PYTHON DATA SCIENCE · COMPLETE FACILITATOR GUIDE

Python Data Science

The full teaching manual for the 21-session course (one 3-hour session, two lessons each). Timing, live-demo scripts, the mistakes to pre-empt, lab facilitation, and checks for understanding, for every session.

21 sessions · 41 lessons | 21 sessions × 3 h · 63 h | Instructor copy · not for students

CONTENTS

S01 **Toolkit and First Arrays**

- 0.1 Your Data Science Toolkit
- 1.1 Arrays vs Lists

S02 **Vectorized Operations and Broadcasting**

- 1.2 Vectorized Operations
- 1.3 Broadcasting

S03 **Randomness and Linear Algebra**

- 1.4 Random Number Generation
- 1.5 Linear Algebra Essentials

S04 **DataFrames and Loading Data**

- 2.1 DataFrames and Series
- 2.2 Loading Data

S05 **Cleaning and Grouping**

- 2.3 Cleaning Data
- 2.4 GroupBy Operations

S06 **Merging and Reshaping**

- 2.5 Merging and Joining
- 2.6 Pivot Tables and Reshaping

S07 **Time Series and Windows**

- 2.7 Time Series
- 2.8 Window Functions

S08 **Features and First Charts**

- 2.9 Feature Engineering Basics
- 3.1 Line, Scatter and Bar Charts

S09 **Composing and Customizing Figures**

- 3.2 Subplots and Composition
- 3.3 Customizing Plots

S10 **Saving Figures; Center and Spread**

- 3.4 Saving Figures
- 4.1 Descriptive Statistics I: Center and Spread

S11 **Shape, Outliers and Distributions**

- 4.2 Descriptive Statistics II: Shape, Position and Outliers
- 4.3 Probability Distributions

S12 **Correlation and Hypothesis Tests**

- 4.4 Correlation Analysis
- 4.5 Hypothesis Testing

S13 **A/B Testing and the EDA Workflow**

- 4.6 A/B Testing
- 4.7 An EDA Workflow

S14 **SQL from Python: Queries and Joins**

- 5.1 Database Connections
- 5.2 Complex Queries

S15 **SQL Windows and Query Speed**

- 5.3 SQL Window Functions
- 5.4 Query Optimization

S16 **Documents and HTTP**

- 5.5 NoSQL with MongoDB
- 6.1 HTTP with Requests

S17 **Scraping Static and Dynamic Pages**

- 6.2 Parsing HTML with Beautiful Soup
- 6.3 Dynamic Pages with Selenium

S18 **APIs: Keys, Tokens and Polite Clients**

6.4 API Authentication

6.5 Rate Limiting and Retries

S19 **Seaborn and Plotly**

7.1 Statistical Graphics with Seaborn

7.2 Interactive Charts with Plotly

S20 **Bokeh and Streamlit**

7.3 Interactive Charts with Bokeh

7.4 Dashboards with Streamlit

S21 **Capstone Studio and Showcase**

8.1 The Pinecrest Report

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 1: Toolkit and First Arrays

How to teach the first session of Python Data Science. Welcome a room of people who can already write Python functions, loops and small programs; prove that every machine has a working environment and the course data; find out what they remember; then get real work out of the toolkit (a first look at 20,302 orders) and meet the structure everything else in the course is built on, the NumPy array. Learners leave having loaded real files, read three tracebacks from the bottom, measured for themselves why an array beats a list for numbers, and been surprised, once and safely, by a view.

`2 lessons` `3 hrs session` `Lessons 0.1–1.1` `Session 1 of 21`

HOW TO USE THIS GUIDE

- **Same rhythm for both lessons:** frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- **Type, do not paste.** Every code block in a demo script below was run to produce the output printed under it. Type it live, ask for a prediction before you run it, and read errors from the last line up, out loud.
- **The lessons are the reference.** Keep `getting_started.html` and `numpy_arrays_vs_lists.html` open in browser tabs for the diagrams (the setup map, the list-versus-array memory picture, the view and copy picture). Labs live in `labs/module-01/`: `_starter.py` files are handouts, `_solution.py` files are yours.
- **Session 1 is different in one way:** it opens with 30 minutes of welcome, environment check and diagnostic instead of the usual 10-minute retrieval warm-up. Lesson 0.1's installation steps are done **before** the session from the setup guide; the room spends its 0.1 time using the toolkit, not installing it.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (0.1, 1.1) match the course site.

THE #1 RISK OF SESSION 1: INSTALLATION EATS THE SESSION

Lesson 0.1 is an install lesson, and installs go wrong in ways that are different on every laptop. If you let the room install together on venue Wi-Fi, you will still be fixing machines at the break and Lesson 1.1 will not happen. Three things prevent it. **Before:** send the setup guide a week ahead and ask every learner to reply with the last line of `check_environment.py`; chase the ones who do not reply. **In the room:** the check gets 12 minutes and no more; a machine that is not ready gets a partner, not your attention. **After:** the NOT READY output is a numbered list the learner can follow at home, and you check it again at the start of Session 2. The second risk is quieter: a learner whose notebook kernel is not the course environment, who gets through today on an old pandas and then sees different output from the lesson pages for weeks. The version lines in the check are there to catch exactly that.

BEFORE THE SESSION

- ☐ **A week before:** email the setup guide (the kit's Setup Guide, which follows Lesson 0.1: Python 3.12 or newer, a `pinecrest-analysis` folder, `.venv`, `python -m pip install -r requirements.txt`, `pinecrest-data.zip` unzipped into it). Attach `labs/module-01/l0_1-orientation/check_environment.py` and ask learners to run it and reply with its last line. No reply means an install risk: chase it.
- ☐ **Files to hand out** (shared folder or USB stick, in case the network fails): `requirements.txt`, `pinecrest-data.zip`, the whole `labs/` folder **without** the `*_solution.py` files (keep `labs_paths.py` in it: every lab imports it), and the workbook. Learners put `labs/` inside `pinecrest-analysis`, next to `data/`, so the labs find the data on their own.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-01` from the `instructor-kit` folder. Every line should say PASS (8 files). Run `python labs/module-01/l0_1-orientation/check_environment.py` on the projector machine so you can show what READY looks like.
- ☐ **Servers to start:** none. Internet is needed only by learners who still have to install; the session itself runs offline.
- ☐ **Projector machine:** a terminal with `(.venv)` active in your own `pinecrest-analysis`, JupyterLab (or VS Code) started from that folder, font size 18 pt or more, two empty notebooks ready: `lesson_0_1` and `lesson_1_1`. Browser tabs: the course home page, `getting_started.html`, `numpy_arrays_vs_lists.html`.
- ☐ **Board, before learners arrive:** the five-step routine (terminal → `cd pinecrest-analysis` → activate, see `(.venv)` → start JupyterLab or VS Code here → kernel is `.venv`), and space for the three array rules you will add after the break.
- ☐ **Print or share:** the workbook (its Start Here page has the diagnostic), and sticky notes for help flags and the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`python` is not recognized, or typing `python` opens the Microsoft Store (Windows)	No environment is active yet, and Windows only provides `py`	Use `py -m venv .venv` to create the environment. Once it is active, plain `python` works.
`Activate.ps1 cannot be loaded because running scripts is disabled on this system`	PowerShell's default execution policy	Once per user: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer Y, activate again.
`ModuleNotFoundError: No module named 'pandas'` in a terminal	The environment is not active in this window	Activate it; look for `(.venv)`. Activation is per terminal window.
`ModuleNotFoundError` in a notebook although the terminal works; or pandas 2.x in the version check	The notebook kernel is a different Python	JupyterLab: start it from the activated terminal. VS Code: **Select Kernel** ▸ the entry that mentions `.venv`.
`FileNotFoundError: ... 'data/orders.csv'`	Python is working in another folder, or the zip was extracted into a nested folder such as `pinecrest-data/data`	`print(Path.cwd())`. Start JupyterLab from `pinecrest-analysis`; move `data` so that `pinecrest-analysis/data/orders.csv` exists.
`ERROR: Could not find a version that satisfies the requirement ...` during the install	Python older than 3.12 (or much newer than the pins)	`python --version`. Install 3.12 or newer, delete `.venv`, create it again, reinstall.
`The virtual environment was not created successfully because ensurepip is not available` (Debian, Ubuntu)	The venv module is a separate system package	`sudo apt install python3-venv python3-pip`, then create the environment again.
`jupyter lab` starts but no browser opens	No default browser, or a remote or WSL session	Copy the `http://localhost:8888/lab?token=...` line from the terminal into a browser.
The install is very slow or fails halfway (venue Wi-Fi)	Several hundred megabytes of downloads	Do not install in the room. This is why the setup guide goes out a week ahead. Today: pair up; install at home.

SYMPTOM	CAUSE	FIX
A locked work laptop: nothing can be installed	No administrator rights	Pair today. Before Session 2: the python.org installer's per-user option usually needs no admin rights; otherwise their IT team. A hosted notebook service works only if they upload the data folder and match the versions.

SESSION 1 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	30	Welcome, environment check and diagnostic	Introductions, how the course runs, `check_environment.py` on every machine, a six-question diagnostic of Python skills (not graded).
0:30	10	**0.1** Frame and mental model	The workshop model; the five-step routine for every time you sit down; notebook or script.
0:40	20	**0.1** Live demo	Version check, where is Python working, a first look at the orders, three errors read from the bottom, a four-line script.
1:00	15	**0.1** Guided lab	`labs/module-01/l0_1-orientation/payment_methods_starter.py`: how do customers pay?
1:15	10	**0.1** Start the lesson project	`first_look.py`, Steps 1 and 2 in a notebook; Steps 3 to 6 go home.
1:25	10	Break	Fix NOT READY machines now. Leave the five-step routine on the board.
1:35	10	**1.1** Frame and mental model	Parking lot and starting grid; one type, fixed size, whole-array operations.
1:45	30	**1.1** Live demo	First array, four facts, `* 2`, memory and speed measured, a CSV column with `nan`, rows and columns, views and copies.
2:15	20	**1.1** Guided lab	`labs/module-01/l1_1-arrays-vs-lists/transactions_week_starter.py`, then `what_if_copy_starter.py` in pairs.
2:35	15	**1.1** Start the lesson project	The benchmark report, Steps 1 and 2; Steps 3 to 6 go home.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 2.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 90 + 80 = 170 minutes for a solo learner, and after the 30-minute opening, the break and the wrap-up the room has 130. So the room does, for each lesson: the mental model, the live demo, one guided kit lab, and the first two steps of the lesson project. Lesson 0.1's installation sections (Install Python, the virtual environment, the packages, downloading the data) are **prework**, verified by the environment check, and are not repeated in the room.

Goes home (2 to 3 hours). `first_look.py` Steps 3 to 6 and the benchmark report Steps 3 to 6; each lesson's Hands-on Exercises (three, plus a stretch); both Quick Quizzes; both journal entries; and Lesson 1.1's "Before the Next Lesson" list. Anyone whose check said NOT READY also fixes their machine.

Protect, in this order: the environment check (nobody leaves without knowing whether their machine is ready and what to do if not); the views and copies step of the 1.1 demo and Lab B; reading three tracebacks from the bottom in the 0.1 demo. **Cut first:** the four-line script and `help(len)` in the 0.1 demo (Steps 11 and 12), then the 0.1 project start (send all of `first_look.py` home), then the speed measurement in the 1.1 demo (Step 6; the project measures it anyway).

OPENING SEGMENT

0:00 Welcome, Environment Check and Diagnostic — 30 min

TIMING

5m	Welcome and introductions
5m	How the course runs: 21 sessions, two lessons each, the room and home split, the journal
12m	Environment check: `check_environment.py` ends with READY on every machine (or the machine gets a helper)
8m	Diagnostic: six predict-the-output questions, not graded

SEQUENCE AND TALKING POINTS

1. **Welcome (5m).** One minute on yourself, including a question you once answered with data that changed someone's mind. Then a fast round: name, where you learned your Python, and one thing you would like to be able to find out from data by the end of the course. Write three or four of those on the board; they come back in today's first journal entry. Housekeeping in one breath: break at 1:25, where the lab files are, and "if you are stuck, put a sticky note on your laptop lid and keep trying while I get to you."

2. **How the course runs (5m).** Show the course home page. Say: "Twenty-one sessions of three hours, two lessons a session. NumPy first, then pandas, charts, statistics, SQL, data from the web, interactive dashboards, and a capstone you choose. Every lesson uses the same made-up company, Pinecrest Outfitters, so by the end you will know its data well." Then the deal: "Each lesson was written for someone working alone for 75 to 110 minutes, so two do not fit in three hours. In this room we do the parts that are hard to do alone: the mental model, a live demo, a guided lab, and the first steps of each lesson's project. At home you finish the projects and do the lesson's exercises, quiz and journal: plan on two to three hours a week." Open `getting_started.html` and scroll once through the pattern: objectives, sections with code and its real output, common mistakes, a project in steps, exercises with hidden solutions, a quiz, a journal prompt.

3. **Environment check (12m).** Say: "Before any new ideas, we prove that every machine is ready." On the projector, then theirs: **1** open a terminal; **2** `cd pinecrest-analysis`; **3** activate and look for `(.venv)`; **4** `python check_environment.py` (the file is in `labs/module-01/l0_1-orientation/`; they copy it into the project folder, or run it by its path). Done = the last line says `READY for Python Data Science`. Anyone who ran it at home and saw READY runs it once more and becomes a helper. A NOT READY machine prints a numbered fix list: activation and the data folder you fix now (one minute each); a missing Python 3.12 or a full package install you do **not** start on venue Wi-Fi: pair that learner with a ready neighbor for today and fix it at the break or at home.

4 **Diagnostic (8m).** Say: "Six quick questions from the Intro and Intermediate courses. This is not a test and nobody collects it. It tells you what to reread and tells me what to slow down for." Learners answer D1 to D6 on the workbook's Start Here page (about four minutes), then show fingers (1 to 4 for A to D) as you reveal each on the slide. Settle any argument by running the code. Keep a tally of wrong answers per question on a sticky note.

DIAGNOSTIC ANSWERS AND HOW TO READ THE TALLY

Answers: **D1** A) `['Sat', 'Sun'] ['Tue', 'Wed']`; **D2** C) `[1, 2, 3, 4] [1, 2, 3]`; **D3** B) `3 2`; **D4** D) `54.0 55.0`; **D5** A) `2 434.9`; **D6** B) `5962.05962.0`.

- **D1 (slicing) and D2 (names and copies)** feed straight into Lesson 1.1 today. If a third of the room misses D2, slow down at the views step of the 1.1 demo and connect it explicitly: "`b = a` gave two names for one list; an array slice does the same thing for part of an array."
- **D3 (dictionaries)** and **D5 (comprehensions)** are the loop-and-count habits that NumPy and pandas replace. Misses here are not urgent: say so.
- **D4 (functions, defaults, return)** matters every time a library call takes keyword arguments. Two or more misses: narrate the arguments of `read_csv` and `genfromtxt` out loud in the demos ("`skip_header=1`: a keyword argument, like `rate=0.10`").
- **D6 (text from files)** predicts trouble with dtypes. Misses here: make a point of printing `.dtype` in every demo step.
- **Anyone with three or more wrong:** not a problem to announce. Pair them with a strong neighbor today, and at the wrap-up hand them `labs/module-01/l0_1-orientation/diagnostic_starter.py` as extra homework: six small functions with a checker, one per skill (`diagnostic_solution.py` is your reference). The workbook's Start Here page names what to review for each question.

WATCH-OUTS

- **Do not debug one machine in front of the room.** Two minutes per machine, then pair them and move on. The trap table in Before the Session covers nearly everything you will see.
- **Mixed backgrounds.** Some learners finished Intermediate Python last week; some learned Python at work years ago. The diagnostic is for them as much as for you.
- **"I already use pandas."** Welcome them and give them a job: helper during the environment check, and the stretch tasks in each lab. Ask them to let Module 1 build the array model before reaching for DataFrame shortcuts.
- **Versions.** The lessons print output from Python 3.12, NumPy 2.5, pandas 3.0. A learner on older versions will see different output (pandas 3 changed how text columns and copies behave). The check prints every version: compare there first.

LESSON PLANS

0.1 Your Data Science Toolkit

55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Install Python 3.12 or newer and create a virtual environment for the course on Windows, macOS or Linux
- Install the course packages from a `requirements.txt` file and confirm they work
- Run code both in a notebook and as a script, and say when each one suits the job
- Download the course data and load a file from it with a relative path
- Read a Python traceback from the bottom up and take a sensible next step when you are stuck

TIMING IN THE ROOM

10m	Frame: the workshop model, the five things you repeat every time you sit down, notebook or script
20m	Live demo: version check, where am I, first look at the orders, three errors on purpose, a four-line script
15m	Guided lab: `payment_methods_starter.py` (how do Pinecrest customers pay?)
10m	Start the lesson project: `first_look.py`, Steps 1 and 2

MENTAL MODEL TO SAY

Say it while you draw four nested boxes on the board: "Python is the building. A virtual environment is one workshop inside it with its own tool wall: the tools hanging there are for this project only. The packages are the tools. A notebook is the workbench where you try things, and the data folder is the lumber."

Then the routine, written on the board and left there all session: **1** open a terminal, **2** `cd pinecrest-analysis`, **3** activate (look for `(.venv)`), **4** start JupyterLab or VS Code from that folder, **5** check that the notebook's kernel is the `.venv` one. "Almost every problem in week one is one of these five being skipped."

One sentence for the data: "Every file belongs to Pinecrest Outfitters, a made-up outdoor-gear shop with a website and four stores. It is fictional, so you can dig into customer records freely, and it was built with real things to find."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The environment check in the opening already proved that every machine works, so this demo is about **using** the toolkit, not installing it. Work in a new notebook called `lesson_0_1` in your own `pinecrest-analysis` folder; learners type along in theirs. Steps 7 to 9 are errors you cause on purpose: read the last line out loud each time before you fix anything.

1. **Prove which Python is running (2m).** Say: "First cell of any new project: ask the tools who they are. If these three lines print, the workshop is stocked."

```
import sys
import numpy as np
import pandas as pd

print("Python", sys.version.split()[0])
print("NumPy ", np.__version__)
print("pandas", pd.__version__)
```

```
Python 3.12.14
NumPy  2.5.4
pandas 3.0.6
```

Last digits may differ in the room; the first two numbers should match. Anyone with pandas 2.x has the wrong kernel or an old environment: send them to the trap table.

2. **Where is Python working? (1m).** Say: "A relative path starts from the folder Python is working in. Before I open a file I ask whether Python can see it."

```
from pathlib import Path

print(Path("data/orders.csv").exists())
```

```
True
```

If anyone gets `False`: `print(Path.cwd())` shows where they really are. Close JupyterLab, `cd` into the project folder, start it again.

3. **Load the orders (2m).** Say: "One line reads the file into a table. pandas calls it a DataFrame; Module 2 teaches it properly. Today we need three moves." Use `.T` so three rows fit the projector.

```
orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
print(orders.shape)
print(orders.head(3).T)
```

```
(20302, 11)
                                0                    1                    2
order_id                   100001               100002               100003
customer_id                 12673                10034                12448
order_date      2024-01-01 00:00:00  2024-01-01 00:00:00  2024-01-01 00:00:00
channel                       store                store               online
store                     Flagstaff            Asheville               Online
status                    completed            completed            completed
payment_method                 cash                 card                 card
promo_code                      NaN                  NaN                  NaN
discount_pct                      0                    0                    0
shipping_cost                   0.0                  0.0                14.95
order_total                   434.9                278.9                 75.9
```

Ask: "What is one row of this file?" (One order.) "How many?" (20,302.) Point at `NaN` in `promo_code`: no value, not zero.

4 **Pick one column and summarize it (2m).** Say: "Square brackets with a column name give one column. `describe()` is the fastest first look at a number."

```
print(orders["order_total"].describe())
```

```
count    20302.000000
mean       194.596663
std        178.760964
min          6.380000
25%         78.850000
50%        146.850000
75%        264.950000
max       4893.000000
Name: order_total, dtype: float64
```

Ask before you explain: "The mean is about $195 but the 50% row is $147. What does that tell you?" (A few large orders pull the mean up.) "And the max?" Leave $4,893 as an open question: it is Exercise 3.

5 **Count the categories (2m).** Say: "`value_counts()` counts how often each value appears. It is the first thing to run on any text column."

```
print(orders["status"].value_counts())
```

```
status
completed    18762
returned      1143
cancelled      397
Name: count, dtype: int64
```

Ask: "If I add up every `order_total`, is that Pinecrest's revenue?" (No: 1,143 returned and 397 cancelled orders are in there.)

6. **Filter the rows, then total (2m).** Say: "The expression in the brackets is a filter: keep the rows where it is true. Read it as 'orders where status equals completed'."

```
completed = orders[orders["status"] == "completed"]

print(len(completed))
print(f"Completed revenue: ${completed['order_total'].sum():,.2f}")
```

```
18762
Completed revenue: $3,624,306.27
```

This is the pattern the lab repeats with a different column.

7. **Error 1: the wrong path (2m).** Say: "Now I break it on purpose. Do not look at the top. Scroll to the **last line**." In the notebook the traceback is about 30 lines; the last one is all you need.

```
orders = pd.read_csv("orders.csv")
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'orders.csv'
```

Read it aloud: "No such file: orders.csv." Then find the arrow at your own line. Say: "The lines in the middle are inside pandas. They are how it got there, not where the mistake is."

8. **Error 2: a column name that is almost right (2m).** Say: "A KeyError from a table means there is no column with exactly this name."

```
orders["Order Date"]
```

```
KeyError: 'Order Date'
```

Do not guess the fix. Ask the table:

9. **Ask the table for its names.**

```
print(orders.columns.tolist())
```

```
['order_id', 'customer_id', 'order_date', 'channel', 'store', 'status', 'payme
```

Say: "Copy the name from here. Lowercase, underscore."

10 **Error 3: a name that does not exist (1m).** Say: "A typo, or a cell that has not been run since the kernel started. Python even offers a guess in the full traceback: Did you mean 'orders'?"

```
print(ordres.shape)
```

```
NameError: name 'ordres' is not defined
```

Now demonstrate the notebook trap: **Kernel ▸ Restart Kernel**, run only the `describe()` cell, and get the same NameError for `orders`. Fix: **Run ▸ Run All Cells** from the top.

11 **The same work as a script (3m).** Say: "A notebook is where I work it out. A script is how I keep it." Create `count_orders.py` in the project folder with these lines, then run `python count_orders.py` in the terminal (check for `(.venv)` first).

```
"""count_orders.py: how many orders, and how many were completed?"""
import pandas as pd

orders = pd.read_csv("data/orders.csv")
completed = orders[orders["status"] == "completed"]
print("Orders:", len(orders), "| completed:", len(completed))
```

```
Orders: 20302 | completed: 18762
```

Say: "It starts fresh every time and prints only what I print. That is why the lessons always use `print()`: the output is the same in both places."

12 **Ask the documentation (1m).** Say: "Python can describe its own tools. In a notebook, `pd.read_csv?` opens the same help in a panel."

```
help(len)
```

```
Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The kernel or the terminal is not the course environment.** Symptom: `ModuleNotFoundError: No module named 'pandas'`, or a pandas version that starts with 2. Coach with a question, not a fix: "What does your prompt start with?" and, in a notebook, "Which kernel does the top-right corner name?" Then have them run `import sys; print(sys.executable)`: the path should contain `.venv`.
- **Python is working in a different folder.** Symptom: `FileNotFoundError` for a file they can see in the file browser, usually because JupyterLab was started from the home folder or the zip was extracted into `pinecrest-data/data`. Coach: "Ask Python where it is: `print(Path.cwd())`. Is there a `data` folder inside that folder?" Do not let them paste an absolute path: it works today and breaks the lesson code tomorrow.
- **Cells run out of order, or a name that is almost right.** Symptom: `NameError` for a variable that is plainly on the screen, or `KeyError: 'Status'`. Coach: "Read me the last line." Then: "When did the cell that creates it last run?" (Restart and run all.) For a KeyError: "Print `orders.columns` and copy the name." Praise reading the error before asking for help; that habit is objective five.

GUIDED LAB

File: learners open `labs/module-01/l0_1-orientation/payment_methods_starter.py`; your reference is `labs/module-01/l0_1-orientation/payment_methods_solution.py`. It practices the lesson's three moves (read a file, pick a column, count and filter) on a question the lesson does not ask: how do customers pay?

Run it first, as shipped (1m). Everyone runs the starter untouched and sees five TODO lines. That proves the lab can find the data. If a learner sees "The Pinecrest course data was not found", their labs folder is not next to a `data` folder: the message names both fixes.

TODO 1 to 2 together (5m), TODO 3 to 5 in pairs (8m). Circulate for `orders["payment_method"]` typed with a capital or a space, and for `=` where `==` is meant inside the filter.

Expected: shape `(20302, 11)`; card 12,392, digital_wallet 4,794, cash 1,614, gift_card 1,502; digital wallet share 23.6%; 1,502 gift-card orders averaging $193.22; all 1,614 cash orders have channel `store`.

Debrief (1m): "Cash is never used online. Is that a finding or a check?" (A check: the data behaves the way the business works. When a check like this fails, you have found a data problem.)

Fast finishers: `labs/module-01/l0_1-orientation/debug_traceback.py`, a script with three bugs to fix one traceback at a time (a wrong file name, a wrong column name, a misspelled variable). Fixed, it prints 20302 orders, 1143 returned, largest returned order $1,168.75.

START THE LESSON PROJECT

The lesson project is `first_look.py`: a summary of the orders file and a chart of completed revenue by month. In the room, do **Steps 1 and 2** only (10 minutes), in a notebook.

Open the lesson page at "Project: First Look at Pinecrest" and read the brief aloud. Everyone types Step 1 (load, count, first and last date) and Step 2 (`value_counts()` on `channel` and `status`). Done in the room = `Orders: 20302`, first order 2024-01-01, last order 2025-12-31, online 12,854 and store 7,448.

Goes home: Step 3 (completed revenue $3,624,306.27), Step 4 (best month 2025-06, slowest 2024-02), Step 5 (the bar chart `monthly_revenue.png` with 24 bars) and Step 6 (assemble the script and run it from the terminal). Tell them the three numbers to expect so they can check themselves.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

1. You open a new terminal tomorrow and `python first_look.py` says there is no module named pandas. What did you skip? **Answer:** Activating the environment in that terminal. Nothing was uninstalled.
2. Why does the lesson say `python -m pip` and not just `pip`? **Answer:** It guarantees the pip that belongs to the Python you are running, which is the environment's when it is active.
3. A traceback is thirty lines long. Which line do you read first, and which do you look for second? **Answer:** The last line (error type and message), then the line that points into your own file.
4. When would you choose a notebook, and when a script? **Answer:** A notebook to explore and look at each result; a script for finished work that must run the same way every time.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) This terminal will use the project's own Python and the packages installed for it; **Q2** D) `python -m pip install -r requirements.txt`; **Q3** `1143`; **Q4** A) Print `orders.columns` and copy the exact column name from there; **Q5** C) A `NameError`: the name `orders` is not defined; **Q6** `True False`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Pair anyone whose check said NOT READY with a ready neighbor for the demo and the lab (one keyboard, two heads) and fix the machine at the break, not in front of the room. For learners who froze on the diagnostic, sit them where you can see their screen and give them the typing role in the pair. Keep the five-step routine on the board and point to it instead of fixing things for them.

Stretch: `debug_traceback.py`; then the lesson's Bonus Challenge (load `daily_sales.csv` and explain why the store totals are so much larger than the store rows in `orders.csv`: the answer is in

`data/README.md`). Ask experienced learners to hold back on pandas tricks they already know: Module 2 gets there.

1.1 Arrays vs Lists

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain how a NumPy array stores numbers differently from a Python list, and why that makes it smaller and faster
- Create arrays from lists, from scratch, and from a CSV file, and read their `dtype`, `shape`, `ndim` and `size`
- Select elements, rows and columns with indexing and slicing in one and two dimensions
- Predict whether an operation gives you a view or a copy, and make a copy when you need one
- Measure memory and speed yourself and choose between a list and an array for a task

TIMING IN THE ROOM

10m	Frame: two containers, two jobs; the parking lot and the starting grid; why pandas sits on arrays
30m	Live demo: first array, four facts, `* 2`, one type only, memory, speed, a CSV column with `nan`, rows and columns, views and copies
20m	Guided lab: `transactions_week_starter.py`, then `what_if_copy_starter.py` in pairs
15m	Start the lesson project: the benchmark report, Steps 1 and 2

MENTAL MODEL TO SAY

Say: "A list is a public parking lot. Motorcycles, sedans and delivery trucks all park there, each in its own space, and the attendant keeps a clipboard saying which vehicle is where. An array is the starting grid of a race: every car the same class, bumper to bumper in numbered positions. Nobody needs a clipboard, and because the grid is uniform the whole field can move as one."

Turn the picture into the three rules you will repeat all session, and write them on the board: **one type** (the dtype), **fixed size** (arrays do not grow), **whole-array operations** (no loop). Add a fourth line under them for the second half: **a slice is a window, not a photocopy.**

Why it matters, in one breath: "pandas stores every column as an array. Matplotlib plots arrays. SciPy computes on arrays. Understand this one container and the rest of the course stops looking like magic spells."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_1`, same project folder. Ask for a prediction before every run: the lesson is built on small surprises (`* 2`, the silent text conversion, the `nan` mean, the view), and a surprise only teaches if the learner committed to a guess first. Steps 5 and 6 measure; the timing in Step 6 will differ on every machine and that is the point.

1. **A first array (2m).** Say: "These are Boise's revenue figures for the first week of January 2024, Monday to Sunday. `np.array` takes a list and lines the values up on the grid."

```
import numpy as np

week_list = [5962.00, 6583.13, 3866.59, 7667.88, 9570.46, 12388.57, 9361.59]
week = np.array(week_list)

print(week)
print(type(week))
```

```
[ 5962.    6583.13  3866.59  7667.88  9570.46 12388.57  9361.59]
<class 'numpy.ndarray'>
```

Point at the screen: no commas. That is how you tell an array from a list at a glance.

2. **Four facts every array knows (2m).** Say: "When something surprises you, print these. They answer most questions."

```
print(week.dtype, week.shape, week.ndim, week.size)
```

```
float64 (7,) 1 7
```

`(7,)` is a tuple with one entry: one dimension, seven long. Ask: "How many bytes is a `float64`?" (8.)

3. **The same operator, two meanings (2m).** Ask first: "What is `week_list * 2`? What is `week * 2`?" Take two guesses for each, then run.

```
print(len(week_list * 2))
print(week * 2)
```

```
14
[11924.   13166.26  7733.18 15335.76 19140.92 24777.14 18723.18]
```

Say: "For a list, star means repeat: fourteen items. For an array, it is arithmetic on every element. No loop."

4. **One type only (2m).** Ask: "What happens if one value is text?"

```
mixed = np.array([42, "tent", 3.5])
print(mixed)
print(mixed.dtype)
```

```
['42' 'tent' '3.5']
<U32
```

Say: "No error. NumPy quietly turned all three into text, because text is the only type that can hold them all. One stray word in a column of numbers and the whole array is strings. When an array misbehaves, check `.dtype` first."

5 **Measure the memory (3m).** Say: "Do not take my word for it that arrays are small. Measure." Type it slowly; the two list lines are the address book and the houses.

```
import sys

n = 1_000_000
numbers_list = list(range(n))
numbers = np.arange(n)

list_bytes = sys.getsizeof(numbers_list) + sum(sys.getsizeof(x) for x in numbe
print(f"list : {list_bytes:>12,} bytes")
print(f"array: {numbers.nbytes:>12,} bytes")
print(f"ratio: {list_bytes / numbers.nbytes:.1f}x")
```

```
list :   36,000,056 bytes
array:    8,000,000 bytes
ratio: 4.5x
```

Say: "The list keeps an 8-byte address for every number, and every number is a full Python object of 28 bytes. The array stores the type once and then only values."

6 **Measure the speed (3m).** Say: "Same job both ways: raise a million prices by 8%. Best of five runs." Your numbers will differ from this page and from each other; write three learners' speedups on the board.

```
import timeit

prices_list = [float(x) for x in range(n)]
prices = np.array(prices_list)

t_list = min(timeit.repeat(lambda: [p * 1.08 for p in prices_list], number=1,
t_array = min(timeit.repeat(lambda: prices * 1.08, number=1, repeat=5))
print(f"speedup: {t_list / t_array:.0f}x")
```

```
speedup: 53x
```

(The figure above is from the machine that built this kit. Anything from about 10x to 100x is normal.) Say: "The list version asks 'what are you?' a million times. The array version makes one call into compiled code that already knows." Name it: vectorization. Session 2 is all about it.

7 **Converting chops, it does not round (2m).** Ask: "What is 2.9 as an integer? And minus 2.9?"

```
print(np.array([2.9, -2.9, 7.5]).astype(np.int64))
```

```
[ 2 -2  7]
```

Say: "The decimal part is dropped. If you want rounding, call `.round()` first."

8 **A real column, and a nan (3m).** Say: "Column 5 of the daily sales file is revenue. One line loads it." Ask for a prediction of the mean before running: "somewhere around what?"

```
revenue = np.genfromtxt("data/daily_sales.csv", delimiter=",", skip_header=1,

print(revenue.shape, revenue.dtype)
print(revenue.mean())
```

```
(2903,) float64
nan
```

Let the `nan` sit for a second. Say: "Not a bug in your code. Six store-days have no revenue recorded, and any arithmetic that touches a missing value gives a missing value."

9 **Count the gaps, then skip them.**

```
print(np.isnan(revenue).sum())
print(np.nanmean(revenue).round(2))
```

```
6
7435.04
```

Say: "An average store-day is about $7,435. Remember the symptom: a `nan` result means a missing value got into the calculation."

10 **Two dimensions: rows, comma, columns (4m).** Say: "Units sold on the first three days of 2024. Rows are days; columns are Asheville, Bend, Boise, Flagstaff." Have the room say "rows, comma, columns" out loud for each line before you run it.

```
units = np.array([[110, 105, 148,  69],
                  [ 88,  84, 152,  82],
                  [103, 133,  90,  92]])

print(units.shape)
print(units[0, 2])       # row 0, column 2
print(units[1])          # all of row 1
print(units[:, 2])       # every row, column 2
print(units[0:2, 1:3])   # rows 0-1, columns 1-2
```

```
(3, 4)
148
[ 88  84 152  82]
[148 152  90]
[[105 148]
 [ 84 152]]
```

Ask: "How would you get Boise's column out of a list of lists?" (A loop or a comprehension.) "Here it is one expression."

11 **A slice is a window (4m).** The most important two minutes of the lesson. Ask for a show of hands: "After I change `window[0]`, has `a` changed?" Count the hands, then run.

```
a = np.array([10, 20, 30, 40, 50])
window = a[1:4]
window[0] = 999

print(a)
print(np.shares_memory(a, window))
```

```
[ 10 999  30  40  50]
True
```

Say: "A list slice is a photocopy. An array slice is a window cut into the wall of the same room: reach through and move the furniture, and it has moved for everyone. NumPy does this because copying a 2 GB array every time you slice it would be a disaster." Tie it to diagnostic D2.

12 **Ask for a copy when you need one (1m).**

```
a = np.array([10, 20, 30, 40, 50])
safe = a[1:4].copy()
safe[0] = 999

print(a)
print(np.shares_memory(a, safe))
```

```
[10 20 30 40 50]
False
```

Rule for the board: "If you will change a slice and still need the original, copy first."

13 **One True or False, please (2m).** Say: "One more error worth meeting on purpose. Read the last line."

```
if week > 6000:
    print("A good week")
```

```
ValueError: The truth value of an array with more than one element is ambiguou
```

Say: "`week > 6000` is seven answers and `if` needs one. Say which question you are asking:"

14 **any or all.**

```
print(week > 6000)
print((week > 6000).any(), (week > 6000).all())
```

```
[False  True False  True  True  True  True]
True False
```

Close the demo by pointing at the board's three rules and asking the room to name where each one showed up.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Expecting a slice to be a copy.** Symptom: "my original numbers changed and I never touched them." There is no error message, which is what makes it hard. Coach: "Run `np.shares_memory(original,`

`piece)`. What does True mean?" Then: "Where should the `.copy()` go?" In the lab this is the whole of Lab B, so let pairs find it before you explain it again.

- **Rows and columns the wrong way round.** Symptom: `week2[1]` when they want Bend's column (it gives Tuesday's row), or `IndexError: index 5 is out of bounds for axis 1 with size 4`. Coach: "Say it out loud: rows, comma, columns. Print `.shape`. Which number is the stores?" Have them point at the row and the column on the printed table with a finger before typing.
- **A silent type change.** Symptom: `102.9` stored in an integer array comes back as `102`; a column that prints with quotes and refuses `.mean()`; a result of `nan`. None of these raise an error. Coach: "What does `.dtype` say?" and for `nan`: "What does `np.isnan(x).sum()` say?" The habit to build is asking the array before asking the instructor.

GUIDED LAB

Lab A (10m, everyone): `labs/module-01/l1_1-arrays-vs-lists/transactions_week_starter.py`; reference `labs/module-01/l1_1-arrays-vs-lists/transactions_week_solution.py`. A different column from the lesson's (`transactions`, column 3): describe the array, slice rows 28 to 55 (the second week of January 2024) and reshape to 7 days by 4 stores, then pull out a row, a column and one cell, and convert to `int16`.

Expected, Lab A: shape `(2903,)`, `float64`, ndim 1, size 2903; 0 missing; Saturday's row `[93. 59. 151. 71.]`; Bend's column `[49. 42. 39. 46. 66. 59. 73.]`; Boise on Saturday 151, which is also the week's maximum; 224 bytes as `float64` against 56 as `int16`; `np.array_equal` is True.

Circulate in Lab A for: `transactions[28:55]` (27 values: `cannot reshape array of size 27 into shape (7,4)`, a perfect moment to say "the stop is left out"); `week2[1]` for Bend; `reshape(4, 7)`, which runs without an error and scrambles the days (ask: "is row 0 one day or one store? Check against the first four values of the file").

Lab B (10m, pairs): `labs/module-01/l1_1-arrays-vs-lists/what_if_copy_starter.py`; reference `labs/module-01/l1_1-arrays-vs-lists/what_if_copy_solution.py`. A manager's what-if ("10 more units a day at Bend") that must not change the real week. One learner types, the other predicts each `np.shares_memory` answer before it runs; swap at Step 5.

Expected, Lab B: Bend's real total 868 printed three times and never changed; what-if total 938; a column slice shares memory (True), the `.copy()` does not (False); storing 97.6 in the `int64` array gives 97, and 97.6 in the `float64` version.

Circulate in Lab B for: `what_if = week1` (a second name, not a copy: the real total becomes 938 and the last line gives it away); `what_if = week1[:, :]` (a view of everything: same result). Do not fix it for them. Ask: "Which printed line tells you the real data changed?"

Fast finishers: `labs/module-01/l1_1-arrays-vs-lists/debug_silent_traps.py`: three blocks that run without an error and print wrong answers (text in a numeric array, an `int8` overflow, a view). Correct answers: average price 48.5, total stock 260, Monday still 148.

START THE LESSON PROJECT

The lesson project is the benchmark report: the same analysis of two years of store data with lists and with arrays, written up from the learner's own measurements. In the room, do **Steps 1 and 2** (15 minutes).

Step 1 together: load `visitors` (column 2) and `revenue` (column 5) with `genfromtxt`; both should print `(2903,) float64`. Ask why the visitor counts print as `201.` with a dot (`genfromtxt` reads everything as `float64` unless told otherwise).

Step 2 on their own: `.tolist()` twins and the memory comparison. Done in the room = `list : 92,952 bytes`, `array: 23,224 bytes`, `ratio: 4.0x`. Anyone who gets 23,280 for the list counted only the container: point at the `sum(sys.getsizeof(x) ...)` half.

Goes home: Step 3 (time spend per visitor both ways), Step 4 (6 missing days; mean spend per visitor $20.87 by two methods), Step 5 (`week1` as a 7 by 4 view; Boise's column totals $55,400.22) and Step 6 (the printed report). **They must keep this notebook:** Lesson 1.2 picks up `revenue` and `visitors` again next session.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

1. An array of one million `int64` values: how many bytes of data? **Answer:** 8,000,000. Eight bytes each, and nothing else per element.
2. `sales` has shape `(3, 4)`: days by stores. What is the shape of `sales[:, 1]`, and what does it hold? **Answer:** `(3,)`: one store's value on each of the three days.
3. I slice `first = revenue[:28]` and set `first[0] = 0`. What happened to `revenue`? **Answer:** Its first value is now 0 too: a slice is a view. `revenue[:28].copy()` would have protected it.
4. You are reading values one at a time and do not know how many are coming. List or array? **Answer:** Collect in a list, convert once with `np.array(the_list)`, compute on the array.
5. The mean of a column prints `nan`. What do you check, and what do you use instead? **Answer:** `np.isnan(x).sum()` to count the missing values, then `np.nanmean(x)`.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** A) `[1, 2, 3, 1, 2, 3] [2 4 6]`; **Q8** `6 [0 4 8]`; **Q9** B) `[5 6 7 8]`; **Q10** D) `float64`; **Q11** `nan 20.0`; **Q12** C) Collecting sensor readings one at a time when you do not know how many there will be. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Give anyone lost in two dimensions the paper version: print (or draw) the 7 by 4 table from Lab A, label the rows Mon to Sun and the columns with the four stores, and have them circle what `week2[5]`, `week2[:, 1]` and `week2[5, 2]` select before typing anything. For views, the lesson's window example with five numbers is easier to hold in the head than the lab's table: rerun it with them and change one number at a time.

Stretch: `debug_silent_traps.py`, then the lesson's Exercise 4 second half (time `np.cumsum` against a running-total loop and explain why the speedup is smaller) or the Bonus Challenge (speedup by size from 10 to 1,000,000 elements: at which size does the array start to win, and why might a list win for ten?).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer on a sticky note or in the workbook's Exit Ticket box and leave it on the way out. Sort the notes into "got it" and "not yet" before Session 2: the pile for question 2 decides how much of the Session 2 warm-up goes to views and copies.

1. Your prompt does not start with `(.venv)`. What will probably go wrong, and what do you type to fix it? **Answer:** Python will not find the course packages (`ModuleNotFoundError`). Activate the environment: `.venv\Scripts\Activate.ps1` on Windows, `source .venv/bin/activate` on macOS and Linux, from the project folder.

2. `a = np.array([1, 2, 3])`, then `b = a[:2]` and `b[0] = 9`. What is `a` now, and what one change would have protected it? **Answer:** `[9 2 3]`: a slice is a view of the same memory. `b = a[:2].copy()` makes an independent array.

3. The mean of a revenue array prints `nan`. What does that tell you, and what do you use instead? **Answer:** At least one value is missing, and arithmetic with a missing value gives a missing value. `np.nanmean(revenue)` skips them (`np.isnan(revenue).sum()` counts them).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** `first_look.py` (Steps 3 to 6: completed revenue $3,624,306.27, best month 2025-06, `monthly_revenue.png` with 24 bars, then run the script from a terminal) and the benchmark report (Steps 3 to 6: 6 missing days, mean spend per visitor $20.87, Boise's week-1 column $55,400.22, the printed report).
- **Lesson exercises.** 0.1: Exercises 1 to 3 (store counts, promo codes, the $4,893 order); Exercise 4 is a stretch. 1.1: Exercises 1 to 3 (inspect and convert, the grid, shrink a column safely); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers are in the Answer Key).
- **Journal:** start `journal.md` in the project folder and write both entries. The 0.1 prompt asks for the exact commands they need each time they sit down: this is the most useful paragraph they will write all week.
- **Before the Next Lesson (from 1.1):** keep the project notebook; load one more column (`transactions`, column 3) and print its shape, dtype, minimum and maximum; predict `np.array([1, 2, 3]) + np.array([10, 20, 30])` before running it.
- **If the environment check said NOT READY:** follow its numbered list, then run it again. Email you the last line.

- **Optional, for anyone who missed two or more diagnostic questions:** `labs/module-01/l0_1-orientation/diagnostic_starter.py`.

> BRING LIST FOR SESSION 2
>
> - A laptop on which `check_environment.py` ends with READY (same environment as today: nothing new to install).
> - The `pinecrest-analysis` folder with `data/` unzipped and the `labs/` folder inside it. Session 2's starters are in `labs/module-02/`.
> - The finished Lesson 1.1 project notebook: Lesson 1.2 picks up the `revenue` and `visitors` arrays.
> - The workbook and the journal. No servers, no accounts, no internet needed.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the environment check result for every learner; the 1.1 views and copies steps and Lab B; at least Steps 7 to 9 (the three errors) of the 0.1 demo.
- **Cut first (saves 4m):** 0.1 demo Steps 11 and 12 (the script and `help`). Say the one sentence, "a notebook to work it out, a script to keep it", and point at Step 6 of the lesson project.
- **Cut second (saves 10m):** the 0.1 project start. Read the brief, give the three check numbers, send all six steps home.
- **Cut third (saves 3m):** the speed measurement in the 1.1 demo. Keep the memory measurement; the project times things anyway.
- **If the opening runs to 40 minutes** (a room with many NOT READY machines): take the 10 from the 0.1 project start, never from Lesson 1.1.
- **Never cut:** the exit ticket. It takes four minutes and it is your only evidence of what landed.

RUNNING SHORT: WHAT TO ADD

- Run `debug_traceback.py` as a whole-room race in pairs: first pair to print the three correct lines explains their routine.
- Do Lesson 1.1's Exercise 2 (the 3 by 4 grid) on the board: learners call out the index expression for (a) to (d) before you type it.
- Start Step 3 of the benchmark report together and compare speedups across the room: why are they different? (Different processors; the pattern holds everywhere.)
- Give five quiet minutes for the first journal entry in the room. Learners who start it here are far more likely to keep the habit.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 2: Vectorized Operations and Broadcasting

How to teach the session where learners stop writing loops over numbers. In the first half they describe what they want done to a whole array: arithmetic between parallel arrays, boolean masks that count, filter and combine, `np.where`, and summaries along an axis. In the second half they learn the two short rules that let NumPy combine a table with a row, a column or a single number, and the habit, checking shapes, that catches the mistakes NumPy will not report. By the end they have compared four stores of very different sizes fairly, in one line.

`2 lessons` `3 hrs session` `Lessons 1.2–1.3` `Session 2 of 21`

HOW TO USE THIS GUIDE

- **Same rhythm for both lessons:** frame (one mental model, the rules on the board) → live demo (you type, they type along) → guided lab (two kit labs per lesson: new questions on the same data) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- **Predict before every run.** Both lessons are about being able to say what will come out: how many values, what shape, error or no error. In 1.3, write the two shapes on the board, right-aligned, before every operation, and take the room's call.
- **Every code block below was run** to produce the output printed under it, on the course data with NumPy 2.5. Type it live; read errors from the last line.
- **The lessons are the reference.** Keep `numpy_vectorized_operations.html` and `numpy_broadcasting.html` open for the diagrams (the mask as punched cardboard, the axis picture, the stamp, the right-aligned shape examples). Labs live in `labs/module-02/`.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (1.2, 1.3) match the course site.

THE #1 RISK OF SESSION 2: WRONG ANSWERS THAT RAISE NO ERROR

Until now a mistake produced a traceback. Today's important mistakes do not. Aggregating along the wrong axis gives numbers that answer a different question. A missing value drops silently out of both halves of a comparison. A row meeting a column builds a grid of sixteen plausible-looking numbers where four were wanted. A learner who has not built the habit of checking will leave thinking everything worked. So make checking the visible routine of the session: before a run, say the expected shape or count out loud; after it, print `.shape` or add the pieces up. Praise a learner who catches a wrong shape more than one who types fast. The second risk is pace: Lesson 1.2 is the longest in the module and it is easy to spend the whole first half on masks. Hold the 1.2 demo to 30 minutes; the lab is where masks stick.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-02` from the `instructor-kit` folder (8 files, all PASS).
- ☐ **Files to hand out:** `labs/module-02/` without the `*_solution.py` files (learners already have `labs_paths.py` from Session 1). Six files: four starters and the handouts `debug_masks.py` and `debug_shapes.py`.
- ☐ **Servers to start:** none. No internet needed.
- ☐ **Projector machine:** `(.venv)` active in `pinecrest-analysis`, JupyterLab or VS Code started from that folder, two empty notebooks `lesson_1_2` and `lesson_1_3`, font 18 pt or more. Browser tabs: both lesson pages.
- ☐ **Board, before learners arrive:** space on the left for the three 1.2 rules; on the right, a hand-drawn 7 by 4 table (rows Mon to Sun, columns Asheville, Bend, Boise, Flagstaff) that you will point at all session.
- ☐ **Sort last session's exit tickets.** If more than a third missed the view question, give warm-up question 3 an extra minute and rerun the five-number window example.
- ☐ **Optional paper:** a half-sheet of shape pairs for the 1.3 support activity (six pairs, two right-aligned boxes each).

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
A learner's `revenue.mean()` is a number, not `nan`	They are running an old notebook where `revenue` was already cleaned, or loaded a different column	Restart the kernel and run the `load` helper cell again. Column 5 is revenue.
`RuntimeWarning: Mean of empty slice` (or `invalid`	NumPy stores `nan` and carries on instead of stopping	Not a crash. Ask what was empty or missing. The project's Step 5 prints this warning on purpose.

SYMPTOM	CAUSE	FIX
`value encountered in divide`)		
`load(1)` returns an array of `nan` instead of store names	The default `dtype=float` cannot read text, and `genfromtxt` stores `nan` without complaining	`load(1, dtype=str)`; for dates, `load(0, dtype="datetime64[D]")`.
`FileNotFoundError: data/daily_sales.csv not found`	The notebook was started from another folder (Session 1's trap)	`print(Path.cwd())`; start JupyterLab from `pinecrest-analysis`.
Different output from the page for string arrays or printing	NumPy older than 2.x in a kernel that is not the course environment	Run `check_environment.py` again; choose the `.venv` kernel.

SESSION 2 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 1 (no notes), homework check, any NOT READY machines from last time.
0:10	10	**1.2** Frame and mental model	The workbench and the assembly line; parallel arrays; three rules on the board.
0:20	30	**1.2** Live demo	Array-with-array arithmetic, ufuncs, masks (count, filter, combine), `np.where` and `np.select`, `axis=`, positions, the `nan` trap.
0:50	25	**1.2** Guided lab	`labs/module-02/l1_2-vectorized-operations/staffing_load_starter.py`, then `eight_weeks_axis_starter.py` in pairs.
1:15	10	**1.2** Start the lesson project	The store scorecard, Steps 1 and 2; Steps 3 to 6 go home.
1:25	10	Break	Leave the three board rules up. Draw the 7 by 4 week table for the second half.
1:35	10	**1.3** Frame and mental model	The rubber stamp; two rules; shapes written right-aligned.
1:45	25	**1.3** Live demo	Table and row, table and column (`keepdims`), `np.broadcast_shapes`, a grid, z-scores, two silent mistakes.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:10	25	**1.3** Guided lab	`labs/module-02/l1_3-broadcasting/fair_visitors_starter.py`, then `target_grid_starter.py` in pairs.
2:35	15	**1.3** Start the lesson project	The weekday index, Steps 1 and 2; Steps 3 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 3.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 85 + 75 = 160 minutes for a solo learner, and after the warm-up, the break and the wrap-up the room has 150. The room does, for each lesson: the mental model, the live demo, two guided kit labs, and the first two steps of the lesson project.

Goes home (2 to 3 hours). The store scorecard Steps 3 to 6 and the weekday index Steps 3 to 6; each lesson's Hands-on Exercises (three, plus a stretch); both Quick Quizzes; both journal entries; each lesson's "Before the Next Lesson" list. Lesson 1.2's sections on sorting (`np.argsort`) and "From Loop to Array Expression" get only a step each in the demo: tell learners to read those two sections in full at home.

Protect, in this order: the 1.3 demo's error-then-`keepdims` pair (Steps 4 and 5) and its two silent mistakes (Steps 10 to 12); Lab A of each lesson; the `and` / parentheses errors in the 1.2 demo. **Cut first:** 1.2 demo Steps 3 and 4 (ufuncs; send learners to the lesson's table), then Lab B of 1.2 (assign it as homework: the workbook has the fill-in), then the 1.3 demo's Step 7 (three ways to add an axis: say `keepdims` and move on).

WARM-UP

0:00 Warm-up: Retrieval from Session 1 — 10 min

HOW TO RUN IT

- Slide up, notes closed, three minutes of silent writing in the workbook's Warm-up box, then answers out loud: cold-call gently, one question per learner.
- Question 3 (views) is the one to watch. If the room is shaky, rerun last session's five-number window example before moving on: Lesson 1.2 contrasts it with masks, which return copies.
- Questions 2 and 5 lead straight into today: whole-array arithmetic, and rows, comma, columns.

FIVE QUESTIONS (ANSWERS FOR YOU)

1. Name the three rules that are true of every NumPy array and not of a Python list. **Answer:** One type for the whole array (the dtype), a fixed size, and operations that act on every element at once.
2. What does `[1, 2, 3] * 2` give? What does `np.array([1, 2, 3]) * 2` give? **Answer:** `[1, 2, 3, 1, 2, 3]` (a list repeats) and `[2 4 6]` (an array does arithmetic).
3. `b = a[1:4]`, then `b[0] = 0`. What happened to `a`, and how do you prevent it? **Answer:** `a[1]` is now 0, because a slice is a view. `b = a[1:4].copy()` makes an independent array.
4. The mean of the revenue array printed `nan`. Why, and what did you use instead? **Answer:** Six values are missing and arithmetic with a missing value gives `nan`. `np.nanmean(revenue)` skips them.
5. `t` has shape `(7, 4)`: days by stores. What does `t[:, 2]` select, and what is its shape? **Answer:** Every row of column 2: one store's value on each of the seven days. Shape `(7,)`.

HOMEWORK CHECK

- Show of hands: who ran `first_look.py` from a terminal? Who has a benchmark report? Write three learners' speedups on the board and ask why they differ.
- Ask for the prediction from 1.1's "Before the Next Lesson": `np.array([1, 2, 3]) + np.array([10, 20, 30])` (it is `[11 22 33]`: first with first, second with second). That is the first idea of Lesson 1.2.
- Anyone whose environment check said NOT READY last time runs it again now. Still not ready: pair them, and fix it at the break.
- Do not re-teach the homework. Two minutes for the single most common question, and the rest go on sticky notes for the break.

LESSON PLANS

1.2 Vectorized Operations

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Combine arrays element by element with arithmetic and with universal functions such as `np.sqrt`, `np.log10` and `np.maximum`
- Build boolean masks with comparisons, combine them with `&`, `|` and `~`, and use them to count, filter and assign
- Choose values by condition with `np.where` and `np.select`
- Summarize a two-dimensional array along rows or columns with `axis=`, and predict the shape of the result
- Rewrite a Python loop over numbers as an array expression and check that both give the same answer

TIMING IN THE ROOM

10m	Frame: the workbench and the assembly line; parallel arrays; the three board rules
30m	Live demo: array-with-array arithmetic, ufuncs, masks (count, filter, combine), `np.where` and `np.select`, `axis=`, positions, the `nan` trap
25m	Guided lab: `staffing_load_starter.py`, then `eight_weeks_axis_starter.py`
10m	Start the lesson project: the store scorecard, Steps 1 and 2

MENTAL MODEL TO SAY

Say: "A Python loop is a craftsperson at a workbench: pick up one item, inspect it, work on it, put it down, pick up the next. Careful, flexible, slow. A vectorized operation is an assembly line. You set the machine up once, 'multiply by 1.08', and the whole batch flows through. The machine can be fast because every item on the belt is the same kind of thing: that is the one-dtype rule from last session."

Then the goal for today, in the lesson's words: "By the end, when someone asks 'which days beat $15,000, and what was the average on sale weekends?', your first thought is an array expression and not a `for` statement."

Write three lines on the board and leave them up: **The sum of a mask counts; its mean is a fraction.** · ** `&`, `|`, `~`, with parentheses around every comparison. · The axis you name is the axis that disappears.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_2`. Learners who finished the Lesson 1.1 project already have `revenue` and `visitors`; everyone reloads them with the helper in Step 1 so the room is in the same place. Ask for a

prediction before every run. Step 10 is two errors on purpose: read the last line of each out loud, because learners will meet both within ten minutes in the lab.

1. **A loading helper and four parallel arrays (3m).** Say: "Every column of the daily sales file is useful today, so wrap last session's `genfromtxt` call in a small function."

```
import numpy as np

def load(col, dtype=float):
    """Read one column of daily_sales.csv into an array."""
    return np.genfromtxt("data/daily_sales.csv", delimiter=",",
                         skip_header=1, usecols=col, dtype=dtype)

visitors = load(2)
transactions = load(3)
units = load(4)
revenue = load(5)

print(visitors.shape, transactions.shape, revenue.shape)
print(visitors[:4], transactions[:4])
```

```
(2903,) (2903,) (2903,)
[201. 200. 333. 144.] [51. 52. 68. 33.]
```

Say: "Position 0 in every array is the same store-day: Asheville on January 1, 2024. Arrays that line up like this are parallel arrays. Everything today depends on that."

2. **Array with array: pair by position (2m).** Ask: "What does dividing one array by another do?" Then: "Two retail measures in two lines, for all 2,903 store-days."

```
conversion = transactions / visitors     # share of visitors who bought someth
basket = revenue / transactions          # dollars per purchase

print(conversion[:4].round(3))
print(basket[:4].round(2))
```

```
[0.254 0.26  0.204 0.229]
[98.37 79.48 87.68 92.78]
```

Read the first pair as a sentence: "51 of 201 visitors bought something, 25.4%, and the average purchase was $98.37."

3. **Universal functions (2m).** Say: "Operators cover arithmetic. For everything else there are ufuncs: functions that apply themselves to every element." Boise's first week again, to keep the output small.

```
week = np.array([5962.00, 6583.13, 3866.59, 7667.88, 9570.46, 12388.57, 9361.5

print(np.round(week, -2))                    # nearest hundred
print(np.abs(week - week.mean()).round(0))   # distance from the weekly aver
```

```
[ 6000.  6600.  3900.  7700.  9600. 12400.  9400.]
[1952. 1331. 4048.  246. 1656. 4474. 1447.]
```

Read the last line slowly: `week.mean()` is one number; subtracting it touches every day; `np.abs` removes the signs. Three vectorized steps, no loop. Mention `math.sqrt(week)` fails with `TypeError: only 0-dimensional arrays can be converted to Python scalars`: use the `np.` function of the same name.

4 **A ufunc of two inputs (1m).** Say: "Boise's daily target is $8,000. How far short was each day, counting days that beat it as zero?"

```
print(np.maximum(8000 - week, 0))
```

```
[2038.   1416.87 4133.41  332.12    0.      0.      0.  ]
```

`np.maximum(x, y)` compares pair by pair and returns an array. `week.max()` collapses one array to one number. Different jobs, similar names.

5 **A comparison makes a mask (2m).** Ask: "What do you get when you compare an array with a number?"

```
busy = week > 9000
print(busy)
print(busy.dtype)
```

```
[False False False False  True  True  True]
bool
```

Say: "A sheet of cardboard with holes punched where the answer is True. It is useful three ways: count, filter, assign."

6 **Count and measure (3m).** Point at board rule one. Ask for the two numbers before running.

```
print(busy.sum(), busy.mean().round(3))

big = revenue > 15000
print(big.sum(), "store-days above $15,000")
print(f"{big.mean():.1%} of all store-days")
```

```
3 0.429
101 store-days above $15,000
3.5% of all store-days
```

Say: "True counts as 1. The sum is how many; the mean is what fraction." This is the single most reused idea of the session.

7 **Filter: the mask goes in the brackets (2m).** Say: "Lay the cardboard over the data and read what shows through the holes."

```
print(week[busy])
print(revenue[big][:5])
print(revenue[big].mean().round(2))
```

```
[ 9570.46 12388.57  9361.59]
[16262.39 15993.3  19117.36 17442.78 22093.08]
17683.61
```

Ask: "Is `week[busy]` a view or a copy?" (A copy: the chosen elements can be scattered anywhere, so NumPy gathers them into a new array. A slice was a view; a mask is not.)

8 **A mask from one column, used on another (3m).** Say: "The mask and the array must have the same shape. They do not have to be the same array."

```
stores = load(1, dtype=str)
promo = load(7)

is_boise = stores == "Boise"
print(is_boise.sum(), "Boise rows")
print(np.nanmean(revenue[is_boise]).round(2))     # mean revenue on Boise days
print(visitors[is_boise].mean().round(1))         # mean visitors on Boise day
```

```
727 Boise rows
10144.21
485.2
```

Ask: "Why `np.nanmean` for revenue and plain `.mean()` for visitors?" (Six revenues are missing; no visitor counts are.)

9 **Combine masks (3m).** Point at board rule two. Say: "Real questions have two parts: Boise AND a sale day."

```
is_promo = promo == 1

print((is_boise & is_promo).sum())     # Boise sale days
print((~is_promo).sum())               # ordinary days, all stores
print(np.nanmean(revenue[is_promo]).round(2))
print(np.nanmean(revenue[~is_promo]).round(2))
```

```
24
2807
13950.46
7211.74
```

There is a finding here: an average sale day brings in $13,950 against $7,212. Say: "Did the sale cause all of that? Sale weekends are also holiday weekends. Module 4 gives you tools for that question. Counting and comparing is where it starts."

10 **Two errors on purpose (3m).** Say: "Now the two mistakes everyone makes this week. First, the word `and`." Read the last line.

```
both = (revenue > 10000) and (promo == 1)
```

```
ValueError: The truth value of an array with more than one element is ambiguou
```

Say: "`and` wants one true-or-false value and each side is 2,903 answers. Same message as `if week > 6000` last session." Now drop the parentheses:

11 **The missing parentheses.**

```
both = revenue > 10000 & promo == 1
```

```
TypeError: ufunc 'bitwise_and' not supported for the input types, and the inpu
```

Say: "Cryptic, but learn the keyword: when an error says `bitwise_and`, look for missing parentheses. Python tried `10000 & promo` first." Fix it on screen: `(revenue > 10000) & (promo == 1)`.

12 **An if/else for every element (3m).** Say: "A mask answers 'which ones?'. `np.where` answers 'this value where it holds, that value where it does not'. For three or more outcomes, `np.select`: first true condition wins, like if/elif."

```
print(np.where(week > 9000, "strong", "normal"))

conditions = [np.isnan(revenue), revenue >= 12000, revenue >= 6000]
tier = np.select(conditions, ["missing", "high", "medium"], default="low")
names, counts = np.unique(tier, return_counts=True)
print(names)
print(counts)
```

```
['normal' 'normal' 'normal' 'normal' 'strong' 'strong' 'strong']
['high' 'low' 'medium' 'missing']
[ 260 1120 1517    6]
```

Ask: "Why is the `isnan` condition first?" (First match wins; a missing value fails both comparisons and would fall through to "low".) `np.unique(..., return_counts=True)` is the quickest tally in NumPy.

13 **Which way to collapse? (3m).** Point at board rule three. Draw the 3 by 4 table on the board and physically squash it flat downward for `axis=0`. Ask for each shape before running.

```
units3 = np.array([[110, 105, 148,  69],
                   [ 88,  84, 152,  82],
                   [103, 133,  90,  92]])

print(units3.sum())           # everything
print(units3.sum(axis=0))     # collapse the rows: one total per store
print(units3.sum(axis=1))     # collapse the columns: one total per day
```

```
1256
[301 322 390 243]
[432 406 418]
```

Say: "Shape `(3, 4)`. `axis=0` squashes the 3 away and leaves 4 numbers. If you cannot remember which is which, print the shape of the result: it tells you what survived."

14 **Positions and parallel arrays (2m).** Say: "`argmax` does not return the largest value. It returns WHERE it is. A position can look something up in a parallel array."

```
days = np.array(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])

print(week.argmax(), days[week.argmax()], week.max())
print(days[np.argsort(week)][::-1][:3])      # the top three days
```

```
5 Sat 12388.57
['Sat' 'Fri' 'Sun']
```

`np.argsort` gives the positions that would sort the array; reversed and cut to three, they index the day names.

15 **The quiet one: nan fails every comparison (2m).** Ask: "Every store-day is either above zero or it is not. So these two counts add up to 2,903. Yes?"

```
print(revenue.size)
print((revenue > 0).sum() + (revenue <= 0).sum())
print((revenue == np.nan).sum(), np.isnan(revenue).sum())
```

```
2903
2897
0 6
```

Say: "Six rows are in neither group, and nothing equals `nan`, not even `nan`. 'The rest' is not the same as 'not the first group'. Test for missing values on purpose with `np.isnan`."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **`and` / `or`, or no parentheses.** Symptoms: `ValueError: The truth value of an array with more than one element is ambiguous`, or `TypeError: ufunc 'bitwise_and' not supported for the input types`. Coach: "Read me the last line. Which word in your line wants a single True or False?" For the second: "What does Python do first here, the `&` or the `>`? Put a pair of parentheses around each question." Do not type the fix for them; point at board rule two.
- **The wrong axis, with no error.** Symptom: 56 numbers where 4 were expected (or the reverse), or a plausible answer to a different question. Coach: "How many answers should there be: one per store or one per day? Print `.shape` of what you got." Then: "Which axis has to disappear to leave that many?" In Lab B make every learner say the expected shape before running each aggregation.
- **Forgetting that `nan` is in neither group.** Symptom: counts that do not add up to 2,903; a mean of `nan` after filtering revenue; `(revenue == np.nan).sum()` giving 0. Coach: "How many store-days are there? How many are in your two groups? Where are the other six?" The habit: any time revenue is involved, ask what happens to the six missing values, and use `np.isnan` and the `nan`-aware functions on purpose.

GUIDED LAB

Lab A (13m, everyone): `labs/module-02/l1_2-vectorized-operations/staffing_load_starter.py`; reference `labs/module-02/l1_2-vectorized-operations/staffing_load_solution.py`. A question the lesson does not ask: how many visitors does

each staff member look after, and when is a store stretched? One division, one mask, two combined masks, `np.select`, `argmax` with parallel arrays.

Expected, Lab A: visitors per staff member: mean 55.3, min 28.2, max 116.5; 814 stretched store-days (60 or more), 28.0%; stretched AND sale day 26; stretched OR sale day 884; labels quiet 361, steady 1,728, stretched 814; most stretched day 2025-02-26 at Bend (233 visitors, 2 staff).

Circulate in Lab A for: `stretched and is_promo`; `per_staff >= 60 & promo == 1` (the `bitwise_and` error); `np.select` conditions in the wrong order (`>= 45` first labels everything from 45 up as steady and the stretched count comes out 0: ask "which condition does a value of 80 meet first?"); `per_staff.max()` printed where the date and store are wanted (ask "that is the value: where is it?").

Debrief Lab A (2m): show the solution's last block: visitors per staff is 52 to 58 at all four stores. Ask why (staff are scheduled from expected demand, so a bigger store simply has more people on duty). That idea returns in Module 4 as a confound.

Lab B (10m, pairs): `labs/module-02/l1_2-vectorized-operations/eight_weeks_axis_starter.py`; reference `labs/module-02/l1_2-vectorized-operations/eight_weeks_axis_solution.py`. A 56-day by 4-store table of transactions; every answer is one aggregation with the right axis. The rule for pairs: the partner who is not typing says the expected shape out loud before each run.

Expected, Lab B: total 13,255; per store `[2935. 3312. 4347. 2661.]`; first five day totals `[204. 196. 196. 234. 287.]`; busiest day per store 2024-02-24, 2024-02-24, 2024-01-13, 2024-01-27; three busiest days 2024-02-24 (392), 2024-01-13 (374), 2024-01-20 (360); days won: Asheville 1, Bend 8, Boise 45, Flagstaff 2.

Circulate in Lab B for: `table.argmax(axis=1)` used for "each store's busiest day" (56 positions, all 0 to 3: ask "can a row number be 3 at most when there are 56 rows?"); `day[top3]` forgotten, so positions 54, 12, 19 are reported as if they were answers; `np.argsort(day_total)[:3]` (the three QUIETEST days).

Fast finishers: `labs/module-02/l1_2-vectorized-operations/debug_masks.py`: two bugs that raise errors (`and`; missing parentheses) and two that do not (the six `nan` rows in neither group; a "copy" that is a second name, so capping it changes the real data). Fixed answers: 15, 15, 6, and 101 capped values with `revenue` untouched.

START THE LESSON PROJECT

The lesson project is the store scorecard: conversion, basket value and sale-weekend lift for all four stores, with no loop over the 2,903 rows. In the room, do **Steps 1 and 2** (10 minutes).

Step 1 together: load seven arrays, including `dates` with `dtype="datetime64[D]"` and `stores` with `dtype=str`. Check: `2903 store-days, 6 with no revenue`. Step 2 on their own: conversion, basket value and units per sale, one array expression each. Done in the room = `23.5%`, `$88.64`, `2.11`.

Goes home: Step 3 (one line per store; the only loop runs four times), Step 4 (sale-day lift 1.93x, weekend mean $10,073.19, the weekday trick with date subtraction), Step 5 (`np.select` labels: the four counts add up to 2,903) and Step 6 (the five biggest days, all Boise sale days, found with `argsort`). Tell them the

headline before they leave so they know where it is going: conversion and basket value are nearly identical at every store; Boise's extra revenue is almost all foot traffic.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

1. `big = revenue > 15000`. What is `big.sum()`? What is `revenue[big].sum()`? **Answer:** The first counts the store-days above $15,000 (101). The second adds up their revenue.
2. How do I write "Flagstaff or Bend" as a mask? **Answer:** `(stores == "Flagstaff") | (stores == "Bend")`, or `np.isin(stores, ["Flagstaff", "Bend"])`.
3. A table has shape `(56, 4)`. What is the shape of `table.mean(axis=0)`, and what does it hold? **Answer:** `(4,)`: one average per store. The 56 days were collapsed.
4. `picked = week[week > 9000]`, then `picked[0] = 0`. Did `week` change? **Answer:** No. Selecting with a mask builds a new array. Assigning through the mask, `week[week > 9000] = 0`, would change it.
5. When is a Python loop still fine? **Answer:** Over a handful of groups (four stores, twelve months), or when each step truly depends on the previous result. Not over the rows.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** A) `3 0.75`; **Q2** C) `((stores == "Bend") & (visitors > 500)).sum()`; **Q3** `[3 6] (3,)`; **Q4** B) `2`; **Q5** `['retry' 'pass' 'pass']`; **Q6** D) `Boise 3`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: For learners who write the loop in their head first: let them. Have them say the loop in words ("for each day, if it is above 9,000, count it") and then find each phrase in the lesson's loop-to-array table, which is printed in the workbook's key ideas. For the axis, use fingers on the printed table: "squash it this way: how many numbers are left?" Pair anyone who missed Session 1's views question with a partner for Lab A.

Stretch: `debug_masks.py`; then the lesson's Exercise 4 (retire the shipping loop with `np.select` and check it with `np.allclose`) or the Bonus Challenge (a 7 by 4 table of mean revenue by weekday and store built from 28 masks, which Lesson 1.3 then standardizes in one line).

1.3 Broadcasting

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- State the broadcasting rules and use them to predict the shape of a result, or that an operation will fail
- Combine a 2-D table with a row of values and with a column of values
- Reshape an array for broadcasting with `np.newaxis`, `reshape` and `keepdims=True`
- Build a grid of every combination from two 1-D arrays
- Recognize the broadcasting mistakes that raise no error, and catch them by checking shapes

TIMING IN THE ROOM

10m	Frame: the rubber stamp; the two rules; shapes written right-aligned
25m	Live demo: table and row, table and column (the error, then `keepdims`), `np.broadcast_shapes`, three ways to add an axis, a grid, z-scores, two silent mistakes
25m	Guided lab: `fair_visitors_starter.py`, then `target_grid_starter.py`
15m	Start the lesson project: the weekday index, Steps 1 and 2

MENTAL MODEL TO SAY

Say: "Picture a paper form with seven lines and a rubber stamp that prints one line of four numbers. To stamp the whole form you press it on line 1, then line 2, down the page. The stamp does not get longer. It is reused. Broadcasting is NumPy doing the stamping: when one array is missing a dimension, or has length 1 along it, NumPy reuses it along that dimension."

The two rules, written on the board as a procedure learners will do on paper all session: **1. Write the shapes one above the other, lined up on the RIGHT. A missing entry counts as 1. 2. Each pair must be equal, or one of them is 1.** The result takes the larger of each pair. Any pair that fails: the "could not be broadcast together" error.

And the pattern to remember, under the rules: **per column (per store): a table and a row, it just works. Per row (per day): a table and a column, so `keepdims=True`.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_3`. Before **every** operation in this demo, write the two shapes on the board one above the other, right-aligned, and have the room call the result or "error". That paper habit is the skill; the code is short. Steps 11 and 12 are mistakes that raise no error: let the wrong number sit on the screen and ask whether anyone would have caught it.

1. **The week as a table (2m).** Say: "The first week of revenue again: 7 days by 4 stores, Asheville, Bend, Boise, Flagstaff. And the broadcast you already know: a table and one number."

```
import numpy as np

def load(col, dtype=float):
    """Read one column of daily_sales.csv into an array."""
    return np.genfromtxt("data/daily_sales.csv", delimiter=",",
                         skip_header=1, usecols=col, dtype=dtype)

revenue = load(5)
week1 = revenue[:28].reshape(7, 4)
print(week1.shape)
print((week1 / 1000).round(1)[:2])        # revenue in thousands: first two da
```

```
(7, 4)
[[5.  4.1 6.  3.1]
 [3.6 3.7 6.6 3.6]]
```

2. **A table and a row (3m).** Board: `(7, 4)` over `(4,)`. Ask: "4 meets 4. 7 meets nothing. Result?" Then run.

```
store_mean = week1.mean(axis=0)
print(store_mean.round(2), store_mean.shape)

diff = week1 - store_mean
print(diff.shape)
print(diff.round(0)[-2:])                 # Saturday and Sunday
```

```
[5099.55 5446.93 7914.32 4681.29] (4,)
(7, 4)
[[1703. 1636. 4474. 1576.]
 [-985. 1272. 1447. 1283.]]
```

Say: "The row of four averages was stamped onto each of the seven rows. Every cell now reads 'dollars above or below this store's own average'."

3. **A ratio reads better across stores (2m).** Say: "Boise is nearly twice Asheville's size, so raw dollars only say 'Boise is bigger'. Divide by each store's own average instead."

```
print((week1 / store_mean).round(2)[-3:])    # Friday, Saturday, Sunday
```

```
[[1.44 0.87 1.21 1.2 ]
 [1.33 1.3  1.57 1.34]
 [0.81 1.23 1.18 1.27]]
```

Read one cell: "Boise's Saturday at 1.57 is 57% above Boise's weekly average." Every store's best day now stands out on the same scale.

4 **A table and a column: the error (3m).** Say: "Next question: what share of each day's total did each store take? Every row divided by its own total." Board: `(7, 4)` over `(7,)`. Ask the room to apply the rules before you run it.

```
day_total = week1.sum(axis=1)
print(day_total.shape)
share = week1 / day_total
```

```
(7,)
ValueError: operands could not be broadcast together with shapes (7,4) (7,)
```

Read the shapes in the message from the right: "4 meets 7. Not equal, neither is 1." Say: "A flat array can only be a row, and a row of seven does not fit a table four wide. We need seven numbers standing as a column."

5 **keepdims keeps the axis (3m).** Say: "Ask the aggregation to keep the axis it collapsed." Board: `(7, 4)` over `(7, 1)`.

```
day_total = week1.sum(axis=1, keepdims=True)
print(day_total.shape)

share = week1 / day_total
print(share.round(3)[:2])
print(share.sum(axis=1))          # every row should add up to 1
```

```
(7, 1)
[[0.276 0.227 0.328 0.168]
 [0.207 0.211 0.377 0.205]]
[1. 1. 1. 1. 1. 1. 1.]
```

The last line is a check worth making every time: shares of a whole must add up to 1.

6 **Let NumPy apply the rules (2m).** Say: "You can ask for the result shape without doing any arithmetic." Have the room predict each line.

```
print(np.broadcast_shapes((7, 4), (4,)))
print(np.broadcast_shapes((7, 4), (7, 1)))
print(np.broadcast_shapes((3, 1), (1, 5)))
```

```
(7, 4)
(7, 4)
(3, 5)
```

The last one is new: both arrays have a 1, in different places, and both are stretched. A column of 3 meets a row of 5: a 3 by 5 table.

7 **Three ways to make a column (2m).** Say: "Most broadcasting work is one move: turn n flat values into a column of shape (n, 1). You will meet all three spellings in other people's code."

```
totals = week1.sum(axis=1)                  # flat: shape (7,)

col_a = totals[:, np.newaxis]               # index with a new axis
col_b = totals.reshape(-1, 1)               # -1 means "work this out for me"
col_c = week1.sum(axis=1, keepdims=True)    # never flatten in the first place
print(totals.shape, col_a.shape, col_b.shape, col_c.shape)
```

```
(7,) (7, 1) (7, 1) (7, 1)
```

Advice to give: if the small array comes from an aggregation of the big one, use `keepdims=True`. It is right for every shape.

8 **Column meets row: a grid (3m).** Say: "Four tent prices, three possible discounts. Every combination, with no loop inside a loop." Board: `(4, 1)` over `(3,)`.

```
prices = np.array([299.95, 217.95, 449.00, 129.50])     # four products
discounts = np.array([0.10, 0.15, 0.20])                # three scenarios

sale_price = prices[:, np.newaxis] * (1 - discounts)
print(sale_price.shape)
print(sale_price.round(2))
```

```
(4, 3)
[[269.96 254.96 239.96]
 [196.16 185.26 174.36]
 [404.1  381.65 359.2 ]
 [116.55 110.08 103.6 ]]
```

Rows are products, columns are discount levels. Ask: "What would `prices * (1 - discounts)` do, without the new axis?" (`(4,)` with `(3,)`: an error.)

9 **Standardize every column in one line (2m).** Say: "Subtract each column's mean, divide by its standard deviation: a z-score. Two broadcasts in one expression, both a table and a row."

```
z = (week1 - week1.mean(axis=0)) / week1.std(axis=0)
print(z.round(2)[-2:])
print(z.std(axis=0))
```

```
[[ 1.29  1.35  1.73  1.36]
 [-0.75  1.05  0.56  1.1 ]]
[1. 1. 1. 1.]
```

Say: "Boise's Saturday is 1.73 standard deviations above Boise's own average. You can now compare stores of different sizes directly. This one line comes back in Module 4 and in Lesson 2.9."

10 **The accidental grid (3m).** Say: "Four actual values, four predictions, and I want the four errors. One array arrived as a column." Run it and say nothing for a moment.

```
actual = np.array([100.0, 200.0, 300.0, 400.0])
predicted = np.array([[110.0], [190.0], [330.0], [380.0]])    # shape (4, 1)

errors = actual - predicted
print(errors.shape)
print(np.abs(errors).mean())      # the average size of an error
```

```
(4, 4)
126.25
```

Ask: "Is 126.25 right?" Board: `(4,)` over `(4, 1)`: a row meets a column, so NumPy built all sixteen combinations. No error, and a plausible number that means nothing. Fix:

11 **Four in, four out.**

```
errors = actual - predicted.ravel()
print(errors.shape, errors, np.abs(errors).mean())
```

```
(4,) [-10.  10. -30.  20.] 17.5
```

Say: "The habit that catches it: four values in should mean four values out. Print the shape."

12 **The square table that hides a wrong axis (2m).** Say: "With seven rows, forgetting `keepdims` gave an error. With four rows and four columns it does not."

```
square = week1[:4]                                  # 4 days x 4 stores
wrong = square - square.mean(axis=1)                # flat array of 4 day-means
right = square - square.mean(axis=1, keepdims=True)

print(np.abs(wrong.sum(axis=1)).round(2))           # should be zeros if the rows
print(np.abs(right.sum(axis=1)).round(2))
```

```
[ 829.24 1563.14  620.76 3013.13]
[0. 0. 0. 0.]
```

Say: "The four day-means happened to fit as a row, so day 1's mean came off column 1, day 2's off column 2. Wrong answer, no complaint. Use `keepdims=True` whenever you aggregate along `axis=1` and combine with the table: then the code is right for every shape."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A flat array where a column was needed.** Symptom: `ValueError: operands could not be broadcast together with shapes (224,4) (224,)`. This is the friendly one. Coach: "The message gives you both shapes. Write them one above the other, right-aligned. Which pair fails?" Then: "What shape would work? Which keyword keeps that axis?" Resist saying `keepdims` first.
- **The accidental grid.** Symptom: no error; a result with far more values than expected, or a summary number that is wildly off (an average error of 1,753 when every forecast was within about 250). Coach: "How many answers should there be? Print `.shape`." Then the board procedure on `(4,)` and `(4, 1)`. Lab B Step 4 is built to produce this; let pairs be puzzled for a minute before you step in.
- **The square table that hides a wrong axis.** Symptom: no error and a right-shaped result that is simply wrong, only when rows and columns happen to be equal in number. Coach: "Check it: if each row was centered on its own mean, what should each row add up to?" The durable fix is a habit, not a patch: `keepdims=True` on every `axis=1` aggregation that is combined with its table, and a check line such as `share.sum(axis=1)`.

GUIDED LAB

Lab A (13m, everyone): `labs/module-02/l1_3-broadcasting/fair_visitors_starter.py`; reference `labs/module-02/l1_3-broadcasting/fair_visitors_solution.py`. Visitors (not the project's revenue) for 224 days by 4 stores. A table and a row to compare each store with its own normal; a table and a column for each store's share of the day. Learners write the two shapes and a prediction in the starter's comments before each operation.

Expected, Lab A: store averages `[281.4 323.4 440.9 281.6]`; ratio shape `(224, 4)`; busiest day at 2.51, 2.45, 2.30 and 2.40 times normal, all four on 2024-05-25; rows of the share table add up to 1; average

share `[0.213 0.244 0.332 0.211]`; Boise's largest share 0.406 on 2024-04-20 and smallest 0.243 on 2024-02-10.

Circulate in Lab A for: skipping the written prediction (insist: it is the skill); `table.mean(axis=1)` for the store averages (224 numbers: ask how many stores there are); the `keepdims` error in Step 4, which the starter asks them to cause on purpose and read from the right.

Debrief Lab A (2m): "All four stores had their busiest day, relative to normal, on the same date. What is special about Saturday May 25, 2024?" (The Memorial Day sale weekend. The project finds the same weekend with z-scores on revenue.)

Lab B (10m, pairs): `labs/module-02/l1_3-broadcasting/target_grid_starter.py`; reference `labs/module-02/l1_3-broadcasting/target_grid_solution.py`. Four shape predictions checked with `np.broadcast_shapes`, a 4 by 3 grid of revenue targets (stores by growth scenarios), a mask on the grid, then a forecast check that silently builds a 4 by 4 grid.

Expected, Lab B: `(4,)` with `(3,)` fails; `(4, 1)` with `(3,)` gives `(4, 3)`; `(224, 4)` with `(4,)` gives `(224, 4)`; `(4,)` with `(4, 1)` gives `(4, 4)`. Targets: Asheville 5875, 5989, 6274; Bend 7208, 7348, 7698; Boise 10449, 10651, 11159; Flagstaff 7096, 7234, 7579. Five of twelve above $7,500; per store `[0 1 3 1]`. Step 4 as shipped: shape `(4, 4)`, average size 1753.02. Fixed: errors `[-196.17 197.87 -255.79 189.75]`, average size 209.9.

Circulate in Lab B for: `current * (1 + growth)` with no new axis (the `(4,)` `(3,)` error: perfect, ask them to draw it); `above.sum(axis=0)` for "per store" (three numbers, one per scenario); pairs who "fix" Step 4 by taking `errors[0]` or the diagonal instead of fixing the shape.

Fast finishers: `labs/module-02/l1_3-broadcasting/debug_shapes.py`: one loud bug and two silent ones (a flat array where a column was needed; a `(7,)` minus `(7, 1)` grid of 49 "gaps"; the square-table trap). Fixed answers: seven zeros; 7 gaps with a largest gap of 6131.55; four zeros.

START THE LESSON PROJECT

The lesson project is the weekday index: how much is a Saturday worth at each store, on a scale where 1.00 means "an average day for this store", plus the days that were unusual for their own store. In the room, do **Steps 1 and 2** (15 minutes).

Step 1 together, slowly, because the reshape only works when no rows are missing: the first 896 rows are exactly 224 days of 4 stores (January 1 to August 11, 2024; Bend's export fails the next day). Check the store order by printing three rows of the reshaped names. Done = `table.shape` is `(224, 4)` and missing values per store are `[1 1 0 1]`.

Step 2 on their own: `store_mean = np.nanmean(table, axis=0)` and `ratio = table / store_mean`. Done in the room = store means `[5697.45 6650.89 9108.3 5895.63]` and a ratio table of shape `(224, 4)`. Ask: "Why does this broadcast as it stands?" (A table and a row.)

Goes home: Step 3 (reshape to `(32, 7, 4)` and average over the weeks: Saturday is about 1.5 at every store), Step 4 (the line chart: four lines that nearly overlap), Step 5 (z-scores above 3: 13 store-days, eight of

them on the Memorial Day sale weekend) and Step 6 (shares with `keepdims=True`: Boise takes about a third).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

1. Shapes `(5, 3)` and `(5,)`. Does it broadcast? **Answer:** No. From the right, 3 meets 5: not equal, neither is 1. The five values need to be a column, `(5, 1)`.
2. I want each store's column minus that store's mean. Do I need `keepdims`? **Answer:** No. Per-column values are a row of shape `(4,)`, which lines up with the table's last axis as it stands.
3. What does a column of shape `(3, 1)` times a row of shape `(5,)` give? **Answer:** A `(3, 5)` table of every combination.
4. Name the three ways to turn a flat array of n values into a column. **Answer:** `arr[:, np.newaxis]`, `arr.reshape(-1, 1)`, or `keepdims=True` on the aggregation that made it.
5. Does broadcasting copy the small array seven times before subtracting? **Answer:** No. The small array is reused from the same memory. Only the result is full size.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** B) An array of shape `(6, 3)`; **Q8** C) `t - t.mean(axis=1, keepdims=True)`; **Q9** `(2, 3) 60`; **Q10** D) `(8, 5)` and `(8,)`; **Q11** `(4, 3)`; **Q12** A) An array of shape `(12, 12)`: every sales value minus every plan value, with no error. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Hand out a half-sheet with six pairs of shapes and two empty right-aligned boxes for each; learners fill in the result or "error" on paper before touching a keyboard (the lesson's Exercise 2 pairs work well). For anyone who is lost in `axis=`, stay with the 7 by 4 week and the two questions "per store or per day?" and "row or column?" until both come quickly; the 224-day table adds nothing new.

Stretch: `debug_shapes.py`; then the lesson's Exercise 4 (a 4 by 4 table of how alike the stores' weekly profiles are, built with two new axes and no loop) or the Bonus Challenge (the 32 by 7 by 4 array: each store's share of its own week, and the week when Saturday's share was highest at Boise).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Three questions on the slide, answered on a sticky note or in the workbook's Exit Ticket box. Question 3 is the one to sort by: a learner who writes `keepdims=True` and can say why has the session.

1. `revenue` and `promo` are parallel arrays. Write one line that counts the sale days (`promo` equal to 1) with revenue above 10,000. **Answer:** `((revenue > 10000) & (promo == 1)).sum()`: `&`, parentheses around each comparison, and the sum of the mask counts.

2. A table has shape `(224, 4)`: days by stores. What is the shape of `table.mean(axis=0)`, and what does each value mean? **Answer:** `(4,)`: one mean per store. The axis you name (the 224 days) is the one that disappears.

3. You want every row of that table divided by its own row total. Write the expression, and say what goes wrong if you leave out the keyword. **Answer:** `table / table.sum(axis=1, keepdims=True)`. Without `keepdims=True` the totals have shape `(224,)`; from the right, 4 meets 224 and NumPy raises "could not be broadcast together".

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The store scorecard (Steps 3 to 6: four store rows, sale-day lift 1.93x, weekend mean $10,073.19, labels that add up to 2,903, the five biggest days) and the weekday index (Steps 3 to 6: Saturday near 1.5 at every store, the chart, 13 store-days with z above 3, Boise's share about a third).
- **Read in full** the two 1.2 sections the room only touched: "Sorting and Picking by Position" and "From Loop to Array Expression".
- **Lesson exercises.** 1.2: Exercises 1 to 3 (units per visitor, two conditions, along each axis); Exercise 4 (retire a loop) is a stretch. 1.3: Exercises 1 to 3 (row then column, predict the shape, scale every store from 0 to 1); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages, and **both journal entries**.
- **Before the Next Lesson (from 1.3):** keep the project notebook; predict what `(8, 1, 3)` and `(5, 1)` give, then check with `np.broadcast_shapes`; and write down a guess: if you picked 30 days at random from Boise's revenue and added them up, then did it again with a different 30, how different would the two totals be?
- **If Lab B of either lesson was not finished in class,** finish it: the starter says what to expect.

BRING LIST FOR SESSION 3

- The same working environment (READY). Nothing new to install: `numpy` and `matplotlib` are already there.
- The `pinecrest-analysis` folder with `data/` and `labs/`. Session 3's starters are in `labs/module-03/`.
- The finished Lesson 1.3 project notebook: the 224-day by 4-store table is used again.
- The written guess from 1.3's "Before the Next Lesson" (two random 30-day totals: how different?). Session 3 opens Lesson 1.4 with it.
- The workbook and the journal. No servers, no accounts, no internet needed.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 1.3 demo Steps 4, 5 and 10 to 12; Lab A of each lesson; the exit ticket.
- **Cut first (saves 3m):** 1.2 demo Steps 3 and 4 (ufuncs). Point at the lesson's ufunc table instead.
- **Cut second (saves 10m):** 1.2 Lab B (`eight_weeks_axis`). Do its fill-in from the workbook on the board in two minutes and assign the file as homework.
- **Cut third (saves 4m):** 1.3 demo Steps 7 and 9 (three ways to add an axis; z-scores). The project teaches z-scores at home.
- **Cut fourth (saves 10m):** the 1.2 project start. Read the brief, give the three check numbers (23.5%, $88.64, 2.11), send all six steps home.
- **Do not cut** the 1.3 project's Step 1 walk-through: the "is this reshape safe?" check is hard to do alone the first time.

RUNNING SHORT: WHAT TO ADD

- Run `debug_masks.py` or `debug_shapes.py` as a pairs race; the first pair explains each bug at the projector.
- Do 1.3's Exercise 2 (predict the shape for five pairs) as a show of fingers: 1 for "works", 2 for "error", then call the shape.
- Do 1.2's Exercise 3 (along each axis) on the board with the hand-drawn 7 by 4 table: learners call `axis=0` or `axis=1` and the shape before you type.
- Start Step 3 of the weekday index together: the reshape to `(32, 7, 4)` and averaging over the weeks is the first three-dimensional array of the course, and it is easier with a picture.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 3: Randomness and Linear Algebra

How to teach the session that closes the NumPy module. In the first half learners make their own numbers: random draws they can reproduce exactly, fair samples and shuffles of real rows, and small simulations that answer "what could happen?", with the discipline of checking a model against reality. In the second half they meet the working parts of linear algebra with no proofs: the dot product as a receipt, the `@` operator, solving for unknowns, and least squares, which is linear regression. They leave having fitted a model to real store data with a matrix they built themselves, and having questioned what its coefficients mean.

`2 lessons` `3 hrs session` `Lessons 1.4–1.5` `Session 3 of 21`

HOW TO USE THIS GUIDE

- **Same rhythm for both lessons:** frame (one mental model) → live demo (you type, they type along) → guided lab (two kit labs per lesson: new questions on the course data) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- **Two different kinds of checking today.** In 1.4 the check is "does my simulation agree with the real data?". In 1.5 it is last session's shape check, plus "put the answer back in" (`np.allclose(A @ x, b)`) and "what else could explain this coefficient?".
- **Every code block below was run** to produce the output printed under it. The 1.4 outputs depend on drawing in the order shown from a fresh generator: type the steps in order, each cell once.
- **The lessons are the reference.** Keep `numpy_random_generation.html` and `numpy_linear_algebra.html` open for the figures: the three distribution histograms, the estimate settling as simulations accumulate, the receipt, the shape rule, and the scatter plot with its least-squares line. Labs live in `labs/module-03/`.
- **This session ends Module 1.** The wrap-up names what carries over to pandas (everything) and hands over to Session 4. Teal blocks are hands-on, amber blocks are watch-outs.

> **THE #1 RISK OF SESSION 3: A CONFIDENT NUMBER THAT NOBODY QUESTIONS**
>
> Both lessons produce impressive output very easily: a probability to three decimals from one line of simulation, a regression coefficient from one call to `lstsq`. The danger is that learners leave trusting whatever comes out. Each lesson carries its own antidote, and both are the parts most easily cut when time is short, so protect them. In 1.4 it is the second gift-bag model (the answer moves from about 4% to about 28% when one assumption changes) and Lab B's reality check. In 1.5 it is reading a slope as "goes with" and quoting a forecast with its typical miss; the staffing trap (Exercise 4) is the homework that makes it stick. The second risk is the title of 1.5. "Linear algebra" frightens people who had a bad time with it in school. Say early and plainly that there are no proofs, that a vector is a 1-D array and a matrix a 2-D array, and keep the receipt on the board.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-03` from the `instructor-kit` folder (8 files, all PASS).
- ☐ **Files to hand out:** `labs/module-03/` without the `*_solution.py` files: four starters and the handouts `debug_random.py` and `debug_linalg.py`.
- ☐ **Servers to start:** none. No internet needed.
- ☐ **Projector machine:** `(.venv)` active in `pinecrest-analysis`, two empty notebooks `lesson_1_4` and `lesson_1_5`, font 18 pt or more. Browser tabs: both lesson pages, scrolled to the distribution histograms (1.4) and the Boise scatter plot (1.5).
- ☐ **Rehearse the 1.4 demo once from a restarted kernel**, in order, so the numbers on your screen match this guide. One accidental rerun of a cell changes everything after it.
- ☐ **Board, before learners arrive:** a blank column headed "Two random 30-day totals: how different?" for the homework guesses. Space for **model → repeat → count**. For the second half: the receipt's four sides and the shape rule.
- ☐ **Sort last session's exit tickets.** If many missed `keepdims`, give warm-up question 5 an extra minute: 1.5 leans on shapes from its first matrix product.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
A learner's random numbers do not match yours or the lesson page	A cell was run twice or out of order; or a different seed; or draws made in a different order	Restart the kernel and run from the top. If it still differs from the lesson page after the dice in Step 2, that is expected: the lesson draws more in between.

SYMPTOM	CAUSE	FIX
`AttributeError: module 'numpy.random' has no attribute 'default_rng'`	A very old NumPy in a kernel that is not the course environment	Run `check_environment.py`; select the `.venv` kernel.
A chart window does not appear, or `plt.show()` prints nothing, in a script	A script run in a terminal without a display; or the figure was closed	Work in a notebook for the project's charts, or save with `fig.savefig("months.png")`. Charts are taught in Module 3; today they are typed as given.
`LinAlgError: Singular matrix` from `np.linalg.solve`	Two rows (or columns) of the matrix carry the same information	Print the matrix. One equation repeats another: a different slip, or `lstsq`, is needed.
`lstsq` returns `[nan nan]`	A missing value in the data	One mask, `& ~np.isnan(revenue)`, applied to every array before building the matrix.
`FutureWarning` about `rcond` from `np.linalg.lstsq`	NumPy older than 2.0	Harmless; `rcond=None` silences it. Check the environment: the course uses NumPy 2.5.

SESSION 3 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 2 (no notes), homework check, collect the "two random months" guesses.
0:10	10	**1.4** Frame and mental model	Why make numbers up; the book of random digits and the seed as a page number.
0:20	25	**1.4** Live demo	A seeded generator, the moving bookmark, `choice`, shuffling, sampling rows, model-repeat-count, precise is not the same as right.
0:45	25	**1.4** Guided lab	`labs/module-03/l1_4-random-generation/mystery_shopper_starter.py`, then `returns_desk_starter.py` in pairs.
1:10	10	**1.4** Start the lesson project	10,000 possible months, Steps 1 and 2; Steps 3 to 7 go home.
1:20	10	Break	Write the receipt's four sides on the board for the second half.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:30	10	**1.5** Frame and mental model	The receipt; vector and matrix are names for arrays you already use; the shape rule.
1:40	30	**1.5** Live demo	Dot product, matrix @ vector, `@` is not `*`, transpose, `solve`, `lstsq` with RMSE and R squared, a prediction, distance.
2:10	25	**1.5** Guided lab	`labs/module-03/l1_5-linear-algebra/payroll_receipt_starter.py`, then `basket_model_starter.py` in pairs.
2:35	15	**1.5** Start the lesson project	A first revenue model for Asheville, Steps 1 and 2; Steps 3 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, Module 1 close, bring list for Session 4.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 85 + 90 = 175 minutes for a solo learner, and after the warm-up, the break and the wrap-up the room has 150. The room does, for each lesson: the mental model, the live demo, two guided kit labs, and the first two steps of the lesson project.

Goes home (about 3 hours: this is the heaviest homework of the module). The month simulation Steps 3 to 7 and the Asheville revenue model Steps 3 to 6; each lesson's Hands-on Exercises (three, plus a stretch, and 1.5's stretch Exercise 4 is strongly recommended); both Quick Quizzes; both journal entries (1.5's prompt reviews all of Module 1); each lesson's "Before the Next Lesson" list. Lesson 1.4's section on distributions (normal, Poisson, binomial, with the three histograms) gets one demo step: tell learners to read that section in full.

Protect, in this order: 1.4 demo Step 12 (the better gift-bag model) and Lab B's reality check; 1.5 demo Steps 9 to 13 (least squares from matrix to prediction); Lab A of 1.5. **Cut first:** 1.5 demo Step 14 (distance: the workbook's worked example covers it), then 1.5 demo Steps 2 and 7 (weighted average, transpose), then Lab A of 1.4 (assign as homework), then the 1.4 project start.

WARM-UP

0:00 Warm-up: Retrieval from Session 2 — 10 min

HOW TO RUN IT

- Slide up, notes closed, three minutes of silent writing in the workbook's Warm-up box, then answers out loud, one question per learner.
- Questions 3 to 5 are the ones today depends on: `axis`, the two rules and `keepdims`. Lesson 1.4 sums a `(10000, 30)` table along `axis=1`, and Lesson 1.5 is shape arithmetic from the first matrix product.
- Keep it to seven minutes so three are left for the homework check.

FIVE QUESTIONS (ANSWERS FOR YOU)

1. `big = revenue > 15000`. What does `big.sum()` tell you? And `big.mean()`? **Answer:** How many store-days are above $15,000, and what fraction of all store-days that is.
2. Write a mask for "Boise and a sale day", given the parallel arrays `stores` and `promo`. **Answer:** `(stores == "Boise") & (promo == 1)`: `&`, with parentheses around each comparison.
3. `table` has shape `(224, 4)`: days by stores. What is the shape of `table.sum(axis=1)` and what does it hold? **Answer:** `(224,)`: one total per day. The axis you name (the 4 stores) disappears.
4. State the two broadcasting rules. **Answer:** Line the shapes up from the right, a missing entry counting as 1. Each pair of lengths must be equal, or one of them 1; the result takes the larger.
5. How do you divide every row of that table by its own row total? **Answer:** `table / table.sum(axis=1, keepdims=True)`: the totals must stand as a column of shape `(224, 1)`.

HOMEWORK CHECK

- Collect the guesses from 1.3's "Before the Next Lesson": two random 30-day totals of Boise revenue, how different? Write four or five on the board and leave them there: the 1.4 project answers the question at 1:10.
- Quick show of hands on the two projects: who got a sale-day lift of 1.93x? Who has a weekday index with Saturday near 1.5 at every store?
- Ask one learner to state the accidental grid in a sentence (a row meets a column: n in, n-squared out, no error). If nobody can, take one minute for it now.
- Unfinished Lab B files from Session 2: note who, and have a word at the break.

LESSON PLANS

1.4 Random Number Generation

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Create a random number generator with `np.random.default_rng` and draw floats, integers and arrays of any shape from it
- Explain what a seed does, and structure your code so its random results can be reproduced
- Sample from an array with and without replacement, shuffle it, and split rows at random
- Draw from the normal, Poisson and binomial distributions and say what kind of quantity each one models
- Estimate a probability by simulation, and judge how far to trust the estimate and the model behind it

TIMING IN THE ROOM

10m	Frame: why make numbers up; the book of random digits; the homework guesses on the board
25m	Live demo: a seeded generator, the moving bookmark, `choice`, shuffling, sampling rows, a binomial, model-repeat-count, precise is not the same as right
25m	Guided lab: `mystery_shopper_starter.py`, then `returns_desk_starter.py`
10m	Start the lesson project: 10,000 possible months, Steps 1 and 2

MENTAL MODEL TO SAY

Start with the homework guess: "If you picked 30 days at random from Boise's revenue and added them up, then did it again with a different 30, how different would the two totals be?" Write four or five guesses on the board, unedited. You will come back to them at the project start.

Say: "Imagine an enormous book filled with digits that pass every test of randomness. You open it at some page and read off numbers as you need them. A computer's random numbers work like that book: a fixed recipe that produces a sequence which looks random. The seed is the page you open at. Open at the same page tomorrow and you read the same numbers in the same order. That is not a weakness. It is what lets someone else rerun your analysis and get your result."

Four uses, one line each, so the room knows why a data course spends a lesson on invented numbers: **sampling** (pick 200 orders to audit, with no bias), **shuffling** (what does "no real difference" look like?), **simulation** (act a question out thousands of times and count), **test data**. Board, for later: **model → repeat → count**, and under it **more simulations: more precise. A better model: more true.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_4`. Run the steps in order, each cell once: because the generator moves forward with every draw, the numbers printed below are what you get from a fresh `default_rng(42)` drawn from in exactly this sequence. They match the lesson page for Step 1 and the dice in Step 2, and then differ, because the lesson draws more in between; say so if a learner compares. If you rerun a cell by accident, your numbers will change. Use it: that is Step 4's point.

1. **A generator, and numbers everyone can reproduce (2m).** Say: "Create a generator once, with a seed, and ask it for numbers. If you typed what I typed, your screen shows exactly these digits." Walk the room and check: it is a small moment of magic worth the thirty seconds.

```
import numpy as np

rng = np.random.default_rng(42)

print(rng.random())        # one float, from 0 up to (not including) 1
print(rng.random(5))       # an array of five
```

```
0.7739560485559633
[0.43887844 0.85859792 0.69736803 0.09417735 0.97562235]
```

Say: "The seed is the page you open the book of random digits at. Same page, same numbers."

2. **Whole numbers and whole tables (2m).** Ask: "A six-sided die: what goes in the brackets?" Take answers before typing.

```
print(rng.integers(1, 7, size=10))            # ten dice rolls: 1 to 6
print(rng.integers(0, 100, size=(2, 4)))      # size can be a shape
```

```
[5 5 5 5 4 1 6 3 4 3]
[[18 92 78 64]
 [40 82 54 44]]
```

Say: "The upper end is left out, like `range` and slicing. `integers(1, 6)` is a die with five faces, and nothing warns you." `size` can be a shape.

3. **Same seed, same sequence (2m).** Say: "Three generators. Two share a seed." Ask for the prediction.

```
a = np.random.default_rng(2024)
b = np.random.default_rng(2024)
c = np.random.default_rng(2025)

print(a.integers(0, 100, size=5))
print(b.integers(0, 100, size=5))
print(c.integers(0, 100, size=5))
```

```
[24 67  9 21 31]
[24 67  9 21 31]
[44 99 99 38 95]
```

The seed's value does not matter. What matters is that it is written in the code.

4 **The bookmark moves (2m).** Ask: "If I draw from `a` again, do I get the same five?"

```
print(a.integers(0, 100, size=5))     # the next five: different

a = np.random.default_rng(2024)       # back to the first page
print(a.integers(0, 100, size=5))     # the first five again
```

```
[30 90 79 91 99]
[24 67  9 21 31]
```

Say: "A generator moves forward with every draw, like a bookmark. This is why rerunning a notebook cell gives you new numbers. Nothing is broken. Reproducible means: same code, run from the top, same answer."

5 **Draw from your own values (2m).** Say: "Names on slips in a hat. With replacement you put the slip back. Without, it stays out."

```
store_names = np.array(["Asheville", "Bend", "Boise", "Flagstaff"])

print(rng.choice(store_names, size=6))                    # with replacement:
print(rng.choice(store_names, size=3, replace=False))     # without: three dif
```

```
['Bend' 'Asheville' 'Asheville' 'Boise' 'Flagstaff' 'Asheville']
['Flagstaff' 'Bend' 'Boise']
```

Ask: "Five prize winners: with or without?" (Without.) "A thousand simulated visitors, each choosing a store?" (With: the draws are independent events.)

6 **Unequal chances, and sampling variation (2m).** Say: "Boise gets about a third of the visitors, so a simulated visitor should be more likely to be in Boise. `p` sets the probabilities."

```
visits = rng.choice(store_names, size=10_000, p=[0.19, 0.24, 0.34, 0.23])
names, counts = np.unique(visits, return_counts=True)
print(names)
print(counts)
```

```
['Asheville' 'Bend' 'Boise' 'Flagstaff']
[1935 2409 3401 2255]
```

Say: "Close to 1,900, 2,400, 3,400 and 2,300, and not exactly. That gap between what the probabilities say and what one sample shows is sampling variation. Reasoning about it is most of statistics."

7 **Two ways to shuffle (2m).** Ask: "What does `rng.shuffle(deck)` return?" Most will say "the shuffled deck".

```
deck = np.arange(10)

mixed = rng.permutation(deck)    # returns a shuffled COPY
print(mixed)
print(deck)                      # the original is untouched

print(rng.shuffle(deck))         # shuffles IN PLACE and returns None
print(deck)
```

```
[4 7 9 2 6 0 1 5 8 3]
[0 1 2 3 4 5 6 7 8 9]
None
[3 4 0 7 6 5 9 1 2 8]
```

Say: "`permutation` returns a shuffled copy. `shuffle` changes your array and returns `None`. Write `deck = rng.shuffle(deck)` and your deck is gone."

8 **Sample rows, keep the columns matched (3m).** Say: "With parallel arrays, the thing to sample is the row position. Draw positions once, use them on every column."

```
def load(col, dtype=float):
    """Read one column of daily_sales.csv into an array."""
    return np.genfromtxt("data/daily_sales.csv", delimiter=",",
                         skip_header=1, usecols=col, dtype=dtype)

dates = load(0, dtype="datetime64[D]")
stores = load(1, dtype=str)
visitors = load(2)
revenue = load(5)

picks = rng.choice(revenue.size, size=4, replace=False)      # four different
print(picks)
print(dates[picks])
print(stores[picks])
print(revenue[picks])
```

```
[2556 2179  902 2107]
['2025-10-04' '2025-07-02' '2024-08-14' '2025-06-14']
['Bend' 'Asheville' 'Asheville' 'Asheville']
[11372.49  6611.42  4739.33  8154.94]
```

Ask: "Why not `rng.choice(revenue, ...)` and `rng.choice(stores, ...)` separately?" (Two separate draws: the revenue would no longer belong to the store next to it.)

9 **Shaped randomness: a binomial (2m).** Say: "Of 485 visitors, each buying with probability 0.236, how many buy? That is a binomial: successes out of a number of tries. Normal is for measurements around a typical value, Poisson for counts of events at some rate. Module 4 studies all three."

```
buyers = rng.binomial(n=485, p=0.236, size=100_000)     # purchases on a 485-v
print(buyers[:10])
print(buyers.mean().round(2), 485 * 0.236)
```

```
[130 118 106 117 110 111 103 112 119 113]
114.46 114.46
```

The average of 100,000 draws sits on 485 × 0.236. A distribution is a recipe for which values are likely.

10 **Simulation: model, repeat, count (2m).** Say: "Boise has 130 gift bags for Saturday's buyers. What is the chance they run out? We already modeled one day and repeated it 100,000 times. Now count: a mask and its mean, exactly as in Lesson 1.2."

```
ran_out = buyers > 130
print(ran_out.sum(), "of", buyers.size, "simulated days")
print(f"estimated chance of running out: {ran_out.mean():.1%}")
```

```
4430 of 100000 simulated days
estimated chance of running out: 4.4%
```

About 1 day in 23. Write **model, repeat, count** on the board.

11 **How many simulations are enough? (1m).** Say: "An estimate from a simulation is itself a little random."

```
for n_days in [100, 10_000, 1_000_000]:
    sim = rng.binomial(n=485, p=0.236, size=n_days)
    print(f"{n_days:>9,} days: {(sim > 130).mean():.4f}")
```

```
      100 days: 0.0400
   10,000 days: 0.0421
1,000,000 days: 0.0445
```

With 100 days the estimate can only move in steps of 0.01. More simulations buy more decimal places, slowly.

12 **The part no amount of simulation fixes (3m).** Say: "That estimate is precise. Is it right? The model said every day has exactly 485 visitors. Real days run from 177 to 1,164. We have the real counts, so use them."

```
boise_visitors = visitors[stores == "Boise"].astype(int)
print(boise_visitors.min(), boise_visitors.max(), boise_visitors.mean().round(

sim_visitors = rng.choice(boise_visitors, size=100_000)      # a random real d
sim_buyers = rng.binomial(n=sim_visitors, p=0.236)           # buyers among th
print(f"estimated chance of running out: {(sim_buyers > 130).mean():.1%}")
```

```
177 1164 485.2
estimated chance of running out: 27.8%
```

Let it land: same question, same 100,000 simulations, and the answer went from about 4% to about 28%. Say: "More simulations make an estimate more precise. Only a better model makes it more true." (And the plan was for a Saturday, the busiest day: Exercise 3 picks that up.)

13 **A mistake on purpose: the generator in the loop (1m).** Say: "Three simulated days."

```
for day in range(3):
    loop_rng = np.random.default_rng(42)          # same seed, every time arou
    print(loop_rng.integers(100, 200, size=4))
```

```
[108 177 165 143]
[108 177 165 143]
[108 177 165 143]
```

Say: "Each pass opened the book at page 42 again. In a real simulation this turns 10,000 runs into one run repeated 10,000 times. Create the generator once, above the loop."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **"My numbers changed and I did not change the code."** Symptom: output that no longer matches a neighbor's, the solution, or the learner's own earlier run. Cause: a cell that draws was run twice, or cells were run out of order, so the bookmark is somewhere else. Coach: "When did the cell that creates `rng` last run? Run from there." Reproducible means "same code, run from the top, same answer". This is the Session 1 notebook lesson again with a new consequence.
- **A seeded generator created inside a loop or a function that is called repeatedly.** Symptom: every simulated day, week or trial is identical; a suspiciously tidy result. No error. Coach: "Print the first two simulated weeks. Should they be the same?" Then: "Where is the generator created? How many times does that line run?" The rule for the board: create the generator once, near the top, and pass it in.
- **Confusing precise with right.** Symptom: a learner answers "how could we improve the estimate?" with "more simulations" when the weak point is an assumption. Coach: "What did we assume about one day? Is it true of the real days? What data do we have that could replace the assumption?" Lab B is built to give them a model that passes its check and one that fails, so they see both.

GUIDED LAB

Lab A (12m, everyone): `labs/module-03/l1_4-random-generation/mystery_shopper_starter.py`; reference `labs/module-03/l1_4-random-generation/mystery_shopper_solution.py`. A schedule nobody can call biased: 2 pilot stores, 12 different Flagstaff days in 2025 (drawn as row positions), proof that the seed reproduces it, and a random 70/30 split. With seed 7, and the draws made in the order the starter gives, every learner's output matches the solution.

Expected, Lab A: 361 Flagstaff rows in 2025; pilot stores Boise and Flagstaff; twelve dates from 2025-01-02 to 2025-11-26; 2 of the 12 on a weekend; same schedule from the same seed: True; group sizes 2,032 and 871; mean visitors 356.8 and 353.8 (all rows 355.9).

Circulate in Lab A for: output that does not match a neighbor's (a draw was repeated or done in a different order: perfect, ask why before fixing); `replace=False` forgotten (it may still print twelve dates, so ask "could a date repeat?"); `rng.choice(dates, ...)` instead of positions (it works for the dates but then

`weekday[picks]` fails: the reason to sample positions); in Step 4, a second generator made but only one of the two draws repeated, so the comparison prints False.

Lab B (13m, pairs): `labs/module-03/l1_4-random-generation/returns_desk_starter.py`; reference `labs/module-03/l1_4-random-generation/returns_desk_solution.py`. Model, repeat, count on a new question (how often will 4 or more of a day's orders be returned?), then a check against the 731 real days, then the same model on a question where it fails.

Expected, Lab B: 731 days, 1,143 returned orders; average 1.564 returns per day; 4 or more returns: simulated 7.4%, real 7.3%; no returns: simulated 21.1%, real 20.7%; real mean 1.56 and variance 1.52. Then orders per day: 40 or more: simulated 1.7%, real 7.3%; real mean 27.77 and variance 64.35.

Debrief Lab B (3m), the point of the session's first half: "The Poisson model passed its check for returns and failed it for orders. Why?" (Returns trickle in at a steady rate; orders have weekends, seasons and sale days, so real days vary far more than one average rate allows. For a Poisson the variance about equals the mean; for orders it is more than twice the mean.) "Would ten million simulations have fixed the orders estimate?" (No.)

Fast finishers: `labs/module-03/l1_4-random-generation/debug_random.py`: a five-faced die, a deck replaced by `None`, three identical "random" weeks, and probabilities that do not add up to 1.

START THE LESSON PROJECT

The lesson project asks how much a month of Boise revenue could vary: 10,000 possible months built from real Boise days, then an honest check against the 24 real months. In the room, do **Steps 1 and 2** (10 minutes).

Step 1 together: load, keep Boise's days with revenue present (one combined mask), and create the generator **once**, with seed 42. Done = `726 days` and a mean of $10,144.21.

Step 2 on their own: two simulated months of 30 days with `rng.choice(boise, size=30)`. With seed 42 and no other draws in between, everyone gets $324,963.33 and $303,054.24. Now point at the guesses on the board: the two totals differ by about $22,000. Ask who was closest.

Goes home: Step 3 (10,000 months in one call with `size=(10_000, 30)`; middle 90% from $271,698 to $339,193), Step 4 (chances as the mean of a mask), Step 5 (the histogram), Step 6 (a function that takes a seed) and Step 7, which matters most: only 10 of the 24 real months fall inside the simulated 90% range, because real days come in seasons. Tell them to expect that, and that catching it is the point.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

1. What does a seed do? **Answer:** It fixes where the generator's sequence starts, so the same code run from the top gives the same numbers.
2. I ran my simulation cell twice and got two different answers. Is the code broken? **Answer:** No. The generator moved forward. Rerun from the cell that creates it.

3. Five prize winners from 4,850 customers: with or without replacement? **Answer:** Without (`replace=False`): nobody should win twice.
4. Returns arriving per day, purchases out of 485 visitors, a purchase amount around $88: which distribution for each? **Answer:** Poisson (counts of events at a rate), binomial (successes out of n tries), normal (a measurement around a typical value).
5. My estimate is stable to three decimals over a million runs. What have I learned about whether it is right? **Answer:** Only that it is precise. Whether it is right depends on the model's assumptions.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) `False True`; **Q2** D) `rng.integers(1, 7, size=10)`; **Q3** `None [0 1 2 3 4]`; **Q4** A) `rng.choice(n, size=50, replace=False)`; **Q5** C) The estimate is precise, but it is only as true as the model; a better model of visitors could change it a lot; **Q6** `(1000, 2) 1 3`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: For learners unsettled by numbers that will not sit still: have them put the generator line and one draw in the SAME cell and run it five times (identical every time), then split them into two cells and run only the second (different every time). That one experiment explains most of the confusion. In Lab B, give the three steps as sentence starters on paper: "One day is modeled as ___. I repeat it ___ times. I count how often ___."

Stretch: `debug_random.py`; then the lesson's Exercise 4 (a permutation test: could shuffling explain the gap between Bend and Flagstaff?) or the Bonus Challenge (simulate next December from real December days only, and say what that model still ignores).

1.5 Linear Algebra Essentials

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Compute a dot product and explain it as a weighted sum
- Multiply matrices and vectors with `@`, predict the shape of the result, and tell `@` apart from `*`
- Solve a system of linear equations with `np.linalg.solve` and check the answer
- Fit a line or a multi-feature linear model with `np.linalg.lstsq`, and interpret its coefficients and its error
- Measure length and distance with `np.linalg.norm`, and standardize features before comparing them

TIMING IN THE ROOM

10m	Frame: the receipt, seen from four sides; vector and matrix are new names for arrays you already use
30m	Live demo: dot product, matrix @ vector, `@` is not `*`, the shape rule, transpose, `solve`, `lstsq` with RMSE and R squared, a prediction, distance
25m	Guided lab: `payroll_receipt_starter.py`, then `basket_model_starter.py`
15m	Start the lesson project: a first revenue model for Asheville, Steps 1 and 2

MENTAL MODEL TO SAY

Say: "A receipt has a column of quantities and a column of prices. Multiply across each line, add down the page, and you have the total. Nearly everything in this lesson is that receipt, seen from a different side."

Write the four sides on the board as you say them, and tick each one off during the demo: **one receipt: a dot product · a stack of receipts with one price list: a matrix times a vector · quantities and totals known, prices unknown: solving a system · the same with noisy totals, so nothing fits perfectly: least squares**.

Take the fear out of the title: "A vector is a 1-D array. A matrix is a 2-D array. You have used both since Session 1. Only the names are new, and there are no proofs today." Then the one rule to add to the board before the first matrix product: **(n, k) @ (k, m) gives (n, m). The inner numbers must match.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_5`. Nothing here is random, so every learner's output should match this page exactly. Keep last session's habit: before each `@`, write the two shapes on the board and take the room's call on the result. Steps 6 and 15 are mistakes on purpose. The demo covers a great deal; if you fall behind, Steps 2, 7 and 14 are the ones to shorten (see If You Run Long).

1. **One receipt: the dot product (3m).** Say: "2 tents, 1 stove, 3 mugs. Lesson 1.2 already gives the total: multiply position by position, then sum. That pattern is so common it has a name, the dot product, and an operator."

```
import numpy as np

quantities = np.array([2, 1, 3])
prices = np.array([299.95, 82.95, 24.50])

print(quantities * prices)              # the three line totals
print((quantities * prices).sum())      # the order total
print(quantities @ prices)              # the same thing: a dot product
```

```
[599.9   82.95  73.5 ]
756.35
756.35
```

Two vectors in, one number out. Ask: "What must be true of the two lengths?" (Equal: every quantity needs its price.)

2. **Any weighted sum is a dot product (2m).** Say: "Three products' ratings, weighted by how many reviews each one has."

```
ratings = np.array([4.0, 3.7, 4.7])
reviews = np.array([8, 3, 41])

print(ratings.mean().round(3))                          # every product counts
print((ratings @ reviews / reviews.sum()).round(3))     # every review counts e
```

```
4.133
4.535
```

The weighted average is higher because the best-rated product also has by far the most reviews.

3. **A stack of receipts: matrix @ vector (3m).** Say: "Three orders, one price list. Stack the orders as rows." Board: `(3, 3) @ (3,)`. Ask for the shape of the result.

```
orders = np.array([[2, 1, 3],      # order A: 2 tents, 1 stove, 3 mugs
                   [1, 0, 2],      # order B
                   [0, 4, 1]])     # order C

totals = orders @ prices
print(totals)
print(orders.shape, prices.shape, totals.shape)
```

```
[756.35 348.95 356.3 ]
(3, 3) (3,) (3,)
```

Say: "The dot product of each row with the price vector: three receipts in one step. Hold on to this picture, because it is also what a prediction looks like: a row of measurements per day, times how much each measurement is worth."

4 **@ is not * (2m).** Ask: "`orders * prices` also runs. What does it give?"

```
print((orders * prices).shape)     # a table of line totals, no error
print((orders @ prices).shape)     # the three order totals
```

```
(3, 3)
(3,)
```

Say: "`*` is position by position and keeps the shape: a table of line totals. `@` multiplies and adds, collapsing an axis. When shapes happen to broadcast, `*` gives you the wrong thing with no complaint. Check the shape."

5 **Matrix @ matrix, and the shape rule (3m).** Say: "Two price lists side by side: regular and sale." Write the rule on the board before running: **(n, k) @ (k, m) gives (n, m). The inner numbers must match.**

```
price_lists = np.array([[299.95, 254.96],     # tent:  regular, sale
                        [ 82.95,  70.51],     # stove
                        [ 24.50,  20.83]])    # mug

both = orders @ price_lists
print(both)
print(orders.shape, "@", price_lists.shape, "gives", both.shape)
```

```
[[756.35 642.92]
 [348.95 296.62]
 [356.3  302.87]]
(3, 3) @ (3, 2) gives (3, 2)
```

First column: the order totals at regular prices, the same numbers as before. Second: at sale prices.

6 **Order matters: read the end of the message (2m).** Say: "Unlike ordinary multiplication, you cannot swap the two sides." Board: `(3, 2) @ (3, 3)`: inner numbers 2 and 3.

```
wrong_order = price_lists @ orders
```

```
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, w
```

Say: "Skip to the end of the message: 'size 3 is different from 2'. The inner numbers do not match. Usually the operands are in the wrong order, or one needs flipping on its side."

7 **Transpose (2m).** Say: "`.T` swaps rows and columns."

```
units_sold = orders.sum(axis=0)            # total units of each product, sha
print(price_lists.T.shape)
print(price_lists.T @ units_sold)          # revenue under each price list
```

```
(2, 3)
[1461.6  1242.41]
```

`(2, 3) @ (3,)`: revenue under each price list, two dot products in one step.

8 **Run the receipt backward: solve (3m).** Say: "Three old order slips: quantities and totals, but the prices have faded. Three equations, three unknowns."

```
slip_totals = np.array([756.35, 348.95, 356.30])

recovered = np.linalg.solve(orders, slip_totals)
print(recovered)
print(np.allclose(orders @ recovered, slip_totals))
```

```
[299.95  82.95  24.5 ]
True
```

Say: "Always put the answer back into the original equation." Mention the inverse only if asked: `solve` is faster and loses less accuracy.

9 **Hundreds of noisy equations: build the matrix (3m).** Say: "Real data: 726 Boise days, two unknowns, and noise. No pair of numbers fits every day. Which pair comes closest? The model is a line, revenue = intercept + slope × visitors. One row per day, one column per unknown."

```
def load(col, dtype=float):
    """Read one column of daily_sales.csv into an array."""
    return np.genfromtxt("data/daily_sales.csv", delimiter=",",
                         skip_header=1, usecols=col, dtype=dtype)

stores = load(1, dtype=str)
visitors, revenue, promo = load(2), load(5), load(7)

keep = (stores == "Boise") & ~np.isnan(revenue)
x = visitors[keep]
y = revenue[keep]

A = np.column_stack([np.ones(x.size), x])
print(A.shape)
print(A[:3])
```

```
(726, 2)
[[  1. 333.]
 [  1. 308.]
 [  1. 290.]]
```

Ask: "What is the column of ones for?" (It multiplies the intercept, so the line is not forced through zero.)

10 **Least squares (3m).** Say: "`lstsq` returns four things; the first is the solution." Read the result as a sentence.

```
coef, *_ = np.linalg.lstsq(A, y)        # keep the first result, discard the o
intercept, slope = coef
print(f"revenue = {intercept:.2f} + {slope:.2f} * visitors")
```

```
revenue = -238.85 + 21.39 * visitors
```

Say: "At Boise, each additional visitor goes with about $21 more revenue. You know another route to that number: 23.6% of visitors buy and an average purchase is $88.66, and 0.236 × 88.66 is $20.92. Two routes, nearly the same number." The intercept is not meaningful: no day has zero visitors.

11 **How good is the line? (2m).** Say: "The gap between an actual value and the prediction is a residual. `A @ coef` is the prediction for every day: a matrix times a vector."

```
residuals = y - A @ coef

print(f"typical miss (RMSE): ${np.sqrt((residuals ** 2).mean()):,.2f}")
r_squared = 1 - (residuals ** 2).sum() / ((y - y.mean()) ** 2).sum()
print(f"R squared: {r_squared:.3f}")
```

```
typical miss (RMSE): $1,344.33
R squared: 0.869
```

RMSE is the typical miss, in dollars. R squared is the share of the day-to-day variation the line accounts for.

12 **More features are more columns (2m).** Say: "To give the model more information, add a column. Nothing else changes."

```
A2 = np.column_stack([np.ones(x.size), x, promo[keep]])
coef2, *_ = np.linalg.lstsq(A2, y)
print(coef2.round(2))

residuals2 = y - A2 @ coef2
print(f"RMSE with visitors and promo: ${np.sqrt((residuals2 ** 2).mean()):,.2f
```

```
[ 189.11   20.34 2502.17]
RMSE with visitors and promo: $1,279.09
```

Intercept, dollars per visitor, and the extra revenue on a sale day beyond what its visitor count predicts. The typical miss shrank from $1,344 to $1,279: the column earned its place.

13 **A prediction is one more @ (1m).** Say: "Three scenarios, columns in the same order as the matrix."

```
scenarios = np.array([[1, 400, 0],      # an ordinary day, 400 visitors
                      [1, 700, 0],      # a busy Saturday
                      [1, 700, 1]])     # the same Saturday in a sale
print((scenarios @ coef2).round(2))
```

```
[ 8325.58 14427.93 16930.1 ]
```

Say: "Quote a forecast with its typical miss: 'about $8,300, give or take $1,300'. And remember what the coefficient is: a description of a pattern across days, not a promise about what one more visitor will do."

14 **Distance, and why you standardize first (2m).** Say: "The norm of a difference is a distance: how alike are two rows? Six tents described by price, weight and how many they sleep. Which is most like the

first?"

```
tents = np.array([[299.95, 1.34, 1],
                  [324.95, 1.07, 1],
                  [217.95, 1.79, 2],
                  [434.95, 2.09, 2],
                  [294.95, 3.27, 4],
                  [511.95, 8.37, 6]])        # price, weight_kg, sleeps

raw = np.linalg.norm(tents - tents[0], axis=1)
print(raw.round(2))

z = (tents - tents.mean(axis=0)) / tents.std(axis=0)
print(np.linalg.norm(z - z[0], axis=1).round(2))
```

```
[  0.    25.    82.01 135.01   6.14 212.18]
[0.   0.28 1.02 1.52 1.84 4.51]
```

On raw numbers the nearest tent (6.14 away) is a four-person model at more than twice the weight: price is in the hundreds and drowned out everything else. After the standardizing step from Lesson 1.3, the nearest (0.28) is the other one-person tent.

15 **A mistake on purpose: one missing value (1m).** Say: "Boise has one day with no revenue recorded. What happens if I forget to filter it out?"

```
boise_all = stores == "Boise"                          # includes the day with no
A_nan = np.column_stack([np.ones(boise_all.sum()), visitors[boise_all]])
coef_nan, *_ = np.linalg.lstsq(A_nan, revenue[boise_all])
print(coef_nan)
```

```
[nan nan]
```

Say: "One `nan` among 727 values ruins every coefficient. There are no nan-aware versions of `solve` or `lstsq`. Build one mask and apply it to every array."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **`*` where `@` was meant.** Symptom: no error, and a table where a column of totals or predictions was expected (shape `(7, 4)` where `(7,)` was wanted). Coach: "Do you want position by position, or multiply and add? Print the shape of what came back." Because the shapes often broadcast, this mistake is silent: it is last session's lesson with a new operator.
- **Shapes that do not chain.** Symptom: `ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0 ... (size 3 is different from 2)`. Coach: "Read the end of the

message. Write the two shapes side by side: which numbers are next to each other in the middle?" Then: "Swap the operands, or does one need `.T`?" Do not let learners try combinations at random until something runs: a wrong product with the right shape is worse than an error.

- **Feeding `lstsq` unfiltered or incomplete data, and over-reading what comes out.** Symptoms: `[nan nan]` for every coefficient (one missing revenue in the data); a slope with no intercept (the column of ones forgotten: it still runs); or a learner announcing that "each staff member adds $1,154 of revenue". Coach the first two with the mask and the matrix: "Print `A[:3]`. What is each column for?" Coach the third with a question: "What else is different about days with more staff?" A fitted line describes a pattern. It does not prove a cause.

GUIDED LAB

Lab A (12m, everyone): `labs/module-03/l1_5-linear-algebra/payroll_receipt_starter.py`; reference `labs/module-03/l1_5-linear-algebra/payroll_receipt_solution.py`. The receipt with payroll on it: a weighted average rating over 234 products (a dot product), a week of staff counts times a daily cost per store (matrix @ vector), the same week under two pay scales (matrix @ matrix), payroll per store, and three hourly rates recovered with `np.linalg.solve`. Learners write the shapes in and the shape out in the starter's blanks before each `@`.

Expected, Lab A: plain average rating 4.138, weighted 4.146; payroll per day `[3172. 2816. 2820. 2996. 4036. 5276. 4396.]`, shape `(7,)`; under two pay scales shape `(7, 2)`, week totals 25,512 and 27,252; payroll per store `[5880. 5984. 8832. 4816.]`; hourly rates `[22. 19. 17.]`; check True.

Circulate in Lab A for: `week * day_cost` in Part 2 (a `(7, 4)` table, no error: "is that seven payroll figures?"); `day_cost @ week` (the matmul error: read the end); the weighted average left as `rating @ reviews` (the dot product without dividing by the total weight); in Part 5, `payroll @ hours` or `hours * rates` in the check.

Debrief Lab A (1m): the two averages are nearly equal (4.138 and 4.146). Ask what that says. (Heavily reviewed products are not rated much better or worse than the rest. In the lesson's three-product example the gap was large because one product had almost all the reviews.)

Lab B (12m, pairs): `labs/module-03/l1_5-linear-algebra/basket_model_starter.py`; reference `labs/module-03/l1_5-linear-algebra/basket_model_solution.py`. The lesson fits revenue against visitors at Boise and Asheville; here it is revenue against transactions at Flagstaff, so the slope is "dollars per purchase", which can be checked by a second route. Then a sale-day column that does **not** earn its place, and three forecasts with one `@`.

Expected, Lab B: 724 days; `revenue = 117.23 + 87.10 * transactions`; RMSE $631.35, R squared 0.945; average dollars per purchase by direct division 88.77; with a sale-day column: intercept 89.82, per purchase 87.52, sale day -172.09, RMSE $630.73 (R squared still 0.945); forecasts $5,341.19, $8,842.11 and $13,046.16, each give or take about $631.

Debrief Lab B (2m): "The sale-day column cut the typical miss by 62 cents. Did it earn its place?" (No.) "In the demo the same column was worth $2,500 at Boise. What is different?" (There the other column was visitors: sale-day visitors are more likely to buy, so a sale day brings more revenue than its visitor count predicts. Here the other column is purchases, and a purchase is worth about the same on a sale day as on any other day. Session 2's worked example found the same thing: sale days bring more buyers, not bigger baskets.) A coefficient always means "with the other columns held fixed".

Fast finishers: `labs/module-03/l1_5-linear-algebra/debug_linalg.py`: `*` for `@`, a product that needs `.T`, one `nan` that ruins a fit, and a missing column of ones.

START THE LESSON PROJECT

The lesson project is a first revenue model for the Asheville store: fitted by least squares, scored, used to predict three scenarios, and checked for its biggest misses. In the room, do **Steps 1 and 2** (15 minutes).

Step 1 together: one mask for "Asheville and revenue present", applied to every column, plus the weekday from date arithmetic. Done = `726 days`, mean visitors 275, mean revenue $5,703.83. Ask: "Why one mask for every array?" (So the arrays stay parallel.)

Step 2 on their own: the two-column matrix and the fit. Done in the room = `revenue = 4.73 + 20.73 * visitors`. Have three learners read the slope aloud as a sentence in dollars per visitor, and ask how it compares with Boise's $21.39.

Goes home: Step 3 (a `score` function: RMSE $902.58, R squared 0.809), Step 4 (four columns: sale day worth about $924, weekend about $130, RMSE $890.88), Step 5 (three scenarios with one `@`, each quoted with a "give or take") and Step 6 (the five biggest misses: all days that did far better than predicted, and the errors grow on busy days). Also Exercise 4, the staffing trap, which is the most important exercise of the module: do not let it be skipped.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

1. In one sentence, what is a dot product? **Answer:** Multiply matching positions and add everything up: a weighted sum.
2. `X` is `(726, 3)` and `coef` is `(3,)`. What is `X @ coef`, in shape and in meaning? **Answer:** Shape `(726,)`: one prediction per day.
3. `(4, 2) @ (3, 4)`: does it work? What about the other way round? **Answer:** No: the inner numbers are 2 and 3. `(3, 4) @ (4, 2)` works and gives `(3, 2)`.
4. When do you reach for `solve`, and when for `lstsq`? **Answer:** `solve` for as many independent equations as unknowns, exact. `lstsq` for more equations than unknowns and noise: the best fit.
5. The slope is 21.39 dollars per visitor. Does that mean bringing one more person in earns $21.39? **Answer:** No. It describes a pattern across days. It is not a promise about one more visitor, and it says little about days outside the range fitted.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** A) `23 [20 0 3]`; **Q8** B) `(100,)`; **Q9** `[3 7] [1 3]`; **Q10** C) `10827.28`; **Q11** `10.0 5.0`; **Q12** D) Standardize each column (subtract its mean, divide by its standard deviation) before measuring distance. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Keep the receipt on paper for anyone lost in shapes: three rows of quantities, one row of prices, and have them compute one order total by hand, then find it in the output of `orders @ prices`. For least squares, the picture matters more than the algebra: open the lesson's scatter plot with its fitted line and point at one dot, its vertical distance to the line (a residual), and the slope. Pair learners who missed last session's shape questions with a partner for Lab A.

Stretch: `debug_linalg.py`; then the lesson's Exercise 4 (the staffing trap: a coefficient of $1,154 per staff member that falls to $78 once visitors are in the model) and the Bonus Challenge (add a visitors-squared column, then compare the two models fairly on a random 80/20 split from Lesson 1.4).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Three questions on the slide, answered on a sticky note or in the workbook's Exit Ticket box. Read question 3 closely afterward: a learner who writes "goes with" and mentions the typical miss has taken the most important idea of the module's last lesson.

1. You run a cell that draws from `rng` twice and get different numbers. Is the code broken? What makes a random result reproducible? **Answer:** No. The generator moves forward with every draw. Reproducible means the same code, run from the top (from where the seeded generator is created), gives the same result.

2. `orders` has shape `(3, 3)` (orders by products) and `prices` has shape `(3,)`. What is the difference between `orders * prices` and `orders @ prices`? **Answer:** `*` multiplies position by position and keeps the shape: a `(3, 3)` table of line totals. `@` multiplies and adds: three order totals, shape `(3,)`.

3. A fit of revenue on visitors gives a slope of 21.39. Write the sentence a manager should hear, and one caution. **Answer:** Each additional visitor goes with about $21 more revenue. It describes a pattern across days: it is not a promise about one more visitor, and a forecast should be quoted with its typical miss (the RMSE).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The month simulation (Steps 3 to 7: middle 90% about $271,698 to $339,193; the chance of a month below $280,000 about 11 or 12%; only 10 of 24 real months inside the simulated range, and a sentence on why) and the Asheville revenue model (Steps 3 to 6: RMSE $902.58 then $890.88; three scenarios with a "give or take"; the five biggest misses).

- **Read in full** Lesson 1.4's section "Distributions: Shaped Randomness" (the room saw only the binomial).
- **Lesson exercises.** 1.4: Exercises 1 to 3 (two dice, an audit sample, gift bags on a Saturday); Exercise 4 (a permutation test) is a stretch. 1.5: Exercises 1 to 3 (a packing list, bundles, ticket prices), and **Exercise 4, the staffing trap**: a stretch on the page, but do it.
- **Both Quick Quizzes** and **both journal entries**. The 1.5 prompt asks for one sentence per Module 1 lesson: bring it, Session 4 opens with it.
- **Before the Next Lesson (from 1.5):** keep the Module 1 notebooks; open `data/daily_sales.csv` in a text editor or spreadsheet and read the header row (in Lesson 2.1 those names become the way you refer to columns); skim `first_look.py` from Lesson 0.1 and see how much of the pandas code you can now guess the meaning of.
- **If a Lab B was not finished in class,** finish it: each starter says what to expect.

BRING LIST FOR SESSION 4

- **The working environment** from Session 1: `check_environment.py` still ends with READY. pandas, pyarrow and openpyxl are already installed by `requirements.txt` and are first used in earnest next session; there is nothing new to install.
- **The unzipped course data:** the whole `data/` folder inside `pinecrest-analysis`, not only `daily_sales.csv`. Session 4 opens `orders.csv`, `customers.csv`, `products.csv`, `sales_2025.xlsx`, `web_sessions.json` and `pinecrest.db`. Anyone who deleted files should unzip `pinecrest-data.zip` again.
- **The finished lesson projects:** the Module 1 notebooks (benchmark report, store scorecard, weekday index, month simulation, revenue model) and `first_look.py` from Lesson 0.1. Lesson 2.1 revisits that script, and redoing a NumPy project in pandas is how the module pays off.
- The `labs/` folder (Session 4's starters are in `labs/module-04/`), the workbook and the journal with its Module 1 review. No servers, no accounts, no internet needed.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 1.4 demo Step 12 and Lab B of 1.4; 1.5 demo Steps 9 to 13; Lab A of 1.5; the exit ticket.
- **Cut first (saves 2m):** 1.5 demo Step 14 (distance). Point at the workbook's worked example and the lesson's tent example.
- **Cut second (saves 4m):** 1.5 demo Steps 2 and 7 (weighted average; transpose). Lab A covers both.
- **Cut third (saves 12m):** Lab A of 1.4 (`mystery_shopper`): assign it as homework. Keep Lab B, which carries the reality check.

- **Cut fourth (saves 10m):** the 1.4 project start. Give the two check totals ($324,963.33 and $303,054.24), compare them with the guesses on the board, and send all seven steps home.
- **Do not cut** Step 2 of the 1.5 project: reading a slope aloud as a sentence, in the room, is worth more than any other single minute of the second half.

RUNNING SHORT: WHAT TO ADD

- Run the lesson's Exercise 1 of 1.4 (two dice, 100,000 rolls) live and ask for the most common total before the tally prints.
- Do the first half of 1.5's Exercise 4 (the staffing trap) together: fit revenue on staff alone and ask the room for the business recommendation. Then add visitors and watch $1,154 become $78. It previews Lesson 4.4.
- A Module 1 round: each learner names the one idea from the five NumPy lessons they most want to keep. Write them on the board; it is the journal prompt, started together.
- Run `debug_random.py` or `debug_linalg.py` as a pairs race, with the first pair explaining each bug at the projector.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 4: DataFrames and Loading Data

How to teach the session where the course moves from NumPy arrays to pandas tables. First learners meet the labeled column and the labeled table, learn to inspect, select, filter, change, sort and count, and see for themselves why chained assignment never works under copy-on-write. Then they learn to get data in properly from CSV, Excel and nested JSON with the right types, and to save results in a format that remembers them.

2 lessons | 3 hrs session | Lessons 2.1–2.2 | Session 4 of 21

HOW TO USE THIS GUIDE

- Each lesson block has the same four beats: **frame** (the mental model, said out loud), **live demo** (you type, they follow), **guided lab** (new practice from `labs/module-04/`) and **start the lesson project** (the first two steps, together). The rest of each project, the lessons' exercises, quizzes and journals are homework.
- The lessons are timed at 90 and 85 minutes for a solo learner; the room has 75 for each. The demo scripts below are therefore compressed paths through the lesson pages, with every output printed from a real run on the course data (pandas 3.0.6).
- **Type, do not paste.** Every code block is what to type. Two steps in 2.1 and one in 2.2 are deliberate mistakes; make them for real and read the last line of the error aloud.
- Keep `pandas_dataframes_series.html` and `pandas_data_loading.html` open in browser tabs for the reference tables (the dtype table, the three selection tools, the `read_csv` options, the readers and writers).
- Starters go to learners; every `*_solution.py` stays with you until after the lab. `debug_filters.py` is a handout that is broken on purpose.

> **THE #1 RISK OF SESSION 4: MISTAKES THAT RAISE NO ERROR**
>
> Most of today's damage is silent. Chained assignment prints a warning and changes nothing. `== np.nan` counts zero missing values. A spreadsheet loaded without `skiprows` gives 15 rows of nonsense and no complaint. A date left as text sorts and filters wrongly weeks later. Train one habit all session and say it every time: **"Run it, then look: `shape`, `dtypes`, `head()`."** The second risk is pace: 2.1 has more new vocabulary than any lesson so far. Protect `.loc` versus `.iloc` and the copy-on-write demonstration; everything else can be looked up.

BEFORE THE SESSION

- ☐ Copy the **starter** files (`*_starter.py`), `debug_filters.py` and `till_export_week08.txt` from `labs/module-04/` to the shared folder or each machine, keeping the two lesson folders and `labs_paths.py` one level up as they are. Keep every `*_solution.py` on your machine.
- ☐ From the kit folder run `python3 labs/check_labs.py module-04`. All 10 lines should say PASS.
- ☐ No servers to start and no internet needed. Files used today: `products.csv`, `orders.csv`, `customers.csv`, `order_items.csv`, `daily_sales.csv`, `survey.csv`, `sales_2025.xlsx`, `web_sessions.json`.
- ☐ Environment traps: `read_excel` needs `openpyxl` and `to_parquet` needs `pyarrow` (both installed in Lesson 0.1; check with `python -c "import openpyxl, pyarrow"`). A learner whose pandas prints `object` for text columns is on an old version: have them activate the course virtual environment.
- ☐ Projector: a notebook called `session04` opened **in the project folder** (the one that contains `data/`), font at least 18 pt, `pd.set_option("display.width", 150)` in the first cell.
- ☐ Whiteboard, before learners arrive: the three-shelf drawing, and the two sentences **"Change a table: `df.loc[rows, \"col\"] = value`." and "Run it, then look: shape, dtypes, head."**
- ☐ Skim learners' Session 3 homework: the 1.4 and 1.5 lesson projects, and Lesson 1.5's 'Before the Next Lesson' (open `daily_sales.csv` and read its header row). That header is your bridge into today.
- ☐ Sticky notes or slips for the exit ticket. Print the workbook for Session 4.

SESSION 4 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 3 (random numbers, linear algebra), homework check, bridge from arrays to tables.
0:10	10	**2.1** Frame and mental model	Two limits of arrays; the labeled shelf; why every later lesson starts with a DataFrame.
0:20	30	**2.1** Live demo	Series and alignment, the four-line greeting, `.loc` / `.iloc`, masks, new columns, chained assignment (two deliberate mistakes), sort and count.
0:50	25	**2.1** Guided lab	`labs/module-04/l2_1-dataframes-series/busiest_days_starter.py`, then `recommend_groups_starter.py`.
1:15	10	**2.1** Start the lesson project	Catalog Health Check, Steps 1 and 2 together; the rest goes home.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:25	10	Break	Leave the `price_band` counts on the projector.
1:35	10	**2.2** Frame and mental model	Loading is translation; the interpreter at the border; readers and writers.
1:45	30	**2.2** Live demo	`read_csv` options, an awkward export, Excel with the header on row 4, nested JSON, memory and chunks, CSV versus Parquet.
2:15	25	**2.2** Guided lab	`labs/module-04/l2_2-loading-data/till_export_starter.py`, then `product_pages_starter.py`.
2:40	10	**2.2** Start the lesson project	The Pinecrest Snapshot, Steps 1 and 2 together; the rest goes home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 5.
Total	180		Ends at 3:00.

In the room: both mental models, both live demos, the guided labs `busiest_days` and `till_export` for everyone (`recommend_groups` and `product_pages` for pairs that finish), and the first two steps of each lesson project. **Goes home:** Steps 3 to 6 of both projects, the lessons' exercises (five each), both Quick Quizzes, both journal entries, and any lab not finished: about 2 to 3 hours.

Protect: the `.loc` / `.iloc` experiment and the chained-assignment demonstration in 2.1; 'load, then look' and the Excel header trap in 2.2; both guided labs. **Cut first if running long:** the Series alignment step (say it in one sentence and point to the lesson), then memory and chunks in 2.2 (homework reading), then the second lab of each block.

WARM-UP

0:00 Warm-up: Retrieval from Session 3 — 10 min

FIVE RETRIEVAL QUESTIONS (NO NOTES, 5 MINUTES)

Slide 3. Learners write answers on paper with notebooks closed, then compare with a neighbor for one minute before you take answers. Wrong answers are welcome: the point is the effort of recalling.

1. You write `rng = np.random.default_rng(42)`. What is `rng`, and what does the 42 do? **Answer:** A random number generator. The seed fixes where its sequence starts, so the same code run from the top gives the same numbers.
2. Can `rng.integers(1, 7, size=5)` ever return a 7? **Answer:** No. The upper end is left out, so the values are 1 to 6 (a die).
3. What does `replace=False` guarantee in `rng.choice(skus, size=10, replace=False)`? **Answer:** Ten distinct items: nothing is picked twice.
4. For two 1-D arrays of length 3, what is the difference between `a * b` and `a @ b`? **Answer:** `a * b` is position by position and gives three numbers; `a @ b` is the dot product, one number: a weighted sum.
5. A matrix of shape `(726, 3)` is multiplied with `@` by a vector of shape `(3,)`. What shape is the result, and what must match? **Answer:** `(726,)`. The inner numbers must match (3 and 3): one dot product per row.

HOMEWORK CHECK (3 MINUTES)

- Ask for a show of hands: who finished the 1.4 project (the simulation) and the 1.5 project (the revenue model)? Note who did not, for a quiet word at the break.
- One volunteer answers Lesson 1.4's rerun question: with a different seed, which printed numbers changed and which conclusions changed? (Numbers move a little; conclusions should not.)
- Ask who opened `daily_sales.csv` and read its header row (Lesson 1.5, 'Before the Next Lesson'). Have someone read the column names aloud.

BRIDGE TO TODAY (2 MINUTES)

Say: "In Module 1 those eight names were in your head, and the numbers were in arrays you had to keep in the same order by hand. Today the names move into the data. That is the whole idea of pandas."

LESSON PLANS

2.1 DataFrames and Series

75 min

OBJECTIVES (THE LESSON'S OWN WORDS)

- Describe a Series as a labeled array and a DataFrame as a set of Series that share one index
- Inspect any table with `shape`, `head()`, `info()`, `dtypes` and `describe()`, and read the dtypes pandas reports
- Select columns with `[]`, rows and cells by label with `.loc` and by position with `.iloc`, and filter rows with boolean masks
- Add and change columns in a way that works under copy-on-write, and explain why chained assignment never does
- Sort a table and count the values in a column with `sort_values()` and `value_counts()`

TIMING INSIDE THIS BLOCK

10 min	Frame: from arrays to tables, the labeled shelf
30 min	Live demo: Series, alignment, the four-line greeting, `.loc` / `.iloc`, masks, new columns, copy-on-write, sort and count
25 min	Guided lab: `busiest_days`, then `recommend_groups`
10 min	Start the lesson project (Catalog Health Check, Steps 1 and 2)

MENTAL MODEL TO SAY

Say: "Picture the stockroom at a Pinecrest store. A Series is one shelf of bins, and every bin has a label on the front. A DataFrame is the whole shelving unit: several shelves lined up so the same label runs across all of them. Pull the label `TNT-1002` and you get that product's name, price and rating together. Underneath, every shelf is still a NumPy array, so everything from Module 1 still works. We are adding names to the race cars, not giving them up." Draw three shelves and one row label across them on the board, and leave it up: you will point at it when `.loc` versus `.iloc` comes up.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Type everything in a notebook called `session04` opened in the project folder (the one that holds `data/`). Run each cell before you talk about it, and make the two deliberate mistakes in steps 8 and 11 for real: the error text is part of the lesson. The code follows the lesson page, so learners can find every line again at home.

1. **A Series is an array with labels (2 min).** Say: "Two columns print, but only the right one is data. The left one is the index: the label on each bin." Ask what `boise["Sat"]` will return before you run it.

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
boise = pd.Series([5962.00, 6583.13, 3866.59, 7667.88, 9570.46, 12388.57, 9361
                   index=days, name="boise")
print(boise["Sat"], boise.mean().round(2))
print(boise[boise > 9000])
```

```
12388.57 7914.32
Fri     9570.46
Sat    12388.57
Sun     9361.59
Name: boise, dtype: float64
```

2. **Alignment by label (3 min).** Bend's figures arrive for four days only, in a different order. Ask: "NumPy would raise an error for lengths 7 and 4. What will pandas do?" Then run it. Say: "It matched Monday with Monday. Where a label has no partner, the answer is NaN. When a calculation gives you NaNs you did not expect, compare the two indexes first."

```
bend = pd.Series([6719.33, 7083.36, 4132.92, 3675.57],
                 index=["Sun", "Sat", "Mon", "Tue"], name="bend")
print(boise + bend)
```

```
Fri         NaN
Mon    10094.92
Sat    19471.93
Sun    16080.92
Thu         NaN
Tue    10258.70
Wed         NaN
dtype: float64
```

3. **Load a real table (2 min).** Say: "You will rarely type data in. Shape is rows first, then columns, exactly as for a 2-D array." Point at the backslash where the wide table wraps.

```
products = pd.read_csv("data/products.csv")
print(products.shape)
print(products[["sku", "name", "category", "price", "rating"]].head(3))
```

```
(242, 13)
        sku                                  name category   price  rating
0  TNT-1001     Granite Fork Drift 1-Person Tent    Tents  299.95     4.0
1  TNT-1002  Cedar Notch Ridgeline 2-Person Tent    Tents  217.95     3.7
2  TNT-1003     Pinecrest Solstice 3-Person Tent    Tents  459.95     4.7
```

4 **The four-line greeting: info() (3 min).** Say: "`shape`, `head()`, `info()`, `describe()`: every new table, every time." Have the room find the two columns whose Non-Null Count is not 242 (`weight_kg` 228, `rating` 234). Then point at `launch_date`: it says `str`. "A date is not a date until you say so." Text is `str` in this version of pandas; older tutorials show `object`.

```
products.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 242 entries, 0 to 241
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sku           242 non-null    str
 1   name          242 non-null    str
 2   category      242 non-null    str
 3   product_type  242 non-null    str
 4   brand         242 non-null    str
 5   price         242 non-null    float64
 6   cost          242 non-null    float64
 7   weight_kg     228 non-null    float64
 8   color         242 non-null    str
 9   rating        234 non-null    float64
 10  review_count  242 non-null    int64
 11  stock_status  242 non-null    str
 12  launch_date   242 non-null    str
dtypes: float64(4), int64(1), str(8)
memory usage: 47.1 KB
```

5 **describe(): scan for things that look wrong (2 min).** Ask: "The median price is $82.95 and the mean is $121.80. What does that tell you?" (A few expensive products pull the mean up.) "`review_count` has a minimum of 0. Why does that explain the 8 missing ratings?"

```
print(products[["price", "rating", "review_count"]].describe().round(2))
```

```
         price  rating  review_count
count  242.00  234.00        242.00
mean   121.80    4.14         70.45
std    117.14    0.39         88.57
min      8.50    2.80          0.00
25%     39.95    3.90         14.00
50%     82.95    4.10         39.00
75%    155.95    4.40         89.75
max    616.95    5.00        561.00
```

6 **Labels: set_index and .loc (3 min).** Say: "`set_index` returned a new table and left `products` alone: 13 columns became 12 because `sku` moved to the index. `.loc` is rows, comma, columns, by label." Ask how many rows the slice will return before you run it: three, because a label slice includes its end.

```
by_sku = products.set_index("sku")
print(products.shape, by_sku.shape)
print(by_sku.loc["TNT-1002", "price"])
print(by_sku.loc["TNT-1001":"TNT-1003", ["name", "price"]])
```

```
(242, 13) (242, 12)
217.95
                                          name   price
sku
TNT-1001     Granite Fork Drift 1-Person Tent  299.95
TNT-1002  Cedar Notch Ridgeline 2-Person Tent  217.95
TNT-1003     Pinecrest Solstice 3-Person Tent  459.95
```

7 **Positions: .iloc, and the one-line experiment (2 min).** Say: "`.iloc` ignores labels and counts from 0, like NumPy." Have everyone write a prediction for the last line on paper. `4 3`: labels 0 through 3 are four rows, positions 0 up to 3 are three. Neither is wrong; they answer different questions.

```
print(products.iloc[0, 5], products.iloc[-1, 0])
print(len(products.loc[0:3]), len(products.iloc[0:3]))
```

```
299.95 ACC-1032
4 3
```

8 **A mask, and the first deliberate mistake (4 min).** Build the mask in two steps so they see the True/False Series. "`sum()` of a mask counts the rows, `mean()` gives the share." Then type the `and` version

exactly as a learner would and read the last line of the error aloud: "A Series is 242 answers, and Python's `and` wants one." Fix it with `&` and one pair of parentheses per condition.

```
is_tent = products["category"] == "Tents"
print(is_tent.sum(), round(is_tent.mean(), 3))
products[products["price"] > 100 and products["rating"] > 4.5]
```

```
24 0.099
ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a
```

9 **Fix it: &, | and ~ with parentheses (2 min).** Say: "Seven tents under $200, and most of them are tarps and footprints. A filter is a quick way to find out a category is broader than its name." Show `.isin()` and `~` in the last line: 44 products are not in stock.

```
picks = products[(products["price"] > 100) & (products["rating"] > 4.5)]
cheap_tents = products[(products["category"] == "Tents") & (products["price"]
print(len(picks), len(cheap_tents))
print((~products["stock_status"].isin(["in_stock"])).sum(), products["rating"]
```

```
14 7
44 8
```

10 **New columns, and changing some rows with .loc (3 min).** Say: "No loop. Two Series subtract row by row and the result becomes a new column." Then the sentence to drill: "In `products`, for the rows where the mask is true, in the `price_band` column, store this. One object, one `.loc`, one assignment."

```
products["margin"] = products["price"] - products["cost"]
products["price_band"] = np.where(products["price"] >= 200, "premium", "standa
products.loc[products["price"] < 25, "price_band"] = "budget"
print(products["price_band"].value_counts())
```

```
price_band
standard    166
premium      44
budget       32
Name: count, dtype: int64
```

11 **The second deliberate mistake: chained assignment (4 min).** Ask the room to predict the printed prices, then run it. A red warning appears and the prices have not moved. Say: "Two sets of brackets,

then an equals sign. The first pair made a temporary table of 24 tents; we wrote into that, and it was thrown away. Under copy-on-write every piece you take behaves as its own copy." In a notebook the warning is red and easy to scroll past: make them find the word `ChainedAssignmentError`.

```
products[products["category"] == "Tents"]["price"] = 0
print(products.loc[products["category"] == "Tents", "price"].head(2).tolist())
```

```
ChainedAssignmentError: A value is being set on a copy of a DataFrame or Serie
[299.95, 217.95]
```

The first line shown above is the warning pandas prints (shortened to its first line); the list under it is the real output.

12 **The fix, on a deliberate copy (2 min).** Say: "Same shape as step 10: one `.loc` on the table you want to change. We work on `demo` so `products` keeps its real prices." Compare the two printed lists.

```
demo = products.copy()
demo.loc[demo["category"] == "Tents", "price"] = demo["price"] * 0.9
print(demo.loc[demo["category"] == "Tents", "price"].head(2).round(2).tolist())
print(products.loc[products["category"] == "Tents", "price"].head(2).tolist())
```

```
[269.96, 196.16]
[299.95, 217.95]
```

13 **Sort and count (3 min).** Point at the index on the left of the sorted table: 19, 12, 20. "The labels traveled with their rows. That is why position and label are different things." Then `value_counts(normalize=True)`: about 82% of the catalog is in stock.

```
top = products.sort_values("price", ascending=False)
print(top[["sku", "price"]].head(3))
print(products["stock_status"].value_counts(normalize=True).round(3))
```

```
         sku   price
19  TNT-1020  616.95
12  TNT-1013  585.95
20  TNT-1021  574.95
stock_status
in_stock        0.818
low_stock       0.120
out_of_stock    0.062
Name: proportion, dtype: float64
```

THREE MISTAKES TO EXPECT

- `and` / `or` **between masks, or** `&` **without parentheses.** Symptoms: `ValueError: The truth value of a Series is ambiguous` (for `and`) or the cryptic `TypeError: Cannot perform 'rand_' with a dtyped [float64] array and scalar of type [bool]` (for missing parentheses). Coach: "Read me the last line. How many True/False values is on each side of your `and`?" Then: "Put a pair of parentheses around each condition and tell me what changed." Do not type the fix for them.
- **Chained assignment:** `df[mask]["col"] = value`. Symptom: a `ChainedAssignmentError` warning that is easy to miss, and a table that did not change. Coach: "Count the pairs of brackets before the equals sign. Which table did the first pair create? Where did it go?" The repair is always the same sentence: one `.loc[rows, "col"]` on the table you mean to change. Learners who read older tutorials will reach for `.copy()` after every filter; tell them it is no longer needed.
- **A position where a label was expected (and the reverse).** Symptom: `KeyError: 0` from `by_sku.loc[0]`, or one row too many from `.loc[0:3]`. Coach: "Is 0 a label in this table or a position? Print `df.index[:3]` and decide." Point at the board drawing: `.loc` reads the label on the bin, `.iloc` counts bins from the left. Also expect `KeyError: 'Price'`: names are case-sensitive, so have them print `df.columns.tolist()` and copy the name.

GUIDED LAB

- `labs/module-04/l2_1-dataframes-series/busiest_days_starter.py` (reference: `busiest_days_solution.py`), 15 min: the greeting, sorting, masks with `&` and `~`, `.idxmax()` with `.loc` on `daily_sales.csv`.
- `labs/module-04/l2_1-dataframes-series/recommend_groups_starter.py` (reference: `recommend_groups_solution.py`), 15 min: a new column filled with `.loc[mask, col] = ...`, `value_counts()`, mask means, and a copy-on-write check on `survey.csv`.

Also in the folder: `labs/module-04/l2_1-dataframes-series/debug_filters.py`. A deliberately broken handout with five bugs (three crashes, two silent) for fast finishers and for homework. It is not run by `check_labs.py`.

- Run `busiest_days` in the room for everyone. Circulate during TODO 3: it is the first time they combine masks on their own, and the parentheses error will appear on several screens within two minutes. Let it.
- At minute 12, stop the room and ask one pair to read their three counts. 101 = 36 + 65. If a pair has numbers that do not add up, debug it on the projector together.
- Pairs that finish move to `recommend_groups`. Everyone else takes it home. Before they run TODO 4, make them write down how many rows of `survey` will say 'checked'.
- Do not show the solution file. If someone is stuck for more than three minutes, give them the next line only.

START THE LESSON PROJECT

- The lesson project is the **Catalog Health Check** (six steps, about 30 minutes solo). In the room do Step 1 (load with `parse_dates`, count dtypes, find the two columns with gaps) and Step 2 (margin, margin percent, launch year) together, each learner in their own `lesson_2_1_project` notebook.
- Checkpoint before you move on: everyone's `catalog.shape` is `(242, 13)` and `launch_date` is `datetime64[us]`.
- Steps 3 to 6 (flags with `.loc`, star products, stock gaps, the printed report) go home. Tell them the target numbers: 8 star products, and a report whose median margin is 45.5%.

CHECKS FOR UNDERSTANDING

1. What three things make up a Series? **Answer:** An array of values, an index of labels of the same length, and an optional name.
2. `products["price"]` and `products[["price"]]`: what is the difference? **Answer:** The first is a Series; the second (a list of names) is a one-column DataFrame.
3. Why does `(products["rating"] == np.nan).sum()` print 0 when 8 ratings are missing? **Answer:** NaN is not equal to anything, itself included. Use `.isna()`.
4. After `tents = products[products["category"] == "Tents"]`, does `tents["price"] = 0` change `products`? **Answer:** No. Under copy-on-write `tents` behaves as an independent copy.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** C) `[6.0, nan, 8.0]`; **Q2** `3 2 40`; **Q3** A) 8 products have no rating; **Q4** D) `((products["category"] == "Tents") | (products["price"] < 20)).sum()`; **Q5** `[10, 50, 90]`; **Q6** B) The original row label of the most expensive product. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Pair anyone who was shaky on boolean masks in Lesson 1.2 with a partner for `busiest_days`, and give them the masks one at a time: first print `big.sum()` alone, then `promo.sum()` alone, then combine. Keep the selection table from the lesson (`[]`, `[[]]`, `.loc`, `.iloc`) on the projector during the lab.

Stretch: `debug_filters.py`, then the lesson's bonus challenge: rewrite a filter with `products.query("price > 100 and rating > 4.5")` and confirm with `.equals()` that the result is the same 14 rows. Ask which form they would rather read in six months.

2.2 Loading Data

75 min

OBJECTIVES (THE LESSON'S OWN WORDS)

- Read a CSV file with the options that matter: columns, dtypes, dates, missing-value markers, separators and header position
- Read a sheet from an Excel workbook whose header is not on the first row
- Flatten nested JSON into tables with `pd.json_normalize()`
- Process a file in chunks and shrink its memory use with `usecols` and `dtype`
- Write a DataFrame to CSV, Excel, JSON and Parquet, and explain what each format keeps and loses

TIMING INSIDE THIS BLOCK

10 min	Frame: loading is translation, the interpreter at the border
30 min	Live demo: `read_csv` options, an awkward export, Excel with the header on row 4, nested JSON, memory and chunks, CSV versus Parquet
25 min	Guided lab: `till_export`, then `product_pages`
10 min	Start the lesson project (The Pinecrest Snapshot, Steps 1 and 2)

MENTAL MODEL TO SAY

Say: "A file is a message written in another language. A CSV says 2024-01-01, and it is only characters: nothing in the file says 'this is a date'. `read_csv` is an interpreter who makes a reasonable guess about each column. Good interpreters can be briefed: column three is a date, the word for missing is n/a, ignore the two lines of notes at the top. Today's options are that briefing. And like any careful traveler, you check the translation before you rely on it: `shape`, `dtypes`, `head()` after every load." Write on the board: **file → reader + options → DataFrame → check → (fix the options) → analyze → writer**.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Same notebook, new section. The pattern for every step is: load, look at `dtypes` or `shape`, name what is wrong, add one option, look again. Resist loading with all the right options at once: the lesson is in the before and after. Step 5 is a deliberate mistake.

1. **Look at the raw text first (2 min).** Say: "Before pandas touches it, it is only text: a header line, commas, and two commas with nothing between them where a value is missing." Ask what the empty spot after `cash` is (the promo code).

```
with open("data/orders.csv", encoding="utf-8") as f:
    for _ in range(3):
        print(f.readline().rstrip())
```

```
order_id,customer_id,order_date,channel,store,status,payment_method,promo_code
100001,12673,2024-01-01,store,Flagstaff,completed,cash,,0,0.0,434.9
100002,10034,2024-01-01,store,Asheville,completed,card,,0,0.0,278.9
```

2 **What pandas guesses, and what it will not guess (3 min).** Run the plain load and have the room find the wrong dtype: `order_date` is `str`. "pandas does not guess dates, because guessing wrong is worse than not guessing." Add `parse_dates` and show what a real date can do: Sunday and Saturday are Pinecrest's busiest days.

```
orders = pd.read_csv("data/orders.csv")
print(orders["order_date"].dtype, orders["order_total"].dtype)
orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
print(orders["order_date"].dtype)
print(orders["order_date"].dt.day_name().value_counts().head(2))
```

```
str float64
datetime64[us]
order_date
Sunday      3336
Saturday    3276
Name: count, dtype: int64
```

3 **Only what you need: usecols, index_col, nrows (2 min).** Say: "Three data columns and an index instead of eleven columns. On this file that is tidiness; on a file with 200 columns it is the difference between a load that takes a second and one that fills your memory. And `nrows=5` is the safe way to look at any file you have never seen."

```
slim = pd.read_csv("data/orders.csv", usecols=["order_id", "order_date", "stor
                   parse_dates=["order_date"], index_col="order_id")
print(slim.shape)
print(pd.read_csv("data/order_items.csv", nrows=2))
```

```
(20302, 3)
   order_item_id  order_id       sku  quantity  unit_price
0              1    100001  SLP-1021         1      371.95
1              2    100001  KIT-1014         1       62.95
```

4 **A whole-number column that loads as float (3 min).** Ask: "A birth year is a whole number. Why does it print as 1989.0?" Point at the `nan` at the top: 442 customers gave no year, and NumPy's `int64` cannot say 'missing'. Then the fix: `"Int64"` with a capital I, the nullable integer, and `"category"` for a column with three distinct values repeated 4,850 times.

```
customers = pd.read_csv("data/customers.csv")
print(customers["birth_year"].dtype, customers["birth_year"].head(3).tolist())
customers = pd.read_csv("data/customers.csv", parse_dates=["signup_date"],
                        dtype={"birth_year": "Int64", "loyalty_tier": "categor
print(customers["birth_year"].dtype, customers["birth_year"].head(3).tolist())
print(customers["birth_year"].isna().sum(), customers["loyalty_tier"].dtype)
```

```
float64 [nan, 1989.0, 1995.0]
Int64 [<NA>, 1989, 1995]
442 category
```

5 **Deliberate mistake: lowercase int64 (1 min).** Type the lowercase version and read the error aloud. "Column 8, counting from 0, is `birth_year`. The message tells you exactly which column has the gaps. One capital letter is the whole fix."

```
pd.read_csv("data/customers.csv", dtype={"birth_year": "int64"})
```

```
ValueError: Integer column has NA values in column 8
```

6 **An awkward export: five options, five things you can see (4 min).** Type the export into the notebook so everyone can see the dialect: semicolons, two lines of notes, `55.400,22`, and two spellings of 'nothing here'. Before each option ask, "Which thing in the text does this one fix?" Then show that `revenue` is `float64` and `returns` has real NaNs. `io.StringIO` wraps a string so it can stand in for a file.

```
export = io.StringIO("""Pinecrest regional export
generated by the till system
store;week;revenue;returns
Boise;2024-W01;55.400,22;n/a
Bend;2024-W01;38.128,49;3
Flagstaff;2024-W01;32.769,02;--
""")
regional = pd.read_csv(export, sep=";", skiprows=2, decimal=",", thousands="."
                       na_values=["n/a", "--"])
print(regional)
print(regional.dtypes.astype(str).to_dict())
```

```
       store      week   revenue  returns
0      Boise  2024-W01  55400.22      NaN
1       Bend  2024-W01  38128.49      3.0
2  Flagstaff  2024-W01  32769.02      NaN
{'store': 'str', 'week': 'str', 'revenue': 'float64', 'returns': 'float64'}
```

7 **Excel: the header is not where pandas looked (4 min).** Run the naive load first and let the room react to the column names. "When you see `Unnamed`, the header is not where pandas looked. And notice: no error. The code ran. This is the dangerous kind of mistake." Then list the sheets and skip three rows: 12 months, six named columns.

```
raw = pd.read_excel("data/sales_2025.xlsx")
print(raw.shape, raw.columns.tolist()[1:3])
print(pd.ExcelFile("data/sales_2025.xlsx").sheet_names)
monthly = pd.read_excel("data/sales_2025.xlsx", sheet_name="Monthly Sales", sk
print(monthly.shape)
print(monthly.head(2))
```

```
(15, 6) ['Unnamed: 1', 'Unnamed: 2']
['Monthly Sales', 'Targets']
(12, 6)
  Month   Boise    Bend  Flagstaff  Asheville  Online
0   Jan  242847  169817     160348     147792   86269
1   Feb  218456  146918     154777     127039   86043
```

8 **JSON is a tree: look before you flatten (3 min).** Say: "CSV and Excel are tables. JSON is a tree: values inside dictionaries inside lists. Load it with the `json` module first, so you can see its shape." The records you want are one level down, under `sessions`. Look at one: simple values, nested dictionaries (`device`, `utm`, `cart`) and a list (`pages`).

```
with open("data/web_sessions.json", encoding="utf-8") as f:
    data = json.load(f)
print(list(data), len(data["sessions"]))
one = data["sessions"][1]
print(list(one))
print(one["device"], one["pages"][0])
```

```
['site', 'period', 'note', 'sessions'] 2400
['session_id', 'customer_id', 'started_at', 'device', 'utm', 'pages', 'cart',
{'type': 'desktop', 'os': 'macOS', 'browser': 'Safari'} {'path': '/', 'seconds
```

9 **json_normalize: one column per inner key (3 min).** Say: "Fourteen columns. `device` became `device.type`, `device.os`, `device.browser`." Then the detail that matters: 916 sessions have no `utm` key at all, and `json_normalize` neither failed nor dropped them. "In JSON, a key that is simply absent is a normal way of saying 'missing'." Ids became floats because some are null: fix with `Int64`.

```
sessions = pd.json_normalize(data["sessions"])
print(sessions.shape)
print(sessions.columns.tolist()[6:])
print(sessions["utm.medium"].isna().sum(), sessions["customer_id"].dtype)
sessions["customer_id"] = sessions["customer_id"].astype("Int64")
sessions["started_at"] = pd.to_datetime(sessions["started_at"])
print(sessions["customer_id"].dtype, sessions["started_at"].dtype, sessions["c
```

```
(2400, 14)
['device.type', 'device.os', 'device.browser', 'cart.items', 'cart.value', 'ut
916 float64
Int64 datetime64[us] 0.096
```

10 **Lists inside records: record_path and meta (2 min).** Say: "`record_path` names the key that holds the list; `meta` names the parent fields to copy onto every row. 2,400 sessions became 9,348 page views, and each row carries its `session_id` so the two tables can be connected again." That is the pattern for nested data: one table per level, linked by a key. Lesson 2.5 joins them back.

```
pages = pd.json_normalize(data["sessions"], record_path="pages", meta=["sessio
print(pages.shape)
print(pages.head(3))
```

```
(9348, 4)
                path  seconds                   at session_id
0                  /       45  2025-11-01T06:31:45     S00001
1  /category/sleeping       13  2025-11-01T06:32:30     S00001
2                  /       18  2025-11-01T06:48:03     S00002
```

11 **Big files: measure, load less, read in chunks (3 min).** Say: "The order to try things in: `nrows` to peek, `usecols` to load less, `dtype` to make it smaller, `chunksize` when it still does not fit." The memory falls from 2.08 MB to 0.71 MB, and the chunked total matches the whole-file total exactly: 65,104 units. "At no point were more than 10,000 rows in memory."

```
items = pd.read_csv("data/order_items.csv")
lean = pd.read_csv("data/order_items.csv", usecols=["order_id", "sku", "quanti
                   dtype={"order_id": "int32", "quantity": "int16", "sku": "ca
for t in (items, lean):
    print(round(t.memory_usage(deep=True).sum() / 1024**2, 2), "MB")
total_units = 0
for chunk in pd.read_csv("data/order_items.csv", chunksize=10_000):
    total_units += chunk["quantity"].sum()
print(total_units, items["quantity"].sum())
```

```
2.08 MB
0.71 MB
65104 65104
```

12 **Writing: CSV forgets, Parquet remembers (4 min).** Ask for a prediction before the read-back: "We wrote a date column to a CSV. What dtype comes back?" `str`: a CSV is only text. Parquet stores each column's type with the data, comes back equal to the original, and is under a third of the size. Close with the habit: `index=False` on every `to_csv`, or a colleague gets a column called `Unnamed: 0`.

```
from pathlib import Path
out = Path("output")
out.mkdir(exist_ok=True)
completed = orders[orders["status"] == "completed"]
completed.to_csv(out / "completed_orders.csv", index=False)
completed.to_parquet(out / "completed_orders.parquet")
print(pd.read_csv(out / "completed_orders.csv")["order_date"].dtype)
back = pd.read_parquet(out / "completed_orders.parquet")
print(back["order_date"].dtype, back.equals(completed))
for name in ["completed_orders.csv", "completed_orders.parquet"]:
    print(f"{name:<28}{(out / name).stat().st_size / 1024:>8,.0f} KB")
```

```
str
datetime64[us] True
completed_orders.csv           1,280 KB
completed_orders.parquet         354 KB
```

THREE MISTAKES TO EXPECT

- **The file is not where Python is looking.** Symptom: `FileNotFoundError: [Errno 2] No such file or directory: 'orders.csv'`. Coach: "Which folder is Python working in? Print `Path.cwd()`. Is there a `data` folder inside it?" This is almost always a notebook opened from the

wrong folder, not a typo. In the labs the question does not arise (`DATA` finds the folder), so expect it in the lesson project instead.

- **Trusting a load that ran without error.** Symptoms: `Unnamed: 1` column names and 15 rows instead of 12 from the Excel sheet; `.dt` raising `AttributeError: Can only use .dt accessor with datetimelike values` because the date is still text; a numeric column that is `str` because of one `n/a`. Coach: "What did `shape` and `dtypes` say right after the load?" If they did not look, that is the fix. Make 'load, then look' a spoken rule today.
- `pd.read_json` **on a nested file, or flattening the wrong level.** Symptom: `ValueError: Mixing dicts with non-Series may lead to ambiguous ordering.` or a DataFrame with whole dictionaries in its cells. Coach: "Print `list(data)`. Which key holds the list of records? Is it the file itself or one level down?" Then `json.load` plus `pd.json_normalize(data["sessions"])`, and `record_path` only when they want one row per item of an inner list.

GUIDED LAB

- `labs/module-04/l2_2-loading-data/till_export_starter.py` (reference: `till_export_solution.py`), 15 min: `sep`, `skiprows`, `decimal`, `thousands`, `na_values`, `parse_dates` with `date_format`, then checking the load against a clean file.
- `labs/module-04/l2_2-loading-data/product_pages_starter.py` (reference: `product_pages_solution.py`), 15 min: `json.load`, `pd.json_normalize` with `record_path` and `meta`, fixing dtypes after the load, `.str` filters, a Parquet round trip into `OUTPUT`.
- `labs/module-04/l2_2-loading-data/targets_vs_actuals_starter.py` (reference: `targets_vs_actuals_solution.py`), 15 min: two Excel sheets with different options (`header=3`), a result table built from a list of dictionaries, `to_excel(..., index=False)` and a read-back.
- Start with `till_export`. Give the room two silent minutes to read the six raw lines and list, on paper, everything the reader must be told. Collect the list on the board before anyone types.
- Circulate during TODO 1. The commonest stall is `skiprows`: they count the header line as a note. Ask them to say which line holds the column names.
- At minute 12, ask for the missing store and day, and the total. 163,755.71 on every screen is the signal to move on.
- `product_pages` next for pairs that are done (it writes to an `output/` folder next to the lab file). `targets_vs_actuals` is homework.
- Labs write only to `OUTPUT`. If someone edits a path to write into `data/`, stop them: the course data is read-only all course long.

START THE LESSON PROJECT

- The lesson project is **The Pinecrest Snapshot**: seven tables from CSV, Excel and JSON, saved as Parquet and verified (six steps, about 30 minutes solo).
- In the room do Step 1 (orders, fully typed: dates and five category columns) and Step 2 (customers with `Int64` birth years, products with real launch dates). Checkpoint: `customers["birth_year"].dtype` prints `Int64`.
- Steps 3 to 6 (both Excel sheets, two tables from the JSON, saving, the verification table) go home. Tell them what done looks like: seven `.parquet` files and a verification table that says `True` on every row.
- Ask them to keep the `snapshot` folder: it is a one-line way to start any later analysis.

CHECKS FOR UNDERSTANDING

1. Why does pandas not turn `order_date` into a date on its own? **Answer:** It does not guess dates, because guessing wrong is worse than not guessing. You say so with `parse_dates`.
2. What is the difference between `"Int64"` and `"int64"`? **Answer:** Capital I is pandas's nullable integer and can hold `<NA>`; lowercase is NumPy's and cannot hold a missing value.
3. You see columns called `Unnamed: 1`, `Unnamed: 2`. What happened? **Answer:** The header is not on the row pandas read as the header. Use `skiprows` or `header`.
4. What does a record with no `utm` key become after `json_normalize`? **Answer:** A row with NaN in the `utm.*` columns. Nothing fails and nothing is dropped.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** B) `[4, <NA>, 6]`; **Q8** A) `pd.read_csv(f, sep=";", skiprows=2, decimal=",", thousands=".")`; **Q9** `(3, 2)`; **Q10** D) Load with `parse_dates=["order_date"]`, or convert with `pd.to_datetime()`; **Q11** `['Unnamed: 0', 'a']`; **Q12** C) Read it with `usecols` and `chunksize`, keeping a running total. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: For `till_export`, hand out the option names on a card (`sep`, `skiprows`, `decimal`, `thousands`, `na_values`, `parse_dates`, `date_format`) and let learners match each to a line of the raw text; they only have to fill in the values. For the JSON lab, draw the tree on the board: file → `sessions` (list) → one session → `pages` (list).

Stretch: `targets_vs_actuals`, then the lesson's bonus challenge: one row per cart line with `record_path=["cart", "items"]`, and an eighth Parquet file in the snapshot. Or ask: "Rewrite the chunk loop so it also gives the mean quantity. What two running numbers do you need?" (A sum and a count.)

CLOSING

2:50 Wrap-up

10 min

EXIT TICKET (3 QUESTIONS, ON PAPER OR STICKY NOTES, 5 MINUTES)

1. You run `df[df["price"] > 100]["price"] = 0`. What happens to `df`, and what should you write instead? **Answer:** Nothing happens to `df` (pandas warns with `ChainedAssignmentError`): the first brackets made a temporary copy. Write `df.loc[df["price"] > 100, "price"] = 0`.
2. On a table with the default index, how many rows do `df.loc[0:3]` and `df.iloc[0:3]` return, and why are they different? **Answer:** 4 and 3. `.loc` slices by label and includes the end; `.iloc` slices by position and leaves out the stop.
3. You save a table that has a date column to `out.csv` and to `out.parquet`. Read back with no options, which file gives the date back as a date, and why? **Answer:** The Parquet file. It stores each column's dtype with the data; a CSV is only text, so the date comes back as `str` unless you pass `parse_dates` again.

HOMEWORK (ABOUT 2 TO 3 HOURS BEFORE SESSION 5)

- Finish the **Catalog Health Check** (2.1, Steps 3 to 6) and **The Pinecrest Snapshot** (2.2, Steps 3 to 6). Keep both: the margin columns and the `snapshot` folder come back later.
- Do each lesson's exercises (1 to 4; number 5 is a stretch), Quick Quiz and journal entry.
- Finish any lab you started; `targets_vs_actuals` and `debug_filters.py` are for home.
- Lesson 2.2, 'Before the Next Lesson': run `pd.read_csv("data/orders_messy.csv", nrows=8)`, print `.columns.tolist()` and `.dtypes`, and write down what you notice. Session 5 starts there.

BRING LIST FOR SESSION 5

- Laptop with the course environment working (`import pandas, openpyxl, pyarrow` runs without error). Nothing new to install.
- The course `data/` folder as it is: `orders_messy.csv`, `orders.csv` and `order_items.csv` are the main files next time.
- Your finished notebooks for 2.1 and 2.2, and your notes on what looked wrong in `orders_messy.csv`.
- This workbook. No servers to start, no accounts and no internet connection are needed for Session 5.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT, WHAT TO CUT FIRST

Protect, in this order: the chained-assignment demonstration and its fix (2.1 demo steps 11 and 12), the `.loc` / `.iloc` experiment (step 7), 'load, then look' with the Excel header trap (2.2 step 7), and one guided lab per lesson. Cut first: the alignment step in 2.1 (one sentence), then 2.2 step 11 (memory and chunks: assign the lesson's 'Large Files' section as reading), then step 6 (the awkward export is practiced in the lab anyway), then the second lab of each block. If 2.1 overruns by more than ten minutes, shorten the 2.2 project start to reading the brief aloud and naming the two checkpoints.

RUNNING SHORT: WHAT TO ADD

Run `recommend_groups` and `product_pages` for the whole room, and have two pairs explain their numbers. Then `debug_filters.py` as a race in pairs: first pair to print 82.95, 14, 7, 8 and 29 explains the two silent bugs. Still time: do Step 3 of the Catalog Health Check together and ask why the order of the three `.loc` lines matters.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 5: Cleaning and Grouping

How to teach the session where the file fights back and then starts answering questions. First learners audit a damaged export of 20,515 rows and repair it one documented step at a time: structure, text, numbers, dates, missing values, and prove the result. Then they learn split-apply-combine: `groupby` with named aggregation, `transform` and `filter`, and the quiet traps that return a plausible wrong number.

2 lessons | 3 hrs session | Lessons 2.3–2.4 | Session 5 of 21

HOW TO USE THIS GUIDE

- Each lesson block has four beats: **frame** (the mental model, said aloud), **live demo** (you type, they follow), **guided lab** (new practice from `labs/module-05/`) and **start the lesson project** (the first two steps together). The rest of each project, the lessons' exercises, quizzes and journals are homework.
- Lesson 2.3 is the longest lesson so far (100 minutes solo) and gets 80 minutes of the room; Lesson 2.4 (90 minutes solo) gets 70. The demo scripts are compressed paths through the lesson pages, on the lessons' own files, so every number on the projector matches the page.
- **Type, do not paste.** In 2.3 the rhythm is look, decide, fix, check: say it at every step. In 2.4 ask 'how many rows will come back?' before every groupby and wait for an answer.
- Keep `pandas_data_cleaning.html` and `pandas_groupby_operations.html` open for the reference tables (the four decisions about missing values; agg, transform, filter).
- Starters go to learners; every `*_solution.py` stays with you. `debug_groupby.py` is broken on purpose. `asheville_may_log.csv` is a data file for the first lab: it must never be edited.

> **THE #1 RISK OF SESSION 5: PLAUSIBLE WRONG NUMBERS**
>
> Both lessons are about results that look fine and are not. In 2.3 it is the value that converts without complaint: a total 100 times too large, a `dropna()` that keeps 2,085 rows of 20,302, a `fillna(0)` that says 40 customers paid nothing. In 2.4 it is the groupby that drops 18,154 rows because their key is missing, a `count` that should have been a `size`, an average of averages. None of these raises an error. The habit to train all session: **"Count the rows before and after. Do the groups add up?"** The second risk is time: the 2.3 demo is long. If you are past 0:50 at the dates step, show the three-format idea in one cell and move on.

BEFORE THE SESSION

- ☐ Copy the **starter** files, `debug_groupby.py` and `asheville_may_log.csv` from `labs/module-05/` to the shared folder or each machine, keeping the folder layout. Keep every `*_solution.py` on your machine.
- ☐ From the kit folder run `python3 labs/check_labs.py module-05`. All 8 lines should say PASS.
- ☐ No servers to start and no internet needed. Files used today: `orders_messy.csv`, `orders.csv`, `order_items.csv`, `daily_sales.csv`, `survey.csv`.
- ☐ Environment traps: none new. A learner whose `raw.dtypes` shows `object` instead of `str` is not in the course environment. If `orders_messy.csv` has been opened and saved in a spreadsheet program, its flaws may have changed: have that learner unzip `pinecrest-data.zip` again.
- ☐ Projector: a notebook called `session05` opened in the project folder, font at least 18 pt, `pd.set_option("display.width", 150)` and `pd.set_option("display.max_columns", 20)` in the first cell.
- ☐ Whiteboard, before learners arrive: the loop **audit → pick one → fix on a copy → check → next**, the order **structure, text, numbers, dates, missing**, and, for after the break, the three shapes with **agg / transform / filter**.
- ☐ Skim the Session 4 homework: the Catalog Health Check and the Pinecrest Snapshot, and each learner's notes on what looked wrong in `orders_messy.csv` (Lesson 2.2, 'Before the Next Lesson'). Those notes open the 2.3 block.
- ☐ Sticky notes for the exit ticket. Print the workbook for Session 5.

SESSION 5 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 4 (DataFrames, loading), homework check, what learners noticed in the messy file.
0:10	10	**2.3** Frame and mental model	The repair bench; the raw file is read-only; audit, fix one thing, check.
0:20	35	**2.3** Live demo	`orders_messy.csv`: audit, structure, text, numbers, an outlier with evidence, dates, missing values, assertions (two deliberate mistakes).
0:55	25	**2.3** Guided lab	`labs/module-05/l2_3-cleaning-data/register_log_starter.py`: a second messy file and their own repair list.
1:20	10	**2.3** Start the lesson project	The Cleaning Pipeline, Steps 1 and 2 together; the rest goes home.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:30	10	Break	Leave the before-and-after audit on the projector.
1:40	10	**2.4** Frame and mental model	Sorting the mail: split, apply, combine. What shape do I want back?
1:50	30	**2.4** Live demo	`orders.csv`: first groupby, counting, named aggregation, rates, two keys, transform, filter, two quiet traps.
2:20	20	**2.4** Guided lab	`labs/module-05/l2_4-groupby-operations/store_days_starter.py`.
2:40	10	**2.4** Start the lesson project	The Store Scorecard, Steps 1 and 2 together; the rest goes home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 6.
Total	180		Ends at 3:00.

In the room: both mental models, both live demos, the guided labs `register_log` and `store_days` for everyone, and the first two steps of each lesson project. **Goes home:** Steps 3 to 6 of both projects, the lessons' exercises, both Quick Quizzes and journals, and the labs `total_row_check`, `basket_sizes` and `debug_groupby.py`: about 3 hours this week, the heaviest homework of Module 2. Say so, and say that the cleaning pipeline is a portfolio piece.

Protect: the audit and the outlier-with-evidence step in 2.3, the `register_log` lab, and in 2.4 named aggregation, `transform` and the two quiet traps. **Cut first if running long:** the status half of the text step and the assertion step in 2.3 (both are in the project), then the computed-key step and `filter` in 2.4 (say the transform-then-mask pattern in one sentence).

WARM-UP

0:00 Warm-up: Retrieval from Session 4 — 10 min

FIVE RETRIEVAL QUESTIONS (NO NOTES, 5 MINUTES)

Slide 3. Paper only, notebooks closed, then one minute with a neighbor. Take answers quickly; the fifth question leads into today.

1. What three things make up a Series? **Answer:** An array of values, an index of labels, and an optional name.
2. On a table with the default index, how many rows do `df.loc[0:3]` and `df.iloc[0:3]` return? **Answer:** 4 and 3: a label slice includes its end, a position slice leaves out the stop.
3. `df[df["price"] > 100]["price"] = 0` changes nothing. What do you write instead? **Answer:** `df.loc[df["price"] > 100, "price"] = 0`: one `.loc` on the table you want to change.
4. A column of birth years loads as `float64` and prints `1989.0`. Why, and what is the fix? **Answer:** Some years are missing and NumPy's `int64` cannot hold a missing value. Load it as `"Int64"` (capital I).
5. After `pd.read_excel(...)` the columns are called `Unnamed: 1`, `Unnamed: 2`. What happened, and what do you check after every load? **Answer:** The header is not on the row pandas read as the header (`skiprows` / `header`). After every load: `shape`, `dtypes`, `head()`.

HOMEWORK CHECK (3 MINUTES)

- Show of hands: whose Pinecrest Snapshot printed `True` for all seven tables? Whose Catalog Health Check found 8 star products?
- Ask three learners what they noticed when they peeked at `orders_messy.csv` with `nrows=8`. List the answers on the board without comment (spaces in the header names, text where numbers should be, odd dates).
- Anyone who did not finish the 2.2 project needs only `orders.csv` today; reassure them.

BRIDGE TO TODAY (2 MINUTES)

Say: "You found three or four problems by looking at eight rows. The file has 22 kinds of damage in 20,515 rows. Today you learn to find them by counting, not by luck, and to prove you fixed them."

2.3 Cleaning Data

80 min

OBJECTIVES (THE LESSON'S OWN WORDS)

- Audit an unfamiliar table and list its problems before changing anything
- Repair structure: untidy headers, summary rows and duplicate rows
- Standardize inconsistent text and convert numbers and dates that were stored as text, including dates in several formats
- Decide, and defend, what to do with each kind of missing or suspicious value: repair, fill, flag or leave
- Verify a cleaning job with a before-and-after audit and assertions, and package it as a function you can rerun

TIMING INSIDE THIS BLOCK

10 min	Frame: the repair bench; audit, fix one thing, check; the raw file is read-only
35 min	Live demo on `orders_messy.csv`: audit, structure, text, numbers, an outlier with evidence, dates, missing values, assertions
25 min	Guided lab: `register_log` (a second messy file, their own repair list)
10 min	Start the lesson project (The Cleaning Pipeline, Steps 1 and 2)

MENTAL MODEL TO SAY

Say: "Every Pinecrest store has a repair bench for rental tents that come back damaged. The technician does the same thing every time. Inspect the whole tent and write down every problem before fixing any. Repair one thing at a time, structural damage first: poles, then seams, then small holes. Check each repair before starting the next. Never throw away the damage report. And some stains do not come out: the honest move is to note them on the tag, not to paint over them. Cleaning data is that routine. The raw file is the tent as it came back, and you never alter it. Your code is the repair log." Write the loop on the board and leave it up: **audit → pick one problem → fix it in code, on a copy → check → next**. Under it, the order: **structure, text, numbers, dates, missing values**.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Type in a notebook called `session05`. This demo is a compressed path through the lesson page on the same file, `data/orders_messy.csv`, so the numbers match the page. The rhythm of every step is the same and worth saying aloud each time: **look, decide, fix, check**. Steps 2 and 10 are deliberate mistakes. Step 9 repairs four columns with moves the room has already seen: go fast there.

1. **Load it as it is: the dtypes are a damage report (2 min).** Say: "No options at all, so we see the file as it is." Ask: "There should be about 20,300 orders. What does 20,515 suggest? Why would an order id be text? Why would a discount be a float?" Collect guesses; do not answer yet.

```
raw = pd.read_csv("data/orders_messy.csv")
print(raw.shape)
print(raw.dtypes)
```

```
(20515, 11)
Order ID              str
 Customer ID          str
Order Date            str
Channel               str
Store                 str
Status                str
Payment Method        str
Promo Code            str
Discount %        float64
Shipping Cost         str
Order Total           str
dtype: object
```

2. **Deliberate mistake: a name that looks right (2 min).** Type the column name as it appears in the printed table, and read the `KeyError`. Then print the names as a list: "A table display hides what a list shows. `' Customer ID'` starts with a space and `'Order Total '` ends with one."

```
print([name for name in raw.columns if name != name.strip()])
raw["Customer ID"]
```

```
[' Customer ID', 'Order Total ']
KeyError: 'Customer ID'
```

3. **The last row is not an order (2 min).** Say: "Someone added a TOTAL line, as people do in spreadsheets. That one row explains two of the odd dtypes: the word TOTAL forced the whole Order ID column to text, and its empty Discount cell forced that column to float." Tell them to remember the number 3,950,701.45: one of today's labs uses it.

```
print(raw.tail(2).iloc[:, [0, 1, 2, 8, 10]])
```

```
      Order ID  Customer ID  Order Date  Discount % Order Total
20513   120302        13152  12/31/2025         0.0        43.40
20514    TOTAL          NaN         NaN         NaN   3950701.45
```

4 **An audit you can rerun (3 min).** Say: "Looking at rows finds problems by luck. Counting finds them systematically." Read the `distinct` column against the business: "Pinecrest has four stores and an online shop. How many distinct values should `Store` have? It has 31. Three statuses: the file has 15. Four payment methods: 7." The number of distinct values is the fastest test there is for a column of categories.

```
def audit(df):
    """One row per column: dtype, missing values, distinct values."""
    return pd.DataFrame({"dtype": df.dtypes.astype(str),
                         "missing": df.isna().sum(), "distinct": df.nunique()})

before = audit(raw)
print(before)
print("rows:", len(raw), " exact duplicate rows:", raw.duplicated().sum())
```

```
                 dtype  missing  distinct
Order ID           str        0     20303
 Customer ID       str      102      3942
Order Date         str       26      1893
Channel            str        1         2
Store              str        1        31
Status             str        1        15
Payment Method     str      409         7
Promo Code         str    17789        10
Discount %     float64        1         5
Shipping Cost      str       32         5
Order Total        str       41      7135
rows: 20515  exact duplicate rows: 212
```

5 **See the variants, then write the repair list (3 min).** Point at the two lines that both say Online: one of them carries a stray space. "To a person these are one store. To a computer they are six." Now stop typing and write the repair list on the board with the room, most structural first: headers; the TOTAL row and 212 duplicates; inconsistent text; numbers stored as text; dates stored as text; missing values.

```
print(raw["Store"].value_counts().head(11))
```

```
Store
Online       11576
Boise         2418
Bend          1609
Flagstaff     1597
Asheville     1316
website        312
Web            299
online         212
ONLINE         204
Online         200
 Online        191
Name: count, dtype: int64
```

6 **Structure: headers, the stray row, duplicates (3 min).** Say: "Work on a copy called `clean`. `raw` stays exactly as loaded." Column names are text, so `.str` works on them. Before dropping duplicates ask the question aloud: "Could two identical rows be legitimate here?" Every row is an order with its own id, so no. Check: 20,302 rows, every `order_id` unique. With the TOTAL row gone, `order_id` converts cleanly: that is why structure comes first.

```
clean = raw.copy()
clean.columns = clean.columns.str.strip().str.lower().str.replace(" ", "_")
clean = clean.rename(columns={"discount_%": "discount_pct"})
clean = clean[clean["order_id"] != "TOTAL"]
print(len(clean), clean.duplicated().sum())
clean = clean.drop_duplicates().reset_index(drop=True)
clean["order_id"] = clean["order_id"].astype("int64")
print(len(clean), clean["order_id"].is_unique)
```

```
20514 212
20302 True
```

7 **Text, mechanical part: strip, then case (2 min).** Say: "Text typed by people varies in three ways: whitespace, case, wording. In that order." Do not assign back yet: look at what is left first. Two mechanical steps took 31 variants down to 11.

```
store = clean["store"].str.strip().str.title()
print(clean["store"].nunique(), "->", store.nunique())
print(store.value_counts().tail(6))
```

```
31 -> 11
store
Website     307
Web         295
Bosie        61
Bend Or      38
Flagstaf     38
Ashville     23
Name: count, dtype: int64
```

8 **Text, judgment part: a mapping dictionary (3 min).** Say: "No rule can know that Bosie means Boise. You write that knowledge down as a dictionary, and the dictionary documents every judgment call." Point at the key `"Bend Or"`: the file says `Bend OR` and title case changed it, which is why the mapping is built from the values after the mechanical steps. Then status, the same three moves. Ask: "Is refunded really the same as returned? Who decides?" (The person who owns the data. A merge of two categories is a claim about the business.)

```
store_fixes = {"Web": "Online", "Website": "Online", "Bosie": "Boise",
               "Flagstaf": "Flagstaff", "Ashville": "Asheville", "Bend Or": "B
clean["store"] = store.replace(store_fixes)
print(clean["store"].value_counts().to_dict())
status = clean["status"].str.strip().str.lower()
clean["status"] = status.replace({"complete": "completed", "refunded": "return
                                  "canceled": "cancelled"})
print(clean["status"].value_counts().to_dict())
```

```
{'Online': 12854, 'Boise': 2594, 'Bend': 1725, 'Flagstaff': 1716, 'Asheville'
{'completed': 18762, 'returned': 1143, 'cancelled': 397}
```

9 **The other text and id columns: the same moves (2 min).** Say: "Nothing new here, so I will go fast; every line is on the lesson page." Payment method: strip, lowercase, one mapping. Promo code: uppercase, then the placeholders `none` and `-` become real missing values with `.where()`, or every count of 'orders with a promo code' is 544 too high. Shipping: the words `free` and `FREE` mean 0, and in-store orders never pay shipping, so their 9 blanks are repaired by a business rule. Customer id: `unknown` becomes missing and the column becomes `Int64`.

```
pay = clean["payment_method"].str.strip().str.lower()
clean["payment_method"] = pay.replace({"credit card": "card"})
promo = clean["promo_code"].str.strip().str.upper()
clean["promo_code"] = promo.where(~promo.isin(["NONE", "-"]))
ship = pd.to_numeric(clean["shipping_cost"].str.strip().str.lower().replace({"
clean["shipping_cost"] = ship.where(~((clean["channel"] == "store") & ship.isn
clean["customer_id"] = pd.to_numeric(clean["customer_id"], errors="coerce").as
print(clean[["payment_method", "promo_code", "shipping_cost"]].nunique().to_di
print(clean["promo_code"].isna().sum(), clean["customer_id"].isna().sum())
```

```
{'payment_method': 4, 'promo_code': 4, 'shipping_cost': 3}
18154 125
```

10 **Deliberate mistake: converting before cleaning (3 min).** Type the `astype` and read the error: "It stops at the first value it cannot convert and shows it to you. A free audit finding." Then the detector: `errors="coerce"` turns what it cannot convert into NaN, so the values that became NaN but were not empty are the troublemakers: 1,421 totals written as currency.

```
as_number = pd.to_numeric(clean["order_total"], errors="coerce")
not_number = as_number.isna() & clean["order_total"].notna()
print(not_number.sum(), clean.loc[not_number, "order_total"].head(3).tolist())
clean["order_total"].astype("float64")
```

```
1421 ['$107.90', '$53.95', '$255.90']
ValueError: could not convert string to float: '$107.90'
```

11 **Convert for real, then describe() (2 min).** Say: "No `errors=` argument now: if anything unconvertible is left, we want an error. That is a guarantee." The moment the column is numeric, `describe()` shows two problems that were invisible while it was text: a minimum of -713.90 and a maximum of 55,690.

```
total = clean["order_total"].str.replace("$", "").str.replace(",", "")
clean["order_total"] = pd.to_numeric(total)
print(clean["order_total"].describe().round(2).loc[["count", "mean", "min", "m
negative = clean["order_total"] < 0
print(negative.sum(), clean.loc[negative, "status"].unique().tolist())
clean["order_total"] = clean["order_total"].abs()
```

```
count    20262.00
mean       201.55
min       -713.90
max      55690.00
Name: order_total, dtype: float64
15 ['returned']
```

12 **An outlier is a question, not a verdict (5 min).** Say: "All 15 negatives were returns, so `.abs()` was safe. But a $55,690 online order? Do not delete it and do not 'fix' it because it is big: the $4,893 order is genuine. Look for independent evidence." The item lines are that evidence. `.map()` uses a Series as a lookup table. Eight orders have a total about 100 times what their items add up to: a lost decimal point. Point at order 102672, recorded as $4,990, barely above the genuine $4,893: "No threshold on the total alone could separate those two. The item evidence does." The mean falls from 201.55 to 194.60: eight rows in twenty thousand had inflated it by 3.6%.

```
items = pd.read_csv("data/order_items.csv")
items["line_total"] = items["quantity"] * items["unit_price"]
item_subtotal = items.groupby("order_id")["line_total"].sum()
subtotal = clean["order_id"].map(item_subtotal)
ratio = clean["order_total"] / subtotal
lost_decimal = ratio > 50
print(clean.loc[lost_decimal, ["order_id", "order_total"]].assign(
    items=subtotal[lost_decimal].round(2), ratio=ratio[lost_decimal].round(1))
clean.loc[lost_decimal, "order_total"] = (clean.loc[lost_decimal, "order_total
print(clean["order_total"].mean().round(2), clean["order_total"].max())
```

```
       order_id  order_total   items  ratio
2671     102672       4990.0   34.95  142.8
5842     105843       8435.0   84.35  100.0
7770     107771      18070.0  180.70  100.0
9848     109849      55690.0  556.90  100.0
11295    111296       5590.0   55.90  100.0
13627    113628      33090.0  330.90  100.0
13949    113950       6035.0   53.40  113.0
17852    117853      16985.0  169.85  100.0
194.6 4893.0
```

13 **Dates in three formats (3 min).** Say: "Three groups of people typed dates, each in their own way. Parse once per format with `errors=\"coerce\"`, so each attempt understands its own format and nothing else. Then combine: `fillna` with another Series fills each gap from the same row." 31 are left: 25 were empty and 6 cannot exist (February 30th, month 13). "`coerce` correctly refused to invent a date."

```
d = clean["order_date"]
iso = pd.to_datetime(d, format="%Y-%m-%d", errors="coerce")
us = pd.to_datetime(d, format="%m/%d/%Y", errors="coerce")
long = pd.to_datetime(d, format="%B %d, %Y", errors="coerce")
print(iso.notna().sum(), us.notna().sum(), long.notna().sum())
clean["order_date"] = iso.fillna(us).fillna(long)
print(clean["order_date"].dtype, clean["order_date"].isna().sum())
print(d[clean["order_date"].isna()].dropna().tolist())
```

```
17632 1827 812
datetime64[us] 31
['2025-04-31', '2024-02-30', '2025-02-29', '2024-06-31', '2024-13-01', '2025-(
```

14 **Missing values are decisions (3 min).** Ask for a prediction: "`dropna()` with no arguments: how many of 20,302 rows survive?" 2,085, because most orders have no promo code, and for promo code 'no value' is the true answer. Name the four decisions: it is not missing; repair (a rule or another source gives the value); leave and report; drop for one analysis with `subset=` and say how many. Then the cost of filling: 0 claims 40 customers paid nothing.

```
print(clean[["order_date", "payment_method", "promo_code", "order_total"]].isn
print(len(clean.dropna()), len(clean.dropna(subset=["order_date", "order_total
print(clean["order_total"].mean().round(2), clean["order_total"].fillna(0).mea
```

```
{'order_date': 31, 'payment_method': 406, 'promo_code': 18154, 'order_total':
2085 20231
194.6 194.22
```

15 **Rules as assertions (2 min).** Say: "An `assert` states something that must be true and stops the program if it is not. A handful of them turn what you know about the business into an automatic test. If next month's export has a new misspelling, the store assertion fails loudly instead of letting a sixth store slip into a report." Close by re-running `audit(clean)` on the projector and comparing it with `before` by eye.

```
assert clean["order_id"].is_unique
assert set(clean["store"]) == {"Online", "Boise", "Bend", "Flagstaff", "Ashevi
assert set(clean["status"]) == {"completed", "returned", "cancelled"}
assert clean["order_total"].dropna().between(0, 5000).all()
assert clean["shipping_cost"].dropna().isin([0, 6.95, 14.95]).all()
print("all checks passed")
print(audit(clean))
```

```
all checks passed
                          dtype  missing  distinct
order_id                  int64        0     20302
customer_id               Int64      125      3941
order_date      datetime64[us]       31       731
channel                    str        0         2
store                      str        0         5
status                     str        0         3
payment_method             str      406         4
promo_code                 str    18154         4
discount_pct           float64        0         5
shipping_cost          float64       18         3
order_total            float64       40      6302
```

THREE MISTAKES TO EXPECT

- **Fixing before looking, or in the wrong order.** Symptoms: `KeyError: 'Customer ID'` (the invisible leading space), `ValueError: could not convert string to float: '$107.90'` (converting before cleaning), an `astype("int64")` that fails because the TOTAL row is still there. Coach: "What did your audit say about that column? What is on your repair list above this item?" Have them print `df.columns.tolist()` and fix structure before contents. Each of these error messages quotes the exact value that broke the rule: teach them to read it as a finding.
- **Case without whitespace, or a mapping written from memory.** Symptoms: `nunique()` barely falls after `.str.title()`; a mapping key such as `"Bend OR"` that never matches because title case already turned it into `Bend Or`; `.str.replace("$", "", regex=True)` that changes nothing. Coach: "Print `value_counts()` after the mechanical steps and build the dictionary from what you see, not from what you remember." Always `.str.strip()` first.
- **Making the NaNs go away.** Symptoms: `dropna()` with no arguments (2,085 rows left of 20,302, no warning); `fillna(0)` on a money column; `df["col"].fillna(..., inplace=True)`, which warns with `ChainedAssignmentError` and fills nothing. Coach: "What are you claiming about those 40 customers when you write 0? How many rows did you have before and after that line?" The rule to repeat: a missing value is a decision, and the decision gets written down.

GUIDED LAB

- `labs/module-05/l2_3-cleaning-data/register_log_starter.py` (reference: `register_log_solution.py`), 25 min: the whole routine on a file they have not been walked through: audit, structure, text, numbers, an outlier with evidence, three date formats, honest blanks, assertions, a scorecard.
- `labs/module-05/l2_3-cleaning-data/total_row_check_starter.py` (reference: `total_row_check_solution.py`), 20 min: using a control total: the same repairs as the lesson, applied one at a time while tracking a sum, then explaining the remainder against the answer key.

- Everyone does `register_log`. Before anyone fixes anything, give the room four silent minutes with the audit output and have each learner write a repair list in the starter file (there is a place for it). Take three lists aloud and compare their order.
- Circulate during TODO 3. The teaching moment is the outlier: learners will want to write `revenue > 20000`. Ask: 'What independent evidence do you have in this row?' (transactions).
- Do not let anyone edit `asheville_may_log.csv`. If someone opens it in a spreadsheet program to 'just fix' a cell, that is the lesson's first rule: stop and say it again.
- At minute 20 ask for scorecards. `still blank: 2 | wrong: 0` is the target. Ask the room: 'Is two blanks a failure?' (No: nothing false.)
- TODO 6 and `total_row_check` are for fast finishers and for homework.

START THE LESSON PROJECT

- The lesson project is **The Cleaning Pipeline**: today's fixes packaged as four small functions chained with `.pipe()`, a `validate()` step, and a scorecard against `orders.csv` (six steps, about 35 minutes solo).
- In the room, do Step 1 (`fix_structure`) and Step 2 (`fix_text`) together in `lesson_2_3_project`. Checkpoints: `(20302, 11) True`, then 5 stores, 3 statuses, 4 payment methods, 4 promo codes.
- Say why a function and not a notebook of cells: when next month's export arrives, the whole job reruns in a second, and every mapping lives in a named dictionary at the top.
- Steps 3 to 6 go home. Done looks like: `validate()` passes and the scorecard prints 0 wrong cells and 623 still blank.

CHECKS FOR UNDERSTANDING

1. Why is the raw file never edited by hand? **Answer:** A hand edit leaves no record, cannot be reviewed and must be redone for every new export. Every fix is a line of code on a copy.
2. What does the `distinct` column of the audit tell you about a category column? **Answer:** Whether it has the number of values the business has. 31 store names for 5 stores means variants to standardize.
3. `pd.to_numeric(s, errors="coerce")` and `pd.to_numeric(s)`: when do you use each? **Answer:** With `coerce` as a detector (find what is not a number). Without it at the end, as a guarantee: it raises if anything unconvertible is left.
4. What made it right to divide eight totals by 100 but leave the $4,893 order alone? **Answer:** Evidence from another table: those eight were about 100 times their item subtotals; the $4,893 order matched its items.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) Print `value_counts()` to see the variants, then strip, fix the case, and map what is left; **Q2** `1 2`; **Q3** C) `[nan, 'TRAIL15', nan, 'TRAIL15']`; **Q4** D) Its item lines add up to $34.95, so the recorded total is over 100 times what was bought; **Q5** `30.0 20.0`; **Q6** A) `trial` is unchanged, and pandas warns with `ChainedAssignmentError`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: Give anyone who is lost the repair list as a checklist (headers; TOTAL row and duplicates; store; promo; revenue; counts; dates) and have them do one item, run, and tick it. For the dates, point them at the three `pd.to_datetime` lines on the lesson page and ask them only to change the column name. Pair weaker typists with a partner for `register_log`: the thinking matters more than the keystrokes.

Stretch: TODO 6 of `register_log` (recover the pending day from the TOTAL row), then `total_row_check`. Further: the lesson's Exercise 4 (recover 30 of the 31 missing dates from neighboring order ids with `ffill` and `bfill`), which is the same idea of repairing on a firm rule.

2.4 GroupBy Operations

70 min

OBJECTIVES (THE LESSON'S OWN WORDS)

- Explain split-apply-combine and read a groupby expression piece by piece
- Summarize groups with built-in methods and with named aggregation, on one key or several
- Choose correctly between `agg` (one row per group), `transform` (one row per original row) and `filter` (keep or drop whole groups)
- Control what happens to missing keys and unused categories with `dropna=` and `observed=`
- Recognize the common traps: lost rows, a key hidden in the index, and averages of averages

TIMING INSIDE THIS BLOCK

10 min	Frame: sorting the mail; split, apply, combine; 'for each...' questions
30 min	Live demo on `orders.csv`: the first groupby, counting, named aggregation, rates, two keys, a computed key, transform, filter, the quiet traps
20 min	Guided lab: `store_days`
10 min	Start the lesson project (The Store Scorecard, Steps 1 and 2)

MENTAL MODEL TO SAY

Say: "Imagine the day's receipts from all of Pinecrest dumped on one table. To get revenue by store you would do three things without thinking. Split: deal the receipts into piles, one per store. Apply: add up each pile. Combine: write the totals on one sheet, one line per pile. That is all `groupby` is, and it has a name: split-apply-combine." Then the question to train all block long: **"What shape do I want back? One row per group, the same rows with a group number on each, or fewer of the same rows?"** Write those three shapes on the board with `agg`, `transform`, `filter` next to them.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New section of the notebook. Load the clean table first: `orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])` (20,302 rows). Before every groupby, ask the room how many rows will come back and wait for a number. Step 6 is a deliberate mistake with an error message; steps 12 and 13 are the quiet kind, with no error at all.

1. **By hand first, then one line (3 min).** Say: "You already have the tools for one store: filter, then sum. That is split and apply by hand, and the combining is left to you." Then the one-liner, read left to right: `groupby("store")` splits, `["order_total"]` selects, `.sum()` applies and combines.

```
for store in ["Bend", "Boise"]:
    one_store = orders[orders["store"] == store]
    print(store, round(one_store["order_total"].sum(), 2))
revenue = orders.groupby("store")["order_total"].sum()
print(revenue.round(2))
```

```
Bend 298263.94
Boise 459714.9
store
Asheville     247133.35
Bend          298263.94
Boise         459714.90
Flagstaff     315077.90
Online       2630511.36
Name: order_total, dtype: float64
```

2 **What comes back (2 min).** Say: "An ordinary Series. Its index is the group key, sorted. So everything from Lesson 2.1 applies: look one group up by label, sort the values." And the GroupBy object by itself computes nothing: it is a plan. `get_group` lets you look inside one pile when a result surprises you.

```
print(type(revenue).__name__, revenue.index.tolist())
print(revenue["Boise"].round(2))
grouped = orders.groupby("store")
print(grouped.ngroups, grouped.get_group("Bend").shape)
```

```
Series ['Asheville', 'Bend', 'Boise', 'Flagstaff', 'Online']
459714.9
5 (1725, 11)
```

3 **Counting: size, count, nunique (3 min).** Three questions that sound alike. "How many rows? `size`. How many non-missing values in this column? `count`. How many distinct values? `nunique`." 12,854 online orders; 1,338 of them have a promo code; 3,456 different customers placed them.

```
print(orders.groupby("channel").size().to_dict())
print(orders.groupby("channel")["promo_code"].count().to_dict())
print(orders.groupby("channel")["customer_id"].nunique().to_dict())
```

```
{'online': 12854, 'store': 7448}
{'online': 1338, 'store': 810}
{'online': 3456, 'store': 2511}
```

4 **Named aggregation: the form to learn by heart (3 min).** Say: "Each argument reads new column equals source column, function. The output has exactly the columns you named, in the order you wrote them, and anyone reading the code can see where each number comes from." Ask how many rows before running: five.

```
summary = orders.groupby("store").agg(
    orders=("order_id", "count"),
    revenue=("order_total", "sum"),
    avg_order=("order_total", "mean"),
    customers=("customer_id", "nunique"),
)
print(summary.round(2))
```

```
           orders     revenue  avg_order  customers
store
Asheville    1413   247133.35     174.90        592
Bend         1725   298263.94     172.91        677
Boise        2594   459714.90     177.22        873
Flagstaff    1716   315077.90     183.61        601
Online      12854  2630511.36     204.65       3456
```

5 **Deliberate mistake: where did the key go? (2 min).** Type `summary["store"]` and read the `KeyError`. "After a groupby the key is the index, not a column." Two cures: `.reset_index()` afterward, or `as_index=False` up front.

```
print(summary.reset_index().columns.tolist())
print(orders.groupby("store", as_index=False)["order_total"].sum().head(2))
summary["store"]
```

```
['store', 'orders', 'revenue', 'avg_order', 'customers']
       store  order_total
0  Asheville    247133.35
1       Bend    298263.94
KeyError: 'store'
```

6 **A rate is the mean of True and False (3 min).** Say: "Make a boolean helper column first. True counts as 1, so its mean within each group is the rate." This is a real finding: online orders are returned twice as often as in-store ones, 6.9% against 3.4%.

```
orders["returned"] = orders["status"] == "returned"
orders["used_promo"] = orders["promo_code"].notna()
rates = orders.groupby("channel").agg(
    orders=("order_id", "count"),
    return_rate=("returned", "mean"),
    promo_rate=("used_promo", "mean"),
)
print(rates.round(3))
```

```
         orders  return_rate  promo_rate
channel
online    12854        0.069       0.104
store      7448        0.034       0.109
```

7 **Two keys, and unstack (3 min).** Say: "A list of keys gives one group per combination that occurs: two channels times three statuses, six groups. The index now has two levels." Show the two ways out: `reset_index` for ordinary columns, `unstack` for the cross-table. (Lesson 2.6 is about that reshaping.)

```
counts = orders.groupby(["channel", "status"]).size()
print(counts)
print(counts.loc[("online", "returned")])
print(counts.unstack())
```

```
channel  status
online   cancelled      360
         completed    11604
         returned       890
store    cancelled       37
         completed     7158
         returned       253
dtype: int64
890
status   cancelled  completed  returned
channel
online         360      11604       890
store           37       7158       253
```

8 **A key you compute on the spot (3 min).** Say: "A key does not have to be a column. Any Series with one value per row will do." Completed revenue by year and channel, unstacked so the years sit side by side and growth is one line of arithmetic: online +18.8%, account-holder store orders +9.1%.

```
completed = orders[orders["status"] == "completed"]
year = completed["order_date"].dt.year.rename("year")
table = completed.groupby([year, "channel"])["order_total"].sum().unstack("yea
table["growth_pct"] = ((table[2025] / table[2024] - 1) * 100).round(1)
print(table.round(2))
```

```
year            2024        2025  growth_pct
channel
online   1077800.75  1280183.70        18.8
store     605498.76   660823.06         9.1
```

9 **transform: a group statistic on every row (3 min).** Ask: "How many values will this return?" 20,302, not 5. "`transform` computes the same five averages and broadcasts each one back to every row of its group, in the original order. So it goes straight into a new column." Order 100001 was 2.37 times the average Flagstaff order.

```
store_avg = orders.groupby("store")["order_total"].transform("mean")
print(store_avg.shape, orders.shape[0])
orders["vs_store_avg"] = orders["order_total"] / store_avg
print(orders[["order_id", "store", "order_total", "vs_store_avg"]].head(3).rou
```

```
(20302,) 20302
   order_id      store  order_total  vs_store_avg
0    100001  Flagstaff        434.9          2.37
1    100002  Asheville        278.9          1.59
2    100003     Online         75.9          0.37
```

10 **filter, and its faster twin (3 min).** Say: "`filter` returns original rows, not a summary: the 496 orders of the 16 customers who each placed 25 or more." Then the twin: a transform and an ordinary mask give the identical rows, with no Python function run per group. Many analysts use that pattern nearly all the time.

```
loyal = orders.groupby("customer_id").filter(lambda g: len(g) >= 25)
print(loyal.shape[0], loyal["customer_id"].nunique())
orders["customer_orders"] = orders.groupby("customer_id")["order_id"].transfor
loyal2 = orders[orders["customer_orders"] >= 25]
print(loyal2.shape[0], loyal2.index.equals(loyal.index))
```

```
496 16
496 True
```

11 **Quiet trap 1: rows lost to a missing key (3 min).** Run the first two lines and ask: "Four groups, 2,148 orders. The table has 20,302. Where did the rest go?" By default `groupby` silently drops every row whose key is missing. `dropna=False` keeps them as their own group. The habit: after a groupby, check that the group sizes add up to `len(df)`. It takes five seconds.

```
by_code = orders.groupby("promo_code")["order_total"].agg(["count", "mean"])
print(by_code["count"].sum(), "of", len(orders))
print(orders.groupby("promo_code", dropna=False)["order_total"].agg(["count",
```

```
2148 of 20302
              count    mean
promo_code
GEARUP5         396  180.80
SUMMIT20        254  148.53
TRAIL15         359  167.28
WELCOME10      1139  177.78
NaN           18154  197.14
```

12 **Quiet trap 2: the average of averages (2 min).** Say: "No error, two different 'average order values'. The first gives tiny Asheville the same weight as the online shop with nine times as many orders." Never average group means unless the groups are the same size: go back to the rows, or carry sums and counts and divide at the end.

```
store_means = orders.groupby("store")["order_total"].mean()
print(store_means.mean().round(2), orders["order_total"].mean().round(2))
print(orders.groupby("store").size().to_dict())
```

```
182.66 194.6
{'Asheville': 1413, 'Bend': 1725, 'Boise': 2594, 'Flagstaff': 1716, 'Online':
```

THREE MISTAKES TO EXPECT

- **Looking for the key among the columns.** Symptom: `KeyError: 'store'` on a groupby result, or a merge or plot later that cannot find the column. Coach: "Print `result.index`. Where is `store` now?" Then let them choose `.reset_index()` or `as_index=False`. A cousin: `orders.groupby("store").mean()` raises `TypeError: dtype 'str' does not support operation 'mean'` because no column was selected.
- `count` **when they meant** `size`**, and rows lost to a missing key.** No error for either. Symptoms: 'orders per payment method' that are ten times too small because `["promo_code"].count()` counted only orders with a promo code; group sizes that add up to 2,148 instead of 20,302. Coach: "Do

your groups add up to the number of rows in the table? Which column did you count, and does it have gaps?" Make 'do the groups add up?' a spoken check after every groupby today.

- **The average of averages.** Symptom: an 'overall average' of 182.66 computed from five store means when the true figure is 194.60. Coach: "How many orders are behind each of those five numbers? Should Asheville count as much as Online?" The cure is to go back to the rows, or to divide a sum of sums by a sum of counts. Expect the same mistake with rates.

GUIDED LAB

- `labs/module-05/l2_4-groupby-operations/store_days_starter.py` (reference: `store_days_solution.py`), 20 min: named aggregation, `size` versus `count`, two keys with `unstack`, a computed year key, `transform`, and the average-of-averages trap on `daily_sales.csv`.
- `labs/module-05/l2_4-groupby-operations/basket_sizes_starter.py` (reference: `basket_sizes_solution.py`), 20 min: choosing the key (per order, per SKU), `agg` versus `transform` versus `filter`, and the transform-then-mask twin on `order_items.csv`.

Also in the folder: `labs/module-05/l2_4-groupby-operations/debug_groupby.py`. A deliberately broken handout: two bugs that raise errors and three quiet ones that print a plausible wrong number. Not run by `check_labs.py`.

- Everyone does `store_days`. Before each TODO, cold-call one learner for the shape: 'How many rows will come back?' (4; 4 with two columns; 4; 2,903.)
- Watch TODO 1: learners who write `days=("revenue", "count")` will get 726 for Asheville and never notice. The two columns `days` and `days_with_revenue` are there to make the difference visible.
- At TODO 2 stop the room for the trap. Ask: 'The two means are 7,212 and 13,950. What is the average store-day?' Take guesses before anyone runs `sales["revenue"].mean()`.
- TODO 3 verifies a claim printed in the lesson project. Say that this is a real analyst's habit: when a document quotes a number, recompute it.
- `basket_sizes` and `debug_groupby.py` are for fast finishers and homework.

START THE LESSON PROJECT

- The lesson project is **The Store Scorecard**: one row per sales channel, then one row per customer (six steps, about 35 minutes solo).
- In the room do Step 1 (helper columns: `returned`, `cancelled`, and `revenue` that counts completed orders only) and Step 2 (the scorecard: one named aggregation with six measures, sorted, plus `revenue_share`).
- Checkpoint: five rows, the `orders` column adds up to 20,302, Online has 65.1% of revenue and a return rate of 6.9%.

- Steps 3 to 6 go home (growth by store, the customer summary, repeat buyers with `transform` and `pd.cut`, saving). Done looks like: completed revenue of $3,624,306.27 and 3,942 customers.
- Ask them to keep `output/customer_summary.csv`: Session 6 attaches names and loyalty tiers to it.

CHECKS FOR UNDERSTANDING

1. Read `orders.groupby("store")["order_total"].sum()` piece by piece. **Answer:** Split the rows into one group per store; from each group take `order_total`; add it up within each group and collect the answers.
2. You want each order's total next to its customer's total spending. `agg` or `transform`? **Answer:** `transform`: one value per original row.
3. What is the difference between `.size()` and `.count()`? **Answer:** `size` counts rows in each group; `count` counts the non-missing values of a column.
4. After a groupby, the group sizes add up to fewer rows than the table has. What happened? **Answer:** Rows with a missing key were dropped. Use `dropna=False`.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** D) `(2, 2) 20.0`; **Q8** `[0.25, 0.75, 1.0]`; **Q9** A) `orders.groupby("channel")["returned"].mean()`; **Q10** C) Rows whose key is missing are dropped by default; `dropna=False` keeps them as a group; **Q11** `[2, 1] [1, 1]`; **Q12** B) `orders.groupby("customer_id").filter(lambda g: len(g) >= 25)`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Have learners who are unsure say each groupby as a sentence before typing it: "For each ___, take ___, and compute ___." Give them the named-aggregation skeleton on a card: `df.groupby(KEY).agg(NEW=(COLUMN, FUNCTION), ...)`. For TODO 4 draw two tables on the board: five rows (agg) and 2,903 rows with a repeated number (transform).

Stretch: `basket_sizes`, then `debug_groupby.py`. Further: the lesson's bonus challenge (each customer's first order with `transform("min")`: what share of first orders used WELCOME10?), or ask them to redo `store_days` TODO 2 by weekday instead of promo day, in calendar order (group by `dt.dayofweek` and `dt.day_name()` together).

CLOSING

2:50 Wrap-up — 10 min

EXIT TICKET (3 QUESTIONS, ON PAPER OR STICKY NOTES, 5 MINUTES)

1. In what order do you repair a damaged table, and why does structure come first? **Answer:** Structure (headers, rows that are not data, duplicates), then text, numbers, dates, missing values. Fixing the shape first makes later steps simpler and counts honest (for example `astype` fails while a TOTAL row is present).
2. Forty of 20,302 order totals are blank and nothing in the file gives their value. You need the average order value. What do you do, and what do you say? **Answer:** Leave them missing, take the mean of the known totals, and report that 40 orders were excluded. Filling with 0 or with the median would invent data.
3. One row per group; one value per original row; fewer of the same rows. Name the groupby tool for each. **Answer:** `agg` (or a built-in such as `sum`); `transform`; `filter` (or a `transform` and a mask).

HOMEWORK (ABOUT 2 TO 3 HOURS BEFORE SESSION 6)

- Finish **The Cleaning Pipeline** (2.3, Steps 3 to 6): `clean_orders()` runs end to end, `validate()` passes, 0 wrong cells and 623 still blank. Keep `output/orders_clean.parquet`.
- Finish **The Store Scorecard** (2.4, Steps 3 to 6): completed revenue $3,624,306.27, 3,942 customers. Keep `output/customer_summary.csv`.
- Each lesson's exercises (2.3: 1 to 3, 4 is a stretch; 2.4: 1 to 4, 5 is a stretch), Quick Quiz and journal entry.
- Labs not finished in class; `total_row_check`, `basket_sizes` and `debug_groupby.py` are for home.
- Lesson 2.4, 'Before the Next Lesson': how many distinct `customer_id` values are in `orders.csv`, and how many rows are in `customers.csv`? Which is larger, and what might that mean? Session 6 starts there.

BRING LIST FOR SESSION 6

- Laptop with the course environment. Nothing new to install.
- The course `data/` folder as it is. Session 6 uses `orders.csv`, `order_items.csv`, `products.csv`, `customers.csv`, `daily_sales.csv`, `store_weather.csv`, `survey.csv` and `sales_2025.xlsx`.
- Your `output/customer_summary.csv` from the 2.4 project (a merge in Session 6 can attach names and loyalty tiers to it), and your answer to the customer-count question.

- This workbook and your notebooks. No servers, accounts or internet connection are needed for Session 6.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT, WHAT TO CUT FIRST

Protect, in this order: the audit (2.3 steps 4 and 5), the outlier-with-evidence step (step 12), the `register_log` lab, named aggregation and `transform` in 2.4 (steps 4 and 9), and the two quiet traps (steps 11 and 12). Cut first: the status half of 2.3 step 8, then step 15 (assertions: they are Step 5 of the project), then 2.4 step 8 (the computed key: it is Step 3 of the project) and step 10 (`filter`). If the 2.3 block overruns by more than ten minutes, start the 2.4 project by reading the brief aloud only, and assign Steps 1 and 2 as homework with the checkpoint numbers.

RUNNING SHORT: WHAT TO ADD

Run TODO 6 of `register_log` with the whole room (recover the pending day from the TOTAL row) and discuss why that is a repair and not a guess. Then `debug_groupby.py` in pairs: the three quiet bugs make a good five-minute discussion. Still time: do Step 3 of the Store Scorecard together and ask which table a growth figure comes from.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 6: Merging and Reshaping

How to teach the session where tables are connected and reshaped. First learners bring customers, orders, order lines and products together by their keys, and learn to notice when a merge quietly loses rows or multiplies them. Then they turn long tables into grids a person can read with `pivot_table` and `crosstab`, turn grids back into rows with `melt`, and learn to read a tidy-looking table with suspicion.

`2 lessons` `3 hrs session` `Lessons 2.5–2.6` `Session 6 of 21`

HOW TO USE THIS GUIDE

- Each lesson block has four beats: **frame** (the mental model, said aloud), **live demo** (you type, they follow), **guided lab** (new practice from `labs/module-06/`) and **start the lesson project** (the first three steps together). The rest of each project, the lessons' exercises, quizzes and journals are homework.
- The lessons are timed at 95 and 85 minutes for a solo learner; the room has 75 for each. The demo scripts are compressed paths through the lesson pages on the lessons' own tables, so every number on the projector matches the page (pandas 3.0.6).
- **Type, do not paste.** In 2.5, ask for three predictions before every merge: the left table, the relationship, the row count. In 2.6, have the room say the four pivot questions as a sentence before every pivot.
- Keep `pandas_merging_joining.html` and `pandas_pivot_tables.html` open for the diagrams and reference tables (the map of the four tables, the four joins, the relationships table, the which-tool-when chart, the six questions before you trust a pivot).
- Starters go to learners; every `*_solution.py` stays with you. `debug_merges.py` is broken on purpose.

> **THE #1 RISK OF SESSION 6: A MERGE OR A PIVOT THAT RUNS AND IS WRONG**
>
> Every serious mistake today is silent. A default inner join drops 739 orders. A one-to-many merge turns $4.0 million into $11.0 million. A pivot without `aggfunc` shows averages under a title that says revenue. A pivot quietly leaves out $34,228 of sales because their category is missing. Two reconciliation habits cover all four, and you should say them until the room says them back: **"Count before, count after"** for every merge, and **"Does the corner match the source?"** for every pivot. The second risk is cognitive load in 2.5: do not teach `merge_asof` in the room. One sentence, and point at the lesson.

BEFORE THE SESSION

- ☐ Copy the **starter** files and `debug_merges.py` from `labs/module-06/` to the shared folder or each machine, keeping the folder layout. Keep every `*_solution.py` on your machine.
- ☐ From the kit folder run `python3 labs/check_labs.py module-06`. All 10 lines should say PASS.
- ☐ No servers to start and no internet needed. Files used today: `orders.csv`, `customers.csv`, `order_items.csv`, `products.csv`, `daily_sales.csv`, `store_weather.csv`, `survey.csv`, `sales_2025.xlsx`.
- ☐ Environment traps: `read_excel` needs `openpyxl` (2.6 demo step 10). Learners on an older pandas will see `stack()` drop empty cells and accept a `dropna=` argument; in the course environment it keeps them. Check `pd.__version__` prints 3.0.6 on any machine that behaves differently.
- ☐ Projector: a notebook called `session06` opened in the project folder, font at least 18 pt, the display options in the first cell, and the four core tables loaded before the session starts (the load takes a few seconds).
- ☐ Whiteboard, before learners arrive: the map **customers → orders → order_items ← products** with the key on each arrow; **"Count before. Count after."**; and, for after the break, **rows? columns? values? how to combine?**
- ☐ Skim the Session 5 homework: the Cleaning Pipeline scorecards (0 wrong, 623 blank) and the Store Scorecard, and each learner's answer to the customer-count question from Lesson 2.4 (3,942 distinct ids in `orders.csv`, 4,850 rows in `customers.csv`). That question opens the 2.5 block.
- ☐ Sticky notes for the exit ticket. Print the workbook for Session 6.

SESSION 6 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 5 (cleaning, groupby), homework check, the customer-count question.
0:10	10	**2.5** Frame and mental model	Why data lives in several tables; the claim ticket; the map of the four tables.
0:20	30	**2.5** Live demo	Four joins, the 739 lost orders, `indicator`, `validate`, rows that multiply, awkward keys, `concat` (two quiet mistakes, two loud ones).
0:50	25	**2.5** Guided lab	`labs/module-06/l2_5-merging-joining/customer_profiles_starter.py`: aggregate, then merge.
1:15	10	**2.5** Start the lesson project	Revenue by Category, Steps 1 to 3 together; the rest goes home.
1:25	10	Break	Leave the `_merge` counts on the projector.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:35	10	**2.6** Frame and mental model	The logbook and the scoreboard; the four pivot questions.
1:45	30	**2.6** Live demo	First pivot, the default-mean trap, totals and empty cells, `crosstab`, `pivot` versus `pivot_table`, `melt`, month order, a total that hides the story.
2:15	25	**2.6** Guided lab	`labs/module-06/l2_6-pivot-tables/payment_returns_starter.py`, then `survey_by_age_starter.py`.
2:40	10	**2.6** Start the lesson project	The Seasonality Report, Steps 1 to 3 together; the rest goes home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 7.
Total	180		Ends at 3:00.

In the room: both mental models, both live demos, the guided labs `customer_profiles` and `payment_returns` for everyone (`survey_by_age` for pairs that finish), and the first three steps of each lesson project. **Goes home:** the rest of both projects (2.5 Steps 4 to 7, 2.6 Steps 4 to 6), the lessons' exercises, both Quick Quizzes and journals, and the labs `orders_weather`, `weather_grid` and `debug_merges.py`: about 2 to 3 hours.

Protect: in 2.5 the lost-rows step, `indicator`, and the double-counting step; in 2.6 the default-mean trap, `normalize` in both directions, `melt`, and the footwear table; both guided labs. **Cut first if running long:** `concat` and the `merge_asof` sentence in 2.5 (reading), then the margins-of-means step and the reshaping-reveals-holes step in 2.6, then the second lab of the 2.6 block.

WARM-UP

0:00 Warm-up: Retrieval from Session 5 — 10 min

FIVE RETRIEVAL QUESTIONS (NO NOTES, 5 MINUTES)

Slide 3. Paper only, notebooks closed, then one minute with a neighbor before you take answers.

1. In what order do you repair a damaged table? **Answer:** Structure (headers, stray rows, duplicates), then text, numbers, dates, and last the missing values.
2. What does `pd.to_numeric(s, errors="coerce")` do with the value `free`, and why is that useful? **Answer:** It turns it into NaN instead of raising an error, so it works as a detector: values that became NaN but were not empty are the ones to look at.
3. `orders.dropna()` kept 2,085 of 20,302 rows. Why, and what should you write? **Answer:** It drops every row with a gap in any column, and most orders have no promo code. Name the columns that matter: `dropna(subset=[...])`.
4. You need each order's total next to its store's average order. Which groupby tool gives one value per original row? **Answer:** `transform` (for example `orders.groupby("store")["order_total"].transform("mean")`).
5. Grouped by `promo_code`, the group sizes add up to 2,148, but there are 20,302 orders. What happened? **Answer:** Rows whose key is missing are dropped by default. `dropna=False` keeps them as a group.

HOMEWORK CHECK (3 MINUTES)

- Show of hands: whose cleaning scorecard said 0 wrong cells and 623 still blank? Whose Store Scorecard totals $3,624,306.27?
- Ask for the answer to the question from Lesson 2.4: how many distinct customer ids are in `orders.csv` (3,942), and how many rows in `customers.csv` (4,850)? Which is larger, and what might that mean?
- Take two or three guesses about what it means and write them on the board without comment.

BRIDGE TO TODAY (2 MINUTES)

Say: "4,850 is larger than 3,942, so some account holders never ordered. True. But there is something the two counts cannot tell you: 150 of those 3,942 ids are not in the customers file at all. You can only see that by putting the two tables side by side, key by key. That is a merge, and it is the next 75 minutes."

2.5 Merging and Joining

75 min

OBJECTIVES (THE LESSON'S OWN WORDS)

- Combine two tables on a key with `merge()` and choose between inner, left, right and outer joins
- Find the keys that have no match on either side with `indicator=True`, and decide what to do about them
- State the relationship between two tables (one-to-one, many-to-one, many-to-many) and enforce it with `validate=`
- Predict and check the row count of a merge, and avoid double counting after a one-to-many merge
- Merge on differently named or multiple keys, stack tables with `concat()`, and recognize when `merge_asof()` is the right tool

TIMING INSIDE THIS BLOCK

10 min	Frame: why data lives in several tables; the claim ticket; the map of the four tables
30 min	Live demo: four joins on toy tables, the 739 lost orders, `indicator`, `validate`, rows that multiply, awkward keys, `concat`
25 min	Guided lab: `customer_profiles` (aggregate, then merge)
10 min	Start the lesson project (Revenue by Category, Steps 1 to 3)

MENTAL MODEL TO SAY

Say: "Leave skis for tuning at a Pinecrest store and you get a numbered ticket. The same number is tied to the skis. Nobody writes your name on the skis: the number connects the two. A key is that ticket number, a column that appears in both tables. A merge walks down one table and, for each row, finds the rows in the other table holding the same ticket. Two questions decide everything that follows: what if a ticket has no match, and what if a ticket matches more than once?" Draw the map from the lesson on the board and leave it up: **customers →(customer_id)→ orders →(order_id)→ order_items ←(sku)← products**. Read each arrow as "one row here can match many rows there". Under it write the habit for the block: **"Count before. Count after."**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Type in a notebook called `session06` and load the four core tables first (`orders`, `customers`, `products`, `items`, as on the lesson page). Before **every** merge, ask the room three things and wait for answers: which table is on the left, what relationship do you expect, how many rows should come out? Steps

3 and 7 are the quiet kind of mistake (no error, a wrong number); steps 6 and 9 are deliberate mistakes that raise errors.

1. **Two tiny tables you can see whole (3 min).** Say: "`left` has keys A, B, C. `right` has B, C, D. So B and C match, A is only on the left, D only on the right." Run the default merge: only B and C. "By default `merge` keeps only keys found in both tables. That default is called an inner join."

```
left = pd.DataFrame({"sku": ["A", "B", "C"], "qty": [2, 1, 5]})
right = pd.DataFrame({"sku": ["B", "C", "D"], "price": [10.0, 20.0, 30.0]})
print(left.merge(right, on="sku"))
```

```
  sku  qty  price
0   B    1   10.0
1   C    5   20.0
```

2. **how= decides what happens to keys with no match (3 min).** Ask the room to call out the keys before each result. Left: A, B, C. Outer: A, B, C, D. "Where a row has no partner, the borrowed columns are NaN. And notice `qty` turned into floats in the outer result: an integer column cannot hold a NaN." The working default for analysis is a left join: take my main table and attach some columns.

```
print(left.merge(right, on="sku", how="left"))
print(left.merge(right, on="sku", how="outer"))
```

```
  sku  qty  price
0   A    2    NaN
1   B    1   10.0
2   C    5   20.0
  sku  qty  price
0   A  2.0    NaN
1   B  1.0   10.0
2   C  5.0   20.0
3   D  NaN   30.0
```

3. **A real merge, and the rows it lost (4 min).** Attach each customer's name, state and tier to the orders, with the default settings. Show the head: "It looks perfect." Then: "Now do the one thing you should do after every merge for the rest of your career: count the rows." 739 orders are gone. No error, no warning. Any revenue total computed from `merged` would be short, and nothing on screen would say so.

```
slim = customers[["customer_id", "first_name", "state", "loyalty_tier"]]
merged = orders.merge(slim, on="customer_id")
print(merged[["order_id", "customer_id", "order_total", "first_name", "loyalty
print(len(orders), "orders before,", len(merged), "rows after,", len(orders) -
```

```
   order_id  customer_id  order_total first_name loyalty_tier
0    100001        12673        434.9     Xavier       Silver
1    100002        10034        278.9       Aiko         Gold
20302 orders before, 19563 rows after, 739 lost
```

4 **The repair: a left join (2 min).** Say: "With `how=\"left\"` all 20,302 orders survive, and the 739 without a customer record have NaN in the borrowed columns: missing matches you can see and count, instead of rows that silently vanish." Then the four kinds on the real tables: inner smallest, outer largest, and the differences are exactly the unmatched rows on each side.

```
with_names = orders.merge(slim, on="customer_id", how="left")
print(len(with_names), with_names["first_name"].isna().sum())
for how in ["inner", "left", "right", "outer"]:
    print(how, len(orders.merge(slim, on="customer_id", how=how)))
```

```
20302 739
inner 19563
left 20302
right 20621
outer 21360
```

5 **indicator=True: unmatched rows are findings (4 min).** Say: "`indicator=True` adds a column `_merge` that says where each row came from. With an outer join it is a complete audit of how two tables overlap." The 739 orphan orders belong to 150 customer ids and are worth $154,274: the README says they are closed accounts whose records were removed. Real sales. And 1,058 people have an account and no order. "Each group of unmatched keys has a cause. Find it, then decide. Never let the default decide for you."

```
audit = orders.merge(slim, on="customer_id", how="outer", indicator=True)
print(audit["_merge"].value_counts())
orphans = audit[audit["_merge"] == "left_only"]
print(len(orphans), orphans["customer_id"].nunique(), round(orphans["order_tot
```

```
_merge
both          19563
right_only     1058
left_only       739
Name: count, dtype: int64
739 150 154273.8
```

6 **Relationships, validate=, and a deliberate mistake (4 min).** Ask: "Is `customer_id` unique in `customers`? In `orders`?" From the orders side this is many-to-one: each order finds at most one customer, so the row count cannot grow. "`validate=` makes pandas check your belief." It passes. Then state something untrue, `one_to_one`, and read the `MergeError` aloud: it names the problem. "A few characters that turn a silent wrong answer into a loud error on the day somebody hands you a customers file with a duplicated id."

```
print(customers["customer_id"].is_unique, orders["customer_id"].is_unique)
checked = orders.merge(slim, on="customer_id", how="left", validate="many_to_o
print(len(checked))
orders.merge(slim, on="customer_id", how="left", validate="one_to_one")
```

```
True False
20302
MergeError: Merge keys are not unique in left dataset; not a one-to-one merge
Duplicates in left:
  customer_id
       12338
       11061
       13819
       12491
       13352 ...
```

7 **One-to-many: correct, and easy to misuse (4 min).** Ask for the row count first: one order matches several lines, so the result has one row per line, 45,490. Look at order 100001: its total of 434.90 now appears twice. "Nothing is wrong yet. It goes wrong the moment you add up an order-level column in the merged table." $11.0 million instead of $4.0 million. "The most expensive merge mistake in practice, because the number looks plausible and the code raises nothing."

```
lines = orders[["order_id", "status", "order_total"]].merge(items, on="order_i
                                                            validate="one_to_ma
print(len(orders), "->", len(lines))
print(lines[lines["order_id"] == 100001])
print(round(orders["order_total"].sum(), 2), round(lines["order_total"].sum(),
```

```
20302 -> 45490
   order_id     status  order_total  order_item_id       sku  quantity  unit_p
0    100001  completed        434.9              1  SLP-1021         1      3
1    100001  completed        434.9              2  KIT-1014         1
3950701.45 11039208.35
```

8 **Two honest ways out (2 min).** Say: "After every merge, finish the sentence: one row of this table is one ... If the answer is one order line, then order-level columns are repeated, and summing or averaging them is wrong." Either go back to one row per order before summing an order-level column, or sum a line-level quantity. (The line totals come to a little more than the order totals because order totals include discounts and shipping.)

```
per_order = lines.drop_duplicates("order_id")
print(round(per_order["order_total"].sum(), 2))
lines["line_total"] = lines["quantity"] * lines["unit_price"]
print(round(lines["line_total"].sum(), 2))
```

```
3950701.45
3963530.35
```

9 **Awkward keys: two columns, different names (4 min).** Daily sales and weather describe the same thing, a store on a day, but one says `store` and the other `city`, and the key is the pair. Type the natural first attempt and read the `KeyError`: "`on=` needs the same names in both tables." Then `left_on` and `right_on`, in matching order, validated one-to-one: 2,903 in, 2,903 out. An outer join with an indicator shows what the 21 extra weather rows are: store-days with no sales row.

```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
sw = daily.merge(weather, left_on=["date", "store"], right_on=["date", "city"]
                 how="left", validate="one_to_one")
print(len(daily), len(weather), len(sw))
both = daily.merge(weather, left_on=["date", "store"], right_on=["date", "city
                   how="outer", indicator=True)
print(both["_merge"].value_counts().to_dict())
daily.merge(weather, on=["date", "store"])
```

```
2903 2924 2903
{'both': 2903, 'right_only': 21, 'left_only': 0}
KeyError: 'store'
```

10 **Stacking with concat, and a word about merge_asof (3 min).** Say: "`merge` puts tables side by side, matching rows by key. `concat` puts tables on top of each other: same columns, more rows. It is what you use when data arrives in pieces, one file per month." `ignore_index=True` renumbers the rows. Then one sentence only on `merge_asof`: "When there is no equal key, only a nearest earlier one, for example a customer's most recent order before a website visit, `pd.merge_asof` is the tool. The lesson has a worked example; you need to recognize the situation, not memorize the call."

```
y2024 = orders[orders["order_date"].dt.year == 2024]
y2025 = orders[orders["order_date"].dt.year == 2025]
stacked = pd.concat([y2024, y2025], ignore_index=True)
print(len(y2024), len(y2025), stacked.shape, stacked.equals(orders))
```

```
9519 10783 (20302, 11) True
```

THREE MISTAKES TO EXPECT

- **An inner join used as an accidental filter.** No error. Symptom: a total that is a few percent short (19,563 rows instead of 20,302), found weeks later or never. Coach: "How many rows did your main table have before the merge? How many after? Which rows went, and why?" The cure is two habits: `how="left"` when you are adding columns to a main table, and a row count after every merge. Then `indicator=True` to look at what did not match.
- **Summing or averaging a repeated column after a one-to-many merge.** No error. Symptoms: order totals that sum to $11.0 million instead of $4.0 million; an 'average order total' of $242.67 instead of $194.60. Coach: "Finish the sentence: one row of this table is one ___. Is `order_total` a fact about a line or about an order?" Have them aggregate each table to the level they need before merging, or total only columns that belong to the row's own level.
- **Keys that do not line up.** These raise errors, which is the easy kind. `ValueError: You are trying to merge on str and int64 columns` (convert one side; ignore the hint about `pd.concat`). `KeyError: 'store'` (the other table calls it `city`: `left_on` / `right_on`). `MergeError: Merge keys are not unique in left dataset` (the `validate=` claim names the wrong side). Coach: "Print the dtype and three values of the key on each side." And one quiet relative: missing keys match each other in pandas, so drop or fill NaN keys before merging on a column that has them.

GUIDED LAB

- `labs/module-06/l2_5-merging-joining/customer_profiles_starter.py` (reference: `customer_profiles_solution.py`), 25 min: aggregate first, then merge; `how="left"`, `validate="one_to_one"`, `indicator=True`; an honest `fillna(0)`; an outer-join audit of both sides.

- `labs/module-06/l2_5-merging-joining/orders_weather_starter.py` (reference: `orders_weather_solution.py`), 20 min: a two-column key with different names (`left_on` / `right_on`), choosing the left table so that days with no orders are kept, `validate="one_to_one"`, `indicator`, then a groupby.

Also in the folder: `labs/module-06/l2_5-merging-joining/debug_merges.py`. A deliberately broken handout: three merges that raise errors and two that run and give a wrong number. Not run by `check_labs.py`.

- Everyone does `customer_profiles`. Before TODO 2 make every learner fill in the prediction line in the file: left table, relationship, rows after. Ask three learners to read theirs out before anyone runs it.
- Circulate at TODO 3. Ask: 'Why is filling with 0 honest here, when filling a missing order total with 0 was wrong in Session 5?' (No orders really is 0 orders and $0 of spend; a missing total is an unknown amount.)
- At minute 18 ask for the three tier percentages. Then ask why Gold and Silver are exactly 0: it is a property of how tiers are defined, not a discovery.
- TODO 5 is the audit of the other side. If time is short, run it on the projector: the 150 ids and $154,273.80 connect straight back to the demo.
- `orders_weather` and `debug_merges.py` are for fast finishers and homework.

START THE LESSON PROJECT

- The lesson project is **Revenue by Category**: three tables joined, every unmatched row accounted for, and a pricing trap (seven steps, about 40 minutes solo).
- In the room do Step 1 (inspect the keys: 609 item lines with no product, 8 products never sold), Step 2 (lines + orders, many-to-one, `45490 -> 45490`) and Step 3 (+ products with an indicator: 609 `left_only`).
- Stop there and ask: 'Those 609 lines are real sales with no category. What would an inner join have done with them?' That question is the homework's Step 4.
- Steps 4 to 7 go home. Done looks like: completed merchandise revenue of $3,715,829.95, Accessories on top with 16.0%, and one sentence explaining why list-price revenue is 1.7% too high.
- Ask them to keep `output/order_lines.parquet`: it is raw material for the pivots after the break.

CHECKS FOR UNDERSTANDING

1. What does the default `merge` do with a key that has no match on the other side? **Answer:** It drops the row (an inner join). No error, no warning.
2. You are adding customer columns to `orders`. Which `how=`, and what should the row count be afterward? **Answer:** `how="left"`, and still 20,302 rows.

3. What does `validate="many_to_one"` check? **Answer:** That the key is unique in the right-hand table. It does not check that every row finds a match.

4. After `orders.merge(items, on="order_id")`, what is one row, and which columns must you not sum? **Answer:** One order line. Order-level columns such as `order_total` are repeated on every line of the order.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** C) `3 4 5`; **Q2** B) `orders.merge(customers, on="customer_id", how="left", validate="many_to_one")`; **Q3** `350 150`; **Q4** A) Orders whose `customer_id` has no row in `customers`; **Q5** `(2, 3) 1`; **Q6** D) `daily.merge(weather, left_on=["date", "store"], right_on=["date", "city"], how="left", validate="one_to_one")`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: Keep the four-join table from the lesson on the projector during the lab. For learners who are unsure which table goes on the left, give them one sentence to complete: "I want every row of _____ to survive." If someone's merge multiplies rows, have them print `key.is_unique` for both tables before touching the merge again.

Stretch: `orders_weather` (the choice of left table changes the answer), then `debug_merges.py`. Further: the lesson's Exercise 5 (completed revenue by tier with closed accounts as their own group, reconciling to $3,624,306.27 to the cent).

2.6 Pivot Tables and Reshaping

75 min

OBJECTIVES (THE LESSON'S OWN WORDS)

- Build a summary grid with `pivot_table()` by choosing its rows, columns, values and aggregation, and add totals with `margins=True`
- Count and compare categories with `pd.crosstab()`, including row and column percentages
- Choose between `pivot()` and `pivot_table()`, and explain the "duplicate entries" error
- Recognize wide and tidy (long) layouts and convert between them with `melt()`, `stack()` and `unstack()`
- Check a pivot before you trust it: its aggregation, its cell counts, its empty cells and its totals

TIMING INSIDE THIS BLOCK

10 min	Frame: the logbook and the scoreboard; the four pivot questions
30 min	Live demo: first pivot, the default-mean trap, empty cells and totals, `crosstab` with `normalize`, `pivot` versus `pivot_table`, `melt`, month order, reading a pivot honestly
25 min	Guided lab: `payment_returns`, then `survey_by_age`
10 min	Start the lesson project (The Seasonality Report, Steps 1 to 3)

MENTAL MODEL TO SAY

Say: "A logbook records one event per line: this order, on this date, at this store, for this amount. It is long and narrow, it grows by adding lines, and it is easy for a program to filter, group and merge. That layout is called long, or tidy. A scoreboard arranges a summary in a grid so a person can compare across a row or down a column at a glance. It is short and wide, and it is what goes into a report. Analysis happens in the logbook. Communication happens on the scoreboard. `pivot_table` turns a logbook into a scoreboard, and `melt` turns a scoreboard back into a logbook." Write the four pivot questions on the board: **rows? columns? values? how to combine?** Learners say all four aloud before they write any pivot today.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New section of the notebook; `orders` is already loaded. Have the room say the four pivot questions as a sentence before each pivot ("for each store, by year, the sum of order totals"). Step 3 is the quiet mistake of this lesson (a wrong table with no error); step 8 is a deliberate mistake with an error message that is worth reading slowly.

1. **The same ten numbers, as a logbook and as a grid (3 min).** Start with what they know: a groupby on two keys gives ten lines, one per store and year. "Correct, and still a logbook: to compare 2024 with 2025 your eyes have to hop between lines." Then the pivot. "Same ten numbers, arranged so each store's two

years sit side by side. A pivot table is a groupby whose result has been spread into a grid. Nothing new is being calculated."

```
done = orders[orders["status"] == "completed"].copy()
done["year"] = done["order_date"].dt.year
print(done.groupby(["store", "year"])["order_total"].sum().head(4))
table = done.pivot_table(index="store", columns="year", values="order_total",
print(table)
```

```
store      year
Asheville  2024    127336.98
           2025    110442.39
Bend       2024    136989.18
           2025    147586.07
Name: order_total, dtype: float64
year              2024        2025
store
Asheville   127336.98   110442.39
Bend        136989.18   147586.07
Boise       212140.12   231246.73
Flagstaff   129032.48   171547.87
Online     1077800.75  1280183.70
```

2 **The result is an ordinary DataFrame (2 min).** Say: "The grid is not a picture. It has an index and columns, and everything you know works on it." With the years side by side, year-over-year change is one line of column arithmetic: that is the reason to go wide. Remind them which table this is: `orders.csv` holds account-holder purchases, so Asheville's -13.3% here is -5.0% in the register totals.

```
print(table.loc["Boise", 2025])
table["growth_pct"] = ((table[2025] / table[2024] - 1) * 100).round(1)
print(table.sort_values("growth_pct", ascending=False))
```

```
231246.73
year              2024        2025  growth_pct
store
Flagstaff   129032.48   171547.87        32.9
Online     1077800.75  1280183.70        18.8
Boise       212140.12   231246.73         9.0
Bend        136989.18   147586.07         7.7
Asheville   127336.98   110442.39       -13.3
```

3 **The quiet mistake: leave out aggfunc (3 min).** Delete `aggfunc` and run it. Ask: "Is this revenue?" No error, and a completely different table: about $170 to $215 per cell. "The default is the mean. This surprises everyone who comes from spreadsheets, where a pivot sums." The habit: always write `aggfunc=`, even when you want the mean, and check one cell by hand.

```
print(done.pivot_table(index="store", columns="year", values="order_total").ro
check = done[(done["store"] == "Boise") & (done["year"] == 2025)]["order_total
print(round(check.sum(), 2), round(check.mean(), 2))
```

```
year          2024    2025
store
Asheville   181.13  168.61
Bend        168.08  176.75
Boise       179.48  175.19
Flagstaff   176.76  186.87
Online      201.01  205.09
231246.73 175.19
```

4 **Empty cells and totals (4 min).** How do customers pay at each store? One cell is NaN: nobody hands cash to a website. "A pivot shows NaN wherever a combination never occurs, and that one NaN turned every count into a float." For a count, 'no rows' honestly means zero, so `fill_value=0`; and `margins=True` adds totals. Point at the corner: 20,302, the number of rows in `orders`. "A free sanity check. If the grand total of a count pivot is not the number of rows you started with, rows fell out." Then the caution: `fill_value=0` is right for counts and sums and wrong for averages.

```
pay = orders.pivot_table(index="payment_method", columns="store", values="orde
print(pay)
pay = orders.pivot_table(index="payment_method", columns="store", values="orde
                         aggfunc="count", fill_value=0, margins=True, margins_
print(pay)
```

```
store           Asheville    Bend   Boise  Flagstaff  Online
payment_method
card                835.0  1050.0  1533.0     1021.0  7953.0
cash                293.0   370.0   577.0      374.0     NaN
digital_wallet      181.0   203.0   325.0      209.0  3876.0
gift_card           104.0   102.0   159.0      112.0  1025.0
store           Asheville  Bend  Boise  Flagstaff  Online  Total
payment_method
card                  835  1050   1533       1021    7953  12392
cash                  293   370    577        374       0   1614
digital_wallet        181   203    325        209    3876   4794
gift_card             104   102    159        112    1025   1502
Total                1413  1725   2594       1716   12854  20302
```

5 **Totals of averages are not averages of totals (2 min).** Say: "Margins use the same `aggfunc` as the cells, applied to all the underlying rows." The `All` corner is the mean of all 18,762 completed orders, 193.17. It is not the average of the five store means above it, 182.07: Online has far more orders and pulls the overall mean toward its own. "pandas gets this right. People reading the table often get it wrong, so it is worth a sentence in your report."

```
aov = done.pivot_table(index="store", columns="year", values="order_total",
                       aggfunc="mean", margins=True).round(2)
print(aov)
print(round(done.groupby("store")["order_total"].mean().mean(), 2))
```

```
year          2024    2025     All
store
Asheville  181.13  168.61  175.10
Bend       168.08  176.75  172.47
Boise      179.48  175.19  177.21
Flagstaff  176.76  186.87  182.39
Online     201.01  205.09  203.20
All        191.46  194.68  193.17
182.07
```

6 **Counting with crosstab, and row percentages (3 min).** Say: "'How many of each combination?' is so common it has its own function. `crosstab` takes two Series and counts." Raw counts are hard to compare because online has almost twice as many orders. `normalize=\"index\"` makes each row add up to 100%: of online orders, 6.9% were returned; of store orders, 3.4%.

```
print(pd.crosstab(orders["channel"], orders["status"], margins=True))
pct = pd.crosstab(orders["channel"], orders["status"], normalize="index") * 10
print(pct.round(1))
```

```
status   cancelled  completed  returned    All
channel
online         360      11604       890  12854
store           37       7158       253   7448
All            397      18762      1143  20302
status   cancelled  completed  returned
channel
online         2.8       90.3       6.9
store          0.5       96.1       3.4
```

7 **The other direction answers a different question (2 min).** Run it and read the returned column: "77.9% of returns are online. Dramatic? 63.3% of all orders are online, so most of that number is just size." Row percentages compare like with like. "Of online orders, what share were returned?" and "Of returned orders, what share were online?" are different questions, and mixing them up is one of the commonest ways to misread a table.

```
by_col = pd.crosstab(orders["channel"], orders["status"], normalize="columns")
print(by_col.round(1))
print(round((orders["channel"] == "online").mean() * 100, 1))
```

```
status   cancelled  completed  returned
channel
online        90.7       61.8      77.9
store          9.3       38.2      22.1
63.3
```

8 **pivot only rearranges, and a deliberate mistake (4 min).** Say: "`pivot`, with no `_table`, does no arithmetic. It needs each row-and-column pair to appear at most once." `daily_sales.csv` fits: one row per store per day. 2,903 rows become 727 rows and 4 columns, and not one number changed. Then try `pivot` on the orders and read the error slowly: "Thousands of orders share a store and a status, and `pivot` has no idea which one belongs in the cell. If you believed the pairs were unique, the error has just told you something about your data."

```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
wide = daily.pivot(index="date", columns="store", values="revenue")
print(wide.shape)
print(wide.head(3))
orders.pivot(index="store", columns="status", values="order_total")
```

```
(727, 4)
store       Asheville     Bend    Boise  Flagstaff
date
2024-01-01    5017.07  4132.92  5962.00    3061.66
2024-01-02    3605.81  3675.57  6583.13    3575.24
2024-01-03    4265.55  6275.05  3866.59    3974.93
ValueError: Index contains duplicate entries, cannot reshape
```

9 **Reshaping reveals holes (2 min).** Ask: "The long table has 6 missing revenue values. How many empty cells does the wide table have?" Eleven. The extra five were never NaN in the file: they are store-days with no row at all. "In a logbook a missing line is invisible. On a grid it is an empty square you can see." Session 7 picks up exactly these gaps.

```
print(daily["revenue"].isna().sum(), wide.isna().sum().to_dict())
```

```
6 {'Asheville': 1, 'Bend': 6, 'Boise': 1, 'Flagstaff': 3}
```

10 **Wide to tidy with melt (3 min).** The finance team's sheet reads nicely, and "total revenue per month" already means naming five columns one by one. "Boise, Bend and the rest are not five variables. They are five values of one variable that have been spread across the header. Test: if a column header is a piece of data, the table is wide." `melt`: say which columns identify a row (`id_vars`), and the rest are stacked into two new columns. 12 rows times 5 store columns: 60 rows.

```
sheet = pd.read_excel("data/sales_2025.xlsx", sheet_name="Monthly Sales", skip
print(sheet.head(2))
tidy = sheet.melt(id_vars="Month", var_name="store", value_name="revenue")
print(tidy.shape)
print(tidy.head(2))
print(tidy.groupby("store")["revenue"].sum().sort_values(ascending=False).head
```

```
  Month   Boise    Bend  Flagstaff  Asheville  Online
0   Jan  242847  169817     160348     147792   86269
1   Feb  218456  146918     154777     127039   86043
(60, 3)
  Month  store  revenue
0   Jan  Boise   242847
1   Feb  Boise   218456
store
Boise        3846407
Flagstaff    2717996
Name: revenue, dtype: int64
```

11 **And back again: the month-order trap (2 min).** Rebuild the grid with `pivot` and look at the row order: Apr, Aug, Dec, Feb. "Both pivot methods sort their labels, and text sorts alphabetically. Month names and weekday names are scrambled by this every time, with no error, and a careless reader sees a trend that is only the alphabet." Fix it with `reindex`, or pivot on the month number.

```
back = tidy.pivot(index="Month", columns="store", values="revenue")
print(back.index.tolist()[:4])
back = back.reindex(index=sheet["Month"], columns=["Boise", "Bend", "Flagstaff
print(back.head(2))
```

```
['Apr', 'Aug', 'Dec', 'Feb']
store   Boise    Bend  Flagstaff  Asheville  Online
Month
Jan    242847  169817     160348     147792   86269
Feb    218456  146918     154777     127039   86043
```

12 **Reading a pivot honestly: a total that hides the story (3 min).** Close with the lesson's best table. Flag the orders that contain footwear, then pivot the return rate. Cover the inside with your hand and read only the `All` column: "footwear orders are returned 10.7% of the time against 4.2%". Now uncover it: online, footwear triples the return rate (14.4% against 4.8%); in stores it makes no real difference. "People can try shoes on in a store. The margin averaged two very different situations into one misleading number." Then name the six questions from the lesson page: one row of the source, the aggregation, the count behind each cell, what empty cells mean, whether totals reconcile, whether a total hides a difference.

```
items = pd.read_csv("data/order_items.csv")
footwear_orders = items.loc[items["sku"].str.startswith("FTW"), "order_id"].un
orders["has_footwear"] = orders["order_id"].isin(footwear_orders)
orders["returned"] = orders["status"] == "returned"
rate = orders.pivot_table(index="has_footwear", columns="channel", values="ret
                          aggfunc="mean", margins=True) * 100
print(rate.round(1))
```

```
channel       online  store   All
has_footwear
False            4.8    3.3   4.2
True            14.4    3.8  10.7
All              6.9    3.4   5.6
```

THREE MISTAKES TO EXPECT

- **Forgetting that the default aggregation is the mean.** No error. Symptom: a table titled 'Revenue by store' whose cells are around $180. Coach: "Read me your `aggfunc`. Now compute one cell by hand with a filter and `.sum()`." A relative with an error message: `TypeError: dtype 'str' does not support operation 'mean'` when the values column is text; they wanted `aggfunc="count"` or `pd.crosstab`.
- `pivot()` **on data with repeats, and two-level column labels.** Symptoms: `ValueError: Index contains duplicate entries, cannot reshape` (several rows per cell: decide how to combine them and use `pivot_table`); `KeyError: 2025` after `aggfunc=["sum", "mean"]` (the year is one level down: `multi[("sum", 2025)]`). Coach: "How many rows of the source land in one cell? Print `df.duplicated([row_key, column_key]).sum()`." and "Print `result.columns`." Also `melt()` without `id_vars`: every column is melted and no row knows its month.
- **Trusting the grid: sorted labels, filled zeros, rows that fell out.** No errors here either. Months in alphabetical order (Apr, Aug, Dec, Feb); `fill_value=0` on a table of means (an 'average cash order online of $0.00'); a grand total that is smaller than the source because rows with a missing key were dropped. Coach: "Does your corner cell match the source? What does an empty cell mean in this table: zero, not applicable, or missing?" In the project this is Step 3: $34,228.25 of revenue with no category quietly left the pivot.

GUIDED LAB

- `labs/module-06/l2_6-pivot-tables/payment_returns_starter.py` (reference: `payment_returns_solution.py`), 15 min: `pivot_table` with `fill_value` and `margins`, a rate as the mean of True/False, the count behind a cell, `crosstab` with `normalize="index"` and `"columns"`, and a total that hides a difference.

- `labs/module-06/l2_6-pivot-tables/survey_by_age_starter.py` (reference: `survey_by_age_solution.py`), 15 min: `melt` with `id_vars` and `value_vars`, `pivot_table` for means and for counts, `idxmin(axis=1)`, `stack()` back to tidy, saving to `OUTPUT`.
- `labs/module-06/l2_6-pivot-tables/weather_grid_starter.py` (reference: `weather_grid_solution.py`), 15 min: `pivot()` when there is one row per cell, `idxmax(axis=1)`, `pivot_table` on month numbers, the alphabetical-months trap and `reindex`, `melt` back to tidy.
- Start everyone on `payment_returns`. Before each pivot have pairs say the four things aloud to each other: rows, columns, values, how to combine.
- At TODO 2 ask the room: 'Should the empty cell be filled with 0?' Take a vote, then ask someone who voted no to explain.
- At minute 12 collect the two sentences from TODO 4 and write them on the board, one under the other. Ask which one a manager who wants to reduce returns should care about.
- TODO 5 is the payoff: the differences in the `All` column mostly vanish inside each channel. Ask: 'So is payment method the cause of anything?'
- `survey_by_age` next (it writes one CSV to `output/`). `weather_grid` is homework or a stretch.

START THE LESSON PROJECT

- The lesson project is **The Seasonality Report**: a category-by-month revenue grid from three files, a silent error caught, percentages, and a tidy copy (six steps, about 30 minutes solo).
- In the room do Step 1 (build the logbook with two validated merges: 41,788 completed lines, $3,715,829.95), Step 2 (pivot with margins) and Step 3 (find the missing revenue).
- Step 2 is staged: the pivot's grand total is smaller than the Step 1 total. Do not explain it. Ask the room to find out why, and give them three minutes before Step 3 reveals the 567 lines with no category ($34,228.25, six discontinued SKUs).
- Steps 4 to 6 go home. Done looks like: month columns 1 to 12 in order, Snow peaking in December and Tents in summer, and `category_seasonality.csv` with 108 rows.
- Ask them to keep `category_seasonality.csv`: Module 3 charts it.

CHECKS FOR UNDERSTANDING

1. Say the four things every pivot table needs. **Answer:** What each row is (`index`), what each column is (`columns`), which number goes in the cells (`values`), and how many rows are combined into one cell (`aggfunc`).
2. What is in the cells if you leave out `aggfunc`? **Answer:** The mean. Always write `aggfunc=`.

3. When does `pivot()` work, and when do you need `pivot_table()`? **Answer:** `pivot()` only rearranges and needs at most one row per cell; with several rows per cell you must say how to combine them, which is `pivot_table()`.

4. How can you tell that a table is wide rather than tidy? **Answer:** Its column headers are pieces of data (store names, months, years) rather than names of measurements.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** A) `20.0 (2, 2)`; **Q8** `(6, 3) 21`; **Q9** D) Of online orders, 6.9% were returned; **Q10** C) 609 lines have a missing category (or month), and rows with a missing key are dropped silently; **Q11** `(2, 2) 3`; **Q12** B) Decide how to combine the rows and use `pivot_table(..., aggfunc="sum")`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: Give unsure learners the sentence frame on a card: "For each _____ (rows), by _____ (columns), the _____ (how to combine) of _____ (values)." Have them fill it in on paper before typing. For `melt`, draw a three-row wide table on the board and melt it by hand with the room, writing the six long rows.

Stretch: `weather_grid`, then the lesson's Exercise 4 (audit the spreadsheet: melt the sheet, compute the same figures from `daily_sales.csv`, merge the two tidy tables and find the largest difference, $0.49). It uses both of today's lessons.

CLOSING

2:50 Wrap-up — 10 min

EXIT TICKET (3 QUESTIONS, ON PAPER OR STICKY NOTES, 5 MINUTES)

1. You merge `orders` (20,302 rows) with `customers` using the default settings and get 19,563 rows. What happened, and what would you write instead? **Answer:** The default inner join dropped the 739 orders whose `customer_id` has no row in `customers`. Write `orders.merge(customers, on="customer_id", how="left", validate="many_to_one")` and count the rows.
2. After `orders.merge(items, on="order_id")`, the sum of `order_total` is $11.0 million instead of $4.0 million. Why? **Answer:** The result has one row per order line, so each order's total is repeated on every one of its lines and counted several times. Sum order-level columns on the orders table (or after `drop_duplicates("order_id")`).
3. A count pivot with `margins=True` has a grand total of 44,881, but the source table has 45,490 rows. What does that tell you, and what do you do? **Answer:** 609 rows have a missing value in the row or column key and were dropped silently. Find them, give them an honest label (for example `fillna("Discontinued")`), and pivot again until the corner matches the source.

HOMEWORK (ABOUT 2 TO 3 HOURS BEFORE SESSION 7)

- Finish **Revenue by Category** (2.5, Steps 4 to 7): 45,490 lines kept, $3,715,829.95 of completed merchandise revenue, the pricing trap explained. Keep `output/order_lines.parquet`.
- Finish **The Seasonality Report** (2.6, Steps 4 to 6): the corner reconciles to the cent, months run 1 to 12, `category_seasonality.csv` has 108 rows. Keep that file for Module 3.
- Each lesson's exercises (2.5: 1 to 4, 5 is a stretch; 2.6: 1 to 3, 4 is a stretch), Quick Quiz and journal entry.
- Labs not finished in class; `orders_weather`, `weather_grid` and `debug_merges.py` are for home.
- Lesson 2.6, 'Before the Next Lesson': rebuild `wide` (daily revenue, one column per store) from memory, then look at `wide.loc["2024-08-10":"2024-08-16", "Bend"]` and write down what you see and what you would want to do about it. Session 7 starts there.

BRING LIST FOR SESSION 7

- Laptop with the course environment. Nothing new to install.
- The course `data/` folder as it is. Session 7 (Time Series and Window Functions) works mainly with `daily_sales.csv` and `orders.csv`.
- Your note on what `wide.loc["2024-08-10":"2024-08-16", "Bend"]` shows, and the one line of code that builds `wide`.

- Your `category_seasonality.csv` stays in your project folder for Module 3; you do not need it next time.
- This workbook and your notebooks. No servers, accounts or internet connection are needed for Session 7.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT, WHAT TO CUT FIRST

Protect, in this order: 2.5 steps 3 to 5 (the lost rows, the left join, the indicator audit), step 7 (rows multiply), the `customer_profiles` lab, 2.6 step 3 (the default mean), steps 6 and 7 (`normalize` both ways), step 10 (`melt`) and step 12 (the footwear table). Cut first: 2.5 step 10 (`concat` and `merge_asof` : assign as reading), then 2.6 step 5 (margins of means) and step 9 (holes), then `survey_by_age` . If 2.5 overruns by more than ten minutes, start the 2.6 project by doing Step 1 only and telling the room that Step 2's total will not match: finding out why is the homework.

RUNNING SHORT: WHAT TO ADD

Run `survey_by_age` for the whole room and ask two pairs to explain the count grid. Then `debug_merges.py` in pairs: bugs 4 and 5 are today's two quiet mistakes and make a good closing discussion. Still time: do Step 4 of Revenue by Category together (recover the category of the 609 discontinued lines from the SKU prefix with `.map()`), and ask why that is a repair and not a guess.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 7: Time Series and Windows

How to teach the session where the calendar becomes part of the data. Before the break, learners turn a text column into a ruler: they select by date string, find the 21 missing store-days in `daily_sales.csv`, decide which are true zeros and which are unknown, and roll days up into weeks, months and years without letting a gap pass as a bad month. After the break they slide windows along that ruler: moving averages, running totals, ranks, and the one idea that matters most, a baseline that never contains the day it judges.

`2 lessons` `3 hrs session` `Lessons 2.7–2.8` `Session 7 of 21`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: **Frame** (the question and a mental model) → **Live demo** (you type, they type along) → **Guided lab** (new practice from the kit) → **Start the lesson project** (everyone runs Step 1; the rest is homework).
- **Type, don't paste.** Every code block in the demo scripts is what to type, in a notebook, one cell per step. The output printed under each block is a real run on the course data with pandas 3.0: if your screen shows something else, stop and find out why before moving on.
- **The lessons are the reference.** Keep `pandas_time_series.html` and `pandas_window_functions.html` open in browser tabs for the three-kinds-of-gap table, the alias table, the trailing-or-centered table and the leakage flowchart.
- **Labs are new practice, not the lesson projects.** Starters and solutions are in `labs/module-07/`. Hand out starters only. The two lesson projects are started in the room and finished at home.
- Teal blocks are hands-on. Amber blocks are watch-outs.

> **THE RISK OF SESSION 7: THE WORST MISTAKES ARE SILENT**
>
> Almost nothing in this session raises an error when it goes wrong. A sum over a month with three missing days returns a number. `diff(7)` on a table with missing rows compares a Saturday with a Wednesday and returns a number. A rolling mean over a long table averages four different stores and returns a number. A baseline that contains the day it judges returns a number. Train two habits all session and write them on the board: **"Count beside every sum"** and **"On that day, could I have known every number in it?"**

BEFORE SESSION 7

- ☐ Run `python3 labs/check_labs.py module-07` from the kit folder: 8 lines, all PASS.
- ☐ Copy the four `*_starter.py` files from `labs/module-07/` to the shared folder (keep the folder structure, and keep `labs_paths.py` one level above the session folder). Keep the `*_solution.py` files on your machine.
- ☐ Projector notebook: a new notebook `session07_demo` in the course folder (the one that contains `data/`), font size 18 pt or larger. First cell already run: `import pandas as pd`, `import numpy as np`, `import matplotlib.pyplot as plt` and the two `pd.set_option` lines from the lesson (`display.width` 150, `display.max_columns` 20).
- ☐ Browser tabs: Lesson 2.7, Lesson 2.8, and `data/README.md` scrolled to the **daily_sales.csv** section (the list of gaps is the documentation learners need in the demo).
- ☐ Board, before learners arrive: **Parse → Index → Complete the calendar → Decide each gap → then resample / shift / compare.** Leave it up all session.
- ☐ Files used today: `data/daily_sales.csv` and `data/orders.csv`. No servers, no internet, no accounts.
- ☐ Sticky notes or scrap paper for the exit ticket.

KNOWN ENVIRONMENT TRAPS

Old frequency aliases. Anyone who copies from a search result or an older book will type `resample("M")`. pandas 3.0 raises `ValueError: Invalid frequency: M ... Please use 'ME' instead.` The demo triggers it on purpose (2.7 step 10) so the room has seen it.

A different pandas. Run `pd.__version__` on a learner machine that prints a `FutureWarning` instead of an error for `"M"`, or shows `datetime64[ns]` where the guide shows `datetime64[us]`: that machine has pandas 2.x. The code still works; send them to the setup guide after class.

Charts in a script. If someone works in a `.py` file and not a notebook, `plt.show()` opens a window that blocks the script until it is closed. Tell them to close the window. (The kit labs never call `plt.show()`.)

The working folder. `FileNotFoundError: data/daily_sales.csv` means the notebook is not in the course folder. The kit labs do not have this problem: they find the data through `labs_paths`.

SESSION 7 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 6 (merging and reshaping), no notes. Homework check: the `wide` table rebuilt from memory.
0:10	10	**2.7** Frame and mental model	Dates are a ruler, not labels. The five-step order of operations goes on the board.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:20	30	**2.7** Live demo	Parse, go wide, select by date string, find the missing days, decide what each gap means, resample with a count, measure change.
0:50	25	**2.7** Guided lab	`labs/module-07/l2_7-time-series/units_calendar_starter.py`: an honest weekly calendar for Bend.
1:15	10	**2.7** Start the lesson project	The Store Trend Report: everyone runs Step 1 and sees `(731, 4)` and `4 days closed`.
1:25	10	Break	Leave the Bend weekly table (with its 4-day week) on the projector.
1:35	10	**2.8** Frame and mental model	The train window: rolling, expanding, weighted. One value per row, not one per group.
1:45	30	**2.8** Live demo	Rolling means and their edges, running totals and year to date, the long-table trap, ranks, and look-ahead leakage.
2:15	25	**2.8** Guided lab	`labs/module-07/l2_8-window-functions/foot_traffic_starter.py`: when did Flagstaff pass Bend?
2:40	10	**2.8** Start the lesson project	The Sale-Day Detector: everyone runs Step 1 and sees 24 promotion days per store.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 8.
Total	180		Ends at 3:00.

The lesson pages estimate 90 + 90 minutes for a solo learner. After the warm-up, break and wrap-up the room has 150, and 50 of those go to the two kit labs. So the room does not cover either lesson whole, and it is not meant to.

> **IN THE ROOM, AT HOME, WHAT TO PROTECT**
>
> **In the room:** both mental models; both live demos (compressed versions of the lesson's core sections); the two guided labs (`units_calendar` for 2.7, `foot_traffic` for 2.8); Step 1 of each lesson project, run by everyone.
>
> **At home:** the rest of both lesson projects (2.7 Steps 2 to 6, 2.8 Steps 2 to 5); each lesson's exercises, Quick Quiz and journal; the two second labs (`returns_trend`, `online_milestones`) if they were not reached; and the sections the room skips: Time Zones in Brief and the quick `.plot()` look in 2.7, exponentially weighted windows and the ranking tie-break table in 2.8.
>
> **Protect:** in 2.7, completing the calendar and deciding what each gap means (demo steps 6 to 9) and the guided lab that repeats it on new data. In 2.8, look-ahead leakage (demo steps 12 and 13). A learner who leaves with only those two ideas has had a good session.
>
> **Cut first if you are running long:** see the last section of this guide for the order.

WARM-UP

0:00 Warm-up: Retrieval from Session 6 — 10 min

HOW TO RUN IT

Closed notebooks, closed laptops. Read each question aloud, give 30 seconds of silent writing, then take one answer from the room before giving yours. Five questions in six minutes; do not re-teach. A question that most of the room misses becomes a two-minute recap at the start of the 2.7 demo, where the same idea returns (question 4 returns in step 4, when you pivot to the wide table).

FIVE RETRIEVAL QUESTIONS (WITH ANSWERS)

1. What is the default `how=` of `merge`, and what happens to a row whose key has no match? **Answer:** `inner`. The row is dropped with no warning. In Lesson 2.5 that lost 739 orders whose customer has no row in `customers.csv`. Use `how="left"` when you are adding columns to a main table.
2. After a merge the result has **more** rows than the left table. What does that tell you? **Answer:** The key repeats on the other side (one-to-many or many-to-many), so left rows were duplicated. Count rows before and after, and state the relationship with `validate=`.
3. What does `pivot_table` compute in each cell if you leave out `aggfunc`? **Answer:** The mean. A table titled "revenue" built that way shows average order values. Always write `aggfunc`.
4. `df.pivot(index="date", columns="store", values="revenue")` raises `ValueError: Index contains duplicate entries, cannot reshape`. What does the error tell you about the data? **Answer:** At least one date and store pair appears on more than one row. `pivot` only rearranges; choose how to combine the repeats and use `pivot_table(..., aggfunc=...)`.
5. A sheet has one row per month and one column per store. Which method turns it into tidy rows of month, store, revenue? **Answer:** `melt(id_vars="Month", var_name="store", value_name="revenue")`. Twelve rows and five store columns become 60 rows.

HOMEWORK CHECK

Lesson 2.6 asked learners to rebuild `wide` (daily revenue, one column per store) from memory and to look at `wide.loc["2024-08-10":"2024-08-16", "Bend"]`. Ask: "Who looked at Bend in August? What did you see?" Take the answer (three `NaN` in a row) and park it: "Hold on to that. In twenty minutes you will know what to do about it, and what not to do." If fewer than half the room did the homework, say so plainly and move on; the demo rebuilds `wide` in step 4.

2.7 Time Series

75 min in the room

OBJECTIVES (THE LESSON'S OWN)

- Tell date text from real datetimes, convert between them, and pull out parts such as year, month and weekday
- Put dates in the index and select days, months and ranges with `.loc`
- Find missing dates with a complete calendar, and choose deliberately between leaving, zeroing and filling a gap
- Change frequency with `resample()` and the current aliases, and check how many days are behind each total
- Measure change with `shift()`, `diff()` and `pct_change()`, including year over year

TIMING

10m	Frame: the trend questions, the ruler, the five steps on the board
30m	Live demo, steps 1 to 14 (steps 13 and 14 are the first to cut)
25m	Guided lab: `units_calendar` (Bend's weekly units)
10m	Start the lesson project: Step 1 of The Store Trend Report

THE MENTAL MODEL TO SAY

Say: "To a computer, the text `2024-03-01` is ten characters, no more related to `2024-02-29` than 'cat' is to 'dog'. A datetime is a point on a ruler. Points on a ruler have an order, distances between them, and regular marks (days, weeks, months) you can group by. Everything today is something you can do with a ruler and cannot do with labels."

Then ask the three questions the lesson opens with and leave them unanswered: Is Asheville's revenue falling? Does Flagstaff grow all year or only in summer? What did the sale weekend add? **Say:** "Each of those is about order in time. By the break you will have answered the first two."

Point at the board and read the order of operations once: parse, index, complete the calendar, decide each gap, then compute. **Say:** "Most time series mistakes come from skipping the third step. Today you will watch that happen twice."

LIVE DEMO: YOU TYPE, THEY FOLLOW (14 STEPS, EVERY OUTPUT BELOW IS A REAL RUN)

Type each block in its own cell of `session07_demo`. Learners type along in their own notebook. Times in brackets add up to 30 minutes.

1. **Text is not a date (2m). Say:** "Load the file the lazy way and ask the date column what it is."

```
raw = pd.read_csv("data/daily_sales.csv")
print(raw["date"].dtype, repr(raw["date"].iloc[0]))
```

```
str '2024-01-01'
```

Ten characters. Now ask the text for its year: `raw["date"].dt.year`. Read the last line of the error aloud: `AttributeError: Can only use .dt accessor with datetimelike values`. **Say:** "That message means one thing: this column is still text."

2. **Parse at load time (2m). Say:** "Same file, one argument more."

```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
first = daily["date"].iloc[0]
print(daily["date"].dtype, "|", first, "|", first.day_name())
print(daily["date"].max() - daily["date"].min())
```

```
datetime64[us] | 2024-01-01 00:00:00 | Monday
730 days 00:00:00
```

Point at the two new things: a `Timestamp` knows its weekday, and subtracting two of them gives a length of time (a `Timedelta`). Text could do neither.

3. **Date parts are ordinary columns (2m). Say:** "`.dt` is to dates what `.str` is to text. And once the weekday is a column, groupby from Session 5 works on it."

```
by_day = daily.groupby(daily["date"].dt.day_name())["revenue"].mean().round(0)
print(by_day.sort_values(ascending=False))
```

```
date
Saturday     10933.0
Sunday        9211.0
Friday        7713.0
Thursday      6321.0
Monday        6138.0
Wednesday     5985.0
Tuesday       5716.0
Name: revenue, dtype: float64
```

Ask before you run it: "Which day wins?" Saturday is almost double Tuesday. **Say:** "Remember this weekly rhythm. After the break it is the reason we use a window of 7."

4 **Dates in the index (2m). Say:** "One row per date, one column per store: the wide table from your homework."

```
wide = daily.pivot(index="date", columns="store", values="revenue")
print(wide.head(3))
print(type(wide.index).__name__, len(wide))
```

```
store       Asheville     Bend    Boise  Flagstaff
date
2024-01-01    5017.07  4132.92  5962.00    3061.66
2024-01-02    3605.81  3675.57  6583.13    3575.24
2024-01-03    4265.55  6275.05  3866.59    3974.93
DatetimeIndex 727
```

Write **727** on the board. "Two years. How many days should there be?" Let someone say 731 (2024 is a leap year). Do not explain the difference yet.

5 **Select with a date string (3m). Say:** "With dates in the index, `.loc` understands a date written as text, and half a date too."

```
flag = wide["Flagstaff"]
print(flag.loc["2025-07-04"])
print(len(flag.loc["2025-07"]), len(flag.loc["2025"]))
print(flag.loc["2024-12-30":"2025-01-02"])
```

```
9937.66
31 363
date
2024-12-30    3600.03
2024-12-31    5352.77
2025-01-01    4442.10
2025-01-02    4827.54
Name: Flagstaff, dtype: float64
```

Three things to point at. `"2025-07"` is a whole month with no start and end typed. The range has four rows: **both ends are included**, unlike a list slice. And 2025 has 363 rows, not 365: "hold that thought".

6 **Check against the calendar (3m). Say:** "A time series is always checked against the calendar. pandas can make the calendar." Point at step 3 on the board.

```
calendar = pd.date_range("2024-01-01", "2025-12-31", freq="D")
closed_days = calendar.difference(wide.index)
print(len(calendar), len(wide))
print([str(d.date()) for d in closed_days])
```

```
731 727
['2024-11-28', '2024-12-25', '2025-11-27', '2025-12-25']
```

Ask: "What are these four dates?" Two Thanksgivings and two Christmases. Every store was closed, so there is no row at all.

7 **One row per calendar day (2m). Say:** "`asfreq` gives every day a row. Days that did not exist show up as `NaN`."

```
wide = wide.asfreq("D")
print(len(wide), wide.index.freq)
print(wide.isna().sum())
```

```
731 <Day>
store
Asheville     5
Bend         10
Boise         5
Flagstaff     7
dtype: int64
```

27 empty cells. **Ask:** "Are they all the same kind of empty?" Switch to the lesson tab and show the three-row table: closed for a holiday (16), export failed (5), revenue blank (6). **Say:** "The data cannot tell you which is which. The README can." Show the gap list in `data/README.md`.

8 **Decide what each gap means (2m). Say:** "A closed store sold zero dollars: that is a fact, so I write it down. An export that failed is a day I know nothing about: it stays unknown."

```
wide.loc[closed_days] = 0
print(wide.loc["2024-12-24":"2024-12-26"])
print(int(wide.isna().sum().sum()), "unknown store-days left")
```

```
store       Asheville     Bend    Boise  Flagstaff
date
2024-12-24    4925.47  5014.47  8001.08    5173.65
2024-12-25       0.00     0.00     0.00       0.00
2024-12-26    3365.72  6551.64  9899.95    5899.79
11 unknown store-days left
```

Mention `ffill()` and `interpolate()` in one sentence: "both are guesses that look exactly like measurements afterward; the lesson shows them on Bend, and if you ever fill, you say so."

9 **Resample: groupby for the calendar, with a count (4m). Say:** "`resample` drops every row into a calendar bucket. `ME` is month end. Watch what I ask for besides the sum."

```
bend = wide["Bend"].resample("ME").agg(["sum", "count", "mean"]).round(0)
print(bend.loc[["2024-08-31", "2025-08-31"]])
```

```
                sum  count    mean
date
2024-08-31  235539.0     28  8412.0
2025-08-31  271392.0     31  8755.0
```

Ask: "Did Bend's August grow 15%?" Point at `count`: 28 days against 31. Three days of sales are missing from the 2024 total. The mean per recorded day (8,412 against 8,755) is the fair comparison: about 4%. **Say and write: "Count beside every sum."** This is the moment the homework question about Bend is answered.

10 **The alias from an older tutorial (1m). Say:** "Half the examples online will tell you to type this."

```
wide["Bend"].resample("M").sum()
```

```
ValueError: Invalid frequency: M. Failed to parse with error message:
ValueError("'M' is no longer supported for offsets. Please use 'ME'
instead.")
```

Read it aloud: it names the replacement. `"M"` is now `"ME"`, `"Q"` is `"QE"`, `"Y"` is `"YE"`, `"H"` is `"h"`.

11 **The headline: whole years (2m). Say:** "Now the question leadership actually asked: how is each store trending?"

```
yearly = wide.resample("YE").sum()
print(yearly.round(0))
print((yearly.pct_change() * 100).round(1).loc["2025"])
```

```
store       Asheville       Bend      Boise  Flagstaff
date
2024-12-31  2123278.0  2476118.0  3518292.0  2270180.0
2025-12-31  2017699.0  2569350.0  3846406.0  2717997.0
store       Asheville  Bend  Boise  Flagstaff
date
2025-12-31       -5.0   3.8    9.3       19.7
```

Three stores grew. One shrank. **Say:** "2,903 rows were hiding that, and it took two lines." `pct_change()` compares each row with the row before it; with one row per year, that is last year.

12 **Year over year by month (3m). Say:** "Comparing February with January measures the season. Retail compares each month with the same month last year: in monthly data that is 12 rows back."

```
monthly = wide.resample("ME").sum()
yoy = (monthly.pct_change(12) * 100).round(1)
print(yoy.loc["2025-06":"2025-09"])
```

```
store       Asheville  Bend  Boise  Flagstaff
date
2025-06-30       -6.0  -1.1    6.5       15.7
2025-07-31      -12.6  -3.6   14.2       22.3
2025-08-31      -11.7  15.2   13.6       21.1
2025-09-30      -11.3  -0.3    5.2       21.3
```

Asheville is down in all four months, Flagstaff up in all four. **Ask:** "Which number on this screen do you not trust?" Bend's August, +15.2: it is flattered by the gap they saw in step 9.

13 **shift moves rows, not days (3m, first to cut). Say:** "`diff(7)` should mean 'against the same weekday last week'. It does, on a complete calendar."

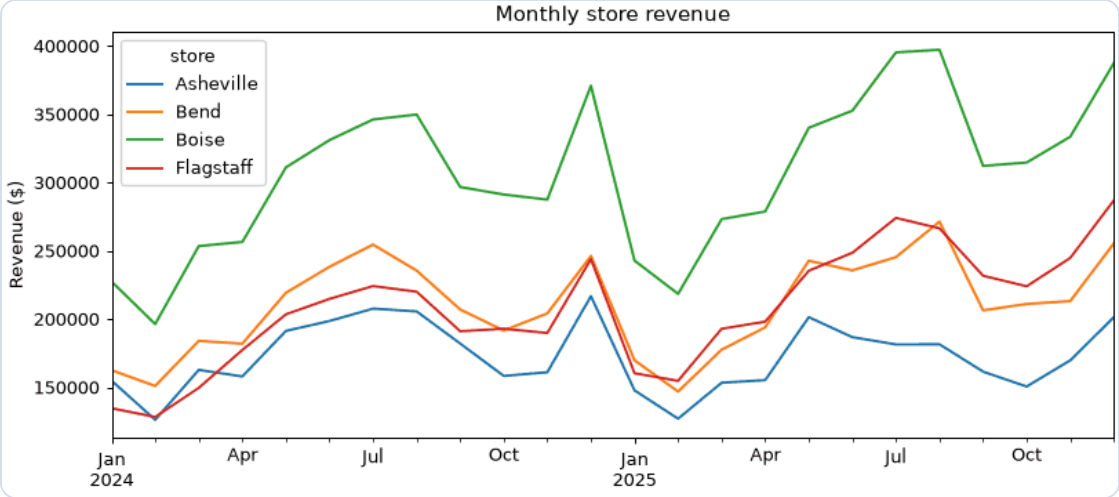
```
bend_rows = daily[daily["store"] == "Bend"].set_index("date")["revenue"]
check = pd.DataFrame({"revenue": wide["Bend"],
                      "diff7_rows_only": bend_rows.diff(7),
                      "diff7_calendar": wide["Bend"].diff(7)})
print(check.loc["2024-08-15":"2024-08-19"])
```

```
             revenue  diff7_rows_only  diff7_calendar
date
2024-08-15   4749.56         -5122.33        -2998.24
2024-08-16   7776.60          1722.87        -1073.78
2024-08-17  13248.68          5785.21         2275.39
2024-08-18   9199.05          1451.25         3338.30
2024-08-19   6094.11         -2756.27             NaN
```

August 17 is a Saturday. The calendar version compares it with the previous Saturday: +2,275. The rows-only version went back seven **rows**, which skipped the hole and landed on a Wednesday: +5,785. No error. Point at the board: this is what skipping step 3 costs.

14 **A sketch for your own eyes (1m, cut if short of time). Say:** "Module 3 teaches charts properly. For a first look, a table with a date index draws itself."

```
fig, ax = plt.subplots(figsize=(9, 4), layout="constrained")
monthly.plot(ax=ax, title="Monthly store revenue",
             ylabel="Revenue ($)", xlabel="")
plt.show()
```



What the room should see: one line per store.

Flagstaff climbs past Bend; Asheville drifts into last place. That picture is where Session 8's second half begins.

THREE MISTAKES LEARNERS WILL MAKE, AND HOW TO COACH EACH

- **Resampling or using `.dt` on text, or on a table with no date index.** Symptoms: `AttributeError: Can only use .dt accessor with datetimelike values`, or `TypeError: Only valid with DatetimeIndex, TimedeltaIndex or PeriodIndex, but got an instance of 'RangeIndex'`. Coach with a question, not the fix: "What does `df.dtypes` say? What is the index right now?" Then: `parse_dates` at load, and `set_index` / `pivot` or `resample("ME", on="date")`.
- **Trusting a total without its count.** There is no symptom: the learner reports "Bend's best week" or "August grew" from a bucket with missing days. Coach: "How many days are behind that number?" Make them add `count` to the `agg` list and read the row again. In the lab this is the 4-day week ending 2024-08-18.
- **Filling every gap the same way.** Learners write `wide.fillna(0)` (which turns five failed exports and six blank values into days with no sales) or `wide.interpolate()` (which invents sales on Christmas).

Coach: "Was the store open that day? How do you know?" Send them to the gap list in `data/README.md`. Closed is 0; unknown stays `NaN` unless you estimate and say so.

GUIDED LAB: FACILITATION NOTES

The lab: `labs/module-07/l2_7-time-series/units_calendar_starter.py` (your reference: `units_calendar_solution.py`). 25 minutes. It repeats demo steps 4 to 9 on a different column (`units`) and a different bucket (weeks), and ends with a judgment: which week-over-week changes would you not report?

Run it: learners open the starter in VS Code and run it cell by cell (`# %%` markers), or run the whole file with `python units_calendar_starter.py`. It runs as shipped and prints which TODOs are still open.

Minute 0 to 10 (TODO 1 to 3): walk the room. The common stall is TODO 1: `calendar.difference(units.index)` typed the wrong way round (`units.index.difference(calendar)` gives an empty index). Ask: "Which one has all 731 days?" Expected after TODO 2: Asheville 4, Bend 7, Boise 4, Flagstaff 6. After TODO 3: Bend 3, Flagstaff 2, the others 0.

Minute 10 to 20 (TODO 4 and 5): the `incomplete` table has two rows: the week ending 2024-08-18 (4 days) and the week ending 2026-01-04 (3 days). **Stop the room at the second one** and ask why it is there: the data ends on a Wednesday, so the last weekly bucket is partial. Nobody lost data; the week is simply not over. Best complete week: ends 2025-08-31, 1,767 units (the Labor Day sale).

Minute 20 to 25 (TODO 6) and debrief: the four changes are -96, +80, -542, +443. Ask for a show of hands: which would you report? The -542 and the +443 are both the gap. **Say:** "One incomplete bucket damages two changes: the one into it and the one out of it."

If the room finishes early: `labs/module-07/l2_7-time-series/returns_trend_starter.py` asks whether the online return rate is getting worse. The honest answer is "no trend" (7.11% in 2024, 6.76% in 2025; monthly rates wobble because a month has only 24 to 51 returns). It is good practice in not finding something.

STARTING THE LESSON PROJECT

The Store Trend Report (lesson page, about 30 minutes solo). In the room, 10 minutes: read the brief aloud, have everyone create the notebook `lesson_2_7_project` and run **Step 1**. They are done when the output shows `(731, 4)`, `4 days closed`, and unknown days of Asheville 1, Bend 6, Boise 1, Flagstaff 3.

Before they leave the step, ask: "Bend showed 3 unknown days in the lab and 6 here. Why?" The lab used `units`, which has no blank values; `revenue` has 3 blank values for Bend on top of the 3 missing rows. Same calendar, different column, different gaps.

Steps 2 to 6 are homework. Tell them the two numbers to check themselves against: Flagstaff +19.7% and Asheville -5.0% in the headline table.

CHECKS FOR UNDERSTANDING (ASK OUT LOUD)

1. How many rows does `flag.loc["2024-02"]` return on the completed calendar? **Answer:** 29. A partial date string selects the whole month, and 2024 is a leap year.

2. A month's revenue total is lower than the same month last year. What is the first thing you check? **Answer:** The `count` of recorded days in both months. A gap looks exactly like a decline.

3. In quarterly data, how many rows back is "the same quarter last year"? **Answer:** 4, so `pct_change(4)`. In monthly data 12, in weekly data 52.

4. Why not `fillna(0)` for the whole table? **Answer:** Zero is only true for the closed days. For a failed export it states that an open store sold nothing.

Workbook self-check, items Q1 to Q6:

Answers: **Q1** B) The column was loaded as text; pass `parse_dates=["date"]` or convert with `pd.to_datetime`; **Q2** `4 4`; **Q3** D) 731 rows; the four new rows hold `NaN`; **Q4** A) `[[40.0, 2.0]]`; **Q5** C) The value seven **rows** earlier, which is ten days back and not a Saturday; **Q6** `[nan, 10.0, 20.0]`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: Pair anyone who is unsure of `pivot` with a neighbor for the first cell of the lab (it is done for them in the starter, but they need to be able to read it). Give them the sentence frame for every gap: "On this day the store was (open / closed), so the value is (unknown / zero)." In the project, they may copy Step 1 from the lesson page instead of typing it.

Stretch: Lesson 2.7's bonus challenge: estimate Bend's three lost August days from the same weekdays in the two weeks before and after, then recompute the August year-over-year growth. Or, in the lab, repeat Steps 4 and 5 for Flagstaff and explain why its list of incomplete weeks has two rows as well (the January 2025 export failure, and the same partial last week).

2.8 Window Functions

75 min in the room

OBJECTIVES (THE LESSON'S OWN)

- Explain how a window calculation differs from a groupby summary, and choose between rolling, expanding and exponentially weighted windows
- Smooth a series with `rolling()`, and control the edges and gaps with `min_periods`, time-based windows and `center=True`
- Build running totals and year-to-date figures with `cumsum()` and `expanding()`
- Apply windows and ranks within groups, and attach the result to the original table without misaligning it
- Recognize look-ahead leakage and write a window that uses only the past

TIMING

10m	Frame: one value per row; the train window; rolling, expanding, weighted
30m	Live demo, steps 1 to 13 (steps 6 and 10 are the first to cut; 11 and 12 never)
25m	Guided lab: `foot_traffic` (visitors, Flagstaff and Bend)
10m	Start the lesson project: Step 1 of The Sale-Day Detector

THE MENTAL MODEL TO SAY

Say: "Before the break, monthly totals smoothed the daily zigzag, but at a price: 731 days became 24 numbers. A window function gives you both. It computes a summary for **every row**, from a small set of nearby rows. The table keeps its length; each row gains some context."

Say: "From a train you never see the whole route. You see what fits in the window, and the view changes a little every meter. A **rolling** window is that pane of glass: always the last 7 days. An **expanding** window is a camera that started filming at the first station and never stops: everything so far. An **exponentially weighted** window is your memory of the trip: the last minutes are vivid, the early stations fade."

Draw two boxes on the board: `groupby().agg()` → one row per group; window → one row per input row. **Ask:** "Which tool from Session 5 also returned one value per row?" (`transform`.) That is the bridge to step 9.

LIVE DEMO: YOU TYPE, THEY FOLLOW (13 STEPS, EVERY OUTPUT BELOW IS A REAL RUN)

Continue in `session07_demo`. Times in brackets add up to 30 minutes plus a one-minute close.

1. **Rebuild the clean calendar (2m). Say:** "This is the one block you may paste today: it is Step 1 of the project you just started."

```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
revenue = daily.pivot(index="date", columns="store", values="revenue")
calendar = pd.date_range(revenue.index.min(), revenue.index.max(), freq="D")
closed_days = calendar.difference(revenue.index)
revenue = revenue.asfreq("D")
revenue.loc[closed_days] = 0
boise = revenue["Boise"]
print(revenue.shape, int(revenue.isna().sum().sum()))
```

```
(731, 4) 11
```

731 days, 4 stores, 11 unknown store-days. Everything after the break stands on this table.

2 **A rolling mean (3m). Say:** "`rolling(7)` means: for each row, this row and the six before it. Like `groupby`, it computes nothing until you say what to do with each window."

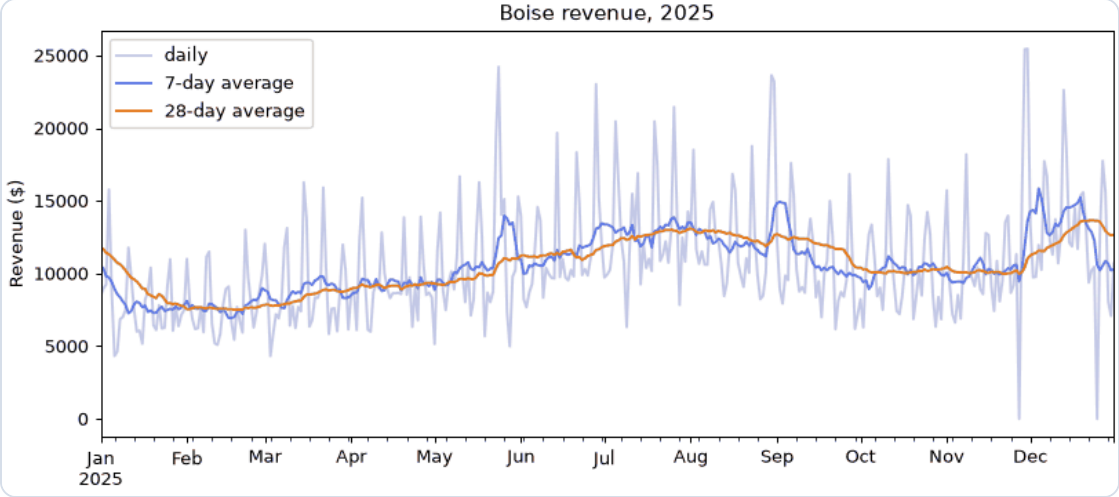
```
week_view = pd.DataFrame({"revenue": boise, "avg_7d": boise.rolling(7).mean()})
print(week_view.head(9).round(2))
```

```
            revenue   avg_7d
date
2024-01-01   5962.00      NaN
2024-01-02   6583.13      NaN
2024-01-03   3866.59      NaN
2024-01-04   7667.88      NaN
2024-01-05   9570.46      NaN
2024-01-06  12388.57      NaN
2024-01-07   9361.59  7914.32
2024-01-08   6173.31  7944.50
2024-01-09   6636.06  7952.07
```

Ask: "Why six `NaN`?" A 7-day average needs 7 days. Check January 7 by hand on a calculator or with `boise.head(7).mean()`. Then point at January 8: the window slid, dropped January 1 and took in January 8. **Why 7?** Call back the weekday table from before the break: a 7-day window always holds exactly one of each weekday, so the weekly zigzag cancels.

3 **See it (2m). Say:** "The same year at three levels of detail."

```
smooth = pd.DataFrame({"daily": boise, "7-day average": boise.rolling(7).mean(
                       "28-day average": boise.rolling(28).mean()}).loc["2025"
fig, ax = plt.subplots(figsize=(9, 4), layout="constrained")
smooth.plot(ax=ax, title="Boise revenue, 2025", ylabel="Revenue ($)", xlabel="
            color=["#c5cae9", "#667eea", "#e67e22"], linewidth=1.5)
plt.show()
```



What the room should see: daily, 7-day and 28-day lines for Boise, 2025.

The 7-day line removes the zigzag and still shows each sale weekend. The 28-day line shows only the season. Note the order: roll over the whole series, **then** slice 2025, so January already has a full window behind it.

4 **One missing day erases a week (3m). Say:** "Boise has one blank revenue value, on December 3, 2024. Watch what it does."

```
gap_view = pd.DataFrame({"revenue": boise, "strict": boise.rolling(7).mean(),
                         "min_6": boise.rolling(7, min_periods=6).mean()})
print(gap_view.loc["2024-12-02":"2024-12-10"].round(0))
```

```
            revenue   strict    min_6
date
2024-12-02  16367.0  14262.0  14262.0
2024-12-03      NaN      NaN  15528.0
2024-12-04   8558.0      NaN  15528.0
2024-12-05  11482.0      NaN  17442.0
2024-12-06  16251.0      NaN  17503.0
2024-12-07  18336.0      NaN  16026.0
2024-12-08  13736.0      NaN  14122.0
2024-12-09  10025.0      NaN  13065.0
2024-12-10  10822.0  12744.0  12744.0
```

One `NaN` cost the strict version seven averages. `min_periods=6` says "answer if at least 6 of the 7 are present". **Say:** "That is a trade, not a fix. Six of seven I can defend. `min_periods=1` makes the `NaN` go away and makes the first 'weekly average' one day's revenue."

5 **Running totals and records (3m). Say:** "An expanding window starts at the first row and never lets go."

```
running = pd.DataFrame({"revenue": boise,
                        "total_so_far": boise.cumsum(),
                        "avg_so_far": boise.expanding().mean(),
                        "best_so_far": boise.cummax()})
print(running.head(4).round(2))
records = boise[boise == boise.cummax()]
print(len(records), "record days; the last:", records.index[-1].date())
```

```
            revenue  total_so_far  avg_so_far  best_so_far
date
2024-01-01  5962.00       5962.00     5962.00      5962.00
2024-01-02  6583.13      12545.13     6272.56      6583.13
2024-01-03  3866.59      16411.72     5470.57      6583.13
2024-01-04  7667.88      24079.60     6019.90      7667.88
12 record days; the last: 2024-08-31
```

A day is a record exactly when it equals the best so far. Boise's last record is the Saturday of Labor Day weekend 2024.

6 **Trailing or centered (3m, cut third). Say:** "A trailing average lags. `center=True` puts the row in the middle of its window: three days back, three days forward."

```
both = pd.DataFrame({"revenue": boise, "trailing": boise.rolling(7).mean(),
                     "centered": boise.rolling(7, center=True).mean()})
print(both.loc["2025-05-21":"2025-05-24"].round(0))
print(both.tail(3).round(0))
```

```
            revenue  trailing  centered
date
2025-05-21   8026.0   10324.0   12425.0
2025-05-22   8755.0   10408.0   12628.0
2025-05-23  19002.0   11292.0   13973.0
2025-05-24  24213.0   12425.0   13791.0
            revenue  trailing  centered
date
2025-12-29   8051.0   10666.0       NaN
2025-12-30   7083.0   10221.0       NaN
2025-12-31  10680.0   10253.0       NaN
```

The Memorial Day sale starts Friday, May 23. The centered average is already higher on May 21, **before the sale has happened**, because its window reaches into the future. It also ends three days early. **Say:** "Keep the phrase 'reaches into the future'. It comes back in ten minutes."

7 **Year to date (2m). Say:** "A business wants a running total that restarts every January 1. 'Within each year' is `groupby`."

```
ytd = revenue.groupby(revenue.index.year).cumsum()
print(ytd.loc["2024-12-31":"2025-01-01"].round(0))
```

```
store       Asheville       Bend      Boise  Flagstaff
date
2024-12-31  2123278.0  2476118.0  3518292.0  2270180.0
2025-01-01     5867.0     4477.0     8442.0     4442.0
```

The total climbs through December 31 and starts again from one day's revenue. The result has the same shape as `revenue`: one value per row, like `transform`.

8 **The long-table trap (2m). Say:** "Most real data is long, not wide. Here is a 4-row rolling mean on the original table."

```
print(daily[["date", "store", "revenue"]].head(5))
print(daily["revenue"].rolling(4).mean().head(5).round(2).tolist())
```

```
        date      store  revenue
0 2024-01-01  Asheville  5017.07
1 2024-01-01       Bend  4132.92
2 2024-01-01      Boise  5962.00
3 2024-01-01  Flagstaff  3061.66
4 2024-01-02  Asheville  3605.81
[nan, nan, nan, 4543.41, 4190.6]
```

Ask: "What is 4,543.41 the average of?" Asheville, Bend, Boise and Flagstaff on January 1. The next one drops Asheville and adds the next day's Asheville. No error, and it means nothing.

9 **Sort, group, window (3m). Say:** "Two fixes: sort so each store's days are in order, and compute the window inside each store. `transform` hands the result back lined up with the rows."

```
long = daily.sort_values(["store", "date"]).copy()
long["avg_7d"] = long.groupby("store")["revenue"].transform(
    lambda s: s.rolling(7).mean())
cols = ["date", "store", "revenue", "avg_7d"]
print(long.loc[long["store"] == "Bend", cols].head(8).round(2))
```

```
         date store  revenue   avg_7d
1  2024-01-01  Bend  4132.92      NaN
5  2024-01-02  Bend  3675.57      NaN
9  2024-01-03  Bend  6275.05      NaN
13 2024-01-04  Bend  5487.11      NaN
17 2024-01-05  Bend  4755.15      NaN
21 2024-01-06  Bend  7083.36      NaN
25 2024-01-07  Bend  6719.33  5446.93
29 2024-01-08  Bend  5111.35  5586.70
```

The quick test: the first rows of **each** group should be `NaN`. Bend's first six are, so its window started fresh and did not borrow the end of Asheville's data. Write **Sort → Group → Window** on the board.

10 **Rank within a group (3m, cut fourth). Say:** "A rank is a value's position in sorted order. Inside a group it gets interesting: rank the four stores against each other on every day."

```
daily["rank_in_day"] = daily.groupby("date")["revenue"].rank(
    ascending=False, method="min")
print(pd.crosstab(daily["store"], daily["rank_in_day"]))
```

```
rank_in_day  1.0  2.0  3.0  4.0
store
Asheville     11   90  172  453
Bend          37  315  272   97
Boise        646   56   22    2
Flagstaff     33  266  261  164
```

Boise came first on 646 of 726 days; Asheville came last on 453. (Six days with a missing revenue have an incomplete ranking.) The lesson has the tie-break table for `method=`: point to it, do not type it.

11 **Leakage: the baseline that contains its own answer (4m, never cut). Say:** "Now the most important idea today. I want to know if a day was unusual, so I compare it with the recent average. Which rows may that average use?"

```
leak = pd.DataFrame({"revenue": boise,
                     "avg7_with_today": boise.rolling(7).mean(),
                     "avg7_before_today": boise.shift(1).rolling(7).mean()})
print(leak.loc["2025-05-21":"2025-05-24"].round(0))
```

```
            revenue  avg7_with_today  avg7_before_today
date
2025-05-21   8026.0          10324.0            10192.0
2025-05-22   8755.0          10408.0            10324.0
2025-05-23  19002.0          11292.0            10408.0
2025-05-24  24213.0          12425.0            11292.0
```

Friday, May 23, first day of the sale. `avg7_with_today` has already risen to 11,292, because the day being judged is inside its own baseline. `avg7_before_today` is 10,408: only what was known when the doors opened. **Say and write:** "On that day, could I have known every number in it?" That question is the whole defense; nothing raises an error.

12 **How much does a leak flatter you? (3m, never cut). Say:** "Pretend each of these is a forecast of the day's revenue. Average miss, in dollars."

```python
candidates = {
    "3-day avg, centered (leaks)": boise.rolling(3, center=True).mean(),
    "3-day avg, with today (leaks)": boise.rolling(3).mean(),
    "3-day avg, before today": boise.shift(1).rolling(3).mean(),
    "7-day avg, before today": boise.shift(1).rolling(7).mean(),
    "same weekday last week": boise.shift(7),
}
for name, guess in candidates.items():
    print(f"{name:<30} average miss ${(boise - guess).abs().mean():>6,.0f}")
```

```
3-day avg, centered (leaks)    average miss $ 1,417
3-day avg, with today (leaks)  average miss $ 2,012
3-day avg, before today        average miss $ 3,132
7-day avg, before today        average miss $ 2,504
same weekday last week         average miss $ 2,158
```

Ask the room to pick a winner before you run it. The two leaking versions look best, because the answer is one third of each of them, and they would be useless tomorrow. Among the honest ones the simplest wins: the same weekday last week. **Say:** "Knowing the shape of your data beat a fancier calculation."

13 **Name what you skipped (1m). Say:** "Two things are in the lesson and not in this demo: a window measured in time, `rolling("7D")`, and exponentially weighted means, `ewm(span=7)`, which weight recent days more. Read both sections at home; neither is needed for the lab."

THREE MISTAKES LEARNERS WILL MAKE, AND HOW TO COACH EACH

- **A window that runs across groups.** On a long table, `df["revenue"].rolling(7).mean()` averages different stores together and raises nothing. Coach with the quick test: "Are the first rows of **each** store `NaN`, or only the top of the table?" Then: sort by group and date, `groupby(...).transform(lambda`

`s: s.rolling(7).mean())`. A related error message, when they assign `groupby(...).rolling(...)` straight back: `TypeError: incompatible index of inserted column with frame index`.

- **`min_periods=1` to make the `NaN` go away.** The numbers appear, and the first "28-day average" is one day. Coach: "How many days is the value on row 3 built from? Would you show that to a manager as a monthly average?" Agree on a threshold they can defend: 6 of 7, 21 of 28.
- **A baseline that includes the row it judges.** `rev / rev.rolling(7).mean()` and anything with `center=True`. No symptom except results that look too good. Coach with the board question: "On that day, could you have known every number in the window?" Fix: `shift(1)` before the window.

GUIDED LAB: FACILITATION NOTES

The lab: `labs/module-07/l2_8-window-functions/foot_traffic_starter.py` (your reference: `foot_traffic_solution.py`). 25 minutes. The clean visitors calendar is built for them; they add a 7-day mean, a 28-day mean with `min_periods`, find a crossover, find record days with `cummax`, and find each store's busiest 7 days with a rolling sum.

Minute 0 to 8 (TODO 1 and 2): expected 28-day averages on 2025-12-31: Asheville 292, Bend 367, Boise 575, Flagstaff 420. If someone's first valid 28-day value is on row 28, they forgot `min_periods=21`; with it the first value is on row 21.

Minute 8 to 18 (TODO 3): the crossover is the thinking part. `ahead[ahead].index[0]` is 2024-12-25, and it deserves a raised eyebrow: on that date the 28-day average holds a closed day (0 visitors) for both stores, and Flagstaff was only just ahead. Part (c) is the better answer to "when did Flagstaff pass Bend": Bend was last ahead on 2025-06-03, so Flagstaff has led since 2025-06-04. Flagstaff was ahead on 324 days in all. **Ask:** "Which of the two dates would you put in a report?"

Minute 18 to 25 (TODO 4 and 5) and debrief: Flagstaff set 18 visitor records, the last on 2025-08-30 (853 visitors, the Saturday of Labor Day weekend). The busiest 7 days: Asheville ended 2024-05-30, Bend 2025-09-04, Boise 2025-09-02, Flagstaff 2025-12-21. Point out what a rolling sum can do that `resample("W")` cannot: the best stretch may start on any day.

If the room finishes early: `labs/module-07/l2_8-window-functions/online_milestones_starter.py`. Its last step is the leakage idea on new data: the shop's best day (2025-06-24, $8,960.15) is 2.20 times a leak-free Tuesday baseline but only 1.60 times a leaky one, and the leaky baseline finds 2 days at twice normal where the honest one finds 25.

STARTING THE LESSON PROJECT

The Sale-Day Detector (lesson page, about 35 minutes solo). In the room, 10 minutes: read the brief aloud (six sale weekends, 24 days, 96 promoted store-days: could you find them from revenue alone?), create `lesson_2_8_project`, run **Step 1**. Done when `revenue` and `promo` are both `(731, 4)` and `promo.sum()` shows 24 for every store.

Tell them what is coming so they do not think they failed: "Your first detector will be wrong three times out of four. That is the point of the project. Step 3 asks you to look at **which** days it gets wrong before you change anything."

Steps 2 to 5 are homework. Check numbers: 260 store-days flagged at first (66 real promotions), then 142 flagged (72 real) with the weekday baseline.

CHECKS FOR UNDERSTANDING (ASK OUT LOUD)

1. How many rows does `revenue.rolling(7).mean()` have, compared with `revenue.resample("W").mean()`? **Answer:** 731 against 105. A window keeps one row per day; a resample returns one row per week.
2. A 28-day rolling mean with the default settings: how many `NaN` rows at the start of a gap-free series? **Answer:** 27. The window is full for the first time on row 28.
3. Which is safe for judging today: `s.rolling(7).mean()`, `s.rolling(7, center=True).mean()` or `s.shift(1).rolling(7).mean()`? **Answer:** Only the third. The first includes today; the second includes today and three future days.
4. What is the quick test that a per-group window did not run across groups? **Answer:** The first rows of every group are `NaN`, not only the first rows of the table.

Workbook self-check, items Q7 to Q12:

Answers: **Q7** C) `[nan, 6.0, 6.0, 8.0]`; **Q8** `[True, False, True, False, True]`; **Q9** D) Sort by store and date, then `df["avg"] = df.groupby("store")["revenue"].transform(lambda s: s.rolling(7).mean())`; **Q10** A) `[1.0, 3.0, 1.0, 2.0]`; **Q11** B) Each day is inside its own baseline, so every spike is understated; use `rev.shift(1).rolling(28).mean()`; **Q12** `[nan, nan, 3.0, 5.0, 7.0]`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: For the crossover (TODO 3), give the three lines one at a time and have the learner say in words what each returns before running it. Anyone lost in `transform(lambda s: ...)` should work on the wide table only: each column is one store, so `visitors.rolling(7).mean()` needs no grouping. That is why the lab is wide.

Stretch: Lesson 2.8's bonus challenge: the detector's remaining false alarms cluster on one weekday because closed days were filled with 0 and now sit inside the weekday baseline; fix the baseline and rescore. Or, in the lab, redo TODO 3 with a 7-day average instead of 28 and count how many times the lead changes hands: a good argument for why the window size is part of the result (the lead changes 13 times on the 28-day average and 33 times on the 7-day one).

CLOSING THE SESSION

2:50 Wrap-up — 10 min

EXIT TICKET (3 QUESTIONS, WITH ANSWERS)

1. You resample daily revenue to monthly totals with `.sum()`. What do you print beside every total, and why? **Answer:** The `count` of days with a value (for example `.agg(["sum", "count"])`). Sums skip `NaN` silently, so a month with missing days looks like a weak month.
2. Write an expression for a 7-day average of the Series `rev` that is safe to use for judging or predicting today. **Answer:** `rev.shift(1).rolling(7).mean()` (or `rev.rolling(7, closed="left").mean()`). Shift first, so today is not inside its own baseline.
3. List the order of operations for a time series, and name the step people skip most often. **Answer:** Parse the dates; put them in the index; complete the calendar with `asfreq`; decide what each gap means; only then resample, shift or compare. The skipped step is completing the calendar.

Reading the tickets: a ticket that says "mean" or "nothing" for question 1 needs the Bend August example again. For question 2, `rev.rolling(7).mean().shift(1)` is also correct (the same numbers); `rev.rolling(7).mean()` and anything with `center=True` are the leak, and those learners get a two-minute recap at the start of Session 8, where leakage returns as the cutoff rule. For question 3, accept any wording that puts the calendar before the arithmetic.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- Finish the Lesson 2.7 project, **The Store Trend Report** (Steps 2 to 6). Check: Flagstaff +19.7%, Asheville -5.0%, and `store_monthly_revenue.csv` saved.
- Finish the Lesson 2.8 project, **The Sale-Day Detector** (Steps 2 to 5). Check: 260 flags at first, then 142 with the weekday baseline, and `sale_day_flags.csv` with 2,924 rows.
- Lesson 2.7: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **Time Zones in Brief**: it was not covered in the room.
- Lesson 2.8: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **Exponentially Weighted Windows**: it was only mentioned in the room.
- Kit labs not reached in the room: `labs/module-07/l2_7-time-series/returns_trend_starter.py` and `labs/module-07/l2_8-window-functions/online_milestones_starter.py` (about 20 minutes each).
- From Lesson 2.8, "Before the Next Lesson": open `data/customers.csv` and list three new columns you could compute for each customer that might help predict whether they will order again.

BRING LIST FOR SESSION 8

- Their Lesson 2.8 project notebook and `sale_day_flags.csv`, and the Lesson 2.7 output `store_monthly_revenue.csv` (Session 8 charts it).
- The list of three customer features from the homework: it opens Lesson 2.9.
- The course data folder with `customers.csv`, `orders.csv`, `daily_sales.csv`, `products.csv` and `store_weather.csv`.
- Nothing new to install: Matplotlib came with `requirements.txt`. Ask them to run `import matplotlib; print(matplotlib.__version__)` once at home (the course uses 3.11) and to tell you before the session if it fails.
- No servers, no accounts, no internet needed.

PREVIEW TO SAY

Next session closes the pandas module and opens the charting one. First, feature engineering: everything from Module 2 (derived columns, date parts, bins, per-customer summaries, windows) becomes columns built so a pattern is easy to see, and leakage comes back as the cutoff rule. Then you draw your first proper charts with Matplotlib: line, scatter, bar and histogram, each chosen from the question it answers.

IF YOU RUN LONG OR SHORT

IF YOU RUN LONG: CUT IN THIS ORDER

1. **2.7 demo step 14** (the quick `.plot()`): say "the lesson shows a one-line sketch" and skip it. Saves 2 minutes.
2. **2.7 demo step 13** (`diff(7)` on rows-only data): show the lesson's output table from the browser tab instead of typing it. Saves 3 minutes. Self-check Q5 covers it.
3. **2.8 demo step 6** (trailing or centered): show the lesson's comparison table instead of typing. Saves 3 minutes.
4. **2.8 demo step 11** (ranking within a day): one sentence, "rank within a group works like transform; the lesson has the table". Saves 3 minutes.
5. **Guided labs:** stop each at its Step 4 and give Step 5 as homework. Do not cut a lab below Step 4: in `units_calendar` that is where the 4-day week appears, in `foot_traffic` it is the record days.
6. **Project starts:** as a last resort, run Step 1 yourself on the projector and have learners run it at home.

Never cut: 2.7 demo steps 6 to 9 and 2.8 demo steps 12 and 13.

IF YOU RUN SHORT

Run the second lab of the lesson you are in (`returns_trend` after the 2.7 lab, `online_milestones` after the 2.8 lab). Both end in a judgment the room can argue about: "is there a trend?" and "why does the leaky baseline find so few unusual days?"

Or continue the lesson project together: 2.7 Step 2 (the completeness check with `days_in_month`) is the best ten extra minutes available in this session.

Or run the Predict, then run boxes in the workbook as a pair activity: predict alone, compare with a neighbor, then run.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 8: Features and First Charts

How to teach the session that closes the pandas module and opens the charting one. Before the break, everything from Module 2 becomes features: learners decide what one row is, build flags, ratios, bins and per-customer summaries, join them back without losing anyone, and meet leakage again as the cutoff rule. After the break their tables become pictures: one Figure, one Axes, and the four charts that answer most questions, each chosen from the question and not from habit.

`2 lessons` `3 hrs session` `Lessons 2.9–3.1` `Session 8 of 21`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: **Frame** → **Live demo** (you type, they type along) → **Guided lab** (new practice from the kit) → **Start the lesson project** (everyone runs Step 1; the rest is homework).
- **Type, don't paste.** Every code block in the demo scripts is what to type, one cell per step. The output under each block is a real run on the course data (pandas 3.0, Matplotlib 3.11). In the charting half, the small picture under a step is what the room should see.
- **The lessons are the reference.** Keep `pandas_feature_engineering_basics.html` and `matplotlib_line_scatter_bar.html` open for the cut-versus-qcut table, the three-kinds-of-leak table, the pyplot translation table and the choosing-a-chart table.
- **Labs are new practice, not the lesson projects.** Starters and solutions are in `labs/module-08/`. Hand out starters only.
- Teal blocks are hands-on. Amber blocks are watch-outs.

> **THE RISK OF SESSION 8: CHARTS AND FEATURES THAT LOOK RIGHT**
>
> Both halves of this session share one danger: the wrong result arrives with no error. An inner merge deletes the 1,058 customers who never ordered. `pd.cut` turns values outside its edges into `NaN`. A feature that counts orders after the cutoff looks brilliant. `ax.bar` on raw rows draws four tidy bars that show each store's single best day. Train one question for each half and write both on the board: **"What is one row?"** and **"What question does this chart answer?"** A learner who cannot answer in a sentence is not ready to write the next line.

BEFORE SESSION 8

- ☐ Run `python3 labs/check_labs.py module-08` from the kit folder: 8 lines, all PASS.

- ☐ Copy the four `*_starter.py` files from `labs/module-08/` to the shared folder (keep the folder structure and `labs_paths.py` one level above the session folder). Keep the `*_solution.py` files on your machine.
- ☐ Projector notebook: a new notebook `session08_demo` in the course folder (the one that contains `data/`), font size 18 pt or larger. First cell already run: `import numpy as np`, `import pandas as pd`, `import matplotlib.pyplot as plt` and the two `pd.set_option` lines from the lessons.
- ☐ Run one throwaway chart on the projector machine before class (`fig, ax = plt.subplots(); ax.plot([1, 2, 3]); plt.show()`) and check that it is readable from the back of the room. If the notebook theme is dark, the default white figure is fine; do not restyle it today.
- ☐ Browser tabs: Lesson 2.9, Lesson 3.1.
- ☐ Board, before learners arrive: **What is one row?** on the left, **What question does this chart answer?** on the right.
- ☐ Files used today: `customers.csv`, `orders.csv`, `order_items.csv`, `daily_sales.csv`, `products.csv`, `store_weather.csv`. No servers, no internet, no accounts.
- ☐ Sticky notes or scrap paper for the exit ticket.

> KNOWN ENVIRONMENT TRAPS
>
> **Charts that do not appear.** In a notebook a figure shows at the end of the cell. In a plain `.py` script, `plt.show()` opens a window and the script waits until it is closed. On a machine with no display (a remote server, some WSL setups) no window opens at all: save with `fig.savefig("chart.png")` and open the file. The kit labs always save to `output/` and never call `plt.show()`, so they work everywhere.
>
> **A line of text above the chart**, such as `[<matplotlib.lines.Line2D object at 0x...>]`, is the return value of the cell's last line. It is harmless; ending the cell with `plt.show()` removes it.
>
> **Old habits from other tutorials.** Learners who have seen Matplotlib before will type `plt.title(...)`. It works with one chart, which is why it is tempting. Hold the line on the one rule (plt creates and shows; everything else is a method on `ax` or `fig`): Session 9 depends on it.
>
> `observed=` **warnings.** Anyone on pandas 2.x sees a `FutureWarning` about `observed` when grouping by a categorical. The code still works; the lab solutions pass `observed=True` explicitly so they are quiet everywhere.

SESSION 8 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 7 (time series and windows), no notes. Homework check: the three customer features, and the first detector score.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:10	10	**2.9** Frame and mental model	Prep before cooking. First decision: what is one row?
0:20	30	**2.9** Live demo	Flags, ratios, date parts, bins, an ordered category, per-customer aggregates joined back, one-hot columns, and the cutoff rule.
0:50	25	**2.9** Guided lab	`labs/module-08/l2_9-feature-engineering/order_features_starter.py`: which orders come back?
1:15	10	**2.9** Start the lesson project	The Customer Feature Table: everyone runs Step 1 and sees 4,474 eligible customers.
1:25	10	Break	Leave the online-footwear table (14.4% returned) on the projector.
1:35	10	**3.1** Frame and mental model	Question first, chart second. Figure and Axes.
1:45	30	**3.1** Live demo	One chart five lines long, then line, scatter, bar and histogram, each from a question; two charts that draw without error and are wrong.
2:15	25	**3.1** Guided lab	`labs/module-08/l3_1-line-scatter-bar/four_questions_starter.py`: four questions, four charts.
2:40	10	**3.1** Start the lesson project	Does Rain Keep Shoppers Away? Everyone runs Step 1 and sees `(2903, 13)`.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 9.
Total	180		Ends at 3:00.

The lesson pages estimate 95 + 90 minutes for a solo learner. After the warm-up, break and wrap-up the room has 150, and 50 of those go to the two kit labs. The room does not cover either lesson whole.

IN THE ROOM, AT HOME, WHAT TO PROTECT

In the room: both mental models; both live demos; the two guided labs (`order_features` for 2.9, `four_questions` for 3.1); Step 1 of each lesson project, run by everyone.

At home: the rest of both lesson projects (2.9 Steps 2 to 6, 3.1 Steps 2 to 6); each lesson's exercises, Quick Quiz and journal; the two second labs (`store_day_features` , `chart_repair`); and the sections the room skips: Scaling by Hand and the categories-with-no-rows detail in 2.9; the missing-days line, `ax.errorbar` and the pandas `.plot` table in 3.1.

Protect: in 2.9, the left merge with honest fills (demo steps 9 and 10) and the cutoff rule (step 12). In 3.1, the five-line first chart (step 1) and the wrong bar chart (step 8). These are the ideas Sessions 9 to 13 lean on.

Cut first if you are running long: see the last section of this guide for the order.

WARM-UP

0:00 Warm-up: Retrieval from Session 7

10 min

HOW TO RUN IT

Closed notebooks. Read each question aloud, 30 seconds of silent writing, one answer from the room, then yours. Question 4 is the bridge to today: leakage returns in the first half as the cutoff rule. If the room is shaky on it, say so: "Good. You will see the same mistake again in forty minutes, at a larger scale."

FIVE RETRIEVAL QUESTIONS (WITH ANSWERS)

1. You resample daily revenue to monthly totals. What do you print beside every total? **Answer:** The `count` of days with a value. Sums skip `NaN` silently, so a month with missing days looks like a weak month.
2. What does `wide.asfreq("D")` do to a table with dates in the index? **Answer:** It gives every calendar day a row. Days that had no row appear, filled with `NaN`, and you then decide what each gap means.
3. On a gap-free daily Series, how many `NaN` values does `s.rolling(7).mean()` have, and where? **Answer:** Six, at the start. A 7-row window is full for the first time on row 7.
4. Write a 7-day average of `rev` that is safe for judging today. **Answer:** `rev.shift(1).rolling(7).mean()`. Shift first, so today is not inside its own baseline.
5. A long table holds four stores stacked in one column. What three things do you do, in order, to get each store's own 7-day average? **Answer:** Sort (by store, then date), group (by store), window (inside `transform`). The first rows of each group should come out `NaN`.

HOMEWORK CHECK

Two quick shows of hands. First: "Who wrote down three customer features from `customers.csv` and `orders.csv`?" Collect five or six on the board without comment (tenure, age, number of orders, total spent, days since last order, share bought online are the usual ones). Leave the list up: the 2.9 demo builds most of them, and step 12 asks which of them could leak. Second: "What did your first sale-day detector score?" (260 flagged, 25% real.) "And after the weekday baseline?" (142 flagged, 51% real.) One sentence: "A better idea of what normal means beat any amount of threshold tuning. Today's first lesson is about building better ideas into columns."

2.9 Feature Engineering Basics

75 min in the room

OBJECTIVES (THE LESSON'S OWN)

- Decide what one row of a feature table represents, and build derived columns and date parts for it
- Turn numbers into groups with `pd.cut()` and `pd.qcut()`, and work with ordered and unordered categoricals
- Summarize many rows per customer and join the summaries back, handling customers with no rows correctly
- Encode categories as numbers and rescale numeric columns by hand, and say when each is needed
- Audit a feature table for leakage using a cutoff date

TIMING

10m	Frame: the marketing question, prep before cooking, what is one row?
30m	Live demo, steps 1 to 12 (steps 7 and 11 are the first to cut; 9, 10 and 12 never)
25m	Guided lab: `order_features` (which orders come back?)
10m	Start the lesson project: Step 1 of The Customer Feature Table

THE MENTAL MODEL TO SAY

Say: "A restaurant kitchen does not cook from whole sacks of onions. Before service, the cooks wash, peel, chop and measure, and set each ingredient out in its own container. Raw tables are the sacks. Features are the prepared containers. A chart, a statistical test or a model is the cooking, and it goes well or badly mostly because of the prep."

Say: "A feature is a column you build so that a pattern becomes visible. And the first decision, the one that is easy to skip, is: **what is one row?**" Show the lesson's three-row table (will this customer order again → a customer; will this order be returned → an order; how busy will the store be tomorrow → a store-day). The guided lab is the second row; the second lab is the third.

Connect to last session: "In Lesson 2.8, 'same weekday last week' beat every smoother. That was a feature. Good features come from knowing the business, which is why a person builds them."

LIVE DEMO: YOU TYPE, THEY FOLLOW (12 STEPS, EVERY OUTPUT BELOW IS A REAL RUN)

Type each block in its own cell of `session08_demo`. Times in brackets add up to 30 minutes.

1. **The question and the unit (2m). Say:** "Marketing asks: which customers are likely to order again? Look at what we have."

```
customers = pd.read_csv("data/customers.csv", parse_dates=["signup_date"])
orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
print(customers.shape, orders.shape)
```

```
(4850, 12) (20302, 11)
```

Ask: "For that question, what should one row of our table be?" A customer. Point at the board. **Say:** "`orders` has 20,302 rows and one row is an order. So most of today is getting from many rows per customer to exactly one."

2 **Derived columns: flag, comparison, arithmetic, choice (3m). Say:** "The simplest features come from other columns in the same row. Four patterns cover almost all of them."

```
feat = orders[["order_id", "customer_id", "order_date", "channel", "promo_code
              "shipping_cost", "order_total"]].copy()
feat["used_promo"] = feat["promo_code"].notna()                    # a flag
feat["is_online"] = feat["channel"] == "online"                    # a test
feat["merch_total"] = feat["order_total"] - feat["shipping_cost"]  # arithmet
feat["size"] = np.where(feat["merch_total"] >= 250,                # a choice
                        "large", "regular")
show = ["promo_code", "used_promo", "is_online", "merch_total", "size"]
print(feat[show].head(4))
```

```
  promo_code  used_promo  is_online  merch_total     size
0        NaN       False      False       434.90    large
1        NaN       False      False       278.90    large
2        NaN       False       True        60.95  regular
3        NaN       False       True       399.80    large
```

Name each as you type it. `merch_total` removes shipping so online and store orders are comparable.

3 **A feature earns its place by separating things (2m). Say:** "Never keep a column because you could build it. Check it against something you care about."

```
by_promo = feat.groupby("used_promo")["merch_total"]
print(by_promo.agg(["mean", "median", "count"]).round(2))
```

```
              mean  median  count
used_promo
False       195.54   147.9  18154
True        171.30   130.1   2148
```

Promo orders are about $24 smaller. The flag separates two groups that behave differently, so it stays. Point at `count`: "the habit from last session, again."

4 **Ratios remove size (3m). Say:** "Boise has more transactions than Asheville because Boise has more visitors. That says nothing about how well either store sells. Divide."

```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
daily["conversion"] = daily["transactions"] / daily["visitors"]
daily["basket"] = daily["revenue"] / daily["transactions"]
cols = ["visitors", "transactions", "conversion", "basket"]
print(daily.groupby("store")[cols].mean().round(3))
```

```
           visitors  transactions  conversion  basket
store
Asheville   274.898        64.536       0.235  88.444
Bend        334.872        78.992       0.235  88.706
Boise       485.166       114.513       0.236  88.655
Flagstaff   328.462        77.746       0.236  88.774
```

Ask the room to predict before you run it: "Which store converts best?" The raw columns differ by almost a factor of two. The ratios are nearly identical: about 24 buyers per 100 visitors everywhere. **Say:** "Whatever is different about Asheville, it is not that its visitors are less willing to buy. The raw columns could not show you that."

5 **Elapsed time needs a reference date (3m). Say:** "How long has this person been a customer? That is a difference between two dates, and one of them is a date **I choose and write down**."

```
as_of = pd.Timestamp("2025-12-31")
customers["tenure_days"] = (as_of - customers["signup_date"]).dt.days
customers["age"] = as_of.year - customers["birth_year"]
show = ["customer_id", "signup_date", "tenure_days", "birth_year", "age"]
print(customers[show].head(3))
print(int(customers["age"].isna().sum()), "customers with no age")
```

```
   customer_id signup_date  tenure_days  birth_year   age
0        10001  2023-08-07          877         NaN   NaN
1        10002  2020-10-10         1908      1989.0  36.0
2        10003  2023-06-11          934      1995.0  30.0
442 customers with no age
```

Say: "Never `pd.Timestamp.today()`: every number would change each time the notebook runs. And the 442 with no birth year have no age. A missing input gives a missing feature. Do not fill it with 0."

6 **cut: you choose the edges (3m). Say:** "The survey reports ages in six bands. To compare, I need the same bands, so the edges are dictated."

```
edges = [17, 24, 34, 44, 54, 64, 120]
labels = ["18-24", "25-34", "35-44", "45-54", "55-64", "65+"]
customers["age_group"] = pd.cut(customers["age"], bins=edges, labels=labels)
print(customers["age_group"].value_counts(sort=False, dropna=False))
```

```
age_group
18-24    466
25-34    693
35-44    780
45-54    788
55-64    717
65+      964
NaN      442
Name: count, dtype: int64
```

Ask: "Why does the first edge say 17 and not 18?" Each bin includes its right edge and excludes its left: (17, 24] is "above 17, up to and including 24". Seven edges make six bins. The 442 stay `NaN`. Mention the trap in one sentence: a value outside the edges becomes `NaN` with no warning (self-check Q3).

7 **qcut: equal-sized groups (2m, cut first). Say:** "No natural edges? `qcut` picks them from the data so each bin holds the same number of rows."

```
print(pd.qcut(feat["merch_total"], q=4).value_counts(sort=False))
```

```
merch_total
(5.999, 75.95]      5101
(75.95, 145.95]     5065
(145.95, 264.63]    5060
(264.63, 4893.0]    5076
Name: count, dtype: int64
```

About 5,075 orders in each. Look how wide the top bin is: a quarter of all orders sit between $265 and $4,893, because order values are skewed.

8 **An ordered category (3m). Say:** "`loyalty_tier` is text. Text sorts alphabetically: Basic, Gold, Silver. That is wrong in every table and chart you will ever make from it."

```
tier_type = pd.CategoricalDtype(["Basic", "Silver", "Gold"], ordered=True)
customers["loyalty_tier"] = customers["loyalty_tier"].astype(tier_type)
print(customers["loyalty_tier"].value_counts(sort=False))
tier = customers["loyalty_tier"]
print((tier >= "Silver").sum(), tier.max())
```

```
loyalty_tier
Basic     2822
Silver    1294
Gold       734
Name: count, dtype: int64
2028 Gold
```

Now the tiers list in rank order, "Silver or better" works, and `.max()` knows Gold is the top. Vocabulary for the board: **ordinal** (has an order: tiers, age bands) and **nominal** (does not: store, channel).

9 **Many rows per customer down to one (3m, never cut). Say:** "The best customer features are not in the customer table. They are summaries of what each customer did. Group and aggregate: Session 5."

```
per_customer = orders.groupby("customer_id").agg(
    n_orders=("order_id", "count"),
    total_spend=("order_total", "sum"),
    avg_order=("order_total", "mean"),
    last_order=("order_date", "max"),
)
print(per_customer.head(3).round(2))
print(len(per_customer))
```

```
             n_orders  total_spend  avg_order last_order
customer_id
10001               8      2129.65     266.21 2025-06-25
10002               4      1013.40     253.35 2025-12-18
10003               9      1873.75     208.19 2025-12-06
3942
```

3,942 rows: one for every customer id that appears in `orders`. Write **3,942** and **4,850** on the board. **Ask:** "Which kind of merge?"

10 **Left merge, then honest fills (3m, never cut). Say:** "Left, from the table that defines the unit. And I state the relationship, so pandas checks it for me."

```
keep = ["customer_id", "acquisition_channel", "loyalty_tier", "tenure_days"]
table = customers[keep].merge(per_customer, on="customer_id", how="left",
                              validate="one_to_one")
print(len(table), int(table["n_orders"].isna().sum()))
table["n_orders"] = table["n_orders"].fillna(0).astype(int)
table["total_spend"] = table["total_spend"].fillna(0)
table["has_ordered"] = table["n_orders"] > 0
check = ["n_orders", "total_spend", "avg_order", "has_ordered"]
print(table[check].isna().sum().tolist())
```

```
4850 1058
[0, 0, 1058, 0]
```

4,850 rows before, 4,850 after. The 1,058 who never ordered are still here: "exactly the people marketing most wants to know about". **Say:** "Zero orders and zero dollars are true statements. An average order of $0.00 is not: there is no average of nothing. So `avg_order` stays missing, and the flag keeps the fact visible." The last line shows it: 0, 0, 1058, 0.

11 **One yes/no column per category (2m, cut second). Say:** "Channel has no order. Number it 0 to 4 and a model reads 'referral is twice paid search'. Give each channel its own column instead."

```
dummies = pd.get_dummies(table["acquisition_channel"], prefix="via", dtype=int
print(dummies.head(3))
```

```
   via_in_store  via_organic  via_paid_search  via_referral  via_social
0             0            0                1             0           0
1             0            0                0             0           1
2             0            0                0             0           1
```

Each row has exactly one 1. For the ordinal tier, `table["loyalty_tier"].cat.codes` gives 0, 1, 2 in rank order. Scaling (`min-max`, z-score, `np.log1p`) is three lines of arithmetic in the lesson: name it and move on.

12 **The cutoff rule (4m, never cut). Say:** "To test who orders again, I pretend to stand on a past date, build features from what was known then, and check them against what happened afterward. Here the date is January 1, 2025, and 'afterward' is the first half of 2025."

```
cutoff = pd.Timestamp("2025-01-01")
base = customers.loc[customers["signup_date"] < cutoff, ["customer_id"]].copy(
past = orders[orders["order_date"] < cutoff]
later = orders[(orders["order_date"] >= cutoff)
               & (orders["order_date"] < "2025-07-01")]

def orders_per_customer(rows):
    counts = rows.groupby("customer_id").size()
    return base["customer_id"].map(counts).fillna(0)

base["n_before"] = orders_per_customer(past)        # honest: before the cutof
base["n_all_time"] = orders_per_customer(orders)    # leaks: includes the futu
base["ordered_again"] = base["customer_id"].isin(later["customer_id"])
print(len(base), round(base["ordered_again"].mean(), 3))
target = base["ordered_again"].astype(int)
print(base[["n_before", "n_all_time"]].corrwith(target).round(2))
never = base["n_all_time"] == 0
print(int(never.sum()), base.loc[never, "ordered_again"].mean())
```

```
4101 0.551
n_before      0.38
n_all_time    0.55
dtype: float64
698 0.0
```

Two columns that both look like "number of orders". The all-time count is far more strongly related to the answer (0.55 against 0.38) and it predicts one group perfectly: all 698 people with zero orders in the whole file did not order again, by definition. **Say:** "That is not insight. It is the answer written into the question. It raises no error, and on a real January 1 that column cannot be computed." Point at the homework list on the board: **which of your three features could leak?** (`loyalty_tier` is the one to catch: it is today's tier, earned partly by ordering in 2025.) The project repeats this at a July cutoff with a full table.

THREE MISTAKES LEARNERS WILL MAKE, AND HOW TO COACH EACH

- **An inner merge that deletes the most interesting rows.** No error; the table shrinks from 4,850 to 3,792 and every average afterward describes buyers only. Coach: "How many rows before the merge? How many after?" Then `how="left"` from the table that defines the unit, plus `validate=`.
- **Filling every `NaN` with 0.** `table.fillna(0)` turns "no average exists" into "average order of $0.00" and drags group means down. Coach: "Is that zero a fact about this customer?" Counts and sums: yes. Averages, shares, recency: no. Add a flag such as `has_ordered`.
- **Bins that silently lose values.** `pd.cut` with edges that do not cover the data (a first edge of 18 instead of 17, a last edge that is too low) produces `NaN` with no warning. Coach: "Compare `.isna().sum()`

before and after the cut." Fix with `-np.inf` and `np.inf` as the outer edges, or `include_lowest=True`. Related error, on `qcut` with many repeated values: `ValueError: Bin edges must be unique`.

GUIDED LAB: FACILITATION NOTES

The lab: `labs/module-08/l2_9-feature-engineering/order_features_starter.py` (your reference: `order_features_solution.py`). 25 minutes. It changes the unit of analysis on purpose: **one row is an order**, and the question is which orders get returned. Same tools as the demo, different table.

Minute 0 to 8 (TODO 1 to 3): the three flags are quick. The stall is TODO 2, named aggregation: `has_footwear=("is_footwear", "any")` asks "does any line of this order contain footwear?" Expected after TODO 3: 20,302 rows before and after, and no order without lines.

Minute 8 to 18 (TODO 4 and 5): six small tables. Read them with the room: online orders are returned twice as often as store orders (6.9% against 3.4%); promo and weekend separate nothing (5.4% against 5.7% both times); orders with footwear are returned 10.7% of the time against 4.2%. **Say:** "Two of your six features turned out to be useless. That is a result too."

Minute 18 to 25 (TODO 6 and 7) and debrief: the interaction is the finding: online orders that contain footwear are returned 14.4% of the time, against 3.3% to 4.8% for the other three combinations. In stores, footwear makes no difference (people try shoes on). Neither feature alone shows that. For TODO 7, take answers aloud: `status` **is** the target. A learner who would "just leave it in" has not yet met the cutoff rule; that is the bridge to the project.

If the room finishes early: `labs/module-08/l2_9-feature-engineering/store_day_features_starter.py`: a store-day feature table for "how busy tomorrow?" Its Step 5 shows that `shift(7)` on rows is not "the same weekday last week" on 104 rows, and its Step 6 is a leak audit where three columns correlate at 0.94 to 0.96 because they are counted during the day being predicted.

STARTING THE LESSON PROJECT

The Customer Feature Table (lesson page, about 40 minutes solo). In the room, 10 minutes: read the brief aloud (one row per customer, features as of July 1, 2025, a target saying whether they ordered in the following six months), create `lesson_2_9_project`, run **Step 1**. Done when the output shows `4850 4474`, then `14497 5805`, then `2025-06-30 2025-07-01`.

Ask before they move on: "Why 4,474 and not 4,850?" Customers who signed up after the cutoff did not exist yet on July 1; there is nothing to know about them.

Steps 2 to 6 are homework. Check numbers: `(4474, 19)` after Step 3; a target rate of 0.569; in the leak audit, correlations of 0.39 (honest) against 0.53 (all-time).

CHECKS FOR UNDERSTANDING (ASK OUT LOUD)

1. One row is a store-day. Name two features and one thing that is not a feature of a store-day. **Answer:** Features: weekday, promotion flag, last week's visitors. Not a feature: one customer's order total (it is a

fact about an order).

2. `pd.cut` or `pd.qcut` for age bands that must match a published survey? **Answer:** `pd.cut`: the edges are dictated. `qcut` would choose its own edges from the data.

3. After a left merge, which of `n_orders`, `total_spend`, `avg_order` do you fill with 0? **Answer:** `n_orders` and `total_spend`. `avg_order` stays missing; add a `has_ordered` flag.

4. Why not use `pd.Timestamp.today()` for tenure? **Answer:** The numbers change on every run, and "now" is later than the data. Fix a reference date in a variable.

Workbook self-check, items Q1 to Q6:

Answers: **Q1** B) The `order_total` of each of the customer's orders; **Q2** `['18-24', '18-24', '25-34', '35+']`; **Q3** C) `2`; **Q4** D) 3,792 rows: every customer who never ordered is gone; **Q5** A) Ordered integer codes for the tier; one yes/no column per channel with `pd.get_dummies`; **Q6** C) The feature looks more predictive than it is, because orders after the cutoff are in both the feature and the target. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: Give the sentence frame "One row is a ___, and this column says ___ about it" and have the learner say it for each column they add. In the lab, TODO 2 can be done one aggregation at a time: start with `n_lines` only, print `per_order.head()`, then add the others.

Stretch: Lesson 2.9's bonus challenge (an RFM segmentation with `qcut`, and the question of which bin edges to use for a brand-new customer). Or, in the lab, add `max_unit_price` and `n_units` bands and look for a second interaction; or rebuild `has_footwear` by merging with `products.csv` and explain why 609 lines lose their category (the six discontinued SKUs from Lesson 2.5).

3.1 Line, Scatter and Bar Charts

75 min in the room

OBJECTIVES (THE LESSON'S OWN)

- Explain what a Figure and an Axes are, and build any chart with `fig, ax = plt.subplots()` and methods on `ax`
- Draw a line chart, a scatter plot, a bar chart and a histogram from a pandas DataFrame, with a title and labeled axes
- Choose a chart type from the question being asked (change over time, relationship, comparison, distribution)
- Plot straight from a DataFrame with `.plot(ax=ax)` and say what pandas is doing for you
- Recognize charts that are drawn without error but show the wrong thing, and fix them

TIMING

10m	Frame: a chart is an answer; four questions, four charts; Figure and Axes
30m	Live demo, steps 1 to 11 (steps 11 and 2 are the first to cut; 1 and 8 never)
25m	Guided lab: `four_questions` (customers and products)
10m	Start the lesson project: Step 1 of Does Rain Keep Shoppers Away?

THE MENTAL MODEL TO SAY

Say: "A table of 24 monthly totals is correct, and almost nobody can see the pattern in it. Draw the same 24 numbers as a line and the seasons, the growth and the odd month out are visible in a second. Your eyes are very good at shapes and very slow at columns of digits."

Say: "A chart is not a decoration. It is an answer to a question, and the kind of question picks the kind of chart. Four tools on a bench: you do not start by picking up the saw, you start by looking at what needs cutting." Write the four on the board under **What question does this chart answer?**: change over time → line; two numbers together → scatter; which group is bigger → bar; what is typical → histogram.

Two nouns: **Figure**, the sheet of paper; **Axes**, one chart drawn on it. Draw a rectangle with a smaller rectangle inside it and label both. "Next session the sheet holds four charts. That is why we hold `ax` in a variable from the very first chart."

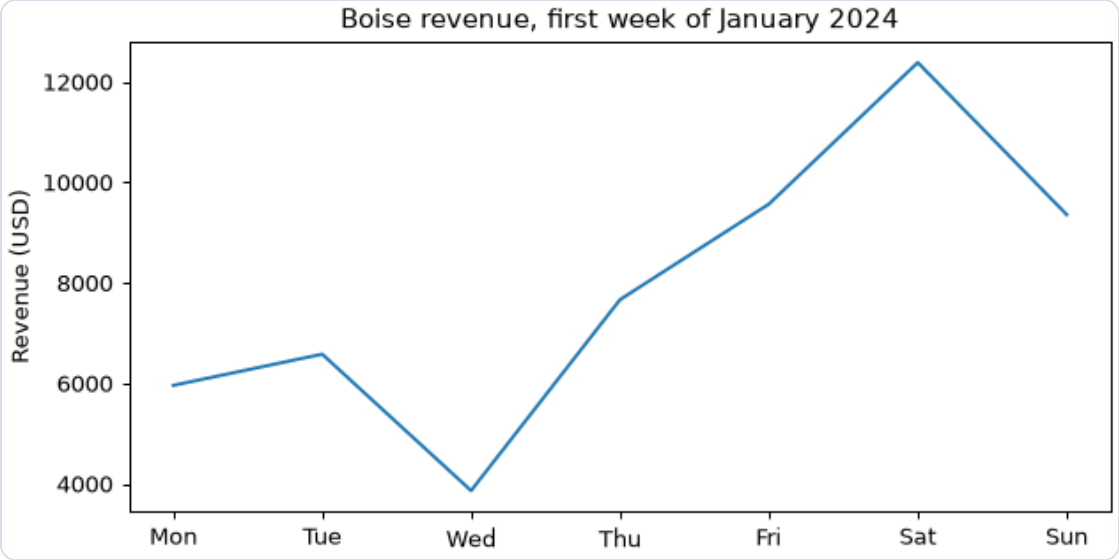
LIVE DEMO: YOU TYPE, THEY FOLLOW (11 STEPS, EVERY OUTPUT BELOW IS A REAL RUN)

Continue in `session08_demo`. Times in brackets add up to 30 minutes. Where a step has a picture under it, that is the chart the room should see.

1 **One chart, five lines (4m, never cut). Say:** "Every chart in this module is a variation on these five lines. Read them slowly with me."

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
week = [5962.00, 6583.13, 3866.59, 7667.88, 9570.46, 12388.57, 9361.59]

fig, ax = plt.subplots(figsize=(7, 3.5), layout="constrained")
ax.plot(days, week)
ax.set_title("Boise revenue, first week of January 2024")
ax.set_ylabel("Revenue (USD)")
plt.show()
```



What the room should see.

Line by line: `plt.subplots()` makes a **Figure** (the sheet of paper) and one **Axes** (one chart on it) and hands back both. `figsize` is inches. `layout="constrained"` stops labels being cut off: "use it every time". `ax.plot(x, y)` draws. `set_title` and `set_ylabel` label: "a chart without labels is a puzzle". Then the naming trap, once: **Axes is not the plural of axis.** An Axes is a whole chart; an axis is one of its number lines.

2 **They are ordinary objects (1m). Say:** "`fig` and `ax` are variables. You can ask them about themselves."

```
print(type(fig).__name__, "and", type(ax).__name__)
print(len(fig.axes), len(ax.lines), ax.get_title())
```

```
Figure and Axes
1 1 Boise revenue, first week of January 2024
```

One Axes, one line so far. These two counts are how the kit labs check a chart without looking at it.

3 **Two styles, one rule (2m). Say:** "Most examples online call functions on `plt`: `plt.plot`, `plt.title`. They draw on whatever chart is 'current'. With one chart that is obvious. With four, the title lands on the wrong panel and nothing tells you why. So: `plt` creates figures and shows them. Everything else is a method on `ax` or `fig`. And this is what happens when the two styles get mixed."

```
ax.title("Boise revenue")
```

```
TypeError: 'Text' object is not callable
```

`ax.title` is the title object, not a function. On an Axes the methods start with `set_`. Show the lesson's translation table (pyplot on the left, this course on the right).

4 **Most of a chart is made in pandas first (3m). Say:** "The question: how did store revenue change over the two years? Daily data is the wrong level of detail for a two-year question. Aggregate first."

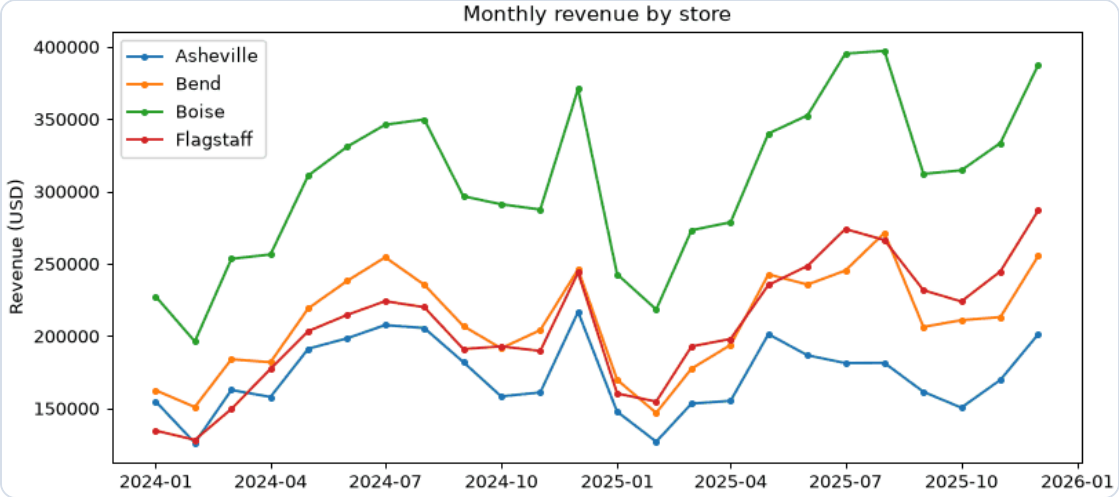
```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
daily["month"] = daily["date"].dt.to_period("M").dt.to_timestamp()
monthly = daily.pivot_table(index="month", columns="store", values="revenue",
                            aggfunc="sum")
print(monthly.head(3).round(0))
```

```
store       Asheville      Bend     Boise  Flagstaff
month
2024-01-01   154850.0  162534.0  227438.0   134690.0
2024-02-01   126216.0  151014.0  196327.0   128296.0
2024-03-01   162774.0  184033.0  253463.0   149609.0
```

One row per month, one column per store: the shape a line chart wants. (`to_timestamp()` turns the month label back into a real date so Matplotlib can place it on a date axis.)

5 **A line per store (3m). Say:** "Call `ax.plot` once per column. Every call adds a line to the same Axes, and the `label` is what the legend shows."

```
fig, ax = plt.subplots(figsize=(9, 4), layout="constrained")
for store in monthly.columns:
    ax.plot(monthly.index, monthly[store], marker="o", markersize=3,
            label=store)
ax.set_title("Monthly revenue by store")
ax.set_ylabel("Revenue (USD)")
ax.legend()
plt.show()
```



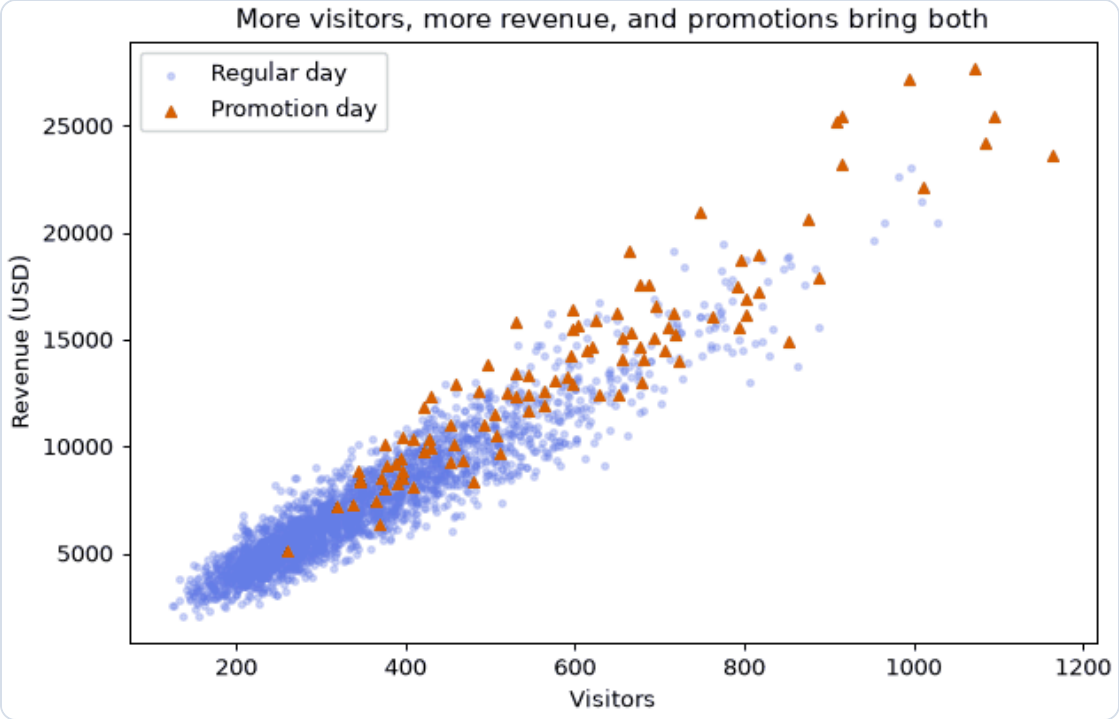
What the room should see: four lines, a legend, date ticks chosen by Matplotlib.

Ask: "What can you see now that the table hid?" A summer peak and a December spike everywhere; Boise on top; Flagstaff climbing past Asheville and Bend. Markers say "these 24 points are the measurements"; with 727 daily points you would leave them off.

6 **Scatter, and the blob (4m). Say:** "New question: when more people walk in, how much more does a store sell? Two numbers, one dot per store-day. I draw the regular days small and see-through, and the promotion days on top."

```
promo = daily["promo_day"] == 1
fig, ax = plt.subplots(figsize=(7, 4.5), layout="constrained")
ax.scatter(daily.loc[~promo, "visitors"], daily.loc[~promo, "revenue"],
           s=8, alpha=0.3, color="#667eea", label="Regular day")
ax.scatter(daily.loc[promo, "visitors"], daily.loc[promo, "revenue"],
           s=22, color="#d95f02", marker="^", label="Promotion day")
ax.set_title("More visitors, more revenue, and promotions bring both")
ax.set_xlabel("Visitors")
ax.set_ylabel("Revenue (USD)")
ax.legend()
plt.show()
per_visitor = daily["revenue"] / daily["visitors"]
print(round(per_visitor[~promo].mean(), 2), round(per_visitor[promo].mean(), 2
```

```
20.79 23.12
```



What the room should see: a translucent band, with orange triangles at its far end.

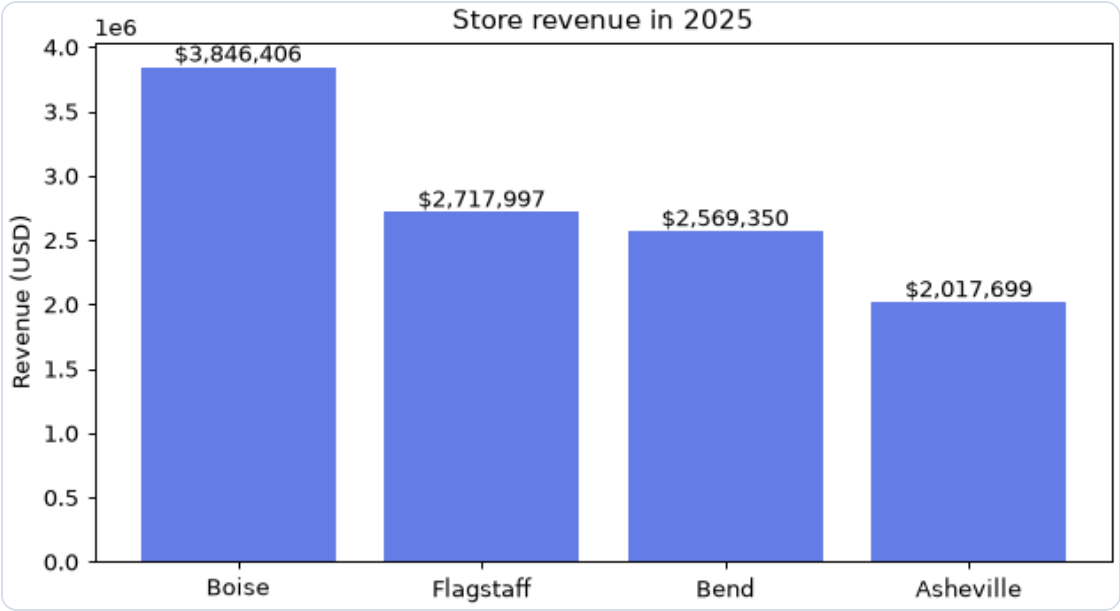
First run it **without** `s=8, alpha=0.3` so the room sees the solid blob, then add them: that is overplotting and its fix. The triangles differ in shape as well as color, so the chart works in gray. The printed numbers check what the eye suspects: about $21 per visitor on regular days, $23 on promotion days. Promotions work mainly by bringing more people in. **Say:** "A scatter shows that two things move together. It cannot say why. Hold back on 'because'."

7 **Bars: aggregate, sort, label (3m). Say:** "Which store sold the most in 2025? One number per group, so: groupby first, then hand the names and the numbers to `ax.bar`."

```
in_2025 = daily[daily["date"].dt.year == 2025]
by_store = in_2025.groupby("store")["revenue"].sum()
by_store = by_store.sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(7, 3.8), layout="constrained")
bars = ax.bar(by_store.index, by_store.values, color="#667eea")
ax.bar_label(bars, fmt="${:,.0f}")
ax.set_title("Store revenue in 2025")
ax.set_ylabel("Revenue (USD)")
plt.show()
print(len(bars), "bars")
```

```
4 bars
```



What the room should see: four sorted bars with their values written on them.

Sorting turns the chart into a ranking; `bar_label` saves the reader a trip to the axis. The `1e6` in the corner means "times one million": Session 9 replaces it with dollar labels. **Bars start at zero.** A bar's length is the value; never cut the axis.

8 **The bar chart that is wrong and looks fine (3m, never cut). Say:** "Now the mistake everyone makes once. Skip the groupby."

```
fig, ax = plt.subplots(figsize=(6, 3), layout="constrained")
bars = ax.bar(daily["store"], daily["revenue"])
print(len(bars), "bars drawn")
print(daily.groupby("store")["revenue"].max())
plt.close(fig)
```

```
2903 bars drawn
store
Asheville    16558.00
Bend         20935.21
Boise        27677.10
Flagstaff    18690.57
Name: revenue, dtype: float64
```

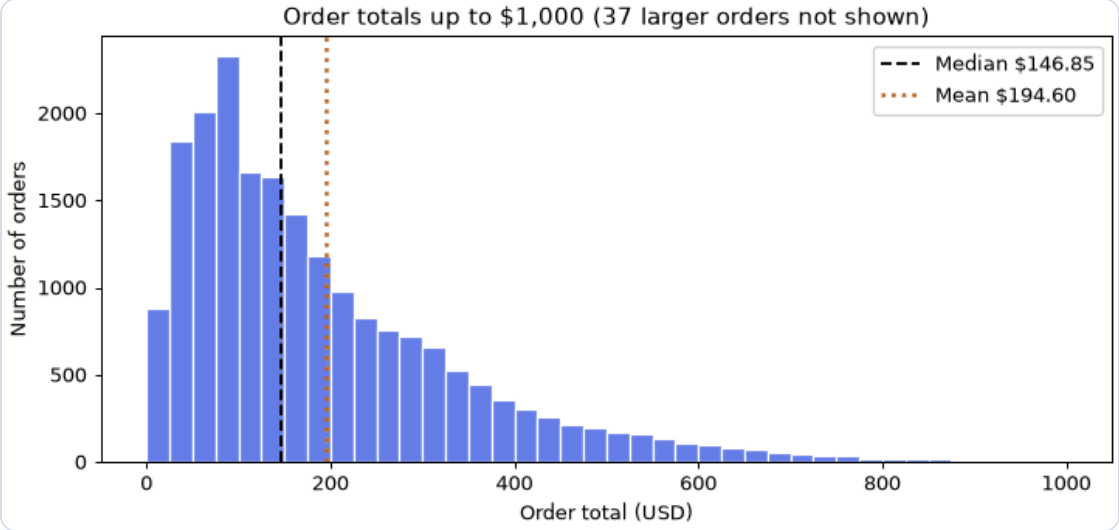
Show the chart before you close it: four tidy bars of about $20,000. **Ask:** "What are these four numbers?" Not totals, not means. Matplotlib drew 2,903 bars stacked at four x positions and you see the tallest in each pile: each store's single best day. **Say and write:** "`ax.bar` never aggregates. One number per bar, and you compute it."

9 **Histogram: the chart that does count (4m). Say:** "What does a typical order cost, and how much do orders vary? One numeric column, many rows, no groupby: the histogram does the counting."

```
orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
totals = orders["order_total"]
above = int((totals > 1000).sum())

fig, ax = plt.subplots(figsize=(8, 3.8), layout="constrained")
counts, edges, patches = ax.hist(totals, bins=40, range=(0, 1000),
                                 color="#667eea", edgecolor="white")
ax.axvline(totals.median(), color="black", linestyle="--",
           label=f"Median ${totals.median():.2f}")
ax.axvline(totals.mean(), color="#d95f02", linestyle=":", linewidth=2,
           label=f"Mean ${totals.mean():.2f}")
ax.set_title(f"Order totals up to $1,000 ({above} larger orders not shown)")
ax.set_xlabel("Order total (USD)")
ax.set_ylabel("Number of orders")
ax.legend()
plt.show()
print(above, edges[:4], counts[:3])
```

```
37 [ 0. 25. 50. 75.] [ 877. 1843. 2009.]
```



What the room should see: a hump at small orders, a long right tail, the mean to the right of the median.

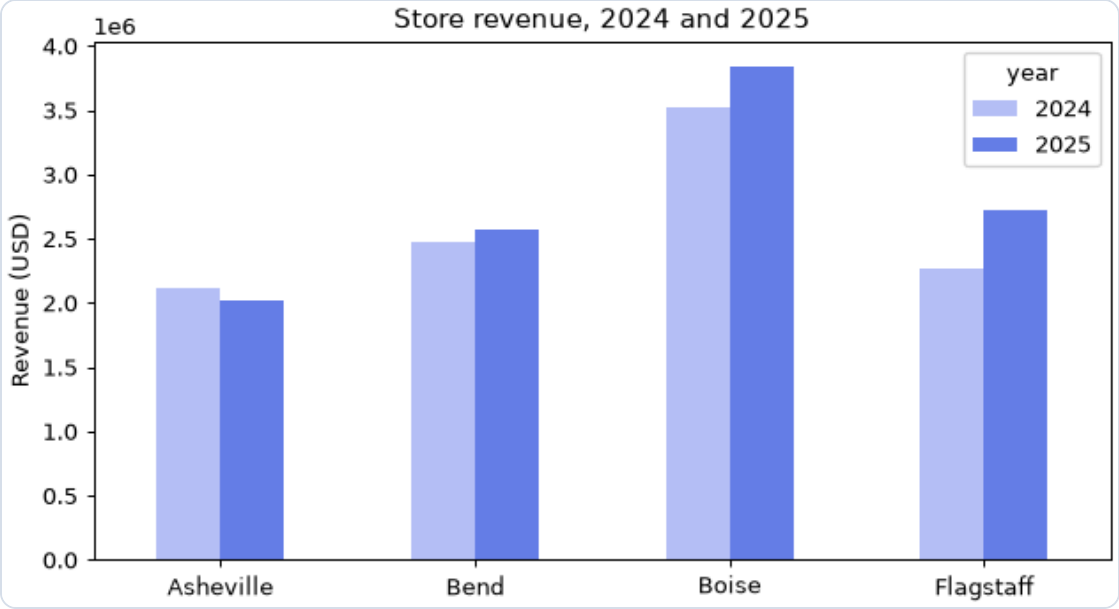
Type it first with only `bins=40`, no `range`: three fat bars and an axis to $5,000. "The chart is not broken. A handful of bulk orders stretch the range." Then add `range=(0, 1000)` **and say so in the title**. The mean and the median are almost $50 apart; the tail drags the mean. "Session 10 is about exactly that difference."

10 **Choosing a chart (2m).** Show the lesson's choosing-a-chart table and read the four questions aloud: **How did it change over time?** Line. **Do two numbers move together?** Scatter. **Which group is bigger?** Bar. **What values are typical, and how spread out?** Histogram. **Say:** "If you cannot finish the sentence 'This chart shows whether...', you are not ready to choose a chart yet. Go back to the data." Then two quick calls from the room: "Orders per weekday?" (bar, in calendar order) and "Temperature against revenue?" (scatter).

11 **The pandas shortcut (3m, cut first). Say:** "`df.plot(ax=ax)` is a shortcut to the same Matplotlib calls. It is at its best for grouped bars."

```
daily["year"] = daily["date"].dt.year
yearly = daily.pivot_table(index="store", columns="year", values="revenue",
                           aggfunc="sum")
fig, ax = plt.subplots(figsize=(7, 3.8), layout="constrained")
returned = yearly.plot.bar(ax=ax, rot=0, color=["#b8c1f5", "#667eea"])
ax.set_title("Store revenue, 2024 and 2025")
ax.set_xlabel("")
ax.set_ylabel("Revenue (USD)")
plt.show()
print(returned is ax)
```

```
True
```



What the room should see: two bars per store, lighter for 2024, darker for 2025.

Stores as rows, years as columns: one cluster per row. It returns the same Axes, so everything from today still applies afterward. **Ask:** "Which store shrank?" Asheville: its darker bar is the shorter one. A lighter and a darker shade of one color say "same measure, earlier and later".

THREE MISTAKES LEARNERS WILL MAKE, AND HOW TO COACH EACH

- **Bars from rows that were never aggregated.** `ax.bar(df["store"], df["revenue"])` draws one bar per row, stacked at each x position; the visible height is each group's largest single value. No error. Coach: "How many bars did you draw? `len(bars)`." Then: groupby first, one number per bar. (The opposite slip is running a groupby before `ax.hist`: the histogram wants the raw rows.)
- **Mixing the two styles.** `ax.title("...")` raises `TypeError: 'Text' object is not callable`; `ax.xlabel(...)` raises `AttributeError`. Coach: "On an Axes the verbs start with `set_`." Point at the translation table. A learner who writes `plt.title(...)` gets no error today; tell them it stops working next session with four panels.
- **Dates or numbers that are still text.** The x axis becomes a solid black band (one tick per row: 727 of them) and the drawing is slow, or the points are in file order instead of date order. No error. Coach: "What is `df["date"].dtype`?" Fix at load time with `parse_dates`. A related error when x and y come from different filters: `ValueError: x and y must have same first dimension`.

GUIDED LAB: FACILITATION NOTES

The lab: `labs/module-08/l3_1-line-scatter-bar/four_questions_starter.py` (your reference: `four_questions_solution.py`). 25 minutes. Four questions about customers and products, each answered by one chart. The learner chooses the chart type **before** reading the TODO. Charts are saved to an `output/` folder next to the lab file; the file never calls `plt.show()`.

How learners see their charts: in VS Code, run the file cell by cell (`# %%`) and each figure appears in the interactive window; or run the whole file and open the PNG files in `output/`. Say this before they start, or half the room will wait for a window that never opens.

Minute 0 to 12 (Questions 1 and 2): the line of new accounts per year has 7 points and peaks in 2024 (838). Watch for a learner who draws bars: not wrong for seven years, so ask what they would do with 84 months. The channel chart has 5 horizontal bars; `sort_values()` (smallest first) puts the longest bar on top, because `barh` draws the first row at the bottom. Organic leads with 1,517.

Minute 12 to 22 (Questions 3 and 4): the scatter has 228 dots: 14 products have no weight, and Matplotlib drops them without a word. **Ask:** "How many dots did you expect? How would a reader know 14 are missing?" (The reference solution says so in the title.) Correlation 0.69. The price histogram has 30 bins of about $20; the median is $82.95 and 21 products cost more than $300: the same long right tail as the order totals.

Debrief (3 min): put one learner's four PNGs on the projector and ask the room for each chart: "What question does it answer?" and "What would you change?" Collect the "what would you change" answers: they are next session's agenda (dollar labels, a title that states the finding, color).

If the room finishes early: `labs/module-08/l3_1-line-scatter-bar/chart_repair_starter.py`. It runs with no error and draws three wrong charts: 1,145 bars for 4 payment methods, a line straight through two unrecorded days, and an x axis with 731 tick labels. Each has a printed clue.

STARTING THE LESSON PROJECT

Does Rain Keep Shoppers Away? (lesson page, about 35 minutes solo). In the room, 10 minutes: read the brief aloud (store managers believe rainy days are slow days; check it with all four chart types), create `lesson_3_1_project`, run **Step 1**. Done when the output shows `(2903, 13)` and a rainy share of 0.134.

Point at the merge in Step 1: the key is called `store` in one file and `city` in the other, so it needs `left_on` and `right_on`; `validate="one_to_one"` is the habit from Session 6.

Steps 2 to 6 are homework. Check numbers: Asheville June 2025 has 10 rainy days of 30; medians of 262 (dry) and 228 (rainy) visitors; a drop of -17.6% to -26.3% across the four stores. Remind them that Step 6 asks for a sentence that includes one thing the charts do **not** prove.

CHECKS FOR UNDERSTANDING (ASK OUT LOUD)

1. What is the difference between a Figure and an Axes? **Answer:** The Figure is the whole canvas and is what gets saved. An Axes is one chart on it, with its own x and y axis, title and data.
2. Which of `ax.bar` and `ax.hist` does the counting for you? **Answer:** `ax.hist`: give it the raw rows. `ax.bar` needs one number per bar that you computed.
3. Orders per day of the week: sort the bars by value? **Answer:** No. Weekdays have a natural order that readers expect. Sort by value only when the categories have no order of their own.
4. You see `plt.xlabel("Visitors")` in a tutorial. What do you write? **Answer:** `ax.set_xlabel("Visitors")`.

Workbook self-check, items Q7 to Q12:

Answers: **Q7** B) A scatter plot: one dot per product, review count against rating; **Q8** `5 1`; **Q9** A) `ax.set_title("Revenue")`; **Q10** C) `range=(0, 1000)`, and say in the title how many orders are not shown; **Q11** `[1.0, 5.0, 0.0, 0.0] 5`; **Q12** D) A smaller `s` and an `alpha` below 1. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Hand out the five-line pattern on a slip of paper (`fig, ax = plt.subplots(...)`, one drawing call, `set_title`, one axis label, `savefig`) and have the learner fill in the drawing call for each question. For Question 3, let them plot without `alpha` first and then add it, so they see what it does.

Stretch: Lesson 3.1's bonus challenge (answer one question with all four chart types and judge which answer it). Or, in the lab, a fifth question with an honest null result: "Do pricier products get better ratings?" (A scatter of `price` against `rating`; the correlation is -0.00.) What does a chart of "no relationship" look like, and is it worth showing?

CLOSING THE SESSION

2:50 Wrap-up — 10 min

EXIT TICKET (3 QUESTIONS, WITH ANSWERS)

1. Before you build any feature, what is the first thing you decide? **Answer:** The unit of analysis: what one row of the table represents (a customer, an order, a store-day). Every feature must be one fact about that unit.
2. State the cutoff rule in one sentence. **Answer:** For every feature: could I have computed this value on the cutoff date, using only what existed then? If not, it leaks.
3. Name the chart for each question: (a) how did it change over time, (b) do two numbers move together, (c) which group is bigger, (d) what values are typical and how spread out are they? **Answer:** (a) line, `ax.plot`; (b) scatter, `ax.scatter`; (c) bar, `ax.bar` or `ax.barh`, after aggregating; (d) histogram, `ax.hist`, on the raw rows.

Reading the tickets: for question 1, "load the data" or "choose the features" means the unit of analysis has not landed; start Session 9 by asking what one row is in the nine-panel project. For question 2, accept any wording with "on that date" and "only what existed then". For question 3, the common slip is (c) and (d) swapped: bar and histogram look alike and answer different questions. Those learners should redo self-check Q8 and Q11.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- Finish the Lesson 2.9 project, **The Customer Feature Table** (Steps 2 to 6). Check: `(4474, 19)` after Step 3, a target rate of 0.569, and `customer_features.csv` with 20 columns.
- Finish the Lesson 3.1 project, **Does Rain Keep Shoppers Away?** (Steps 2 to 6). Check: a rainy-day drop at all four stores, between -17.6% and -26.3%, and a written finding that says one thing the charts do not prove.
- Lesson 2.9: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **Scaling by Hand**: it was only mentioned in the room.
- Lesson 3.1: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **What a Line Does with Missing Days** and **Points with a Range**.
- Kit labs not reached in the room: `labs/module-08/l2_9-feature-engineering/store_day_features_starter.py` and `labs/module-08/l3_1-line-scatter-bar/chart_repair_starter.py` (about 20 minutes each).
- From Lesson 3.1, "Before the Next Lesson": find one pyplot-style example (`plt.plot`, `plt.title`) and rewrite it with `fig, ax = plt.subplots()`.

BRING LIST FOR SESSION 9

- Their Lesson 3.1 notebook: Lesson 3.2 uses the `daily` and `monthly` tables again.
- `customer_features.csv` and the other Module 2 outputs stay in the project folder (they return in Module 4).
- The pyplot-style example they rewrote, to show a neighbor.
- The course data folder with `daily_sales.csv`, `store_weather.csv`, `orders.csv`, `order_items.csv` and `products.csv`.
- Nothing new to install. No servers, no accounts, no internet needed.

PREVIEW TO SAY

Next session one Figure holds several Axes. You will put the four stores side by side on a shared scale, build an uneven layout by drawing it as text, and see why a chart with two different y scales can be made to say anything. Then you take a correct chart and make it clear: a title that states the finding, axes people can read, and color that points at what matters. Holding `ax` in a variable, as you practiced today, is what makes all of it easy.

IF YOU RUN LONG OR SHORT

IF YOU RUN LONG: CUT IN THIS ORDER

1. **2.9 demo step 7** (`qcut`): one sentence, " `qcut` picks the edges so each group is the same size; the lesson has the comparison table". Saves 2 minutes.
2. **2.9 demo step 11** (one-hot): show the lesson's output instead of typing. Saves 2 minutes. Self-check Q5 covers it.
3. **3.1 demo step 11** (the pandas `.plot` shortcut): say " `df.plot(ax=ax)` is a shortcut to the same calls; the lesson has the table". Saves 3 minutes.
4. **3.1 demo step 2** (figure and axes are objects): fold it into step 1 as one spoken sentence. Saves 1 minute.
5. **Guided labs:** stop `order_features` after Step 6 (Step 7 is one spoken question); stop `four_questions` after three charts and give the fourth as homework.
6. **Project starts:** as a last resort, run Step 1 yourself on the projector.

Never cut: 2.9 demo steps 9, 10 and 12; 3.1 demo steps 1 and 8.

IF YOU RUN SHORT

Run the second lab of the lesson you are in. `store_day_features` ends in a debate worth having: is `staff_on_duty` a leak? (It is known the evening before, so it passes the cutoff rule, but it is somebody's forecast written into a column.) `chart_repair` is three charts that draw without an error and are wrong: good pair work.

Or continue a lesson project together: 2.9 Step 5 (the leak audit) is the most valuable ten minutes; 3.1 Step 2 (a line with the rainy days marked) is the most satisfying.

Or play "name that chart": read out five questions about Pinecrest and have the room call out line, scatter, bar or histogram, and the pandas step that prepares it.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 9: Composing and Customizing Figures

How to teach the session where charts become figures for other people. Before the break, learners put several charts on one page: grids returned as arrays, shared scales chosen on purpose, uneven layouts drawn as a picture of names, and the twin-axis chart that can be made to say anything. After the break they edit one chart from Matplotlib's first draft to a finished figure, one decision at a time, and capture the repeated decisions in a house style.

2 lessons | 3 hrs session | Lessons 3.2-3.3 | Session 9 of 21

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: **Frame** → **Live demo** (you type, they type along) → **Guided lab** (new practice from the kit) → **Start the lesson project** (everyone runs Step 1; the rest is homework).
- **Type, don't paste**, with one exception: the six small functions of the 3.3 demo build on each other, so if the room falls behind, paste the earlier ones from this guide and type the new one. Every output and every small picture below is a real run (pandas 3.0, Matplotlib 3.11).
- **The lessons are the reference.** Keep `matplotlib_subplots_composition.html` and `matplotlib_customizing_plots.html` open for the subplots-shape table, the mosaic table, the palette table and the formatter table.
- **Labs are new practice, not the lesson projects.** Starters and solutions are in `labs/module-09/`. Every lab saves PNG files to `output/` next to the lab file.
- Teal blocks are hands-on. Amber blocks are watch-outs.

> **THE RISK OF SESSION 9: CHARTS THAT PERSUADE INSTEAD OF INFORM**
>
> Both lessons are about the same power: the person who draws the chart decides what the reader sees first. Separate y scales make a small store look as big as a large one. A twin axis can make two lines hug or fly apart without changing a number. A finding title can claim more than the data shows; a cut-off bar axis can quadruple a difference. Write the two questions on the board and come back to them all session: **"Could I redraw this to tell a different story without changing the data?"** and **"Did I check the headline with a number?"**

BEFORE SESSION 9

- ☐ Run `python3 labs/check_labs.py module-09` from the kit folder: 8 lines, all PASS.
- ☐ Copy the four `*_starter.py` files from `labs/module-09/` to the shared folder (keep the folder structure and `labs_paths.py` one level above the session folder). Keep the `*_solution.py` files on your machine.
- ☐ Projector notebook `session09_demo` in the course folder, font size 18 pt or larger. First cell already run: `import matplotlib as mpl`, `import matplotlib.dates as mdates`, `import matplotlib.pyplot as plt`, `import numpy as np`, `import pandas as pd`, `from matplotlib.ticker import FuncFormatter, PercentFormatter`, and the two `pd.set_option` lines.
- ☐ Have the six 3.3 helper functions (`new_chart`, `draw_lines`, `add_titles`, `format_axes`, `label_line_ends`, `annotate_story`, `clean_up`) in a text file you can paste from, in case the room falls behind.
- ☐ Browser tabs: Lesson 3.2, Lesson 3.3, and a news site with a twin-axis chart if you can find one this week (the lesson's "Before the Next Lesson" asks learners to look for one).
- ☐ Board, before learners arrive: **Could I redraw this to tell a different story without changing the data?** and **Did I check the headline with a number?**
- ☐ Files used today: `daily_sales.csv`, `store_weather.csv`, `orders.csv`, `order_items.csv`, `products.csv`. No servers, no internet, no accounts.
- ☐ Sticky notes for the exit ticket.

KNOWN ENVIRONMENT TRAPS

Small screens. A 3 by 3 grid at `figsize=(9, 6.5)` is unreadable on a laptop at 125% zoom in a narrow notebook pane. If learners squint, have them widen the pane or drop to `figsize=(8, 6)`; do not shrink fonts.

`tight_layout` **from old habits.** Calling `fig.tight_layout()` on a figure made with `layout="constrained"` prints a warning that the layout engine changed. Use one or the other; this course uses constrained.

Fonts. Text on the projector machine and on learner machines may use different fonts. DejaVu Sans ships with Matplotlib and is always available; nobody needs to install anything.

Charts in VS Code cells vs files. The kit labs save PNGs and never call `plt.show()`. Run them cell by cell in VS Code to see each figure, or open the files in `output/`.

SESSION 9 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 8 (features and first charts), no notes. Homework check: the rain finding, and the pyplot example rewritten.
0:10	10	**3.2** Frame and mental model	Panels on a page: the Figure is the page, each Axes a panel.
0:20	30	**3.2** Live demo	A grid and its array, shared scales, a named layout, the twin-axis trap and what to do instead, small multiples.
0:50	25	**3.2** Guided lab	`labs/module-09/l3_2-subplots-composition/traffic_panels_starter.py`: foot traffic on one page.
1:15	10	**3.2** Start the lesson project	Nine Categories, Twelve Months: everyone runs Step 1 and sees `(12, 9)`.
1:25	10	Break	Leave the two twin-axis charts of the same data on the projector.
1:35	10	**3.3** Frame and mental model	You are the editor: a default chart is a first draft.
1:45	30	**3.3** Live demo	One chart edited six times: finding title, formatters, gray plus one, direct labels, annotations, clean-up; then a house style.
2:15	25	**3.3** Guided lab	`labs/module-09/l3_3-customizing-plots/finish_chart_starter.py`: from first draft to finished chart.
2:40	10	**3.3** Start the lesson project	The Footwear Returns Chart: everyone runs Step 1 and sees Footwear at 0.144.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 10.
Total	**180**		Ends at 3:00.

The lesson pages estimate 85 + 90 minutes for a solo learner. After the warm-up, break and wrap-up the room has 150, and 50 of those go to the two kit labs. The room does not cover either lesson whole.

IN THE ROOM, AT HOME, WHAT TO PROTECT

In the room: both mental models; both live demos; the two guided labs (`traffic_panels` for 3.2, `finish_chart` for 3.3); Step 1 of each lesson project, run by everyone.

At home: the rest of both lesson projects (3.2 Steps 2 to 5, 3.3 Steps 2 to 5); each lesson's exercises, Quick Quiz and journal; the two second labs (`honest_axes` , `color_jobs`); and the sections the room only names: layout without the constrained engine and `width_ratios` in 3.2; the heatmap, the palette strip and ready-made style sheets in 3.3.

Protect: in 3.2, shared scales (demo step 4) and the twin-axis pair (steps 8 and 9). In 3.3, the finding title checked with a number (steps 2 and 3) and gray plus one with direct labels (step 6). Session 10 opens by saving exactly this kind of finished chart.

Cut first if you are running long: see the last section of this guide for the order.

WARM-UP

0:00 Warm-up: Retrieval from Session 8 — 10 min

HOW TO RUN IT

Closed notebooks. Read each question aloud, 30 seconds of silent writing, one answer from the room, then yours. Questions 4 and 5 are today's foundation: if the room hesitates on Figure versus Axes, draw the page-and-panels picture now, before the 3.2 frame.

FIVE RETRIEVAL QUESTIONS (WITH ANSWERS)

1. Before you build a feature table, what is the first thing you decide? **Answer:** The unit of analysis: what one row represents. Every feature must be one fact about that unit.
2. After a left merge of per-customer summaries, which `NaN` values become 0 and which stay missing? **Answer:** Counts and sums become 0 (true statements). Averages, shares and recency stay missing; add a flag such as `has_ordered`.
3. State the cutoff rule. **Answer:** For every feature: could I have computed this value on the cutoff date, using only what existed then? If not, it leaks.
4. What is the difference between a Figure and an Axes? **Answer:** The Figure is the whole canvas, the thing that gets saved. An Axes is one chart on it, with its own x and y axis.
5. `ax.bar(daily["store"], daily["revenue"])` draws four bars and no error. What are their heights? **Answer:** Each store's single highest day: 2,903 bars pile up at four positions and the tallest shows. `ax.bar` never aggregates; group first.

HOMEWORK CHECK

Two quick checks. First, ask two volunteers to read their rain finding from the Lesson 3.1 project aloud, including the sentence about what the charts do **not** prove. Praise the second sentence more than the first: today's second half is about exactly that honesty. Second: "Who rewrote a pyplot example? What did `plt.title` become?" (`ax.set_title`.) Then: "Today that habit stops being style and starts being necessary. With four charts on a page there is no 'current' chart."

3.2 Subplots and Composition

75 min in the room

OBJECTIVES (THE LESSON'S OWN)

- Create a grid of Axes with `plt.subplots(nrows, ncols)` and reach each one by index or in a loop
- Decide when panels should share an x or y axis, and explain what the reader loses when they do not
- Build an uneven layout with `plt.subplot_mosaic()` and address its panels by name
- Explain why a chart with two different y scales can mislead, and replace it with stacked panels or a ratio
- Draw small multiples from grouped data with one function and one loop

TIMING

10m	Frame: panels on a page; real questions have several answers
30m	Live demo, steps 1 to 12 (steps 5 and 10 are the first to cut; 4, 8 and 9 never)
25m	Guided lab: `traffic_panels` (foot traffic on one page)
10m	Start the lesson project: Step 1 of Nine Categories, Twelve Months

THE MENTAL MODEL TO SAY

Say: "Think of a newspaper's front page. The page is one object, divided into panels, and the arrangement tells you how to read them: panels in a row invite comparison, a wide panel across the top is the headline, smaller panels underneath are the detail. In Matplotlib the page is the Figure and each panel is an Axes. You already draw on an Axes. Today is about arranging them."

Draw the lesson's decision on the board: **Same chart for each group?** Yes → a grid (`plt.subplots`), and share the axes so sizes compare. No, a mix → a layout (`subplot_mosaic`), with the biggest panel for the main point.

Say: "With several Axes there is no single 'current chart' any more. That is why we have held `ax` in a variable since the first chart."

LIVE DEMO: YOU TYPE, THEY FOLLOW (12 STEPS, EVERY OUTPUT BELOW IS A REAL RUN)

Type each block in its own cell of `session09_demo`. Times in brackets add up to 30 minutes. A picture under a step is what the room should see.

1. **The monthly table again (2m). Say:** "The same table as Session 8: one row per month, one column per store. Today it gets four panels."

```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
daily["month"] = daily["date"].dt.to_period("M").dt.to_timestamp()
monthly = daily.pivot_table(index="month", columns="store", values="revenue",
                            aggfunc="sum")
stores = list(monthly.columns)
print(stores, monthly.shape)
```

```
['Asheville', 'Bend', 'Boise', 'Flagstaff'] (24, 4)
```

2 **A grid is an array (2m). Say:** "Give `plt.subplots` rows and columns and the second thing it hands back is no longer one Axes."

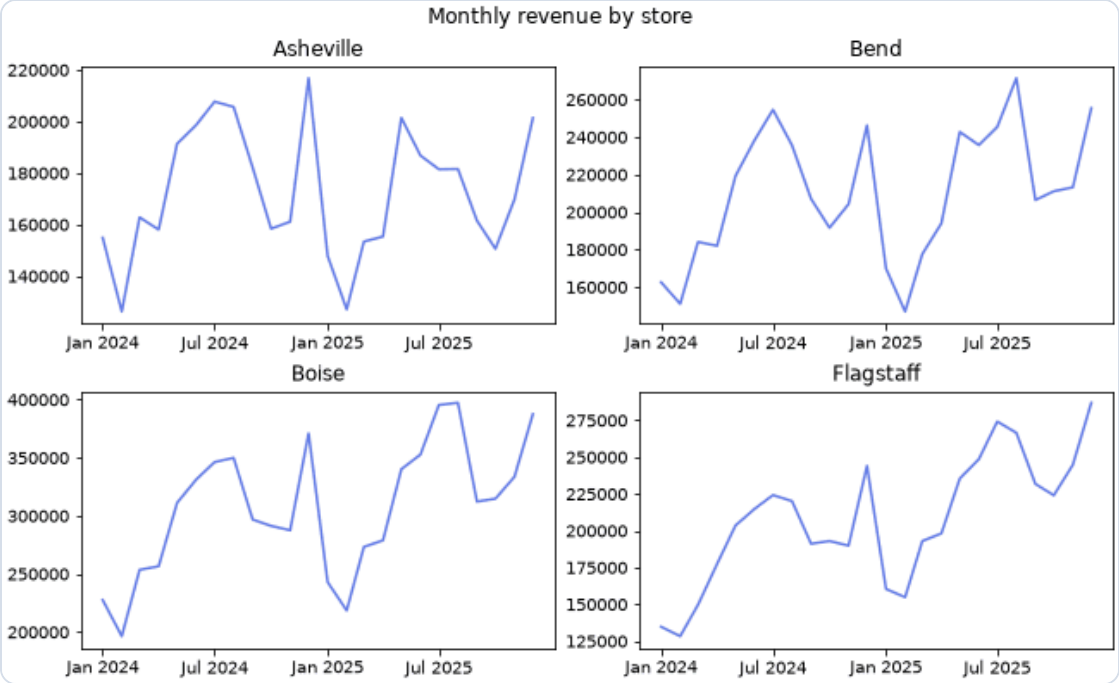
```
fig, axs = plt.subplots(2, 2, figsize=(9, 5.5), layout="constrained")
print(type(axs).__name__, axs.shape, type(axs[0, 1]).__name__, len(fig.axes))
plt.close(fig)
```

```
ndarray (2, 2) Axes 4
```

A NumPy array of shape (2, 2), every element an Axes. **Ask:** "Which panel is `axs[0, 1]`?" Row 0, column 1: top right. Everything from Lesson 1.1 about 2-D indexing applies.

3 **Loop, don't repeat (3m). Say:** "`axs.flat` walks the panels row by row; `zip` pairs each panel with a store. One loop fills any grid."

```
tick_dates = pd.to_datetime(["2024-01-01", "2024-07-01",
                             "2025-01-01", "2025-07-01"])
tick_labels = ["Jan 2024", "Jul 2024", "Jan 2025", "Jul 2025"]
fig, axs = plt.subplots(2, 2, figsize=(9, 5.5), layout="constrained")
for ax, store in zip(axs.flat, stores):
    ax.plot(monthly.index, monthly[store], color="#667eea")
    ax.set_title(store)
    ax.set_xticks(tick_dates, labels=tick_labels)
fig.suptitle("Monthly revenue by store")
plt.show()
```



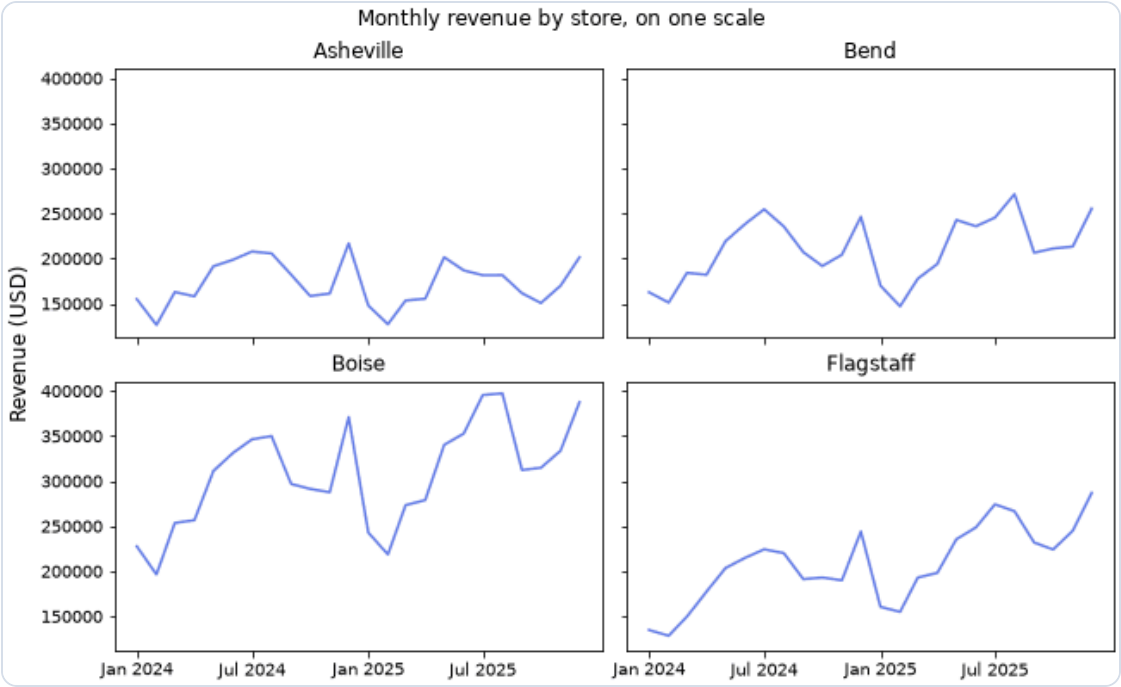
What the room should see: four panels, each with its own y scale.

Ask: "Which store is biggest?" Let someone answer from the picture. Then point at the y axes: Boise tops out near 400,000, Asheville near 220,000, and the lines reach the same height. **Say:** "Each panel chose a scale to fit its own data. For one chart that is right. For a comparison it lies."

4 **One scale for all four (3m, never cut). Say:** "`sharey=True`: one y scale for every panel. And one y label for the whole figure."

```
fig, axs = plt.subplots(2, 2, figsize=(9, 5.5), sharex=True, sharey=True,
                        layout="constrained")
for ax, store in zip(axs.flat, stores):
    ax.plot(monthly.index, monthly[store], color="#667eea")
    ax.set_title(store)
    ax.set_xticks(tick_dates, labels=tick_labels)
fig.suptitle("Monthly revenue by store, on one scale")
fig.supylabel("Revenue (USD)")
plt.show()
print(axs[0, 0].get_ylim() == axs[1, 1].get_ylim())
```

```
True
```



What the room should see: one scale, inner tick labels removed, Boise visibly biggest.

Now height means the same thing everywhere, and Matplotlib dropped the inner tick labels. **Say:** "Sharing is a decision, not a default. Shared answers 'which is bigger?'. Separate answers 'what shape does each have?'. Choose for the question, and when you do not share, say so in the title."

5 **What shape comes back? (2m, cut first). Say:** "The one surprise: the type you get depends on the grid."

```
f1, a = plt.subplots(1, 3)
f2, b = plt.subplots()
f3, c = plt.subplots(1, 3, squeeze=False)
print(a.shape, type(b).__name__, c.shape)
plt.close("all")
```

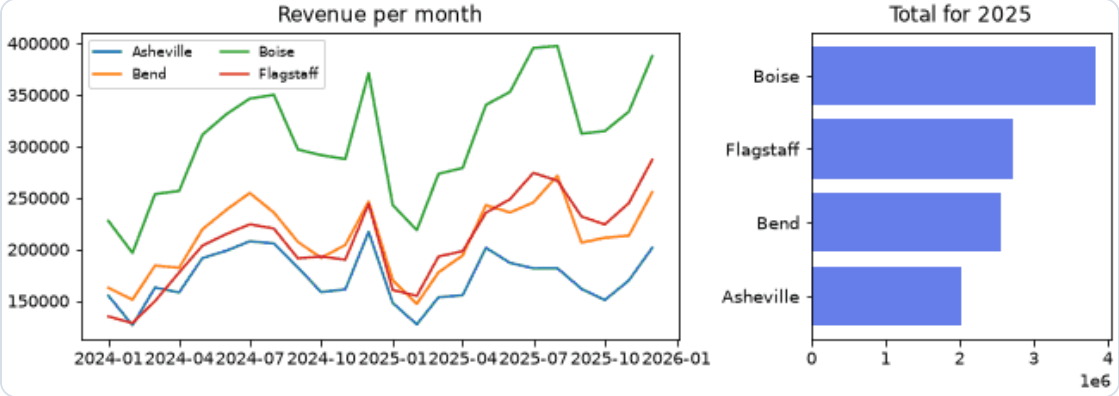
```
(3,) Axes (1, 3)
```

One row: a 1-D array, one index. One panel: a single Axes, no array. `squeeze=False`: always 2-D. `axs.flat` works on all of them, which is another reason to loop with it.

6 **Two panels, unequal widths (2m). Say:** "A one-row grid unpacks straight into names. `width_ratios` gives the main chart more room."

```
y2025 = daily[daily["date"].dt.year == 2025]
totals_2025 = y2025.groupby("store")["revenue"].sum().sort_values()
fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.2), width_ratios=[2, 1],
                                  layout="constrained")
for store in stores:
    left.plot(monthly.index, monthly[store], label=store)
left.set_title("Revenue per month")
left.legend(ncols=2, fontsize=8)
right.barh(totals_2025.index, totals_2025.values, color="#667eea")
right.set_title("Total for 2025")
plt.show()
print(round(left.get_position().width / right.get_position().width, 1))
```

```
2.0
```



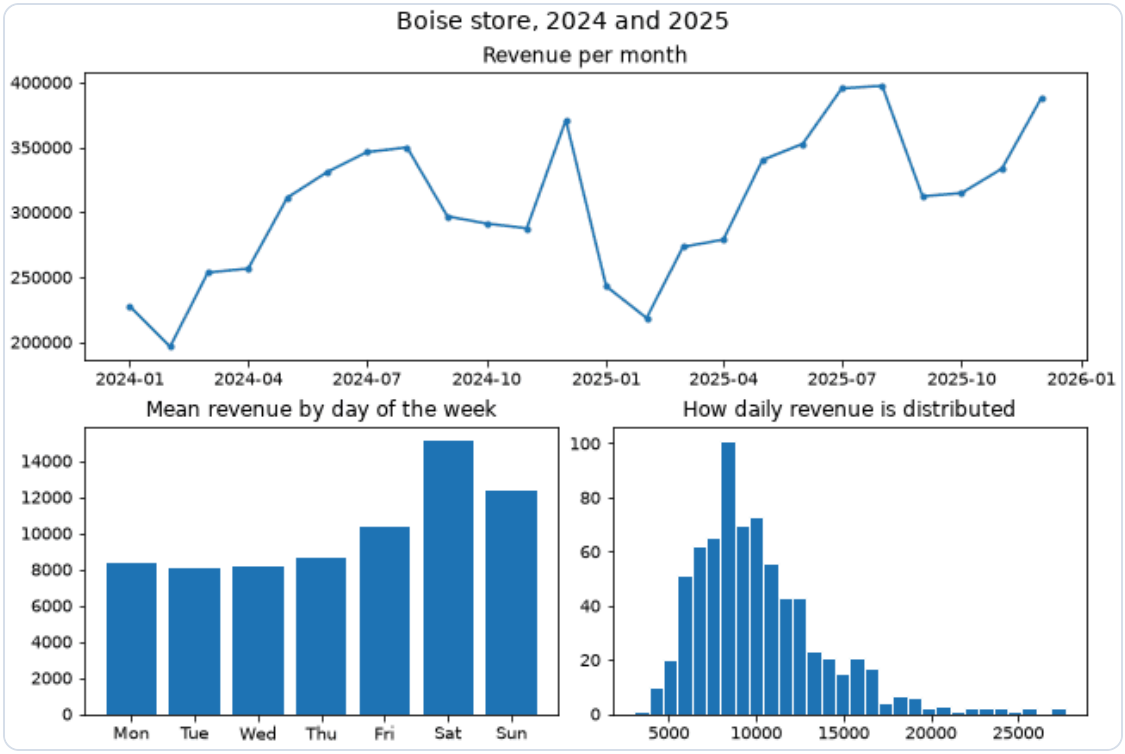
Two chart types on one figure; the time series gets twice the width.

`left` and `right` read better than `axs[0]` and `axs[1]`. The printed ratio confirms 2 to 1.

7 **A layout drawn as a picture (4m). Say:** "Real summary figures are not regular grids. `subplot_mosaic` lets you draw the layout with names. A name repeated across cells spans them."

```
fig, axd = plt.subplot_mosaic([["trend", "trend"],
                               ["week", "spread"]],
                              figsize=(9, 6), layout="constrained")
print(type(axd).__name__, list(axd))
boise = daily[daily["store"] == "Boise"]
by_weekday = boise.groupby(boise["date"].dt.dayofweek)["revenue"].mean()
axd["trend"].plot(monthly.index, monthly["Boise"], marker="o", markersize=3)
axd["trend"].set_title("Revenue per month")
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
axd["week"].bar(days, by_weekday.values)
axd["week"].set_title("Mean revenue by day of the week")
axd["spread"].hist(boise["revenue"].dropna(), bins=30, edgecolor="white")
axd["spread"].set_title("How daily revenue is distributed")
fig.suptitle("Boise store, 2024 and 2025", fontsize=14)
plt.show()
```

```
dict ['trend', 'week', 'spread']
```



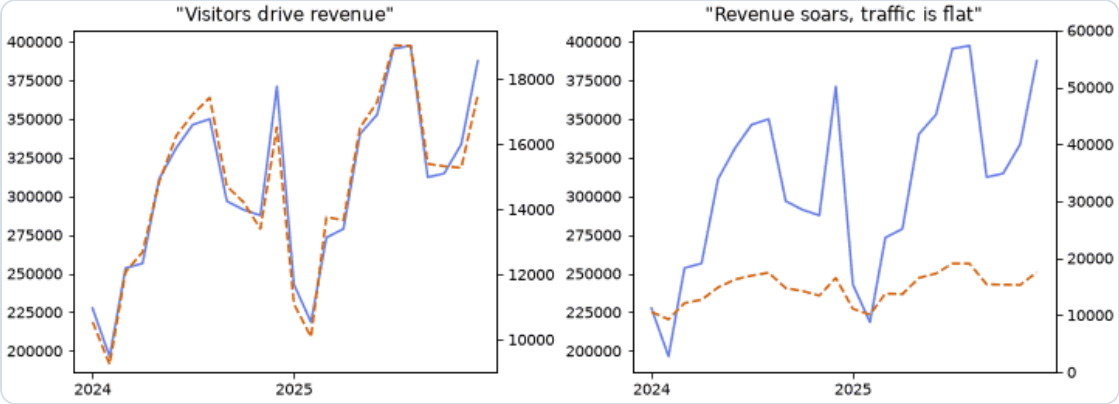
What the room should see: one wide headline panel, two supporting panels.

This time a dictionary comes back. The code that fills a panel says what it is: `axd["trend"]`, not `axs[0]`. Move the histogram by editing the picture; nothing else changes.

8 **The twin axis that says anything (4m, never cut). Say:** "Someone will ask you to put revenue and visitors on one chart. `twinx()` makes a second y axis on the right. Watch what I can do with it. Same data twice; only the right-hand limits differ."

```
bm = boise.groupby("month")[["revenue", "visitors"]].sum()
fig, axs = plt.subplots(1, 2, figsize=(10, 3.6), layout="constrained")
versions = [((9000, 19500), '"Visitors drive revenue"'),
            ((0, 60000), '"Revenue soars, traffic is flat"')]
for ax, (limits, claim) in zip(axs, versions):
    ax.plot(bm.index, bm["revenue"], color="#667eea")
    ax.set_xticks(pd.to_datetime(["2024-01-01", "2025-01-01"]),
                  labels=["2024", "2025"])
    right = ax.twinx()
    right.plot(bm.index, bm["visitors"], color="#d95f02", linestyle="--")
    right.set_ylim(limits)
    ax.set_title(claim)
plt.show()
print(len(fig.axes), "Axes: each panel has a twin")
```

```
4 Axes: each panel has a twin
```



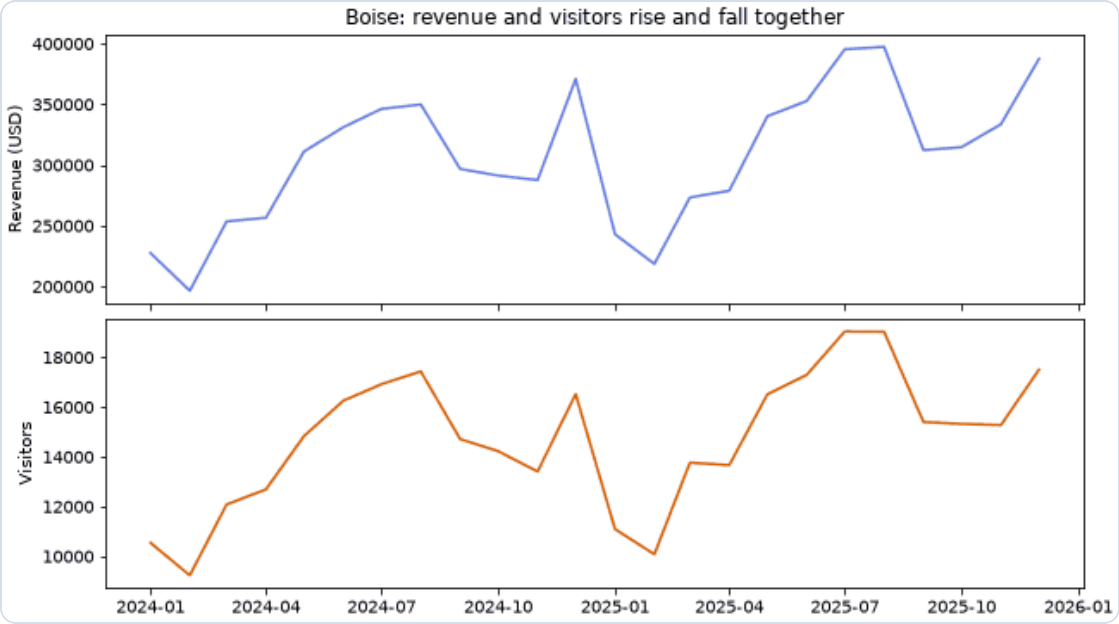
Identical data. Only the right-hand axis limits differ; each title is a claim the picture seems to support.

Leave this on the screen through the break. **Say:** "Neither chart has a wrong number. A crossing point on a twin-axis chart is where two unrelated rulers happen to line up." Point at the board question: could I redraw this to tell a different story? Here, yes, in one line.

9 **What to do instead (3m, never cut). Say:** "Stack the two measures and share the x axis. Or compute the relationship itself."

```
fig, (top, bottom) = plt.subplots(2, 1, figsize=(9, 5), sharex=True,
                                  layout="constrained")
top.plot(bm.index, bm["revenue"], color="#667eea")
top.set_ylabel("Revenue (USD)")
top.set_title("Boise: revenue and visitors rise and fall together")
bottom.plot(bm.index, bm["visitors"], color="#d95f02")
bottom.set_ylabel("Visitors")
plt.show()
per_visitor = bm["revenue"] / bm["visitors"]
print(per_visitor.min().round(2), per_visitor.max().round(2))
```

```
19.85 22.46
```



Each measure on its own honest scale; the months line up vertically.

Revenue per visitor stays between about $20 and $22.50 in every month: that answers the question more directly than any pair of lines. **The test for a second axis:** could I change one axis and make the chart tell a different story? If the two axes are tied by a formula (°F and °C, with `ax.secondary_yaxis`), a second axis is fine.

10 **One quantity, two units (1m, cut second). Say:** "The honest use of a second axis, in one line: show the lesson's Flagstaff temperature chart, with Fahrenheit on the left and Celsius on the right, tied together by `secondary_yaxis("right", functions=(f_to_c, c_to_f))`. Nobody chose those scales; a formula did."

11 **Small multiples with context (3m). Say:** "The four-store grid has a name: small multiples. The clean way to build them is a function that draws one panel on the Axes it is given, plus a loop."

```
def draw_store(ax, store):
    """One store in color, the other stores in gray behind it."""
    for other in stores:
        if other != store:
            ax.plot(monthly.index, monthly[other], color="lightgray", lw=1)
    ax.plot(monthly.index, monthly[store], color="#667eea", linewidth=2)
    ax.set_title(store)
    years = pd.to_datetime(["2024-01-01", "2025-01-01"])
    ax.set_xticks(years, labels=["2024", "2025"])

fig, axs = plt.subplots(1, 4, figsize=(10, 2.8), sharex=True, sharey=True,
                        layout="constrained")
for ax, store in zip(axs, stores):
    draw_store(ax, store)
fig.suptitle("Monthly revenue: each store against the others")
plt.show()
```



Every panel carries the full context; the one colored line says which store it is about.

Flagstaff climbing through the pack and Asheville sliding to the bottom are both easy to see. **Say:** "The function does not care where its Axes lives. That is the payoff of holding `ax` in a variable since Session 8. And gray for context, one color for the subject, is the first edit after the break."

12 **The bug the ax style prevents (2m). Say:** "Last session `plt.title` worked. Here is why it stops working."

```
fig, axs = plt.subplots(2, 2, layout="constrained")
for ax, store in zip(axs.flat, stores):
    ax.plot(monthly.index, monthly[store])
    plt.title(store)                    # pyplot style: which Axes is this?
print([ax.get_title() for ax in axs.flat])
plt.close(fig)
```

```
['', '', '', 'Flagstaff']
```

Three panels without a title and no error. `plt.title` writes on the "current" Axes, the last one created, four times over. **Say:** "When a grid misbehaves, print `type(axs)` and `axs.shape` first. Experienced people do exactly that."

THREE MISTAKES LEARNERS WILL MAKE, AND HOW TO COACH EACH

- **Plotting on the array.** `axs.plot(...)` raises `AttributeError: 'numpy.ndarray' object has no attribute 'plot'`; so does `axs[0].plot(...)` on a 2-D grid, because `axs[0]` is a whole row. Coach: "Print `type(axs)` and `axs.shape`. Which single panel do you mean?" Then `axs[0, 0]` or a loop over `axs.flat`.
- **Two indexes on a one-row grid.** `axs[0, 1]` after `plt.subplots(1, 3)` raises `IndexError: too many indices for array: array is 1-dimensional, but 2 were indexed`. Coach with the shape table: one row comes back 1-D. Fix: `axs[1]`, or `squeeze=False` when the grid size varies.
- **Comparing heights across unshared panels**, or a pyplot call inside the loop. Neither raises anything: the reader concludes the stores are the same size, or three panels lose their titles. Coach: "Is size the point of this figure? Then `sharey=True`." And: "Which Axes does `plt.title` write on?"

GUIDED LAB: FACILITATION NOTES

The lab: `labs/module-09/l3_2-subplots-composition/traffic_panels_starter.py` (your reference: `traffic_panels_solution.py`). 25 minutes. The demo used revenue; the lab uses mean visitors per day, so the learner rebuilds every figure from scratch on a new table and a new question: whose foot traffic is growing?

Minute 0 to 8 (TODO 1 and 2): the own-scales grid prints four different y ranges (Asheville roughly 203 to 341, Boise 303 to 629). Then the shared grid prints `True`. Walk the room during TODO 1 checking for a `suptitle` that **says** the scales differ: that is the habit, and most learners skip it.

Minute 8 to 18 (TODO 3): `draw_store(ax, table, store)` with gray context in a 1 by 4 row. Each panel should hold 4 lines. The common slip is drawing the colored line first, so the gray lines paint over it: the order of the calls is the drawing order (or pass `zorder=3`). Ask the room the lab's question: visitors grew 21.0% at Flagstaff and 9.2% at Boise from 2024 to 2025, 2.5% at Bend, and fell 7.5% at Asheville. Which design shows that best?

Minute 18 to 25 (TODO 4) and debrief: the Flagstaff mosaic: a wide trend, weekday bars (Saturday busiest at about 482 visitors, the quietest weekday about 252) and a histogram. `list(axd)` prints `['trend', 'week', 'spread']`. Put one learner's three PNGs on the projector: "Which panel is the headline? Does the layout say so?"

If the room finishes early: `labs/module-09/l3_2-subplots-composition/honest_axes_starter.py`: Asheville's visitors against the number of rainy days per month, first as two twin-axis charts of the same data, then honestly. The monthly correlation is only 0.28, but per day the difference is clear: 286.8 visitors on dry days against 236.4 on rainy days (-17.6%). The level at which you compare changes the answer, which is worth a minute of discussion.

STARTING THE LESSON PROJECT

Nine Categories, Twelve Months (lesson page, about 35 minutes solo). In the room, 10 minutes: read the brief (buyers need to know when each category has its season), create `lesson_3_2_project`, run **Step 1**. Done when the output shows `41788 lines on completed orders; 567 use discontinued products with no category`, then `(12, 9)` and the three-row Snow/Tents/Apparel table.

Point at the 567 lines: the six discontinued SKUs from Lesson 2.5 again. They drop out of a category chart because they have no category, and the project says so in its first print.

Steps 2 to 5 are homework. Check numbers: Accessories is the largest category over the year (594,912), Snow the smallest (221,470); in percent of its own year, Snow peaks in December at 28.0% and falls to 0.1% in July.

CHECKS FOR UNDERSTANDING (ASK OUT LOUD)

1. `fig, axs = plt.subplots(3, 1)`: how do you reach the bottom panel? **Answer:** `axs[2]` (or `axs[-1]`): one column comes back as a 1-D array.
2. Four store panels with separate y scales: what must the title or caption say? **Answer:** That each panel has its own scale, so heights cannot be compared.
3. What does `plt.subplot_mosaic` return for the panels? **Answer:** A dictionary keyed by the names in the layout picture.
4. When is a second y axis honest? **Answer:** When both sides show the same quantity in two units, tied by a formula (`secondary_yaxis`).

Workbook self-check, items Q1 to Q6:

Answers: **Q1** `(2, 3) 6 Axes`; **Q2** B) A one-row grid comes back as a 1-D array, so it takes one index: `axs[2]`; **Q3** C) When the reader should compare sizes across panels; separate scales show each shape but hide the differences in size; **Q4** `['A', 'B', 'C'] 3`; **Q5** D) Temperature in °F on the left and the same values in °C on the right, with `secondary_yaxis`; **Q6** A) Two empty frames with ticks from 0 to 1, until you remove them with `ax.remove()`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Let the learner sketch the grid on paper first, writing the index into each cell (`[0, 0]`, `[0, 1]`...). For TODO 3, give them the two-line skeleton of `draw_store` (the gray loop, then the colored line) and let them fill in the arguments.

Stretch: Lesson 3.2's bonus challenge (gray context inside the nine-category panels, and a different arrangement). Or, in the lab, add a fifth panel "All stores" to a 2 by 3 grid and remove the empty sixth with `ax.remove()`; or one shared figure legend with `fig.legend(..., loc="outside lower center")`.

3.3 Customizing Plots

75 min in the room

OBJECTIVES (THE LESSON'S OWN)

- Write a chart title that states the finding, with a subtitle that says what is plotted
- Format tick labels as money, percentages and readable dates with formatters and locators
- Use color for a purpose (highlight, category, magnitude) and choose palettes that work for color-blind readers
- Replace a legend with direct labels, and add annotations and shaded spans that point at the evidence
- Remove chart junk, and capture your choices in rcParams so every chart starts out styled

TIMING

10m	Frame: you are the editor; a default chart is a first draft
30m	Live demo, steps 1 to 11 (steps 5 and 10 are the first to cut; 2, 3 and 6 never)
25m	Guided lab: `finish_chart` (online revenue, 2024 against 2025)
10m	Start the lesson project: Step 1 of The Footwear Returns Chart

THE MENTAL MODEL TO SAY

Say: "The charts you made so far were for you: quick pictures to find out what the data says. A chart for someone else has a different job. It does not ask 'what is in here?'. It says 'here is what I found'."

Say: "Think of Matplotlib's default output as a first draft from a reporter: accurate, complete, and with no idea what the story is. Your job is the editor's: write the headline, cut what does not serve the story, and make sure the eye lands on the important part first. Customizing is editing, not decorating. Every change today answers one question: does this help the reader get the point faster?"

Point at the second board question: **Did I check the headline with a number?** "A finding title is a claim. Claims can be wrong."

LIVE DEMO: YOU TYPE, THEY FOLLOW (11 STEPS, EVERY OUTPUT BELOW IS A REAL RUN)

Continue in `session09_demo`. One chart is edited from step 1 to step 8, each edit in a small function that takes an `ax`. Times in brackets add up to 30 minutes.

1. **The first draft (3m). Say:** "Two helper functions, then Matplotlib's default chart. Look at it the way a stranger would."

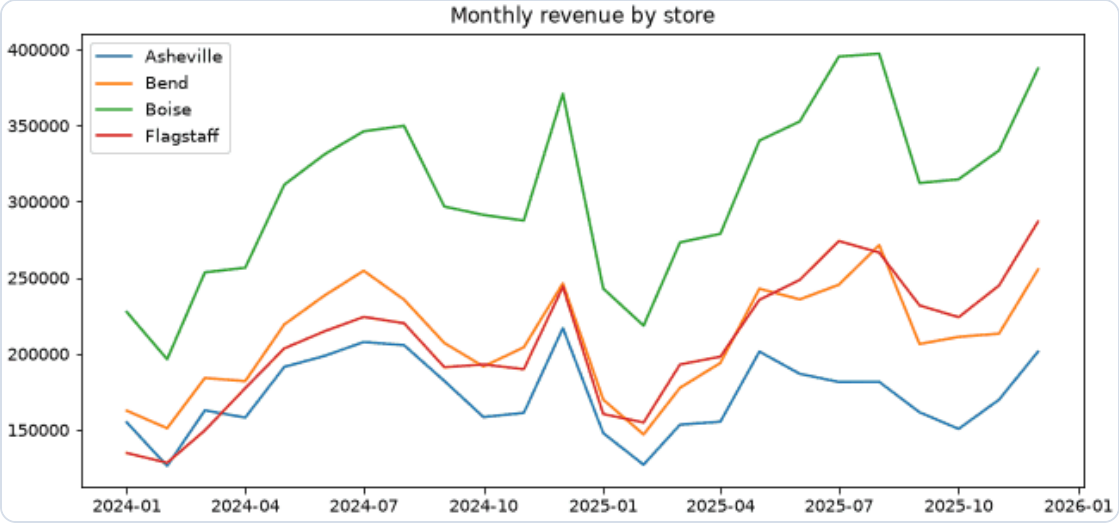
```
import matplotlib as mpl
import matplotlib.dates as mdates
from matplotlib.ticker import FuncFormatter

daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
daily["month"] = daily["date"].dt.to_period("M").dt.to_timestamp()
monthly = daily.pivot_table(index="month", columns="store", values="revenue",
                            aggfunc="sum")
stores = list(monthly.columns)

def new_chart():
    return plt.subplots(figsize=(9, 4.2), layout="constrained")

def draw_lines(ax, focus=None):
    """Four store lines; with focus, that store in color and the rest gray."""
    for store in stores:
        if focus is None:
            ax.plot(monthly.index, monthly[store], label=store)
        elif store == focus:
            ax.plot(monthly.index, monthly[store], label=store,
                    color="#667eea", linewidth=2.8, zorder=3)
        else:
            ax.plot(monthly.index, monthly[store], label=store,
                    color="0.6", linewidth=1.3)

fig, ax = new_chart()
draw_lines(ax)
ax.set_title("Monthly revenue by store")
ax.legend()
plt.show()
```



The first draft: accurate, complete, and no idea what the story is.

Collect what a stranger struggles with and write the list on the board: the title names the data but not the point; four equally loud colors; a legend to look up four times; the y axis says 150000 (dollars? units?); the year repeats on every label. **Say:** "Each of the next steps crosses one item off this list." (`draw_lines` already has a `focus` option; it is used in step 6.)

2. **Decide the story, and check it (3m, never cut). Say:** "In 3.2 we noticed Flagstaff climbing. Before that goes in a headline, check it with numbers."

```
print(monthly.iloc[0].sort_values().round(0))     # January 2024
print(monthly.iloc[-1].sort_values().round(0))    # December 2025
```

```
store
Flagstaff    134690.0
Asheville    154850.0
Bend         162534.0
Boise        227438.0
Name: 2024-01-01 00:00:00, dtype: float64
store
Asheville    201331.0
Bend         255359.0
Flagstaff    286876.0
Boise        387449.0
Name: 2025-12-01 00:00:00, dtype: float64
```

Flagstaff: smallest store in January 2024, second largest in December 2025. **Say:** "That is the story. A title that states it is a claim, and now I know the claim is true."

3. **A headline that states the finding (3m, never cut). Say:** "The title is the most-read text on any chart. Spend it on the finding. The description moves to a smaller subtitle."

```
def add_titles(ax, headline, subtitle):
    ax.set_title(headline, loc="left", fontsize=13, fontweight="bold", pad=24)
    ax.text(0, 1.03, subtitle, transform=ax.transAxes, fontsize=10,
            color="0.35")

headline = ("Flagstaff grew from Pinecrest's smallest store "
            "to its second largest")
subtitle = "Monthly in-store revenue, January 2024 to December 2025"
fig, ax = new_chart()
draw_lines(ax)
add_titles(ax, headline, subtitle)
print(ax.get_title(loc="left"))
plt.close(fig)
```

```
Flagstaff grew from Pinecrest's smallest store to its second largest
```

Explain `transform=ax.transAxes` once: "(0, 1.03) means left edge, just above the top, as fractions of the Axes, whatever the data is." `"0.35"` is a gray level. Then the warning: "Flagstaff grew" is on the chart; "Flagstaff grew because of the new manager" is not. Keep causes out of titles.

4 **Axes people can read (3m). Say:** "A locator decides where ticks go; a formatter decides what they say. 250000 becomes $250K, 2024-07 becomes Jul 2024."

```python
def thousands(x, pos):
    return f"${x / 1000:,.0f}K"

def format_axes(ax):
    ax.yaxis.set_major_formatter(FuncFormatter(thousands))
    ax.xaxis.set_major_locator(mdates.MonthLocator(bymonth=[1, 7]))
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
    ax.set_ylim(bottom=0)

fig, ax = new_chart()
draw_lines(ax)
format_axes(ax)
fig.canvas.draw()
print([t.get_text() for t in ax.get_xticklabels()])
print([t.get_text() for t in ax.get_yticklabels()][:4])
plt.close(fig)
```

```
['Jan 2024', 'Jul 2024', 'Jan 2025', 'Jul 2025', 'Jan 2026']
['$0K', '$50K', '$100K', '$150K']
```

The simplest formatter is a string with a field named `x`: `"${x:,.0f}"`. When you need arithmetic, write a two-argument function and wrap it in `FuncFormatter`. The y axis starts at zero because the reader is meant to compare store sizes.

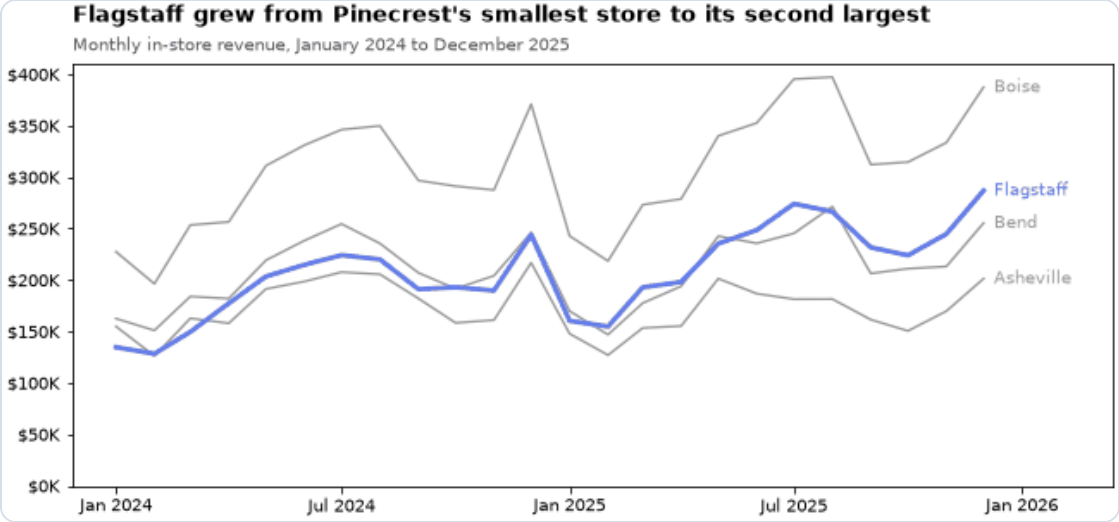
5 **What job is color doing? (2m, cut third). Say:** "Color has four jobs: highlight one series, tell unordered groups apart, show how much, show above or below a midpoint. Each has its palette type." Show the lesson's palette strip. **Say:** "Okabe-Ito for categories, viridis for low-to-high, a diverging map like RdBu around a midpoint. Never red against green as the only difference, never color as the only cue."

6 **Gray plus one, and direct labels (4m, never cut). Say:** "This chart's story is one store. So one store gets color and weight, the rest become context. Then the legend is useless, so each name goes at the end of its own line."

```
def label_line_ends(ax):
    """Each line's label just right of its last point, in its own color."""
    for line in ax.lines:
        x, y = line.get_xdata()[-1], line.get_ydata()[-1]
        ax.annotate(line.get_label(), xy=(x, y), xytext=(6, 0),
                    textcoords="offset points", va="center", fontsize=10,
                    color=line.get_color())
    left, right = ax.get_xlim()
    ax.set_xlim(left, right + 0.09 * (right - left))

fig, ax = new_chart()
draw_lines(ax, focus="Flagstaff")
add_titles(ax, headline, subtitle)
format_axes(ax)
label_line_ends(ax)
plt.show()
print([(line.get_label(), line.get_color()) for line in ax.lines])
```

```
[('Asheville', '0.6'), ('Bend', '0.6'), ('Boise', '0.6'), ('Flagstaff', '#667e
```



No legend. The eye goes straight to the one colored line.

Every line object remembers its data, label and color, so one function labels any chart. `textcoords="offset points"` keeps a 6-point gap at any figure size.

7 **Point at the evidence (3m). Say:** "An annotation is what you would say while pointing at the slide. Two callouts carry the numbers behind the headline; a shaded band explains the December spikes."

```
def annotate_story(ax):
    for year in (2024, 2025):
        ax.axvspan(pd.Timestamp(f"{year}-11-01"), pd.Timestamp(f"{year}-12-31"
                   color="0.93", zorder=0)
    flag = monthly["Flagstaff"]
    arrow = {"arrowstyle": "->", "color": "0.3"}
    ax.annotate(f"Jan 2024: ${flag.iloc[0] / 1000:.0f}K\nthe smallest store",
                xy=(flag.index[0], flag.iloc[0]),
                xytext=(pd.Timestamp("2024-02-15"), 55000),
                arrowprops=arrow, fontsize=9)
    ax.annotate(f"Dec 2025: ${flag.iloc[-1] / 1000:.0f}K\nsecond only to Boise
                xy=(flag.index[-1], flag.iloc[-1]),
                xytext=(pd.Timestamp("2025-06-10"), 95000),
                arrowprops=arrow, fontsize=9)
```

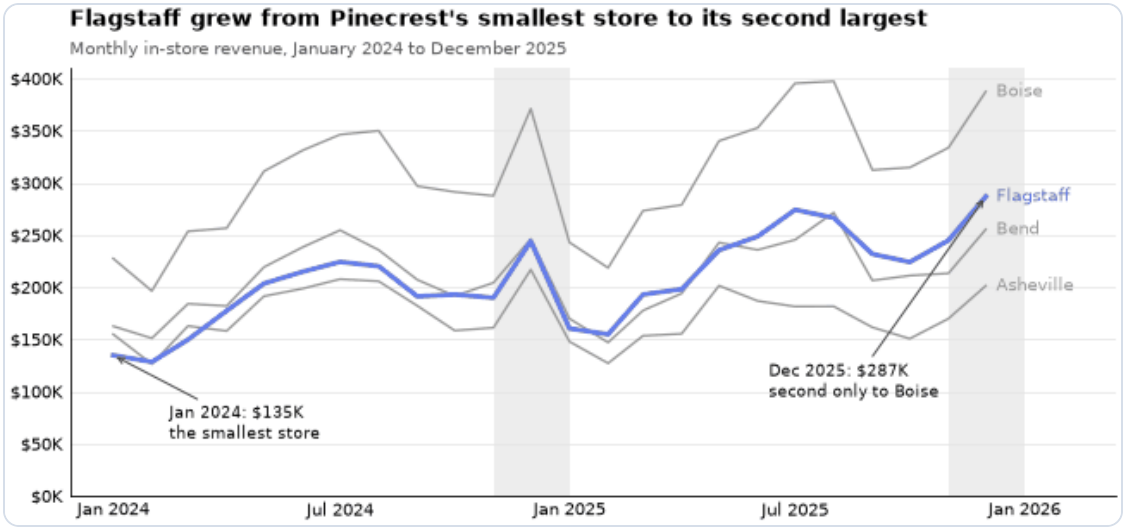
No output yet: this only defines the function. Positions for `xytext` take a few tries; adjusting and rerunning is quicker than calculating. **Say:** "One to three annotations. If you want six, you have two charts."

8 **Delete what does not work (3m). Say:** "The editor's last pass is deletion: the top and right sides of the frame, the tick marks, and a grid so faint it only helps."

```
def clean_up(ax):
    ax.spines[["top", "right"]].set_visible(False)
    ax.tick_params(length=0)
    ax.grid(axis="y", color="0.9")
    ax.set_axisbelow(True)

fig, ax = new_chart()
draw_lines(ax, focus="Flagstaff")
add_titles(ax, headline, subtitle)
format_axes(ax)
label_line_ends(ax)
annotate_story(ax)
clean_up(ax)
plt.show()
print([side for side in ax.spines if ax.spines[side].get_visible()])
```

```
['left', 'bottom']
```



The finished chart. The data is identical to the first draft; six function calls each made one decision.

Scroll back to the first draft and put the two side by side. **Ask:** "Which edit changed it most for you?" (It is the lesson's journal prompt.)

9 **A bar chart that lies with a straight face (2m). Say:** "One more deletion people make: the bottom of a bar chart's axis, 'to show the difference better'."

```
totals = daily[daily["date"].dt.year == 2025].groupby("store")["revenue"].sum(
boise_t, ash_t = totals["Boise"], totals["Asheville"]
print("True ratio, Boise to Asheville:", round(boise_t / ash_t, 1))
cut = 1_500_000
drawn = (boise_t - cut) / (ash_t - cut)
print("Ratio of the bar heights drawn:", round(drawn, 1))
```

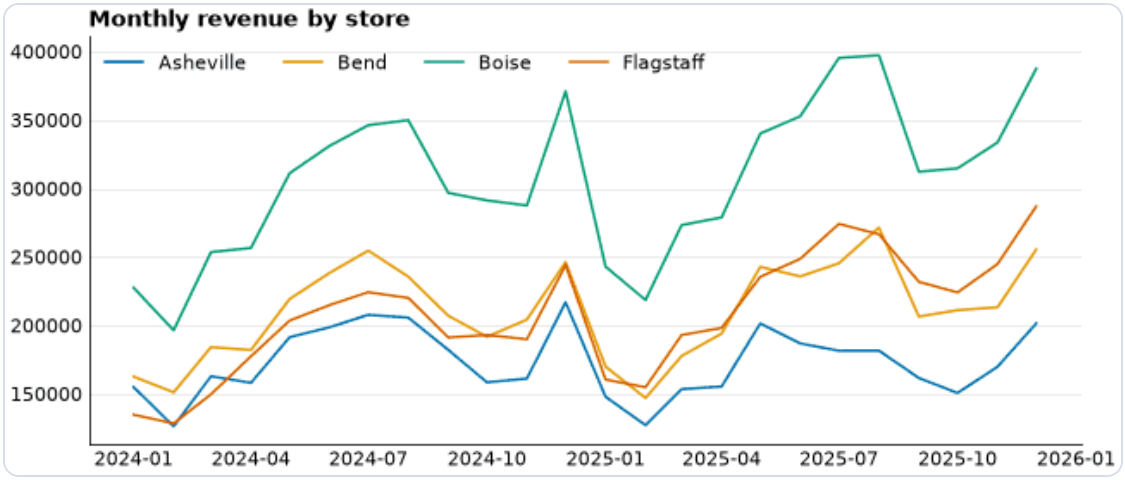
```
True ratio, Boise to Asheville: 1.9
Ratio of the bar heights drawn: 4.5
```

A bar's length is its value. Cut the axis at $1.5 million and Boise looks four and a half times Asheville. Bars start at zero; lines may zoom in when the change itself is the point.

10 **A house style (3m, cut fourth). Say:** "You removed spines and chose fonts by hand. Do it on every chart and you will tire of it. `rcParams` holds every default; `rc_context` changes them for one block only."

```
okabe_ito = ["#0072b2", "#e69f00", "#009e73", "#d55e00", "#cc79a7", "#56b4e9"]
pinecrest_style = {
    "font.size": 11, "axes.titlelocation": "left", "axes.titleweight": "bold",
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "axes.grid.axis": "y", "grid.color": "0.9",
    "axes.axisbelow": True, "xtick.major.size": 0, "ytick.major.size": 0,
    "legend.frameon": False, "axes.prop_cycle": mpl.cycler(color=okabe_ito),
}
with plt.rc_context(pinecrest_style):
    fig, ax = plt.subplots(figsize=(9, 3.8), layout="constrained")
    for store in stores:
        ax.plot(monthly.index, monthly[store], label=store)
    ax.set_title("Monthly revenue by store")
    ax.legend(ncols=4)
    plt.show()
print(len(plt.rcParams), "settings; outside the block, top spine:",
      plt.rcParams["axes.spines.top"])
```

```
339 settings; outside the block, top spine: True
```



Plain plotting code inside the block; everything else came from the style.

A style takes care of what is the same on every chart, so your attention goes to what is different. It cannot write your headline. **Tell the room to save `pinecrest_style` and the helper functions:** Session 10 starts from them.

11 **The formatter typo (1m). Say:** "Last one: the most common customization error, and why it is confusing."

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3], [1000, 2500, 1800])
ax.yaxis.set_major_formatter("${:,.0f}")        # no x inside the braces
try:
    fig.canvas.draw()          # labels are formatted when the figure is drawn
finally:
    plt.close(fig)
```

```
IndexError: tuple index out of range
```

The error appears when the figure is drawn, in a notebook at the end of the cell, far from the line that caused it. Matplotlib passes each tick by name, as `x`: the fix is `"${x:,.0f}"`.

THREE MISTAKES LEARNERS WILL MAKE, AND HOW TO COACH EACH

- **A format string without `x`.** `ax.yaxis.set_major_formatter("${:,.0f}")` raises `IndexError: tuple index out of range`, and only when the figure is drawn, at the end of the cell. Coach: "Matplotlib passes each tick value by name. What is the name?" Fix: `"${x:,.0f}"`. The neighbor error: `ValueError: The number of FixedLocator locations (3) ... does not match the number of labels (2)` from `set_xticks` with too few labels.
- **A finding title that overstates.** No error. "Footwear is returned twice as often as anything else" when Apparel is at 9.3%; "returns are rising" from a wobbling line. Coach: "Which number on the chart proves that sentence?" When the headline overstates, edit the headline, not the data.
- **Decorating instead of editing.** More colors, a thicker frame, a legend kept "just in case", a red and green pair, a bar axis cut to "show the difference". Coach with the checklist: "If you removed this, would the reader lose anything? Does color carry the message alone?" Bars start at zero; color needs a second cue.

GUIDED LAB: FACILITATION NOTES

The lab: `labs/module-09/l3_3-customizing-plots/finish_chart_starter.py` (your reference: `finish_chart_solution.py`). 25 minutes. The starter saves Matplotlib's first draft of completed online revenue by month, 2024 against 2025. Learners make five edits, each in a small function, and save `finished.png`.

Minute 0 to 5 (TODO 1): numbers before words. The check: 2025 beat 2024 in all 12 months; the largest gap is August (+$33,144), the smallest December (+$4,803); the year grew 18.8%. Only now may they write the headline. Push back on "Online revenue is booming": what number says booming?

Minute 5 to 15 (TODO 2 and 3): the subtitle says what is plotted (completed online orders, revenue per calendar month, USD). The x axis is month numbers 1 to 12 here, not dates, so the tick labels come from `ax.set_xticks(range(1, 13), labels=MONTHS)` rather than a date formatter: a good moment to ask why.

Minute 15 to 25 (TODO 4 and 5) and debrief: 2024 in gray, 2025 in color, years written at the line ends, an arrow at August. Expected checks: y labels `$0K, $20K, $40K, $60K...`, visible spines left and bottom, `ax.get_legend()` is `None`. Put a draft and a finished PNG side by side on the projector and let the room name each edit.

If the room finishes early: `labs/module-09/l3_3-customizing-plots/color_jobs_starter.py`: a sequential heatmap of visitors (store by month) and a diverging heatmap of revenue change against 2024, centered on zero. Flagstaff and Boise grew in all 12 months; Asheville shrank in 9 and Bend in 5. Ask why limits taken from the data's own minimum and maximum (-12.6% to 29.0%) would put the neutral color at about +8% and mislead.

STARTING THE LESSON PROJECT

The Footwear Returns Chart (lesson page, about 35 minutes solo). In the room, 10 minutes: read the brief (the online team suspects some categories come back far more than others), continue in the lesson notebook (the project reuses `add_titles`), and run **Step 1**. Done when nine sorted rates print with Footwear last at 0.144, and `All online orders: 0.069`.

Connect to Session 8: "You found this in the `order_features` lab: online orders with footwear came back 14.4% of the time. Today you make a chart that says it in five seconds."

Steps 2 to 5 are homework. Check: Footwear is 2.1 times the overall online rate, and the lesson's own warning about the headline ("twice as often as anything else" is contradicted by Apparel at 9.3%).

CHECKS FOR UNDERSTANDING (ASK OUT LOUD)

1. What goes in the title, and what goes in the subtitle? **Answer:** The title states the finding; the subtitle says what is plotted, the units and the period.
2. Which formatter turns 0.25 into 25%? **Answer:** `PercentFormatter(xmax=1)`; or a format string `"{x:.0%}"`.
3. Why gray for three stores and color for one? **Answer:** Color directs attention to the story; gray keeps the others as context.
4. What does `plt.rc_context(style)` do after the `with` block ends? **Answer:** It puts every setting back, so later charts are not affected.

Workbook self-check, items Q7 to Q12:

Answers: **Q7** C) The online return rate held near 7% in both years; **Q8** `$1,235K 7.1%`; **Q9** C) A diverging map such as RdBu, with the limits set symmetric around zero; **Q10** A) The reader no longer has to look a color up in a box, and the chart works even when colors are hard to tell apart; **Q11** `True False True`; **Q12** B) Starting the y axis at $1.5 million to "show the differences better". Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: Provide the six helper functions from this guide as a file. The learner's job in the lab then becomes choosing and ordering the edits, not typing them. For the headline, give the frame "[Subject] [did what] [by

how much / when]".

Stretch: Lesson 3.3's bonus challenge (finish the nine-panel seasons figure with the house style and gray-plus-one panels, then check it in grayscale). Or, in the lab, shade the November-December holiday season with `ax.axvspan(10.5, 12.5, ...)` and decide whether it helps this chart; or rebuild the finished chart inside `plt.rc_context(pinecrest_style)` and delete every line the style made unnecessary.

CLOSING THE SESSION

2:50 Wrap-up — 10 min

EXIT TICKET (3 QUESTIONS, WITH ANSWERS)

1. What does `plt.subplots(2, 2)` give you in `axs`, and what is the safest way to loop over the panels? **Answer:** A 2-D NumPy array of four Axes. Loop with `for ax, item in zip(axs.flat, items):`, which works for every grid shape.
2. Why is a twin-axis chart of two different measures risky, and what do you draw instead? **Answer:** The two scales are arbitrary, so where the lines sit, cross or overlap can be changed without changing the data. Stack two panels with `sharex=True`, or plot the ratio itself.
3. Rewrite the title "Monthly online revenue, 2024 and 2025" as a finding, and say what you checked first. **Answer:** For example "Online revenue beat 2024 in every month of 2025", after checking that the 2025 value is higher in all 12 months (it is; the smallest gap is December, $4,803).

Reading the tickets: question 1 answered with "a list" or with `for ax in axs` (which loops over rows of a 2-D array) needs the shape table again at the start of Session 10. For question 2 accept "the scales can be chosen" in any words; "it looks cluttered" misses the point. For question 3 the check matters more than the wording: a ticket with a headline and no check is the learner to talk to first next time.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- Finish the Lesson 3.2 project, **Nine Categories, Twelve Months** (Steps 2 to 5). Check: `by_month` is `(12, 9)`, every column of `share` adds up to 100, and Snow peaks in December at 28.0%.
- Finish the Lesson 3.3 project, **The Footwear Returns Chart** (Steps 2 to 5). Check: Footwear at 14.4%, 2.1 times the overall online rate of 6.9%, and a headline you compared with the numbers.
- Lesson 3.2: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **Layout: Size and Spacing** and the secondary-axis example.
- Lesson 3.3: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read the **heatmap** section and **Ready-Made Styles**.
- Kit labs not reached in the room: `labs/module-09/l3_2-subplots-composition/honest_axes_starter.py` and `labs/module-09/l3_3-customizing-plots/color_jobs_starter.py` (about 20 minutes each).
- From Lesson 3.3, "Before the Next Lesson": save `pinecrest_style` and your helper functions (`add_titles`, `format_axes`, `label_line_ends`, `clean_up`) in a file or a clearly marked cell. Session 10 uses them.

BRING LIST FOR SESSION 10

- Their Lesson 3.3 notebook with `pinecrest_style` and the helper functions saved in a clearly marked cell or a `.py` file: Lesson 3.4 reuses them.
- The two finished project figures (the nine-panel seasons figure and the footwear returns chart): Session 10 saves them for the web, a slide and print.
- The course data folder (`daily_sales.csv`, `orders.csv`, `order_items.csv`, `products.csv`, `store_weather.csv`).
- Nothing new to install: SciPy, used in one Lesson 4.1 bonus exercise, is already in `requirements.txt`. No servers, no accounts, no internet needed.

PREVIEW TO SAY

Next session your charts leave the notebook. Lesson 3.4: how figure size and dpi become pixels, PNG or SVG or PDF for each destination, why text comes out tiny, and one function that applies the house style and saves a whole set of charts. Then Module 4 begins: two honest numbers for 20,302 orders, one for what is typical and one for how much they vary, and why the mean you have been printing all course can mislead.

IF YOU RUN LONG OR SHORT

IF YOU RUN LONG: CUT IN THIS ORDER

1. **3.2 demo step 5** (what shape do you get back): show the lesson's table instead of typing. Saves 2 minutes; self-check Q1 and Q2 cover it.
2. **3.2 demo step 10** (secondary axis): one sentence and the lesson's figure. Saves 2 minutes.
3. **3.3 demo step 5** (naming colors and palettes): show the lesson's palette strip instead. Saves 2 minutes.
4. **3.3 demo step 10** (`rc_context` house style): show it running, do not type the dictionary; it goes home with the homework. Saves 3 minutes.
5. **Guided labs:** stop `traffic_panels` after Step 3 (Step 4, the mosaic, becomes homework); stop `finish_chart` after Step 4 and annotate together on the projector.
6. **Project starts:** as a last resort, run Step 1 yourself on the projector.

Never cut: 3.2 demo steps 4, 8 and 9; 3.3 demo steps 2, 3 and 6.

IF YOU RUN SHORT

Run the second lab of the lesson you are in. `honest_axes` lets the room make the same twin-axis data tell two stories, then fix it. `color_jobs` builds a sequential and a centered diverging heatmap and asks why an uncentered one would mislead.

Or have pairs swap their `finish_chart` PNGs and write one edit for the other's chart on a sticky note: "remove...", "add...".

Or continue the 3.2 project to Step 4 together: sorting the nine panels by peak month is the moment the figure starts telling a story.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 10: Saving Figures; Center and Spread

How to teach the session where Module 3 ends and Module 4 begins. Before the break learners get their charts out of the notebook: they predict a saved figure's size from `figsize` and `dpi`, find out why text comes out tiny, choose between PNG, SVG and PDF, and build one preset-driven save function. After the break they start statistics with its two most used ideas, center and spread, and discover that Pinecrest's "average order" of $194.60 describes almost nobody.

`2 lessons` `3 hrs session` `Lessons 3.4-4.1` `Session 10 of 21`

HOW TO USE THIS GUIDE

- Same rhythm as every session: **Frame** (the goal and a mental model) → **Live demo** (you type, they type along) → **Guided lab** (the kit's lab files) → **Start the lesson project** (so nobody opens it cold at home). The lesson's exercises, quiz and journal are homework.
- **Type, do not paste.** Every code block in the demo scripts is what to type, in a Jupyter notebook in the course project folder, one cell per step. The one exception is the setup cell of 3.4, which learners may copy from the lesson page. Every output shown was produced by running that code with the course environment (pandas 3.0, Matplotlib 3.11).
- **Ask for a prediction before you run.** Both lessons are full of results that surprise people (same pixels, different chart; 63% of orders below the average order). The surprise only teaches if they committed to a guess first.
- **The lessons are the reference.** Keep `matplotlib_saving_figures.html` and `statistics_descriptive_part1.html` open in browser tabs for the figures (the two same-pixel charts, the raster zoom, the histogram with mean and median) and the tables (destinations, formats, pairing center with spread).
- Teal = hands-on, amber = watch-outs. Labs for this session live in `labs/module-10/`: starters for learners, solutions for you.

THE RISK OF SESSION 10: TWO LESSONS WHOSE MISTAKES RAISE NO ERROR

A chart saved at the wrong size, a blank PNG from `plt.savefig()`, a mean reported for skewed data, an SD that silently used the other `ddof`: none of these stop the program. Train one habit in each half. Before the break: **do not trust, check** (`plt.imread(path).shape`, then open the file). After the break: **every mean gets a median next to it**, and every summary gets its `.count()`. The second risk is the change of subject at the break. Some learners arrive nervous about statistics; say plainly that today has no formulas to memorize and that every number is one method call.

BEFORE SESSION 10

- ☐ Run the lab check from the kit folder: `python3 labs/check_labs.py module-10`. Eight lines should say PASS.
- ☐ Copy the starters and the two debug handouts (`*_starter.py`, `debug_*.py`) from `labs/module-10/` to the shared folder. Keep every `*_solution.py` on your machine.
- ☐ Projector machine: Jupyter (or VS Code notebooks) opened in the course project folder so that `data/daily_sales.csv` and `data/orders.csv` resolve; font size at least 18 pt; a file explorer or image viewer open beside it, ready to show the `charts` folder.
- ☐ Files used today: `data/daily_sales.csv`, `data/orders.csv`, `data/store_weather.csv`, `data/products.csv`, `data/order_items.csv`. No servers to start, no accounts, nothing new to install (`matplotlib`, `numpy`, `pandas` and `scipy` are already in the course environment).
- ☐ Run the 3.4 demo once on the projector machine and delete the `charts` folder afterwards, so the first `out.mkdir(exist_ok=True)` visibly creates it.
- ☐ Have a PDF viewer that can zoom to 800% and, if you can, a slide program with one dark slide for the transparent-background trap (optional; the corner-pixel printout makes the same point).
- ☐ Whiteboard, written before learners arrive: **pixels = inches x dpi** on the left; on the right two empty columns headed **Center** and **Spread**.
- ☐ **Known environment traps.** (1) Notebooks show figures at their own resolution and trim the margins, so the preview is not the file: always open the saved file. (2) File sizes in KB differ a little between machines (fonts); pixel sizes do not. (3) On Windows a PDF that is open in a viewer cannot be overwritten (`PermissionError`): close the viewer before re-running a cell. (4) In VS Code the working folder is the workspace root, so `charts/` appears there, not next to the notebook. (5) `plt.imread` on a PNG returns values from 0 to 1 with 4 channels. (6) pandas 3 prints `dtype: str` for text; a learner on pandas 2 sees `object`. (7) `np.mean` on data with a missing value prints `nan`, not an error.
- ☐ Skim the Session 9 homework: the nine-panel category figure (Lesson 3.2 project) and the footwear returns chart (Lesson 3.3 project). The returns chart is your bridge: "It is finished. Now get it onto a slide."
- ☐ Sticky notes or slips of paper for the exit ticket.

SESSION 10 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 9 (subplots, customizing), homework check.
0:10	10	**3.4** Frame and mental model	The sheet of paper and the scanner; pixels = inches x dpi.
0:20	25	**3.4** Live demo	Save, read back, change dpi, the tiny-text problem, formats, presets, a loop that closes.
0:45	20	**3.4** Guided lab	`labs/module-10/l3_4-saving-figures/pixel_budget_starter.py`, then the first two TODOs of `category_price_pack_starter.py`.
1:05	10	**3.4** Start the lesson project	The Pinecrest Chart Pack, Step 1: everyone prints the four growth figures.
1:15	10	Break	Leave the two same-pixel charts side by side on the projector.
1:25	15	**4.1** Frame and mental model	Two questions about any column: what is typical, and how much does it vary?
1:40	30	**4.1** Live demo	Mean and median of order totals, the tail, mode, variance by hand, ddof, IQR, missing values.
2:10	25	**4.1** Guided lab	`labs/module-10/l4_1-center-and-spread/orders_per_buyer_starter.py`, then Steps 1 to 3 of `visitor_spread_starter.py`.
2:35	15	**4.1** Start the lesson project	The Typical Order Memo, Step 1: `center_and_spread()` prints six numbers.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 11.
Total	**180**		Ends at 3:00.

The lesson pages estimate 75 + 85 = 160 minutes for a solo learner, and the room has 150 after the warm-up, break and wrap-up, of which 45 go to the kit labs. So the room runs each lesson's frame, a compressed demo of its main thread, the guided lab and the **first step of each lesson project**. **Done in the room:** the mental models, saving and verifying a figure, the tiny-text demonstration, presets and the closing loop, mean against median on the order totals, a variance by hand, `ddof`, the IQR, both `pixel_budget` and `orders_per_buyer` labs. **Goes home:** the rest of both projects (Chart Pack Steps 2 to 4, Typical Order Memo Steps 2 to 5), the unfinished lab steps, the lessons' exercises, the Quick Quizzes, both journals, and the reading-only parts: the raster zoom figure, `rasterized=True` for heavy scatter plots, the dark-style chart, `PdfPages`, the n - 1 simulation and the FAQs.

SESSION AT A GLANCE

LESSON	TOPIC	ROOM TIME	LEARNERS LEAVE ABLE TO…
3.4	Saving Figures	65 of 75 min	predict a saved figure's pixel size, choose PNG, SVG or PDF for a destination, and export a set of charts through one preset-driven function
4.1	Descriptive Statistics I: Center and Spread	85 of 85 min	compute and explain mean, median, mode, SD and IQR, say which library divides by n or n - 1, and pair the right center with the right spread

WARM-UP

0:00 Warm-up: Retrieval from Session 9

10 min

TIMING

6m	Five questions, no notes: answer alone on paper, compare with a neighbor, then one answer per question out loud
3m	Homework check: the two Session 9 projects
1m	Bridge to today

FIVE RETRIEVAL QUESTIONS (NO NOTES, PAIRS, 60 SECONDS EACH)

1. `fig, axs = plt.subplots(2, 2)`: what kind of object is `axs`, and what loop visits every panel whatever the grid shape? **Answer:** A 2-D NumPy array of Axes. `for ax, group in zip(axs.flat, groups):` works for every grid shape.
2. What does `sharey=True` buy you in a grid of panels, and what does it cost? **Answer:** Heights become comparable across panels. The cost: small series are flattened, so shape is harder to see. Choose on purpose and tell the reader.
3. Why is a twin-axis chart with two different measures risky, and what should you draw instead? **Answer:** The two scales are arbitrary, so the chart can be made to say almost anything. Use stacked panels with a shared x axis, or plot a ratio.
4. What should a finished chart's title say? And the subtitle? **Answer:** The title states the finding. The subtitle says what is plotted, the units and the period.
5. You have a dictionary of style settings. How do you apply it to one block of plotting code without changing the rest of the session? **Answer:** `with plt.rc_context(style_dict):` around the block.

HOMEWORK CHECK

Ask for a show of hands: who finished the nine-category small-multiples figure (Lesson 3.2) and the footwear returns chart (Lesson 3.3)? Have one volunteer hold up or share the returns chart and read its title out loud: it should state a finding (the lesson's is "Online orders with footwear come back twice as often": about 14% against the overall online rate, 2.1 times). Ask the lesson's own question of it: "Twice as often as what?" Anyone who did not finish keeps going at home; today's first lesson does not depend on those notebooks beyond the style dictionary, and the setup cell is on the lesson page.

BRIDGE TO TODAY

Say: "That chart is finished and it lives in your notebook, where its audience is one person. Before the break we get charts out: onto a web page, a slide, a printed report. After the break we change subject completely

and start the statistics module, with the simplest question there is: what is a typical order?"

LESSON PLANS

3.4 Saving Figures

65 min

TIMING

10m	Frame: the last step is part of the work; the sheet of paper and the scanner
25m	Live demo: 10 short steps, learners type along in a new notebook
20m	Guided lab: `pixel_budget`, then the start of `category_price_pack`
10m	Start the lesson project: the Pinecrest Chart Pack, Step 1

OBJECTIVES (THE LESSON'S OWN)

- Save a figure with `fig.savefig()` and predict its size in pixels from `figsize` and `dpi`
- Choose between raster formats (PNG) and vector formats (SVG, PDF) for a given destination
- Explain why text looks tiny or huge in a saved chart, and fix it with the figure size
- Control the saved area and background with `bbox_inches`, `facecolor` and `transparent`
- Write one function that applies a house style and saves for web, slides or print, then use it in a loop that closes each figure

MENTAL MODEL TO SAY

Say: "A Matplotlib Figure is a sheet of paper with a size in inches. Everything on it, including the text, is laid out on that sheet. Saving to an image is like scanning the sheet: the scanner's resolution, in dots per inch, decides how many pixels you get. A bigger sheet and a finer scan both give more pixels, but they are not the same thing. A finer scan makes everything sharper. A bigger sheet makes the text smaller relative to the page."

Hold up a sheet of paper with one word written on it, then a sheet twice the size with the same word at the same handwriting size. "Same pen. Which page looks empty?" Write on the board and leave it up all session: **pixels = inches x dpi** and under it **figsize sets how big the text looks; dpi sets how many pixels you get.** The lesson's decision chart (printed or resized: PDF or SVG; a slide: PNG at slide size; web or email: PNG at about 110 dpi) is worth showing from the lesson page once.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `session10_saving.ipynb` in the course project folder (so that `data/` is next to it). Everything is saved into a `charts` folder that the code creates. Do not call `plt.show()` in the demo: in a notebook the figure appears under the cell, and the habit to build today is **judge the saved file, not the notebook preview**. Keep a file explorer or image viewer open on the `charts` folder.

1. **Set the stage (3m).** This first cell is the lesson's own setup: the one cell today that learners may copy from the lesson page instead of typing. Say: "Nothing new here. It is the monthly table from Lesson 3.1 and a chart function in the style of Lesson 3.3. Today is about what happens after the chart is drawn."

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.ticker import FuncFormatter

daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
daily["month"] = daily["date"].dt.to_period("M").dt.to_timestamp()
monthly = daily.pivot_table(index="month", columns="store",
                            values="revenue", aggfunc="sum")
stores = list(monthly.columns)

def thousands(x, pos):
    return f"${x / 1000:,.0f}K"

def revenue_chart(store, figsize=(8, 4)):
    """Draw one store's monthly revenue. Returns the figure and its Axes."""
    fig, ax = plt.subplots(figsize=figsize, layout="constrained")
    ax.plot(monthly.index, monthly[store], color="#667eea",
            marker="o", markersize=3)
    ax.set_title(f"{store}: monthly in-store revenue",
                 loc="left", fontweight="bold")
    ax.yaxis.set_major_formatter(FuncFormatter(thousands))
    ax.set_ylim(bottom=0)
    ax.spines[["top", "right"]].set_visible(False)
    return fig, ax

fig, ax = revenue_chart("Boise")
print(monthly.shape, stores)
```

```
(24, 4) ['Asheville', 'Bend', 'Boise', 'Flagstaff']
```

24 months by 4 stores. In a notebook the Boise chart appears under the cell.

2. **Save it, then prove it (3m).** Say: "Saving is one method call on the figure. Then we do not trust, we check: read the file back." Ask before running the last line: "8 by 4 inches. How many pixels?"

```
out = Path("charts")
out.mkdir(exist_ok=True)

fig.savefig(out / "boise.png")

saved = out / "boise.png"
print(saved.exists(), f"{saved.stat().st_size / 1024:.0f} KB")
print(plt.imread(saved).shape)
```

```
True 35 KB
(400, 800, 4)
```

Point at the shape: rows first (400 high), then columns (800 wide), then 4 channels (red, green, blue, alpha). The default dpi is 100. The size in KB can differ a little on other machines.

3. **One multiplication (3m).** Write **pixels = inches x dpi** on the board. Have the room predict all three lines before you run.

```
for dpi in (72, 150, 300):
    path = out / f"boise_{dpi}dpi.png"
    fig.savefig(path, dpi=dpi)
    height, width, channels = plt.imread(path).shape
    kb = path.stat().st_size / 1024
    print(f"dpi={dpi:>3}: {width} x {height} pixels, {kb:>4.0f} KB")
```

```
dpi= 72: 576 x 288 pixels,   24 KB
dpi=150: 1200 x 600 pixels,   57 KB
dpi=300: 2400 x 1200 pixels,  128 KB
```

Say: "Same sheet of paper, scanned at three resolutions. The picture is identical. Only the number of pixels describing it changes."

4. **The tiny-text problem (4m).** The most important two minutes of the lesson. Say: "I need 1,200 pixels wide. Twelve inches at 100, or six inches at 200. Same pixels. Same chart?" Take a show of hands, then run and open both files side by side in the file explorer or an image viewer.

```
big_canvas, _ = revenue_chart("Boise", figsize=(12, 6))
big_canvas.savefig(out / "canvas_12in.png", dpi=100)

small_canvas, _ = revenue_chart("Boise", figsize=(6, 3))
small_canvas.savefig(out / "canvas_6in.png", dpi=200)

for name in ("canvas_12in.png", "canvas_6in.png"):
    print(name, plt.imread(out / name).shape[:2])
plt.close(big_canvas)
plt.close(small_canvas)
```

```
canvas_12in.png (600, 1200)
canvas_6in.png (600, 1200)
```

Say: "Font sizes are in points, and a point is 1/72 of an inch: a physical length on the sheet. On the 6-inch sheet the same title is twice as prominent. **figsize controls how big the text looks. dpi controls how many pixels you get.**" Leave the two files on screen for the break.

5 **Raster or vector (3m).** Say: "A PNG is a grid of colored squares. An SVG or a PDF is a list of drawing instructions, sharp at any size." The format comes from the extension.

```
for extension in ("png", "svg", "pdf"):
    path = out / f"boise_formats.{extension}"
    fig.savefig(path)
    print(f"{path.name:<20} {path.stat().st_size / 1024:>5.1f} KB")
```

```
boise_formats.png     35.0 KB
boise_formats.svg     31.6 KB
boise_formats.pdf     13.3 KB
```

Open the PDF and zoom to 800%: still sharp, and the title can be selected as text. Then zoom the PNG. Rule of thumb for the board: PNG for screens and slides, PDF or SVG for print and anything that will be resized, never JPEG for charts.

6 **The tight bounding box changes the size (2m).** Say: "You will see `bbox_inches="tight"` in every tutorial. Watch what it does to the size we asked for."

```
fig.savefig(out / "boise_default.png", dpi=100)
fig.savefig(out / "boise_tight.png", dpi=100, bbox_inches="tight")

for name in ("boise_default.png", "boise_tight.png"):
    rows, cols = plt.imread(out / name).shape[:2]
    print(f"{name:<20} {cols} x {rows}")
```

```
boise_default.png    800 x 400
boise_tight.png      811 x 411
```

Say: "It guarantees nothing is cut off and gives up the exact size. Our figures use `layout="constrained"`, so everything already fits. When a slide needs exactly 1,920 by 1,080, leave it off."

7. **Backgrounds (2m).** Say: "Transparent sounds perfect for slides. Look at the corner pixel."

```
fig.savefig(out / "boise_transparent.png", transparent=True)
corner = plt.imread(out / "boise_transparent.png")[0, 0]
print("top-left pixel (red, green, blue, alpha):", corner)
```

```
top-left pixel (red, green, blue, alpha): [1. 1. 1. 0.]
```

Alpha 0 means fully see-through. Say: "The background is gone. The title is still black. On a dark slide it disappears." The safe default is `fig.savefig(path, facecolor="white")`; for a dark slide, draw in a dark style (the lesson shows it; do not type it now).

8. **Put the decisions in one place (4m).** Say: "I know the settings now. Typing them correctly every time is how mistakes creep in. One table, two small functions." Type the table slowly and say each pixel size out loud as you type the row.

```
PRESETS = {
    "web":   {"figsize": (9, 4.5),   "dpi": 110, "extension": "png"},
    "slide": {"figsize": (9.6, 5.4), "dpi": 200, "extension": "png"},
    "print": {"figsize": (6.5, 3.5), "dpi": 300, "extension": "pdf"},
}

def new_figure(kind="web"):
    """Create a figure sized for 'web', 'slide' or 'print'."""
    size = PRESETS[kind]["figsize"]
    return plt.subplots(figsize=size, layout="constrained")

def save_chart(fig, name, kind="web", folder="charts"):
    """Save fig as folder/name_kind.ext. Returns the path."""
    preset = PRESETS[kind]
    folder = Path(folder)
    folder.mkdir(parents=True, exist_ok=True)
    path = folder / f"{name}_{kind}.{preset['extension']}"
    fig.savefig(path, dpi=preset["dpi"], facecolor="white")
    return path

def draw_revenue(ax, store):
    ax.plot(monthly.index, monthly[store], color="#667eea",
            marker="o", markersize=3)
    ax.set_title(f"{store}: monthly in-store revenue",
                 loc="left", fontweight="bold")
    ax.yaxis.set_major_formatter(FuncFormatter(thousands))
    ax.set_ylim(bottom=0)

for kind, preset in PRESETS.items():
    width, height = preset["figsize"]
    dpi = preset["dpi"]
    print(kind, round(width * dpi), "x", round(height * dpi))
```

```
web 990 x 495
slide 1920 x 1080
print 1950 x 1050
```

The figure is created at the right size **before** drawing, because the size decides how the text relates to the chart. (The lesson also wraps the loop in the `pinecrest_style` dictionary from Lesson 3.3; learners who have it should use it.)

9 **Many charts: create, draw, save, close (3m).** Say: "pyplot keeps every figure alive until you close it. A notebook hides that. A loop does not." Run the loop, then ask what the last line will print.

```
plt.close("all")
for store in stores:
    chart, ax = new_figure("web")
    draw_revenue(ax, store)
    path = save_chart(chart, store.lower())
    plt.close(chart)
    rows, cols = plt.imread(path).shape[:2]
    print(path, f"{cols} x {rows}")

print("open figures:", len(plt.get_fignums()))
```

```
charts/asheville_web.png 990 x 495
charts/bend_web.png 990 x 495
charts/boise_web.png 990 x 495
charts/flagstaff_web.png 990 x 495
open figures: 0
```

Comment out `plt.close(chart)`, run again and show `open figures: 4`. Say: "Four is harmless. Four hundred is a memory problem." Put the line back.

10 **Make the classic mistake on purpose (2m).** Say: "Here is the bug that reaches the meeting." Type it exactly, with `plt.savefig`.

```
chart, ax = revenue_chart("Bend")     # the chart you care about
scratch = plt.figure()                # later, another figure is created
plt.savefig(out / "bend_oops.png")    # saves the *current* figure

image = plt.imread(out / "bend_oops.png")
print("every pixel is white:", bool((image[..., :3] == 1).all()))
plt.close("all")
```

```
every pixel is white: True
```

No error, and a blank file. Say: "`plt.savefig()` saves whatever pyplot thinks is current. You are holding the figure in a variable, so say which one: `chart.savefig(...)`."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Raising dpi to fix small text.** The slide's labels are tiny, so the learner saves again at `dpi=600`: a file many times larger with text exactly as small. Coach with the board rule: "Which knob changes how big the text is on the sheet?" Then have them halve the `figsize` and double the dpi, and compare `plt.imread(path).shape` before and after: same pixels, bigger text.

- `plt.savefig()` **instead of** `fig.savefig()` (often after `plt.show()` in a script, or after another figure was created). No error, a blank or wrong file. Coach: "Which figure did you ask to be saved? Show me its name in that line." The check that catches it is the lesson's: `bool((plt.imread(path)[..., :3] == 1).all())`.
- **Reading** `imread().shape` **as (width, height), or expecting an exact size with** `bbox_inches="tight"`. Coach the first with "rows first, like a DataFrame: `shape` is (rows, columns)". For the second, ask them to print the shape with and without the argument (800 x 400 against 811 x 411 in the demo) and decide whether their destination needs an exact size.

GUIDED LAB

First 12 minutes, everyone: `labs/module-10/l3_4-saving-figures/pixel_budget_starter.py` (reference: `labs/module-10/l3_4-saving-figures/pixel_budget_solution.py`). Before anyone types, have them write the Step 1 prediction in the comment: 7 x 3.5 inches at 100 dpi. Walk the room during TODO 2; the usual stall is realizing the big sheet needs dpi 100 and the small one dpi 200. Stop the room after TODO 3 and take three answers to "which is easier to read, and why?" Expected by the end: `email: (700, 350)`, two `(1400, 700)` lines, `5 files`, `open figures: 0`.

Last 8 minutes: `labs/module-10/l3_4-saving-figures/category_price_pack_starter.py`, TODO 1 and TODO 2 only (the two presets and the one-line save). TODO 3 to 4 finish at home; the manifest at the bottom tells them when they are done (7 files, web 960 x 480, slides 1920 x 1080, 0 open figures). Labs write into an `output` folder next to the lab file and never call `plt.show()`: learners open the saved files, which is the point.

Early finishers: `labs/module-10/l3_4-saving-figures/debug_blank_slide.py`, a script that runs without an error and gets three things wrong.

START THE LESSON PROJECT

The lesson's project is **The Pinecrest Chart Pack** (about 30 minutes alone): one chart per store, saved for web and print, plus a four-page PDF, verified by a manifest. In the room do **Step 1 only**: learners continue in the demo notebook (the project reuses `PRESETS`, `new_figure()` and `save_chart()`), type the `yearly` pivot and `growth`, and print it. Everyone should see Asheville -5.0, Bend 3.8, Boise 9.3, Flagstaff 19.7 before the break. Read the completion checklist out loud so they know what "done" means at home: 9 files, every web PNG exactly 990 by 495, 4 pages, 0 open figures. Steps 2 to 4 are homework.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold call, then pair-share):

1. "I need an image 1,000 pixels wide. Give me two different `figsize` and `dpi` pairs. How will the two files differ?" **Answer:** For example 10 inches at 100 dpi and 5 inches at 200 dpi. Same pixels; in the 5-inch version the text and lines are twice as large relative to the chart.
2. "The designer will resize my chart for the annual report. Which format, and why?" **Answer:** PDF or SVG: vector files are redrawn sharply at any size. A PNG only has the pixels it was saved with.

3. "My loop saves 300 charts and the kernel dies. What is missing?" **Answer:** `plt.close(fig)` after each save. pyplot keeps every figure alive until it is closed.

Workbook self-check, items Q1 to Q6 (do two in the room if time allows; the rest is homework):

Answers: **Q1** B) 1,080 wide by 600 high; **Q2** `(150, 200)`; **Q3** D) In B the text, lines and markers are twice as large relative to the chart; **Q4** A) Pass `rasterized=True` to the scatter layer and keep saving as PDF; **Q5** C) Only the background became transparent; the text is still black, on a dark slide; **Q6** `1`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: give learners the `PRESETS` table on paper and have them fill in a fourth column, the pixel size, by hand before running anything. Pair anyone who was absent for Lesson 3.3 with a neighbor for the setup cell; they do not need `pinecrest_style` to follow today. If typing is slow, let them copy the `save_chart()` cell from the lesson page and spend the time on the loop.

Stretch: lesson Exercise 2 (add a `"square"` preset of 1,080 by 1,080 and choose the inches so the text is comfortable), or Exercise 4 (write PNG metadata and read it back with Pillow). Strong learners can start the bonus challenge: move the presets and both functions into `charting.py` and import them.

4.1 Descriptive Statistics I: Center and Spread

85 min

TIMING

15m	Frame: two questions about any column; the restaurant review; why this module exists
30m	Live demo: 11 short steps on order totals, weather and five days of tent sales
25m	Guided lab: `orders_per_buyer`, then Steps 1 to 3 of `visitor_spread`
15m	Start the lesson project: the Typical Order Memo, Step 1

OBJECTIVES (THE LESSON'S OWN)

- Compute the mean, median and mode of a column with pandas and say what each one measures
- Recognize when the mean misleads, and choose the median instead, using Pinecrest's skewed order totals
- Compute range, variance, standard deviation and interquartile range (IQR), and explain each in the units of the data
- Explain the difference between dividing by `n` and by `n - 1`, and predict which one NumPy and pandas use by default
- Pair the right measure of center with the right measure of spread for a given column

MENTAL MODEL TO SAY

Say: "Nobody reads 1,000 reviews before choosing a restaurant. You look at the average rating, and then, if you are careful, at how much the reviews disagree. A 4.2 where everyone gave a 4 is a different restaurant from a 4.2 made of fives and ones. Two numbers, what is typical and how much it varies, are the least you need to describe anything honestly."

Name the two questions and write them as column headings on the board: **Center: what is typical?** (mean, median, mode) and **Spread: how much does it vary?** (range, variance, SD, IQR). Fill the columns in as the demo goes. Open Module 4 in one sentence: "Modules 1 to 3 gave you tables and charts. This module turns them into claims you can defend." Then the hook: "'Our average order is $195' is true. And most orders are nowhere near $195. By the break after next you will be able to say why."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `session10_center_spread.ipynb`. The numbers in the demo are the lesson's own, so learners can find every step again on the lesson page tonight. Ask for a prediction before every cell that prints a comparison. Step 4 draws a histogram: save it with `fig.savefig` as practiced before the break.

1. **The mean is a balance point (3m).** New notebook. Say: "The marketing lead wants the average order value for a slide. Easy: add them up, divide by how many."

```
import numpy as np
import pandas as pd

orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
totals = orders["order_total"]

print(orders.shape)
print(totals.sum(), len(totals))
print(totals.sum() / len(totals))
print(totals.mean())
```

```
(20302, 11)
3950701.45 20302
194.59666289035565
194.59666289035565
```

Say: "About $3.95 million over 20,302 orders: $194.60. Picture every order as a weight on a ruler. The mean is where the ruler balances."

2. **One far-away weight (3m).** Say: "Five ordinary orders. Then the same five plus one of Pinecrest's real bulk orders." Have learners predict the second mean.

```
five = np.array([42.50, 88.00, 150.00, 163.25, 271.00])
six = np.append(five, 4893.00)

print("five orders: mean", five.mean(), " median", np.median(five))
print("six orders : mean", six.mean(), " median", np.median(six))
```

```
five orders: mean 142.95  median 150.0
six orders : mean 934.625  median 156.625
```

Say: "One order moved the mean from about $143 to about $935, a number that describes none of the six. The median moved by $6.63. A statistic that barely reacts to extremes is called **robust**."

3. **The real column (3m).** Ask first: "What share of orders is below the average order? Half?" Write the guesses on the board.

```
print("mean  :", round(totals.mean(), 2))
print("median:", totals.median())
below = totals < totals.mean()
print("share of orders below the mean:", round(below.mean(), 3))
```

```
mean  : 194.6
median: 146.85
share of orders below the mean: 0.626
```

Say: "63% of orders are smaller than the average order. Neither number is wrong; they answer different questions." Dwell on the last line: a comparison gives True/False, True counts as 1, so **the mean of a boolean column is a proportion**. They will use that all module.

4 **See the tail (3m).** Say: "A picture explains the $50 gap faster than a formula." Type the histogram; in a notebook it appears under the cell.

```
from pathlib import Path
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(9, 4), layout="constrained")
ax.hist(totals[totals <= 1000], bins=50, color="#667eea", edgecolor="white")
ax.axvline(totals.median(), color="#15803d", linewidth=2.5,
           label=f"median ${totals.median():,.2f}")
ax.axvline(totals.mean(), color="#b91c1c", linewidth=2.5, linestyle="--",
           label=f"mean ${totals.mean():,.2f}")
ax.set_xlabel("Order total ($)")
ax.set_ylabel("Number of orders")
ax.legend()
Path("charts").mkdir(exist_ok=True)
fig.savefig("charts/order_totals.png", dpi=110)

print("orders above $1,000 (not drawn):", (totals > 1000).sum())
```

```
orders above $1,000 (not drawn): 37
```

Name the shape: **right-skewed**, the normal shape of anything about money. The tail has leverage on the balance point, so the mean sits to the right of the median. (This is also the Session's first half in action: the chart is saved for the web.)

5 **Check the tempting explanation (3m).** Say: "Someone will say it is just the twelve bulk orders. Check before you believe it."

```
is_bulk = totals > 2000
print("bulk orders:", is_bulk.sum())
print("mean without them  :", round(totals[~is_bulk].mean(), 2))
print("median without them:", totals[~is_bulk].median())

big = totals > 500
print("orders above $500:", big.sum(), f"({big.mean():.1%} of all orders)")
```

```
bulk orders: 12
mean without them  : 192.63
median without them: 146.85
orders above $500: 1131 (5.6% of all orders)
```

Say: "Removing them moves the mean by about $2. The gap comes from the whole tail: 1,131 ordinary orders above $500, a tent and a sleeping bag." The honest conclusion for the board: **mean for totals, median for a typical order, report both and say which is which.**

6 **The mode (2m).** Say: "The only center that works for categories."

```
print(orders["payment_method"].mode())
print(pd.Series([1, 1, 2, 2, 3]).mode().tolist())
top = totals.mode()[0]
print(top, "appears", (totals == top).sum(), "times out of", len(totals))
```

```
0    card
Name: payment_method, dtype: str
[1, 2]
53.9 appears 114 times out of 20302
```

Three points: `.mode()` returns a Series because ties happen (take `[0]` for one value); pandas 3 prints `dtype: str` (older versions say `object`); and the mode of a measured quantity is mostly an accident ($53.90 is mostly one popular $46.95 item plus $6.95 shipping).

7 **Same center, different spread (3m).** Say: "Center is half the story. Two store cities, daily high temperature."

```
weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
boise = weather.loc[weather["city"] == "Boise", "high_f"]
asheville = weather.loc[weather["city"] == "Asheville", "high_f"]

for name, city in [("Boise", boise), ("Asheville", asheville)]:
    print(f"{name:<10} mean {city.mean():5.1f}  median {city.median():5.1f}"
          f"  range {city.max() - city.min():3d}  SD {city.std():5.1f}")
```

```
Boise      mean  62.4  median  63.0  range 102  SD  22.5
Asheville  mean  66.5  median  66.0  range  79  SD  15.5
```

Say: "Four degrees apart on average. Nobody who has lived in both would call the climates alike. The range uses just two days, the two strangest ones. The SD uses all 731."

8 **A variance by hand (4m).** Do this one on the board first, then in code. Five days of tent sales: 4, 7, 5, 10, 4.

```
tents = np.array([4, 7, 5, 10, 4])
deviations = tents - tents.mean()

print("mean              :", tents.mean())
print("deviations        :", deviations, " sum:", deviations.sum())
squared = deviations ** 2
print("squared deviations:", squared, " sum:", squared.sum())
print("divide by n       :", squared.sum() / 5)
print("divide by n - 1   :", squared.sum() / 4)
print("square root of 6.5:", round(np.sqrt(6.5), 2))
```

```
mean              : 6.0
deviations        : [-2.  1. -1.  4. -2.]  sum: 0.0
squared deviations: [ 4.  1.  1. 16.  4.]  sum: 26.0
divide by n       : 5.2
divide by n - 1   : 6.5
square root of 6.5: 2.55
```

Say: "Plain deviations cancel to zero: the balance point again. Squaring makes them positive and counts big misses more. The variance is in tents squared, so take the square root to get back to tents: the standard deviation. Six tents a day, give or take about 2.5."

9 **Two libraries, two answers (3m).** Ask the room to predict which of the three lines will match.

```
print("NumPy  np.std(tents)          :", round(np.std(tents), 4))
print("pandas pd.Series(tents).std() :", round(pd.Series(tents).std(), 4))
print("NumPy  np.std(tents, ddof=1)  :", round(np.std(tents, ddof=1), 4))
```

```
NumPy  np.std(tents)          : 2.2804
pandas pd.Series(tents).std() : 2.5495
NumPy  np.std(tents, ddof=1)  : 2.5495
```

Say: "NumPy divides by n: `ddof=0`. pandas divides by n - 1: `ddof=1`. Population, when the data is the whole group you care about; sample, when it stands in for something bigger. If unsure, `ddof=1`.

With 20,302 rows the two differ by less than half a cent, so it is a small-sample question." Why n - 1 at all: a sample is measured against its own mean, the point it is closest to, so its deviations come out a little too small. The lesson proves it with 100,000 simulated samples; that simulation is homework reading.

10 **Spread that shrugs off extremes (3m).** Say: "An SD of $179 around a mean of $195 suggests some orders are below zero. For skewed data the SD is inflated by the same tail as the mean. The robust partner of the median is the IQR."

```
q1 = totals.quantile(0.25)
q3 = totals.quantile(0.75)
print("Q1:", q1, " Q3:", q3, " IQR:", round(q3 - q1, 2))

with_typo = pd.concat([totals, pd.Series([1_000_000.0])], ignore_index=True)
typo_iqr = with_typo.quantile(0.75) - with_typo.quantile(0.25)
print("mean  :", round(totals.mean(), 2), "->", round(with_typo.mean(), 2))
print("median:", totals.median(), "->", with_typo.median())
print("SD    :", round(totals.std(), 2), "->", round(with_typo.std(), 2))
print("IQR   :", round(q3 - q1, 2), "->", round(typo_iqr, 2))
```

```
Q1: 78.85  Q3: 264.95  IQR: 186.1
mean  : 194.6 -> 243.84
median: 146.85 -> 146.85
SD    : 178.76 -> 7019.02
IQR   : 186.1 -> 186.13
```

Say it as a sentence: "The middle half of Pinecrest's orders are between $78.85 and $264.95." Then the stress test: one bad row out of 20,303 multiplied the SD by 39 and did not move the IQR. Two standard pairs for the board: **mean with SD** for symmetric data, **median with IQR** for skewed data.

11 **Missing values (2m).** Say: "One practical matter. Six store-days have no revenue recorded."

```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
revenue = daily["revenue"]

print("rows:", len(revenue), " values present:", revenue.count())
print("pandas mean:", round(revenue.mean(), 2))
print("NumPy mean :", np.mean(revenue.to_numpy()))
print("np.nanmean :", round(np.nanmean(revenue.to_numpy()), 2))
```

```
rows: 2903  values present: 2897
pandas mean: 7435.04
NumPy mean : nan
np.nanmean : 7435.04
```

Say: "pandas skips missing values without telling you. NumPy returns `nan` without an error. Two habits: print `.count()` next to `len()`, and ask whether the missing rows could be different from the rest." Next session's survey data is where that question bites.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Two libraries, two standard deviations.** A learner's SD does not match a neighbor's or the lesson's, because one used `np.std` (divide by n) and the other `.std()` on a Series (divide by n - 1). Coach: "When your number does not match someone else's, check `ddof` first." Have them write the choice into the code, `np.std(x, ddof=1)`, so it stops being an accident.
- **Averaging averages.** To get an overall mean they average the group means from a `groupby` (the lesson's example gives $182.66 instead of $194.60). Coach: "Does Asheville's 1,413 orders count as much as Online's 12,854 in your number?" Fix: compute from the original rows, or `np.average(means, weights=counts)`.
- **Mean and SD for skewed data out of habit.** "Orders average $195 plus or minus $179" is arithmetic that paints a symmetric hump that does not exist. There is no error message, so it is the most common. Coach with one question every time a mean is reported: "What is the median? Show me the histogram." If mean and median disagree noticeably, lead with median and IQR.

GUIDED LAB

First 12 minutes, everyone: `labs/module-10/l4_1-center-and-spread/orders_per_buyer_starter.py` (reference: `labs/module-10/l4_1-center-and-spread/orders_per_buyer_solution.py`). The loyalty team says "our customers order about five times". Have learners write a guess for the order of mean, median and mode before TODO 1. Expected: mean 5.15, median 4.0, mode 1; 65.1% of buyers below the mean; mean x buyers = 20302 but median x buyers = 15768. Debrief with one question: "Which number goes in which sentence?"

Next 13 minutes: `labs/module-10/l4_1-center-and-spread/visitor_spread_starter.py`, Steps 1 to 3 (variance by hand on five days, matching NumPy and pandas, the four-store table). Expected in Step 1: 510.96 dividing by n and 638.7 dividing by n - 1. Steps 4 and 5 (relative spread, the stress test) go home. Watch for `spread()` still returning the `np.nan` placeholders: the table prints all NaN and the starter says so.

Early finishers: `labs/module-10/l4_1-center-and-spread/debug_average_of_averages.py`: three findings that run without an error and are wrong (an average of averages, a silent `nan`, two `ddof` defaults).

START THE LESSON PROJECT

The lesson's project is **The Typical Order Memo** (about 35 minutes alone, in a new notebook `lesson_4_1_project`). In the room do **Step 1**: everyone types `center_and_spread(values)`, which returns a dictionary of six numbers, and runs it on `orders["order_total"]`. They should see n 20,302.00, mean 194.60, median 146.85, sd 178.76, iqr 186.10 and mean_vs_median_pct 32.51. Say the rule of

thumb from the lesson: when the mean is more than about 10% above the median, look at a histogram before reporting a mean. If minutes remain, start Step 2 (the three-row table). Steps 2 to 5 are homework; the memo must be generated from variables, with no number typed by hand.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold call, then pair-share):

1. "The mean order is $194.60 and the median is $146.85. Which one is wrong?" **Answer:** Neither. The mean is the balance point and is pulled up by the long right tail; the median is the middle order. Mean for totals, median for a typical order.
2. "`np.std(x)` gave 2.28 and `pd.Series(x).std()` gave 2.55 on the same five numbers. Why?" **Answer:** NumPy divides by n (`ddof=0`), pandas by n - 1 (`ddof=1`). For a sample standing in for something bigger, use `ddof=1`.
3. "Your column is delivery times in days, strongly right-skewed, maybe with typing errors. Which two numbers do you report?" **Answer:** Median and IQR: both ignore the extremes. Add the mean only if someone needs a total.

Workbook self-check, items Q7 to Q12 (do two in the room if time allows; the rest is homework):

Answers: **Q7** B) `30.0`; **Q8** C) The mode; **Q9** `2.0 2.5`; **Q10** A) The mean, $194.60; **Q11** D) Median and interquartile range; **Q12** A) `200.0 nan 2`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: keep the board's two columns (center, spread) visible and have learners copy them into the workbook's notes box as each term appears. For the variance, do the five-number table by hand on paper (value, deviation, squared deviation) before touching code. Learners who are anxious about statistics should hear early that today has no formulas to memorize: every number is one method call, and the skill is choosing which to report.

Stretch: lesson Exercise 4 (break the catalog: one price loses its decimal point; compare mean, median, SD, IQR and the 5% trimmed mean from `scipy.stats.trim_mean`), or the bonus challenge (rerun the n - 1 simulation for sample sizes 2, 5, 10, 30, 100 and compare with (n - 1) / n).

CLOSING

EXIT TICKET (3 QUESTIONS, ON PAPER OR STICKY NOTES, NO NOTES)

1. A figure has `figsize=(6, 4)` and is saved with `dpi=150`. How many pixels wide and high is the PNG? **Answer:** 900 wide by 600 high (6 x 150 and 4 x 150).
2. A column of order totals is strongly right-skewed. Which measure of center and which measure of spread do you report for a typical order? **Answer:** The median and the IQR. (The mean is for totals.)

3. `np.std(x)` and `pd.Series(x).std()` give different answers for the same small list. Why? **Answer:** NumPy divides by n (`ddof=0`); pandas divides by n - 1 (`ddof=1`).

Read the slips before Session 11. If more than a quarter miss question 1, open Session 11 with one more pixel prediction. If question 3 is weak, the 4.2 demo's z-score step (which uses the SD) is the place to revisit `ddof`.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Lesson 3.4 project:** finish the Pinecrest Chart Pack (Steps 2 to 4): 9 files, every web PNG 990 x 495, a 4-page PDF, 0 open figures.
- **Lesson 3.4 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the five-question Quick Quiz, and the journal prompt (it also asks for one line on Module 3 as a whole).
- **Lesson 4.1 project:** finish the Typical Order Memo (Steps 2 to 5): the three-row table, the bulk-order check, the two-panel figure saved as `typical_order.png`, and the memo built from variables.
- **Lesson 4.1 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz, and the journal prompt. Read the section "Sample or Population? The n - 1 Question" and run its simulation once.
- **Kit labs:** finish `category_price_pack` (TODO 3 and 4) and `visitor_spread` (Steps 4 and 5).
- **Workbook:** the 12 self-check items and anything left in Predict, then run.
- **Before Lesson 4.2** (from the lesson page): run `orders["order_total"].describe()` and name every line you can; guess, then check, whether `visitors` in `daily_sales.csv` is skewed by comparing its mean and median.

BRING LIST FOR SESSION 11

- Their Lesson 4.1 notebook with a working `center_and_spread()` function: Lesson 4.2 extends it.
- A place they can copy `pinecrest_style`, `new_figure()` and `save_chart()` from (a marked cell or `charting.py`): charts appear in every remaining session.
- Data files for Session 11: `orders.csv`, `daily_sales.csv`, `store_weather.csv`, `survey.csv`, `products.csv` (all already in `data/`).
- Installs: none. Session 11 uses `scipy.stats`, which is in the course environment; anyone unsure runs `python -c "import scipy; print(scipy.__version__)"` at home.
- Servers to start: none. Accounts: none needed.
- This workbook, with the Session 10 self-check done.

IF YOU RUN LONG OR SHORT

Protect: the tiny-text demonstration (demo step 4) with both files opened side by side, the `pixel_budget` lab, mean against median on the real order totals (demo steps 3 to 5), the variance by hand, and the first step of each lesson project.

Running long, cut in this order: (1) the backgrounds step of the 3.4 demo (say the rule, show the lesson's three-panel figure); (2) the `bbox_inches` step (one sentence: "it changes the size"); (3) `category_price_pack` in the room (all of it goes home); (4) the missing-values step of the 4.1 demo (it returns in Session 11); (5) `visitor_spread` in the room. Do not cut either project start: two minutes of "your first cell runs" saves twenty at home.

Running short: run the debug handouts as a whole-room hunt (one on the projector, pairs call out the faults), do the two Predict items per lesson from the workbook together, then let learners continue the Chart Pack to Step 2 or the Memo to Step 2.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 11: Shape, Outliers and Distributions

How to teach the session that finishes describing data and starts reasoning about data not yet seen. Before the break learners measure the shape of a column, locate values in it with percentiles and z-scores, run two outlier rules that disagree, and learn that a flag is a question, not a verdict. After the break they meet four workhorse distributions through `scipy.stats`, watch the central limit theorem turn a skew of 5.3 into a bell, and build their first confidence interval.

`2 lessons` `3 hrs session` `Lessons 4.2-4.3` `Session 11 of 21`

HOW TO USE THIS GUIDE

- Same rhythm as every session: **Frame** (the goal and a mental model) → **Live demo** (you type, they type along) → **Guided lab** (the kit's lab files) → **Start the lesson project**. The lesson's exercises, quiz and journal are homework.
- **Type, do not paste.** Every code block in the demo scripts is what to type, one notebook cell per step, in the course project folder. Every output shown was produced by running that code with the course environment (pandas 3.0, SciPy 1.18).
- **Ask for a prediction before you run.** Rank the columns by skew, guess the share of days above $12,000, guess whether two outlier rules agree, guess the shape of 20,000 sample means. The surprise only teaches if they committed first.
- **The lessons are the reference.** Keep `statistics_descriptive.html` and `statistics_probability_distributions.html` open in browser tabs for the figures (the labeled box plot, the stacked survey bars, the normal curve over the spend histogram, the four-panel central limit figure, the fifty intervals) and the tables (skewness cutoffs, what to do with an outlier, the four methods, which distribution when).
- Teal = hands-on, amber = watch-outs. Labs for this session live in `labs/module-11/`: starters for learners, solutions for you.

> **THE RISK OF SESSION 11: THE DENSEST THREE HOURS OF MODULE 4**
>
> Lesson 4.2 has eight ideas and Lesson 4.3 has more: four distributions, four methods, simulation, the central limit theorem, the standard error and a confidence interval. Nobody holds all of that after one pass. Decide in advance what must land in the room: before the break, **a flag is a question, not a verdict**, and **every grouped number carries its n**; after the break, **freeze a distribution, ask it a question, check it against data**, and **the 95% describes the method**. The rest is exposure that the lesson pages and the homework will consolidate. Say so to the room: "You will not remember all four distributions tonight. You will remember how to ask one a question." The second risk is random numbers: a learner who runs a cell twice gets different results from yours and thinks they made an error. Tell them in advance that the conclusions will match even when the digits do not.

BEFORE SESSION 11

- ☐ Run the lab check from the kit folder: `python3 labs/check_labs.py module-11`. Eight lines should say PASS.
- ☐ Copy the starters and the two debug handouts (`*_starter.py`, `debug_*.py`) from `labs/module-11/` to the shared folder. Keep every `*_solution.py` on your machine.
- ☐ Projector machine: Jupyter (or VS Code notebooks) opened in the course project folder; font size at least 18 pt.
- ☐ Files used today: `data/orders.csv`, `data/order_items.csv`, `data/daily_sales.csv`, `data/products.csv`, `data/survey.csv`, `data/store_weather.csv`. No servers to start, no accounts, nothing new to install: `scipy` is already in the course environment (check once with `python -c "import scipy; print(scipy.__version__)"`).
- ☐ Run the 4.3 demo once, top to bottom, on the projector machine and confirm your outputs match this guide (they will, with `default_rng(42)` and the cells run once each in order). Then restart the kernel so the generator is fresh for class.
- ☐ Whiteboard, written before learners arrive: on the left **center · spread · shape · position** and **A flag is a question, not a verdict.**; on the right an empty four-row table headed **cdf · sf · ppf · pmf**.
- ☐ Optional, for the central limit theorem: a handful of dice (three per learner).
- ☐ **Known environment traps.** (1) Random results depend on the order in which cells are run; restart and run all to get back in step. (2) `ax.boxplot()` on a column with missing values draws an empty frame with no error: `dropna()` first. (3) Matplotlib 3.9 and later use `orientation="horizontal"` and `tick_labels=` in `boxplot`; a learner on an older Matplotlib needs `vert=False` and `labels=`. (4) pandas `.quantile()` wants 0 to 1 and raises a ValueError for 25; `np.percentile()` wants 0 to 100 and raises nothing for 0.25. (5) `scipy.stats.zscore` divides by n (`ddof=0`) unless told otherwise; the lesson computes z-scores by hand with the pandas SD. (6) `stats.skew()` (NumPy-style, used on arrays in the CLT step) and pandas `.skew()` use slightly different formulas; on 20,302 rows both print 5.48.

- ☐ Skim the Session 10 homework: the Chart Pack manifest (9 files) and the Typical Order Memo. Two "before the next lesson" tasks feed today's opening: which line of `describe()` could they not name, and is `visitors` skewed (its mean is 355.9 and its median 322, so yes, to the right).
- ☐ Sticky notes or slips of paper for the exit ticket.

SESSION 11 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 10 (saving figures, center and spread), homework check.
0:10	10	**4.2** Frame and mental model	A pile of sand: center, spread, shape, and one value's position in the pile.
0:20	25	**4.2** Live demo	Skew, quantiles, z-scores, two outlier rules that disagree, describe(), a box plot, survey answers by group.
0:45	20	**4.2** Guided lab	`labs/module-11/l4_2-shape-position-outliers/catalog_weights_starter.py`, then TODO 1 to 3 of `price_satisfaction_starter.py`.
1:05	10	**4.2** Start the lesson project	The Store Profile Sheet, Step 1: the grouped profile table prints.
1:15	10	Break	Leave the four-store profile table on the projector.
1:25	15	**4.3** Frame and mental model	The vending machine: a distribution as a stocking list; discrete or continuous.
1:40	30	**4.3** Live demo	Simulate, freeze a normal, the four methods, binomial, Poisson, the central limit theorem, a confidence interval.
2:10	25	**4.3** Guided lab	`labs/module-11/l4_3-probability-distributions/conversion_model_starter.py`, then Steps 1 to 4 of `lines_per_order_ci_starter.py`.
2:35	15	**4.3** Start the lesson project	The Returns Forecast, Steps 1 and 2: 731 days counted, mean against variance.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 12.
Total	180		Ends at 3:00.

The lesson pages estimate 90 + 100 = 190 minutes for a solo learner, and the room has 150 after the warm-up, break and wrap-up, of which 45 go to the kit labs. So this session leans on homework more than most. **Done in the room:** both mental models; skewness and the log trick; quantiles and z-scores; both outlier rules and the investigation of the bulk orders; `describe()` and the box plot; the survey table with its n; simulation with one generator; the normal model and the four methods; one binomial and one Poisson example with its mean-against-variance check; the central limit theorem table; one confidence interval and its coverage; the `catalog_weights` and `conversion_model` labs; the first step or two of each lesson project. **Goes home:** the rest of both projects (Store Profile Sheet Steps 2 to 6, Returns Forecast Steps 3 to 6), the unfinished lab steps, the lessons' exercises, Quick Quizzes and journals, and the reading-only parts: quantile interpolation, histogram bins, named aggregation with `q1` and `q3`, the full survey section (the distribution table, the stacked bar chart), the binomial check against 507 real batches, the fifty-interval figure and the six common mistakes of 4.3.

SESSION AT A GLANCE

LESSON	TOPIC	ROOM TIME	LEARNERS LEAVE ABLE TO…
4.2	Descriptive Statistics II: Shape, Position and Outliers	65 of 90 min	put a number on skew, locate a value with percentiles and z-scores, run both outlier rules and investigate what they flag, read `describe()` and a box plot, and summarize survey answers by group with their n
4.3	Probability Distributions	85 of 100 min	choose among uniform, normal, binomial and Poisson, answer probability questions with `cdf`, `sf`, `ppf` and `pmf`, check a model against data, and build and interpret a 95% confidence interval for a mean

WARM-UP

0:00 Warm-up: Retrieval from Session 10

10 min

TIMING

6m	Five questions, no notes: answer alone on paper, compare with a neighbor, then one answer per question out loud
3m	Homework check: the Chart Pack, the Typical Order Memo, the two "before the next lesson" tasks
1m	Bridge to today

FIVE RETRIEVAL QUESTIONS (NO NOTES, PAIRS, 60 SECONDS EACH)

1. A figure with `figsize=(10, 5)` is saved with `dpi=200`. How many pixels wide and high is the PNG? **Answer:** 2,000 wide by 1,000 high: pixels = inches x dpi.
2. The labels on your slide image are too small. Which two settings do you change, and what stays the same? **Answer:** A smaller `figsize` and a higher `dpi`, so the pixel size stays the same. Raising the dpi alone does not make text bigger.
3. Pinecrest's mean order is $194.60 and the median is $146.85. Why do they differ, and which describes a typical order? **Answer:** The totals are right-skewed: a long tail of large orders pulls the mean up. The median describes a typical order; the mean is for totals.
4. `np.std(x)` and `pd.Series(x).std()`: which divides by n and which by n - 1? **Answer:** NumPy divides by n (`ddof=0`); pandas divides by n - 1 (`ddof=1`).
5. What is the IQR, and why prefer it to the SD for a skewed column? **Answer:** Q3 minus Q1, the width of the middle half of the data. It is robust: extreme values cannot inflate it.

HOMEWORK CHECK

Show of hands: whose Chart Pack manifest showed 9 files with every web PNG at 990 x 495? Whose memo was generated entirely from variables? Then the two tasks from the end of Lesson 4.1. "You ran `orders["order_total"].describe()`. Which line could you not name?" (Most say `std` or the percent lines; all of them are explained in today's first demo.) "Is `visitors` skewed?" Take a guess, then the numbers: mean 355.9, median 322, so the mean is about 10% above the median: a right skew.

BRIDGE TO TODAY

Say: "Last session we described a column with a center and a spread, and we kept saying 'it depends on the shape'. Today we measure the shape, locate single values inside it, and decide what to do with the ones that stand apart. After the break the subject changes from data you have to data you have not seen yet."

LESSON PLANS

4.2 Descriptive Statistics II: Shape, Position and Outliers

65 min

TIMING

10m	Frame: the pile of sand; shape is the first thing to check
25m	Live demo: 11 short steps on order totals, daily revenue and the survey
20m	Guided lab: `catalog_weights`, then the start of `price_satisfaction`
10m	Start the lesson project: the Store Profile Sheet, Step 1

OBJECTIVES (THE LESSON'S OWN)

- Describe the shape of a distribution with a histogram and a skewness value, and say what the shape implies for your choice of summary
- Locate a value within its column using percentiles, quantiles and z-scores
- Apply the IQR and z-score outlier rules, explain why they disagree, and choose a defensible action for flagged values
- Read every line of `describe()` and every part of a box plot
- Summarize by group with `groupby().agg()`, and summarize 1-to-5 survey answers as the ordered categories they are

MENTAL MODEL TO SAY

Say: "Pour every value of a column onto a number line, like sand. The pile that forms is the distribution, and a histogram is a photograph of it. Center says where the pile sits. Spread says how wide it is. Shape says whether it is one even mound, a mound with a long ramp on one side, or two mounds. Every other summary is an attempt to describe that pile without showing the photograph."

Draw three piles on the board: an even mound, a mound with a ramp to the right, two mounds. Under them write the four words of the module so far: **center, spread, shape, position**. Last session covered the first two; today adds the other two and the question that follows from them: what do you do with the values that stand apart? Give the answer away early and repeat it all session: **a flag is a question, not a verdict.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `session11_shape.ipynb`. The numbers are the lesson's own, so learners can find every step again on the lesson page tonight. The demo saves one figure with `fig.savefig` (in a notebook it also appears under the cell). Ask for a prediction before each comparison. If time is short, say steps 4 and 8 instead of typing them.

1. **Put a number on shape (3m).** New notebook. Say: "Last session we judged shape by eye and by mean against median. Skewness is the number: zero for a symmetric pile, positive when the long tail is on the right." Have the room rank the four columns from most symmetric to most skewed before you run.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
products = pd.read_csv("data/products.csv")
totals = orders["order_total"]
daily["spend_per_visitor"] = daily["revenue"] / daily["visitors"]
spend = daily["spend_per_visitor"].dropna()

print("product ratings   :", round(products["rating"].skew(), 2))
print("spend per visitor :", round(spend.skew(), 2))
print("daily revenue     :", round(daily["revenue"].skew(), 2))
print("order totals      :", round(totals.skew(), 2))
no_bulk = totals[totals <= 2000]
print("  without the 12 bulk orders:", round(no_bulk.skew(), 2))
print("  log(order totals)         :", round(np.log(totals).skew(), 2))
```

```
product ratings   : -0.02
spend per visitor : 0.27
daily revenue     : 1.51
order totals      : 5.48
  without the 12 bulk orders: 1.64
  log(order totals)         : -0.41
```

Write the convention on the board: between -0.5 and 0.5 roughly symmetric (mean and SD are fair), beyond 1 strongly skewed (median and IQR). Two cautions from the output: skewness is not robust (twelve orders move it from 1.64 to 5.48), and it says nothing about the number of humps. The last line is the log trick: 5.48 becomes -0.41.

2. **Position by rank (3m).** Say: "Center, spread and shape describe the whole pile. Position describes one value's place in it. 'This order was $517' says little. 'Larger than 95% of orders' says a lot."

```
cuts = [0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99]
print(totals.quantile(cuts).to_string())

print("share of orders at or below $500:", round((totals <= 500).mean(), 4))
```

```
0.10     41.9500
0.25     78.8500
0.50    146.8500
0.75    264.9500
0.90    405.3950
0.95    516.7975
0.99    737.4430
share of orders at or below $500: 0.9443
```

Read down the list out loud: one order in ten is under $42, half are under $147, one in a hundred is over $737. Then the reverse question, with last session's tool: the mean of a boolean is a proportion, so a $500 order sits at about the 94th percentile. Warn now: pandas `.quantile()` takes 0 to 1, NumPy's `np.percentile()` takes 0 to 100.

3 **Position by distance: the z-score (3m).** Write **z = (value - mean) / SD** on the board. Ask: "Is a $12,000 day a big deal?" Let them argue, then run.

```python
for store in ["Asheville", "Boise"]:
    revenue = daily.loc[daily["store"] == store, "revenue"]
    z = (12000 - revenue.mean()) / revenue.std()
    share = (revenue >= 12000).mean()
    print(f"{store:<9} mean {revenue.mean():7,.0f} SD {revenue.std():5,.0f}"
          f"  z = {z:5.2f}  days at or above: {share:.1%}")
```

```
Asheville mean   5,704 SD 2,064  z =  3.05  days at or above: 1.5%
Boise     mean  10,144 SD 3,715  z =  0.50  days at or above: 24.3%
```

Say: "The same $12,000 is three standard deviations above Asheville's average and half a standard deviation above Boise's. The raw number cannot tell you that."

4 **Standardizing does not change the shape (2m).** Say: "Convert the whole column." Ask what the mean and SD of a z-scored column must be.

```python
z_totals = (totals - totals.mean()) / totals.std()
print("mean of z:", abs(round(z_totals.mean(), 6)),
      "  SD of z:", round(z_totals.std(), 6))
print("largest z:", round(z_totals.max(), 1),
      "  smallest z:", round(z_totals.min(), 2))
```

```
mean of z: 0.0   SD of z: 1.0
largest z: 26.3   smallest z: -1.05
```

Mean 0 and SD 1, always. But the largest order is 26 SDs above the mean and the smallest barely 1 below: the pile is as lopsided as before. "95% within 2 SDs" belongs to one bell-shaped distribution, which is the second half of today.

5 **Outlier rule 1: the IQR fences (3m).** Say: "An outlier is a value far from the rest. 'Far' needs a definition. Rule one: a fence 1.5 IQRs beyond each quartile. This is the rule box plots use."

```
q1, q3 = totals.quantile(0.25), totals.quantile(0.75)
iqr = q3 - q1
lower_fence = q1 - 1.5 * iqr
upper_fence = q3 + 1.5 * iqr
flag_iqr = (totals < lower_fence) | (totals > upper_fence)

print("fences:", round(lower_fence, 2), "to", round(upper_fence, 2))
print("flagged:", flag_iqr.sum(), f"({flag_iqr.mean():.1%} of orders)")
```

```
fences: -200.3 to 544.1
flagged: 848 (4.2% of orders)
```

Point at the lower fence: minus $200.30. No order can ever be flagged as too small, and 4% are flagged as too large.

6 **Outlier rule 2 disagrees (3m).** Say: "Rule two: z beyond 3. Same data. Same answer?"

```
flag_z = z_totals.abs() > 3
print("flagged:", flag_z.sum(), f"({flag_z.mean():.1%} of orders)")
print("smallest flagged order:", totals[flag_z].min())

print(f"IQR-flagged orders: {flag_iqr.mean():.1%} of orders, "
      f"{totals[flag_iqr].sum() / totals.sum():.1%} of revenue")
```

```
flagged: 217 (1.1% of orders)
smallest flagged order: 731.85
IQR-flagged orders: 4.2% of orders, 15.4% of revenue
```

848 or 217. Ask why. (The mean and SD are inflated by the very values being judged; the quartiles are not.) Then the last line: deleting what the IQR rule flags would throw away 4% of the orders and over 15% of the money. Say the sentence of the lesson: **"A flag is a question, not a verdict."**

7 **Investigate before you remove (3m).** Say: "So ask the question. Are the twelve largest orders errors? Look them up."

```
items = pd.read_csv("data/order_items.csv")
bulk_ids = orders.loc[totals > 2000, "order_id"]
bulk_lines = items[items["order_id"].isin(bulk_ids)]
largest = bulk_lines.groupby("order_id")["quantity"].max()

print("orders above $2,000:", len(bulk_ids))
print("largest quantity on each:", sorted(largest.tolist()))
print(orders.loc[totals > 2000, ["channel", "status"]].value_counts())
```

```
orders above $2,000: 12
largest quantity on each: [25, 26, 27, 28, 30, 30, 30, 32, 35, 38, 39, 39]
channel  status
online   completed    12
Name: count, dtype: int64
```

Every one is a completed online order with one line of 25 to 39 units: outfitters buying in bulk. Real sales. Walk the lesson's decision chart on the board: provable error? fix or remove it. Not an error but not part of your question? analyze separately and say so. Otherwise keep it and stay robust (median, IQR).

8 **Read describe() properly (2m).** Say: "You have run this since the first lesson. You can now read every line."

```
print(totals.describe().round(2).to_string())
```

```
count    20302.00
mean       194.60
std        178.76
min          6.38
25%         78.85
50%        146.85
75%        264.95
max       4893.00
```

Three checks to do in your head every time: is `count` lower than the number of rows; are `mean` and `50%` far apart; are the two halves of the box unequal (264.95 - 146.85 against 146.85 - 78.85)? `std` is the sample SD (`ddof=1`).

9 **A box plot is the IQR rule, drawn (3m).** Say: "A box plot shows the median, the quartiles, and the fence rule in one shape." Type it with `dropna()` and say why: with a missing value Matplotlib draws an empty frame and raises nothing.

```
asheville = daily.loc[daily["store"] == "Asheville", "revenue"].dropna()
a_q1, a_med, a_q3 = asheville.quantile([0.25, 0.5, 0.75])
a_iqr = a_q3 - a_q1
whisker_hi = asheville[asheville <= a_q3 + 1.5 * a_iqr].max()
whisker_lo = asheville[asheville >= a_q1 - 1.5 * a_iqr].min()

fig, ax = plt.subplots(figsize=(9, 2.6), layout="constrained")
ax.boxplot(asheville, orientation="horizontal", widths=0.5)
ax.set_yticks([])
ax.set_xlabel("Asheville daily revenue ($)")
fig.savefig("asheville_box.png", dpi=110)

print("Q1, median, Q3:", round(a_q1), round(a_med), round(a_q3))
print("whiskers end at:", round(whisker_lo), "and", round(whisker_hi))
print("points beyond the upper fence:",
      (asheville > a_q3 + 1.5 * a_iqr).sum())
```

```
Q1, median, Q3: 4263 5208 6652
whiskers end at: 2268 and 10229
points beyond the upper fence: 23
```

Point at the picture: the line in the box is the median, not the mean; the box is the middle half; each whisker ends at the last real data point inside the fence, not at the fence and not at the maximum; separate points are what the rule flags. A box plot compares groups well and hides humps.

10 **Summaries by group carry their n (2m).** Say: "Most real questions are comparisons, so most real summaries are grouped."

```
by_store = daily.groupby("store")["revenue"].agg(
    ["count", "mean", "median", "std"])
print(by_store.round(0))
```

```
           count     mean  median     std
store
Asheville    726   5704.0  5208.0  2064.0
Bend         721   6998.0  6456.0  2546.0
Boise        726  10144.0  9360.0  3715.0
Flagstaff    724   6890.0  6405.0  2686.0
```

Say: "A mean with no n beside it cannot be judged: is that group 700 days or 7?" The counts differ by the six missing revenue values. The lesson goes on to named aggregation with `q1` and `q3` functions; that is the project's Step 1.

11 **Survey answers are ordered categories (4m).** Say: "Pinecrest surveyed 1,800 customers on a 1-to-5 scale. That looks like numeric data. It is not, quite: 4 is more satisfied than 3, but nothing says the steps are equal. The word is **ordinal**."

```
survey = pd.read_csv("data/survey.csv")

def top_two(values):
    return (values.dropna() >= 4).mean() * 100

def bottom_two(values):
    return (values.dropna() <= 2).mean() * 100

staff = survey.groupby("store")["sat_staff"].agg(
    n="count", rows="size", median="median",
    satisfied_pct=top_two, dissatisfied_pct=bottom_two)
print(staff.round(1))
```

```
             n  rows  median  satisfied_pct  dissatisfied_pct
store
Asheville  193   202     4.0           57.5              14.0
Bend       228   232     4.0           78.9               4.4
Boise      320   329     4.0           76.9               2.2
Flagstaff  206   213     4.0           74.8               4.4
Online     248   824     4.0           74.6               2.8
```

Two findings in one table. First: the median is 4 in every store and tells you nothing, while the shares do: Asheville has 57.5% satisfied against 75 to 79% elsewhere, and 14% dissatisfied. Second: compare `n` with `rows` for Online: 248 answers from 824 respondents. Most online shoppers never met the staff, so the answers are **not missing at random**. Whether Asheville's gap could be a fluke is a question for Session 12; describing it clearly, with counts, comes first.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Passing a percentage where a fraction is wanted, or the reverse.** `totals.quantile(25)` raises `ValueError: percentiles should all be in the interval [0, 1]`; the dangerous twin raises nothing: `np.percentile(totals, 0.9)` quietly returns the 0.9th percentile. Coach: "Which function, and which scale does it take? pandas `.quantile()`: 0 to 1. NumPy `np.percentile()`: 0 to 100." Have them check a result against `describe()`.
- **Deleting whatever the rule flags.** "The IQR rule says they are outliers, so I removed them." Coach with two questions from the lesson's chart: "Is it a provable error? Is it part of your question?" Then make it concrete: the 848 flagged orders are 15.4% of revenue. Remove only provable errors, write down what was removed, and use the median and IQR so the rest cannot distort the summary.

- **A grouped statistic without its n, or a Likert mean on its own.** A table of group means looks finished. Coach: "How many answers is each mean based on? Add `count` and `size`." For 1-to-5 answers: "Show me the distribution, or the share satisfied, next to that mean." A related silent failure worth naming: `ax.boxplot()` on a column with missing values draws an empty frame; `dropna()` first.

GUIDED LAB

First 12 minutes, everyone: `labs/module-11/l4_2-shape-position-outliers/catalog_weights_starter.py` (reference: `labs/module-11/l4_2-shape-position-outliers/catalog_weights_solution.py`). The shipping team asked to "remove the outlier weights first". Learners run the whole workflow on `weight_kg`: shape, position, both rules, then look at what was flagged. Expected: skew 3.46 (0.18 after a log); 26 products flagged by the IQR rule and 4 by the z-score rule; 11 of the 26 are tents; nothing removed. Watch Step 2: 14 products have no weight, and comparing a missing value with 2 gives False, so the share needs `dropna()` (89.9%, not 84.7%). Debrief with one question: "What do you send back to the shipping team?"

Last 8 minutes: `labs/module-11/l4_2-shape-position-outliers/price_satisfaction_starter.py`, TODO 1 to 3 (the distribution table, `bottom_two()`, the grouped table). The grouped table is the payoff: ages 18 to 24 are 19.4% satisfied and 38.2% dissatisfied with prices, against about 44% and 15% for everyone else, and the medians hide it. TODO 4 to 6 (who skipped, saving the chart, the sentence) go home. The chart is saved to `OUTPUT`; the lab never calls `plt.show()`.

Early finishers: `labs/module-11/l4_2-shape-position-outliers/debug_summary_report.py`: four results that run without an error and are wrong.

START THE LESSON PROJECT

The lesson's project is **The Store Profile Sheet** (about 40 minutes alone, in a new notebook `lesson_4_2_project`). The operations manager wants a profile of daily revenue at the four stores and has asked to "take out the weird days first". In the room do **Step 1**: drop the six store-days with no revenue (2,903 rows become 2,897), add the weekday name, and build the grouped profile with `q1` and `q3` functions and named aggregation. Everyone should see four rows: Asheville's median 5,208, Bend's 6,456, Boise's 9,360, Flagstaff's 6,405, and Boise's maximum of 27,677 far beyond its 95th percentile of 16,834. Do not give away the ending (all 91 flagged days turn out to be weekends, promotions or December days). Steps 2 to 6 are homework.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold call, then pair-share):

1. "Daily revenue has a skewness of 1.51. Which center and spread do you report, and what would you draw first?" **Answer:** Median and IQR (beyond 1 is strongly skewed), and a histogram first: skewness does not show how many humps there are.

2. "A value is flagged by the IQR rule. What are the two questions you ask before doing anything?" **Answer:** Is it a provable error? Is it part of the question I am answering? Remove only provable errors; otherwise keep it (and stay robust) or analyze it separately and say so.

3. "On a box plot, where does the upper whisker end?" **Answer:** At the largest data value that is still within 1.5 IQRs above Q3. Not at the fence itself, and not at the maximum unless nothing is flagged.

Workbook self-check, items Q1 to Q6 (do two in the room if time allows; the rest is homework):

Answers: **Q1** C) On the log scale the pile is roughly symmetric, so methods that prefer symmetric data will behave better there; **Q2** `25.0`; **Q3** A) The day is far above Asheville's own average, measured in Asheville's SDs, and only a little above Boise's; **Q4** D) The mean and SD are inflated by the same large orders being judged, which pushes the z-score line outward; the quartiles are not; **Q5** `14.5 1`; **Q6** B) Report it with its n and say who answered: 70% of Online shoppers skipped the question, so it describes the minority who dealt with staff. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: keep the lesson's outlier decision chart on screen during the lab and have learners trace it with a finger for one flagged product. For z-scores, do one by hand on the board with round numbers (a store with mean 300 and SD 100: what is the z of a 500-visitor day?) before any code. Give anyone lost in named aggregation the pattern on paper: `agg(new_name="function")`.

Stretch: lesson Exercise 4 (the outlier rule on the log scale), or extend the lab: z-scores of weight within each category with `groupby().transform()`, the technique the project's Step 2 uses. Does a 6-person tent still look unusual among tents?

4.3 Probability Distributions

85 min

TIMING

15m	Frame: the vending machine; discrete or continuous; why the rest of the module needs this
30m	Live demo: 12 short steps, one seeded generator, run in order
25m	Guided lab: `conversion_model`, then Steps 1 to 4 of `lines_per_order_ci`
15m	Start the lesson project: the Returns Forecast, Steps 1 and 2

OBJECTIVES (THE LESSON'S OWN)

- Explain what a random variable and a probability distribution are, and tell discrete from continuous
- Choose between the uniform, normal, binomial and Poisson distributions for a situation, and check the choice against data
- Answer probability questions with `scipy.stats` using `pmf` / `pdf`, `cdf`, `sf` and `ppf`
- Simulate random outcomes reproducibly with `np.random.default_rng` and use simulation to check a calculation
- Demonstrate the central limit theorem by simulation, compute a standard error, and build and correctly interpret a 95% confidence interval for a mean

MENTAL MODEL TO SAY

Say: "Picture a vending machine that drops a random snack each time you press the button. You cannot say what the next press will give you. But if someone tells you the machine is stocked with 50% chips, 30% candy and 20% cookies, you know a great deal: what to expect most often, how surprised to be by three cookies in a row, and roughly what 100 presses will produce. That stocking list is a probability distribution: every possible outcome, with its probability."

Then the two kinds, as two columns on the board: **discrete** (counted: returns in a day; binomial, Poisson; one exact value has a real probability) and **continuous** (measured: spend per visitor; uniform, normal; only ranges have probability). Finish the frame with why this lesson exists: "Every tool in the rest of this module, tests, A/B tests, confidence intervals, is built by asking 'what would chance alone produce?' Distributions are how that question gets an answer."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `session11_distributions.ipynb`. **Create the generator once, in the first cell, and run every cell once, in order.** A generator hands out its numbers in sequence: run a cell twice or skip one and every random result after it changes a little (the conclusions do not). The outputs below come from running these cells top to bottom with `default_rng(42)`. The lesson page runs a longer sequence of

cells, so its random results (the auditor's sample, the coverage percentage) differ slightly from the ones here: same method, different draws. That difference is itself a teaching point.

1. **One generator, and the law of large numbers (3m).** New notebook. Say: "Until now we described data that exists. A business also plans for data that does not exist yet. We cannot know tomorrow's number. We can know how it behaves." Create the generator once and say so.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(42)

orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])

many = rng.integers(1, 7, size=100_000)      # 100,000 rolls of a die
shares = pd.Series(many).value_counts(normalize=True).sort_index()
print(shares.round(3).tolist())
print("mean of the rolls:", round(many.mean(), 3))
```

```
[0.167, 0.167, 0.165, 0.167, 0.168, 0.166]
mean of the rolls: 3.5
```

Each face comes up very close to 1/6, and the average roll settles on 3.5, the **expected value**: no single roll is ever 3.5. Say: "That is why simulation works. If you can simulate a process, you can estimate any probability about it by counting." Tell learners their random numbers will match yours only if they run the same cells in the same order, once each.

2. **Two routes to one answer (2m).** Say: "A customer is equally likely to walk in at any moment of an hour. Chance they arrive in the first 15 minutes?"

```
arrivals = rng.uniform(0, 60, size=100_000)       # minutes past the hour
print("simulated:", round((arrivals < 15).mean(), 3))

arrival = stats.uniform(loc=0, scale=60)          # a frozen distribution
print("exact    :", arrival.cdf(15))
```

```
simulated: 0.25
exact    : 0.25
```

The simulation counted; the scipy object calculated. Name the object: a **frozen distribution**, parameters fixed, ready to answer questions. For the uniform, `scale` is the width, not the end.

3 **Freeze a normal with the data's own mean and SD (3m).** Say: "Last lesson's even mound: revenue per visitor. A normal distribution is completely described by two numbers."

```
spend = (daily["revenue"] / daily["visitors"]).dropna()
mu, sigma = spend.mean(), spend.std()
spend_model = stats.norm(loc=mu, scale=sigma)
print("mean:", round(mu, 2), " SD:", round(sigma, 2), " n:", len(spend))

for k in (1, 2, 3):
    in_data = ((spend - mu).abs() < k * sigma).mean()
    in_theory = stats.norm.cdf(k) - stats.norm.cdf(-k)
    print(f"within {k} SD:  data {in_data:.1%}   normal {in_theory:.1%}")
```

```
mean: 20.87  SD: 3.0  n: 2897
within 1 SD:  data 68.6%   normal 68.3%
within 2 SD:  data 95.1%   normal 95.4%
within 3 SD:  data 99.6%   normal 99.7%
```

The 68-95-99.7 rule, and its fine print: it is a property of the normal shape. The spend data follows it to within a fraction of a percentage point; order totals would not. Show the lesson's histogram with the curve laid over it from the lesson page.

4 **Four questions, four methods (4m).** Draw the lesson's table on the board as you go: `cdf(x)` this or less; `sf(x)` more than this; `ppf(q)` which value has q below it; `pmf(k)` exactly k (discrete). Have learners say the method before you type each line.

```
print("P(spend below $15): model", round(spend_model.cdf(15), 3),
      " data", round((spend < 15).mean(), 3))
print("P(spend above $25): model", round(spend_model.sf(25), 3),
      " data", round((spend > 25).mean(), 3))
print("P($18 to $24)     : model",
      round(spend_model.cdf(24) - spend_model.cdf(18), 3),
      " data", round(((spend >= 18) & (spend <= 24)).mean(), 3))
print("95th percentile   : model", round(spend_model.ppf(0.95), 2),
      " data", round(spend.quantile(0.95), 2))
```

```
P(spend below $15): model 0.025  data 0.019
P(spend above $25): model 0.084  data 0.087
P($18 to $24)     : model 0.683  data 0.686
95th percentile   : model 25.8  data 26.01
```

Say: "A model summarizes 2,897 numbers with two. It buys the ability to ask about values you have never seen, at the cost of small errors you can measure." It is close in the middle and a little off in the tails. One warning: for a continuous variable `pdf` is the height of the curve, not a probability. Probabilities are areas: differences of `cdf`.

5 **Binomial: successes out of n tries (3m).** Say: "A counting question. Out of the next 40 orders, how many will come back?" Fixed number of tries, independent, same probability each time.

```
is_returned = orders["status"] == "returned"
p_return = is_returned.mean()
returns_in_40 = stats.binom(n=40, p=p_return)
print("return rate:", round(p_return, 4),
      " expected in 40:", round(returns_in_40.mean(), 2))

for k in range(4):
    print(f"P(exactly {k} returns) = {returns_in_40.pmf(k):.3f}")
print(f"P(5 or more)        = {returns_in_40.sf(4):.3f}")
```

```
return rate: 0.0563  expected in 40: 2.25
P(exactly 0 returns) = 0.098
P(exactly 1 returns) = 0.235
P(exactly 2 returns) = 0.273
P(exactly 3 returns) = 0.207
P(5 or more)        = 0.072
```

Circle `sf(4)`: "more than 4" is "5 or more" for a count. Off-by-one slips here are the most common binomial error. Mention the assumption that matters at Pinecrest: online orders come back about twice as often as store orders (6.9% against 3.4%), so an all-online batch is not this model.

6 **Check it by simulation (1m).** Say: "Do not take the formula on faith. Count."

```
sim = rng.binomial(n=40, p=p_return, size=100_000)
print("simulated P(5 or more):", round((sim >= 5).mean(), 3))
```

```
simulated P(5 or more): 0.073
```

Formula (0.072) and simulation agree to within a thousandth. The lesson adds a third check, against 507 real batches.

7 **Poisson: events in an interval (4m).** Say: "A different counting question. How many returned orders does a day produce? There is no fixed number of tries, just events scattered through time. One parameter: the average rate."

```
all_days = pd.date_range("2024-01-01", "2025-12-31")
returns_per_day = (orders[is_returned].groupby("order_date").size()
                   .reindex(all_days, fill_value=0))
lam = returns_per_day.mean()
print("days:", len(returns_per_day), " mean:", round(lam, 3),
      " variance:", round(returns_per_day.var(), 3))

returns_model = stats.poisson(mu=lam)
observed = returns_per_day.value_counts().sort_index()
expected = returns_model.pmf(observed.index) * len(returns_per_day)
print(pd.DataFrame({"days observed": observed,
                    "Poisson expects": expected.round(1)}))
```

```
days: 731  mean: 1.564  variance: 1.526
   days observed  Poisson expects
0            151            153.1
1            239            239.3
2            190            187.1
3             98             97.5
4             39             38.1
5             11             11.9
6              2              3.1
7              1              0.7
```

Two things to stress. The `reindex(..., fill_value=0)` puts the zero-return days back; without it the mean is wrong. And the check: **mean about equal to variance** is the fingerprint of a Poisson. The fit is striking: 151 zero-return days observed against 153 expected.

8 **A count that is not Poisson (2m).** Say: "Do not assume every count is Poisson. Total orders per day."

```
orders_per_day = orders.groupby("order_date").size()
print("mean    :", round(orders_per_day.mean(), 1))
print("variance:", round(orders_per_day.var(), 1))

print("count covering 95% of days:", returns_model.ppf(0.95))
print("P(5 or more returns)      :", round(returns_model.sf(4), 3))
```

```
mean    : 27.8
variance: 64.4
count covering 95% of days: 4.0
P(5 or more returns)      : 0.022
```

The variance is more than twice the mean: weekends, seasons and sale days move the rate. The word is **overdispersion**, and a Poisson model there would be far too confident. The last two lines are the

planning questions the returns model can answer; they are the heart of the lesson project.

9 **The central limit theorem, watched (4m).** The big idea of the lesson. Say: "Order totals are about as far from a bell as data gets: skew 5.5. Draw a sample of n orders, take its mean. Do that 20,000 times. What does the pile of means look like?"

```
population = orders["order_total"].to_numpy()
sd = population.std()
print("    n  mean of means  SD of means  sigma/sqrt(n)   skew")
for n in (5, 30, 200):
    means = rng.choice(population, size=(20_000, n)).mean(axis=1)
    print(f"{n:>5} {means.mean():>14.2f} {means.std():>12.2f} "
          f"{sd / np.sqrt(n):>14.2f} {stats.skew(means):>6.2f}")
```

```
    n  mean of means  SD of means  sigma/sqrt(n)   skew
    5         194.58        79.08          79.94   2.21
   30         194.70        32.42          32.64   0.93
  200         194.43        12.49          12.64   0.39
```

Three things, one per column: the means center on the true mean ($194.60); their spread shrinks like sigma over root n (to halve it you need four times the data); their shape approaches the normal (skew falls toward 0). Then what the CLT does **not** say: it does not make the data normal. A million orders are exactly as skewed as before. Show the lesson's four-panel figure.

10 **One sample, a standard error, an interval (4m).** Say: "In real work you get one sample and never see the population. An auditor pulls 200 orders."

```
sample = rng.choice(population, size=200, replace=False)
x_bar, s = sample.mean(), sample.std(ddof=1)
se = s / np.sqrt(len(sample))
low, high = stats.t.interval(0.95, df=len(sample) - 1, loc=x_bar, scale=se)

print("sample mean:", round(x_bar, 2), " sample SD:", round(s, 2))
print("standard error:", round(se, 2),
      " scipy.stats.sem:", round(stats.sem(sample), 2))
print(f"95% CI: {low:.2f} to {high:.2f}")
print("true mean:", round(population.mean(), 2))
```

```
sample mean: 182.93  sample SD: 147.89
standard error: 10.46  scipy.stats.sem: 10.46
95% CI: 162.31 to 203.55
true mean: 194.6
```

Write **SE = s / sqrt(n)** and **mean plus or minus about 2 SE** on the board. SD against SE: the SD (about $148 here) says how much single orders vary and does not shrink with more data; the SE (about $10) says how much the sample mean would vary from sample to sample. This auditor's sample mean is $12 below the truth, and the interval still contains $194.60. (The lesson page's sample differs, because its generator had been used for different draws: same method, different 200 orders.)

11 **What "95% confident" means (2m).** Say: "The 95% describes the method, not this one interval. We own the population, so we can check: 10,000 samples, 10,000 intervals."

```
n = 200
big = rng.choice(population, size=(10_000, n))
m = big.mean(axis=1)
e = big.std(axis=1, ddof=1) / np.sqrt(n)
t_mult = stats.t.ppf(0.975, df=n - 1)
covered = ((m - t_mult * e <= population.mean())
           & (population.mean() <= m + t_mult * e))
print(f"{covered.mean():.1%} of 10,000 intervals contain the true mean")
```

```
93.9% of 10,000 intervals contain the true mean
```

Close to 95%, a little short because the data is so skewed. Say exactly this: "If we repeated the whole procedure many times, about 95% of the intervals would contain the true mean. Any single interval either contains it or does not." And what an interval is not: it is not a range that holds 95% of the orders.

12 **Make the classic mistake on purpose (1m).** Say: "scipy rarely complains. Watch."

```
wrong = stats.norm(loc=mu, scale=sigma ** 2)     # variance passed as scale
right = stats.norm(loc=mu, scale=sigma)
print(round(wrong.cdf(15), 3), round(right.cdf(15), 3))
```

```
0.257 0.025
```

`scale` is the standard deviation. Passing the variance makes the bell three times too wide, with no error: 25.7% instead of 2.5%. Textbooks write N(mean, variance), which is where the habit comes from.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Giving scipy the wrong meaning for an argument.** `scale=sigma ** 2` (the variance where the SD is wanted), or `stats.uniform(10, 20)` read as low and high when the second argument is the width (the range is 10 to 30). No error either way. Coach: "Name the arguments, `loc=` and `scale=`, and print `model.ppf([0.025, 0.975])` to see whether the range is believable."
- **Off by one with `sf` on a count, or `cdf` and `ppf` swapped.** "5 or more" is `sf(4)`, not `sf(5)`. And `ppf(15)` returns `nan` because `ppf` takes a probability and returns a value. Coach: "Say the

question in words first: this or less, more than this, or which value has this much below it? Then pick the method."

- **Misreading a confidence interval.** Two versions: "95% of orders are between $162 and $204" (it is a statement about the mean, not about single orders: only about a tenth of orders fall in such a range), and "there is a 95% probability the true mean is in this interval" (the 95% describes the method: about 95 of 100 intervals built this way contain the true mean). Coach by rerunning the coverage cell and asking: "What is the 95% a percentage of?"

GUIDED LAB

First 13 minutes, everyone: `labs/module-11/l4_3-probability-distributions/conversion_model_starter.py` (reference: `labs/module-11/l4_3-probability-distributions/conversion_model_solution.py`). Conversion is transactions divided by visitors. Learners describe it, freeze `stats.norm`, check the 68-95-99.7 rule against the data, then answer three managers' questions. Have them write the method name next to each question before coding it (the starter has blanks for this). Expected: mean 0.2354, SD 0.0271, skew 0.05; within 1, 2 and 3 SD the data has 69.4%, 95.6%, 99.5%; P(below 20%) model 0.096 against 0.090 in the data; a bad day (5th percentile) is about 19.1%.

Next 12 minutes: `labs/module-11/l4_3-probability-distributions/lines_per_order_ci_starter.py`, Steps 1 to 4: an intern's sample of 150 orders, the standard error two ways, a 95% t interval, and the prediction in Step 4 (what share of individual orders lies inside the interval?). Expected: sample mean 2.093, SE 0.0821, interval 1.931 to 2.256, which contains the true mean of 2.241. Steps 5 and 6 (coverage over 5,000 repeats; four times the sample) go home. The file is seeded: run top to bottom, everyone gets the same numbers.

Early finishers: `labs/module-11/l4_3-probability-distributions/debug_delivery_model.py`: four model answers that run without an error, all wrong.

START THE LESSON PROJECT

The lesson's project is **The Returns Forecast** (about 40 minutes alone, in a new notebook `lesson_4_3_project`): a checked Poisson model of daily returns, with capacity numbers and a confidence interval. In the room do **Steps 1 and 2**. Step 1 builds the daily series including the zero days (`reindex(days, fill_value=0)`): 731 days, 1,143 returned orders, busiest day 7, 151 days with none. Step 2 checks the model before trusting it: mean 1.564 against variance 1.526, then the observed-against-expected table, where no count is off by more than about three days. Say what the remaining steps answer so nobody opens them cold: how much capacity covers 95% of days, what a simulated year looks like, and a margin of error on the rate. Steps 3 to 6 are homework.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold call, then pair-share):

1. "A warehouse checks 60 parcels, each with the same 2% chance of damage, independently. Which distribution, and which parameters?" **Answer:** Binomial with n = 60 and p = 0.02: a fixed number of independent tries with the same probability.

2. "I have `model = stats.norm(loc=20.87, scale=3.0)`. Which method tells me the spend that only 5% of days fall below?" **Answer:** `model.ppf(0.05)`: `ppf` takes a probability and returns a value (about $15.94).

3. "The standard deviation of my sample is $148 and the standard error is $10. Which one shrinks if I collect four times as much data, and by how much?" **Answer:** The standard error: it halves (it divides by the square root of n). The SD describes single orders and stays about the same.

Workbook self-check, items Q7 to Q12 (do two in the room if time allows; the rest is homework):

Answers: **Q7** D) Poisson; check that the mean and the variance are about equal; **Q8** `0.5`; **Q9** A) `batch.sf(2)`; **Q10** B) About $8.94; **Q11** C) 800; **Q12** A) They are identical: each pass builds a new generator from the same seed, which replays the same numbers. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: give learners the lesson's four-methods table on paper and have them tick the method for each question before typing. For the CLT, do it physically if you can: three dice per person, each person reports the mean of their three, and you tally the means on the board; the tally is a mound even though one die is flat. Learners who are shaky on simulation should rerun the dice cell with 10, 1,000 and 100,000 rolls and watch the shares settle.

Stretch: lesson Exercise 4 (how the interval's width depends on n, for 50, 200, 800 and 3,200), or extend the lab: is conversion more variable than a binomial model allows? For a store-day with 330 visitors and p = 0.235, the binomial SD of the rate is sqrt(p(1 - p) / 330); compare it with the observed SD (0.0271) and explain the gap.

CLOSING

◆ EXIT TICKET (3 QUESTIONS, ON PAPER OR STICKY NOTES, NO NOTES)

1. The IQR rule flags 91 store-days as unusually high. What do you do before removing any of them? **Answer:** Investigate them: are they provable errors, and are they part of the question? (In the lesson's project all 91 are weekends, promotions or December days: real, and kept.)

2. You have a frozen distribution called `model`. Which method returns the value that 95% of the distribution lies below? **Answer:** `model.ppf(0.95)`. (`cdf` goes from a value to a probability; `ppf` goes back.)

3. A 95% confidence interval for the mean order total is $162 to $204. Does that mean 95% of orders are between $162 and $204? If not, what does the 95% describe? **Answer:** No. It is a statement about the mean, not about

single orders. The 95% describes the method: about 95 of 100 intervals built this way contain the true mean.

Read the slips before Session 12. Question 3 matters most: Session 12's hypothesis tests are built on the same standard error and the same "what would chance alone produce" logic. If more than a quarter answer "yes, 95% of orders", open Session 12 with the coverage simulation again.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Lesson 4.2 project:** finish the Store Profile Sheet (Steps 2 to 6): z-scores within each store, the flagged days and what they turn out to be, the box plot saved as `store_profile.png`, the written profile.
- **Lesson 4.2 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the five-question Quick Quiz, and the journal prompt. Read the survey section in full, including "Who Did Not Answer?"
- **Lesson 4.3 project:** finish the Returns Forecast (Steps 3 to 6): planning questions with `pmf`, `sf` and `ppf`, a simulated year, a week, a confidence interval for the rate.
- **Lesson 4.3 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz, and the journal prompt. Read "Common Mistakes" in full: all six are silent.
- **Kit labs:** finish `price_satisfaction` (TODO 4 to 6) and `lines_per_order_ci` (Steps 5 and 6).
- **Workbook:** the 12 self-check items and anything left in Predict, then run.
- **Before Lesson 4.4** (from the lesson page): make a scatter plot of `visitors` against `revenue` from `daily_sales.csv` and describe the pattern in one sentence. Then predict: do stores take more revenue on days when more staff are on duty? Why might that be true even if adding staff did nothing?

BRING LIST FOR SESSION 12

- Their notes on the pattern **mean plus or minus t x s / sqrt(n)**: the t-test in Lesson 4.5 is built on the same standard error.
- The scatter plot of visitors against revenue and their written prediction about staffing: Session 12 opens with it.
- Data files for Session 12: `daily_sales.csv`, `store_weather.csv`, `products.csv`, `order_items.csv`, `survey.csv`, `orders.csv`, `customers.csv` (all already in `data/`).
- Installs: none new. Session 12 uses `scipy.stats` and, for two cells, `statsmodels`; both are in the course environment. Anyone unsure runs `python -c "import statsmodels; print(statsmodels.__version__)"` at home.
- Servers to start: none. Accounts: none needed.
- This workbook, with the Session 11 self-check done.

IF YOU RUN LONG OR SHORT

Protect: the two outlier rules disagreeing and the bulk-order investigation (4.2 demo steps 5 to 7), the survey table with `n` beside `rows` (step 11), the `catalog_weights` lab, freezing a normal and asking it questions (4.3 demo steps 3 and 4), the central limit theorem table (step 9), one confidence interval with its coverage check (steps 10 and 11), and the first step of each lesson project.

Running long, cut in this order: (1) 4.2 demo step 4 (standardizing: one sentence) and step 8 (`describe()`: point at the lesson's table); (2) `price_satisfaction` in the room (all of it goes home; show the solution's table for one minute instead); (3) 4.3 demo step 6 (the simulation check) and step 8 (overdispersion: say it, show the two numbers from the lesson); (4) the binomial step shortened to the frozen object and `sf(4)`; (5) `lines_per_order_ci` in the room. Never cut the central limit theorem or the coverage check: Session 12 stands on them.

Running short: do the dice version of the central limit theorem physically, run a debug handout as a whole-room hunt, do the Predict items together, then let learners continue the Store Profile Sheet to Step 3 (the flagged days) and ask them to guess what the flagged days are before Step 4.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 12: Correlation and Hypothesis Tests

How to teach the session that turns description into claims. Before the break learners measure how two columns move together, choose between Pearson and Spearman, and take apart a correlation of 0.88 between staffing and revenue that points to exactly the wrong decision. After the break they ask whether a difference could be chance: they build a test by shuffling, read a p-value correctly, and report every test with an effect size and a confidence interval. Lesson 4.4 contains no hypothesis tests; they begin after the break.

`2 lessons` `3 hrs session` `Lessons 4.4–4.5` `Session 12 of 21`

HOW TO USE THIS GUIDE

- Same rhythm as every session: **Frame** → **Live demo** (you type, they type along) → **Guided lab** (the kit's lab files) → **Start the lesson project**. The lesson's exercises, quiz and journal are homework.
- **Type, do not paste.** Every code block in the demo scripts is what to type, one notebook cell per step, in the course project folder. Every output shown was produced by running that code with the course environment (pandas 3.0, SciPy 1.18, statsmodels 0.15).
- **Keep the halves apart.** Lesson 4.4 describes association and never tests it: if a learner asks "is 0.46 significant?" before the break, write the question on the board and answer it after. In 4.5, never let a p-value appear without an effect size and an interval, in your own speech included.
- **Words matter more than code today.** Write the definition of the p-value on the board before class and point at it every time someone reads a p-value out loud.
- **The lessons are the reference.** Keep `statistics_correlation_analysis.html` and `statistics_hypothesis_testing.html` open for the figures (the four scatter panels, the heatmap, the staffing diagram, the pooled and like-with-like scatters, the shuffle histogram, the power curve) and the tables (choosing Pearson or Spearman, the five explanations, what a p-value is not, the test chooser).
- Teal = hands-on, amber = watch-outs. Labs for this session live in `labs/module-12/`: starters for learners, solutions for you.

> **THE RISK OF SESSION 12: CONFIDENT MISREADINGS**
>
> Nothing in today's code is hard. The danger is in the sentences: "r = 0.88, so more staff sell more", "p = 0.03, so there is a 3% chance the stores are the same", "p = 0.57, so they are the same", "p = 0.003, so it is a big problem". Each one sounds right, and learners will hear all four in their working lives. Spend the session on the sentences. Two board rules carry it: **every correlation gets the question "how did this data come to be?"**, and **every p-value travels with an effect size and a confidence interval**. Never phrase a p-value as the probability that a hypothesis is true, including in passing; learners copy the instructor's wording. Second risk: the session ends Module 4's teaching, and Session 13 opens with a checkpoint. Announce it clearly at the wrap-up so nobody is surprised.

BEFORE SESSION 12

- ☐ Run the lab check from the kit folder: `python3 labs/check_labs.py module-12`. Eight lines should say PASS.
- ☐ Copy the starters and the two debug handouts (`*_starter.py`, `debug_*.py`) from `labs/module-12/` to the shared folder. Keep every `*_solution.py` on your machine.
- ☐ Projector machine: Jupyter (or VS Code notebooks) opened in the course project folder; font size at least 18 pt.
- ☐ Files used today: `data/daily_sales.csv`, `data/store_weather.csv`, `data/products.csv`, `data/order_items.csv`, `data/survey.csv`, `data/orders.csv` (and `data/customers.csv` for the lesson's null-result example). No servers to start, no accounts, nothing new to install: `scipy` and `statsmodels` are in the course environment. Check once: `python -c "import scipy, statsmodels; print(scipy.__version__, statsmodels.__version__)"`.
- ☐ Whiteboard, written before learners arrive: on the left **plot first · direction, form, strength · how did this data come to be?**; on the right the definition **p-value = the probability, if the null hypothesis were true, of a result at least as extreme as the one observed.**
- ☐ Read the Session 13 plan from its author (lessons 4.6 A/B Testing and 4.7 the EDA workflow, with Checkpoint 1) so your wrap-up announcement matches: a short open-notebook checkpoint on Modules 1 to 4.
- ☐ **Known environment traps.** (1) `daily.corr()` on a table with text raises a ValueError: select numeric columns or pass `numeric_only=True`. (2) `np.corrcoef` returns `nan` with missing values; pandas `.corr()` drops missing pairs silently, so print the n. (3) SciPy tests return `nan` with missing values. (4) `stats.ttest_ind` defaults to Student's test; always type `equal_var=False`. (5) Result objects print NumPy wrappers (`np.float64(...)`): format with f-strings. (6) `permutation_test` takes a few seconds; `rng=` is the keyword in current SciPy (older versions used `random_state=`). (7) A `pearsonr` result carries a `.pvalue`; in 4.4 use only `.statistic`.
- ☐ Skim the Session 11 homework: the Store Profile Sheet (all 91 flagged days were weekends, promotions or December) and the Returns Forecast (capacity 4 a day; rate 1.56 with a CI of 1.47 to 1.65). Collect the "before the

next lesson" predictions about staffing: they are the bridge into the staffing puzzle.

- ☐ Sticky notes or slips of paper for the exit ticket.

SESSION 12 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 11 (shape, outliers, distributions, the CLT), homework check.
0:10	10	**4.4** Frame and mental model	Two dancers: in step, but who is leading, or is it the music? Plot first.
0:20	25	**4.4** Live demo	r from z-scores, Spearman, a matrix, the staffing confound tested three ways, a regression line.
0:45	20	**4.4** Guided lab	`labs/module-12/l4_4-correlation-analysis/reviews_and_sales_starter.py`, then Steps 1 and 2 of `staffing_rule_starter.py`.
1:05	5	**4.4** Start the lesson project	The Rain Check, Step 1: the merge keeps 2,903 rows.
1:10	10	Break	Leave the staffing verdict table (0.88, 0.14, 0.07, 0.03) on the projector.
1:20	15	**4.5** Frame and mental model	The courtroom: assume nothing is going on, ask how surprising the data would be.
1:35	35	**4.5** Live demo	A permutation test by hand, the p-value, Welch with CI and effect size, paired, chi-square, Mann-Whitney, false alarms.
2:10	25	**4.5** Guided lab	`labs/module-12/l4_5-hypothesis-testing/returned_order_size_starter.py`, then Step 1 of `footwear_returns_starter.py`.
2:35	15	**4.5** Start the lesson project	Does Rain Keep Shoppers Away? Steps 1 and 2: hypotheses written before any test.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, the Session 13 checkpoint, bring list.
Total	**180**		Ends at 3:00.

The lesson pages estimate 95 + 110 = 205 minutes for a solo learner, the longest pair in the course, and the room has 150 after the warm-up, break and wrap-up. So this session protects the two ideas everything after it depends on (the confound, and the correct reading of a test) and sends the breadth home. **Done in the room:** the scatter-first habit, r

from z-scores, Spearman, a correlation matrix, the staffing confound tested three ways, a first regression line, a permutation test built by hand, the p-value and what it is not, Welch with its interval and Cohen's d, significance against importance, one paired test, one chi-square test, one Mann-Whitney test, false alarms by simulation, the `reviews_and_sales` and `returned_order_size` labs, and the first steps of both projects. **Goes home:** the rest of both projects (Rain Check Steps 2 to 6; Does Rain Keep Shoppers Away? Steps 3 to 7), the unfinished labs, the exercises, Quick Quizzes and journals, and the reading-only parts: the heatmap code, Simpson's paradox (a simulated example), the regression cautions in full, `permutation_test`, the null chi-square result on loyalty tiers, the power curve and `solve_power`, the 30-test scan, and the assumption checks.

SESSION AT A GLANCE

LESSON	TOPIC	ROOM TIME	LEARNERS LEAVE ABLE TO…
4.4	Correlation Analysis	60 of 95 min	plot before measuring, compute and choose between Pearson and Spearman, read a correlation matrix, test a suspected confounder by comparing like with like, and read a regression line in the units of the data
4.5	Hypothesis Testing	90 of 110 min	explain what a p-value is and is not, build a permutation test, choose and run Welch, one-sample, paired, chi-square and Mann-Whitney tests, and report every test with an effect size, a confidence interval and its caveats

WARM-UP

0:00 Warm-up: Retrieval from Session 11

10 min

TIMING

6m	Five questions, no notes: answer alone on paper, compare with a neighbor, then one answer per question out loud
3m	Homework check: the Store Profile Sheet, the Returns Forecast, the staffing prediction
1m	Bridge to today

FIVE RETRIEVAL QUESTIONS (NO NOTES, PAIRS, 60 SECONDS EACH)

1. A column has a skewness of 1.5. Which center and spread do you report, and what do you draw first? **Answer:** Median and IQR (strongly skewed, long right tail), and a histogram first: skewness does not show humps.
2. An outlier rule flags 4% of your rows. Which two questions do you ask before removing anything? **Answer:** Is it a provable error? Is it part of the question I am answering? Remove only provable errors, and say what you removed.
3. You have a frozen distribution `model`. Which method gives P(X at most x), and which gives P(X more than x)? **Answer:** `model.cdf(x)` and `model.sf(x)`.
4. In one sentence: what does the central limit theorem say becomes normal? **Answer:** The distribution of the sample mean (not the data): centered on the population mean, with SD sigma / sqrt(n).
5. A 95% confidence interval for the mean order total is $165 to $209. What does the 95% describe? **Answer:** The method: about 95 of 100 intervals built this way contain the true mean. It is not a range for 95% of orders.

HOMEWORK CHECK

Show of hands: who found out what the 91 "weird days" in the Store Profile Sheet were? (All weekends, promotions or December days: kept.) Who has a capacity number from the Returns Forecast? (4 a day, exceeded on about 2% of days.) Then the bridge: "You were asked to predict whether stores take more revenue on days with more staff, and why that might be true even if adding staff did nothing. What did you write?" Collect two or three answers; somebody will say "busy days get more staff". Write it on the board and leave it: the 4.4 demo proves them right.

BRIDGE TO TODAY

Say: "Until now we described one column at a time. Today, two columns, and two of the most useful and most misused ideas in statistics. Before the break: do two things move together, and does that mean what it

seems? After the break: could a difference be nothing but chance?"

4.4 Correlation Analysis

60 min

TIMING

10m	Frame: two dancers; plot first; direction, form, strength
25m	Live demo: 11 short steps, ending with the staffing verdict and a regression line
20m	Guided lab: `reviews_and_sales`, then the start of `staffing_rule`
5m	Start the lesson project: the Rain Check, Step 1 (the merge)

OBJECTIVES (THE LESSON'S OWN)

- Read direction, form and strength from a scatter plot, and explain why the plot comes before the number
- Compute Pearson and Spearman correlations with pandas and scipy, and choose between them
- Build a correlation matrix and draw it as an annotated heatmap with Matplotlib
- Identify a confounding variable and test for it by comparing like with like
- Fit and interpret a simple regression line, and state what it does and does not tell you

MENTAL MODEL TO SAY

Say: "Watch two people on a dance floor. If they step forward and back together, they are positively correlated. If one steps forward whenever the other steps back, negatively. If each moves with no regard for the other, uncorrelated. Correlation measures how well their movements are synchronized. It says nothing about why: one may be leading, or both may be following the same music."

Write **plot first** on the board, and under it the three things to read from any scatter plot: **direction, form, strength**. Then the question that follows every correlation, which is the whole lesson: **how did this data come to be?** Tell the room up front that today's lesson has no p-values; "could this be chance?" is the second half of the session.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `session12_correlation.ipynb`. The numbers are the lesson's own. Two steps are said, not typed (the list of explanations, and the heatmap, which the lab practices). If time runs short, cut step 4 to the printed matrix and step 6 to one sentence. Do not cut steps 5, 7 and 8: the staffing verdict is the lesson.

1. **Plot first (3m).** New notebook. Say: "Every lesson so far looked at one column. Most real questions involve two. The first move with two numeric columns is always the same: a scatter plot." Read three things from it out loud: direction, form, strength.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
sales = daily.dropna(subset=["revenue"]).copy()
products = pd.read_csv("data/products.csv")
items = pd.read_csv("data/order_items.csv")
units_sold = items.groupby("sku")["quantity"].sum().rename("units_sold")
catalog = products.merge(units_sold, on="sku")   # products that sold
print(sales.shape, catalog.shape)

fig, ax = plt.subplots(figsize=(7, 4), layout="constrained")
ax.scatter(sales["visitors"], sales["revenue"], s=6, alpha=0.3)
ax.set_xlabel("Visitors")
ax.set_ylabel("Revenue ($)")
fig.savefig("visitors_revenue.png", dpi=110)
```

```
(2897, 8) (234, 14)
```

A rising, straight, tight band. Then show the lesson's four-panel figure (visitors and revenue; price and units; price and rating; a perfect curve scored as 0) and say: "Two of these four are warnings. The number measures one specific thing, and you need the picture to know whether that is what your data is doing."

2 **What r is made of (3m).** Say: "You already have the ingredients: z-scores. Multiply them row by row and average." Have the room predict whether the four routes agree.

```
x, y = sales["visitors"], sales["revenue"]
zx = (x - x.mean()) / x.std()
zy = (y - y.mean()) / y.std()
print("by hand        :", round((zx * zy).sum() / (len(x) - 1), 4))
print("pandas .corr() :", round(x.corr(y), 4))
print("np.corrcoef    :", round(np.corrcoef(x, y)[0, 1], 4))
print("scipy pearsonr :", round(stats.pearsonr(x, y).statistic, 4))
```

```
by hand        : 0.9435
pandas .corr() : 0.9435
np.corrcoef    : 0.9435
scipy pearsonr : 0.9435
```

Three properties follow: no units (try revenue in cents), symmetric, and not robust. Then the warning for the board: **r is not a slope and not a percentage.** 0.94 does not mean 94 cents per visitor. (`pearsonr`

also carries a `.pvalue`; say "that is the next lesson's question" and move on.)

3. **Spearman: correlation of ranks (3m).** Say: "Price against units sold. Obviously related, and it scores a feeble -0.32. It is just not straight."

```
price, units = catalog["price"], catalog["units_sold"]
print("Pearson         :", round(price.corr(units), 3))
print("Spearman        :", round(price.corr(units, method="spearman"), 3))
print("Pearson on ranks:", round(price.rank().corr(units.rank()), 3))
print("Pearson of logs :", round(np.log(price).corr(np.log(units)), 3))
```

```
Pearson         : -0.319
Spearman        : -0.6
Pearson on ranks: -0.6
Pearson of logs : -0.628
```

Spearman is Pearson on the ranks: it measures whether one goes consistently up as the other goes down, straight or not. When the two disagree, there is a curve or an influential point. Spearman's other home is ordinal data: overall satisfaction and the recommend score give 0.63 in the lesson. Habit for the board: **compute both.**

4. **Many pairs at once (2m).** Say: "With more than two columns, compute every pair."

```
columns = ["visitors", "transactions", "units", "revenue",
           "staff_on_duty", "promo_day"]
corr = sales[columns].corr().round(2)
print(corr.rename(columns=lambda name: name[:6]))
```

```
              visito  transa  units  revenu  staff_  promo_
visitors        1.00    0.97   0.96    0.94    0.93    0.32
transactions    0.97    1.00   0.99    0.98    0.90    0.37
units           0.96    0.99   1.00    0.99    0.89    0.36
revenue         0.94    0.98   0.99    1.00    0.88    0.37
staff_on_duty   0.93    0.90   0.89    0.88    1.00    0.32
promo_day       0.32    0.37   0.36    0.37    0.32    1.00
```

Read it: diagonal of 1s, mirror image, so only one triangle matters. Then the important sentence: "A solid block of very high correlations is not five discoveries. It is one fact seen five ways: some days are busier. Look for the common driver." The lesson draws it as an annotated heatmap (`imshow`, a diverging colormap, scale fixed at -1 to 1); the lab has learners draw one.

5. **The staffing puzzle (3m).** Say: "One cell deserves a closer look: staff and revenue, 0.88. Imagine it arriving on an executive's desk."

```
staff, revenue = sales["staff_on_duty"], sales["revenue"]
line = stats.linregress(staff, revenue)
print("r:", round(staff.corr(revenue), 3))
print("revenue with 4 staff:", round(revenue[staff == 4].mean()))
print("revenue with 8 staff:", round(revenue[staff == 8].mean()))
print("slope:", round(line.slope), "dollars per extra staff member")
```

```
r: 0.882
revenue with 4 staff: 4759
revenue with 8 staff: 9109
slope: 1154 dollars per extra staff member
```

Let the proposal write itself ("schedule more staff, sell more"), then ask the question that follows every correlation: **how did this data come to be?** The data dictionary: staff are "scheduled in advance". Draw the lesson's diagram: expected demand drives both staffing and visitors. A third variable that drives both is a **confounder**.

6 **The calendar moves both (2m).** Say: "Check the story before you believe it."

```
sales["weekday"] = sales["date"].dt.day_name()
by_day = sales.groupby("weekday")[["staff_on_duty", "revenue"]].mean()
print(by_day.loc[["Tuesday", "Saturday"]].round(1))
print(sales.groupby("promo_day")[["staff_on_duty", "revenue"]]
      .mean().round(1))
```

```
           staff_on_duty  revenue
weekday
Tuesday              5.0   5716.1
Saturday             9.6  10932.9
            staff_on_duty  revenue
promo_day
0                     6.3   7211.7
1                    10.8  13950.5
```

Saturdays have the most staff and the most revenue, Tuesdays the least of both; promotion days have 10.8 staff against 6.3. The schedule and the sales both follow the calendar.

7 **Test 1: compare like with like (3m).** Say: "If staff raise revenue, then among days expected to be equally busy, the days with an extra person should still sell more." Explain `transform` as "the group's mean, written on every row".

```
sales["month"] = sales["date"].dt.to_period("M")
alike = sales.groupby(["store", "month", "weekday", "promo_day"])
sales["staff_dev"] = (sales["staff_on_duty"]
                      - alike["staff_on_duty"].transform("mean"))
sales["revenue_dev"] = sales["revenue"] - alike["revenue"].transform("mean")
print("groups:", alike.ngroups)
print("r, all days pooled       :", round(staff.corr(revenue), 3))
print("r, within comparable days:",
      round(sales["staff_dev"].corr(sales["revenue_dev"]), 3))
```

```
groups: 768
r, all days pooled       : 0.882
r, within comparable days: 0.143
```

From 0.88 to 0.14. Among comparable days, more staff than usual tells you very little about revenue.

8 **Tests 2 and 3 (3m).** Say: "Two more angles. Hold visitors constant. And ask whether each visitor spends more on well-staffed days."

```
def residuals(target, predictor):
    fit = stats.linregress(predictor, target)
    return target - (fit.intercept + fit.slope * predictor)

staff_left = residuals(staff, sales["visitors"])
revenue_left = residuals(revenue, sales["visitors"])
print("partial r, controlling for visitors:",
      round(staff_left.corr(revenue_left), 3))
print("r, staff and spend per visitor     :",
      round(staff.corr(revenue / sales["visitors"]), 3))
```

```
partial r, controlling for visitors: 0.068
r, staff and spend per visitor     : 0.032
```

Put the verdict table on the board: 0.88 pooled, 0.14 like with like, 0.07 holding visitors constant, 0.03 with spend per visitor. Three checks agree. Then the honest limit: this does not prove staffing has **no** effect (no day in the data is badly understaffed). Observational data can rule a simple story out; to rule one in you need an experiment, which is Lesson 4.6.

9 **Correlation is not causation, in a list (1m).** Write the lesson's five explanations for any correlation on the board without code: A causes B; B causes A; a confounder causes both; selection; chance. Ask which ones apply to staffing. Reverse arrow (expected revenue causes the schedule) and confounder (the calendar). Chance is ruled out by 2,897 days; selection by how the file was built.

10 **A first regression line (3m).** Say: "Correlation tells you how tightly points follow a line. Regression tells you which line."

```
fit = stats.linregress(sales["visitors"], sales["revenue"])
print("slope    :", round(fit.slope, 2))
print("intercept:", round(fit.intercept, 2))
print("r squared:", round(fit.rvalue ** 2, 3))

predicted = fit.intercept + fit.slope * sales["visitors"]
leftover = sales["revenue"] - predicted
print("predicted for 400 visitors:", round(fit.intercept + fit.slope * 400))
print("typical miss:", round(leftover.std()))
```

```
slope    : 21.19
intercept: -107.93
r squared: 0.89
predicted for 400 visitors: 8367
typical miss: 1081
```

Read every number in units: about $21 more revenue per extra visitor, on average; the intercept only positions the line (no day is near zero visitors); visitors account for 89% of the variance. Three cautions: do not extrapolate beyond 125 to 1,164 visitors; the line from revenue to visitors is a different line; **a slope is not a causal effect** (the $1,154 per staff member was a correct slope and almost entirely confound).

11 **Two mistakes on purpose (2m).** Say: "Two that give a clean-looking answer."

```
print(np.corrcoef(daily["visitors"], daily["revenue"])[0, 1])

u = pd.Series(np.arange(-5, 6))
print(u.corr(u ** 2))
```

```
nan
0.0
```

First: six missing revenues and NumPy returns `nan`; pandas drops missing pairs. Second: a perfect curve scores 0. r near zero means **no linear** relationship; only the scatter plot can say there is none.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading r as a slope or a percentage.** "r = 0.94, so 94 cents a visitor" or "r = 0.5, half related". Coach: "r measures how tightly the points hug a line, not how steep it is. For the slope, fit the line. For 'how much does it account for', square r." Have them print `r ** 2` for 0.3, 0.5, 0.7, 0.9.

- **Reading a correlation as a cause.** The staffing proposal is the example everyone will make in real life. Coach with the five explanations on the board and one question: "Was either variable decided by someone using information related to the other?" Then: "Does it survive comparing like with like?"
- **Trusting r without the picture.** `r = 0` for a curve, `nan` from `np.corrcoef` with a missing value, or a `ValueError` from `daily.corr()` on a table with text. Coach: "Show me the scatter plot." For the errors: use pandas `.corr()` (which drops missing pairs and says nothing, so print the n), and select numeric columns or pass `numeric_only=True`.

GUIDED LAB

First 12 minutes, everyone: `labs/module-12/l4_4-correlation-analysis/reviews_and_sales_starter.py` (reference: `labs/module-12/l4_4-correlation-analysis/reviews_and_sales_solution.py`). Marketing claims "products with more reviews sell more". Learners plot first (the starter asks for one sentence per panel before any number), then Pearson and Spearman, a Spearman matrix and heatmap saved to `OUTPUT`, and a partial correlation holding price constant. Expected: reviews and units 0.456 and 0.442; rating and units about 0; price -0.319 and -0.600; partial 0.424. Debrief with the direction question: "Which way could the arrow run?" (Sales produce reviews.)

Last 8 minutes: `labs/module-12/l4_4-correlation-analysis/staffing_rule_starter.py`, Steps 1 and 2: a regression line that recovers the managers' scheduling rule (1.59 staff per 100 visitors, r 0.925) and its typical miss. Steps 3 to 5 go home; Step 4's prediction (is the reversed line just 1 / slope?) is a good opener for Session 13.

Early finishers: `labs/module-12/l4_4-correlation-analysis/debug_correlation_claims.py`: four claims about r, all wrong.

START THE LESSON PROJECT

The lesson's project is **The Rain Check** (about 40 minutes alone, in `lesson_4_4_project`): does rain keep shoppers away, once you compare like with like? In the room do **Step 1** only: merge the weather into the sales with `left_on=["date", "store"], right_on=["date", "city"], how="left"` and check the row count: 2,903 before, 2,903 after, 0 missing weather. Tell them where it goes: the pooled correlation looks weak (about -0.19), and comparing like with like doubles it. The same merged table starts the 4.5 project after the break, so everyone should leave this step with it working. Steps 2 to 6 are homework.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold call, then pair-share):

1. "Pearson says -0.32 and Spearman says -0.60 for the same two columns. What do you suspect, and what do you look at?" **Answer:** A consistent relationship that is curved (or an influential point). Look at the scatter plot, perhaps on log axes.
2. "r = 0.88 between staff and revenue. Name two explanations other than 'staff cause revenue'." **Answer:** A confounder (expected demand, the calendar, drives both) and the reverse arrow (expected revenue

decides the schedule).

3. "The line says $21 of revenue per extra visitor and the intercept is -$108. Does a store with no visitors lose $108?" **Answer:** No. No day is anywhere near zero visitors; the intercept only positions the line. Do not interpret it, and do not extrapolate.

Workbook self-check, items Q1 to Q6 (do two in the room if time allows; the rest is homework):

Answers: **Q1** B) Products with more reviews tend to have sold more; the arrow may run from sales to reviews, and the data alone cannot say; **Q2** `-1.0`; **Q3** C) Subtract each group's mean (same store, month, weekday and promotion status) from both columns with `groupby().transform("mean")`, then correlate the deviations; **Q4** D) 1-to-5 overall satisfaction and the 0-to-10 recommend score; **Q5** `the line accounts for 25% of the variance`; **Q6** A) A slope describes how revenue differs across days with different staffing, not what changing the staffing would do; here it is mostly the calendar acting on both. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: give learners the five explanations for a correlation on a card and have them sort each example from the demo onto it. For like-with-like, do it first with four rows on the board (two Saturdays, two Tuesdays): subtract each group's mean by hand, then correlate. `transform("mean")` is then just "the group mean written on every row".

Stretch: lesson Exercise 4 (a reusable `partial_corr` function: staff and revenue controlling for visitors, and the reverse), or the lab extension: the reviews-units link within each category (`groupby("category")`). Does it hold everywhere?

4.5 Hypothesis Testing

90 min

TIMING

15m	Frame: the courtroom; the Asheville question; the four steps of every test
35m	Live demo: 12 short steps from a test built by hand to false alarms and power
25m	Guided lab: `returned_order_size`, then Step 1 of `footwear_returns`
15m	Start the lesson project: Does Rain Keep Shoppers Away?, Steps 1 and 2

OBJECTIVES (THE LESSON'S OWN)

- Explain the logic of a hypothesis test in plain words, and say what a p-value is and what it is not
- Build a permutation test by simulation and match it to SciPy's ready-made tests
- Choose and run a one-sample, Welch two-sample or paired t-test, a chi-square test, or a rank-based test for the data in front of you
- Report every test with an effect size and a confidence interval, and check its assumptions on the data
- Describe Type I and Type II errors and power, and correct for multiple comparisons

MENTAL MODEL TO SAY

Say: "A court starts from 'innocent until proven guilty'. It does not ask the defendant to prove innocence. It asks whether the evidence would be surprising if the defendant were innocent. If it would be very surprising, the court rejects innocence. If not, the verdict is 'not guilty', which is not the same as 'proven innocent'. A hypothesis test works the same way. The starting assumption, the null hypothesis, is 'nothing is going on: any difference is chance'. Then you ask how surprising your data would be if that were true."

Write the two verdicts on the board: **reject the null** and **fail to reject the null** (never "accept"). Then say why this lesson matters more than any other in the module: "The p-value is the most misread number in statistics. By the end of today you will read it correctly, and you will never report one alone."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `session12_tests.ipynb`. The numbers are the lesson's own; the two simulations use `default_rng(42)` created inside their cells, so they reproduce on every machine. Step 4 is said, not typed. If time runs short, say step 7 in one sentence and show step 10's numbers from the lesson page; never cut steps 2 to 6 (the test built by hand, the p-value, Welch with its interval and effect size).

1. **The question (3m).** New notebook. Say: "Head office noticed that Asheville shoppers give a lower recommend score. Before anyone schedules a meeting about Asheville: could that gap be luck?"

```
import numpy as np
import pandas as pd
from scipy import stats

survey = pd.read_csv("data/survey.csv")
in_store = survey[survey["store"] != "Online"]
is_ash = in_store["store"] == "Asheville"
asheville = in_store.loc[is_ash, "recommend"].dropna()
others = in_store.loc[~is_ash, "recommend"].dropna()

for name, g in [("Asheville", asheville), ("Other stores", others)]:
    print(f"{name:<12} n = {len(g)}  mean = {g.mean():.2f}"
          f"  sd = {g.std():.2f}")
observed = asheville.mean() - others.mean()
print(f"Observed difference: {observed:.3f}")
```

```
Asheville    n = 197  mean = 6.98  sd = 2.05
Other stores n = 752  mean = 7.34  sd = 2.04
Observed difference: -0.357
```

About a third of a point on an eleven-point scale, with answers spread about 2 points either side. Is 0.36 a lot? Looking will not tell you. You need to know how big a gap chance alone produces with groups of this size.

2 **Build chance yourself: shuffle the labels (4m).** Say: "If the store makes no difference, the 'Asheville' label is a sticker that could have landed on anyone. Peel off all 949 stickers, shuffle, stick them back, measure the gap. Ten thousand times."

```
rng = np.random.default_rng(42)
pooled = np.concatenate([asheville, others])
n_ash = len(asheville)
shuffled_diffs = np.empty(10_000)
for i in range(10_000):
    shuffled = rng.permutation(pooled)
    shuffled_diffs[i] = shuffled[:n_ash].mean() - shuffled[n_ash:].mean()

print(shuffled_diffs[:5].round(3))
print(f"typical chance gap (SD): {shuffled_diffs.std():.3f}")
```

```
[-0.03   0.034 -0.216  0.277  0.124]
typical chance gap (SD): 0.165
```

Each number is a gap between two groups that cannot truly differ. Their spread, about 0.165, is a standard error measured by simulation: Session 11's idea, for a difference. The real gap is a little more

than two of those from zero. Show the lesson's histogram with the real gap marked in the tail.

3. **Count: the p-value (3m).** Say: "How often did chance produce a gap at least this large, in either direction?"

```
as_extreme = np.abs(shuffled_diffs) >= abs(observed)
print(as_extreme.sum(), "of 10,000 shuffles gave a gap at least this large")
print(f"p-value = {as_extreme.mean():.3f}")
```

```
300 of 10,000 shuffles gave a gap at least this large
p-value = 0.030
```

Write the definition on the board and leave it there all session: **the p-value is the probability, if the null hypothesis were true, of a result at least as extreme as the one observed.** Then the four steps of every test: state what "nothing going on" means; work out what that produces; see where the real result falls; report how rare it is. Vocabulary: H0, H1, alpha = 0.05 chosen before looking, "reject" or "fail to reject" (never "accept").

4. **What a p-value is not (2m).** Read the lesson's table aloud, no code. Have learners say "no" out loud to each. "There is a 3% chance the null is true": no, p is computed **assuming** the null. "97% chance Asheville is worse": no, same mistake turned around. "The gap is large": no, p mixes effect size with sample size. "p = 0.20 means the stores are the same": no, not detected is not absent. And 0.049 against 0.051 is the same evidence: report the number.

5. **The shortcut: Welch's t-test (3m).** Say: "For a difference in means there is a classic shortcut: the difference divided by its standard error, read off the t distribution. Welch's version does not assume equal variances. SciPy's default does, so type `equal_var=False` every time."

```
res = stats.ttest_ind(asheville, others, equal_var=False)
ci = res.confidence_interval(confidence_level=0.95)
print(f"t = {res.statistic:.2f}, df = {res.df:.0f}, p = {res.pvalue:.3f}")
print(f"95% CI for the difference: {ci.low:.2f} to {ci.high:.2f}")
```

```
t = -2.17, df = 305, p = 0.030
95% CI for the difference: -0.68 to -0.03
```

The same p as the shuffle: one by simulation, one by theory. The interval is the more useful half: Asheville is between 0.68 and 0.03 points lower. It excludes 0, which is the same fact as p below 0.05, and it shows what p cannot: the true gap might be meaningful or almost nothing.

6. **How big? Cohen's d (2m).** Say: "A p-value never travels alone. It comes with an effect size and an interval."

```
def cohens_d(x, y):
    nx, ny = len(x), len(y)
    pooled = (((nx - 1) * np.var(x, ddof=1) + (ny - 1) * np.var(y, ddof=1))
              / (nx + ny - 2))
    return (np.mean(x) - np.mean(y)) / np.sqrt(pooled)

print(f"Cohen's d = {cohens_d(asheville, others):.2f}")
```

```
Cohen's d = -0.17
```

Near 0.2 small, 0.5 medium, 0.8 large: this is small. Say the honest sentence: "Asheville shoppers rated Pinecrest 0.36 points lower on average (95% CI 0.03 to 0.68 lower; Welch p = 0.030; d = -0.17, small)." Evidence worth following up, nowhere near a crisis.

7. **Significant is not the same as important (2m).** Say: "Now the opposite case. Target score 7.5, all 1,746 answers."

```
everyone = survey["recommend"].dropna()
res1 = stats.ttest_1samp(everyone, popmean=7.5)
ci1 = res1.confidence_interval()
print(f"n = {len(everyone)}, mean = {everyone.mean():.2f}")
print(f"p = {res1.pvalue:.4f}")
print(f"95% CI {ci1.low:.2f} to {ci1.high:.2f}, "
      f"d = {(everyone.mean() - 7.5) / everyone.std():.2f}")
```

```
n = 1746, mean = 7.36
p = 0.0028
95% CI 7.27 to 7.45, d = -0.07
```

A smaller p than Asheville's and a tiny shortfall (d = -0.07). With 1,746 answers even a small miss is detectable. Write on the board: **statistically significant = unlikely to be chance alone; practically significant = big enough to matter.** The first is a calculation, the second a judgment.

8. **Pairs: the same days, two stores (3m).** Say: "Bend against Flagstaff, visitors, on the same dates. Watch the verdict change with the test."

```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
visitors = daily.pivot(index="date", columns="store", values="visitors")
pair = visitors[["Bend", "Flagstaff"]].dropna()
wrong = stats.ttest_ind(pair["Bend"], pair["Flagstaff"], equal_var=False)
right = stats.ttest_rel(pair["Bend"], pair["Flagstaff"])
print(len(pair), "days; mean difference",
      round((pair["Bend"] - pair["Flagstaff"]).mean(), 1))
print(f"as independent groups: p = {wrong.pvalue:.3f}")
print(f"as pairs             : p = {right.pvalue:.4f}")
```

```
722 days; mean difference 6.5
as independent groups: p = 0.274
as pairs             : p = 0.0024
```

Traffic swings from a February Tuesday to an August Saturday in both stores together. The independent test sees all that swing as noise; pairing by date subtracts it out. Real and small: about 6.5 visitors a day. Caveat to say once: consecutive days are not fully independent, so daily-data p-values are approximate.

9 **Categories: chi-square (3m).** Say: "Means are for numbers. When both variables are categories, compare counts."

```
orders = pd.read_csv("data/orders.csv")
orders["returned"] = orders["status"] == "returned"
table = pd.crosstab(orders["channel"], orders["returned"])
chi = stats.chi2_contingency(table)
print(table)
print(f"chi-square = {chi.statistic:.1f}, df = {chi.dof}")
print(f"p = {chi.pvalue:.1e}")
print("smallest expected count:", round(chi.expected_freq.min(), 1))
rates = orders.groupby("channel")["returned"].mean()
print(f"online {rates['online']:.1%}, store {rates['store']:.1%}")
```

```
returned  False  True
channel
online    11964    890
store      7195    253
chi-square = 109.7, df = 1
p = 1.1e-25
smallest expected count: 419.3
online 6.9%, store 3.4%
```

The null: channel and returning are unrelated. The expected counts are the assumption check (about 5 or more in every cell; the smallest is 419). The effect: online orders come back twice as often (6.9%

against 3.4%). Cramér's V is only 0.07 here, so for a 2 x 2 table report the rates and their ratio: that is what a reader can act on.

10 **Ratings: Mann-Whitney (3m).** Say: "A t-test compares means, and a mean assumes equal steps between 1, 2, 3, 4 and 5. For ratings, use ranks."

```
staff_ash = in_store.loc[is_ash, "sat_staff"].dropna()
staff_oth = in_store.loc[~is_ash, "sat_staff"].dropna()
mw = stats.mannwhitneyu(staff_ash, staff_oth)
print(f"n = {len(staff_ash)} and {len(staff_oth)}, p = {mw.pvalue:.1e}")
print("probability of superiority:",
      round(mw.statistic / (len(staff_ash) * len(staff_oth)), 2))
```

```
n = 193 and 754, p = 3.0e-09
probability of superiority: 0.37
```

This is the gap learners described in Session 11 (57.5% satisfied against 75 to 79%). Now it has a test: p about 3 in a billion. The effect size: pick one Asheville shopper and one other-store shopper at random; the Asheville rating is the higher one 37% of the time (50% would be no difference).

11 **False alarms, watched (3m).** Say: "A test can be wrong two ways. Here is the first one, happening." Draw two groups from the same pool, so the null is true by construction.

```
rng = np.random.default_rng(42)
pool = in_store["recommend"].dropna().to_numpy()
p_null = np.array([
    stats.ttest_ind(rng.choice(pool, 197), rng.choice(pool, 752),
                    equal_var=False).pvalue
    for _ in range(2_000)])
share = (p_null < 0.05).mean()
print(f"{share:.1%} of 2,000 tests of a true null were significant")
```

```
5.3% of 2,000 tests of a true null were significant
```

About 5%, as alpha promises: one test in twenty rings the alarm when nothing is going on (a **Type I error**). The other way to be wrong is a miss (**Type II**); the chance of catching a real effect is **power**, and it depends on sample size: with 50 per group a small real effect (d = 0.2) is caught only 17% of the time. Then multiple comparisons in one line: 30 tests of true nulls give a 79% chance of at least one false alarm. The lesson's scan of 6 age groups x 5 items finds 6 "significant" results, 3 after Holm's correction. Count every test you ran.

12 **A mistake on purpose (1m).** Say: "SciPy does not skip missing values."

```
raw = in_store.loc[in_store["store"] == "Asheville", "recommend"]
print(stats.ttest_ind(raw, others, equal_var=False).pvalue)
print(round(stats.ttest_ind(raw, others, equal_var=False,
                            nan_policy="omit").pvalue, 3))
```

```
nan
0.03
```

`nan` with no error. Fix: `.dropna()` first, or `nan_policy="omit"`. Close with the reporting template from the lesson, on the board: **what was compared (and n) · the estimate · its 95% CI · the test and p · the effect size in words · caveats (how the data were collected, how many tests).**

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading the p-value as the probability that the null is true** ("a 3% chance the stores are the same", "97% sure Asheville is worse"). Coach with one question every time: "What was assumed when p was computed?" Then have them rewrite the sentence starting with "If there were no difference, ...". The board definition stays up all session.
- **"Not significant" read as "no difference", and "significant" read as "big".** Coach both with the effect size and the interval: "What does the confidence interval allow?" For a small sample show the lesson's pilot (10 answers, p = 0.82 for a real gap). For a huge sample show the target test (p = 0.003, d = -0.07). Two separate questions: is it chance? does it matter?
- **Silent mechanical errors.** `nan` from a missing value (SciPy does not skip: `.dropna()` or `nan_policy="omit"`); `stats.ttest_ind(x, y)` without `equal_var=False` (Student's test, no warning); `chi2_contingency` on raw columns instead of a `crosstab` table; a paired test on two columns dropped separately (misaligned days); `alternative="less"` chosen after looking. Coach: "Read your call out loud and say what each argument means."

GUIDED LAB

First 15 minutes, everyone: `labs/module-12/l4_5-hypothesis-testing/returned_order_size_starter.py` (reference: `labs/module-12/l4_5-hypothesis-testing/returned_order_size_solution.py`). Customer service believes "people send back the big orders". The starter has a TODO 0: write H0, H1 and alpha before anything runs; check that every learner did. Expected: difference $25.54 (95% CI $15.29 to $35.79), p = 1.1e-06, d = 0.14, permutation p 0.0002; online only $23.85. The debrief is the point of the lab: **real, and small**. Ask a volunteer to read their report and the room to find any sentence that misreads p.

Last 10 minutes: `labs/module-12/l4_5-hypothesis-testing/footwear_returns_starter.py`, Step 1: the online table, the rates (14.4% against 4.8%) and the chi-square test with its expected counts. Steps 2 to 4 (stores, where nothing is detected; Holm; the report) go home.

Early finishers: `labs/module-12/l4_5-hypothesis-testing/debug_test_report.py`: five conclusions, five misreadings.

START THE LESSON PROJECT

The lesson's project is **Does Rain Keep Shoppers Away?** (about 40 minutes alone, in `lesson_4_5_project`). In the room do **Steps 1 and 2**: the same merge as the Rain Check plus `days["rainy"] = days["precip_in"] >= 0.10` (rainy-day shares: Asheville 0.24, Bend 0.07, Boise 0.09, Flagstaff 0.14), then the table of visitors by store and rain. Before anyone runs a test, everyone writes the hypotheses and alpha as a comment, exactly as the lesson shows; walk the room and read them. Steps 3 to 7 (one store completely, a permutation check, all four stores with Holm, the seasonal challenge, the written finding) are homework.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold call, then pair-share):

1. "Finish the sentence: a p-value of 0.03 means that, if ..." **Answer:** "... there were no real difference, a result at least this extreme would turn up in about 3% of samples." Not the probability that the null is true.
2. "Bend and Flagstaff visitor counts on the same 722 dates. Which test, and why?" **Answer:** A paired t-test (`ttest_rel`): the days pair one to one, and pairing removes the shared swing of the calendar.
3. "You ran 30 tests and 6 came out below 0.05. What do you do before reporting any of them?" **Answer:** Correct for multiple comparisons (Holm), report how many tests were run, and label scanned findings as exploratory.

Workbook self-check, items Q7 to Q12 (do two in the room if time allows; the rest is homework):

Answers: **Q7** A) In a world where the store makes no difference, about 3% of shuffles produced a gap at least as large as the real one; **Q8** D) A Mann-Whitney U test; **Q9** `nan`; **Q10** C) It is below 0.05; **Q11** B) We could not detect a difference with this sample; it is too small to say much; **Q12** `23%`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: keep the four steps of a test and the definition of the p-value on the board, and have learners say the four steps for each test in the demo. Give anyone overwhelmed by the menu of tests the lesson's decision chart on paper (category outcome? rating? one group against a target? the same units twice? otherwise Welch). Reassure the room that the permutation test is always available: if a formula feels like magic, shuffle.

Stretch: lesson Exercise 5 (simulate the power of a Mann-Whitney test at 25, 50, 100 and 200 answers per group), or extend Lab 4.5-B: in the store result, how large a difference could the data have missed? (A confidence interval for a difference in proportions, or a power calculation.)

CLOSING

EXIT TICKET (3 QUESTIONS, ON PAPER OR STICKY NOTES, NO NOTES)

1. Staff on duty and revenue correlate at r = 0.88. Give two explanations other than "more staff cause more revenue". **Answer:** A confounder (expected demand, the calendar, drives both) and the reverse arrow (expected revenue decides the schedule). Selection and chance also count as explanations in general.
2. Complete the sentence: "A p-value is the probability of _____, assuming _____." **Answer:** "... a result at least as extreme as the one observed, assuming the null hypothesis is true." It is not the probability that the null is true.
3. Besides the p-value, which two numbers belong in every test report? **Answer:** An effect size (how big?) and a confidence interval (how sure, and how big it could be).

Read the slips before Session 13. Question 2 is the one to watch: any answer that turns the p-value into the probability of a hypothesis means the learner needs the "What a p-value Is Not" table again before the A/B-testing lesson, which leans on it. Session 13's author can open with it if several miss it; pass on the count.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Lesson 4.4 project:** finish the Rain Check (Steps 2 to 6): look, measure two ways, compare like with like, the effect in visitors, the finding with its limits.
- **Lesson 4.4 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz, and the journal prompt.
- **Lesson 4.5 project:** finish Does Rain Keep Shoppers Away? (Steps 3 to 7), ending with a written finding in the reporting template.
- **Lesson 4.5 practice:** Exercises 1 to 4 (Exercise 5 is a stretch), the Quick Quiz, and the journal prompt. Reread the "What a p-value Is Not" table tomorrow.
- **Kit labs:** finish `staffing_rule` (Steps 3 to 5) and `footwear_returns` (Steps 2 to 4).
- **Workbook:** the 12 self-check items and anything left in Predict, then run.
- **Before Lesson 4.6** (from the lesson page): open `data/ab_test.csv` and look at its columns. Before running anything, write down which test from Lesson 4.5 you would use to compare conversion between the two variants, and why.
- **Prepare for the Session 13 checkpoint:** a short open-notebook task on Modules 1 to 4 (NumPy, pandas, Matplotlib, statistics). Tidy your lesson notebooks so you can find things, and skim the key ideas pages of this workbook from Session 1 on. No memorizing needed: it is open-notebook.

BRING LIST FOR SESSION 13

- **Session 13 includes Checkpoint 1: a short open-notebook analysis task on Modules 1 to 4** (about 30 minutes inside the session). Learners bring their lesson notebooks from Modules 1 to 4 and this workbook; they may use them, the lesson pages and the documentation. Not graded on speed.
- Their `cohens_d` function and the reporting template (Lesson 4.6 uses both), and their note on which test they would use for `ab_test.csv`.
- Data files: `ab_test.csv` for Lesson 4.6, and all the Pinecrest CSV files in `data/` for Lesson 4.7 and the checkpoint (already in place).
- Installs: none new. Lesson 4.6 uses `statsmodels` power functions, already in the course environment (checked in this session).
- Servers to start: none. Accounts: none needed.
- A charged laptop that can run the course environment for the whole three hours.

IF YOU RUN LONG OR SHORT

Protect: the staffing confound tested like with like (4.4 demo steps 5 to 8), the permutation test and the definition of the p-value (4.5 steps 2 to 4), Welch with its interval and effect size and the significant-is-not-important contrast (steps 5 to 7), the `returned_order_size` lab with its "real and small" debrief, the merge that starts both projects, and the Session 13 checkpoint announcement.

Running long, cut in this order: (1) 4.4 step 4 (the matrix: show the lesson's heatmap instead) and step 11 (the two mistakes: one sentence each); (2) `staffing_rule` in the room; (3) 4.5 step 8 (the paired test: show the two p-values from the lesson and say why they differ); (4) 4.5 step 10 (Mann-Whitney: one sentence and the result, since learners know the gap from Session 11); (5) `footwear_returns` in the room. Never cut the p-value definition, the effect size, or the exit ticket.

Running short: run both debug handouts as whole-room hunts (one sentence per conclusion, the room votes on what is wrong), then let learners continue the 4.5 project to Step 3 (Asheville in full) and read their reports aloud against the template.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 13: A/B Testing and the EDA Workflow

How to teach the session that closes Module 4: one real experiment from the plan written before launch to the ship decision, a seven-step routine for data nobody has explained to you, and a 30-minute checkpoint in which learners run that routine alone.

`2 lessons + checkpoint` `3 hrs session` `Lessons 4.6–4.7` `Session 13 of 21`

HOW TO USE THIS GUIDE

- This is the last session of Module 4 and the fullest of the course: two lessons **and** Checkpoint 1 in 180 minutes. The lesson blocks are 60 and 55 minutes instead of the usual 75 to 80. The guide says exactly what is typed in the room, what is shown from the lesson page, and what goes home.
- Same rhythm as every session, compressed: **Frame** (goal and mental model) → **Live demo** (you type, they follow) → **Guided lab** (new practice from the kit) → **Start the project** (five minutes: the brief and the first cell). Then the checkpoint.
- Type, don't paste, with one stated exception (the 55-test scan in the 4.7 demo). Every code block below was run on the course data; the gray block under it is the real output.
- Labs for this session live in `labs/module-13/`: starters for learners, solutions for you. Teal blocks are hands-on, amber blocks are watch-outs.
- Keep `statistics_ab_testing.html` and `statistics_eda_workflow.html` open in browser tabs for the diagrams and the three charts you show but do not type (the daily p-value chart, the six bar panels, the heatmap of gaps).

> **THE #1 RISK OF SESSION 13: THE CLOCK**
>
> The lesson pages estimate 100 + 110 = 210 minutes for a solo learner. After the warm-up, the break, the checkpoint and the wrap-up the room has 115. The checkpoint is fixed at 2:15: do not let the EDA lab run into it. If you are behind at the break, shorten the 4.7 demo (see "If You Run Long"), never the checkpoint. The second risk is tone: a checkpoint in the last hour makes some learners tense from 0:00. Say in the first minute that it is practice for the capstone, that it is open notebook, and that it is not graded.

BEFORE THE SESSION

- ☐ From the `instructor-kit/` folder run `python3 labs/check_labs.py module-13`. Expect `14 passed, 0 failed` (six lab pairs and the checkpoint pair).

- ☐ Copy the **starter files only** to the shared folder: `labs/module-13/l4_6-ab-testing/*_starter.py`, `labs/module-13/l4_7-eda-workflow/*_starter.py` and `labs/module-13/checkpoint-1/checkpoint1_starter.py`. Keep every `*_solution.py` on your machine; `checkpoint1_solution.py` is the checkpoint's model answer.
- ☐ Files used today, all in `data/`: `ab_test.csv`, `survey.csv`, `customers.csv`, `orders.csv`, `order_items.csv`, `products.csv`, `daily_sales.csv`. No servers, no internet, no accounts.
- ☐ Projector machine: two empty notebooks, `session13_ab.ipynb` and `session13_eda.ipynb`, in the folder that holds `data/`. Font 18 pt or larger. Run the statsmodels import cell once before class: the first import takes several seconds.
- ☐ Board, before learners arrive: the six plan lines (unit, primary metric, guardrail, hypotheses, sample size, decision rule) on the left; the seven EDA steps on the right; between them "A failed health check means STOP" and "Missing for WHOM?".
- ☐ Print or share the Checkpoint 1 task sheet (it is in the workbook) and decide how files are handed in. Have a visible 30-minute timer ready.
- ☐ Read the model answer and the seven weak answers in the Answer Key before class. Five minutes now saves twenty when marking.
- ☐ Skim the Session 12 homework: the Rain Check and "Does Rain Keep Shoppers Away?" projects, and the Lesson 4.5 note that asked which test fits `ab_test.csv`. The warm-up uses it.

KNOWN ENVIRONMENT TRAPS

statsmodels missing or old. `from statsmodels.stats.proportion import confint_proportions_2indep` fails on a very old statsmodels. The course pins 0.15: `python -m pip install -r requirements.txt` inside the course environment fixes it. **Wrong working folder.** `FileNotFoundError: data/ab_test.csv` means the notebook is not next to the `data/` folder; the labs do not have this problem (they find the data through `labs_paths`). **A chart window that blocks.** In a plain terminal `plt.show()` waits for the window to close; in the checkpoint tell terminal users to `savefig` instead. **NaN from SciPy.** `stats.ttest_ind` returns `nan` if either input contains a missing value: today that is a learning moment in the checkpoint, not an environment fault. Do not fix it for them.

SESSION 13 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 12 (correlation, hypothesis tests), no notes. Homework check: the rain projects.
0:10	10	**4.6** Frame and mental model	Why experiments beat before/after comparisons. "The coin does the controlling." The six-line plan.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:20	25	**4.6** Live demo	Size the test, check the randomization, the primary result with its interval, the weekly peeking table, device segments, revenue per visitor.
0:45	20	**4.6** Guided lab	`labs/module-13/l4_6-ab-testing/novelty_check_starter.py`. Fast finishers: `followup_test_plan_starter.py`.
1:05	5	**4.6** Start the lesson project	The Decision Memo: read the brief, everyone runs the Step 1 health-check cell.
1:10	10	Break	Leave the weekly peeking table on the projector.
1:20	10	**4.7** Frame and mental model	The detective at the scene. Seven steps, written on the board. Four questions before any code.
1:30	20	**4.7** Live demo	The survey: `overview()`, missing for whom, who answered, one variable, the 55-test scan with Holm, one competing explanation.
1:50	20	**4.7** Guided lab	`labs/module-13/l4_7-eda-workflow/shipping_rules_starter.py`: work a rule out of the data and write the note.
2:10	5	**4.7** Start the lesson project	Why Do Orders Come Back? Learners write their four questions; the three tables load.
2:15	30	Checkpoint 1	Open-notebook analysis task on Modules 1 to 4, individual. Task sheet in the workbook.
2:45	5	Checkpoint hand-in and debrief	Collect files. Two-minute debrief on one chart from the model answer. No full solution yet.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 14.
Total	180		Ends at 3:00.

In the room: both mental models; the checkout experiment from sample size to ship decision in twelve cells; the survey from questions to a challenged finding in ten cells; one guided lab per lesson (a novelty check on the experiment; working the shipping rule out of the orders); the first cell of each lesson project; Checkpoint 1.

Moves home, exactly: Lesson 4.6: the day-by-day p-value chart and the peeking simulation ("Simulating a Peeker"); the by-hand z-test and its match with chi-square; the device forest plot and the Holm column; the buyers-only order-value comparison (the room does revenue per visitor only); Common Mistakes 1 to 6 as reading; Decision Memo steps 2 to 5; Exercises 1 to 5; the quiz and the journal. **Lesson 4.7**: the six-panel distribution chart and the heatmap of gaps; the recommend-by-satisfaction table; the Asheville revenue follow-up in Step 7 and the

full findings note; Common Mistakes 1 to 6 as reading; the whole returns project after its Step 2 (about 45 minutes); Exercises 1 to 4; the quiz and the journal. **Kit labs**: `followup_test_plan`, `aa_test`, `order_size_scan` and `customer_overview`. That is about three and a half hours, the heaviest homework week of the course: say so, and say which parts come first (the two projects).

Protect: the health checks and the interval in 4.6 (steps 3 to 7), "missing for whom" and the corrected scan in 4.7 (steps 4 and 9), the written note at the end of the shipping lab, and all 30 minutes of the checkpoint. **Cut first:** see "If You Run Long or Short" at the end.

WARM-UP

0:00 Warm-up: Retrieval from Session 12

10 min

TIMING

6m	Five retrieval questions on paper, no notes, then answers by show of hands
3m	Homework check and the bridge to today
1m	Today's roadmap and the checkpoint, said plainly

FIVE RETRIEVAL QUESTIONS (LESSONS 4.4 AND 4.5)

1. Across store-days, staff on duty and revenue correlate at r = 0.88. Give one reason this does not mean "add staff, sell more". **Answer:** Staff are scheduled from expected demand (weekday, season, promotions), and expected demand also drives revenue: a confounder. Comparing like with like (same store, month, weekday, promo status) the correlation falls to 0.14.

2. For the same two columns Pearson's r is -0.32 and Spearman's is -0.61. What does the gap tell you? **Answer:** The relationship is consistent in direction but curved (or one influential point is distorting Pearson). Spearman works on ranks, so it measures any monotonic trend. Plot it.

3. Finish the sentence: "A p-value is the probability of ... assuming ..." **Answer:** ...a result at least as extreme as the one observed, assuming the null hypothesis is true. It is not the probability that the null is true, and it says nothing about the size of the effect.

4. Which test: (a) the mean of two independent groups of different sizes; (b) whether two categorical columns are related? **Answer:** (a) Welch's t-test, `stats.ttest_ind(x, y, equal_var=False)`. (b) Chi-square on a table of counts, `stats.chi2_contingency(pd.crosstab(a, b))`. Hold on to (b): today's first test is the same test in another form.

5. You run 20 tests at alpha = 0.05 and one comes back with p = 0.04. What do you say about it? **Answer:** About one false alarm is expected from 20 tests when nothing is going on. Count every test, correct (Holm), and call a scanned result exploratory.

HOMEWORK CHECK AND BRIDGE

Ask who finished the two rain projects and what number they would give a store manager (rainy days have about 20% fewer visitors: -18% to -26% by store). Then the Lesson 4.5 note: "You were asked to look at `ab_test.csv` and write down which test compares conversion between the two variants. What did you write?" (Chi-square on the two-by-two table, or a two-proportion test: the same test. Today's first half is that test with everything that has to happen around it.)

Roadmap, said plainly: "Two lessons and one checkpoint. Before the break, an experiment from plan to decision. After the break, a routine for data you have never seen. At 2:15, thirty minutes on your own with a small question: open notebook, not graded, practice for the capstone."

WATCH-OUTS

- Question 3 is the one most rooms get wrong ("the probability the null is true"). Do not move on until someone has said "assuming the null is true" aloud. Today's peeking and scanning both depend on it.
- Someone missed Session 12: pair them now. They need p-values, Welch's t-test and the idea of multiple comparisons; point them at Lesson 4.5's "A Template for Reporting Any Test" and do not re-teach.

4.6 A/B Testing

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Write an experiment plan that names the metric, the unit, the sample size and the decision rule before any data are collected
- Calculate the sample size for a conversion test with statsmodels power functions
- Check that randomization worked, then run a two-proportion test and report the difference with its confidence interval
- Explain why peeking inflates false alarms and why segment results are exploratory
- Use a guardrail metric and practical significance to turn a result into a written recommendation

TIMING INSIDE THE BLOCK (60 MINUTES)

10m	Frame and mental model
25m	Live demo, steps 1 to 12
20m	Guided lab: novelty check
5m	Start the Decision Memo project

MENTAL MODEL TO SAY

"Imagine sorting a crowd into two rooms by flipping a coin for each person. Tall people, impatient people, people on phones, people who were going to buy anyway: every kind ends up split about evenly, including kinds you never thought to measure. The rooms are alike in everything except what you do to them next. So if they behave differently afterward, either the treatment did it or chance did, and last session taught you how to judge chance."

Bridge from Session 12: "Last time rain and visitors: we had to work hard to argue that rain itself was responsible, because rain comes with seasons. An experiment removes that weakness by design. It does not make the statistics optional. Most of what goes wrong in A/B testing goes wrong AROUND the test."

Then put the six-line plan on the board and leave it there all session: **Unit** (a visitor who reaches checkout) · **Primary metric** (conversion per visitor) · **Guardrail** (order value, revenue per visitor) · **Hypotheses** (two-sided, alpha 0.05) · **Sample size and duration** (computed, then fixed) · **Decision rule** (ship if conversion is significantly higher and the guardrail holds). Ask: "Why two-sided when everyone hopes the page is better?" (A new page can be worse, and you would need to know.)

LIVE DEMO (YOU TYPE, THEY FOLLOW): THE CHECKOUT EXPERIMENT, PLAN TO DECISION

One notebook, `session13_ab.ipynb`, in the folder that holds `data/`. Twelve short cells. The clock is tight: the marked minutes add up to 25. If you are two minutes behind at step 6, say steps 2 and 9 instead of typing them.

1. **Imports and the sample size (3m).** Say: "Before any data exists, four ingredients: the baseline, the smallest lift worth catching, alpha and power. statsmodels wants the two rates as ONE number, Cohen's h." Type the imports once; they serve the whole demo.

```
import numpy as np
import pandas as pd
from scipy import stats
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import (proportion_effectsize, proportions_z
                                          confint_proportions_2indep)

baseline, target = 0.089, 0.100
h = proportion_effectsize(target, baseline)
n_per_arm = NormalIndPower().solve_power(effect_size=h, alpha=0.05, power=0.80
print(f"Cohen's h = {h:.4f}")
print(f"Visitors needed per arm: {n_per_arm:,.0f}")
```

```
Cohen's h = 0.0376
Visitors needed per arm: 11,092
```

Ask before running: "More or fewer than 2,000 per arm?" Most rooms guess far too low. "Conversion is yes-or-no. Each visitor carries very little information."

2. **Small lifts cost time (1m).** Say: "The checkout page gets about 1,000 visitors a day. Watch the days column."

```
for goal in [0.094, 0.100, 0.109]:
    h_goal = proportion_effectsize(goal, baseline)
    n = NormalIndPower().solve_power(effect_size=h_goal, alpha=0.05, power=0.8
    print(f"detect 8.9% -> {goal:.1%}: {n:>7,.0f} per arm, "
          f"about {2 * n / 1000:>4.0f} days")
```

```
detect 8.9% -> 9.4%:  52,187 per arm, about  104 days
detect 8.9% -> 10.0%:  11,092 per arm, about   22 days
detect 8.9% -> 10.9%:   3,492 per arm, about    7 days
```

"Half a point takes three and a half months. The team planned for 1.1 points and rounded 22 days up to 28: four complete weeks, so every weekday is counted four times."

3 **Load and check the unit (2m).** Say: "The test has run. Before a single conversion: is this the experiment the plan describes?"

```
ab = pd.read_csv("data/ab_test.csv", parse_dates=["date"])
print("Rows:", len(ab), " unique visitors:", ab["visitor_id"].nunique())
print("Dates:", ab["date"].min().date(), "to", ab["date"].max().date(),
      f"({ab['date'].nunique()} days)")
print(ab["variant"].value_counts().to_string())
```

```
Rows: 28000  unique visitors: 28000
Dates: 2025-09-08 to 2025-10-05 (28 days)
variant
treatment    14079
control      13921
```

One row per visitor, 28 days. The arms are not equal: 14,079 against 13,921. "Is that a problem?" Let them argue for ten seconds.

4 **Health check 1: the split (2m).** Say: "A fair coin rarely lands exactly half and half. `binomtest` asks whether THIS imbalance is more than a coin produces."

```
n_treat = int((ab["variant"] == "treatment").sum())
srm = stats.binomtest(n_treat, n=len(ab), p=0.5)
print(f"Treatment share: {n_treat / len(ab):.2%}")
print(f"p-value against a 50/50 split: {srm.pvalue:.3f}")
```

```
Treatment share: 50.28%
p-value against a 50/50 split: 0.348
```

"One of the rare places where you hope for a LARGE p-value." Write on the board: a failed check means stop, not adjust.

5 **Health check 2: balance (2m).** Say: "Both arms should have the same mix of anything fixed before the visitor saw a page."

```
for col in ["device", "returning"]:
    mix = pd.crosstab(ab["variant"], ab[col], normalize="index").round(3)
    p_mix = stats.chi2_contingency(pd.crosstab(ab["variant"], ab[col])).pvalue
    print(mix.to_string())
    print(f"chi-square p = {p_mix:.2f}\n")
```

```
device     desktop  mobile  tablet
variant
control      0.376   0.551   0.073
treatment    0.379   0.554   0.067
chi-square p = 0.11

returning  False  True
variant
control    0.652  0.348
treatment  0.646  0.354
chi-square p = 0.27
```

About 55% mobile and 35% returning in both arms. "The experiment is healthy, so its results can be read."

6 **The primary result: counts first (2m).** Say: "Now the planned analysis, run once. Counts before rates."

```
summary = ab.groupby("variant")["converted"].agg(
    conversions="sum", visitors="count", rate="mean")
print(summary.round(4))
c_c, n_c = summary.loc["control", ["conversions", "visitors"]]
c_t, n_t = summary.loc["treatment", ["conversions", "visitors"]]
rate_c, rate_t = c_c / n_c, c_t / n_t
print(f"\nDifference: {(rate_t - rate_c) * 100:+.2f} percentage points")
print(f"Relative lift: {rate_t / rate_c - 1:+.1%}")
```

```
           conversions  visitors    rate
variant
control           1233     13921  0.0886
treatment         1412     14079  0.1003

Difference: +1.17 percentage points
Relative lift: +13.2%
```

Stop here. "1.17 points AND 13.2%. Both true. Which will the head of e-commerce hear if you say 'conversion improved 13%'?" (From 9% to 22%.)

7 **The test and its interval (3m).** Say: "Same pattern as last session: difference, divided by its standard error. `proportions_ztest` takes COUNTS, treatment first."

```
z, p = proportions_ztest([c_t, c_c], [n_t, n_c])
low, high = confint_proportions_2indep(c_t, n_t, c_c, n_c, compare="diff")
print(f"z = {z:.2f}, p = {p:.4f}")
print(f"95% CI for the difference: {low * 100:+.2f} to {high * 100:+.2f} point
```

```
z = 3.35, p = 0.0008
95% CI for the difference: +0.49 to +1.86 points
```

"If the pages converted equally, a gap this large would turn up in fewer than 1 experiment in 1,000." Then break it on purpose: pass the rates `[0.1003, 0.0886]` instead of counts and show `p = 0.98`. "No error. A wrong number that looks fine."

8 **Wrap it in a function (2m).** Say: "We will run this comparison on slices all afternoon. One function, one dictionary of everything worth reporting." Type it carefully; the lab and the project reuse it.

```python
def compare_rates(df):
    """Treatment vs control conversion for the rows in df."""
    s = df.groupby("variant")["converted"].agg(["sum", "count"])
    c_c, n_c = s.loc["control"]
    c_t, n_t = s.loc["treatment"]
    r_c, r_t = c_c / n_c, c_t / n_t
    ci = confint_proportions_2indep(c_t, n_t, c_c, n_c, compare="diff")
    return {"visitors": int(n_c + n_t), "control": r_c, "treatment": r_t,
            "diff": r_t - r_c, "ci_low": ci[0], "ci_high": ci[1],
            "se": np.sqrt(r_c * (1 - r_c) / n_c + r_t * (1 - r_t) / n_t),
            "p": proportions_ztest([c_t, c_c], [n_t, n_c])[1]}

overall = compare_rates(ab)
for name in ["control", "treatment", "diff", "ci_low", "ci_high", "p"]:
    print(f"{name:<10}{overall[name]:.4f}")
```

```
control   0.0886
treatment 0.1003
diff      0.0117
ci_low    0.0049
ci_high   0.0186
p         0.0008
```

9 **Statistical against practical (1m).** Say: "The product team said half a point is the least worth having. Lay the interval next to that."

```python
worthwhile = 0.005      # half a percentage point, from the plan
print("Whole interval above zero:", overall["ci_low"] > 0)
print("Whole interval above the worthwhile lift:", overall["ci_low"] > worthwh
```

```
Whole interval above zero: True
Whole interval above the worthwhile lift: False
```

"Real effect, probably worthwhile, size still uncertain. That sentence goes in the memo as a caveat."

10 **What a weekly peeker would have seen (3m).** Say: "The plan said: analyze once, after day 28. Someone opens the dashboard every Monday. Predict: is the p-value below 0.05 after week 1?"

```
ab["week"] = (ab["date"] - ab["date"].min()).dt.days // 7 + 1
for wk in [1, 2, 3, 4]:
    r = compare_rates(ab[ab["week"] <= wk])
    print(f"after week {wk}: {r['visitors']:>6,} visitors   "
          f"control {r['control']:.2%}   treatment {r['treatment']:.2%}   "
          f"p = {r['p']:.4f}")
```

```
after week 1:  7,018 visitors   control 8.20%   treatment 9.25%   p = 0.1188
after week 2: 14,083 visitors   control 8.82%   treatment 9.73%   p = 0.0622
after week 3: 21,043 visitors   control 9.12%   treatment 9.99%   p = 0.0308
after week 4: 28,000 visitors   control 8.86%   treatment 10.03%   p = 0.0008
```

"After week 1 an impatient team abandons a page that works. Stopping at the first p below 0.05 is stopping at a lucky moment by construction." Quote the lesson's simulation, do not run it: 4.4% false alarms with one look, 12.2% with four looks, about 27% with daily looks. The day-by-day chart and the simulation are homework reading.

11 **Segments: handle with care (3m).** Say: "Did it work for everyone? Same function, three slices."

```
by_device = pd.DataFrame({dev: compare_rates(g)
                          for dev, g in ab.groupby("device")}).T
show = ["control", "treatment", "diff", "ci_low", "ci_high"]
print((by_device[show] * 100).round(2))
print(by_device["p"].round(5).to_string())
mob, desk = by_device.loc["mobile"], by_device.loc["desktop"]
gap = mob["diff"] - desk["diff"]
gap_se = np.sqrt(mob["se"] ** 2 + desk["se"] ** 2)
p_gap = 2 * stats.norm.sf(abs(gap / gap_se))
print(f"\nDifference between the lifts: {gap * 100:.2f} points, p = {p_gap:.3f
```

```
         control  treatment  diff  ci_low  ci_high
desktop    11.73      12.00  0.27   -0.97     1.50
mobile      6.82       8.58  1.76    0.92     2.60
tablet      9.40      10.83  1.43   -1.24     4.13
desktop    0.67236
mobile     0.00004
tablet     0.29476

Difference between the lifts: 1.49 points, p = 0.049
```

Let the room write the tempting headline first ("works on mobile, not on desktop"). Then three objections: nobody planned it; desktop's interval still contains +1.17; and the claim is about a DIFFERENCE, which needs its own test: p = 0.049, on the line, unplanned. "A lead, not a finding. The word for the memo is exploratory."

12 **The guardrail, on every visitor (1m).** Say: "More orders that are each much smaller would not be a win. Buyers are not a randomized group, visitors are: count a non-buyer as $0."

```
ab["revenue"] = ab["order_value"].fillna(0)
rev_c = ab.loc[ab["variant"] == "control", "revenue"]
rev_t = ab.loc[ab["variant"] == "treatment", "revenue"]
res = stats.ttest_ind(rev_t, rev_c, equal_var=False)
print(f"Revenue per visitor: control ${rev_c.mean():.2f}, "
      f"treatment ${rev_t.mean():.2f}")
print(f"Welch p = {res.pvalue:.4f}")
```

```
Revenue per visitor: control $7.90, treatment $8.94
Welch p = 0.0045
```

"Conversion up, revenue per visitor up, guardrail holds: ship it. You write that memo in the project."

THREE MISTAKES TO EXPECT

- **The raw difference as the effect size.** Someone types `solve_power(effect_size=0.011, ...)` and gets 129,733 per arm. No error. Coach: "What does the documentation call that argument? Which function turns two rates into it?" (`proportion_effectsize(target, baseline)` ; the right answer is 11,092.) In the lab this shows up as a follow-up test that would take a year.
- **Rates instead of counts into the z-test.** `proportions_ztest([0.1003, 0.0886], [14079, 13921])` prints `z = 0.024, p = 0.98` . Coach: "Does your result reproduce the rates you printed a cell ago? What would 'a tenth of one conversion' mean?" The habit to teach: counts in, and check the output against the summary table.
- **Two p-values compared to declare a difference.** "Week 4 is significant and weeks 1 to 3 are not, so it only worked in week 4." Coach: "Read me the four lifts, not the four p-values. Does any weekly interval exclude +1.17?" Then: "A claim about a difference needs a test of the difference." The lab is built on exactly this.

GUIDED LAB (20 MIN)

Learners open `labs/module-13/l4_6-ab-testing/novelty_check_starter.py` ; your reference is `labs/module-13/l4_6-ab-testing/novelty_check_solution.py` . The question is new: a product manager fears a novelty effect (a lift that was big in week 1 and has been shrinking), and a colleague reads the weekly table as "it only worked in week 4". `compare_rates` and `describe` are given.

- **Minutes 0 to 5, TODO 1 and 2.** Walk the room. The week column is the demo's line; the trap is using `<=` (cumulative, the peeking table) instead of `==` (each week by itself). Expected weekly lifts: +1.05, +0.76, +0.80, +2.08 points; only week 4 has p below 0.05 (0.0026).
- **Minutes 5 to 12, TODO 3.** Halves: +0.91 points (weeks 1 to 2) and +1.43 (weeks 3 to 4). Difference +0.52 points, standard error 0.70, p = 0.46. If someone subtracts the p-values, send them back to the demo's device cell.
- **Minutes 12 to 17, TODO 4.** Everyone writes the sentence. Ask two learners to read theirs aloud. A good one: "Every week shows a positive lift and every weekly interval contains the overall +1.17. The late lift is not smaller than the early one (p = 0.46 for the difference), so there is no sign of a novelty effect. One week alone is too small to be significant."
- **Minutes 17 to 20, debrief.** Two questions: "Why is week 4 significant alone and week 2 not?" (Chance plus 7,000 visitors per week: a week has little power.) "Was this check in the plan?" (No: exploratory, and say so.)
- **Fast finishers** start `labs/module-13/l4_6-ab-testing/followup_test_plan_starter.py` (size a mobile-only confirmation test: 7,458 per arm, about 27 days, for 6.8% to 8.0%). It goes home for everyone else, with `labs/module-13/l4_6-ab-testing/aa_test_starter.py` as the stretch.

START THE LESSON PROJECT (5 MIN)

Open the lesson page at "Project: The Decision Memo". Read the brief aloud (one notebook, one new segment question, a one-page memo to the head of e-commerce). Everyone creates `lesson_4_6_project.ipynb` and runs the Step 1 health-check cell: four printed lines ending `Balance on returning: p = 0.27`. Say what they will meet at home in Step 3: a colleague who reads two p-values (0.008 and 0.051) on opposite sides of 0.05. "You have just done the same thing with weeks." Steps 2 to 5 are homework; the memo is the point.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q1 to Q6 (learners do them at home; use two as cold calls if time allows). Good cold calls: Q4 (the broken split) and Q6 (the memo sentence).

Answers: **Q1** C) Groups that are alike in everything except the page, including things nobody measured; **Q2** A) `effect_size` expects Cohen's h from `proportion_effectsize(0.100, 0.089)`, not the raw difference in rates; **Q3** `1.17 0.132`; **Q4** D) Stop and find where the control visitors went: the experiment did not run as planned, so its results cannot be read; **Q5** B) About 12%, because each look is another chance to cross the line by luck; **Q6** C) The gain appears concentrated on mobile; the evidence that devices differ is borderline and exploratory, and worth a follow-up test. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pair anyone who was shaky on p-values last session with a confident neighbor for the lab, and give them the sentence frame for TODO 4: "Every week ... The difference between the halves is ... so ...". They may skip step 2 of the demo notebook (the days table) and copy `compare_rates` from the lesson page instead

of typing it. **Stretch:** `followup_test_plan` then `aa_test`; or the lesson's bonus challenge (find the per-look threshold that brings four looks back to 5%). Ask stretch learners: "Your follow-up plan uses the observed 8.6% as the target. Why is that optimistic?" (A test that crossed the line tends to overstate its effect.)

BREAK (1:10 TO 1:20)

Leave the weekly peeking table on the projector. Over the break, open the second notebook and the Lesson 4.7 page, and tick nothing on the board yet.

4.7 An EDA Workflow

55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Follow a seven-step exploratory data analysis (EDA) routine on an unfamiliar dataset, starting from written questions
- Establish what one row means, which columns are keys, and which values are missing and why
- Summarize single variables, pairs of variables and segments with the right table, chart or test for each
- Tell a finding from a lead: correct for scanning many comparisons and check an obvious competing explanation
- Write up findings, caveats and next questions so that someone else can act on them

TIMING INSIDE THE BLOCK (55 MINUTES)

10m	Frame and mental model
20m	Live demo, steps 1 to 10
20m	Guided lab: who pays for shipping?
5m	Start the returns project

MENTAL MODEL TO SAY

"A good detective does not walk in and arrest the first person who looks nervous. They ask what they are trying to find out, secure the scene, note what is there and what is missing, examine each piece of evidence, look for connections, and write a report that separates what is established from what is suspected. Same order every time, because the routine is what stops them from overlooking the obvious."

Write the seven steps on the board, numbered, and leave room beside each for a tick: **1 Questions · 2 Structure · 3 Quality · 4 One variable · 5 Two variables · 6 Segments · 7 Findings and caveats**. Draw the arrow from 7 back to 1. Say: "There is very little new syntax in this lesson. What is new is the order, and the judgment about what each result means. And in forty minutes you will do it alone, on the clock." (The checkpoint is a small run of this routine; saying so focuses the room.)

Connect to the first half: "Plan first, separate confirmed from exploratory, write it down. Same habits, without the luxury of a coin flip."

LIVE DEMO (YOU TYPE, THEY FOLLOW): THE CUSTOMER SURVEY IN TEN CELLS

A second notebook, `session13_eda.ipynb`. Tick each step on the board as you reach it. The marked minutes add up to 20. The lesson's two charts (six bar panels, the heatmap of gaps) are shown from the lesson page, not typed.

1 **Questions first (1m).** Say: "The head of customer experience has budget for one or two projects. Before the file is open, what do we want to know?" Type the four questions as a comment (or have them on the board):

```
# 1. Which parts of the experience do customers rate lowest?
# 2. Which part is most closely tied to overall satisfaction and recommending?
# 3. Is any store or age group noticeably less satisfied, and with what?
# 4. Who answered? Can these responses speak for Pinecrest's customers in gene
```

2 **Structure: one function for every new table (3m).** Say: "The most important fact about a table is its grain: what one row is. I want the same first look at every table, so I write it once."

```
import numpy as np
import pandas as pd
from scipy import stats
pd.set_option("display.width", 150)

survey = pd.read_csv("data/survey.csv", parse_dates=["survey_date"])

def overview(df):
    """One row per column: type, missing values, distinct values and an exampl
    return pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "missing": df.isna().sum(),
        "missing_pct": (df.isna().mean() * 100).round(1),
        "unique": df.nunique(),
        "example": df.apply(lambda col: str(col.dropna().iloc[0])[:22]),
    })

print(survey.shape)
print(overview(survey))
```

```
(1800, 12)
                         dtype  missing  ...  unique                    example
response_id              int64        0  ...    1800                          1
customer_id              int64        0  ...    1800                      13742
survey_date    datetime64[us]        0  ...      46     2025-10-01 00:00:00
age_group                 str        0  ...       6                        65+
store                     str        0  ...       5                  Asheville
sat_overall           float64       53  ...       5                        5.0
sat_selection         float64       53  ...       5                        5.0
sat_price             float64       47  ...       5                        5.0
sat_staff             float64      605  ...       5                        5.0
sat_website           float64      472  ...       5                        3.0
recommend             float64       54  ...      11                        6.0
comment                   str      977  ...      26  Love this store. Will

[12 rows x 5 columns]
```

Read it aloud with them: `response_id` and `customer_id` have 1,800 distinct values in 1,800 rows (grain: one response per customer). Ratings are float only because of the gaps. Then point: "`sat_staff` 33.6% missing, `sat_website` 26.2%. Too many to be carelessness. Park it for one minute."

3 **Quality: impossible values, duplicates (1m).** Say: "Quick checks first: anything outside its scale, any repeated row?"

```
items = ["sat_overall", "sat_selection", "sat_price", "sat_staff", "sat_websit
print(survey[items + ["recommend"]].agg(["min", "max"]))
print("Fully duplicated rows:", survey.duplicated().sum())
```

```
     sat_overall  sat_selection  sat_price  sat_staff  sat_website  recommend
min          1.0            1.0        1.0        1.0          1.0        0.0
max          5.0            5.0        5.0        5.0          5.0       10.0
Fully duplicated rows: 0
```

4 **Quality: missing for whom? (3m).** Say: "The useful question is never only 'how many are missing?' It is 'missing for WHOM?' Predict: who skipped the staff question?"

```
survey["shops"] = np.where(survey["store"] == "Online", "online", "in store")
missing_share = survey.groupby("shops")[items].agg(lambda col: col.isna().mean
print(missing_share.round(2))
```

```
          sat_overall  sat_selection  sat_price  sat_staff  sat_website
shops
in store         0.03           0.03       0.02       0.03         0.46
online           0.03           0.03       0.03       0.70         0.03
```

"70% of online shoppers skipped the staff question. People left out the part of Pinecrest they do not use. So `sat_staff` is really 'what store shoppers think of the staff'. Fill those gaps with the average and you change who the analysis is about." Then the two silent mistakes, one line each: `len(survey.dropna())` (295 of 1,800 left) and `survey['sat_staff'].fillna(0).mean()` (2.67 instead of 4.02).

5 **Quality: who is in the data? (2m).** Say: "Question 4. The customer table can tell us who answered. `validate` makes pandas prove the relationship I believe in."

```
customers = pd.read_csv("data/customers.csv")
joined = survey.merge(customers[["customer_id", "loyalty_tier"]], on="customer
                      how="left", validate="one_to_one")
who = pd.DataFrame({
    "survey": joined["loyalty_tier"].value_counts(normalize=True),
    "all_customers": customers["loyalty_tier"].value_counts(normalize=True),
}).round(3)
print(who)
```

```
              survey  all_customers
loyalty_tier
Basic          0.456          0.582
Silver         0.349          0.267
Gold           0.195          0.151
```

"Silver and Gold are 54% of respondents and 42% of customers. Engaged customers answer surveys. That is a caveat for every sentence we write from here on."

6 **One variable at a time (2m).** Say: "Ratings are ordinal: median and the share at each end, the mean as a convenient extra. Divide by the number who ANSWERED."

```
summary = pd.DataFrame({
    "answers": survey[items].count(),
    "median": survey[items].median(),
    "mean": survey[items].mean().round(2),
    "pct_4_5": (survey[items].ge(4).sum() / survey[items].count() * 100).round
})
print(summary.sort_values("mean"))
```

```
               answers  median  mean  pct_4_5
sat_price         1753     3.0  3.31     41.5
sat_website       1328     4.0  3.54     51.9
sat_overall       1747     4.0  3.92     68.3
sat_staff         1195     4.0  4.02     73.3
sat_selection     1747     4.0  4.04     75.1
```

Question 1 answered: price is weakest (median 3, 41.5% satisfied), then website. The six-panel bar chart of the distributions is in the lesson; show it from the lesson page, do not type it.

7 **Two variables (2m).** Say: "Question 2: which item goes with overall satisfaction and recommending? Ordinal data, so Spearman."

```
corr = survey[items + ["recommend"]].corr(method="spearman")
print(corr[["sat_overall", "recommend"]].round(2))
```

```
               sat_overall  recommend
sat_overall           1.00       0.63
sat_selection         0.49       0.55
sat_price             0.45       0.53
sat_staff             0.49       0.58
sat_website           0.43       0.48
recommend             0.63       1.00
```

"0.43 to 0.58. No single item dominates, and a customer in a good mood rates everything higher. This step tells us where to look, not what causes what."

8 **Segments: the table of means (1m).** Say: "Question 3. Eleven groups by five items: 55 numbers. Find the odd ones by eye. You have ten seconds."

```
by_store = survey.groupby("store")[items].mean()
by_age = survey.groupby("age_group")[items].mean()
print(pd.concat([by_store, by_age]).round(2))
```

```
           sat_overall  sat_selection  sat_price  sat_staff  sat_website
Asheville         3.77           4.05       3.35       3.62         3.58
Bend              3.96           4.01       3.30       4.16         3.63
Boise             3.86           4.01       3.24       4.11         3.50
Flagstaff         3.99           4.00       3.40       4.02         3.50
Online            3.96           4.07       3.30       4.07         3.54
18-24             3.95           4.06       2.80       4.05         3.69
25-34             3.88           3.99       3.32       4.02         3.67
35-44             3.99           4.07       3.38       3.94         3.67
45-54             3.95           4.08       3.38       4.01         3.66
55-64             3.85           3.99       3.44       4.04         3.72
65+               3.92           4.05       3.33       4.05         3.04
```

They will find Asheville staff (3.62), 18-24 price (2.80) and 65+ website (3.04). "You just ran 55 comparisons with your eyes. That is multiple testing, whether or not you call it that."

9 **Discipline 1: count the comparisons and correct (3m).** Say: "Each group against everyone else, on each item, Mann-Whitney for ratings, then Holm." This is the one block today you may paste from the lesson page; read it line by line instead of typing it.

```python
from statsmodels.stats.multitest import multipletests

rows = []
for seg in ["store", "age_group"]:
    for level in sorted(survey[seg].unique()):
        inside = survey[seg] == level
        for item in items:
            x = survey.loc[inside, item].dropna()
            y = survey.loc[~inside, item].dropna()
            rows.append({"group": level, "item": item, "n": len(x),
                         "gap": x.mean() - y.mean(),
                         "p": stats.mannwhitneyu(x, y).pvalue})
scan = pd.DataFrame(rows)
scan["p_holm"] = multipletests(scan["p"], method="holm")[1]
print(f"{len(scan)} tests, {(scan['p'] < 0.05).sum()} below 0.05 before "
      f"correction, {(scan['p_holm'] < 0.05).sum()} after")
top = scan.sort_values("p").head(4).round({"gap": 2, "p": 6, "p_holm": 4})
print(top.to_string(index=False))
```

```
55 tests, 9 below 0.05 before correction, 4 after
    group        item   n   gap        p  p_holm
      65+ sat_website 291 -0.64 0.000000  0.0000
    18-24   sat_price 186 -0.57 0.000000  0.0000
Asheville   sat_staff 193 -0.47 0.000000  0.0000
    55-64 sat_website 210  0.21 0.000919  0.0478
```

Nine before, four after. Three are the dark cells. The fourth: 55-64 year olds rate the website +0.21 higher. "Look at the size and at the cause: 'everyone else' includes the unhappy 65+ group. A side effect of how I built the scan. It does not go in the findings."

10 **Discipline 2: could something else explain it? (2m).** Say: "Asheville's staff rating is the result most likely to lead to action, so challenge it. Maybe Asheville just has a different age mix."

```
in_store = survey[survey["store"] != "Online"]
staff_table = in_store.pivot_table(index="store", columns="age_group",
                                   values="sat_staff", aggfunc="mean")
print(staff_table.round(2))
```

```
age_group  18-24  25-34  35-44  45-54  55-64   65+
store
Asheville   3.67   3.71   3.17   3.57   3.79  3.82
Bend        4.22   4.08   4.07   4.27   4.15  4.17
Boise       4.18   4.13   4.10   4.15   4.02  4.11
Flagstaff   3.85   4.11   4.05   4.05   3.94  4.09
```

"Asheville is the lowest store in every age group. Age mix does not explain it. That does not prove the staff are the cause: it removes one competing explanation and leaves the finding standing." Close by showing the lesson's findings note on the projector: findings with numbers, caveats, next questions. Step 7 is what they do in the lab.

THREE MISTAKES TO EXPECT

- `dropna()` **on the whole table.** In the lab or the checkpoint someone drops every row with any gap. On the survey that leaves 295 of 1,800 responses. Coach: "How many rows did you have before and after that line? Which column does THIS question need?" (`dropna(subset=[...])`.)
- **A merge that multiplies rows.** `survey.merge(orders, on='customer_id')` turns 1,800 rows into 9,230 and no message appears. Coach: "What is one row now? Say it out loud." Then: "What relationship did you expect? Put it in `validate=` and let pandas check." Summarize to one row per customer first, then merge.
- **The best of many comparisons reported as if it were planned.** Learners will scan a table, find the biggest gap and write it up with its raw p-value. Coach: "How many groups did you look at before you chose that one?" The three-part fix: count, correct, call it exploratory, and look at the size of the gap, not only its p-value.

GUIDED LAB (20 MIN)

Learners open `labs/module-13/l4_7-eda-workflow/shipping_rules_starter.py`; your reference is `labs/module-13/l4_7-eda-workflow/shipping_rules_solution.py`. Nobody at Pinecrest can

find the shipping policy: the learners work the rule out of the data by running steps 1 to 7 on one column. Questions and the structure cell are given.

- **Minutes 0 to 4, TODO 1 (quality).** The crosstab of channel by charge shows store orders never pay shipping (7,448 at 0.00). "So which rows are the question about?" Online only.
- **Minutes 4 to 7, TODO 2 (one variable).** Online: 71.4% ship free, 21.4% pay 6.95, 7.2% pay 14.95.
- **Minutes 7 to 13, TODO 3 and 4 (two variables).** The key move is `goods = order_total - shipping_cost`. If someone groups by `order_total` the threshold is blurred by the shipping itself: ask "What is included in order_total?" (It is in the data dictionary.) The min/max table gives it away: free orders start at exactly 75.00, the 6.95 orders stop at 74.96. The two-by-three table has zeros where a rule with no exceptions should.
- **Minutes 13 to 16, TODO 5 (segments).** The 14.95 charge sits at 6% to 9% of orders in every size band: not tied to order size. "What could it be?" (Faster delivery, chosen by the customer: a hypothesis. No column records it.)
- **Minutes 16 to 20, TODO 6 (the note).** Everyone writes findings and at least one caveat. Push on the caveat: "Did you READ the rule or INFER it?" Take one note on the projector and mark each sentence F (finding), C (caveat) or N (next question).
- Goes home: `labs/module-13/l4_7-eda-workflow/order_size_scan_starter.py` (a 22-test scan with one survivor and a competing explanation; the best preparation for the capstone) and `labs/module-13/l4_7-eda-workflow/customer_overview_starter.py` (stretch).

START THE LESSON PROJECT (5 MIN)

Open the lesson page at "Project: Why Do Orders Come Back?". Read the brief. Everyone creates `lesson_4_7_project.ipynb`, writes their own four questions in a Markdown cell **before** scrolling to the lesson's list, and runs the Step 2 cell (three tables, three grains: one row per order, per order line, per product). Tell them the project is 45 minutes at home and that Step 6 is where the finding is. Do not reveal it.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q7 to Q12. Good cold calls: Q9 (who answered?) and Q12 (finding or caveat?).

Answers: **Q7** B) Its grain: what one row represents, and which column identifies a row; **Q8** `0 2`; **Q9** D) Customers who answered, mostly online shoppers, rate the website 3.54; **Q10** `4 5`; **Q11** A) The one that survived, with the size of its gap, the number of comparisons, and the result of checking an obvious competing explanation; the other two are leads; **Q12** B) Every respondent has placed an order, so customers who never ordered are not heard. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: in the lab, give the `goods` line to anyone stuck at minute 9 and let them spend their time on the tables and the note. For the scan in the demo, they follow on the lesson page instead of typing. **Stretch:** `order_size_scan` in the room, then the lab's next question ("do customers add items to reach $75.00?": look at the distribution of `goods` between 60 and 90). Ask stretch learners to say what an inner merge of

orders and customers silently does (drops 739 orders from 150 closed accounts; `customer_overview` shows it).

CHECKPOINT 1

2:15 Checkpoint 1: analysis task on Modules 1 to 4

30 + 5 min

WHAT IT IS

A 30-minute, individual, open-notebook analysis task on Modules 1 to 4: one small question on the course data that no lesson has worked, answered with the routine the learners have just practiced. It is formative. Nothing rides on the score; it tells each learner, and you, where they stand before the course turns to databases and the web, and it is a rehearsal for the capstone at one-tenth the size. The rubric uses the same four levels and the same row names as the capstone rubric in Lesson 8.1 (without the Reproducibility row), so the feedback carries forward.

The question (also on the task sheet in the workbook):

> **CHECKPOINT 1 QUESTION**
>
> Pinecrest's store buyers believe that **December shoppers buy more expensive things**. Finance wants a number before it plans next December's stock. Using `data/daily_sales.csv`: is the average price per unit sold (revenue ÷ units) higher in December than in the rest of the year? By how much, how sure are you, and is it the same in all four stores?

Why this question: it needs a derived column, a check for missing values, a fair comparison of two groups, a segment check, one chart and an honest note, and it has a second layer for strong learners (December is the peak of a winter plateau, so "December against the rest" understates the gap). It is not one of the four capstone questions and no lesson, project or exercise computes revenue per unit by month.

RUN IT IN THE ROOM

1. **Before 2:15.** Close the returns project notebooks. Each learner opens a NEW notebook (or the scaffold `labs/module-13/checkpoint-1/checkpoint1_starter.py`, which has the question, the load line and six empty headings). Workbooks open at the Checkpoint 1 task sheet.
2. **Rules, said once (1 minute).** "Thirty minutes. Your own work. Open notebook means: your notes, your earlier notebooks, your helper functions, the lesson pages and the library documentation are all allowed. Your neighbor and AI assistants are not. I will answer questions about what the question means and about a crashing environment, not about method. A finished small answer beats an unfinished big one."
3. **2:15 start.** Put the clock and the question on the projector (the deck has a slide for it). Say: "The first three minutes are for the plan. No code yet."
4. **Time calls.** At 3 minutes: "Plan written? Start coding." At 12: "You should have your column and your checks." At 20: "You should have your comparison. Make the figure." At 25: "Write the note now, even if

the analysis is not finished. Say what you would do next." At 30: "Stop. Save."

5. **While they work.** Walk once at minute 8 and once at minute 18 and write down, per learner, one thing you see (plan written or not; NaN handled or not; interval present or not). Those notes make the marking fast. Do not coach method. If someone is frozen at minute 10, say only: "What is one row? What would you compute for each row?"

6. **2:45 hand-in.** Learners save the notebook as `checkpoint1_<name>.ipynb` (or `.py`) and hand it in the way your course collects work (shared folder, email, USB stick). The note inside the notebook is the part you mark first. On paper is fine for the note if machines misbehave.

7. **2:47 two-minute debrief.** Show ONE thing from the model answer: the twelve monthly means (or the saved figure). Ask: "Who compared December with all other months? Look at January and February. What is hiding in 'the rest'?" Do not show the numbers of the full model answer if another group sits the checkpoint later; otherwise post the model answer after marking.

MARK IT

Use the 5-row rubric in the Answer Key (4 levels, 20 points; Proficient in every row is 15). Budget five minutes per learner: read the note first, then scan the notebook for the plan, the missing-value handling, the interval and the figure. The Answer Key has the full model answer from a real run and seven common weak answers with what each reveals.

- **Marking in the room** is realistic only for the Question-and-plan row (your walk at minute 8) and the Data-handling row (your walk at minute 18). Mark the rest afterward and return it at the start of Session 14, before the warm-up.
- **The numbers a correct answer contains:** 2,897 usable store-days (6 dropped for missing revenue); December $44.95 against $41.65; difference $3.31 (+7.9%), 95% CI $2.97 to $3.65; Cohen's d = 1.33; gaps of $2.86 to $3.79 in the four stores. Mann-Whitney instead of Welch is equally acceptable; so is the ratio of sums ($45.06 against $41.53).
- **Return the rubric, not a grade.** Circle one cell per row and write one sentence: the single change that would move the lowest row up a level. Anyone with a row at Beginning gets a two-minute conversation at the next break.
- **What to do with the results.** If more than a third of the room has no interval or effect size, open Session 14's warm-up with a three-minute redo of the Welch cell from the model answer. If more than a third skipped the plan, start the Session 14 lab with "write the question first" as an explicit step. Note learners at Beginning in two or more rows: they are the ones to pair for support through Module 5 and to check on at the capstone's first milestone.

Accommodations: extra time is fine (let the learner finish during the wrap-up and hand in at 3:00); a learner who missed Session 12 may do the task with the Lesson 4.5 page open at "A Template for Reporting Any Test". A learner who is absent today does it at home on the honor system before reading any answers.

CLOSING

EXIT TICKET (3 QUESTIONS, ON A STICKY NOTE OR IN THE CHAT; ANSWERS)

1. Name two things you check about an A/B test before you look at a single conversion. **Answer:** Any two of: one row per unit (`visitor_id` is unique); the split is consistent with the design (`stats.binomtest` against 50/50: here p = 0.35); the arms are balanced on things fixed before the page was shown (device, returning). If a check fails: stop, do not adjust.
2. "Significant on mobile, not significant on desktop." What is the right next step, and what word goes in the memo? **Answer:** Test the difference between the two lifts directly (here 1.49 points, p = 0.049) and correct for the slices. The word is "exploratory": the comparison was not planned.
3. Write the first three steps of the EDA routine and say what "grain" means. **Answer:** Questions, structure, quality (then one variable, two variables, segments, findings and caveats). Grain: what one row of the table represents.

HOMEWORK (ABOUT 3.5 HOURS; THE TWO PROJECTS FIRST)

- **Lesson 4.6:** finish the Decision Memo (Steps 2 to 5; write your own memo before reading the lesson's), Exercises 1 to 4 (5 is a stretch), the quiz and the journal. Read "The Peeking Problem" from the daily chart to the end, and "Common Mistakes".
- **Lesson 4.7:** the returns project, all seven steps (about 45 minutes), Exercises 1 to 3 (4 is a stretch), the quiz and the journal. Read Step 7 of the lesson and compare your note with the lesson's only after writing yours.
- **Kit labs:** `labs/module-13/l4_6-ab-testing/followup_test_plan_starter.py` and `labs/module-13/l4_7-eda-workflow/order_size_scan_starter.py`. Stretch: `aa_test` and `customer_overview`.
- **Keep:** save `compare_rates`, `overview` and `rate_by` in a file of your own helpers. The capstone reuses them.
- **Workbook:** self-check Q1 to Q12.

BRING LIST FOR SESSION 14 (LESSONS 5.1 AND 5.2: SQL FROM PYTHON)

☐ `data/pinecrest.db` in your data folder (it came in the same zip as the CSV files; check with `python -c "import sqlite3; print(sqlite3.connect('data/pinecrest.db').execute('SELECT COUNT(*) FROM orders').fetchone())"`, which should print `(20302,)`). ☐ SQLAlchemy installed (it is in `requirements.txt`; check with `python -c "import sqlalchemy; print(sqlalchemy.__version__)"`, which should print a 2.1 version). ☐ Your sketch, on paper, of how `customers`, `orders`, `order_items` and `products` connect: which column links each pair and which side is "one" (Lesson 4.7, "Before the Next Lesson"). ☐ Your helpers file. ☐ No SQL is assumed, no server to install, no account needed: SQLite ships with Python.

IF YOU RUN LONG OR SHORT

Long. The checkpoint starts at 2:15 whatever happens. Cut in this order: (1) 4.6 demo step 2 (the days table) and step 9 (the threshold): say them. (2) 4.7 demo step 7 (the correlation columns): say "0.43 to 0.58, nothing dominates". (3) 4.7 demo step 5 (who answered): show the output from the lesson page. (4) "Start the project" for 4.7 becomes one sentence and homework. (5) The 4.6 lab's TODO 4 sentence is written at home. (6) Last resort: the shipping lab stops after TODO 4 (the rule is found) and the note is homework. Never cut: the health checks, the interval, "missing for whom", the corrected scan, the checkpoint.

Short (a small or fast group). In 4.6 run `followup_test_plan` in the room and ask for the one-paragraph plan aloud. In 4.7 run `order_size_scan` after the shipping lab: its Step 3 (is June about tents?) is the best ten minutes of capstone preparation in the kit. If the checkpoint itself finishes early for some learners, their stretch is on the task sheet (is it the mix? use `order_items` and `products`). Do not start the debrief before 2:45.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 14: SQL from Python: Queries and Joins

How to teach the first SQL session: connect to a database from Python, ask one table for exactly what you need, pass values safely, then combine tables with joins, CTEs and CASE without losing or inventing a single row.

`2 lessons` `3 hrs session` `Lessons 5.1–5.2` `Session 14 of 21`

HOW TO USE THIS GUIDE

- Module 5 starts here. For most learners this is their first SQL, and both lessons are dense. Every idea has a pandas twin they already own: say the pandas version aloud whenever a query looks foreign.
- Same rhythm as every session: **Frame → Live demo** (you type, they follow) **→ Guided lab** (new practice from the kit) **→ Start the project**. The lesson projects are finished at home.
- Type SQL one clause per line and run it after each clause. Every code block below was run on the course data; the gray block under it is the real output.
- Labs live in `labs/module-14/`: starters for learners, solutions for you. Teal blocks are hands-on, amber blocks are watch-outs.
- Keep `sql_database_connections.html` and `sql_complex_queries.html` open in browser tabs for the flow diagrams, the join table and the recursion diagram.

> **THE #1 RISK OF SESSION 14: RESULTS THAT ARE WRONG AND LOOK RIGHT**
>
> In pandas a bad merge at least changes `.shape` in front of you. In SQL an inner join drops 739 orders, a fan-out triples revenue and a comparison with NULL skips 18,154 rows, all without a word. The habit to build all session: **say the row count you expect before you run the query, then count**. The second risk is safety: one learner will build SQL with an f-string out of habit. Stop the room when you see it.

BEFORE THE SESSION

- ☐ From the `instructor-kit/` folder run `python3 labs/check_labs.py module-14`. Expect `12 passed, 0 failed`.
- ☐ Copy the **starter files only** to the shared folder: `labs/module-14/l5_1-database-connections/*_starter.py` and `labs/module-14/l5_2-complex-queries/*_starter.py`. Keep every `*_solution.py` on your machine.

- ☐ Files used today: `data/pinecrest.db` (3.5 MB, six tables) and, for the project checks at home, `orders.csv`, `order_items.csv`, `products.csv`. No server, no internet, no accounts: SQLite ships inside Python.
- ☐ On the projector machine, and on any learner machine you can reach, run `python -c "import sqlite3; print(sqlite3.sqlite_version)"` and `python -c "import sqlalchemy; print(sqlalchemy.__version__)"`. You need SQLAlchemy 2.x.
- ☐ Projector: one empty notebook, `session14_sql.ipynb`, in the folder that holds `data/`. Font 18 pt or larger. Delete any old `pinecrest_work.db` in that folder so step 1 starts clean.
- ☐ Board: on the left the running order **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT**; on the right, covered until after the break, **739 · 1,058 · 609 · 8**; across the top, "Say the row count before you run it."
- ☐ Checkpoint 1 rubrics, marked, one sentence of advice each. They go back in the first two minutes.
- ☐ Skim the Session 13 homework: the Decision Memo, the returns project (its finding, online footwear, is today's 5.2 project in SQL) and the paper sketch of the four tables.

KNOWN ENVIRONMENT TRAPS

An old SQLite. `RIGHT JOIN` and `FULL OUTER JOIN` need SQLite 3.39 or later (Lesson 5.2 shows a rewrite that works everywhere); nothing in the demo or the labs needs them. A version below 3.25 means Session 15's window functions will not run: note the learner and get them a newer Python before then. **A typo in the file name** creates a new, empty database and the error arrives one line later as `OperationalError: no such table: orders`. Look for a stray small `.db` file and delete it. **The wrong working folder** gives the same message for the same reason. The labs avoid both (they find the data through `labs_paths` and open it read-only). `database is locked`: two notebooks hold the same working copy and one has an uncommitted write; commit or restart one kernel. **Windows** will not overwrite `pinecrest_work.db` while a connection to it is open (`PermissionError` on the `shutil.copy` line when rerunning): restart the kernel first. **SQLAlchemy 2** refuses bare strings (`ObjectNotExecutableError`): wrap SQL in `text(...)`.

SESSION 14 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Checkpoint 1 rubrics back. Five retrieval questions on Session 13 (experiments, the EDA routine), no notes. Homework check.
0:10	10	**5.1** Frame and mental model	The binder and the librarian. SQL as a second language for ideas they already know from pandas.
0:20	30	**5.1** Live demo	A working copy, the catalog, cursors, `SELECT` clause by clause, `GROUP BY` and `HAVING`, placeholders against an f-string, a rollback,

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
			SQLAlchemy and `pd.read_sql`.
0:50	25	**5.1** Guided lab	`labs/module-14/l5_1-database-connections/staff_report_starter.py`, then `safe_lookup_starter.py`.
1:15	10	**5.1** Start the lesson project	The Store Scorecard: Steps 1 and 2 in the room (1,320 completed Boise orders in 2025).
1:25	10	Break	Leave the SQL-to-pandas table on the projector.
1:35	10	**5.2** Frame and mental model	Two stacks of index cards. The four unmatched-row numbers, written on the board.
1:45	30	**5.2** Live demo	Read-only connection, `JOIN` and its missing 739 rows, `LEFT JOIN` and the anti-join, three tables, the fan-out trap, a CTE, a recursive CTE, `CASE`, NULL.
2:15	25	**5.2** Guided lab	`labs/module-14/l5_2-complex-queries/join_row_counts_starter.py`, then the first two steps of `stock_vs_sales_starter.py`.
2:40	10	**5.2** Start the lesson project	The Returns Investigation: Step 1 in the room (6.9% online, 3.4% in stores), Step 2 read together.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 15.
Total	180		Ends at 3:00.

In the room: both mental models; SQL from the first `SELECT` to a parameterized `pd.read_sql` in thirteen cells; joins, the fan-out trap, a CTE, a recursive CTE, CASE and NULL in twelve cells; four short labs on the `employees` and `inventory` tables; the first step or two of each lesson project.

Moves home, exactly: Lesson 5.1: `sqlite3.Row` and iterating a cursor; `IN`, `BETWEEN`, `LIKE` and `DISTINCT`; grouping by `strftime`; named placeholders with sqlite3 and "What Placeholders Cannot Do"; `contextlib.closing`; `engine.begin()` and `inspect`; `chunksize`; the PostgreSQL comparison table; Common Mistakes 1 to 6; Store Scorecard Steps 3 to 6; Exercises 1 to 4; the quiz and the journal. **Lesson 5.2**: `RIGHT JOIN`, `FULL OUTER JOIN` and the rewrite; `CROSS JOIN`; scalar, `IN` and correlated subqueries and `EXISTS`; the org chart printed from the `path` column; CASE as size bands; the set operations; Common Mistakes 1 to 4; Returns Investigation Steps 2 to 6; Exercises 1 to 5; the quiz and the journal. **Kit labs**: `save_summary`, the CTE half of `stock_vs_sales`, and `org_payroll` (stretch). About three hours.

Protect: the placeholder demonstration (5.1 step 10) and the lab's injection step; the row-count habit (5.2 steps 2 and 3), the fan-out trap (step 6) and the lab's predictions. **Cut first:** see "If You Run Long or Short".

WARM-UP

0:00 Warm-up: Retrieval from Session 13

10 min

TIMING

2m	Checkpoint 1 rubrics back, one sentence to the room
6m	Five retrieval questions on paper, no notes, then answers
2m	Homework check and today's roadmap

CHECKPOINT 1 RUBRICS

Hand the rubrics back face down. One sentence to the room, chosen from what you saw when marking, for example: "Most of you had the right answer; fewer of you said how sure you were. An estimate without an interval is half an answer." If more than a third of the room had no interval or effect size, spend three warm-up minutes re-running the Welch cell from the model answer instead of questions 1 and 2. Individual conversations happen at the break.

FIVE RETRIEVAL QUESTIONS (LESSONS 4.6 AND 4.7)

1. Name three of the six lines of an experiment plan. **Answer:** Any three of: unit, primary metric, guardrail metric, hypotheses (two-sided, alpha), sample size and duration, decision rule.
2. Conversion went from 8.86% to 10.03%. Say the lift two ways. **Answer:** +1.17 percentage points (absolute) and +13% (relative). Both are correct; always say which.
3. Why is "stop the test the first time p is below 0.05" a problem? **Answer:** Each look is another chance to cross the line by luck. With weekly looks the false-alarm rate was about 12% instead of 5%; with daily looks about 27%.
4. What does "grain" mean, and what is the grain of `order_items`? **Answer:** What one row represents. One product line on an order (45,490 rows for 20,302 orders). Today's joins live or die on this.
5. Merging 1,800 survey rows with `orders` on `customer_id` gave 9,230 rows. What happened, and what guards against it? **Answer:** Each response was repeated once per order of that customer. Guard: summarize to one row per customer first, and state the relationship with `validate=`. In SQL the same trap is called fan-out, and you will see it turn $3.7 million into $10.3 million after the break.

HOMEWORK CHECK AND ROADMAP

Ask what the returns project found (online orders containing footwear are returned 14.4% of the time against 4.8%; in stores footwear makes no difference). "Keep that number. You will find it again today, in a different language." Ask one learner to describe the link between `orders` and `order_items` from their sketch.

Roadmap: "Every dataset so far arrived as a file. In most organizations the data lives in a database. Before the break: ask one table for what you need, safely. After the break: combine tables without losing or inventing rows."

WATCH-OUTS

- Questions 4 and 5 are the bridge to today. If the room is unsure about "grain", spend the extra minute now.
- A learner who missed Session 13 does Checkpoint 1 at home on the honor system (task sheet in the Session 13 workbook). Nothing today depends on it.

LESSON PLANS

5.1 Database Connections

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Open a SQLite database from Python, run a query through a cursor, and close the connection cleanly
- Write SELECT queries that filter, sort, aggregate and group rows, and say which pandas operation each clause matches
- Pass values into a query with parameters, and explain why building SQL with f-strings is dangerous
- Use a SQLAlchemy engine with text(), and move data both ways with pd.read_sql and DataFrame.to_sql
- Name the main things that change when the database is PostgreSQL instead of SQLite

TIMING INSIDE THE BLOCK (75 MINUTES)

10m	Frame and mental model
30m	Live demo, steps 1 to 13
25m	Guided lab: the staff report, then a safe stock lookup
10m	Start the Store Scorecard: Steps 1 and 2

MENTAL MODEL TO SAY

"A CSV file is a photocopied binder. To find last March's returned orders you carry the whole binder to your desk and flip through every page yourself. A database is a library with a librarian at the desk. You hand over a request slip, 'returned orders, March, Boise store, sorted by total', and the librarian brings back only those pages. The request slip is written in SQL, and the librarian is the database engine."

Then the reassurance: "You already know the ideas. Filtering, sorting, grouping and merging are what you did in Module 2. Today you learn to say the same things in a second language. When a query feels foreign, translate it into pandas in your head." Put the SQL-to-pandas table (it is in the workbook) on the projector and leave it up through the break.

One line on the engine: "SQLite is a complete SQL database in a single file, with no server and no password, and it ships inside Python. Nearly everything you write today runs unchanged on PostgreSQL." Have learners take out the paper sketch of the four tables from their homework: it comes into play after the break.

LIVE DEMO (YOU TYPE, THEY FOLLOW): FROM CONNECTION TO DATAFRAME

One notebook, `session14_sql.ipynb`, in the folder that holds `data/`. Thirteen cells; the marked minutes add up to 30. Type SQL one clause per line and run after each new clause, so every error is one clause old. The SQLite version printed in step 1 will differ from machine to machine.

1. **A working copy, then connect (2m).** Say: "First habit of this module: work on a COPY. Some lessons create tables and indexes, and the original must stay exactly as documented."

```
import shutil
import sqlite3
import pandas as pd

shutil.copy("data/pinecrest.db", "pinecrest_work.db")    # a private working c
conn = sqlite3.connect("pinecrest_work.db")
print("SQLite library version:", sqlite3.sqlite_version)
```

```
SQLite library version: 3.53.1
```

Your version will differ. "A connection is an open line to the file. Everything goes through it." Mention once: `connect` never complains about a wrong file name; it creates an empty database.

2. **What is in there? (2m).** Say: "Every SQLite database keeps a catalog of itself. This is your first SQL query: read it left to right."

```
tables = conn.execute(
    "SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY name")
print([name for (name,) in tables])
for row in conn.execute("PRAGMA table_info(orders)"):
    print(row)
```

```
['customers', 'employees', 'inventory', 'order_items', 'orders', 'products']
(0, 'order_id', 'INTEGER', 0, None, 1)
(1, 'customer_id', 'INTEGER', 1, None, 0)
(2, 'order_date', 'TEXT', 1, None, 0)
(3, 'channel', 'TEXT', 1, None, 0)
(4, 'store', 'TEXT', 1, None, 0)
(5, 'status', 'TEXT', 1, None, 0)
(6, 'payment_method', 'TEXT', 1, None, 0)
(7, 'promo_code', 'TEXT', 0, None, 0)
(8, 'discount_pct', 'INTEGER', 1, None, 0)
(9, 'shipping_cost', 'REAL', 1, None, 0)
(10, 'order_total', 'REAL', 1, None, 0)
```

Four old friends and two tables that exist only here: `employees` and `inventory` (today's labs live in those). Point at `order_date TEXT`: "SQLite has no date type. `2024-03-05` sorts correctly as text, so date filters still work."

3. **A cursor walks the rows (1m).** Say: "Running a statement gives a cursor: a pointer that moves forward through the result. Rows are tuples."

```
cur = conn.cursor()
cur.execute("SELECT order_id, store, order_total FROM orders")
print(cur.fetchone())        # the next row
print(cur.fetchmany(2))      # the next two, as a list
print(len(cur.fetchall()))   # everything that is left
```

```
(100001, 'Flagstaff', 434.9)
[(100002, 'Asheville', 278.9), (100003, 'Online', 75.9)]
20299
```

1 + 2 + 20,299 = 20,302. "Once a row is fetched it is gone from the cursor."

4 **A helper so we can read results (2m).** Say: "Reading tuples by hand gets old. pandas can run a query and hand back a DataFrame. This helper is all we need for an hour."

```
def show(sql, params=()):
    """Run a query and print the result as a table."""
    print(pd.read_sql(sql, conn, params=params).to_string(index=False))

show("SELECT order_id, order_date, store, status, order_total FROM orders LIMI
```

```
 order_id order_date     store    status  order_total
   100001 2024-01-01 Flagstaff completed        434.9
   100002 2024-01-01 Asheville completed        278.9
   100003 2024-01-01    Online completed         75.9
```

5 **Rows: WHERE (3m).** Say: "WHERE is the boolean mask of SQL. Text in SINGLE quotes, equality is a single `=`, conditions join with AND, OR, NOT." Type it clause by clause; run after each new line.

```
show("""
    SELECT name, price, rating
    FROM products
    WHERE category = 'Tents'
      AND price < 250
      AND rating >= 4.5
""")
```

```
                            name  price  rating
          Quillback Wayfinder Tarp 132.95     4.9
Quillback Alpenglow Tent Footprint  67.95     5.0
```

Ask for the pandas version aloud: `products[(products.category == 'Tents') & (products.price < 250) & (products.rating >= 4.5)]`. "Same idea, new spelling." `IN`, `BETWEEN` and `LIKE` are homework reading.

6 **Order and top N (1m).** Say: "Without ORDER BY the order of rows is not guaranteed, even if it looks stable today. ORDER BY with LIMIT is how SQL says top N."

```
show("""
    SELECT order_id, order_date, store, order_total
    FROM orders
    WHERE status = 'completed'
    ORDER BY order_total DESC, order_id
    LIMIT 3
""")
```

```
 order_id order_date  store  order_total
   106479 2024-09-15 Online       4893.0
   114268 2025-06-24 Online       4184.9
   113836 2025-06-11 Online       4150.4
```

"There is the $4,893 bulk order from Lesson 0.1, found without loading 20,301 other rows."

7 **Aggregates, and three kinds of COUNT (3m).** Say: "An aggregate turns many rows into one value. The five you will use daily: COUNT, SUM, AVG, MIN, MAX."

```
show("""
    SELECT COUNT(*)                  AS orders,
           ROUND(SUM(order_total), 2) AS revenue,
           ROUND(AVG(order_total), 2) AS avg_order
    FROM orders
    WHERE status = 'completed'
""")
show("""
    SELECT COUNT(*)             AS all_rows,
           COUNT(phone)         AS with_phone,
           COUNT(DISTINCT city) AS cities
    FROM customers
""")
```

```
 orders    revenue  avg_order
  18762 3624306.27     193.17
 all_rows  with_phone  cities
     4850        4570      35
```

"$3,624,306.27: the number you have computed with pandas since Module 2, answered by the engine in one row." Then the three COUNTs: rows, non-NULL values, different values (`.nunique()`). "Aggregates skip NULL the way pandas skips NaN."

8 **GROUP BY: one row per group (3m).** Say: "Split, apply, combine: Lesson 2.4. The rule for SQL: every column in SELECT is either in the GROUP BY or inside an aggregate."

```
show("""
    SELECT store,
           COUNT(*)                  AS orders,
           ROUND(SUM(order_total), 2) AS revenue,
           ROUND(AVG(order_total), 2) AS avg_order
    FROM orders
    WHERE status = 'completed'
    GROUP BY store
    ORDER BY revenue DESC
""")
```

```
    store  orders    revenue  avg_order
   Online   11604 2357984.45     203.20
    Boise    2502  443386.85     177.21
Flagstaff    1648  300580.35     182.39
     Bend    1650  284575.25     172.47
Asheville    1358  237779.37     175.10
```

9 **HAVING filters the groups (2m).** Say: "Which stores have an average order above $180? HAVING is the answer; then watch me put the condition in WHERE instead." The last line is the wrong version; read the error aloud.

```
show("""
    SELECT store, ROUND(AVG(order_total), 2) AS avg_order
    FROM orders
    WHERE status = 'completed'
    GROUP BY store
    HAVING AVG(order_total) > 180
    ORDER BY avg_order DESC
""")
conn.execute("SELECT store FROM orders WHERE AVG(order_total) > 180 GROUP BY s
```

```
    store  avg_order
   Online     203.20
Flagstaff     182.39
OperationalError: misuse of aggregate: AVG()
```

`misuse of aggregate`: "WHERE runs before the groups exist, so there is nothing to average yet." Draw the running order on the board: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT.

10 **Parameters, and the wrong way demonstrated safely (4m).** Say: "Real queries are rarely fixed text. The store comes from a dropdown, the name from a form. How the value gets into the query is the most important safety habit in this module."

```
def count_customers_unsafe(last_name):
    sql = f"SELECT COUNT(*) FROM customers WHERE last_name = '{last_name}'"  #
    return conn.execute(sql).fetchone()[0]

def count_customers_safe(last_name):
    sql = "SELECT COUNT(*) FROM customers WHERE last_name = ?"
    return conn.execute(sql, (last_name,)).fetchone()[0]

print(count_customers_unsafe("Soto"), count_customers_safe("Soto"))
sneaky = "nobody' OR '1'='1"
print(count_customers_unsafe(sneaky), count_customers_safe(sneaky))
```

```
45 45
4850 0
```

Write the SQL the unsafe function built on the board: `WHERE last_name = 'nobody' OR '1'='1'`. "The quote in the input closed the string early and the rest became SQL. That is SQL injection. Had this returned names and emails, the visitor would have the customer list." Leave the rule on the board: **never f-string, .format, % or + a value into SQL**.

11 **Writing: create, insert, commit (2m).** Say: "Analysts mostly read. When you do write: placeholders again, `executemany` for a list, and `commit` to make it permanent."

```
conn.execute("""
    CREATE TABLE store_targets (
        store          TEXT NOT NULL,
        quarter        TEXT NOT NULL,
        target_revenue REAL NOT NULL,
        PRIMARY KEY (store, quarter)
    )
""")
targets = [("Boise", "2025-Q1", 720000.0), ("Bend", "2025-Q1", 505000.0)]
conn.executemany("INSERT INTO store_targets VALUES (?, ?, ?)", targets)
conn.commit()
show("SELECT * FROM store_targets")
```

```
store quarter  target_revenue
Boise 2025-Q1        720000.0
 Bend 2025-Q1        505000.0
```

12 **A transaction is all or nothing (2m).** Say: "Two inserts in one `with conn:` block. The second breaks the primary key. Predict: how many rows afterward, two or three?"

```
insert = "INSERT INTO store_targets VALUES (?, ?, ?)"
try:
    with conn:
        conn.execute(insert, ("Asheville", "2025-Q1", 390000.0))
        conn.execute(insert, ("Boise", "2025-Q1", 999999.0))    # duplicate ke
except sqlite3.IntegrityError as e:
    print("IntegrityError:", e)
show("SELECT store, quarter FROM store_targets")
```

```
IntegrityError: UNIQUE constraint failed: store_targets.store, store_targets.(
store quarter
 Bend 2025-Q1
Boise 2025-Q1
```

Two. "The Asheville row was fine, and it was rolled back anyway." One warning: for sqlite3, `with conn:` ends the transaction; it does not close the connection.

13 **SQLAlchemy and pandas: the pattern you will keep (3m).** Say: "sqlite3 speaks only to SQLite. SQLAlchemy gives one way to connect to any database, and it is what pandas prefers. Three pieces: an engine, `text()`, and `:name` parameters in a dictionary."

```
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///pinecrest_work.db")
query = text("""
    SELECT store, COUNT(*) AS returned
    FROM orders
    WHERE status = :status AND order_date >= :since
    GROUP BY store
    ORDER BY returned DESC
""")
with engine.connect() as sa_conn:
    returned = pd.read_sql(query, sa_conn,
                           params={"status": "returned", "since": "2025-01-01"
print(returned)
rows = returned.to_sql("returns_2025", engine, if_exists="replace", index=Fals
print("rows written:", rows)
```

```
       store  returned
0     Online       467
1      Boise        36
2       Bend        34
3  Flagstaff        30
4  Asheville        30
rows written: 5
```

"Database to DataFrame with `read_sql`, DataFrame to table with `to_sql`. Change the engine URL and the same code talks to PostgreSQL." Warn: `if_exists="replace"` drops the table first. The PostgreSQL comparison table is homework reading.

THREE MISTAKES TO EXPECT

- **A string where a tuple of parameters belongs.** `conn.execute("... WHERE store = ?", "Boise")` raises `ProgrammingError: Incorrect number of bindings supplied. The current statement uses 1, and there are 5 supplied.` Coach: "How many characters in Boise?" Fix: `("Boise",)` with the trailing comma. It appears in the lab's Step 3 on purpose.
- **An aggregate in WHERE** (or a selected column that is neither grouped nor aggregated). The message is `misuse of aggregate`. Coach with the running order on the board: "Point to where WHERE runs. Do the groups exist yet?" SQLite silently allows the ungrouped-column version; PostgreSQL refuses. Hold them to the rule either way.
- **The f-string out of habit.** They have built strings with f-strings for two courses, and one will do it in the lab "because the value comes from my own code". Coach: "What happens to that query when the store is called O'Malley's?" The rule has no exceptions for values. (For a column NAME, which a placeholder cannot carry, the lab shows the allowed-list check.)

GUIDED LAB (25 MIN)

Two short labs on tables the lesson does not touch. Learners open `labs/module-14/l5_1-database-connections/staff_report_starter.py` first; your references are the matching `_solution.py` files. Both labs open the database read-only, so nothing a learner types can change it.

- **Minutes 0 to 12, the staff report (`employees`, 55 rows).** TODO 1 and 2 are cursor mechanics: 10 columns; the first hire is Clara Soto, 2016-02-01; the last `fetchone()` prints `None` because the cursor is spent (ask for predictions first). TODO 3 returns the four store managers (Helga Shah 77,800 first). TODO 4: Retail 36 people and 1,841,600 of payroll, Operations 11, E-commerce 6, Executive 2. TODO 5 needs HAVING: HQ 17, Boise 10, Asheville 8, Bend 8. Walk the room at TODO 5: half the room will try WHERE first, which is the point.
- **Minutes 12 to 25, a safe stock lookup (`labs/module-14/l5_1-database-connections/safe_lookup_starter.py`, `inventory`, 1,210 rows).** An unsafe function is given. TODO 1: the placeholder version returns the same counts (Boise out of stock 65; Warehouse at 5 or fewer 39). TODO 2: the crafted location `nowhere' OR '1'='1' --` makes the unsafe function count all 1,210 rows; the safe one returns 0. Have a volunteer write the built SQL on the board and underline what the quote and the two dashes did. TODO 3 is the one-item tuple (the message says 8 because `TNT-1001` has eight characters). TODO 4, the allowed list for `sort_by`, is for fast finishers.
- **Debrief (last 2 minutes).** "In `low_stock` there IS an f-string in the SQL. Why is that one acceptable?" (It inserts a column name checked against a fixed list; values still travel through `params`.)
- Goes home: `labs/module-14/l5_1-database-connections/save_summary_starter.py` (SQLAlchemy, `to_sql`, a rollback, on a copy in the output folder).

START THE LESSON PROJECT (10 MIN)

Open the lesson page at "Project: The Store Scorecard". The brief: one parameterized report function for any store and any year, saved back as a table and checked against pandas. In the room, everyone does **Step 1** (a fresh copy called `scorecard.db` and a SQLAlchemy engine; the six table names print) and **Step 2** (the headline query with `:store`, `:start`, `:end` for Boise in 2025). On track: one row with `1320` orders, `672` customers and revenue `240660.6`. Ask why the year is written as `>= :start AND < :end`. Steps 3 to 6 are homework.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q1 to Q6. Good cold calls: Q3 (the bindings error) and Q6 (predict the rollback).

Answers: **Q1** C) `FROM`, because the engine has to pick the table before it can filter, group or select; **Q2** `(3, 2, 6)`; **Q3** A) Pass a one-item tuple: `("Boise",)`; **Q4** B) `AVG(order_total) > 190`; **Q5** D) `conn.execute("SELECT * FROM orders WHERE store = ?", (store,))`; **Q6** `0`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners new to SQL type less and read more: let them run the demo cells from the lesson page and spend the lab on the staff report only, with the SQL-to-pandas table open. Give them the first line of each query (`SELECT ... FROM employees`). **Stretch:** `safe_lookup` TODO 4, then `save_summary` in the room. Learners who know SQL from work: have them explain to a neighbor why `with conn:` did not keep the Asheville row, then try the lesson's bonus (`describe_database(engine)` with `inspect`).

> **BREAK (1:25 TO 1:35)**
> Leave the SQL-to-pandas table on the projector. Over the break: uncover the four numbers on the board, and have the short Checkpoint 1 conversations with anyone who had a row at Beginning.

5.2 Complex Queries

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Choose between inner, left, right, full outer, cross and self joins, and predict how many rows each returns when keys do not all match
- Use subqueries and common table expressions (CTEs) to break a hard question into named steps
- Walk a hierarchy of any depth with a recursive CTE
- Build categories and side-by-side counts with CASE, and combine results with UNION, INTERSECT and EXCEPT
- Explain how NULL behaves in comparisons, and avoid the three bugs it causes most often

TIMING INSIDE THE BLOCK (75 MINUTES)

10m	Frame and mental model
30m	Live demo, steps 1 to 12
25m	Guided lab: revenue by brand, then stock against sales
10m	Start the Returns Investigation: Step 1

MENTAL MODEL TO SAY

"Picture a stack of order cards and a stack of customer cards. A join goes through the order cards one at a time, finds the customer card with the same customer_id, and staples the two together into one wider card. Every kind of join is the same stapling. They differ only in what happens to a card that finds no partner: throw it away, or keep it with a blank where the partner would be."

Then uncover the four numbers on the board: **739** orders belong to 150 closed accounts that are not in `customers` · **1,058** customers have never ordered · **609** order lines use 6 discontinued SKUs that are not in `products` · **8** products have never sold. Ask learners to look at their paper sketch: which side of each link is "one" and which is "many"?

Say what is coming: "Three results today will be wrong with no error message. I will make each mistake on purpose so that you recognize it when it is yours." (An inner join that drops rows, a sum after a fan-out, a comparison with NULL.)

LIVE DEMO (YOU TYPE, THEY FOLLOW): JOINS WITHOUT LOSING OR INVENTING ROWS

Continue in the same notebook under a new heading, with a NEW connection (step 1 replaces `conn` and `show`). Twelve cells; the marked minutes add up to 30. Before every join, ask the room for the row count they expect.

1. **A read-only connection (2m).** Say: "This lesson only reads. A second way to protect the file, besides a copy: open it read-only and let the engine refuse."

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("file:data/pinecrest.db?mode=ro", uri=True)

def show(sql, params=()):
    """Run a query and print the result as a table."""
    print(pd.read_sql(sql, conn, params=params).to_string(index=False))

conn.execute("DELETE FROM orders")
```

```
OperationalError: attempt to write a readonly database
```

"Exactly what you want from a connection you only mean to read with." A bonus: a typo in the file name is now an error instead of a new empty database.

2. **INNER JOIN: only the matches (3m).** Say: "A join goes in FROM: the first table, then JOIN the second ON the condition that pairs the rows. Short aliases, and every column wears its alias." Before the second query: "20,302 orders. How many rows come through?"

```
show("""
    SELECT o.order_id, o.order_total, c.first_name, c.last_name, c.city
    FROM orders AS o
    JOIN customers AS c ON c.customer_id = o.customer_id
    ORDER BY o.order_id
    LIMIT 3
""")
show("""
    SELECT (SELECT COUNT(*) FROM orders) AS orders_in_table,
           COUNT(*)                      AS rows_after_inner_join
    FROM orders AS o
    JOIN customers AS c ON c.customer_id = o.customer_id
""")
```

```
 order_id  order_total first_name last_name     city
   100001        434.9     Xavier  Madrigal  Phoenix
   100002        278.9       Aiko    Larkin     Bend
   100003         75.9     Kareem      Pike Missoula
 orders_in_table  rows_after_inner_join
           20302                  19563
```

"739 orders vanished: the closed accounts. No error, no warning." The habit: count after every join, the way you checked `.shape` after a merge.

3 **LEFT JOIN keeps the unmatched (3m).** Say: "LEFT JOIN keeps every row of the table named first. Where there is no partner, the other side is NULL."

```
show("""
    SELECT COUNT(*)                  AS all_rows,
           COUNT(c.customer_id)      AS with_customer,
           SUM(c.customer_id IS NULL) AS without_customer
    FROM orders AS o
    LEFT JOIN customers AS c ON c.customer_id = o.customer_id
""")
```

```
 all_rows  with_customer  without_customer
    20302          19563               739
```

4 **The anti-join: which ones are missing? (2m).** Say: "Start FROM the table you care about, LEFT JOIN the other, keep the rows with nothing on the other side."

```
show("""
    SELECT c.loyalty_tier, COUNT(*) AS never_ordered
    FROM customers AS c
    LEFT JOIN orders AS o ON o.customer_id = c.customer_id
    WHERE o.order_id IS NULL
    GROUP BY c.loyalty_tier
""")
```

```
loyalty_tier  never_ordered
       Basic           1058
```

1,058, all Basic (higher tiers are earned by spending). Tick the second board number.

5 **Three tables: choose the join one link at a time (3m).** Say: "Revenue by category: orders to order_items to products. For each link ask: if a row has no partner here, do I still want it?"

```
show("""
    SELECT p.category,
           COUNT(*)                                  AS lines,
           ROUND(SUM(i.quantity * i.unit_price), 2) AS item_revenue
    FROM orders AS o
    JOIN order_items AS i ON i.order_id = o.order_id
    LEFT JOIN products AS p ON p.sku = i.sku
    WHERE o.status = 'completed'
    GROUP BY p.category
    ORDER BY item_revenue DESC
""")
```

```
    category  lines  item_revenue
 Accessories  11080     567699.20
    Sleeping   3386     516302.45
       Tents   2110     447114.75
     Apparel   4554     435424.65
    Footwear   4142     391188.90
       Packs   3103     387082.35
Camp Kitchen   7298     367311.60
    Climbing   3157     272994.05
        Snow   2391     216940.15
         NaN    567      34228.25
```

The last row has no category: 567 lines, $34,228.25 on discontinued SKUs. "Real revenue that an inner join would have silently dropped." The lab names it with `COALESCE`.

6 **The fan-out trap (3m).** Say: "Total completed revenue, but I have joined the lines first."

```
show("""
    SELECT ROUND(SUM(o.order_total), 2) AS after_join,
           COUNT(*)                     AS rows_summed,
           COUNT(DISTINCT o.order_id)   AS orders
    FROM orders AS o
    JOIN order_items AS i ON i.order_id = o.order_id
    WHERE o.status = 'completed'
""")
show("""
    SELECT ROUND(SUM(order_total), 2) AS true_revenue
    FROM orders
    WHERE status = 'completed'
""")
```

```
 after_join  rows_summed  orders
10073626.41        41788   18762
 true_revenue
   3624306.27
```

"$10.3 million instead of $3.7 million, and it looks plausible. Each order's total was added once per line." The rule for the board: **sum a column at the level where it lives**. Connect to Session 13: the survey merged with orders went from 1,800 rows to 9,230. Same trap.

7 **A table joined to itself (2m).** Say: "`manager_id` holds the `employee_id` of another row in the same table. Two aliases, two roles."

```
show("""
    SELECT e.first_name || ' ' || e.last_name AS employee,
           e.title,
           m.first_name || ' ' || m.last_name AS manager
    FROM employees AS e
    LEFT JOIN employees AS m ON m.employee_id = e.manager_id
    ORDER BY e.employee_id
    LIMIT 5
""")
```

```
     employee                    title      manager
   Clara Soto Chief Executive Officer          NaN
 Tamar Kimura            VP of Retail   Clara Soto
  Arjun Silva       VP of E-commerce   Clara Soto
Elise Delgado       VP of Operations   Clara Soto
 Elena Mistry  Regional Manager, West Tamar Kimura
```

"LEFT JOIN, so the CEO is not dropped. A self join reaches exactly one level up. Hold that thought."

8 **A query used as a table (2m).** Say: "How many orders does a typical buyer place? Count per customer first, then summarize the counts."

```
show("""
    SELECT COUNT(*)               AS buyers,
           ROUND(AVG(n_orders), 2) AS avg_orders,
           MAX(n_orders)           AS most_orders
    FROM (
        SELECT customer_id, COUNT(*) AS n_orders
        FROM orders
        GROUP BY customer_id
    ) AS per_customer
""")
```

```
 buyers  avg_orders  most_orders
   3942        5.15           41
```

9 **CTEs: name the steps (3m).** Say: "Nested queries read inside out. `WITH name AS (...)` gives a step a name. Does the loyalty rule hold: Silver from $600, Gold from $1,500?"

```
show("""
    WITH spend AS (
        SELECT customer_id, COUNT(*) AS orders, SUM(order_total) AS total_spen
        FROM orders
        WHERE status = 'completed'
        GROUP BY customer_id
    )
    SELECT c.loyalty_tier,
           COUNT(*)                    AS customers,
           ROUND(MIN(s.total_spent), 2) AS lowest_spend,
           ROUND(MAX(s.total_spent), 2) AS highest_spend
    FROM spend AS s
    JOIN customers AS c ON c.customer_id = s.customer_id
    GROUP BY c.loyalty_tier
    ORDER BY lowest_spend
""")
```

```
loyalty_tier  customers  lowest_spend  highest_spend
       Basic       1710          7.50         599.91
      Silver       1294        600.43        1499.56
        Gold        734       1502.00        7945.54
```

"The rule holds exactly. And spend has ONE row per customer before it is joined to anything: that is the cure for fan-out."

10 **A recursive CTE walks the whole tree (3m).** Say: "How many levels does the company have? A recursive CTE is SQL's loop: an anchor, UNION ALL, and a part that joins the table to the rows found last round."

```
ORG_CTE = """
    WITH RECURSIVE org (employee_id, name, level) AS (
        SELECT employee_id, first_name || ' ' || last_name, 1
        FROM employees
        WHERE manager_id IS NULL                  -- anchor: top of the tree
        UNION ALL
        SELECT e.employee_id, e.first_name || ' ' || e.last_name, org.level +
        FROM employees AS e
        JOIN org ON e.manager_id = org.employee_id   -- reports of the last ro
    )
"""
show(ORG_CTE + "SELECT level, COUNT(*) AS people FROM org GROUP BY level")
```

```
 level  people
     1       1
     2       4
     3       7
     4      14
     5       8
     6      21
```

Trace two rounds aloud: the anchor returns Clara Soto at level 1; the recursive part finds the four people who report to her; it runs again on those four. "It stops when a round finds nobody." The lesson's `path` column (an org chart) is homework reading.

11 **CASE inside an aggregate (2m).** Say: "CASE is SQL's if / elif / else. Put it inside SUM and you count only the rows that meet a condition."

```
show("""
    SELECT channel,
           COUNT(*)                                          AS orders,
           SUM(CASE WHEN status = 'returned' THEN 1 ELSE 0 END) AS returned,
           ROUND(100.0 * SUM(CASE WHEN status = 'returned' THEN 1 ELSE 0 END)
                 / COUNT(*), 1)                              AS return_pct
    FROM orders
    GROUP BY channel
""")
show("SELECT 890 / 12854 AS int_division, 100.0 * 890 / 12854 AS correct")
```

```
channel  orders  returned  return_pct
 online   12854       890         6.9
  store    7448       253         3.4
 int_division  correct
            0 6.923915
```

"Why `100.0` and not `100`? An integer divided by an integer is an integer." This table is Step 1 of the project.

12 **NULL means unknown (2m).** Say: "NULL is not zero and not an empty string. It is 'unknown', and any comparison with unknown is unknown. WHERE keeps a row only when the condition is TRUE."

```
show("""
    SELECT COUNT(*)                     AS all_orders,
           SUM(promo_code = 'TRAIL15')  AS trail15,
           SUM(promo_code <> 'TRAIL15') AS not_trail15,
           SUM(promo_code IS NULL)      AS no_code
    FROM orders
""")
show("""
    SELECT (SELECT COUNT(*) FROM employees
            WHERE employee_id NOT IN (SELECT manager_id FROM employees)) AS no
           (SELECT COUNT(*) FROM employees AS e
            WHERE NOT EXISTS (SELECT 1 FROM employees AS x
                WHERE x.manager_id = e.employee_id))            AS not_exists
""")
```

```
 all_orders  trail15  not_trail15  no_code
      20302      359         1789    18154
 not_in  not_exists
      0          32
```

"A condition and its opposite do not add up: 359 + 1,789 is not 20,302." And `NOT IN` with one NULL in the list returns nothing: zero instead of 32. "Ask of every filter on a column that can be empty: what should happen to the rows with no value?"

THREE MISTAKES TO EXPECT

- **Not counting after a join.** The query runs, a table appears, and nobody asks how many rows it should have. Coach before they run it: "Write the number you expect in a comment. Now run it." The lab's first step forces it (45,490 lines: 44,881 after the inner join, 45,490 after the left join).
- **Summing after a fan-out.** In `stock_vs_sales` both sides are "many" and both sums inflate (154,470 units sold instead of 30,894; 3,693,132 on hand instead of 22,815). Coach: "What is one row after this join? At what level does `quantity` live?" The cure: aggregate each side to the grain you need first (a CTE), then join.
- **A filter on the right-hand table in WHERE, which undoes the LEFT JOIN, and its cousin `= NULL`.** Listing every customer with their December 2025 orders, `WHERE o.order_date >= '2025-12-01'` after the left join keeps only 898 customers instead of 4,850, because the NULL order rows fail the

comparison. Coach: "Which table is that condition about? Conditions on the right table of a left join go in the ON clause." And `= NULL` never matches: `IS NULL`.

GUIDED LAB (25 MIN)

Learners open `labs/module-14/l5_2-complex-queries/join_row_counts_starter.py`. The buying team wants item revenue by brand for completed orders: three tables. The starter has a line for three row-count predictions; do not let anyone skip it.

- **Minutes 0 to 5, predictions and TODO 1.** 45,490 lines; inner join to products 44,881; left join 45,490. Anyone who predicted correctly used the board number 609.
- **Minutes 5 to 12, TODO 2 and 3.** The tempting inner join reports 3,602,058.10; the left join with `COALESCE(p.brand, 'Discontinued')` shows nine rows that add up to 3,636,286.35. "How much did the inner join lose?" 34,228.25, on 567 lines. Top brand: Wrenfield, 515,811.85.
- **Minutes 12 to 15, TODO 4** (the anti-join): six SKUs, led by APL-1033 at 11,432.85; all last sold in September 2024. TODO 5 (`NOT EXISTS`) is for fast finishers: Granite Fork has 3 unsold products.
- **Minutes 15 to 25, `labs/module-14/l5_2-complex-queries/stock_vs_sales_starter.py`, Steps 1 and 2 only.** Everyone computes the two true totals (30,894 units sold in 2025; 22,815 on hand), then writes the tempting three-table join and sees 154,470 and 3,693,132. Stop the room: "By what factor is units sold off, and why exactly that factor?" (Five: each SKU has five inventory rows.) The CTE fix, Steps 3 to 5, is homework; show the shape of the answer from the solution file for thirty seconds.
- Goes home: the rest of `stock_vs_sales`, and `labs/module-14/l5_2-complex-queries/org_payroll_starter.py` (a recursive CTE that carries each branch's head down the tree, plus three NULL checks) as the stretch.

START THE LESSON PROJECT (10 MIN)

Open the lesson page at "Project: The Returns Investigation". Learners who finished the Lesson 4.7 project already know the ending (online footwear): say so, and say what is new. "Last week you found it with merges. This week you pin it down in SQL across four tables, and prove both routes agree." In the room: **Step 1** (read-only connection, return rate by channel with conditional aggregation: online 6.9%, store 3.4%) and a read-through of **Step 2**'s plan: why the CTE uses `DISTINCT` and why the link to products is a LEFT JOIN with COALESCE. Steps 2 to 6 are homework.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q7 to Q12. Good cold calls: Q7 (count the rows) and Q10 (where did 18,154 orders go?).

Answers: **Q7** B) 19,563; **Q8** `350.0`; **Q9** A) `SELECT c.* FROM customers AS c LEFT JOIN orders AS o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL`; **Q10** D) For the 18,154 orders with no code the comparison with NULL is unknown, and WHERE keeps only rows where the condition is true; **Q11** `(3, 3.5, 25)`; **Q12** C) A round of the recursive part finds no new rows. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pair anyone new to SQL with someone who is not, and let them stop after TODO 3 of `join_row_counts`; that lab alone covers the core skill. In the demo they may copy the recursive CTE from the lesson page. **Stretch:** finish `stock_vs_sales` in the room, then `org_payroll`. Ask: "Your CTE joins `products` to `on_hand` with an inner join and to `sold` with a left join. Why the difference?" (Every product has inventory rows; a product that never sold has no `sold` row and must still appear, with 0.)

CLOSING

EXIT TICKET (3 QUESTIONS; ANSWERS)

1. In one sentence each: what does WHERE filter, and what does HAVING filter? **Answer:** WHERE filters rows before they are grouped (so it cannot mention an aggregate). HAVING filters groups after GROUP BY (so it can).
2. Why do values go into SQL through placeholders and never through an f-string? **Answer:** With a placeholder the SQL text and the value travel separately, so a value can never be read as part of the command. An f-string pastes the value into the command: a quote in the input can change the query (SQL injection), and an honest name like O'Brien breaks it.
3. `orders` has 20,302 rows and 739 have no matching customer. How many rows from `orders JOIN customers`, and how many from `orders LEFT JOIN customers`? **Answer:** 19,563 and 20,302. The inner join drops the unmatched rows silently; the left join keeps them with NULL on the customer side.

HOMEWORK (ABOUT 3 HOURS)

- **Lesson 5.1:** finish the Store Scorecard (Steps 3 to 6), Exercises 1 to 3 (4 is a stretch), the quiz and the journal. Read "Where PostgreSQL Differs" and "Common Mistakes".
- **Lesson 5.2:** the Returns Investigation, Steps 2 to 6 (about 40 minutes), Exercises 1 to 4 (5 is a stretch), the quiz and the journal. Read the sections on set operations and on `RIGHT` / `FULL OUTER JOIN`.
- **Kit labs:** `labs/module-14/l5_1-database-connections/save_summary_starter.py` and Steps 3 to 5 of `labs/module-14/l5_2-complex-queries/stock_vs_sales_starter.py`. Stretch: `labs/module-14/l5_2-complex-queries/org_payroll_starter.py`.
- **Keep:** your `show()` helper and the `ORG_CTE` string. Session 15 uses both.
- **Workbook:** self-check Q1 to Q12.

BRING LIST FOR SESSION 15 (LESSONS 5.3 AND 5.4: SQL WINDOWS AND QUERY SPEED)

☐ `data/pinecrest.db`, untouched (Lesson 5.4 creates indexes, so it works on a copy; about 10 MB of free disk space). ☐ SQLite 3.25 or later for window functions: `python -c "import sqlite3; print(sqlite3.sqlite_version)"`. ☐ Your notebook from today with `show()` and `ORG_CTE`. ☐ A glance back at Lesson 2.8 (pandas `rolling`, `shift`, `cumsum`, `rank`): Lesson 5.3 is the SQL version of that lesson. ☐ Nothing to install, no server, no account.

IF YOU RUN LONG OR SHORT

Long. Cut in this order: (1) 5.1 demo step 3 (the cursor walk): say it, the lab practices it. (2) 5.1 step 6 (top N). (3) 5.1 step 11 becomes "here is a table I made earlier": keep the rollback in step 12. (4) 5.2 steps 7 and 8 (self join, query used as a table): say them. (5) The second lab of each block (`safe_lookup` TODO 3 and 4; `stock_vs_sales`) becomes homework. (6) "Start the project" shrinks to five minutes. Never cut: the injection demonstration, the 739 missing rows, the fan-out trap.

Short (a group that has used SQL before). Run `save_summary` in the room after the first lab block, and all of `stock_vs_sales` after the second. Add the lesson's set-operations cell (2024 and 2025 buyers: 3,942 either year, 2,488 both, 431 lost) to the 5.2 demo, and give `org_payroll` to pairs. Experienced learners can rewrite one demo query for PostgreSQL using the lesson's comparison table.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 15: SQL Windows and Query Speed

How to teach window functions (a row that can see its neighbors without giving up its place) and query optimization (reading the engine's plan and changing it with evidence), with every result checked by a second route.

`2 lessons` `3 hrs session` `Lessons 5.3–5.4` `Session 15 of 21`

HOW TO USE THIS GUIDE

- Two SQL lessons that look advanced and are not: window functions are the pandas `transform`, `rank`, `shift`, `cumsum` and `rolling` that learners already know, and query optimization is careful measuring. Say both of those things early.
- Same rhythm as every session: **Frame** → **Live demo** (you type, they follow) → **Guided lab** (new practice from the kit) → **Start the project**. The lesson projects are finished at home.
- Every code block below was run on the course data. Times in the 5.4 demo come from the author's machine: they are printed so you know the size of the effect, and yours will differ. Plans and row counts will not.
- Labs live in `labs/module-15/`: starters for learners, solutions for you. Teal blocks are hands-on, amber blocks are watch-outs.
- Keep `sql_window_functions.html` and `sql_query_optimization.html` open for the train picture, the frame table, the B-tree diagram and the seven-day moving-average chart (shown, not typed).

> **THE #1 RISK OF SESSION 15: ANSWERS THAT ARE WRONG IN ONE ROW, AND SPEED CLAIMS FROM ONE STOPWATCH READING**
>
> A missing PARTITION BY or the default frame with ties produces a table that is right in almost every row. Teach the three questions (partition, order, frame) until the room says them with you. In 5.4 the risk is the opposite kind: learners will believe a single timing. The kit never asserts a time; it asserts plans and identical results. Hold the room to the same standard.

BEFORE THE SESSION

- ☐ From the `instructor-kit/` folder run `python3 labs/check_labs.py module-15`. Expect `10 passed, 0 failed`. (The 5.4 labs copy the database to a temporary folder; the checker fails any lab that touches `data/`.)

- ☐ Copy the **starter files only** to the shared folder: `labs/module-15/l5_3-sql-window-functions/*_starter.py` and `labs/module-15/l5_4-query-optimization/*_starter.py`. Keep every `*_solution.py` on your machine.
- ☐ Files: `data/pinecrest.db` only, untouched (check that it has no `idx_` indexes: `SELECT name FROM sqlite_master WHERE type = 'index'` should list only `sqlite_autoindex_...` entries). About 10 MB of free disk space per learner for working copies. No server, no internet, no accounts.
- ☐ SQLite 3.25 or later on every machine for window functions: `python -c "import sqlite3; print(sqlite3.sqlite_version)"`. The course Python bundles 3.53. Learners flagged in Session 14 with an old version need a newer Python before today.
- ☐ Projector: two empty notebooks, `session15_sql.ipynb` and `session15_tuning.ipynb`, in the folder that holds `data/`. Delete any old `pinecrest_tuning.db` there. Close other heavy programs: timing demos are noisier on a busy laptop.
- ☐ Board: a train car labeled PARTITION BY / ORDER BY / ROWS BETWEEN on the left; on the right, "Read the plan, form a guess, change one thing, measure."
- ☐ Skim the Session 14 homework: the Store Scorecard and the Returns Investigation (both should report matching SQL and pandas numbers), and who finished `stock_vs_sales`.

KNOWN ENVIRONMENT TRAPS

Plan wording. SQLite 3.36 and later say `SCAN orders`; older versions say `SCAN TABLE orders`. Same meaning. **A stale plan** after `DROP INDEX`: Python's sqlite3 caches prepared statements by their text; connect with `cached_statements=0` (as the lesson, the demo and the labs do) or open a new connection. `index ... already exists` when a cell runs twice: use `CREATE INDEX IF NOT EXISTS`. **Windows and OneDrive folders**: a working copy inside a synced folder can be locked by the sync client (`database is locked`, or `PermissionError` on the copy). Work in a local folder. **Timings** on laptops in power-saving mode or on shared machines jump around; best-of-N absorbs most of it. Never let a learner conclude from a single number. **The shipped database changed**: if a learner ran `CREATE INDEX` on `data/pinecrest.db` itself, later lessons' plans will differ. Fix: unzip `pinecrest-data.zip` again (or `python data/make_data.py`).

SESSION 15 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 14 (queries, joins, NULL), no notes. Homework check: the Scorecard and the Returns Investigation.
0:10	10	**5.3** Frame and mental model	A seat on the train: partition, order, frame. GROUP BY collapses rows; a window keeps them.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:20	30	**5.3** Live demo	`OVER (PARTITION BY ...)`, a share of the total, the three ranking functions, top N per group, `LAG`, running totals and year to date, moving averages and frames, the tie trap.
0:50	25	**5.3** Guided lab	`labs/module-15/l5_3-sql-window-functions/staff_rankings_starter.py`, then `silver_milestone_starter.py`.
1:15	10	**5.3** Start the lesson project	The Store Performance Table: Step 1 in the room (120 rows: 5 stores × 24 months), Step 2 read together.
1:25	10	Break	Leave the train picture on the projector: partition, order, frame.
1:35	10	**5.4** Frame and mental model	The book with no index. SCAN against SEARCH. Measure, then change one thing.
1:45	30	**5.4** Live demo	A copy with `cached_statements=0`, `EXPLAIN QUERY PLAN`, one index and the stopwatch, join plans, a composite index and the leftmost prefix, a function that defeats an index, `ANALYZE`, asking for less.
2:15	25	**5.4** Guided lab	`labs/module-15/l5_4-query-optimization/customer_lookup_plans_starter.py`, then `n_plus_one_starter.py`.
2:40	10	**5.4** Start the lesson project	The Tuning Report: Steps 1 and 2 in the room (the workload and its baseline plans).
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 16.
Total	180		Ends at 3:00.

In the room: both mental models; eleven window functions from a group average to the tie trap; nine cells from the first plan to "ask for less"; four labs (two per lesson) that each end in a check by a second route; the first step of each project.

Moves home, exactly: Lesson 5.3: every "The pandas Way" block (the phrase book in the workbook summarizes them); the seven-day moving-average chart; NTILE, FIRST_VALUE and the named WINDOW clause (the Silver lab uses a named window, so learners see one); Common Mistakes 3 (LAST_VALUE) and 4; Store Performance Table Steps 2 to 5; Exercises 1 to 3 (4 is a stretch); the quiz and the journal. **Lesson 5.4**: covering indexes in detail; wildcard patterns and low-selectivity columns ("When an Index Does Not Help" cases 2 to 4); `sqlite_stat1`; "What Indexes Cost" (file size and insert speed); "Where PostgreSQL Differs"; Common Mistakes 1 to 5; Tuning Report Steps 3 to 6; Exercises 1 to 3 (4 is a stretch); the quiz and the journal. **Kit labs**: `weekly_trend`, the Silver lab's TODO 4, and `n_plus_one` if it was not reached. About three hours.

Protect: top N per group (5.3 step 5), the frame cells and the tie trap (steps 9 and 10), the Silver lab's check against 2,028; in 5.4 reading plans (step 2), one index measured properly (step 3) and the function-on-a-column case (step 6). **Cut first:** see "If You Run Long or Short".

WARM-UP

0:00 Warm-up: Retrieval from Session 14

10 min

TIMING

6m	Five retrieval questions on paper, no notes, then answers
3m	Homework check: did SQL and pandas agree?
1m	Roadmap

FIVE RETRIEVAL QUESTIONS (LESSONS 5.1 AND 5.2)

1. In one sentence each: what does WHERE filter, and what does HAVING filter? **Answer:** WHERE filters rows before grouping; HAVING filters groups after GROUP BY. (Today adds a third stage: window functions run after both, in the SELECT step.)

2. `orders` has 20,302 rows and 739 have no customer row. How many rows from `JOIN customers`, and from `LEFT JOIN customers`? **Answer:** 19,563 and 20,302.

3. Total revenue came out as $10.3 million instead of $3.7 million after joining `order_items`. What happened, and what is the rule? **Answer:** Fan-out: each order's total was repeated once per line, then summed. Sum a column at the level where it lives; aggregate first (a CTE), then join.

4. Why does `WHERE employee_id NOT IN (SELECT manager_id FROM employees)` return 0 rows? **Answer:** The list contains a NULL (the CEO's `manager_id`), and a comparison with NULL is unknown, so NOT IN is never true. Use `NOT EXISTS` or filter the NULLs out of the list.

5. A value travels into SQL through a placeholder. What can a placeholder NOT stand for, and what do you do then? **Answer:** A table name, a column name or a keyword. Check the name against a fixed list of allowed names in Python, and only then put it in the query text.

HOMEWORK CHECK AND ROADMAP

Ask two learners what their project checks printed: the Store Scorecard's revenue match (True for five stores) and the Returns Investigation's four rows adding up to 20,302. "Two routes, one answer: today every lab ends that way." Ask who finished `stock_vs_sales` and what the factor was (five: each SKU has five inventory rows).

Roadmap: "Before the break, a row learns to see its neighbors: rankings, running totals, month over month, without losing the rows. After the break, the same queries, fast: how to ask the engine what it is doing and change it with evidence."

WATCH-OUTS

- Question 1 sets up the first half: if the room cannot say where WHERE and HAVING run, redraw the running order now; window functions go in at the SELECT step.
- A learner with SQLite below 3.25 cannot run 5.3. Pair them with a neighbor for the first half; they do the 5.4 labs on their own machine.

5.3 SQL Window Functions

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what OVER, PARTITION BY and the window's ORDER BY each control, and how a window function differs from GROUP BY
- Number and rank rows with ROW_NUMBER, RANK and DENSE_RANK, and select the top N rows of every group
- Compare a row with earlier or later rows using LAG and LEAD
- Compute running totals and moving averages with an explicit window frame, and say what the default frame does
- Translate each of these between SQL and pandas (transform, rank, shift, cumsum and expanding, rolling with min_periods)

TIMING INSIDE THE BLOCK (75 MINUTES)

10m	Frame and mental model
30m	Live demo, steps 1 to 11
25m	Guided lab: pay within each location, then the Silver milestone
10m	Start the Store Performance Table: Step 1

MENTAL MODEL TO SAY

"GROUP BY is the conductor's report: 'car 3 has 48 passengers'. The passengers have become a number. A window function leaves you in your seat and lets you look around. Through the window you can see the rest of your car, your **partition**; you know who sits in front of you and behind you, the **order**; and you can choose to look only at the two seats ahead, the **frame**. You get your answer, and you are still a passenger with a seat."

Draw a train car on the board with three labels: PARTITION BY (which car), ORDER BY (seat order), ROWS BETWEEN (how far you look). Leave it up through the break.

Bridge to what they own: "You have done all of this in pandas. `transform` from Lesson 2.4; `rank`, `shift`, `cumsum`, `rolling` from Lesson 2.8. Today is the SQL spelling." The workbook has the two-column phrase book; put it on the projector after the demo.

LIVE DEMO (YOU TYPE, THEY FOLLOW): ELEVEN WINDOWS ON PINECREST'S ORDERS

A notebook `session15_sql.ipynb` in the folder that holds `data/`. Read-only connection. The marked minutes add up to 30. For each window, say the three questions aloud before you type it: partition, order, frame.

1. **Setup and a version check (1m).** Say: "Read-only again, the same `show()` helper. Window functions need SQLite 3.25 or later; this line checks."

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("file:data/pinecrest.db?mode=ro", uri=True)

def show(sql, params=()):
    """Run a query and print the result as a table."""
    print(pd.read_sql(sql, conn, params=params).to_string(index=False))

print(sqlite3.sqlite_version_info >= (3, 25, 0))
```

```
True
```

2. **OVER (PARTITION BY): a group value on every row (3m).** Say: "Each order next to its own store's average. Five orders in, five rows out, each carrying a number computed from thousands of other rows."

```
show("""
    SELECT order_id, store, order_total,
           ROUND(AVG(order_total) OVER (PARTITION BY store), 1) AS store_avg,
           ROUND(order_total - AVG(order_total) OVER (PARTITION BY store), 2)
               AS vs_avg
    FROM orders
    WHERE status = 'completed'
    ORDER BY order_id
    LIMIT 4
""")
```

```
 order_id     store  order_total  store_avg  vs_avg
   100001 Flagstaff        434.9      182.4  252.51
   100002 Asheville        278.9      175.1  103.80
   100003    Online         75.9      203.2 -127.30
   100004    Online        399.8      203.2  196.60
```

Write the comparison on the board: GROUP BY store gives 5 rows; OVER (PARTITION BY store) gives 18,762. pandas twin: `groupby("store")["order_total"].transform("mean")`.

3 **An empty OVER (): a share of the whole (2m).** Say: "Windows run after GROUP BY, so they can work on grouped rows. An empty OVER() is one window holding every row."

```
show("""
    WITH by_store AS (
        SELECT store, ROUND(SUM(order_total), 2) AS revenue
        FROM orders
        WHERE status = 'completed'
        GROUP BY store
    )
    SELECT store, revenue,
           ROUND(100.0 * revenue / SUM(revenue) OVER (), 1) AS pct_of_total
    FROM by_store
    ORDER BY revenue DESC
""")
```

```
    store    revenue  pct_of_total
   Online 2357984.45          65.1
    Boise  443386.85          12.2
Flagstaff  300580.35           8.3
     Bend  284575.25           7.9
Asheville  237779.37           6.6
```

4 **Three ways to rank, and what they do with ties (3m).** Say: "The two cheapest items in Tents share a price. Predict the three columns for those two rows."

```
show("""
    SELECT name, price,
           ROW_NUMBER() OVER (ORDER BY price) AS row_number,
           RANK()       OVER (ORDER BY price) AS rank,
           DENSE_RANK() OVER (ORDER BY price) AS dense_rank
    FROM products
    WHERE category = 'Tents'
    ORDER BY price, name
    LIMIT 4
""")
```

```
                           name  price  row_number  rank  dense_rank
  Cedar Notch Cairn Tent Footprint  41.95           2     1           1
 Quillback Foothill Tent Footprint  41.95           1     1           1
Quillback Alpenglow Tent Footprint  67.95           3     3           2
             Pinecrest Drift Tarp  94.95           4     4           3
```

"RANK is a race: two golds, no silver. DENSE_RANK does not skip. ROW_NUMBER picks arbitrarily among ties unless the ORDER BY breaks them": point at the two footprints, numbered in an order nothing in the query decided. pandas: `rank(method="min" / "dense" / "first")`.

5 **Top N per group: number, wrap, filter (4m).** Say: "The two best-selling products in every category. `ORDER BY ... LIMIT 2` gives two rows in total, not two per category. The recipe has three steps, and step 2 is not optional."

```
show("""
    WITH product_revenue AS (
        SELECT p.category, p.name,
               ROUND(SUM(i.quantity * i.unit_price), 2) AS revenue
        FROM order_items AS i
        JOIN orders AS o ON o.order_id = i.order_id
        JOIN products AS p ON p.sku = i.sku
        WHERE o.status = 'completed'
        GROUP BY p.sku
    ),
    ranked AS (
        SELECT *,
               ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC)
        FROM product_revenue
    )
    SELECT category, rn, name, revenue
    FROM ranked
    WHERE rn <= 2
    ORDER BY category, rn
    LIMIT 6
""")
```

```
    category  rn                                name   revenue
 Accessories   1 Cedar Notch Sawtooth First-Aid Kit 148120.20
 Accessories   2      Larkspur Ridge Mesa Multitool  47968.95
     Apparel   1   Granite Fork Cascade Rain Jacket 104981.25
     Apparel   2        Tallgrass Drift Rain Jacket  51019.70
Camp Kitchen   1 Quillback Timberline Insulated Mug  46581.65
Camp Kitchen   2   Granite Fork Meadow Water Bottle  28923.80
```

Then break it on purpose: move `rn <= 2` into the first SELECT with the window. The engine answers `misuse of window function ROW_NUMBER()`. "WHERE runs before windows exist. The CTE turns the number into an ordinary column."

6 **LAG and LEAD: the neighbors (3m).** Say: "Monthly revenue is the series for the next three steps. Store the CTE in a Python string so we never retype it."

```
MONTHLY = """
    WITH monthly AS (
        SELECT strftime('%Y-%m', order_date) AS month,
               ROUND(SUM(order_total), 2)    AS revenue
        FROM orders
        WHERE status = 'completed'
        GROUP BY month
    )
"""
show(MONTHLY + """
    SELECT month, revenue,
           LAG(revenue) OVER (ORDER BY month) AS prev_month,
           ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
                 / LAG(revenue) OVER (ORDER BY month), 1) AS pct_change
    FROM monthly
    ORDER BY month
    LIMIT 4
""")
```

```
  month   revenue  prev_month  pct_change
2024-01 106287.63         NaN         NaN
2024-02 104687.91   106287.63        -1.5
2024-03 128400.63   104687.91        22.7
2024-04 129716.87   128400.63         1.0
```

"January 2024 has nothing before it: NULL, the honest answer." pandas: `shift(1)` and `pct_change()`. One difference to say aloud: pandas shifts by position, so you must sort first; SQL takes the order from the OVER.

7 **LAG within a group: days between a customer's orders (2m).** Say: "With PARTITION BY, 'previous' means previous within the group. `julianday()` turns a date into a day count."

```
show("""
    WITH gaps AS (
        SELECT order_date,
               LAG(order_date) OVER (PARTITION BY customer_id
                                     ORDER BY order_date, order_id) AS prev_da
        FROM orders
    )
    SELECT COUNT(*)         AS orders,
           COUNT(prev_date) AS with_earlier,
           ROUND(AVG(julianday(order_date) - julianday(prev_date)), 1) AS avg_
    FROM gaps
""")
```

```
 orders  with_earlier  avg_days
  20302         16360      83.1
```

20,302 orders minus 3,942 first orders = 16,360 gaps, averaging about 83 days. "order_id breaks ties when a customer orders twice in one day, so the result is the same every run."

8 **Running total and year to date (3m).** Say: "Give an aggregate window an ORDER BY and it covers the partition from the start to the current row: a running total. Partition by year and it restarts each January."

```
show(MONTHLY + """
    SELECT month, revenue,
           ROUND(SUM(revenue) OVER (ORDER BY month
             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS running_
           ROUND(SUM(revenue) OVER (PARTITION BY substr(month, 1, 4) ORDER BY
             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS year_to_
    FROM monthly
    ORDER BY month
    LIMIT 4 OFFSET 10
""")
```

```
   month   revenue  running_total  year_to_date
2024-11 143304.56     1498118.16    1498118.16
2024-12 185181.35     1683299.51    1683299.51
2025-01 132423.89     1815723.40     132423.89
2025-02 126414.41     1942137.81     258838.30
```

"I wrote the frame out on purpose. In ten minutes you will see why." pandas: `cumsum()`, and `groupby(year).cumsum()` for year to date.

9 **Frames: a moving average, and what the first rows do (3m).** Say: "The frame is the part of the partition the function looks at: from here to here. Three months, and a column that counts how many rows the frame really held."

```
show(MONTHLY + """
    SELECT month, revenue,
           ROUND(AVG(revenue) OVER (ORDER BY month
                 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3
           COUNT(*) OVER (ORDER BY month
                 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)     AS rows_in_fram
           ROUND(AVG(revenue) OVER (ORDER BY month
                 ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING), 2) AS prior_3_avg
    FROM monthly
    ORDER BY month
    LIMIT 4
""")
```

```
  month   revenue  moving_avg_3  rows_in_frame  prior_3_avg
2024-01 106287.63     106287.63              1          NaN
2024-02 104687.91     105487.77              2    106287.63
2024-03 128400.63     113125.39              3    105487.77
2024-04 129716.87     120935.14              3    113125.39
```

Two points. (1) SQL averages whatever part of the frame exists: January's "three-month average" is one month. pandas `rolling(3)` gives NaN there unless `min_periods=1`. (2) A frame that ends at `1 PRECEDING` never sees the current row: the leak-free feature from Lesson 2.8, `shift(1).rolling(3, min_periods=1)`.

10 **The tie trap in the default frame (3m).** Say: "Bend's completed orders as a running total by date, written with the default frame and with an explicit one. Three orders share January 1."

```
show("""
    SELECT order_id, order_date, order_total,
           ROUND(SUM(order_total) OVER (ORDER BY order_date), 2) AS default_fr
           ROUND(SUM(order_total) OVER (ORDER BY order_date, order_id
             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS explicit
    FROM orders
    WHERE store = 'Bend' AND status = 'completed'
    ORDER BY order_date, order_id
    LIMIT 4
""")
```

```
 order_id order_date  order_total  default_frame  explicit_rows
   100011 2024-01-01        43.90         687.14          43.90
   100015 2024-01-01       241.97         687.14         285.87
   100017 2024-01-01       401.27         687.14         687.14
   100036 2024-01-02       211.80         898.94         898.94
```

"The default frame treats rows that tie in the window's order as arriving together, so all three January 1 rows show the same total. No error." The habit for the board: **aggregating over an ordered window? Write the ROWS frame.**

11 **Forgetting PARTITION BY with LAG (3m).** Say: "One more silent mistake. Two stores, one row per month, and a 'previous month' column with and without a partition. Look at the middle row."

```
show("""
    WITH store_month AS (
        SELECT store, strftime('%Y-%m', order_date) AS month,
               ROUND(SUM(order_total), 2) AS revenue
        FROM orders
        WHERE status = 'completed' AND store IN ('Asheville', 'Bend')
        GROUP BY store, month
    )
    SELECT store, month, revenue,
           LAG(revenue) OVER (ORDER BY store, month)             AS prev_wrong
           LAG(revenue) OVER (PARTITION BY store ORDER BY month) AS prev_right
    FROM store_month
    ORDER BY store, month
    LIMIT 3 OFFSET 23
""")
```

```
    store   month  revenue  prev_wrong  prev_right
Asheville 2025-12  8253.61     8067.69     8067.69
     Bend 2024-01  7563.67     8253.61         NaN
     Bend 2024-02  7726.66     7563.67     7563.67
```

"Bend's first month was compared with Asheville's last. The partition is the series." Close the demo with the three questions to ask of every window: **what is the partition, what is the order, does it need a frame?**

THREE MISTAKES TO EXPECT

- **Filtering on a window function in the same query.** `WHERE ROW_NUMBER() OVER (...) <= 3` raises `OperationalError: misuse of window function ROW_NUMBER()`, and giving it an alias does not help. Coach with Session 14's running order: "Where do window functions run? After WHERE. So the filter needs another SELECT around it: a CTE."
- **A missing PARTITION BY.** LAG over several series without a partition compares one series with the last row of another (demo step 11). No error, one wrong row in the middle of a correct table. Coach: "How many series are in this table? Then what is the partition?"
- **The default frame with ties, and an order that does not decide.** A running total ordered by date gives tied rows the same total; ROW_NUMBER over a date with two orders on one day numbers them

arbitrarily. Coach: "Is your ORDER BY unique? Add the key as a tiebreaker. Aggregating over an ordered window? Write the ROWS frame." The Silver lab depends on both.

GUIDED LAB (25 MIN)

Learners open `labs/module-15/l5_3-sql-window-functions/staff_rankings_starter.py` first. Read-only; your references are the `_solution.py` files. Both labs end with a check against a second route (pandas, or a number already known).

- **Minutes 0 to 10, pay within each location (`employees`).** TODO 1: Boise's ten employees next to the location average, 43,950: "ten rows; GROUP BY would give one." TODO 2 finds the tie at 39,000: RANK goes 5, 5, 7, DENSE_RANK 5, 5, 6. TODO 3 is the top-N recipe: 12 rows, two per location (HQ: Clara Soto 185,000, Tamar Kimura 140,700). Walk the room at TODO 3: someone will try the filter in the same SELECT.
- **Minutes 10 to 23, the Silver milestone (`labs/module-15/l5_3-sql-window-functions/silver_milestone_starter.py`).** The question is new: on which order does a customer's completed spend cross $600? TODO 1 builds the running total with a named WINDOW and an explicit frame (customer 10034 crosses on order 4, at 691.45). TODO 2: the first row per customer with `spent_so_far >= 600` (MIN(n) grouped): 2,028 customers, on order 3.5 on average. TODO 3 is the moment to stop the room: the customers table has exactly 2,028 Silver and Gold customers. "Two routes, one number: your window reproduced the business rule."
- **Minutes 23 to 25.** TODO 4 (gaps before the 2nd, 3rd ... order: 118 days falling to 57) goes home, with its caution: later orders come only from customers who keep buying, so the table describes who gets that far, not a change in anyone.
- Goes home: `labs/module-15/l5_3-sql-window-functions/weekly_trend_starter.py` (online orders week by week: two frames, the three sale-weekend spikes, a pandas check).

START THE LESSON PROJECT (10 MIN)

Open the lesson page at "Project: The Store Performance Table". The brief: the monthly table leadership rebuilds by hand (revenue, change from last month, year to date, trend, rank, growth over the same month last year), in one query, verified column by column against pandas. In the room: **Step 1**, the base CTE: `(120, 4)`, five stores × 24 months, first row Asheville 2024-01 with 40 orders and 7,259.22. Then read **Step 2** together and ask: "What is the partition for month-over-month change? For year to date?" (Store; store and year.) Steps 2 to 5 are homework.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q1 to Q6. Good cold calls: Q3 (why the filter fails) and Q5 (predict the tie).

Answers: **Q1** B) `orders.groupby("store")["order_total"].transform("sum")`; **Q2** `(40, 4, 4, 3)`; **Q3** A) WHERE runs before window functions are computed; compute `rn` in a CTE and filter it in the outer query; **Q4** B) `ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING`; **Q5** `[30, 30, 35]`; **Q6** D) On the first three rows: SQL

averages the rows that exist, pandas gives NaN until four rows exist. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: give learners the three questions on a card (partition? order? frame?) and the phrase-book table, and let them do `staff_rankings` only, with the pandas Step 4 as their check. In the demo they may copy steps 5 and 8 from the lesson page. **Stretch:** `weekly_trend` in the room, then the lesson's bonus (a cohort table with `FIRST_VALUE`). Ask: "In the Silver lab, why join to `customers` at all?" (Closed accounts have no tier; without the join the count could not match.)

BREAK (1:25 TO 1:35)

Leave the train picture on the board. Over the break, open the second notebook and delete any old `pinecrest_tuning.db` so step 1 copies a clean file.

5.4 Query Optimization

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Describe how SQLite turns a query into a plan, and read EXPLAIN QUERY PLAN output line by line
- Create single-column, composite and covering indexes, and show their effect on both the plan and the measured time
- Recognize the situations where an index does not help, and rewrite a filter so that an index can be used
- Measure what indexes cost in file size and write speed, and use ANALYZE to give the planner statistics
- Decide what work belongs in SQL and what belongs in pandas, and fetch only what you need

TIMING INSIDE THE BLOCK (75 MINUTES)

10m	Frame and mental model
30m	Live demo, steps 1 to 9
25m	Guided lab: plans for a lookup screen, then one query instead of 300
10m	Start the Tuning Report: Steps 1 and 2

MENTAL MODEL TO SAY

"A table is a thick reference book whose pages are in the order they were written. To find every mention of 'crampons' you have one strategy: start at page 1 and read to the end. That is a scan. An index is the alphabetical list at the back: each term, then the pages where it appears. Jump to 'crampons', turn straight to those pages. That is a search. The index is extra paper, and somebody has to update it every time a page changes."

Then the method, which matters more than any single index: "SQL says WHAT you want; the planner decides HOW. We will ask it how, change one thing, and measure again. **Read the plan, form a guess, change one thing, measure.**" Write that sentence on the board.

Set expectations about numbers: "On a 3.5 MB database everything is fast. We look at the plan first and the stopwatch second, and every time I print a time I will say: your numbers will differ."

LIVE DEMO (YOU TYPE, THEY FOLLOW): PLANS, INDEXES AND THE STOPWATCH

A second notebook, `session15_tuning.ipynb`. The demo creates indexes, so step 1 copies the database to `pinecrest_tuning.db` first. Nine cells; the marked minutes add up to 30. The times below are from the author's machine; yours will differ and that is the point of best-of-N.

1. **A copy, a plan printer, a stopwatch (3m).** Say: "This lesson creates indexes, so it works on a copy. Two helpers: `plan()` prints the engine's strategy, `timed()` runs a query many times and keeps the best.

And one detail in the connect line, which I will explain in step 7."

```
import os
import shutil
import sqlite3
import time
import pandas as pd

shutil.copy("data/pinecrest.db", "pinecrest_tuning.db")
conn = sqlite3.connect("pinecrest_tuning.db", cached_statements=0)

def plan(sql, params=()):
    """Print the query plan, one step per line."""
    for row in conn.execute("EXPLAIN QUERY PLAN " + sql, params):
        print("   ", row[3])

def timed(sql, param_list, repeat=5):
    """Best time in milliseconds to run the query once per item of param_list.
    best = float("inf")
    for _ in range(repeat):
        start = time.perf_counter()
        for params in param_list:
            conn.execute(sql, params).fetchall()
        best = min(best, time.perf_counter() - start)
    return best * 1000
```

`plan()` here prints the steps without indentation; the lesson's version indents nested steps.

2 **Reading a plan: SCAN and SEARCH (3m).** Say: "EXPLAIN QUERY PLAN returns the strategy instead of running the query. Nothing is read or changed, so it is always safe. Two verbs do nearly all the work."

```
print("by primary key")
plan("SELECT * FROM orders WHERE order_id = ?", (100500,))
print("by another column")
plan("SELECT * FROM orders WHERE customer_id = ?", (12673,))
print("filter, then sort")
plan("SELECT * FROM orders WHERE store = ? ORDER BY order_date", ("Bend",))
```

```
by primary key
    SEARCH orders USING INTEGER PRIMARY KEY (rowid=?)
by another column
    SCAN orders
filter, then sort
    SCAN orders
    USE TEMP B-TREE FOR ORDER BY
```

SCAN: read every row and test each. SEARCH: jump. TEMP B-TREE: a separate sort. "SCAN is not a dirty word: for a yearly total it is the best plan. It is a problem when you wanted a few rows and paid for all of them."

3 **One index, the plan and the stopwatch (4m).** Say: "The classic slow query here: the lines of one order. order_items has 45,490 rows and no index on order_id. Two hundred lookups, before and after."

```
ITEMS_SQL = "SELECT * FROM order_items WHERE order_id = ?"
order_ids = [(100000 + 100 * k,) for k in range(1, 201)]      # 200 different
plan(ITEMS_SQL, (100500,))
before = timed(ITEMS_SQL, order_ids)
conn.execute("CREATE INDEX idx_order_items_order_id ON order_items (order_id)"
conn.commit()
plan(ITEMS_SQL, (100500,))
after = timed(ITEMS_SQL, order_ids)
print(f"200 lookups: {before:.1f} ms before, {after:.1f} ms after")
```

```
    SCAN order_items
    SEARCH order_items USING INDEX idx_order_items_order_id (order_id=?)
200 lookups: 151.8 ms before, 1.3 ms after
```

Your timings will differ: say so every time, and read the plan line before the number. "SCAN became SEARCH ... USING INDEX. The query did not change; the planner found the index by itself."

4 **Joins: the inner loop should be a SEARCH (3m).** Say: "A join runs as a nested loop: for each row of the outer table, find the matches in the inner one. The plan lists the outer table first. One customer's history, with one index, then two."

```
HISTORY_SQL = """
    SELECT o.order_id, o.order_date, i.sku, i.quantity
    FROM orders AS o
    JOIN order_items AS i ON i.order_id = o.order_id
    WHERE o.customer_id = ?
"""
print("index on order_items(order_id) only:")
plan(HISTORY_SQL, (12673,))
conn.execute("CREATE INDEX idx_orders_customer_id ON orders (customer_id)")
conn.commit()
print("plus an index on orders(customer_id):")
plan(HISTORY_SQL, (12673,))
```

```
index on order_items(order_id) only:
    SCAN o
    SEARCH i USING INDEX idx_order_items_order_id (order_id=?)
plus an index on orders(customer_id):
    SEARCH o USING INDEX idx_orders_customer_id (customer_id=?)
    SEARCH i USING INDEX idx_order_items_order_id (order_id=?)
```

"No scans at all: search for the customer's few orders, then search for each order's lines. This is the plan you want behind a 'show me one customer' page. And notice: the planner chose the outer table, not the order in your FROM clause."

5 **Composite index and the leftmost prefix (4m).** Say: "A store manager's report filters on a store and a date range. One index on two columns, in the order a phone book uses: last name, then first name."

```
REPORT_SQL = ("SELECT order_id, order_total FROM orders "
              "WHERE store = ? AND order_date BETWEEN ? AND ?")
plan(REPORT_SQL, ("Bend", "2025-07-01", "2025-07-07"))
conn.execute("CREATE INDEX idx_orders_store_date ON orders (store, order_date)")
conn.commit()
plan(REPORT_SQL, ("Bend", "2025-07-01", "2025-07-07"))
print("date only:")
plan("SELECT order_id FROM orders WHERE order_date = ?", ("2025-07-04",))
```

```
    SCAN orders
    SEARCH orders USING INDEX idx_orders_store_date (store=? AND order_date>?
date only:
    SCAN orders USING COVERING INDEX idx_orders_store_date
```

"Equality columns first, the range or the sort last. And the index is no help for the date alone: dates are sorted only within each store, like finding everyone named Maria in a phone book."

6 **A function around the column hides it (3m).** Say: "Both of these count Bend's orders in July 2025. Same answer. Look at how much of the index each can use."

```
FUNC = ("SELECT COUNT(*) FROM orders "
        "WHERE store = ? AND strftime('%Y-%m', order_date) = ?")
RANGE = ("SELECT COUNT(*) FROM orders "
         "WHERE store = ? AND order_date >= ? AND order_date < ?")
plan(FUNC, ("Bend", "2025-07"))
plan(RANGE, ("Bend", "2025-07-01", "2025-08-01"))
print(conn.execute(FUNC, ("Bend", "2025-07")).fetchone()[0],
      conn.execute(RANGE, ("Bend", "2025-07-01", "2025-08-01")).fetchone()[0])
```

```
    SEARCH orders USING COVERING INDEX idx_orders_store_date (store=?)
    SEARCH orders USING COVERING INDEX idx_orders_store_date (store=? AND orde
83 83
```

"The index is sorted by `order_date`, not by `strftime('%Y-%m', order_date)`. Keep the bare column on one side of the comparison." A pattern that starts with `%` and a column where most rows match (status: 92% completed) are the other two cases; they are in the lesson, as reading.

7 **DROP INDEX and plan again: why cached_statements=0 (2m).** Say: "Experiments need undoing. Drop the composite index and ask for the same plan text again."

```
conn.execute("DROP INDEX idx_orders_store_date")
conn.commit()
plan(REPORT_SQL, ("Bend", "2025-07-01", "2025-08-01"))
```

```
    SCAN orders
```

"Back to SCAN, as it should be. Python's sqlite3 normally caches prepared statements by their text, and a cached one can still carry the plan it was made with while the index existed. That is why this connection was opened with `cached_statements=0`. Harmless in normal work; confusing in a lesson about plans. A new connection does the same job."

8 **ANALYZE: statistics for the planner (3m).** Say: "So far the planner has chosen with no knowledge of the data. ANALYZE measures it. Watch a date-only query on a three-column index."

```
conn.execute("CREATE INDEX idx_orders_store_date_total "
             "ON orders (store, order_date, order_total)")
conn.commit()
DATE_SQL = "SELECT COUNT(*) FROM orders WHERE order_date = ?"
plan(DATE_SQL, ("2025-07-04",))
conn.execute("ANALYZE")
conn.commit()
plan(DATE_SQL, ("2025-07-04",))
```

```
    SCAN orders USING COVERING INDEX idx_orders_store_date_total
    SEARCH orders USING COVERING INDEX idx_orders_store_date_total (ANY(store)
```

"The statistics told the planner that store has only five values, so five small searches, one per store, beat a scan: a skip-scan, `ANY(store)`." The `sqlite_stat1` table and what indexes cost in file size and insert time are reading; the project measures the size.

9 **Ask for less: aggregate where the data lives (5m).** Say: "The cheapest row is the one you never fetch. Item revenue by store, two ways."

```
def pandas_way():
    df = pd.read_sql("SELECT * FROM order_items AS i "
                     "JOIN orders AS o ON o.order_id = i.order_id", conn)
    df = df[df["status"] == "completed"]
    return (df["quantity"] * df["unit_price"]).groupby(df["store"]).sum().roun

def sql_way():
    return pd.read_sql("""
        SELECT o.store, ROUND(SUM(i.quantity * i.unit_price), 2) AS revenue
        FROM order_items AS i JOIN orders AS o ON o.order_id = i.order_id
        WHERE o.status = 'completed'
        GROUP BY o.store""", conn).set_index("store")["revenue"]

for func in (pandas_way, sql_way):
    start = time.perf_counter()
    result = func()
    print(f"{func.__name__}: {(time.perf_counter() - start) * 1000:.1f} ms")
print("same numbers:", (pandas_way() - sql_way()).abs().max() < 0.01)
```

```
pandas_way: 143.0 ms
sql_way: 14.0 ms
same numbers: True
```

"Same answer; one moved 45,490 rows into Python, the other five." Three habits for the board: **name your columns, LIMIT while exploring, one query not a loop of queries**. The second lab is about the third.

THREE MISTAKES TO EXPECT

- **Assuming the index is used.** Creating an index proves nothing; the planner may ignore it (a function on the column, a wildcard at the start, a filter that matches most rows). Coach: "Show me the plan line for the query it was meant for. Which index does it name?" In the lab, the city-only query still scans after the composite index.
- **Trusting one timing.** A single before and after can show only the cache warming up. Coach: "How many times did you run it? Which number did you keep?" Both labs time with best-of-several and print "your numbers will differ"; nothing in the kit asserts a time.
- **A stale plan after DROP INDEX, or editing the shipped database.** Re-asking the same plan text on a connection that caches statements can show the old plan; the demo and labs connect with `cached_statements=0`. And an index created on `data/pinecrest.db` itself changes the file every later lesson expects. Coach: "Which file is your connection open on?" Always a copy.

GUIDED LAB (25 MIN)

Learners open `labs/module-15/l5_4-query-optimization/customer_lookup_plans_starter.py`. Both labs copy the database into their output folder first and open the copy with `cached_statements=0`.

- **Minutes 0 to 12, plans for a lookup screen (`customers`).** TODO 1: predictions first (in a comment), then three plans, all SCAN, one with a TEMP B-TREE. TODO 2: one index on `(state, city)`: state-and-city becomes a SEARCH; city alone is still a SCAN (of the index: the leftmost-prefix rule); the newest-first query still sorts. The speedup is modest (488 of 4,850 customers live in Boise, ID: an index pays most when it skips most of the table). TODO 3: a third column, `signup_date`, removes the sort step. TODO 4: DROP and plan again: SCAN.
- **Minutes 12 to 25, one query instead of 300 (`labs/module-15/l5_4-query-optimization/n_plus_one_starter.py`).** A colleague's script runs one query per Gold customer. TODO 1 plans it (SCAN orders) and times the loop. TODO 2 writes ONE `IN (...) GROUP BY` query (one placeholder per id, so values still travel as parameters) and checks "same answers: True". TODO 3 adds `idx_orders_customer_id` and times both again. Debrief: "Either fix works here. Which one survives a server database, where each loop query also pays a network round trip?" (The single query.)
- **If the room is slow**, stop after TODO 2 of the first lab; the second goes home whole. Its point (the N+1 problem) is one sentence in the lesson, so it is worth doing.

START THE LESSON PROJECT (10 MIN)

Open the lesson page at "Project: The Tuning Report". The brief: three dashboard queries, the fewest indexes that fix them, before-and-after plans and times, proof that results are unchanged, and the cost in file size. In the room: **Step 1** (a fresh `tuning_project.db` and the workload: `{'customer history': 100, 'product sales by month': 50, 'store daily report': 100}`) and **Step 2** (the baseline: every plan contains a SCAN of a large table). Ask the room to propose the indexes from the WHERE and ON clauses before anyone opens Step 3. Steps 3 to 6 are homework.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q7 to Q12. Good cold calls: Q8 (which filter cannot seek) and Q12 (one timing each).

Answers: **Q7** C) `SEARCH orders USING COVERING INDEX idx_orders_store_date_total (store=? AND order_date>? AND order_date<?)`; **Q8** A) `WHERE order_date = '2025-07-04'`; **Q9** C) Python's sqlite3 reused its cached prepared statement, planned while the index existed; connect with `cached_statements=0` or open a new connection; **Q10** `SCAN t`; **Q11** D) Slower inserts and a larger file, because every index is a sorted copy that must be kept up to date; **Q12** C) Repeat each measurement several times, take the best, and check the plan changed from SCAN to SEARCH. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the plan-words table in the workbook is the whole vocabulary; learners who are lost read plans aloud with it ("SCAN customers: read every customer"). Let them do the first lab only, TODO 1 and 2. **Stretch:** the lesson's Exercise 4 (is a covering index worth it?) after `n_plus_one`, or measure the file-size cost of the

lab's two indexes with `os.path.getsize`. Ask: "Why does the composite index put state before city, and could city first ever be better?"

CLOSING

EXIT TICKET (3 QUESTIONS; ANSWERS)

1. `orders` has 20,302 rows and 5 stores. How many rows from `SELECT store, AVG(order_total) FROM orders GROUP BY store`, and from `SELECT store, AVG(order_total) OVER (PARTITION BY store) FROM orders`? **Answer:** 5 and 20,302. GROUP BY collapses the rows; a window function keeps every row and attaches the group value to it.
2. Give the three steps of "top N per group". **Answer:** Number the rows with ROW_NUMBER() OVER (PARTITION BY group ORDER BY measure DESC, tiebreaker); put that in a CTE; filter the CTE with WHERE rn <= N. (WHERE cannot see a window function in the same SELECT.)
3. You have just created an index. Name two things you check before saying it helped. **Answer:** Any two of: the plan for the query it was meant for now says SEARCH ... USING INDEX (or COVERING INDEX) instead of SCAN; best-of-several timings before and after; the query returns exactly the same rows; what it costs in file size and write speed.

HOMEWORK (ABOUT 3 HOURS)

- **Lesson 5.3:** finish the Store Performance Table (Steps 2 to 5; all five window columns must match pandas), Exercises 1 to 3 (4 is a stretch), the quiz and the journal. Read the "pandas Way" blocks you did not see in the room.
- **Lesson 5.4:** the Tuning Report, Steps 3 to 6, including the written recommendation; Exercises 1 to 3 (4 is a stretch); the quiz and the journal. Read "What Indexes Cost".
- **Kit labs:** `labs/module-15/l5_3-sql-window-functions/weekly_trend_starter.py`; TODO 4 of the Silver lab; `labs/module-15/l5_4-query-optimization/n_plus_one_starter.py` if it was not finished.
- **Tidy up:** delete the working copies (`pinecrest_tuning.db`, `tuning_project.db`, the labs' `output/` folders), or keep them out of any shared folder.
- **Workbook:** self-check Q1 to Q12.

BRING LIST FOR SESSION 16 (LESSONS 5.5 AND 6.1: DOCUMENTS AND HTTP)

☐ `pymongo` and `mongomock` installed (both are in `requirements.txt`): `python -c "import pymongo, mongomock; print(pymongo.__version__, mongomock.__version__)"`. No MongoDB server is needed: Lesson 5.5 runs on mongomock. ☐ `requests` installed (also in `requirements.txt`): `python -c "import requests; print(requests.__version__)"`. ☐ The local mock API tested once: from the course folder run `python data/mock_api.py`, open http://127.0.0.1:8000/health in a browser (it answers `{"status": "ok"}`), then stop it with Ctrl+C. If port 8000 is taken, `python data/mock_api.py --port 8050`. ☐ `data/web_sessions.json` in your data folder (Lesson 5.4 asked you to look at the first session in a text editor). ☐ No accounts needed: the mock API's demo key and password are public teaching values given in Session 16.

IF YOU RUN LONG OR SHORT

Long. Cut in this order: (1) 5.3 step 3 (share of the total) and step 7 (days between orders): say them. (2) 5.3 step 11 (missing PARTITION BY) becomes a slide; the lab and Q-cards cover it. (3) 5.4 step 4 (join plans): show the output from the lesson page. (4) 5.4 step 8 (ANALYZE) becomes one sentence. (5) The second lab of each block (`silver_milestone` after TODO 3; `n_plus_one`) goes home. (6) "Start the project" shrinks to the first cell. Never cut: top N per group, the frame and tie cells, reading a plan, one index measured properly.

Short. Run `weekly_trend` after the first lab block and the lesson's Exercise 4 (Pareto: 41 products make half of revenue) as a class discussion. In 5.4 add the lesson's "What Indexes Cost" cell (file size and insert time with 0, 1, 2 and 4 indexes) and have pairs propose the Tuning Report indexes before reading Step 3.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 16: Documents and HTTP

How to teach the session where the data stops being a file someone prepared. The first half finishes Module 5 with a document database: nested website sessions, queried with dictionaries and aggregation pipelines, and flattened back into pandas. The second half opens Module 6 with the conversation every web page and API is built on: one request, one response, and the habits (rules first, identify yourself, timeout, check the status) that make a program a good guest.

`2 lessons` `3 hrs session` `Lessons 5.5–6.1` `Session 16 of 21`

HOW TO USE THIS GUIDE

- Session 16 closes Module 5 and opens Module 6. Before the break learners query a document database (Lesson 5.5); after it they make their first HTTP requests (Lesson 6.1). The link between the halves is JSON: the nested documents of the first hour are what web APIs send in the second.
- Same rhythm as every session: **frame** (the mental model, said aloud) → **live demo** (you type, they follow) → **guided lab** (the kit lab) → **start the lesson project**. Finishing the projects, the lessons' exercises, quizzes and journals is homework.
- Every code block below was run for this guide: mongomock 4.3 and pandas 3.0.6 on `data/web_sessions.json`; requests 2.34.2 against the practice shop and `data/mock_api.py`. The output shown is real. Where your room will see something different (dates, the local copy of the shop), the guide says so.
- Type, do not paste. Ask for a prediction before every line whose output is a number. Two of today's most valuable moments are lines that raise **no** error and print a wrong answer (a misspelled field in 5.5, a 404 page parsed as data in 6.1).
- Labs live in `labs/module-16/`: starters for learners, solutions for you. Teal blocks are hands-on; amber blocks are watch-outs.

> **THE RISK IN SESSION 16: QUIET WRONG ANSWERS**
>
> Both lessons teach tools that are forgiving in the same dangerous way. A collection accepts a query on a field nobody has and answers 0. `requests` accepts a 404 and hands you an error page to parse. Learners who have spent five lessons being corrected by SQL's error messages will trust the zero. Say the rule early and repeat it in both halves: **"Be suspicious of zero. Look at one real document, or one real response, before you believe a count."**

BEFORE THE SESSION

- ☐ Run `python3 labs/check_labs.py module-16` from the kit folder. Expect `8 passed, 0 failed` (four starters, four solutions; the two network labs run from saved responses).
- ☐ Copy the starter files only (`*_starter.py`) plus the whole `_fixtures/` folder and `labs_paths.py` to the shared folder, keeping the folder layout. Keep every `*_solution.py` on your machine.
- ☐ **MongoDB: nothing to install and no server to start.** Lesson 5.5 runs on `mongomock`, which was installed with the course environment in Lesson 0.1. Check it on the projector machine: `python -c "import mongomock, pymongo; print(mongomock.__version__, pymongo.version)"` (this kit was run with 4.3.0 and 4.18.3). If a learner's import fails: `pip install mongomock pymongo` inside the course environment.
- ☐ **The practice shop, online.** Open `https://rays-python-datascience.netlify.app/practice/shop/` in a browser **on the venue network**, not at home. You should see the Pinecrest Outfitters catalog with a banner that says it is a practice site. Then open `https://rays-python-datascience.netlify.app/practice/robots.txt`. If either fails, use the offline copy below for the whole room.
- ☐ **The practice shop, offline fallback.** In the project folder (the one that holds `data/`), in a second terminal: `python -m zipfile -e data/pinecrest-practice-site.zip .` and then `python -m http.server 8001`. Leave it running. Everyone sets `BASE_URL = "http://127.0.0.1:8001/practice/shop/"`. (The lesson writes the first command for a zip downloaded into the project folder; from the course data folder the path is `data/pinecrest-practice-site.zip`.) Each learner serves their own copy, or you serve one and give out your machine's address if the venue network allows it.
- ☐ **The practice API.** In another terminal, from the project folder: `python data/mock_api.py`. It prints `Pinecrest mock API on http://127.0.0.1:8000 (limit: 30 requests per 10 s; Ctrl+C to stop)`. Lesson 6.1 needs no flags. Check it: `http://127.0.0.1:8000/health` in a browser shows `{"status": "ok"}`. Start it during the break, not before: a terminal left open for 80 minutes gets closed by accident.
- ☐ Browser tabs: `sql_nosql_mongodb.html`, `webscraping_requests.html`, the practice shop, and the API's `/health` page.
- ☐ Whiteboard, before learners arrive: "**Be suspicious of zero.**" and, for the second half, "**get → raise_for_status() → use the body**".
- ☐ Print the workbook chapter for Session 16, or share the HTML. Sticky notes for the exit ticket.

WHAT DIFFERS ON THE OFFLINE COPY (CHECKED FOR THIS KIT)

The kit's author unzipped `pinecrest-practice-site.zip` in an empty folder and served it with `python -m http.server`, exactly as Lesson 6.1 describes. The lesson's code runs unchanged. These printed details differ from the lesson and from this guide:

- `Content-Type` is `text/html` with **no charset**, and the `Server` header is `SimpleHTTP/0.6 Python/3.12.x` instead of `Netlify`.
- Because no charset is sent, `r.encoding` is `ISO-8859-1`, and `r.text` shows `â˜…` where the stars should be: on the front page `r.text.count("★")` is 0 until you set `r.encoding = "utf-8"` (then 20). The byte count is the same (11,783); the character count is 11,783 before the fix and 11,725 after. This is the lesson's encoding section happening for real: use it.
- A missing page returns Python's own short 404 page (335 bytes, `text/html;charset=utf-8`), not the course site's 404 page (2,514 bytes).
- `robots.txt` is at `http://127.0.0.1:8001/practice/robots.txt` and is served as `text/plain` with no charset. `RobotFileParser` gives the same answers (`True`, `False`, crawl delay 1).
- A `HEAD` request (lesson Exercise 3) returns `text/html` instead of `text/html; charset=UTF-8`.
- Both kit labs print the right answers on the offline copy (the labs set the encoding themselves); only the `type` column and the size of the 404 body differ.

ENVIRONMENT VARIABLES AND CHROME: NOT TODAY, BUT SET THEM UP NOW

Session 18 reads three environment variables, and Session 17 drives Chrome. Neither is needed today. Both fail in ways that eat class time, so check them on the projector machine this week and send learners home with the Chrome check (it is on the bring list).

The three variables (public teaching values for the practice API; Session 18):

```
# Windows PowerShell (this terminal only)
$env:PINECREST_API_KEY = "pinecrest-demo-key"
$env:PINECREST_USER = "analyst"
$env:PINECREST_PASSWORD = "trailmix"

# macOS and Linux (this terminal only)
export PINECREST_API_KEY="pinecrest-demo-key"
export PINECREST_USER="analyst"
export PINECREST_PASSWORD="trailmix"
```

Start Jupyter or VS Code **from that same terminal**, or the notebook will not see them. Lesson 6.4 also teaches the `.env` file route.

Chrome and Selenium (Session 17): Google Chrome must be installed; Selenium 4 downloads a matching driver by itself the first time, which needs internet once. The one-minute check is on the Session 17 bring list below. On a locked-down venue machine where Chrome cannot be installed or the driver download is blocked, Session 17 pairs learners up on a machine that works, and Lesson 6.3's JSON route (request the data file the page loads) needs no browser at all.

SESSION 16 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 15 (SQL windows, query plans), no notes. Homework check: who looked inside `web_sessions.json`?
0:10	10	**5.5** Frame and mental model	The ledger and the folder. One session document on the projector; SQL words mapped to MongoDB words.
0:20	25	**5.5** Live demo	mongomock, queries as dictionaries, dot notation, missing versus null, `$elemMatch`, two pipelines, two silent mistakes, back to pandas.
0:45	25	**5.5** Guided lab	`labs/module-16/l5_5-nosql-mongodb/sale_page_audit_starter.py`, then `landing_pages_starter.py` for those who finish.
1:10	10	**5.5** Start the lesson project	The Funnel Report: read the brief together; everyone runs Step 1 and sees 2400.
1:20	10	Break	Start `python data/mock_api.py` on the projector machine and open the practice shop in a browser tab.
1:30	15	**6.1** Ground rules and mental model	First 2 minutes: the six ground rules for Module 6, read aloud. Then the order counter: request, response, status, headers, body.
1:45	25	**6.1** Live demo	`robots.txt` first, a page and its envelope, a 404 that is not an exception, encoding, the practice API, timeouts, `fetch()`.
2:10	25	**6.1** Guided lab	`labs/module-16/l6_1-http-requests/category_check_starter.py`, then `api_price_check_starter.py`.
2:35	15	**6.1** Start the lesson project	The Polite Snapshot: Steps 1 and 2 in the room (`polite_get()` refuses the staff page).
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 17 (Chrome check).
Total	180		Ends at 3:00.

In the room: both mental models, both live demos, the four kit labs (two are finish-if-fast), Step 1 of the Funnel Report and Steps 1 and 2 of the Polite Snapshot. **At home (about 2 to 3 hours):** the rest of both lesson projects, each lesson's four exercises, the two five-question lesson quizzes and the journals. The lesson pages estimate 95 + 90 =

185 minutes for a solo learner; the room has 160 after the warm-up and wrap-up, so the projects are started here and finished at home by design.

Protect: in 5.5, demo steps 6, 7 and 11 (missing versus null, `$elemMatch`, the two silent mistakes) and the first lab; in 6.1, the ground rules, demo steps 6 and 7 (the 404 that is not an exception) and the first lab. **Cut first if long:** demo step 13 of 5.5 (updates: point to the lesson), then the second lab of each lesson (they become homework), then demo step 14 of 6.1 (sessions and saving: the project teaches it).

WARM-UP

0:00 Warm-up: Retrieval from Session 15 — 10 min

Five questions on the projector, answered on paper with notes closed (4 minutes), then answers aloud (3 minutes). Retrieval practice works because it is effortful: do not let it turn into a re-teach. Anything more than half the room misses goes on the board as a note for the break.

1. How does a window function differ from `GROUP BY`? **Answer:** A window function computes over a set of related rows and **keeps every row**; `GROUP BY` collapses each group to one row.
2. Three orders tie for first place. What do `ROW_NUMBER`, `RANK` and `DENSE_RANK` give the fourth row? **Answer:** `ROW_NUMBER` 4, `RANK` 4, `DENSE_RANK` 2. They differ only in how they treat ties.
3. You want the top 3 products per category. Why does `WHERE rn <= 3` fail in the same query, and what is the fix? **Answer:** `WHERE` runs before window functions are computed. Number the rows in a CTE, then filter in the outer query.
4. In `EXPLAIN QUERY PLAN`, what is the difference between `SCAN` and `SEARCH`? **Answer:** `SCAN` reads the whole table; `SEARCH` jumps to the rows it needs through a primary key or an index.
5. Name one situation in which an index does not help. **Answer:** Any one of: the column is wrapped in a function, the `LIKE` pattern starts with a wildcard, most rows match, the table is small.

Homework check (3 minutes). Lesson 5.4 asked everyone to open `data/web_sessions.json` in a text editor and decide which parts would be awkward to store in one flat table. Take two answers (the list of pages; the cart's items; the `utm` block that is sometimes missing). That is the bridge: "Today that file stops being awkward." Ask who finished the 5.4 tuning report; note names of anyone who did not start it and check in at the break.

LESSON PLANS

5.5 NoSQL with MongoDB

70 min

OBJECTIVES (FROM THE LESSON)

- Explain how a document differs from a row, and judge when a document store fits a dataset better than relational tables
- Insert, find, update and delete documents with pymongo
- Write queries with operators, dot notation and array conditions, and choose the fields returned with a projection
- Build an aggregation pipeline with `$match`, `$group`, `$sort`, `$unwind` and `$addFields`, and relate each stage to SQL and pandas
- Create indexes on a collection and flatten nested results into a tidy DataFrame

TIMING (70 MINUTES)

10m	Frame: the ledger and the folder; the SQL-to-MongoDB word table
25m	Live demo, steps 1 to 13
25m	Guided lab: the sale page audit (then landing pages)
10m	Start the Funnel Report: Step 1 runs for everyone

MENTAL MODEL TO SAY

"A relational database is a set of **ledgers**. Each ledger has fixed columns, every entry fills the same columns, and the facts about one website visit are spread over several ledgers that point at each other by number. A document database is a filing cabinet of **folders**. Each folder holds everything about one thing, in whatever shape that thing has: one visit's pages, cart and all. Pull one folder and you have the whole story, with no assembly. And two folders need not contain the same sheets."

Put the lesson's comparison table on the projector and read it as a translation exercise: table → collection, row → document, column → field, primary key → `_id`, a join to a child table → an embedded list. Then the honest caveat, in one sentence: "Neither is better. Orders, customers and products point at each other constantly, so they live in `pinecrest.db`. Web sessions are self-contained, nested and irregular, so they suit documents."

One sentence on tooling, so nobody goes looking for a server: "We use `mongomock`, which imitates MongoDB inside Python. Every line you write is real pymongo code; only the line that connects would change."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

One notebook, `session16_mongo`, started in the project folder so that `data/web_sessions.json` resolves. Thirteen short steps; the minutes are in the timing table above.

1. **One session is not a row.** Say: "This is one visit to the website, exactly as it is stored. Count the levels with me: a device object, a list of pages, a cart with a list inside it. Which three tables would this be in SQL?"

```
import json
from pprint import pprint

with open("data/web_sessions.json", encoding="utf-8") as f:
    sessions = json.load(f)["sessions"]

print(len(sessions), "sessions")
pprint(sessions[2], sort_dicts=False, width=100, depth=2)
```

```
2400 sessions
{'session_id': 'S00003',
 'customer_id': 11550,
 'started_at': '2025-11-01T07:07:11',
 'device': {'type': 'desktop', 'os': 'Windows', 'browser': 'Edge'},
 'pages': [{...}, {...}, {...}, {...}, {...}, {...}, {...}],
 'cart': {'items': [...], 'value': 122.9},
 'converted': True,
 'order_id': 118263}
```

2. **Two folders, different sheets.** Say: "Same collection, different keys. The second one has no `utm` at all: the visitor typed the address. In a table that is NULL in three columns. Here the field is simply not there. Remember this; it comes back in five minutes."

```
print(sorted(set(sessions[1]) - set(sessions[0])))
print(sum("utm" not in s for s in sessions), "sessions have no utm key")
```

```
['utm']
916 sessions have no utm key
```

3. **A database in one line.** Say: "`mongomock` imitates a MongoDB server inside Python: no install, no server, same methods as pymongo. Nothing is created until we insert. On a real server only the first line changes." Point at the count going from 0 to 2400.

```
import mongomock
import pandas as pd

client = mongomock.MongoClient()      # real server: pymongo.MongoClient(uri)
coll = client["pinecrest"]["sessions"]
print(coll.count_documents({}))

result = coll.insert_many(sessions)
print(len(result.inserted_ids), "inserted;",
      coll.count_documents({}), "in the collection")
```

```
0
2400 inserted; 2400 in the collection
```

4 **A query is a dictionary.** Say: "This replaces WHERE. Field, value. Two pairs in one dictionary mean AND. To reach inside the device object, join the names with a dot, in quotes." Ask before running the third line: "More or fewer than 1,326?"

```
print(coll.count_documents({"converted": True}))
print(coll.count_documents({"device.type": "mobile"}))
print(coll.count_documents({"device.type": "mobile", "converted": True}))
```

```
230
1326
130
```

5 **Operators start with a dollar sign.** Say: "Anything that is not 'equals' becomes a small dictionary with an operator. Read `$gte` as 'greater than or equal'. `$in` is SQL's IN." Have learners type the second line themselves and call out the number.

```
print(coll.count_documents({"cart.value": {"$gte": 500}}))
print(coll.count_documents({"device.os": {"$in": ["iOS", "iPadOS"]},
                            "cart.value": {"$gte": 500}}))
print(sorted(coll.distinct("device.os")))
```

```
73
32
['Android', 'Linux', 'Windows', 'iOS', 'iPadOS', 'macOS']
```

6 **Missing is not the same as null.** Say: "Predict each line before I run it. `customer_id` is always present and is null for anonymous visitors. `utm` is absent for direct visits." After the output: "`None` in a query matches null AND missing. `$exists` is the precise tool."

```
n = coll.count_documents
print("customer_id is null :", n({"customer_id": None}))
print("customer_id missing :", n({"customer_id": {"$exists": False}}))
print("utm missing         :", n({"utm": {"$exists": False}}))
print("utm equals None     :", n({"utm": None}))
```

```
customer_id is null : 1547
customer_id missing : 0
utm missing         : 916
utm equals None     : 916
```

7 **Asking inside a list.** Say: "`pages` is a list of objects. `pages.path` means 'the path of ANY page in the list'. Now the trap: I want visits that spent ninety seconds or more ON the cart page. Which of these two is that question?" Let the room vote, then run.

```
print("saw the cart page:", coll.count_documents({"pages.path": "/cart"}))

any_page = {"pages.path": "/cart", "pages.seconds": {"$gte": 90}}
same_page = {"pages": {"$elemMatch": {"path": "/cart",
                                      "seconds": {"$gte": 90}}}}
print("without $elemMatch:", coll.count_documents(any_page))
print("with $elemMatch   :", coll.count_documents(same_page))
```

```
saw the cart page: 543
without $elemMatch: 304
with $elemMatch   : 122
```

8 **Choose the fields, sort, limit.** Say: "The second argument of `find` is the projection: the SELECT list. `1` keeps a field, `_id` comes along unless you switch it off. `sort` takes -1 for descending. This is ORDER BY ... DESC LIMIT 3."

```
cursor = (coll.find({"device.type": "tablet"},
                    {"_id": 0, "session_id": 1, "device.os": 1,
                     "cart.value": 1})
          .sort("cart.value", -1)
          .limit(3))
for doc in cursor:
    print(doc)
```

```
{'session_id': 'S01248', 'device': {'os': 'iPadOS'}, 'cart': {'value': 1293.8}
{'session_id': 'S02140', 'device': {'os': 'iPadOS'}, 'cart': {'value': 1007.85
{'session_id': 'S00727', 'device': {'os': 'iPadOS'}, 'cart': {'value': 743.9}}
```

9 **The first pipeline: group and count.** Say: "`find` returns documents. To group and summarize we send a list of stages. Read `$group` slowly: `_id` is the key; a string that starts with a dollar sign means 'the value of this field'; `{'$sum': 1}` adds one per document, which is how you count." Write the SQL next to it on the board: `SELECT os, COUNT(*), AVG(cart_value) ... GROUP BY os ORDER BY 2 DESC`.

```
pipeline = [
    {"$group": {"_id": "$device.os",
                "sessions": {"$sum": 1},
                "avg_cart": {"$avg": "$cart.value"}}},
    {"$sort": {"sessions": -1}},
]
by_os = pd.DataFrame(list(coll.aggregate(pipeline)))
print(by_os[["_id", "sessions", "avg_cart"]].round(2).to_string(index=False))
```

```
     _id  sessions  avg_cart
     iOS       749     62.25
 Android       609     51.32
 Windows       574     53.64
   macOS       272     50.12
  iPadOS       127     73.86
   Linux        69     56.99
```

10 **Match, unwind, group, match again.** Say: "Which product pages do desktop visitors look at most? The page views are inside lists, so `$unwind` makes one document per page view first: pandas calls this `explode`. `$match` appears twice. Before the group it is WHERE; after the group it is HAVING. Position gives a stage its meaning."

```
pipeline = [
    {"$match": {"device.type": "desktop"}},
    {"$unwind": "$pages"},
    {"$match": {"pages.path": {"$regex": "^/product/"}}},
    {"$group": {"_id": "$pages.path", "views": {"$sum": 1},
                "avg_seconds": {"$avg": "$pages.seconds"}}},
    {"$match": {"views": {"$gte": 9}}},
    {"$sort": {"views": -1, "_id": 1}},
    {"$limit": 4},
]
top = pd.DataFrame(list(coll.aggregate(pipeline)))
print(top[["_id", "views", "avg_seconds"]].round(1).to_string(index=False))
```

```
              _id  views  avg_seconds
/product/CLM-1019     12         43.3
/product/APL-1017     10         60.6
/product/KIT-1027     10         40.7
/product/ACC-1004      9         49.2
```

11 **Two mistakes that raise no error.** Say: "I am going to make two typos on purpose. Watch for the error message." There is none. "SQL would say 'no such column'. A collection has no list of valid fields, so the answer to a question about a field nobody has is zero. And without the dollar sign, `device.type` is just a piece of text, the same for every document: one group." Write on the board: **Be suspicious of zero.**

```
print(coll.count_documents({"device.typ": "mobile"}))
wrong = coll.aggregate([{"$group": {"_id": "device.type",
                                    "sessions": {"$sum": 1}}}])
print(list(wrong))
```

```
0
[{'sessions': 2400, '_id': 'device.type'}]
```

12 **Back to a DataFrame.** Say: "Every source in this course ends in the same place. `json_normalize` spreads nested objects into dotted columns; sessions with no `utm` get NaN there. From this line on it is ordinary pandas." Ask: "Why did I leave `pages` and `cart.items` out of the projection?" (They are lists: they need one row per element, which is `record_path`.)

```
docs = list(coll.find({}, {"_id": 0, "pages": 0, "cart.items": 0}))
flat = pd.json_normalize(docs)
print(flat.shape)
print(list(flat.columns)[5:])
with_cart = flat[flat["cart.value"] > 0]
print(with_cart.groupby("device.type")["cart.value"].median().to_string())
print(flat["utm.medium"].isna().sum(), "rows have no utm.medium")
```

```
(2400, 12)
['device.type', 'device.os', 'device.browser', 'cart.value', 'utm.source', 'ut
device.type
desktop    171.90
mobile     161.90
tablet     210.85
916 rows have no utm.medium
```

13 **Count before you change anything.** Say: "An update takes two dictionaries: which documents, and what to do, written with an operator such as `$set`. The habit: run `count_documents` with the same query first, then compare with `matched_count`. Then I put the collection back with `$unset`." If time is short, show this step without typing it.

```
big = {"cart.value": {"$gte": 1000}}
expected = coll.count_documents(big)
r = coll.update_many(big, {"$set": {"vip_cart": True}})
print(expected, r.matched_count, r.modified_count)

r = coll.update_many({}, {"$unset": {"vip_cart": ""}})
print(coll.count_documents({"vip_cart": {"$exists": True}}))
```

```
17 17 17
0
```

THREE MISTAKES TO EXPECT

- **A misspelled field or a value in the wrong case returns 0, and they report the 0.** `{"device.typ": "mobile"}` and `{"device.type": "Mobile"}` both count nothing. Coach: "What does one real document look like? Run `coll.find_one({}, {"_id": 0, "device": 1})`. Now which values exist? `coll.distinct("device.type")`." Do not point at the typo; let the document show it.
- **The dollar sign in the wrong place.** In a query or `$match` the field name is the key, with no dollar: `{"device.type": "mobile"}`. Inside `$group`, `$addFields` or `$project` a field's **value** is written `"$device.type"`. Learners write `"_id": "device.type"` and get one group of 2,400.

Coach: "How many groups did you expect? How many did you get? What is the `_id` of the one you got?"

- **Two list conditions without `$elemMatch`.** In the lab the loose query counts 218 and the strict one 81. Learners who skip the strict version report 218 for "a minute or more on the sale page". Coach: "Read your query aloud as a sentence. Does it say the minute was spent on /sale, or on some page?"

Also common: `list(cursor)` twice (the second is empty: a cursor reads once), and `update_one(query, {"reviewed": True})` without `$set` (`ValueError: update only works with $ operators`, which is a kindness).

GUIDED LAB (25 MINUTES)

Lab 5.5-A, everyone: `labs/module-16/l5_5-nosql-mongodb/sale_page_audit_starter.py` (reference: `sale_page_audit_solution.py`). Question: who looks at the /sale page, do they stay, and do they buy more often than everyone else? Five TODOs, each one query.

- Minutes 0 to 5: everyone runs the starter as shipped. It prints the field list and a row of `None`s. Have them read the field list: that habit is the cure for mistake 1.
- Minutes 5 to 18: TODOs 1 to 5 in pairs. Walk the room during TODO 3; that is where the thinking is.
- Minutes 18 to 22: results on the projector. Reference answers: 290 sessions saw /sale; 37 of them ordered; 20 were on mobile with a cart of $200 or more; loose 218 versus strict 81; 92 arrived with no `utm`; biggest carts S00039 ($1,265.85), S01848 ($1,202.85), S00806 ($1,023.90); order rate 12.8% of /sale visitors versus 9.6% of all sessions.
- Minutes 22 to 25: the discussion that matters. "12.8% against 9.6%. Should marketing conclude that the sale page causes orders?" (No: people who browse more pages are more likely to see /sale and more likely to buy. Module 4's lesson, in a new costume.)

Lab 5.5-B, for pairs who finish (or homework): `labs/module-16/l5_5-nosql-mongodb/landing_pages_starter.py`. Two pipelines and a pandas check: landing pages with at least 80 sessions (most visits start at `/`, 1,301 sessions, 9.6% order rate; the highest rate is `/category/apparel`, 11.8%, 10 orders of 85) and the five SKUs with the most units in carts (SNW-1016 leads with 16 units in 8 carts). The check with `json_normalize(record_path=["cart", "items"])` gives 734 lines, 1,106 units and 543 sessions with a cart.

START THE LESSON PROJECT (10 MINUTES)

Open the lesson at "Project: The Funnel Report". Read the brief aloud: where do visitors drop out between arriving and ordering, and does it differ by device? Everyone creates `lesson_5_5_project`, types Step 1 and sees `2400`. Then read the `STEPS` dictionary of Step 2 together without running it and ask: "Which of these five queries uses something you have not typed today?" (`"cart.items.0": {"$exists": True}`: element 0 of the list exists, so the cart is not empty.) Steps 2 to 6 are homework; the checklist numbers to aim for are 2,400 → 1,610 → 543 → 230 → 230 and 313 abandoned carts.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q1 to Q6. Use Q5 as a hinge question before the lab: if fewer than two thirds of hands choose the right letter, redo demo step 7 with the two tiny documents from Q5 on the board.

Answers: **Q1** B) `{"utm": {"$exists": False}}`; **Q2** D) `2 1`; **Q3** `[('Bend', 6), ('Boise', 5)]`; **Q4** A) `$unwind`; **Q5** C) `2 1`; **Q6** B) `[{'n': 3, '_id': 'store'}]`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: pair on TODO 3 and give the sentence frame "one page that is ___ and lasted ___". Hand out the SQL-to-MongoDB table from the workbook; let them write the SQL first and translate. Skip Lab B entirely.

Stretch: in Lab B, add a third pipeline: abandoned-cart value by landing page. Or the lesson's bonus challenge: write the three relational tables with `to_sql` and answer one question both ways.

6.1 HTTP with Requests

80 min

OBJECTIVES (FROM THE LESSON)

- Describe an HTTP request and response (method, URL, headers, status code, body) and read each part from Python
- Check a site's `robots.txt` and decide whether, and how fast, your program may fetch a page
- Fetch HTML and JSON with `requests.get()`, sending a User-Agent that identifies you, query parameters and a timeout
- Tell success from failure with status codes and `raise_for_status()`, and handle timeouts and HTTP errors without crashing
- Reuse settings with a Session, fix a wrong text encoding, and save raw responses to disk

TIMING (80 MINUTES)

2m	Ground rules for Module 6, read aloud
13m	Frame: the order counter; the three-part shape; status code ranges
25m	Live demo, steps 1 to 14
25m	Guided lab: three category pages (then the API price check)
15m	Start the Polite Snapshot: Steps 1 and 2

GROUND RULES: THE 2-MINUTE OPENER FOR EVERY MODULE 6 SESSION

Before any fetching code, put these six on the projector and read them aloud, one learner per rule. Say why first: "A program can do in a minute what would take a person a week. A website is somebody's property, runs on somebody's bill, and often holds information about real people."

1. **Read `robots.txt` and obey it.** It is the owner's published request to programs: where they may go and how fast.
2. **Read the terms of use.** If they forbid automated collection, the answer is no, however easy the code would be.
3. **Prefer an API or a download** when one exists. It is the front door.
4. **Go slowly and say who you are.** Pause between requests; send a User-Agent with a way to contact you.
5. **Never go around a login, a paywall or a block.** If you are asked to stop, stop.

6. **Do not collect personal data.** Names, emails, profiles and photos of real people are protected by law in many places.

Then the reassurance: "Every request in this module goes to the practice shop or to the practice API on your own computer. You have the owner's permission, nothing on them is real, and you cannot break anything. We practice the polite habits here so they are automatic everywhere else." This is not legal advice, and the lesson says so; if someone asks "is scraping legal?", the lesson's FAQ has the careful answer.

MENTAL MODEL TO SAY

"Think of the counter at a deli. You hand over a **slip**: what you want, plus a few notes about yourself. A moment later a **tray** comes back. On top is a ticket with a number that says how it went: order filled, we are out of that, you may not order that, the kitchen is on fire. Under the ticket is the food, if there is any. That is the whole of HTTP. The slip is a request, the tray is a response. The server never calls you; it only answers. And it does not remember your last order unless you remind it."

Draw the three-part shape twice on the board, side by side: first line (method and URL | status code), headers (who is asking | what the answer is), body (usually empty for GET | the page or the JSON). Then the status ranges as "whose move is it": 2xx use it, 3xx nothing to do, 4xx fix **your** request, 5xx wait and try later.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

A new notebook, `session16_http`. The practice API must be running in a second terminal from step 9 on. The outputs below were produced against a local copy of the practice shop that sends the same headers as the course website; on the public site only the `Date` header differs. If your room uses the offline copy, read "What differs on the offline copy" above before step 4.

1. **One address, defined once.** Say: "Every request this module makes goes to one of two places built for this course: the practice shop and the practice API. Put your own email in the User-Agent. In about ten minutes you will see who reads it."

```
import requests
from urllib.parse import urljoin

BASE_URL = "https://rays-python-datascience.netlify.app/practice/shop/"
USER_AGENT = "pinecrest-course-bot/1.0 (learning project; contact: you@example
HEADERS = {"User-Agent": USER_AGENT}
print(requests.__version__)
```

```
2.34.2
```

2. **The first page a polite program asks for.** Say: "Before the catalog, the rule book. `urljoin` resolves `../robots.txt` the way a browser resolves a link." Read the file aloud: who it applies to, where not

to go, how fast. "That was a complete HTTP conversation: one request out, one response back. 200 means 'here you go'."

```
robots_url = urljoin(BASE_URL, "../robots.txt")
r = requests.get(robots_url, headers=HEADERS, timeout=10)
print(r.status_code, robots_url)
print(r.text)
```

```
200 https://rays-python-datascience.netlify.app/practice/robots.txt
# Practice rules for the Python Data Science course (fictional shop).
# A real robots.txt lives at the root of a site; this one is a teaching copy
# that the lessons read with urllib.robotparser.
User-agent: *
Disallow: /practice/shop/staff/
Crawl-delay: 1

User-agent: BadBot
Disallow: /
```

3 **Let the standard library read the rules.** Say: "You do not interpret the file by eye. Two questions matter: may I fetch this, and how long do I wait?" Ask before running: "What will line two say?" Then: "Nothing in the software stops you from fetching the staff page. Honoring the file is your job."

```
from urllib.robotparser import RobotFileParser

robots = RobotFileParser()
robots.parse(r.text.splitlines())
print(robots.can_fetch(USER_AGENT, urljoin(BASE_URL, "category-tents.html")))
print(robots.can_fetch(USER_AGENT, urljoin(BASE_URL, "staff/")))
print(robots.crawl_delay(USER_AGENT))
```

```
True
False
1
```

4 **A page, and the envelope it came in.** Say: "Status line, headers, body: the same three parts every time. `r.ok` is True for anything below 400. `Content-Type` tells you what the body is and how its text is encoded." Open the same URL in the browser next to the notebook so learners connect the two.

```
url = urljoin(BASE_URL, "category-tents.html")
r = requests.get(url, headers=HEADERS, timeout=10)
print(r.status_code, r.reason, r.ok)
print(r.headers["Content-Type"])
cards = r.text.count('class="product-card"')
print(len(r.content), "bytes;", cards, "product cards")
```

```
200 OK True
text/html; charset=UTF-8
12997 bytes; 24 product cards
```

5 **What did we send?.** Say: "The response keeps a copy of the request. We wrote one header; requests added the rest. Without ours, the server log would say only `python-requests`: an anonymous script. With it, the owner can write to you before reaching for the block button."

```
print(r.request.method, r.request.url)
for name, value in r.request.headers.items():
    print(f"{name}: {value}")
```

```
GET https://rays-python-datascience.netlify.app/practice/shop/category-tents.h
User-Agent: pinecrest-course-bot/1.0 (learning project; contact: you@example.c
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

6 **A 404 is not an exception.** Say: "Predict: does this line crash?" It does not. "From requests' point of view the conversation went perfectly: it asked, the server answered 'not found' and sent a friendly error page. If you count products in that body you get zero, and nothing tells you why."

```
url = urljoin(BASE_URL, "category-golf.html")
missing = requests.get(url, headers=HEADERS, timeout=10)
print(missing.status_code, missing.reason, missing.ok)
print(len(missing.content), "bytes of", missing.headers["Content-Type"])
print(missing.text.count('class="product-card"'), "product cards")
```

```
404 Not Found False
2514 bytes of text/html; charset=utf-8
0 product cards
```

7 **The one-line check.** Say: "Make it a reflex: get, then `raise_for_status()`, then use the body. A loud failure here is far cheaper than a quiet zero three steps later." Have everyone type this and read the error message aloud: it names the status and the URL.

```
missing.raise_for_status()
```

```
HTTPError: 404 Client Error: Not Found for url: https://rays-python-datascienc
```

8 **Bytes, text and the wrong table.** Say: "Computers send bytes. `r.text` is those bytes decoded with the encoding named in the header. Watch what happens to the rating stars when I decode with the wrong table." Name it: mojibake. "If text ever looks like this, print `r.encoding` and the Content-Type header first."

```
print(r.encoding, "|", len(r.content), "bytes ->", len(r.text), "characters")
r.encoding = "cp1252"
print(r.text.count("★"), "stars;", r.text.count("â˜…"), "pieces of junk")
r.encoding = "utf-8"
print(r.text.count("★"), "stars;", r.text.count("â˜…"), "pieces of junk")
```

```
UTF-8 | 12997 bytes -> 12945 characters
0 stars; 24 pieces of junk
24 stars; 0 pieces of junk
```

9 **An API: parameters in, JSON out.** Say: "HTML is written for people. An API is the front door for programs. Start the practice API in a second terminal first: `python data/mock_api.py`. I pass the parameters as a dictionary and let requests build the URL: look at what it did with the space."

```
API = "http://127.0.0.1:8000"

r = requests.get(f"{API}/products",
                 params={"category": "Camp Kitchen", "per_page": 3},
                 headers=HEADERS, timeout=10)
r.raise_for_status()
print(r.url)
print(r.headers["Content-Type"])
data = r.json()
print(list(data))
print(data["total"], "in the category;", len(data["items"]), "on this page")
```

```
http://127.0.0.1:8000/products?category=Camp+Kitchen&per_page=3
application/json; charset=utf-8
['page', 'per_page', 'total', 'total_pages', 'next', 'items']
29 in the category; 3 on this page
```

10 **From JSON to a DataFrame.** Say: "A list of flat dictionaries is exactly what `pd.DataFrame` wants. Three lines from a URL to a table, and the numbers arrive as numbers. That is why ground rule three says: prefer the API."

```python
import pandas as pd

kitchen = pd.DataFrame(data["items"])
print(kitchen[["sku", "name", "price", "stock_status"]].to_string(index=False))
```

```
     sku                                      name  price stock_status
KIT-1001 Larkspur Ridge Headwater Backpacking Stove 154.95     in_stock
KIT-1002                Pinecrest Wayfinder Cookset  40.95     in_stock
KIT-1003         Quillback Timberline Insulated Mug  24.95     in_stock
```

11 **Errors speak JSON too.** Say: "A well-made API explains its errors in the body. The status code tells your program what happened; the message tells you."

```python
r = requests.get(f"{API}/products/KIT-9999", headers=HEADERS, timeout=10)
print(r.status_code, r.ok)
print(r.json())
```

```
404 False
{'error': {'code': 'not_found', 'message': "No product with SKU 'KIT-9999'."}}
```

12 **Timeouts, always.** Say: "A fact that surprises people: with no timeout, requests waits forever. `/slow` answers late on purpose. I ask it to take three seconds and I am willing to wait one." After the error: "That is the behavior you want: a clear error, at a moment you chose."

```python
requests.get(f"{API}/slow", params={"seconds": 3}, headers=HEADERS, timeout=1)
```

```
ReadTimeout: HTTPConnectionPool(host='127.0.0.1', port=8000): Read timed out.
```

13 **One function that does it right.** Say: "Four habits: say who you are, set a timeout, check the status, expect failure. Typing all four on every call is how they get skipped, so they go in one function. Specific

exceptions first, the general one last." Type it together; this is the function Session 17 builds on.

```
def fetch(url, **kwargs):
    """GET a URL politely. Return the response, or None after saying why."""
    kwargs.setdefault("headers", HEADERS)
    kwargs.setdefault("timeout", 10)
    try:
        resp = requests.get(url, **kwargs)
        resp.raise_for_status()
    except requests.exceptions.Timeout:
        print(f"TIMEOUT   {url}")
    except requests.exceptions.HTTPError as err:
        print(f"HTTP {err.response.status_code}  {url}")
    except requests.exceptions.RequestException as err:
        print(f"FAILED    {url} ({type(err).__name__})")
    else:
        return resp
    return None

print(fetch(urljoin(BASE_URL, "category-tents.html")))
print(fetch(urljoin(BASE_URL, "category-golf.html")))
print(fetch(f"{API}/slow", params={"seconds": 3}, timeout=1))
```

```
<Response [200]>
HTTP 404  https://rays-python-datascience.netlify.app/practice/shop/category-g
None
TIMEOUT   http://127.0.0.1:8000/slow
None
```

14 **A session, a pause, and the raw bytes on disk.** Say: "A Session keeps the connection open and remembers the headers. It does NOT remember a timeout. `time.sleep(1)` is the crawl delay we read at the start. And we save `resp.content`, the bytes, so every parsing experiment next session runs without a new request."

```python
import time
from pathlib import Path

raw = Path("raw")
raw.mkdir(exist_ok=True)

with requests.Session() as session:
    session.headers.update(HEADERS)
    for name in ["category-tents.html", "category-packs.html"]:
        resp = session.get(urljoin(BASE_URL, name), timeout=10)
        resp.raise_for_status()
        (raw / name).write_bytes(resp.content)
        print(name, resp.status_code, len(resp.content), "bytes saved")
        time.sleep(1)

print(sorted(p.name for p in raw.iterdir()))
```

```
category-tents.html 200 12997 bytes saved
category-packs.html 200 12863 bytes saved
['category-packs.html', 'category-tents.html']
```

THREE MISTAKES TO EXPECT

- **Parsing an error page as if it were data.** A loop over `page-12.html` to `page-15.html` reports 20, 2, 0, 0 products and nobody notices that the zeros are 404s. Coach: "What was the status of that response? Where in your loop would you find out?" The fix is one line after every `get`.
- **No timeout, or `ConnectionError` because the API is not running.** A cell that says "running" for a minute has a call with no `timeout=`. A long red message ending in `Connection refused` means the second terminal was closed. Coach: "Is the API's terminal still open? What does `http://127.0.0.1:8000/health` show in your browser?"
- **Building the query string by hand, or calling `.json()` on HTML.** `f"{API}/products?category=Camp Kitchen"` breaks on the space; `r.json()` on a page gives `JSONDecodeError: Expecting value: line 1 column 1 (char 0)`. Coach: "Print `r.url`. Print `r.headers["Content-Type"]`. Print `r.text[:200]`. What did you really receive?"

Also common: leaving off `https://` (`MissingSchema`), and on the offline copy, stars that print as `â˜…` (set `r.encoding = "utf-8"`).

GUIDED LAB (25 MINUTES)

Lab 6.1-A, everyone: `labs/module-16/l6_1-http-requests/category_check_starter.py` (reference: `category_check_solution.py`). Question: how many products do the Footwear, Snow and Skis category pages list, and how many show a rating star? One of the three pages does not exist.

- **How the lab reaches the network.** One function, `get()`, hides the choice. Normally it sends a real request to `BASE_URL` and pauses one second. With `LABS_OFFLINE=1` in the environment it rebuilds real `Response` objects from `labs/module-16/_fixtures/` and touches nothing. Teaching with no network at all: `LABS_OFFLINE=1 python category_check_starter.py` (PowerShell: `$env:LABS_OFFLINE = "1"; python category_check_starter.py`). Teaching from the unzipped copy: change the `BASE_URL` line, or set `PINECREST_BASE_URL`.
- Minutes 0 to 4: run the starter as shipped. It already fetches; the report shows status 200, 200, 404 and `TODO 2: status not checked yet`.
- Minutes 4 to 18: TODO 1 (ask `robots.txt`), TODO 2 (check the status before counting), TODO 4 (decode with the wrong table, then the right one).
- Minutes 18 to 25: results. Reference answers: `robots.txt` allows the three category pages and refuses `staff/`, crawl delay 1; Footwear 28 products, 25 with a star, 3 not rated yet; Snow 22 and 22; Skis is a 404 whose body (2,514 bytes of HTML) contains 0 product cards. Under cp1252 the Snow page has 0 stars and 22 pieces of `â˜…`; under utf-8, 22 stars. 11,987 bytes became 11,939 characters.
- Ask: "Step 3 counted zero product cards on the Skis page. Is 'Skis: 0 products' a true statement?" (No. There is no such page. Zero and 'no answer' are different results, and only the status code tells them apart.)

Lab 6.1-B, for pairs who finish (or homework): `labs/module-16/l6_1-http-requests/api_price_check_starter.py`, which needs the practice API running (or `LABS_OFFLINE=1`). Reference answers: the URL is `http://127.0.0.1:8000/products?category=Camp+Kitchen&per_page=100`; 75 products in three categories; median price Tents $298.45 (24 products), Snow $78.45 (22), Camp Kitchen $39.95 (29); category `Skis` answers 200 with total 0; product `TNT-9999` answers 404 with the message `No product with SKU 'TNT-9999'.` The point of steps 3 and 4: an empty result is not an error, and a real error explains itself in the body.

START THE LESSON PROJECT (15 MINUTES)

Open the lesson at "Project: The Polite Snapshot". Learners type Step 1 (session, `robots.txt`, crawl delay) and Step 2 (`polite_get()`), and run the last two lines of Step 2: the staff URL comes back as `skipped: robots.txt` **without a request being sent**. Stop there and ask: "Where in the function does the robots check sit, and why there?" (Before the network is touched: the only place it does any good.) Steps 3 to 5 are homework. Step 5 needs the practice API; remind them how to start it at home.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q7 to Q12. Q7 is the hinge question for this lesson: ask it by show of hands right after demo step 6, before you run step 7.

Answers: **Q7** D) A normal response comes back; `r.ok` is `False` and nothing is raised; **Q8** `1 3 False`; **Q9** A) `r.encoding` and the `Content-Type` header; **Q10** `http://127.0.0.1:8000/orders?status=returned&since=2025-12-01`; **Q11** C) Skip the staff page, and wait at least one second between its other requests; **Q12** B) `False` then `caught`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give the three-line pattern on a card: `r = requests.get(url, headers=HEADERS, timeout=10)` / `r.raise_for_status()` / use `r.text` or `r.json()`. In Lab A, TODO 2 is already written in comments: uncommenting and indenting it is a legitimate way through.

Stretch: lesson Exercise 4 (`get_json()` that returns `(data, problem)`), or the bonus challenge: turn `polite_get()` into a cache that reads `raw/` before it touches the network.

CLOSING

EXIT TICKET (3 QUESTIONS, ON A STICKY NOTE; ANSWERS)

1. In a pymongo query, what does `{"utm": None}` match, and which operator asks the precise question? **Answer:** Documents where `utm` is null **and** documents where it is missing. `$exists` tells the two apart.
2. `requests.get()` comes back with status 404. What has happened in your program, and which one line makes the failure loud? **Answer:** Nothing was raised; you hold a normal response whose body is an error page. `r.raise_for_status()`.
3. Name any two of the six ground rules. **Answer:** Any two of: obey `robots.txt`; read the terms; prefer an API; go slowly and identify yourself; never go around a login or block; no personal data.

Read the notes before Session 17. If more than a third miss question 2, open Session 17 by parsing a 404 page on purpose.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- Lesson 5.5: finish the Funnel Report (Steps 2 to 6), do the four exercises, the quiz and the journal prompt.
- Lesson 6.1: finish the Polite Snapshot (Steps 3 to 5), do the four exercises, the quiz and the journal prompt.
- Kit labs not finished in the room: `landing_pages_starter.py` and `api_price_check_starter.py`.
- Put your real contact address in `USER_AGENT`.

BRING LIST FOR SESSION 17 (SCRAPING STATIC AND DYNAMIC PAGES)

- Your Lesson 6.1 notebook and the `raw/` folder. Lesson 6.2 reuses `BASE_URL`, the User-Agent header and the idea behind `fetch()`.
- **Google Chrome, installed and up to date.** Session 17 drives it with Selenium. Nothing else to install: `beautifulsoup4` and `selenium` came with the course environment in Lesson 0.1.
- **Run the one-minute Chrome check at home, while you have internet** (the first run downloads a driver):

```
from selenium import webdriver

options = webdriver.ChromeOptions()
options.add_argument("--headless=new")
driver = webdriver.Chrome(options=options)
driver.get("https://rays-python-datascience.netlify.app/practice/shop/")
print(driver.title)
driver.quit()
```

It should print the shop's page title and return within about ten seconds. If it raises an error, bring the exact message: Session 17 has a plan for machines where Chrome cannot run (pairing, and a route that needs no browser).

- `data/pinecrest-practice-site.zip` in your project folder, in case the venue network is down.
- Before the session, open the practice shop in a browser, right-click a product name and choose Inspect. You do not need to understand the HTML yet.
- Accounts: none needed. Servers: none to start for Session 17 unless you work offline.

IF YOU RUN LONG OR SHORT

Running long. At 1:10 you should be starting the Funnel Report; if you are still in the lab, skip the project start (assign Step 1 as homework: it is five lines) and take the break on time. In the second half the fixed points are the ground rules, demo steps 6 and 7, and Lab A. Demo steps 10, 11 and 14 can be shown from the lesson page instead of typed. If the practice API will not start in the room, do steps 9 to 12 from the guide's printed output and move Lab B to homework; Lab A does not need the API.

Running short. Run Lab 5.5-B in the room and compare the pipeline with the pandas check. In 6.1, do lesson Exercise 2 together (count products per category by asking for `per_page=1` and reading `total`: Tents 24, Snow 22, Footwear 28), then Exercise 3 (a `HEAD` request: status 200 and 0 bytes of body). Or take the lesson's "Common Questions" as a five-minute discussion: "pandas can read a URL directly; why not just use `pd.read_csv(url)`?"

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 17: Scraping Static and Dynamic Pages

How to teach the scraping session. The first half turns static HTML into a tidy, checked DataFrame with Beautiful Soup: labels over positions, one block one dictionary, missing values on purpose, pagination without counting. The second half meets pages that JavaScript fills in, teaches learners to look for the data file first, and drives headless Chrome with explicit waits when nothing else will do.

2 lessons | 3 hrs session | Lessons 6.2–6.3 | Session 17 of 21

HOW TO USE THIS GUIDE

- Session 17 is the scraping session. Before the break learners turn static HTML into a checked DataFrame with Beautiful Soup (Lesson 6.2); after it they meet pages that JavaScript fills in, learn to look for the data file first, and drive headless Chrome with Selenium when they must (Lesson 6.3).
- Same rhythm as every session: **frame → live demo** (you type, they follow) **→ guided lab → start the lesson project**. Finishing the projects, the exercises, quizzes and journals is homework.
- Every code block below was run for this guide: Beautiful Soup 4.15, pandas 3.0.6, requests 2.34.2 and Selenium 4.51 with headless Chromium, against a local copy of the practice shop that sends the course website's headers. The output is real; on the public site only the addresses you see are different, and they are shown as the public ones.
- The single most useful moment in the first half is a crash: `parse_card()` works on the first cards and raises `KeyError: 'data-rating'` on the Packs page. Let it happen live. The most useful idea in the second half is the order of the routes: data file first, browser last.
- Labs live in `labs/module-17/`. The Selenium labs fall back to saved finished pages when Chrome cannot start, so nobody is left without a lab.

> **THE RISK IN SESSION 17: THE BROWSER AS A HAMMER**
>
> Selenium is exciting, and learners who have just watched it click buttons will want to use it for everything, with `time.sleep()` sprinkled in until it works. Both habits are expensive. Keep two sentences on the board all afternoon: **"Is the data in the HTML? Is it in a file the page loads? Only then a browser."** and **"Never sleep: wait for something by name."**

BEFORE THE SESSION

- ☐ Run `python3 labs/check_labs.py module-17` from the kit folder. Expect `8 passed, 0 failed` (offline: the Selenium labs read saved finished pages and start no browser).
- ☐ Copy the starters, `_fixtures/` and `labs_paths.py` to the shared folder, keeping the layout. Keep the `*_solution.py` files on your machine.
- ☐ **The practice shop, online.** Open `https://rays-python-datascience.netlify.app/practice/shop/` and its Deals page **on the venue network**. The Deals page should show 8 deals after a second or two. If the shop does not load, use the offline copy for the whole room.
- ☐ **The practice shop, offline fallback.** In the project folder: `python -m zipfile -e data/pinecrest-practice-site.zip .` then `python -m http.server 8001` (leave it running). Everyone sets `BASE_URL = "http://127.0.0.1:8001/practice/shop/"`; for the kit labs, set `PINECREST_BASE_URL` to the same address or edit the `BASE_URL` line. Selenium loads local pages the same way, and the deals and search pages work offline because their data files are in the zip.
- ☐ **The practice API is not needed today.** Neither 6.2 nor 6.3 uses it. (Session 18 starts it with flags: `python data/mock_api.py --token-ttl 5` for Lesson 6.4 and `python data/mock_api.py --rate 10 --per 5` for Lesson 6.5.)
- ☐ **Chrome and Selenium on the projector machine.** Run the one-minute check from the Session 16 bring list (below). The first run downloads a driver, so do it on the venue network before learners arrive. Keep the printed page title on screen as proof.
- ☐ **MongoDB:** nothing needed (Session 16 ran on mongomock, no server).
- ☐ Browser tabs: `webscraping_beautifulsoup.html`, `webscraping_selenium.html`, the shop's Packs category page and Deals page, and the Deals page with developer tools open on the Network tab (Fetch/XHR filter).
- ☐ Whiteboard: "**Is the data in the HTML? In a file the page loads? Only then a browser.**" and "**Never sleep: wait for something by name.**"

THE CHROME CHECK (FROM THE SESSION 16 BRING LIST)

```
from selenium import webdriver

options = webdriver.ChromeOptions()
options.add_argument("--headless=new")
driver = webdriver.Chrome(options=options)
driver.get("https://rays-python-datascience.netlify.app/practice/shop/")
print(driver.title)
driver.quit()
```

It prints `All products, page 1 | Pinecrest Outfitters (practice site)` within about ten seconds. What the failures mean:

- `NoSuchDriverException` or "unable to obtain driver": Selenium Manager could not download the driver (blocked network or proxy) or could not find Chrome. Run it once on an open network, or install Chrome.
- "cannot find Chrome binary": Chrome is not installed, or is installed somewhere unusual. The kit labs accept `PINECREST_CHROME` = the path of a Chrome or Chromium program.
- It hangs for a minute, then fails: a corporate machine blocking automation. Treat it as locked down.

A LOCKED-DOWN VENUE MACHINE: THE PLAN

- **Pair up.** Seat learners whose Chrome failed with a neighbor whose check worked; the driver types, the navigator reads the lab aloud, and they swap at TODO 3.
- **Use the JSON route.** Lesson 6.3 teaches it first for a reason: `deals.json` and `products.json` hold the same data the browser shows, and need nothing but `requests`. Learners without a browser do the lesson's first section, Exercise 4 and lab step 6 of `budget_search` in full.
- **The kit labs never strand anyone.** If Chrome cannot start, they print `[no browser] Chrome could not be started (...)` and continue with the saved finished page. Everything after the browser step still runs.
- **Nothing to install on the spot.** If a learner can install Chrome, do it at the break, not during the demo.

WHAT DIFFERS ON THE OFFLINE COPY (CHECKED FOR THIS KIT)

- Python's server sends `text/html` with no charset, so `resp.text` shows `â˜…` for stars. Lesson 6.2 passes `resp.content` to Beautiful Soup, which reads the page's `<meta charset>`, so every parsed value is identical. Only learners who use `resp.text` see junk: a live reminder of why the lesson does it this way.
- `pd.read_html(io.BytesIO(resp.content), encoding="utf-8")` gives the same two tables.
- A missing page is Python's own 335-byte 404 page, not the site's. `raise_for_status()` behaves the same.
- Selenium: the deals appear after the same 1.5 seconds and the search form produces the same URLs with `127.0.0.1:8001` in front. Links read with `get_attribute("href")` start with the local address.
- All four kit labs give the same answers on the offline copy (checked against both servers).

ENVIRONMENT VARIABLES: SET THEM UP NOW, USED NEXT TIME

Session 18 reads three environment variables (the practice API's public demo values). Show the commands at the end of today so nobody meets them for the first time under checkpoint time pressure.

```
# Windows PowerShell (this terminal only)
$env:PINECREST_API_KEY = "pinecrest-demo-key"
$env:PINECREST_USER = "analyst"
$env:PINECREST_PASSWORD = "trailmix"

# macOS and Linux (this terminal only)
export PINECREST_API_KEY="pinecrest-demo-key"
export PINECREST_USER="analyst"
export PINECREST_PASSWORD="trailmix"
```

Start Jupyter or VS Code from that same terminal, or the notebook will not see them.

SESSION 17 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 16 (MongoDB, HTTP), no notes. Homework check: funnel numbers, the `raw/` folder, and the Chrome check (who could not run it?).
0:10	2	Ground rules opener	The six Module 6 rules, read aloud by six learners.
0:12	13	**6.2** Frame and mental model	A page is a tree; scraping is reading the labels on a filing cabinet. Inspect one product card together in the browser.
0:25	25	**6.2** Live demo	`get_soup()`, find and select, sturdy selectors, one card one dictionary, the crash that teaches, pagination, checks, `read_html`.
0:50	25	**6.2** Guided lab	`labs/module-17/l6_2-beautifulsoup/sleeping_bag_shortlist_starter.py`, then `price_list_tables_starter.py`.
1:15	10	**6.2** Start the lesson project	Category Price Report: Steps 1 and 2 (discover the nine category URLs; look before you loop).
1:25	10	Break	Projector machine: run the Chrome check once more; pair anyone whose Chrome failed with a neighbor whose worked.
1:35	12	**6.3** Frame and mental model	Flat-pack furniture: the page that comes back empty, and the order of the two honest routes.
1:47	25	**6.3** Live demo	Look for the data first; headless Chrome; explicit waits; `page_source` to Beautiful Soup; a click loop; a form; quit in `finally`.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:12	25	**6.3** Guided lab	`labs/module-17/l6_3-selenium/budget_search_starter.py`, then `page_triage_starter.py`.
2:37	13	**6.3** Start the lesson project	Deals Watch: Steps 1 and 2 (the answer key first, then `collect_deals()`).
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 18 (API flags, three environment variables, Checkpoint 2).
Total	180		Ends at 3:00.

In the room: the ground rules, both mental models, both demos, Lab 6.2-A and Lab 6.3-A (the B labs if a pair finishes early), Steps 1 and 2 of each lesson project. **At home (about 2 to 3 hours):** the rest of both projects, each lesson's four exercises, the two quizzes, the journals and the B labs. The lessons estimate 95 + 90 = 185 minutes solo; the room has 158 after the warm-up, opener and wrap-up.

Protect: 6.2 demo steps 7 to 9 (the crash and the decision about missing values) and Lab A; 6.3 demo steps 1 to 5 (empty page, data file first, the wait) and step 12 (`quit()` in `finally`), and Lab A. **Cut first if long:** 6.2 step 12 (`read_html`: Lab B covers it), then 6.3 steps 9 and 10 (the form: Lab B covers it), then the project starts (assign Step 1 of each as homework).

WARM-UP

0:00 Warm-up: Retrieval from Session 16

10 min

Five questions on the projector, answered on paper, notes closed (4 minutes), then aloud (3 minutes).

1. In a pymongo query, what does `{"utm": None}` match? **Answer:** Documents where `utm` is null **and** documents where it is missing. `$exists` tells the two apart.
2. Which aggregation stage does what pandas `explode` does? **Answer:** `$unwind`: one document per element of a list.
3. `requests.get()` comes back with status 404. Has an exception been raised? **Answer:** No. You hold a normal response with `.ok` False; `r.raise_for_status()` makes it loud.
4. Why does every request in this course carry `timeout=`? **Answer:** Without it `requests` waits forever for a server that has gone quiet.
5. You see `â˜…` where a star should be. What happened, and what is one fix? **Answer:** UTF-8 bytes were decoded with the wrong table (no charset sent). Set `r.encoding = "utf-8"`, or hand `r.content` to a parser that reads the page's `<meta charset>`.

Homework check (3 minutes). Funnel Report: hands up if your funnel read 2,400 → 1,610 → 543 → 230 → 230. Polite Snapshot: four files in `raw/` and a six-row manifest. **Chrome check:** "Who saw the page title print? Who got an error?" Write the names of anyone with an error on a sticky note: they are your pairing list for the second half.

GROUND RULES OPENER (2 MINUTES)

0:10 The Six Ground Rules for Module 6

2 min

Every Module 6 session opens here. Six learners read one rule each, aloud. Add one sentence for today: "A browser is a heavier guest than `requests`: it downloads every script, style and image. That is one more reason to reach for it last."

1. **Read `robots.txt` and obey it.** It is the owner's published request to programs: where they may go and how fast.
2. **Read the terms of use.** If they forbid automated collection, the answer is no, however easy the code would be.
3. **Prefer an API or a download** when one exists. It is the front door.
4. **Go slowly and say who you are.** Pause between requests; send a User-Agent with a way to contact you.
5. **Never go around a login, a paywall or a block.** If you are asked to stop, stop.
6. **Do not collect personal data.** Names, emails, profiles and photos of real people are protected by law in many places.

6.2 Parsing HTML with Beautiful Soup

73 min

OBJECTIVES (FROM THE LESSON)

- Read a page's HTML as a tree and locate the elements that hold the data you want
- Select elements with `find()`, `find_all()` and CSS selectors, and choose selectors that survive small changes to a page
- Write a function that turns one repeated block of HTML into a dictionary, including when optional pieces are missing
- Follow "next" links to the last page of a paginated listing, politely, and visit detail pages for extra fields
- Read HTML tables with `pandas.read_html()`, validate what you scraped, and save a tidy DataFrame

TIMING (73 MINUTES)

13m	Frame: the filing cabinet; Inspect one card in the browser
25m	Live demo, steps 1 to 13
25m	Guided lab: the sleeping bag shortlist (then the price list tables)
10m	Start the Category Price Report: Steps 1 and 2

MENTAL MODEL TO SAY

"A page is a **filing cabinet**. Drawers hold folders, folders hold sheets, and nearly everything has a label on it: a class such as `price`, a `data-sku` such as `TNT-1001`. You do not find a document by counting 'third drawer, second folder, fifth sheet'. You read the labels: 'the sheet labeled price inside the folder labeled product-card'. Scraping well is mostly the skill of choosing good labels."

Then the whole job in one line on the board: **fetch → parse → select the repeated block → one dictionary per block → DataFrame → check it.** Open the Packs category page, right-click a product name, Inspect, and walk up to the `li.product-card`. Read its labels aloud.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `session17_soup`. Thirteen steps; each runs in seconds except step 10, which pauses one second between three requests.

1. **Same polite setup, one new helper.** Say: "Everything from Session 16 stays: one address, an honest User-Agent, a timeout, raise_for_status. The new line is the last one in the function: we hand the BYTES to Beautiful Soup, which reads the page's own charset tag." Run it and point at the title.

```
import re
import time
from urllib.parse import urljoin

import pandas as pd
import requests
from bs4 import BeautifulSoup

BASE_URL = "https://rays-python-datascience.netlify.app/practice/shop/"
HEADERS = {"User-Agent": "pinecrest-course-bot/1.0 (learning project; contact:
DELAY = 1        # the Crawl-delay in robots.txt


def get_soup(url):
    """Fetch one page politely and return it as a parsed tree."""
    resp = requests.get(url, headers=HEADERS, timeout=10)
    resp.raise_for_status()
    return BeautifulSoup(resp.content, "html.parser")


packs = get_soup(urljoin(BASE_URL, "category-packs.html"))
print(packs.title.string)
```

```
Packs | Pinecrest Outfitters (practice site)
```

2 **An element: a name, attributes, text.** Say: "Predict: is the first `li` on the page a product?" It is not: it is the breadcrumb. "That is why we search by LABEL, not by tag name alone. Attributes behave like a dictionary; class is a LIST, because an element can have several."

```
print(repr(packs.find("li").get_text(strip=True)))
card = packs.find("li", class_="product-card")
print(card.attrs)
print(card["data-sku"])
print(card.get_text(" | ", strip=True))
```

```
'Shop'
{'class': ['product-card'], 'data-sku': 'PCK-1001'}
PCK-1001
Wrenfield Canyon 22L Daypack | Packs | $136.95 | ★ 4.7 | (200 reviews) | In st
```

3 **find, find_all, and the underscore.** Say: "`find` gives the first match or None. `find_all` gives a list, maybe empty. `class_` has an underscore because `class` is a Python keyword. And notice the key move: once we hold one card, we search INSIDE it."

```
cards = packs.find_all("li", class_="product-card")
print(len(cards))
link = cards[0].find("a")
print(link.get_text(), "->", link["href"])
print(cards[0].find("p", class_="price").get_text())
```

```
24
Wrenfield Canyon 22L Daypack -> products/PCK-1001.html
$136.95
```

4 **CSS selectors: the labels on the drawers.** Say: "`select` returns a list, `select_one` the first match or None. Read each selector aloud as a sentence: 'the price paragraph inside the card whose data-sku is PCK-1003'."

```
print(len(packs.select("li.product-card")))
print(packs.select_one("p.result-count").get_text())
print(len(packs.select("p.stock.out-of-stock")), "out of stock on this page")
print(packs.select_one('li[data-sku="PCK-1003"] p.price').get_text())
```

```
24
24 products
3 out of stock on this page
$182.95
```

5 **Fragile and sturdy.** Say: "Both lines find the same price today. The first is directions by position: it breaks silently the day the shop adds a badge or reorders the list. The second reads the labels. Choose meaning over position, and keep it short."

```
fragile = packs.select_one("main > ul > li:nth-of-type(3) > p:nth-of-type(2)")
sturdy = packs.select_one('li[data-sku="PCK-1003"] p.price')
print(fragile.get_text(), sturdy.get_text())
```

```
$182.95 $182.95
```

6 **Take the machine-readable value.** Say: "The text is for people; the data- attribute is for the site's own scripts. Either way, HTML gives you strings: converting is your job."

```
price = cards[0].select_one("p.price")
print(repr(price.get_text()), repr(price["data-price"]))
print(float(price["data-price"]) * 2)
```

```
'$136.95' '136.95'
273.9
```

7 **One card, one dictionary (first version).** Say: "This is the function that makes scraping manageable: it knows the HTML so the rest of the code does not have to. Predict: will it work on all 24 packs?" Let them vote, then run.

```
def parse_card(card):
    """Turn one product card into a flat dictionary. First version."""
    link = card.select_one("h2.product-name a")
    return {
        "sku": card["data-sku"],
        "name": link.get_text(strip=True),
        "price": float(card.select_one("p.price")["data-price"]),
        "rating": float(card.select_one("p.rating")["data-rating"]),
    }


rows = [parse_card(c) for c in packs.select("li.product-card")]
```

```
KeyError: 'data-rating'
```

8 **Ask the page, do not guess.** Say: "A KeyError on an attribute. Which card? Don't guess: ask." After the output: "Two packs have no reviews yet, so the shop writes different HTML for them. Real sites are full of this. The crash was the process working."

```
for c in packs.select("li.product-card"):
    tag = c.select_one("p.rating")
    if "data-rating" not in tag.attrs:
        print(c["data-sku"], tag)
```

```
PCK-1002 <p class="rating no-rating">No reviews yet</p>
PCK-1009 <p class="rating no-rating">No reviews yet</p>
```

9 **Decide what missing means, field by field.** Say: "`.get()` returns None for a missing attribute instead of raising. A product without a rating is normal: record missing. A product without a price is

NOT normal: keep the square brackets so that one crashes loudly."

```
def parse_card(card):
    """Turn one product card into a flat dictionary."""
    link = card.select_one("h2.product-name a")
    rating = card.select_one("p.rating").get("data-rating")      # None if unra
    return {
        "sku": card["data-sku"],
        "name": link.get_text(strip=True),
        "price": float(card.select_one("p.price")["data-price"]),
        "rating": float(rating) if rating else None,
        "stock": card.select_one("p.stock")["class"][1],
    }


packs_df = pd.DataFrame([parse_card(c) for c in packs.select("li.product-card"
print(packs_df.shape)
print(packs_df.loc[packs_df["rating"].isna(), ["sku", "name", "rating"]])
print(packs_df["stock"].value_counts().to_dict())
```

```
(24, 5)
        sku                                name  rating
1  PCK-1002    Pinecrest Headwater 30L Daypack     NaN
8  PCK-1009  Juniper Vane Sawtooth 30L Daypack     NaN
{'in-stock': 20, 'out-of-stock': 3, 'low-stock': 1}
```

10 **Follow "next" until there is none.** Say: "The catalog is paginated. We never count pages by hand: we follow the next link until a page has none. `max_pages` is a seat belt. I start at page 11 so the demo sends three requests, not thirteen."

```
def scrape(start_url, max_pages=50):
    """Follow 'next' links from start_url. Returns (rows, pages_fetched)."""
    rows, url, pages = [], start_url, 0
    while url and pages < max_pages:
        soup = get_soup(url)
        rows.extend(parse_card(c) for c in soup.select("li.product-card"))
        pages += 1
        nxt = soup.select_one("nav.pagination a.next")
        url = urljoin(url, nxt["href"]) if nxt else None
        time.sleep(DELAY)
    return rows, pages


rows, pages = scrape(urljoin(BASE_URL, "page-11.html"))
print(pages, "pages,", len(rows), "products")
print(rows[-1]["sku"], rows[-1]["name"])
```

```
3 pages, 42 products
ACC-1032 Larkspur Ridge Mesa Multitool
```

11 **Check before you trust.** Say: "Scraped data deserves more suspicion than a file someone handed you. Three questions: does the count match what the page claims, is the key unique, do the values look plausible?"

```
claimed = int(re.search(r"\d+", packs.select_one("p.result-count").get_text())
print("claimed", claimed, "| scraped", len(packs_df), "| match", claimed == le
print("unique SKUs:", packs_df["sku"].is_unique)
print(packs_df["price"].describe().round(2)[["min", "50%", "max"]].to_dict())
```

```
claimed 24 | scraped 24 | match True
unique SKUs: True
{'min': 20.95, '50%': 118.95, 'max': 335.95}
```

12 **A real table? read_html.** Say: "When the data is already a `<table>`, skip the selectors. Fetch it yourself so your User-Agent and timeout apply, wrap the bytes, and pick the table by its id, not its position."

```
import io

resp = requests.get(urljoin(BASE_URL, "price-list.html"), headers=HEADERS, tim
resp.raise_for_status()
hours = pd.read_html(io.BytesIO(resp.content), encoding="utf-8",
                     attrs={"id": "store-hours"})[0]
print(hours)
```

```
         City State     Open daily           Phone
0       Boise    ID   9 am to 8 pm  (208) 555-0142
1        Bend    OR   9 am to 7 pm  (541) 555-0118
2  Flagstaff    AZ  10 am to 7 pm  (928) 555-0167
3  Asheville    NC  10 am to 8 pm  (828) 555-0193
```

13 **Two errors you will see today.** Say: "Read the type in the message. NoneType means your search found nothing. ResultSet means you have a LIST and treated it as one element: Beautiful Soup even tells you which method you probably meant."

```
for code in ['packs.select_one("p.sale-price").get_text()',
             'packs.select("p.price").get_text()']:
    try:
        eval(code)
    except AttributeError as err:
        print(str(err).split(". ")[0])
```

```
'NoneType' object has no attribute 'get_text'
ResultSet object has no attribute "get_text"
```

THREE MISTAKES TO EXPECT

- `AttributeError: 'NoneType' object has no attribute 'get_text'`. The selector found nothing (usually a typo in a class name, or a class that the card does not have). Coach: "How many does `len(soup.select(your_selector))` find? Now Inspect the page: what is the class really called?"
- **A list treated as one element,** `soup.select("p.price").get_text()`. Beautiful Soup's own message says "You're probably treating a list of elements like a single element". Coach: "Do you want one, or all of them? One: `select_one`. All: a loop."
- `KeyError` **on an optional attribute, "fixed" with** `try/except: pass`. That hides the product. Coach: "What should a missing rating MEAN in your table?" Then `.get()` and `None`. Keep square brackets for fields that must exist (SKU, price) so those crash loudly.

Also common: `class=` instead of `class_=` (a `SyntaxError`), and requesting a relative `href` (`MissingSchema`: use `urljoin`).

GUIDED LAB (25 MINUTES)

Lab 6.2-A, everyone: `labs/module-17/l6_2-beautifulsoup/sleeping_bag_shortlist_starter.py` (reference: `sleeping_bag_shortlist_solution.py`). Question: a customer wants a sleeping bag for $260 or less that can be bought today, as light as possible. Which bag, and what do we not know?

- Minutes 0 to 3: run it as shipped. The listing prints with `None` for price, rating and stock: that is TODO 2.
- Minutes 3 to 15: TODO 2 (finish `parse_card()`: `data-price`, `.get("data-rating")`, the second stock class) and TODO 3 (the shortlist).
- Minutes 15 to 22: TODO 4 (the specs table as a dictionary) and TODO 5 (clean weight and temperature). The lab fetches only the shortlisted detail pages: 6, not 24. Ask why that matters.
- Minutes 22 to 25: results. 24 cards (the page claims 24); 12 sleeping bags, 6 at $260 or less and available; lightest SLP-1002 Juniper Vane Foothill 30F, 0.81 kg, $132.95; SLP-1019 shows `Not listed` for weight.
- Ask: "Is SLP-1019 the heaviest bag?" (We do not know. Missing is 'unknown', not 'zero' and not 'heaviest'. `sort_values` puts NaN last, which is not the same as saying it is heavy.)

Lab 6.2-B, for pairs who finish (or homework): `labs/module-17/l6_2-beautifulsoup/price_list_tables_starter.py`. `read_html` on bytes, both tables chosen by id, availability by category (Packs has the most out of stock: 3), the 8 unrated products, store hours (Boise open longest, 11 hours) and a card-by-card cross-check of the Accessories rows (32 rows, 32 cards, prices and ratings agree).

START THE LESSON PROJECT (10 MINUTES)

Open the lesson at "Project: Category Price Report". Read the brief, then type Steps 1 and 2: the nine category URLs come from the front page's links (no typed list), and one category page is inspected before looping (24 cards, a different count line, no pagination). Stop there and ask: "Why does Step 2 matter before Step 3?" (The count line has a different wording, which would break the check.) Steps 3 to 5 are homework; the target is a nine-row `category_report.csv` sorted by median price, Tents first at $298.45.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q1 to Q6. Q1 is the hinge question: ask it right after demo step 13.

Answers: **Q1** C) `soup.select_one("p.sale-price")`; **Q2** `['A1', None]`; **Q3** A) On the last page "Next" is a plain `span`, so `select_one("nav.pagination a.next")` returns `None`; **Q4** D) `['stock', 'low-stock']` `low_stock`; **Q5** B) So that your User-Agent, your timeout and `raise_for_status()` apply, and pandas reads the exact bytes you checked; **Q6** `str` `1299.0`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: hand out the selector table from the workbook and the five-line `parse_card()` skeleton; in Lab A, TODO 2 has one line per field. Pair a learner who is lost in HTML with one who is comfortable in the browser's Inspect panel.

Stretch: lesson Exercise 4 (the `application/ld+json` block) or the bonus challenge (a `get_soup()` that caches pages in `raw/`, reusing Session 16's folder).

6.3 Dynamic Pages with Selenium

75 min

OBJECTIVES (FROM THE LESSON)

- Tell a static page from one that JavaScript fills in, and explain why `requests` sees no data on the second kind
- Find the JSON a dynamic page loads and use it directly when you can
- Start headless Chrome with Selenium 4, open a page, and locate elements with `By` locators
- Wait for content with `WebDriverWait` and expected conditions instead of `time.sleep()`
- Click buttons, fill in a form, take a screenshot, and always quit the browser, even after an error

TIMING (75 MINUTES)

12m	Frame: flat-pack furniture; the order of the routes
25m	Live demo, steps 1 to 12
25m	Guided lab: the budget shelf (then page triage)
13m	Start Deals Watch: Steps 1 and 2

MENTAL MODEL TO SAY

"A static page arrives like a **finished bookcase**: what comes off the truck is what you see. A dynamic page arrives **flat-packed**: a nearly empty frame (the HTML), a bag of parts to be fetched (data files), and an instruction sheet (JavaScript). Your browser does the assembly after delivery. `requests` is a delivery driver, not a carpenter: it hands you the box exactly as shipped and never opens the instructions."

Then draw the decision on the board: **visible in the browser → in the raw HTML? Beautiful Soup. → No: does the page load a data file? Request it. → No: drive a browser.** "We will collect the deals both ways today, because you will check the browser's answer against the file."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `session17_selenium`. One browser stays open from step 3 to step 12; if the demo is interrupted, run `driver.quit()` before anything else. On your projector machine Chrome is found automatically; this kit's run used a Chromium program set with two extra options that learners do not need.

1. **The page that comes back empty.** Say: "In the browser the Deals page shows eight offers. Watch what requests receives." After the output: "The request worked. The list is there. It is empty, next to a message saying the deals are loading. They never will, in this program."

```
import re
import time
from urllib.parse import urlencode, urljoin

import pandas as pd
import requests
from bs4 import BeautifulSoup

BASE_URL = "https://rays-python-datascience.netlify.app/practice/shop/"
USER_AGENT = "pinecrest-course-bot/1.0 (learning project; contact: you@example
HEADERS = {"User-Agent": USER_AGENT}
DEALS_URL = urljoin(BASE_URL, "deals.html")

resp = requests.get(DEALS_URL, headers=HEADERS, timeout=10)
resp.raise_for_status()
soup = BeautifulSoup(resp.content, "html.parser")
print(len(soup.select("li.deal")), "deals in the HTML")
print(soup.select_one("#loading").get_text())
print(soup.select_one("#deal-list"))
```

```
0 deals in the HTML
Loading deals…
<ul class="product-grid" id="deal-list"></ul>
```

2 **Look for the data first.** Say: "Flat-pack furniture: the HTML is the frame, the data is a bag of parts, the JavaScript is the instruction sheet. Before automating a browser, find out what the page loads. In Chrome: F12, Network, Fetch/XHR, reload. In Python: read the script."

```
script = requests.get(urljoin(BASE_URL, "shop.js"), headers=HEADERS, timeout=1
script.raise_for_status()
print(re.findall(r"fetch\('(.+?)'\)", script.text))

resp = requests.get(urljoin(BASE_URL, "deals.json"), headers=HEADERS, timeout=
resp.raise_for_status()
deals_json = pd.DataFrame(resp.json()["deals"])
print(deals_json.shape)
```

```
['deals.json', 'products.json']
(24, 6)
```

3 **Start a browser from Python.** Say: "Selenium drives a real Chrome. Selenium Manager finds Chrome and fetches the driver the first time (half a minute, once). Headless means no window; a fixed window size means the same layout every run; and the browser says who it is, like our requests code does."

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By

options = Options()
options.add_argument("--headless=new")
options.add_argument("--window-size=1000,800")
options.add_argument(f"--user-agent={USER_AGENT}")
driver = webdriver.Chrome(options=options)
print(driver.name)
```

```
chrome
```

4 **Ask too early.** Say: "`get()` returns when the page's files are loaded. The deals arrive a second and a half later. We are in a race with the page." Point at the zero.

```
driver.get(DEALS_URL)
print(driver.title)
print(len(driver.find_elements(By.CSS_SELECTOR, "li.deal")), "deals right afte
```

```
Deals | Pinecrest Outfitters (practice site)
0 deals right after get()
```

5 **Wait for the thing you need, by name.** Say: "Never sleep. Say what you are waiting for and how long you will give it. Selenium checks twice a second and continues the moment it is true." Write the double parentheses on the board: the locator is ONE tuple.

```
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

wait = WebDriverWait(driver, 10)
first = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "#deal-lis
print(len(driver.find_elements(By.CSS_SELECTOR, "li.deal")), "deals now")
print(driver.find_element(By.ID, "deal-status").text)
```

```
8 deals now
Showing 8 of 24 deals
```

6 **Reading a live element.** Say: "A WebElement is a handle on a live element. `.text` is what a person sees; `get_attribute` reads an attribute. The browser has already resolved the link to a full URL."

```
print(first.get_attribute("data-sku"))
link = first.find_element(By.CSS_SELECTOR, ".product-name a")
print(link.text)
print(link.get_attribute("href"))
```

```
FTW-1013
Juniper Vane Talus Hiking Boots
https://rays-python-datascience.netlify.app/practice/shop/products/FTW-1013.ht
```

7 **Let the browser render, let Beautiful Soup parse.** Say: "Each find_element is a round trip to the browser. Division of labor: Selenium does what only a browser can, then we take the finished HTML and use the tool we already know."

```
def parse_deals(html):
    """Parse the deal cards in a rendered deals page into a DataFrame."""
    soup = BeautifulSoup(html, "html.parser")
    return pd.DataFrame([{
        "sku": li["data-sku"],
        "price": float(li.select_one("s.was").get_text().lstrip("$")),
        "deal_price": float(li.select_one("span.deal-price").get_text().lstrip
    } for li in soup.select("#deal-list li.deal")])


print(parse_deals(driver.page_source).head(3))
```

```
       sku   price  deal_price
0  FTW-1013  155.95      132.56
1  ACC-1017   14.50       11.60
2  KIT-1003   24.95       18.71
```

8 **Clicking, one wait per click.** Say: "A person would click Load more until it disappears. After each click we wait until there are MORE deals than before: a custom condition, a tiny function that receives the driver."

```
load_more = driver.find_element(By.ID, "load-more")
while load_more.is_displayed():
    shown = len(driver.find_elements(By.CSS_SELECTOR, "li.deal"))
    load_more.click()
    wait.until(lambda d: len(d.find_elements(By.CSS_SELECTOR, "li.deal")) > sh
    print(driver.find_element(By.ID, "deal-status").text)

all_deals = parse_deals(driver.page_source)
print(all_deals.shape, "| same SKUs as deals.json:",
      all_deals["sku"].tolist() == deals_json["sku"].tolist())
```

```
Showing 16 of 24 deals
Showing 24 of 24 deals
(24, 3) | same SKUs as deals.json: True
```

9 **Filling in a form.** Say: "Form fields are easiest to find by their name. `Select` wraps a drop-down. Submitting loads a NEW page, so two waits: the address changes, then the new results finish (the page marks its list with data-loaded)."

```
from selenium.webdriver.support.ui import Select

ready = EC.presence_of_element_located(
    (By.CSS_SELECTOR, '#search-results[data-loaded="true"]'))
driver.get(urljoin(BASE_URL, "search.html"))
wait.until(ready)
Select(driver.find_element(By.NAME, "category")).select_by_visible_text("Packs
driver.find_element(By.NAME, "max_price").send_keys("100")
driver.find_element(By.CSS_SELECTOR, "#search-form button[type='submit']").cli
wait.until(EC.url_contains("category=Packs"))
wait.until(ready)
print(driver.current_url.split("?")[1])
print(driver.find_element(By.ID, "search-count").text)
```

```
q=&category=Packs&max_price=100
11 products found
```

10 **The address bar is telling you something.** Say: "The form uses GET, so filling it in is a slow way of building a URL. Build it yourself." Ask: "And where does this page get its data?" (products.json: no browser needed at all.)

```
query = urlencode({"q": "stove", "in_stock": 1})
driver.get(urljoin(BASE_URL, "search.html") + "?" + query)
wait.until(ready)
print(driver.find_element(By.ID, "search-count").text)
print([li.get_attribute("data-sku")
       for li in driver.find_elements(By.CSS_SELECTOR, "#search-results li")])
```

```
4 products found
['KIT-1001', 'KIT-1009', 'KIT-1017', 'KIT-1025']
```

11 **Two errors to read on purpose.** Say: "Selenium's tracebacks are long; read the first line. NoSuchElement right after get(): not YET. A TimeoutException: the thing never came, often a typo. Give every wait a message so the error says what you were waiting for."

```
from selenium.common.exceptions import NoSuchElementException, TimeoutExceptio

driver.get(DEALS_URL)
try:
    driver.find_element(By.CSS_SELECTOR, "li.deal")
except NoSuchElementException as err:
    print(type(err).__name__, "|", err.msg.split(":")[0])
try:
    WebDriverWait(driver, 3).until(
        EC.presence_of_element_located((By.CSS_SELECTOR, "li.deals")),
        message="no li.deals after 3 seconds")
except TimeoutException as err:
    print(type(err).__name__, "|", err.msg)
```

```
NoSuchElementException | no such element
TimeoutException | no li.deals after 3 seconds
```

12 **You own a running program: quit it.** Say: "driver is a real Chrome process. It runs until you quit it, even after your script crashes. So: one function that starts it, and try/finally around the work, every time."

```
driver.quit()


def make_driver():
    """Start Chrome with this course's settings. Whoever calls this must quit
    options = Options()
    options.add_argument("--headless=new")
    options.add_argument("--window-size=1000,800")
    options.add_argument(f"--user-agent={USER_AGENT}")
    return webdriver.Chrome(options=options)


driver = make_driver()
try:
    driver.get(DEALS_URL)
    WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.CSS_SELECTOR, "li.deal")))
    print(driver.find_element(By.ID, "deal-status").text)
finally:
    driver.quit()
    print("browser closed")
```

```
Showing 8 of 24 deals
browser closed
```

THREE MISTAKES TO EXPECT

- `time.sleep()` **to make it work.** It passes in class and fails at home on a slow connection. Coach: "What exactly are you waiting for? Name it, and wait for that."
- **Missing the inner parentheses** in `EC.presence_of_element_located((By.ID, "x"))`. The error is a `TypeError` about arguments. Coach: point at the board: the locator is ONE tuple.
- **Orphaned browsers.** A crash before `quit()` leaves Chrome running; after a few runs the laptop is slow. Coach: "Where is your `finally`?" On Windows, Task Manager shows them; on macOS and Linux, Activity Monitor or `ps`.

Also common: `NoSuchElementException` right after `get()` (not yet: wait), a `TimeoutException` caused by a selector typo (give every wait a `message=`), `StaleElementReferenceException` after a reload, and old tutorials' `find_element_by_id` (removed in Selenium 4).

GUIDED LAB (25 MINUTES)

Lab 6.3-A, everyone (pairs where a Chrome failed): `labs/module-17/l6_3-selenium/budget_search_starter.py` (reference: `budget_search_solution.py`). Question:

marketing wants a "Gear under $50" shelf. Which products cost $50 or less and can be bought today, by category?

- **How it reaches a browser.** One function, `rendered()`, runs your browser steps and always quits Chrome. With `LABS_OFFLINE=1`, or when Chrome cannot start, it prints a clear line and parses the saved finished page from `_fixtures/` instead, so the rest of the lab works for everyone.
- Minutes 0 to 15: TODO 1 (the query string with `urlencode`), TODO 2 (the wait: no sleep), TODO 3 and 4 (parse the price text, read the count line).
- Minutes 15 to 25: TODO 5 (the same search in `products.json` with pandas), then the comparison. Reference answers: through `requests` the page has 0 results and says `Loading…`; the finished page shows 60 rows but its count line says **73 products found**; `products.json` has 73 products at $50 or less and available; the 13 not shown are all Accessories.
- The discussion: "The page shows only the first 60 matches. Who would have reported 60?" The page's own count line is what caught it, and the data file is what answered the question.

Lab 6.3-B (finish-if-fast, or homework): `labs/module-17/l6_3-selenium/page_triage_starter.py`. Three pages, raw HTML versus finished page: Climbing category 24 / 24 (static), Deals 0 / 8 and Search 0 / 24 (dynamic: request the data file first); the Climbing search done through the form returns the same 24 SKUs as the category page.

START THE LESSON PROJECT (13 MINUTES)

Open "Project: Deals Watch". Step 1 fetches `deals.json` first and keeps it as the answer key (24 rows). Step 2 writes `collect_deals()`: wait, click Load more with a limit, screenshot, parse, and quit in `finally`. Learners without a working browser do Step 1, then read Step 2 with their partner. Steps 3 to 5 (verify, the business question, the comparison of the two routes) are homework.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q7 to Q12. Q9 is the hinge question: show the three wait lines on the projector and vote.

Answers: **Q7** B) Find the data file the page loads (Network tab, or read its script) and, if you may, request it directly; **Q8** `q=&category=Snow&max_price=&in_stock=1`; **Q9** C) `WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, "deal-status")))`; **Q10** D) The OLD page's results are already marked loaded, so the second wait alone could pass instantly, before the new page arrives; **Q11** A) The handle pointed at an element in the page that was replaced; find the element again after anything changes the page; **Q12** D) The page's HTML as it is now, after its scripts ran: all 24 deals are in it. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the browser parts of Lab A are three lines; let a struggling pair do the `products.json` route (TODO 5) first, so they have the answer before they automate anything.

Stretch: the lesson's bonus challenge (save `failure.png` and `failure.html` when a wait times out), or lesson Exercise 3 (the form, the human way).

CLOSING

EXIT TICKET (3 QUESTIONS, ON A STICKY NOTE; ANSWERS)

1. Which Beautiful Soup method returns `None` when nothing matches, and which returns an empty list? **Answer:** `select_one()` (and `find()`) return `None`; `select()` (and `find_all()`) return a list, empty if nothing matches.
2. Why is `WebDriverWait(...).until(...)` better than `time.sleep(5)`? **Answer:** It continues the moment the condition is true and raises a clear `TimeoutException` if it never is. A sleep is always too short or too long.
3. Before you start a browser for a page that JavaScript fills in, what do you look for first? **Answer:** The data file or API the page loads (Network tab, or the page's script). If you may request it, do that instead.

If more than a third miss question 3, open Session 18 with the decision drawn on the board again: it is the same question APIs answer.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- Lesson 6.2: finish the Category Price Report (Steps 3 to 5), the four exercises, the quiz and the journal.
- Lesson 6.3: finish Deals Watch (Steps 3 to 5), the four exercises, the quiz and the journal.
- Kit labs not finished in the room: `price_list_tables_starter.py` and `page_triage_starter.py`.
- Check that no stray browsers are running.

BRING LIST FOR SESSION 18 (APIS: KEYS, TOKENS AND POLITE CLIENTS)

- **The practice API, with flags.** From the project folder: `python data/mock_api.py --token-ttl 5` (Lesson 6.4: tokens expire after five seconds so you can watch one expire). Lesson 6.5 restarts it as `python data/mock_api.py --rate 10 --per 5`. Check that it starts at home: `http://127.0.0.1:8000/health` shows `{"status": "ok"}`.
- **The three environment variables** (`PINECREST_API_KEY`, `PINECREST_USER`, `PINECREST_PASSWORD`), set with the commands your instructor showed, in the terminal you start Jupyter or VS Code from.
- **Checkpoint 2 (15 minutes, open notes) is in Session 18:** one SQL query against `data/pinecrest.db` and one authenticated, paginated pull from the practice API into a DataFrame, answering one small question. Review Lesson 5.1 (opening the database read-only, `pd.read_sql_query`) and Lesson 6.1 (`params`, `raise_for_status`).
- Your 6.2 and 6.3 notebooks. Nothing new to install. Accounts: none.

IF YOU RUN LONG OR SHORT

Running long. By 1:15 you should be starting the 6.2 project; if not, skip it (Step 1 is three lines of homework) and take the break on time. In 6.3, steps 9 and 10 can be shown from the guide's printed output; Lab B becomes homework. Never cut step 12 (`quit()` in `finally`). If the venue network blocks the driver download, move the whole second half to the JSON route and the labs' saved pages, and demo the browser from your own machine.

Running short. Run Lab 6.2-B in the room and compare the price-list total (29,475.90) with the card scrape. In 6.3, do lesson Exercise 4 together (the search without a browser: 9 tents) and discuss "which would you rather maintain?" Or open a real public website's Network tab together (look only, no scraping) and count how much of the page arrives as JSON.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 18: APIs: Keys, Tokens and Polite Clients

How to teach the session that closes Module 6. Before the break, API keys and bearer tokens: secrets from the environment, 401 versus 403, logging in, expiry and renewing once, pagination into a DataFrame. After the break, the habits of a welcome client: reading the limit, pacing, honoring 429, backoff with jitter, retry policies, caching and resumable jobs. Then Checkpoint 2, a 15-minute data-access task on Modules 5 and 6.

`2 lessons` `3 hrs session` `Lessons 6.4-6.5` `Checkpoint 2` `Session 18 of 21`

HOW TO USE THIS GUIDE

- Session 18 closes Module 6 with the front door: APIs. Before the break, keys and tokens (Lesson 6.4); after it, the habits that make a client welcome and dependable (Lesson 6.5). Then **Checkpoint 2**, 15 minutes, on data access across Modules 5 and 6.
- Same rhythm as every session: frame → live demo → guided lab → start the project. The checkpoint takes 15 minutes and its debrief 10, so the lesson blocks are a little shorter than usual and both projects are mostly homework.
- Every code block below was run for this guide against `data/mock_api.py`: with `--token-ttl 5` for 6.4 and `--rate 10 --per 5` for 6.5, with the three demo environment variables set. Tokens, and some timings, will differ on your screen; the guide says where.
- The practice shop is not used today. Everything happens on `http://127.0.0.1:8000`, a server on each learner's own computer.
- Labs and the checkpoint live in `labs/module-18/`. With `LABS_OFFLINE=1` they replay real saved responses and run their waits on a pretend clock.

> **THE RISK IN SESSION 18: SECRETS ON SCREENS, LOOPS THAT HAMMER**
>
> Two habits to make automatic today. **Secrets never appear in code, output or logs**: the projector itself is a test (never print a key; print a hint). **Every retry has a reason, a wait and a limit**: retry 429, 5xx and timeouts; never 400, 403 or 404; renew an expired token once. A loop that retries a 401 forever looks like an attack.

BEFORE THE SESSION

- ☐ Run `python3 labs/check_labs.py module-18` from the kit folder. Expect `10 passed, 0 failed` (offline; the checker sets the three variables to the demo values).
- ☐ Copy the starters (including `checkpoint-2/checkpoint2_starter.py`), `_fixtures/` and `labs_paths.py` to the shared folder. Keep every `*_solution.py` on your machine; the checkpoint solution is the model answer.
- ☐ **The practice API, with the flags of each lesson.** Lesson 6.4: `python data/mock_api.py --token-ttl 5` (tokens last five seconds so learners can watch one expire). Lesson 6.5: stop it with Ctrl+C at the break and start `python data/mock_api.py --rate 10 --per 5` (ten requests every five seconds, so the limit is easy to hit). The checkpoint works with either. Check: `http://127.0.0.1:8000/health` shows `{"status": "ok"}`, and the start line names the limit, for example `(limit: 10 requests per 5 s; Ctrl+C to stop)`.
- ☐ **Why restart at the break:** the 6.5 demos need the small limit to show a 429 within seconds, and the 6.4 token demos need short tokens. With default flags (30 per 10 s, 60-second tokens) nothing goes wrong, it just takes too long to see.
- ☐ **The three environment variables** on the projector machine, set before starting Jupyter or VS Code from the same terminal (commands below). Test with the first demo cell: it prints `key loaded: pine... (18 characters)`.
- ☐ **The practice shop:** not used today. If a learner asks, it is at `https://rays-python-datascience.netlify.app/practice/shop/`, with the offline copy served from `data/pinecrest-practice-site.zip` on port 8001 (Session 16).
- ☐ **Chrome and Selenium:** not used today (Session 17). **MongoDB:** nothing needed (Session 16 ran on mongomock).
- ☐ Print the Checkpoint 2 task sheet from the workbook, or keep it on the projector during the checkpoint.
- ☐ Whiteboard: "**401: who are you? 403: no.**" and "**Retry: a reason, a wait, a limit.**"

THE THREE ENVIRONMENT VARIABLES (PUBLIC DEMO VALUES FOR THE PRACTICE API)

```
# Windows PowerShell (this terminal only)
$env:PINECREST_API_KEY = "pinecrest-demo-key"
$env:PINECREST_USER = "analyst"
$env:PINECREST_PASSWORD = "trailmix"

# macOS and Linux (this terminal only)
export PINECREST_API_KEY="pinecrest-demo-key"
export PINECREST_USER="analyst"
export PINECREST_PASSWORD="trailmix"
```

- **The trap:** variables set this way exist only in that terminal and in programs started from it afterward. A notebook started earlier, or from the Start menu, does not see them: close it, set the variables, start it again from that terminal. The demo's `require_env()` turns this into a clear message instead of a mystery.
- **Other routes (Lesson 6.4):** `getpass.getpass("API key: ")` to type a secret into a hidden box, or a `.env` file read by `python-dotenv`, which is safe only if `.env` is listed in `.gitignore` before the first commit.
- These are teaching values for a server on your own computer. Real keys are personal: never share one on a slide or in a chat.

SESSION 18 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 17 (Beautiful Soup, Selenium), no notes. Homework check; are the three environment variables set?
0:10	2	Ground rules opener	The six Module 6 rules, read aloud. Today's angle: keys are personal, limits are the owner's rule.
0:12	10	**6.4** Frame and mental model	The festival wristband: a key at the gate, a timed wristband inside. 401 versus 403.
0:22	23	**6.4** Live demo	Secrets from the environment, the key in a header, `ApiError`, POST `/login`, Bearer, expiry, renew once, pagination.
0:45	22	**6.4** Guided lab	`labs/module-18/l6_4-api-authentication/cancelled_orders_starter.py`, then `token_lifecycle_starter.py`.
1:07	8	**6.4** Start the lesson project	December Returns Brief: Steps 1 and 2 (`client.me()`, the probe).
1:15	10	Break	Restart the practice API as `python data/mock_api.py --rate 10 --per 5`; everyone does the same.
1:25	10	**6.5** Frame and mental model	The freeway on-ramp light: politeness (pace, obey, never ask twice) and resilience (retry what is temporary).
1:35	23	**6.5** Live demo	Rate-limit headers, a burst into 429, a Throttle, Retry-After, backoff with jitter, `/flaky`, urllib3 `Retry`, a cache, a resumable job.
1:58	20	**6.5** Guided lab	`labs/module-18/l6_5-rate-limiting/stock_check_starter.py`, then

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
			`retry_and_cache_starter.py`.
2:18	7	**6.5** Start the lesson project	The Resumable Order Pull: Step 1 (plan before you pull).
2:25	15	Checkpoint 2	Open notes, own work: one SQL query on `pinecrest.db` and one authenticated, paginated API pull. `labs/module-18/checkpoint-2/checkpoint2_starter.py`.
2:40	10	Checkpoint debrief and Module 6 recap	The answer on the projector; three common slips; the module in one picture.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 19 (stop the API; check seaborn and Plotly).
Total	180		Ends at 3:00.

In the room: the ground rules, both demos, Lab 6.4-A and Lab 6.5-A, the first step of each project, Checkpoint 2 and its debrief. **At home (about 2 to 3 hours):** both lesson projects (December Returns Brief, Resumable Order Pull), the exercises, quizzes and journals, and the B labs. The lessons estimate 90 + 95 minutes solo; the room has 133 minutes for them after the warm-up, opener, checkpoint, debrief and wrap-up.

Protect: Checkpoint 2 (start it at 2:25 whatever happens); 6.4 demo steps 1, 3, 6 to 9; 6.5 demo steps 2, 3, 4, 7. **Cut first if long:** 6.5 steps 9 to 11 (cache, logging, resumable: the project teaches them), then 6.4 steps 12 and 13, then the 6.5 project start. Never cut the debrief to under 5 minutes: it is where the checkpoint becomes learning.

WARM-UP

0:00 Warm-up: Retrieval from Session 17

10 min

Five questions, notes closed, then answers aloud.

1. Which Beautiful Soup method returns `None` when nothing matches? **Answer:** `select_one()` (and `find()`). `select()` and `find_all()` return a list, empty when nothing matches.
2. Why does `get_soup()` pass `resp.content`, not `resp.text`? **Answer:** From bytes, Beautiful Soup reads the page's own `<meta charset>`, so the text decodes correctly whatever the server said.
3. How does a pagination loop know it has reached the last page of the catalog? **Answer:** There is no `a.next` link on the last page, so `select_one("nav.pagination a.next")` returns `None`.
4. `requests` sees an empty list where the browser shows deals. What do you try before Selenium? **Answer:** Find the data file the page loads (Network tab or the script) and request it directly, if allowed.
5. Why `WebDriverWait(...).until(...)` instead of `time.sleep()`, and where does `driver.quit()` go? **Answer:** The wait continues as soon as the condition is true and raises clearly if it never is. `quit()` goes in a `finally` block.

Homework check (3 minutes). Category Price Report: nine rows, Tents first. Deals Watch: 24 rows equal to `deals.json`. Then: "Who has the three variables set and the API started with `--token-ttl 5`?" Fix the rest now, while the room does question 5.

GROUND RULES OPENER (2 MINUTES)

0:10 The Six Ground Rules for Module 6

2 min

Six learners, one rule each, aloud. Today's sentence: "An API is the front door rule 3 asked for. It comes with its own rules: a key that is yours alone, and a limit the owner sets. Keep both."

1. **Read `robots.txt` and obey it.** It is the owner's published request to programs: where they may go and how fast.
2. **Read the terms of use.** If they forbid automated collection, the answer is no, however easy the code would be.
3. **Prefer an API or a download** when one exists. It is the front door.
4. **Go slowly and say who you are.** Pause between requests; send a User-Agent with a way to contact you.
5. **Never go around a login, a paywall or a block.** If you are asked to stop, stop.
6. **Do not collect personal data.** Names, emails, profiles and photos of real people are protected by law in many places.

6.4 API Authentication

63 min

OBJECTIVES (FROM THE LESSON)

- Send an API key in a request header and tell a 401 from a 403
- Keep keys and passwords out of your code with environment variables
- Log in for a bearer token, use it, and recover when it expires
- Turn an API's error responses into clear Python exceptions, and decide by status code what to do next
- Follow pagination to the last page and build a DataFrame, using a small reusable client class

TIMING (63 MINUTES)

10m	Frame: the wristband; 401 versus 403; where secrets live
23m	Live demo, steps 1 to 13
22m	Guided lab: cancelled orders (then the token lifecycle)
8m	Start the December Returns Brief: Steps 1 and 2

MENTAL MODEL TO SAY

"Think of a festival. At the gate you show your **ticket** once (your username and password). They give you a **wristband** (the token) and you wear it everywhere inside: no one asks for the ticket again. The wristband is dated: tomorrow it does not work, and you go back to the gate. Some stalls only want to see your **membership card** (the API key) every time. And two different refusals: 'I cannot see a wristband' is 401; 'I see it, and you are not allowed in here' is 403."

Draw the decision table from the lesson on the board: 400 fix the code, 401 renew once or stop, 403 stop, 404 stop or record, 429 wait and retry, 5xx wait and retry a few times.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `session18_auth`, with the practice API started as `python data/mock_api.py --token-ttl 5`. Step 8 and step 9 each wait six seconds on purpose: talk while they run.

1. **Secrets come from the environment.** Say: "Before any request: where does the key live? Not in this notebook. You set three variables in the terminal before starting Jupyter. When you need to confirm a secret loaded, print a HINT of it, never the value: output cells are saved with the notebook."

```
import os
import time

import pandas as pd
import requests

API = "http://127.0.0.1:8000"
HEADERS = {"User-Agent": "pinecrest-course-bot/1.0 (learning project; contact:


def require_env(name):
    """Read a required setting from the environment, or stop with a clear mess
    value = os.environ.get(name)
    if not value:
        raise RuntimeError(f"Set the environment variable {name} before runnin
    return value


API_KEY = require_env("PINECREST_API_KEY")
USERNAME = require_env("PINECREST_USER")
PASSWORD = require_env("PINECREST_PASSWORD")
print("key loaded:", API_KEY[:4] + "...", f"({len(API_KEY)} characters)")
```

```
key loaded: pine... (18 characters)
```

2. **A variable that is not set.** Say: "This is the message a learner sees when Jupyter was started from a different terminal. It says exactly what to do."

```
require_env("PINECREST_REPORTS_KEY")
```

```
RuntimeError: Set the environment variable PINECREST_REPORTS_KEY before runnir
```

3. **401 and 403 are different answers.** Say: "No key: 401, 'I cannot tell who you are'. A wrong key: 403, 'I understood, and the answer is no'. Sending a 403 again never helps." Ask before running the second request: "Which status?"

```
r = requests.get(f"{API}/orders", headers=HEADERS, timeout=10)
print(r.status_code, r.json()["error"]["code"])
r = requests.get(f"{API}/orders", headers={**HEADERS, "X-API-Key": "pinecrest-
                 timeout=10)
print(r.status_code, r.json()["error"]["code"])
```

```
401 missing_api_key
403 invalid_api_key
```

4 **The key in a header, never in the URL.** Say: "The documentation says where the key goes: a header named X-API-Key. URLs end up in logs, browser history and bug reports; headers mostly do not."

```
KEY_HEADERS = {**HEADERS, "X-API-Key": API_KEY}
r = requests.get(f"{API}/orders", headers=KEY_HEADERS,
                 params={"status": "cancelled", "since": "2025-12-15", "per_pa
                 timeout=10)
data = r.json()
print(r.status_code, {k: v for k, v in data.items() if k != "items"})
print([(o["order_id"], o["store"], o["order_total"]) for o in data["items"]])
```

```
200 {'page': 1, 'per_page': 3, 'total': 15, 'total_pages': 5, 'next': '/orders
[(119812, 'Online', 81.9), (119840, 'Online', 53.9), (119860, 'Online', 62.9)]
```

5 **Errors you can act on.** Say: "raise_for_status throws the API's explanation away. A tiny exception class keeps the status, the code and the message, so code can branch on err.code."

```
class ApiError(Exception):
    """An error response from the API, with its status, code and message."""
    def __init__(self, status, code, message):
        super().__init__(f"{status} {code}: {message}")
        self.status, self.code, self.message = status, code, message


def check(resp):
    """Return the response if it succeeded. Otherwise raise ApiError."""
    if resp.ok:
        return resp
    try:
        error = resp.json()["error"]
        code, message = error["code"], error["message"]
    except (requests.exceptions.JSONDecodeError, KeyError, TypeError):
        code, message = "http_error", resp.reason
    raise ApiError(resp.status_code, code, message)


check(requests.get(f"{API}/orders", headers=KEY_HEADERS,
                   params={"since": "Dec 1 2025"}, timeout=10))
```

```
ApiError: 400 bad_request: 'since' must look like 2025-01-31, got 'Dec 1 2025
```

6 **Your first POST: log in for a token.** Say: "GET reads; POST sends. json= encodes the dictionary as a JSON body. The password crosses the network once; what you carry afterward stops working on its own. We started the API with --token-ttl 5, so this one lasts five seconds."

```
resp = check(requests.post(f"{API}/login",
                           json={"username": USERNAME, "password": PASSWORD},
                           headers=HEADERS, timeout=10))
body = resp.json()
print(list(body))
print(body["token_type"], "token, valid for", body["expires_in"], "seconds")
token = body["access_token"]
print(token[:6] + "...  (your token will differ)")
```

```
['access_token', 'token_type', 'expires_in']
Bearer token, valid for 5 seconds
clP_eI...  (your token will differ)
```

7 **Bearer means bearer.** Say: "The header is the word Bearer, a space, the token. Whoever bears it gets in, no other questions: treat it like a key."

```
auth = {**HEADERS, "Authorization": f"Bearer {token}"}
me = check(requests.get(f"{API}/me", headers=auth, timeout=10)).json()
print(me["username"], "|", me["name"], "|", me["role"])
```

```
analyst | Demo Analyst | analyst
```

8 **The wristband runs out.** Say: "Wait six seconds and use the same token. Nothing is wrong with your account. This is the normal life of a token, and your code must expect it in the middle of a job."

```
time.sleep(6)
try:
    check(requests.get(f"{API}/me", headers=auth, timeout=10))
except ApiError as err:
    print(err.status, err.code)
    print(err.message)
```

```
401 token_expired
The token has expired. Log in again with POST /login.
```

9 **Renew and retry ONCE.** Say: "Recover from token_expired by logging in again and repeating the request one time. Anything else, and a second failure, goes up to the caller. A loop that retries a 401 forever looks like an attack and gets accounts locked."

```
def login():
    resp = check(requests.post(f"{API}/login", timeout=10, headers=HEADERS,
                               json={"username": USERNAME, "password": PASSWOR
    return resp.json()["access_token"]


def get_me(token):
    try:
        return check(requests.get(f"{API}/me", timeout=10,
                                  headers={**HEADERS, "Authorization": f"Beare
    except ApiError as err:
        if err.code != "token_expired":
            raise
    print("  (token expired: logging in again)")
    return check(requests.get(f"{API}/me", timeout=10,
                              headers={**HEADERS, "Authorization": f"Bearer {l


old = login()
time.sleep(6)
print(get_me(old).json()["username"])
```

```
  (token expired: logging in again)
analyst
```

10 **One small request first: how big is the job?.** Say: "Before downloading, ask for one record with the same filter and read the total. The size of the job for the price of one request."

```
params = {"status": "returned", "since": "2025-12-15", "per_page": 1}
probe = check(requests.get(f"{API}/orders", headers=KEY_HEADERS, params=params
                           timeout=10)).json()
print(probe["total"], "orders;", probe["next"])
```

```
34 orders; /orders?page=2&per_page=1&since=2025-12-15&status=returned
```

11 **Follow next to the end.** Say: "The same loop as the catalog in Session 17, with JSON. The filters go in params for the FIRST request only: next already carries them, so we set params to None."

```
def get_all(path, params=None):
    """Collect every item from a paginated endpoint by following 'next'."""
    items, url = [], f"{API}{path}"
    while url:
        data = check(requests.get(url, headers=KEY_HEADERS, params=params,
                                  timeout=10)).json()
        items.extend(data["items"])
        url = f"{API}{data['next']}" if data["next"] else None
        params = None              # 'next' already carries the filters
    return items


late = pd.DataFrame(get_all("/orders", {"status": "returned", "since": "2025-1
                                        "per_page": 10}))
print(late.shape, "| unique ids:", late["order_id"].is_unique)
print(late.groupby("channel")["order_total"].agg(["count", "sum"]).round(2))
```

```
(34, 11) | unique ids: True
         count      sum
channel
online      28  6618.27
store        6   567.10
```

12 **Check it against the source.** Say: "As always: check what you collected. In this course you hold the answer key."

```
csv = pd.read_csv("data/orders.csv")
expected = csv[(csv["status"] == "returned") & (csv["order_date"] >= "2025-12-
print(len(expected), "expected | same ids:",
      late["order_id"].tolist() == expected["order_id"].tolist())
```

```
34 expected | same ids: True
```

13 **Two mistakes with no traceback to warn you.** Say: "Leave out the word Bearer and the server does not recognize a token at all. Send data= instead of json= and you sent a form, not JSON."

```
t = login()
r = requests.get(f"{API}/me", headers={**HEADERS, "Authorization": t}, timeout
print(r.status_code, r.json()["error"]["code"])
r = requests.post(f"{API}/login", data={"username": USERNAME, "password": PASS
                  headers=HEADERS, timeout=10)
print(r.status_code, r.json()["error"]["message"])
```

```
401 missing_token
400 The request body must be JSON.
```

THREE MISTAKES TO EXPECT

- `RuntimeError: Set the environment variable ...` because Jupyter was started before the variables were set, or from another terminal. Coach: close it, set them, start it from that terminal. Do not let anyone "fix" it by typing the key into the notebook.
- **Leaving out the word Bearer**, or using `data=` instead of `json=` for the login (`401 missing_token`; `400 The request body must be JSON.`). Coach: "Print the headers you are sending. What does the documentation say the value looks like?"
- `KeyError: 'access_token'` after a failed login. The KeyError is a symptom: the login answered 401 and the body is an error object. Coach: "What was the status of the line BEFORE?" Then `check()`.

Also common: sending `params` again on every page (filters doubled), and a retry loop around a 401 with no limit.

GUIDED LAB (22 MINUTES)

Lab 6.4-A, everyone: `labs/module-18/l6_4-api-authentication/cancelled_orders_starter.py` (reference: `cancelled_orders_solution.py`). Question: which stores and channels account for the orders cancelled since October 1, 2025, and what were they worth?

- **How the lab reaches the API.** One function, `make_http()`, decides: online, a `Session` to `http://127.0.0.1:8000` (or `PINECREST_API`); offline, the same `Session` with an adapter that replays saved responses and a pretend clock.
- Minutes 0 to 12: TODO 1 (the key from the environment, a hint, the header), TODO 3 (one probe for the size of the job), TODO 4 (follow `next`).
- Minutes 12 to 20: TODO 5 (group by store; the online share) and TODO 6 (a date the API cannot read: a 400 that explains itself).
- Reference answers: no key 401 `missing_api_key`, wrong key 403 `invalid_api_key`; 70 cancelled orders since 2025-10-01, 3 requests at 25 per page; Online 62 ($9,788.71), Flagstaff 4, Boise 3, Asheville 1; total $10,604.57; 88.6% online, against 65.5% of all orders in the same period; `orders.csv` agrees.

- Ask: "88.6% of cancellations were online. Does that mean online customers are fickle?" (Partly a base-rate effect: 65.5% of all orders are online. Compare rates, not counts.)

Lab 6.4-B (finish-if-fast, or homework): `labs/module-18/l6_4-api-authentication/token_lifecycle_starter.py`. Log in, use the token, make three requests that must fail (`missing_token`, `invalid_token`, `invalid_credentials`), watch the token expire, renew once. The journal at the end says which answer earns a second try: only `token_expired`, exactly once.

START THE LESSON PROJECT (8 MINUTES)

"Project: December Returns Brief". Learners type the `PinecrestClient` class from the lesson (or paste it from their notes: it is the lesson's code, not new material), then Step 1 (`client.me()` prints `Demo Analyst - analyst`) and Step 2 (the probe: 1,145 orders since 2025-12-01, 12 requests at 100 per page). Steps 3 to 5 are homework.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q1 to Q6. Q1 is the hinge question: ask it right after demo step 3.

Answers: **Q1** A) 401 for no key, 403 for the misspelled one; **Q2** `401 token_expired: Log in again.` then `True True`; **Q3** C) `['a', 'b', 'c'] 3`; **Q4** B) The `next` address already carries the filters; sending them again would attach every filter twice; **Q5** D) `Authorization: Bearer <token>`; **Q6** B) `print("key loaded:", API_KEY[:4] + "...")`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give the four-line pattern on a card: `require_env` → headers with the key → `check(requests.get(...))` → `.json()`. Learners who struggle with classes skip the client and use `get_all()` from the demo for the project.

Stretch: lesson Exercise 4 (a `CatalogClient` subclass) or Exercise 3 (watch a token run out).

6.5 Rate Limiting and Retries

60 min

OBJECTIVES (FROM THE LESSON)

- Read an API's rate-limit headers and pace your requests to stay inside the limit
- Respond correctly to 429 Too Many Requests by honoring `Retry-After`
- Retry temporary failures with exponential backoff and jitter, and recognize errors that must not be retried
- Configure automatic retries on a Session with urllib3's `Retry` and `HTTPAdapter`
- Cache responses on disk, make a long download resumable, and log what your job did

TIMING (60 MINUTES)

10m	Frame: the on-ramp light; politeness and resilience
23m	Live demo, steps 1 to 11
20m	Guided lab: the stock check (then retry and cache)
7m	Start the Resumable Order Pull: Step 1

MENTAL MODEL TO SAY

"Many freeway on-ramps have a light that lets one car go every few seconds. Nobody enjoys it, but without it everyone merges at once and the whole freeway crawls, including the cars from that ramp. A rate limit is that light. A good driver arrives at the pace the light allows, waits when it is red for exactly as long as it says, and does not honk."

Two halves on the board: **politeness** (pace yourself, obey Retry-After, never fetch the same thing twice) and **resilience** (retry what is temporary with backoff, give up clearly, save work so an interruption loses nothing).

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `session18_limits`, with the API restarted as `python data/mock_api.py --rate 10 --per 5`. `fresh_window()` waits five seconds so each demonstration starts from zero: use the pause to ask the next prediction. The 429 wait in step 4 can print 4 or 5 seconds depending on where in the window the burst landed.

1. **The limit, in the headers.** Say: "The API was restarted with --rate 10 --per 5: ten requests every five seconds. Every response tells you where you stand. Read them like a fuel gauge."

```
import hashlib
import json
import logging
import random
import sys
import time
from pathlib import Path

import requests

API = "http://127.0.0.1:8000"
HEADERS = {"User-Agent": "pinecrest-course-bot/1.0 (learning project; contact:
LIMIT, PER = 10, 5


def fresh_window():
    """Wait out the current window so a demonstration starts from zero."""
    time.sleep(PER)


r = requests.get(f"{API}/products", params={"per_page": 1}, headers=HEADERS, t
print(r.status_code, {k: v for k, v in r.headers.items() if k.startswith("X-Ra
```

```
200 {'X-RateLimit-Limit': '10', 'X-RateLimit-Remaining': '9', 'X-RateLimit-Res
```

2 **Ignore it, and hit the wall.** Say: "Thirteen requests as fast as Python can. Predict how many succeed." After: "429 is the server holding up a hand, with an instruction: Retry-After says how long to wait. The refused requests were pure waste."

```
fresh_window()
statuses = []
for i in range(13):
    r = requests.get(f"{API}/products", params={"per_page": 1}, headers=HEADER
                     timeout=10)
    statuses.append(r.status_code)
print(statuses)
print("Retry-After:", r.headers["Retry-After"], "|", r.json()["error"]["messag
```

```
[200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 429, 429, 429]
Retry-After: 5 | Too many requests. Try again in 5 second(s).
```

3 **Plan A: pace yourself.** Say: "The freeway on-ramp light. Ten per five seconds is one every half second; add ten percent for clock drift. The Throttle sleeps only for whatever is left of the interval."

```
class Throttle:
    """Keep calls at least `interval` seconds apart."""
    def __init__(self, interval):
        self.interval, self.last = interval, None

    def wait(self):
        if self.last is not None:
            remaining = self.interval - (time.monotonic() - self.last)
            if remaining > 0:
                time.sleep(remaining)
        self.last = time.monotonic()


interval = PER / LIMIT * 1.1
fresh_window()
throttle, statuses, started = Throttle(interval), [], time.monotonic()
for i in range(13):
    throttle.wait()
    statuses.append(requests.get(f"{API}/products", params={"per_page": 1},
                                 headers=HEADERS, timeout=10).status_code)
print(f"every {interval:.2f} s:", statuses.count(200), "of 13 succeeded,",
      f"about {time.monotonic() - started:.0f} s")
```

```
every 0.55 s: 13 of 13 succeeded, about 7 s
```

4 **Plan B: when told to wait, wait.** Say: "Pacing is plan A; limits change and keys get shared, so you still need plan B. Three details make it trustworthy: a default for Retry-After, a maximum number of tries, and retrying ONLY 429."

```
def get_polite(url, max_tries=5, **kwargs):
    """GET that waits and retries when the server answers 429."""
    kwargs.setdefault("headers", HEADERS)
    kwargs.setdefault("timeout", 10)
    for attempt in range(1, max_tries + 1):
        resp = requests.get(url, **kwargs)
        if resp.status_code != 429:
            return resp
        wait = float(resp.headers.get("Retry-After", 1))
        print(f"  429 (try {attempt}): waiting {wait:.0f} s, as asked")
        time.sleep(wait)
    return resp


fresh_window()
codes = [get_polite(f"{API}/products", params={"per_page": 1}).status_code
         for i in range(15)]
print(codes.count(200), "of", len(codes), "succeeded")
```

```
  429 (try 1): waiting 5 s, as asked
15 of 15 succeeded
```

5 **Backoff, and why it needs jitter.** Say: "No instruction from the server? Wait a little, then twice as long, then twice again, up to a cap. And add randomness, or a hundred clients that failed together retry together, in waves." The seed is only so this page matches the screen.

```
def backoff_delay(attempt, base=0.5, cap=8.0):
    """Seconds to wait after failed attempt number `attempt` (from 0)."""
    return min(cap, base * 2 ** attempt)


print([backoff_delay(n) for n in range(6)])
rng = random.Random(7)
print([round(rng.uniform(0, backoff_delay(n)), 2) for n in range(6)])
```

```
[0.5, 1.0, 2.0, 4.0, 8.0, 8.0]
[0.16, 0.15, 1.3, 0.29, 4.29, 2.93]
```

6 **A flaky endpoint.** Say: "/flaky fails twice with 503 and then succeeds, over and over. Notice its 503 also carries Retry-After: an instruction always beats your guess."

```
fresh_window()
for i in range(3):
    r = requests.get(f"{API}/flaky", headers=HEADERS, timeout=10)
    print(r.status_code, "| Retry-After:", r.headers.get("Retry-After"))
```

```
503 | Retry-After: 1
503 | Retry-After: 1
200 | Retry-After: None
```

7 **Retry what is temporary, return what is not.** Say: "Temporary: 429, 5xx, a timeout, a dropped connection. Not temporary: 400, 401, 403, 404. A missing product will still be missing in two seconds, so it comes back at once."

```
RETRY_STATUSES = {429, 500, 502, 503, 504}


def get_with_backoff(url, max_tries=5, base=0.5, **kwargs):
    """GET with retries: obeys Retry-After, otherwise backs off with jitter.""
    kwargs.setdefault("headers", HEADERS)
    kwargs.setdefault("timeout", 10)
    for attempt in range(max_tries):
        try:
            resp = requests.get(url, **kwargs)
            if resp.status_code not in RETRY_STATUSES:
                return resp
            problem, asked = f"HTTP {resp.status_code}", resp.headers.get("Ret
        except (requests.exceptions.Timeout, requests.exceptions.ConnectionErr
            problem, asked = type(err).__name__, None
        if attempt == max_tries - 1:
            break
        delay = float(asked) if asked else random.uniform(0, backoff_delay(att
        print(f"  try {attempt + 1}: {problem}; next try in {delay:.1f} s")
        time.sleep(delay)
    raise RuntimeError(f"gave up after {max_tries} tries (last problem: {probl


resp = get_with_backoff(f"{API}/flaky")
print(resp.status_code, resp.json())
print(get_with_backoff(f"{API}/products/SNW-9999").status_code)
```

```
  try 1: HTTP 503; next try in 1.0 s
  try 2: HTTP 503; next try in 1.0 s
200 {'status': 'ok', 'attempts_needed': 3}
404
```

8 **The same policy, built in.** Say: "For everyday work, urllib3's Retry does all of this, mounted on a Session. allowed_methods keeps retries to requests that only read: repeating a POST that places an order could place it twice."

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry


def make_session(total=5, backoff_factor=0.5):
    """A Session that identifies itself and retries temporary failures."""
    retry = Retry(total=total, backoff_factor=backoff_factor,
                  status_forcelist=[429, 500, 502, 503, 504],
                  allowed_methods=["GET", "HEAD"], respect_retry_after_header=
    adapter = HTTPAdapter(max_retries=retry)
    session = requests.Session()
    session.mount("https://", adapter)
    session.mount("http://", adapter)
    session.headers.update(HEADERS)
    return session


fresh_window()
session = make_session()
resp = session.get(f"{API}/flaky", timeout=10)
print(resp.status_code, "| retried after:", [a.status for a in resp.raw.retrie
```

```
200 | retried after: [503, 503]
```

9 **The politest request is the one you never send.** Say: "Same request, same file name: built from the URL and the parameters, hashed. check() runs BEFORE anything is written, so an error is never cached."

```
class ApiError(Exception):
    def __init__(self, status, code, message):
        super().__init__(f"{status} {code}: {message}")
        self.status, self.code, self.message = status, code, message


def check(resp):
    if resp.ok:
        return resp
    error = resp.json()["error"]
    raise ApiError(resp.status_code, error["code"], error["message"])


CACHE_DIR = Path("cache")


def cache_path(url, params=None):
    key = url + "?" + json.dumps(params or {}, sort_keys=True)
    return CACHE_DIR / (hashlib.sha256(key.encode("utf-8")).hexdigest()[:16] +


def cached_get_json(session, url, params=None):
    """Return (data, source): the cache when it can, the network when it must.
    path = cache_path(url, params)
    if path.exists():
        return json.loads(path.read_text(encoding="utf-8")), "cache"
    resp = check(session.get(url, params=params, timeout=10))
    CACHE_DIR.mkdir(exist_ok=True)
    path.write_text(resp.text, encoding="utf-8")
    return resp.json(), "network"


for category in ["Packs", "Climbing", "Packs", "Climbing", "Packs"]:
    data, source = cached_get_json(session, f"{API}/products",
                                   {"category": category, "per_page": 5})
    print(category, data["total"], "from", source)
```

```
Packs 24 from network
Climbing 24 from network
Packs 24 from cache
Climbing 24 from cache
Packs 24 from cache
```

10 **A log you can read afterward.** Say: "A job that runs overnight needs a record: what was asked, what failed, what was retried. The level is a filter. And never log a key, a token or an Authorization header."

```
log = logging.getLogger("pinecrest")
log.setLevel(logging.INFO)
handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(logging.Formatter("%(levelname)-7s %(message)s"))
log.handlers = [handler]
log.debug("fetched page 3")             # below INFO: not shown
log.info("starting download: %s pages", 7)
log.warning("rate limited; waiting %s s", 5)
```

```
INFO    starting download: 7 pages
WARNING rate limited; waiting 5 s
```

11 **Resumable: one file per page, skip what is done.** Say: "A long download WILL be interrupted. Save each page as it arrives; on the next run, skip pages already on disk. The .part rename means a half-written file is never mistaken for a finished one."

```
def download_pages(session, path, params, out_dir, throttle, max_new=None):
    """Save every page as JSON, skipping pages already saved."""
    out_dir = Path(out_dir)
    out_dir.mkdir(exist_ok=True)
    page, fetched, skipped = 1, 0, 0
    while True:
        file = out_dir / f"page_{page:04d}.json"
        if file.exists():
            data = json.loads(file.read_text(encoding="utf-8"))
            skipped += 1
        else:
            if max_new is not None and fetched >= max_new:
                return {"complete": False, "fetched": fetched, "skipped": skip
            throttle.wait()
            resp = check(session.get(f"{API}{path}", params={**params, "page":
                                     timeout=10))
            partial = file.with_suffix(".part")
            partial.write_text(resp.text, encoding="utf-8")
            partial.replace(file)
            data, fetched = resp.json(), fetched + 1
        if page >= data["total_pages"]:
            return {"complete": True, "fetched": fetched, "skipped": skipped}
        page += 1


throttle = Throttle(interval)
job = {"per_page": 50}
print(download_pages(session, "/products", job, "product_pages", throttle, max
print(download_pages(session, "/products", job, "product_pages", throttle))
print(download_pages(session, "/products", job, "product_pages", throttle))
```

```
{'complete': False, 'fetched': 2, 'skipped': 0}
{'complete': True, 'fetched': 3, 'skipped': 2}
{'complete': True, 'fetched': 0, 'skipped': 5}
```

THREE MISTAKES TO EXPECT

- `int(resp.headers.get("Retry-After"))` **with no default**: `TypeError` exactly when things are already going wrong. Coach: "What if the header is not there?" `.get("Retry-After", 1)`.
- **Retrying everything**, 404s and 403s included, or retrying with no wait and no limit. Coach: "Could this exact request succeed in two seconds? If not, why ask again?"
- **Caching before checking**: one 429 saved and served forever. Coach: "Which line writes the file, and which line checks the status? Swap them."

Also common: thinking a retry policy replaces pacing (it catches the rare miss; the Throttle is plan A), and running several copies of a script to go faster (the limit is per client).

GUIDED LAB (20 MINUTES)

Lab 6.5-A, everyone: `labs/module-18/l6_5-rate-limiting/stock_check_starter.py` (reference: `stock_check_solution.py`). Question: which categories have the most products out of stock or low, and what happens to thirteen requests sent at once?

- Minutes 0 to 6: TODO 1 (the headers and a polite pace: one request every 0.55 s) and TODO 2 (`Retry-After` with a default).
- Minutes 6 to 16: TODO 3 (plan B: on 429, wait as asked, a few tries) and TODO 4 (plan A: the Throttle, one line).
- Reference answers: the burst gets 10 answers and 3 refusals, `Retry-After: 5`; plan B ends with 0 refused after one wait (about 5 s); plan A ends with 0 refused in about 7 s; Packs has the most out of stock (3); two of the four SKUs are 404 and are asked once.
- Minutes 16 to 20: TODO 5, the sentence: only the 429s earned a second try; a product that does not exist will not exist in five seconds either.

Lab 6.5-B (finish-if-fast, or homework): `labs/module-18/l6_5-rate-limiting/retry_and_cache_starter.py`. Backoff with jitter, `/flaky` (success on try 3), what not to retry, a urllib3 `Retry` policy (offline it says why that step is skipped), and a cache that answers six questions with three requests and never stores a 404.

START THE LESSON PROJECT (7 MINUTES)

"Project: The Resumable Order Pull", Step 1 only: one probe gives 2,056 orders since 2025-11-01, 21 requests, about 12 seconds at 0.55 s each. Say: "Do this arithmetic before every download. For 50,000 requests it tells you to run overnight." Steps 2 to 6 (log file, interruption, resume, prove it, read the log) are homework.

CHECKS FOR UNDERSTANDING

Workbook self-check, Q7 to Q12. Q12 is worth a 30-second discussion: someone in the room will have tried it.

Answers: **Q7** C) Wait about 3 seconds, until the window starts over; **Q8** `3`; **Q9** `[0.2, 0.4, 0.8, 1.6, 2.0]`; **Q10** A) Repeating a POST can repeat its effect, such as placing an order twice; **Q11** D) The saved error body is served from the cache, as if it were data; **Q12** B) The limit applies to you, so the copies hit 429 three times as often and the job is no faster. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: in Lab A, TODO 3 and TODO 4 are independent: a learner who is stuck on the loop can do the one-line Throttle first and see 0 refusals.

Stretch: lesson Exercise 4 (a cache with `max_age`) or the bonus challenge.

CHECKPOINT 2 (15 MINUTES)

2:25 Checkpoint 2: Data Access (Modules 5 and 6) 15 min

What it checks: that each learner can get data out of a database and out of an authenticated, paginated API on their own, check both, and answer a question in a sentence. It is open notes (lessons, their own notebooks), own work, not graded for the record: it tells you and them what to review before the capstone.

The task (workbook task sheet): Finance is closing the year. For orders with status `returned` placed on or after 2025-11-01, do the company database and the orders API agree? Report the count and total value from each source and whether the order ids are the same.

- **Minute 0:** everyone opens `labs/module-18/checkpoint-2/checkpoint2_starter.py`. It already runs, sets up the session and prints the mode. Read the task aloud once. Say: "One query, one paginated pull, one sentence."
- **Minutes 1 to 13:** silence and circulation. Answer questions about the environment (variables, the API not running), not about the solution.
- **Minute 13:** two-minute warning: "Print what you have. A partial answer with a clear sentence about what is missing scores better than nothing."
- **API will not start, or the network stack is broken on a machine:** `LABS_OFFLINE=1 python checkpoint2_starter.py` runs the same file against saved responses. The database part never needs the network.
- **Collect:** the file plus its printed output (a screenshot, or the output pasted at the bottom of the file).

Scoring: the 5-row rubric and the model answer are in the Answer Key. The model answer prints `118 returned orders since 2025-11-01, total $25,191.92` from both sources, 3 pages at 50 per page, and the same ids.

2:40 Checkpoint Debrief and Module 6 Recap

10 min

- Run the model answer on the projector (offline is fine) and read the three printed parts.
- The three slips you will have seen: values glued into the SQL string instead of `?` parameters; `params` sent again on every page (or only page 1 read); no status check, so a 401 body became "data".
- The module in one picture, on the board: **an API or a data file first → static HTML with Beautiful Soup → a browser last**, all of it with the six rules, a User-Agent, timeouts, status checks, pacing and retries with limits.
- Tell learners what their result means: a full answer means Modules 5 and 6 are ready for the capstone; a partial one means one lesson to revisit this week (name it on their sheet).

CLOSING

EXIT TICKET (3 QUESTIONS, ON A STICKY NOTE; ANSWERS)

1. What is the difference between a 401 and a 403, and what does your program do with each? **Answer:** 401: no valid credential (missing, malformed, expired): fix how you send it, or renew an expired token and retry once. 403: understood and refused: stop; a person must fix the key or permission.
2. The server answers 429 with `Retry-After: 5`. What does your client do? **Answer:** Wait at least 5 seconds, then send the request again (with a maximum number of tries). Better still: pace requests so this rarely happens.
3. Name one failure that must NOT be retried, and say why. **Answer:** Any of 400, 401 (except an expired token, once), 403, 404: they describe the request, so the same request gets the same answer.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- Lesson 6.4: finish the December Returns Brief (Steps 3 to 5), the exercises, the quiz and the journal.
- Lesson 6.5: finish the Resumable Order Pull (Steps 2 to 6), the exercises, the quiz and the journal.
- Kit labs not finished: `token_lifecycle_starter.py`, `retry_and_cache_starter.py`.
- Checkpoint 2: if your answer was partial, finish it at home and compare with the sentence on the debrief slide.

BRING LIST FOR SESSION 19 (SEABORN AND PLOTLY)

- **Stop the practice API** (Ctrl+C). Module 7 does not use it, nor the practice shop.
- **Check the plotting libraries** (installed with the course environment in Lesson 0.1): `python -c "import seaborn, plotly; print(seaborn.__version__, plotly.__version__)"`.
- **Files** (all in `data/`): `daily_sales.csv`, `store_weather.csv`, `orders.csv`, `survey.csv` for Lesson 7.1; `order_items.csv`, `products.csv`, `ab_test.csv` as well for Lesson 7.2.
- Reopen a Matplotlib notebook from Module 3 and remind yourself how `fig, ax = plt.subplots()` works: seaborn builds on it.
- Keep `orders_since_november.csv` and your Module 6 notebooks: a small portfolio of responsible data collection. Accounts: none. Servers: none.

IF YOU RUN LONG OR SHORT

Running long. The checkpoint starts at 2:25 no matter what: take the time from the 6.5 project start, then from the 6.5 demo (steps 9 to 11 become "read the lesson"), then from Lab 6.5-A (TODO 5 becomes homework). If the API restart at the break fails for some learners, they run the 6.5 lab offline (`LABS_OFFLINE=1`) and still see the same 429s and waits.

Running short. Do lesson Exercise 2 together (`polite_interval`: 100,000 requests at 5,000 per hour take 22 hours) and Exercise 1 (watch `X-RateLimit-Remaining` count down). Or extend the debrief: learners swap checkpoint files and run each other's offline.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 19: Seaborn and Plotly

How to teach the session where charts start doing statistics and then start answering back. Before the break, seaborn: describe a chart by naming columns, and learn to say exactly what an error bar shows. After it, Plotly: a chart the reader can hover, zoom and filter, in one HTML file that carries its data with it.

`2 lessons` `3 hrs session` `Lessons 7.1–7.2` `Session 19 of 21`

HOW TO USE THIS GUIDE

- **Same rhythm as every session:** warm-up, then for each lesson a frame (the goal and a mental model), a live demo (you type, they type along), a guided lab from the kit, and the first step of the lesson's own project. The rest of each project, the lesson exercises, quiz and journal are homework.
- **The lessons are the reference.** Keep `visualization_seaborn.html` and `visualization_plotly.html` open in browser tabs for the diagrams (the function-family tree, the figure-to-browser flow, the "who will read it?" chooser). The kit's labs for today live in `labs/module-19/`: starters for learners, solutions for you.
- **Type, do not paste.** Every code block in the two demos is what to type live; every output was produced by running it on the course data with the course environment (seaborn 0.13.2, Plotly 7.1.0, pandas 3.0). Break something on purpose once per lesson and read the message aloud.
- **Teal blocks are hands-on, amber blocks are watch-outs.**

> THE ONE RISK OF SESSION 19: CHARTS THAT LOOK RIGHT AND SAY SOMETHING ELSE
>
> Both libraries will happily compute or skip a statistic without telling you. Seaborn **aggregates silently** (`lineplot`, `barplot` and `pointplot` draw a mean and an interval you did not ask for). Plotly Express **does not aggregate at all** (`px.bar` stacks every row). Neither raises an error. Say the two rules early and keep pointing at them: "In seaborn, ask what the bar shows. In Plotly, aggregate first." The second risk comes at the end: an HTML chart carries its data. Learners are two sessions from a capstone they may publish, so the habit of passing a figure only the columns it needs starts today.

BEFORE THE SESSION

- ☐ Run the kit check from the kit folder: `python3 labs/check_labs.py module-19`. Expect `8 passed, 0 failed` (four starters, four solutions).

- ☐ Hand out **starters only**: `labs/module-19/l7_1-seaborn/tier_order_sizes_starter.py`, `website_by_age_starter.py`, `labs/module-19/l7_2-plotly/product_map_starter.py`, `promo_share_trend_starter.py`, together with `labs/labs_paths.py`. Keep every `*_solution.py` on your machine.
- ☐ Projector machine: JupyterLab or VS Code started **from the course folder** (the one that holds `data/`), font at 18 pt or larger, a browser for the two lesson pages. Check versions: `python -c "import seaborn, plotly; print(seaborn.__version__, plotly.__version__)"` should print `0.13.2 7.1.0` (any 0.13 and any 7.x is fine).
- ☐ **Plotly in the room's notebooks.** Run one `px.line(...).show()` on a learner machine before class. If the cell shows a blank box, restart the kernel; if it still does, `import plotly.io as pio; pio.renderers.default = "browser"` sends every chart to a browser tab. `fig.write_html("chart.html")` is the fallback that cannot fail. Write that line on the board.
- ☐ **No internet in the room?** `include_plotlyjs="cdn"` files open blank offline. For the demo's last step and both Plotly labs tell learners to write `include_plotlyjs=True` instead (files of about 4.7 MB). Everything else today is offline already: no servers, no accounts, no API.
- ☐ **Static images are optional today.** `fig.write_image` needs Kaleido (installed) plus a Chrome it can drive. Do not spend class time on it: the lab skips it with a message. If you want to show it, run `plotly_get_chrome` on your machine beforehand.
- ☐ Whiteboard, before learners arrive: the two boxes (**axes-level** | **figure-level**), "What does this bar show: spread, or uncertainty?", and for after the break "figure = data + layout" and "Aggregate first."
- ☐ Skim the Session 18 homework: the Lesson 6.5 project file `orders_since_november.csv` (2,056 orders, pulled through an interruption). One volunteer explains in the warm-up how the job knew where to resume.
- ☐ Sticky notes or cards for the exit ticket.

SESSION 19 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 18 (keys, tokens, 429, backoff), no notes. Homework check: the resumable order pull.
0:10	12	**7.1** Frame and mental model	The drafting assistant: name the columns, seaborn groups, computes and draws on a Matplotlib Axes.
0:22	28	**7.1** Live demo	One call instead of a loop, the theme, lineplot's hidden mean, error bars four ways, axes-level against figure-level.
0:50	25	**7.1** Guided lab	`labs/module-19/l7_1-seaborn/tier_order_sizes_starter.py`, then `website_by_age_starter.py` for those who finish.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:15	10	**7.1** Start the lesson project	The Store Performance Brief: read the brief, run Step 1 (the two tables), sketch the four panels.
1:25	10	Break	Leave the four-error-bar figure on the projector. Ask: which panel would you put in front of a manager?
1:35	12	**7.2** Frame and mental model	The wall map and the map app: Python builds a description, the browser draws it.
1:47	28	**7.2** Live demo	First figure, a figure is data, aggregate before px.bar, design the hover, finish the layout, write_html and what is inside it.
2:15	25	**7.2** Guided lab	`labs/module-19/l7_2-plotly/product_map_starter.py`, then `promo_share_trend_starter.py`.
2:40	10	**7.2** Start the lesson project	The Sales Explorer: Step 1 (tidy, minimal tables) and the first chart with a unified hover.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 20.
Total	180		Ends at 3:00.

The two lesson pages are timed at 90 + 90 = 180 minutes for a solo learner; after the warm-up, break and wrap-up the room has 150. **Protect** in each lesson: the mental model, the demo steps marked as the heart (7.1 error bars four ways; 7.2 `px.bar` does not aggregate, and the look inside the HTML file), the first guided lab, and starting the project. **Goes home:** the remaining project steps, all lesson exercises, both quizzes and journals, and the second lab of each lesson for anyone who did not reach it: about three hours.

WARM-UP

0:00 Warm-up: Retrieval from Session 18 — 10 min

HOW TO RUN IT

Six minutes, no notes, answers on paper, then two minutes of answers called out and two for the homework check. These reach back to Lessons 6.4 and 6.5: the point is retrieval, not a grade.

FIVE QUESTIONS (WITH ANSWERS)

1. A request comes back **401**. Another comes back **403**. What does each one tell you, and what should your client do next? **Answer:** 401: the server does not know who you are (no token, or `token_expired`): log in again and repeat the request once. 403: it knows who you are and refuses: stop and fix the credential or the permission. Neither is retried in a loop.

2. Where does an API key live while you work on a notebook you will share, and where in the request does it travel? **Answer:** In an environment variable (or typed at a prompt), read at run time; never in the code. It travels in a header (`X-API-Key`), not in the URL, because URLs end up in logs and browser history.

3. The server answers **429** with the header `Retry-After: 4`. What does a polite client do? What if the header is missing? **Answer:** Wait at least 4 seconds, then send the same request again. With no header, fall back to a default wait (exponential backoff with jitter), with a maximum number of tries.

4. What does this print?

```
print([min(4.0, 1.0 * 2 ** n) for n in range(4)])
```

   Answer: `[1.0, 2.0, 4.0, 4.0]`: the delay doubles (1, 2, 4) and then the cap of 4.0 holds it. That is exponential backoff with a ceiling; real clients add random jitter on top.

5. Name two failures a retry policy should retry and two it never should. Then: what made the Lesson 6.5 download resumable? **Answer:** Retry: 429, 5xx, timeouts (temporary). Never: 400, 401, 403, 404 (the same request will fail the same way). Resumable: each page is saved to its own file as soon as it arrives, written completely or not at all, and pages whose files exist are skipped.

HOMEWORK CHECK (2 MIN)

Hands up: whose `orders_since_november.csv` has 2,056 rows? Ask one volunteer: "You interrupted the job. How did it know what was left?" (One file per page; pages whose files exist are skipped.) Then the bridge: "Five lessons getting data. From today, showing it. Open the notebook from Module 3 in your head: `fig, ax = plt.subplots()`. Seaborn draws on that `ax`."

7.1 Statistical Graphics with Seaborn

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Draw relational, distribution and categorical charts from a tidy DataFrame with one seaborn call each
- Tell an axes-level function from a figure-level one, and choose the right one for a job
- Split a chart by `hue`, `col` and `row` instead of writing loops
- Say exactly what an error bar or shaded band represents, and set it with `errorbar=`
- Finish a seaborn chart with Matplotlib, and recognize when plain Matplotlib is the better tool

IN THE ROOM / AT HOME

Room (75 min): the frame, the eleven-step demo, lab 7.1-A (and 7.1-B for fast finishers), and Step 1 of the Store Performance Brief. **Home:** the rest of the project (four panel functions, the assembled `store_brief.png`, three sentences for the manager), Exercises 1 to 5, the quiz and the journal. Heatmaps and pair plots are not demonstrated: learners read that section at home and meet `sns.heatmap` in the project's Step 4.

MENTAL MODEL TO SAY (12 MIN FRAME)

"Matplotlib is a drawing table with excellent pens: put a dot here, a line there. Seaborn is an **assistant at the same table** who takes orders at a higher level: revenue against visitors, one color per store. The assistant splits the table into groups, picks colors, computes any averages and intervals, draws with the same pens and writes the legend. When the assistant is done, you can still pick up a pen."

Draw two boxes on the board, **axes-level** (takes `ax=`, returns an Axes) and **figure-level** (`relplot`, `displot`, `catplot`: makes its own figure, `col=` / `row=`, returns a FacetGrid), and leave them up all session. Then write the second idea under them: **"What does this bar show: spread, or uncertainty?"** Show the lesson page's function-family diagram for thirty seconds; do not read the theme table aloud.

LIVE DEMO (YOU TYPE, THEY FOLLOW) · 28 MIN

Work in one notebook, `session19_seaborn.ipynb`, started from the course folder so that `data/` resolves. Every output below is from a real run. End each chart cell with `plt.show()` as written.

1. **Load and look (2m).** Say: "Same table as Module 3. One row per store per day, one column per variable: that is tidy, and tidy is the shape seaborn wants."

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

sales = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
sales["year"] = sales["date"].dt.year
sales["month"] = sales["date"].dt.month
sales["weekday"] = sales["date"].dt.day_name().str[:3]
print(sns.__version__, sales.shape)
print(sales[["date", "store", "visitors", "revenue", "weekday"]].head(3))
```

```
0.13.2 (2903, 11)
        date      store  visitors  revenue weekday
0 2024-01-01  Asheville       201  5017.07     Mon
1 2024-01-01       Bend       200  4132.92     Mon
2 2024-01-01      Boise       333  5962.00     Mon
```

2 **The Module 3 way: you do the grouping (2m).** Type the loop slowly. Say: "Four lines of bookkeeping before any thinking. Count them."

```
fig, ax = plt.subplots(figsize=(8, 4), layout="constrained")
for store, group in sales.groupby("store"):
    ax.scatter(group["visitors"], group["revenue"], s=8, alpha=0.5, label=stor
ax.set_xlabel("visitors")
ax.set_ylabel("revenue")
ax.legend(title="store")
plt.show()
print(len(ax.collections), "scatter collections, one per store")
```

```
4 scatter collections, one per store
```

3 **The seaborn way: name the roles (3m).** Say: "Name the table, then say which column plays which role. Watch what comes back." Ask before running the print: "Is `returned` something new, or our own `ax`?"

```
fig, ax = plt.subplots(figsize=(8, 4), layout="constrained")
returned = sns.scatterplot(data=sales, x="visitors", y="revenue", hue="store",
                           s=12, alpha=0.5, ax=ax)
plt.show()
print(type(returned).__name__, returned is ax, "| x label:", ax.get_xlabel())
```

```
Axes True | x label: visitors
```

It is the same Axes: seaborn drew on **your** canvas and labeled the axes from the column names. Everything from Module 3 still works on `ax`.

4. **Set the theme once (2m).** Say: "Importing seaborn changes nothing. `set_theme` changes Matplotlib's global settings, so every later figure gets it, plain Matplotlib included." Re-run the scatter cell to show the grid and the colorblind palette.

```
sns.set_theme(style="whitegrid", palette="colorblind")
print(plt.rcParams["axes.grid"], plt.rcParams["axes.facecolor"])
```

```
True white
```

5. **lineplot computes something you did not ask for (4m).** Draw it first and let the room look. Ask: "Nobody computed a mean. What is the line? What is the shaded band?" Then check against pandas.

```
fig, ax = plt.subplots(figsize=(8, 4), layout="constrained")
sns.lineplot(data=sales, x="month", y="revenue", hue="store", ax=ax)
ax.set(xticks=range(1, 13), ylabel="daily revenue ($)")
plt.show()
boise = sales[sales["store"] == "Boise"]
print(boise.groupby("month")["revenue"].agg(["count", "mean"]).round(0).head(3
```

```
       count    mean
month
1         62  7585.0
2         57  7277.0
3         62  8494.0
```

Say: "About 60 days per store and month. A line needs one y per x, so seaborn took the **mean** and shaded a **95% confidence interval** for it. A band you did not expect means seaborn found several rows per x."

6. **Aggregate yourself when the total is the point (3m).** Say: "If the chart is total revenue per month, compute it with groupby and plot that table. One row per x: nothing left to average."

```
sales["month_start"] = sales["date"].dt.to_period("M").dt.to_timestamp()
monthly = sales.groupby(["store", "month_start"], as_index=False)["revenue"].s
per_x = monthly.groupby(["store", "month_start"]).size().max()
print(monthly.shape, "| rows per store and month:", per_x)
fig, ax = plt.subplots(figsize=(8, 4), layout="constrained")
sns.lineplot(data=monthly, x="month_start", y="revenue", hue="store", ax=ax)
plt.show()
```

```
(96, 3) | rows per store and month: 1
```

7 **A skewed distribution, two groups (3m).** Say: "Order totals are right-skewed (Lesson 4.2). `log_scale=True` is the fix, and an ECDF needs no bins at all." Point at where each curve crosses 0.5.

```
orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
completed = orders[orders["status"] == "completed"]
fig, axes = plt.subplots(1, 2, figsize=(10, 3.6), layout="constrained")
sns.histplot(data=completed, x="order_total", hue="channel", log_scale=True,
             bins=50, element="step", stat="density", common_norm=False, ax=ax
sns.ecdfplot(data=completed, x="order_total", hue="channel", log_scale=True,
             ax=axes[1])
axes[1].axhline(0.5, color="gray", linewidth=1, linestyle="--")
plt.show()
print(completed.groupby("channel")["order_total"].median().round(2).to_dict())
```

```
{'online': 153.95, 'store': 133.46}
```

8 **Same means, four error bars (5m). This is the heart of the lesson.** Build the loop, run it, then stop talking for ten seconds. Ask: "Four panels, identical dots. Which panel says Asheville is different? Which says customers disagree with each other?"

```
survey = pd.read_csv("data/survey.csv")
stores = ["Boise", "Bend", "Flagstaff", "Asheville"]
in_store = survey[survey["store"].isin(stores)]
choices = [("ci", 95), "se", "sd", ("pi", 50)]
fig, axes = plt.subplots(1, 4, figsize=(10, 3.4), sharey=True, layout="constra
for ax, choice in zip(axes, choices):
    sns.pointplot(data=in_store, x="store", y="sat_staff", order=stores,
                  errorbar=choice, seed=42, linestyle="none", capsize=0.2, ax=
    ax.set_title(f"errorbar={choice!r}", fontsize=10)
    ax.tick_params(axis="x", rotation=45)
plt.show()
by_store = in_store.groupby("store")["sat_staff"]
summary = by_store.agg(n="count", mean="mean", sd="std", se="sem")
print(summary.loc[stores].round(2))
```

```
             n  mean    sd    se
store
Boise      320  4.11  0.82  0.05
Bend       228  4.16  0.86  0.06
Flagstaff  206  4.02  0.86  0.06
Asheville  193  3.62  1.02  0.07
```

Say: "Short bars: how well we know the mean. Long bars: how much people vary. Both are true. The bar you draw decides which one the reader hears, so the caption must say which it is." Note `seed=42`: the default interval is bootstrapped, so without a seed it wiggles between runs.

9 **Figure-level: seaborn builds the grid (3m).** Say: "With four stores on one scatter the points pile up. `col=` gives one panel per store." Ask the room to predict the two printed values before you run.

```
g = sns.relplot(data=sales, x="visitors", y="revenue", col="store", col_wrap=2
                hue="promo_day", s=12, alpha=0.6, height=2.8, aspect=1.5)
g.set_axis_labels("visitors per day", "revenue ($)")
g.set_titles("{col_name}")
plt.show()
print(type(g).__name__, g.axes.shape, type(g.figure).__name__)
```

```
FacetGrid (4,) Figure
```

Size is `height` and `aspect` per panel, not `figsize`. The grid has its own methods (`set_titles`, `set_axis_labels`); for anything else reach in through `g.figure` and `g.axes`.

10 **Break it on purpose, twice (3m).** Say: "Two mistakes everyone makes this week. First: handing my own Axes to a figure-level function." Read the warning aloud. "Second: treating the grid like an Axes."

```
import warnings
fig, ax = plt.subplots(figsize=(6, 3))
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    sns.relplot(data=sales, x="visitors", y="revenue", ax=ax)
print(caught[0].category.__name__ + ":", caught[0].message)
g = sns.catplot(data=sales, x="store", y="revenue", kind="box", height=3)
try:
    g.set_title("Revenue by store")
except AttributeError as error:
    print("AttributeError:", error)
```

```
UserWarning: relplot is a figure-level function and does not accept the `ax` p
AttributeError: 'FacetGrid' object has no attribute 'set_title'
```

Say: "When a seaborn object lacks a method you expected, print `type(obj)`. It tells you which of the two kinds you are holding."

11 **Finish with Matplotlib, and save (3m).** Say: "Seaborn got us most of a chart in one line. The last part, a title that states the finding, dollars on the axis, the legend moved, is Module 3 work on the Axes that came back."

```
from matplotlib.ticker import StrMethodFormatter
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
fig, ax = plt.subplots(figsize=(8, 4), layout="constrained")
sns.barplot(data=sales, x="weekday", y="revenue", hue="year", order=days,
            errorbar=("ci", 95), seed=42, ax=ax)
ax.set_title("Saturday is the biggest day of the week, in both years", loc="le
ax.set(xlabel="", ylabel="mean daily revenue per store (bars: 95% CI)")
ax.yaxis.set_major_formatter(StrMethodFormatter("${x:,.0f}"))
sns.move_legend(ax, "upper left", title="Year")
fig.savefig("weekday_revenue.png", dpi=150)
by_day = sales.groupby("weekday")["revenue"].mean()
print(by_day.loc[["Tue", "Sat"]].round(0).to_dict())
```

```
{'Tue': 5716.0, 'Sat': 10933.0}
```

THREE MISTAKES TO EXPECT

- **Passing `ax=` to `relplot`, `displot` or `catplot`** (or the reverse: expecting `col=` to work on `scatterplot`). Symptom: two figures, one empty, and a UserWarning that scrolls by unread. Coach: "Read the last sentence of the warning. Who owns the figure in your plan, you or seaborn?" Point at the two boxes on the board.

- **Reading a band or bar without knowing what it is.** A learner draws `lineplot` on daily rows and describes the band as "the range". Coach: "How many rows share each x? Print `groupby(...).size()`. What did seaborn have to do to get one y?" Then: "Which `errorbar=` would give you the range you had in mind?" (`("pi", 100)`.)
- **Categories in first-seen order.** Weekdays come out Mon, Tue, ... only by luck of the file; age groups come out 65+, 18-24, 25-34. There is no error, so it reaches the saved figure. Coach: "Read your x axis aloud, left to right. Is that the order you would say them in?" Fix: `order=`, `hue_order=`, `col_order=`.

GUIDED LAB · 25 MIN

Lab 7.1-A `labs/module-19/l7_1-seaborn/tier_order_sizes_starter.py` (everyone; reference: `tier_order_sizes_solution.py`). Question: do Gold members get to their spend with bigger orders or with more of them? Two axes-level calls in one `plt.subplots(1, 2)` figure, finished with Matplotlib and saved to `OUTPUT`.

- **Minutes 0 to 5:** everyone runs the starter as shipped and reads the merge check line aloud: 18,762 completed orders, 18,079 with a customer row. Ask: "Where did 683 orders go?" (Closed accounts: the README's 150 missing customer ids.) This is the capstone habit, rehearsed.
- **Minutes 5 to 18:** TODO 1 to 5. Walk the room during TODO 2: the common stall is `hue=` spelled `color=` (next lesson's word) or a missing `ax=axes[0]`, which draws both charts on the last Axes.
- **Minutes 18 to 25:** TODO 6 and the debrief. Ask two pairs for their panel titles. A good title states the finding: "Gold orders are somewhat larger... but mostly there are many more of them."
- **Expected numbers:** Merge check: 18,762 completed orders, 18,079 with a customer row, 683 dropped (closed accounts). 3,738 customers. Median order total: Basic $110.66, Silver $146.98, Gold $162.28. Mean completed orders per customer: Basic 2.05, Silver 5.12, Gold 10.81. Gold against Basic: order size ×1.47, orders per customer ×5.26.
- **Lab 7.1-B** `labs/module-19/l7_1-seaborn/website_by_age_starter.py` (fast finishers now, everyone at home; reference: `website_by_age_solution.py`). 1,800 responses, 472 with no website rating. Means by age group: 3.69, 3.67, 3.67, 3.66, 3.72 and 65+ 3.04 (n = 291, 95% CI 2.94 to 3.15); everyone else 3.68, a gap of 0.64 points. The gap is the same for online shoppers (65+ 3.06) and store shoppers (65+ 3.02). catplot returns a FacetGrid with axes shape (1, 2). No website rating: 3% of online shoppers, 46% of store shoppers.
- Debrief line for 7.1-B: the caveat in Step 4 is the point. Nearly half of store shoppers skipped the website question, so the store panel describes the store shoppers who use the website.

START THE LESSON PROJECT · 10 MIN

Open the lesson at **Project: The Store Performance Brief**. Read the brief aloud (one page for the regional manager: trend, busy times, rain). Everyone types and runs **Step 1** only; the check is the printed `(96, 3)` `(2903, 17)` and the weather counts `dry 2514`, `rainy 389`. Then two minutes on paper: sketch the 2 by 2 layout and write which seaborn function each panel needs. Say: "Each panel is a function that takes

`ax`. That is the pattern from Lesson 3.2, and it is why axes-level functions exist." Steps 2 to 5 are homework.

CHECKS FOR UNDERSTANDING

Use the workbook self-check, items Q1 to Q6, in the last five minutes of the lab or as a cold-call round after the break.

Answers: **Q1** C) `sns.displot`; **Q2** A) `Axes`; **Q3** `(4,)`; **Q4** B) One line per store through the monthly means, with no band; **Q5** D) One standard deviation either side of each mean; **Q6** `['M', 'S', 'L'] [3.0, 3.0, 2.0]`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pair programming for the lab with one keyboard; give the pair the two-box board picture as a printed card and have them write `axes-level` or `figure-level` as a comment above every seaborn call. Offer the first `sns.ecdfplot` line complete and let them write the second chart. **Stretch:** in lab 7.1-A replace the point plot with `sns.boxplot` of orders per customer and say what the box plot shows that the mean hides; in 7.1-B redo Step 1 with `errorbar=("pi", 50)` and explain why every bar now covers whole numbers only; or start Exercise 5 (the Spearman heatmap with a mask).

7.2 Interactive Charts with Plotly

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Build line, scatter, bar, box and heatmap charts with one Plotly Express call each
- Control what a reader sees on hover, and format axes, legends and facet titles
- Decide when animation helps and when small multiples say the same thing better
- Drop down to graph objects to combine trace types, add a second y axis and annotate
- Save a chart as a standalone HTML file, export a static image, and explain what data travels inside the file

IN THE ROOM / AT HOME

Room (75 min): the frame, the eleven-step demo, lab 7.2-A (and 7.2-B for fast finishers), and Step 1 plus the first chart of the Sales Explorer. **Home:** the rest of the project (three charts assembled into `sales_explorer.html`), Exercises 1 to 4, the quiz and the journal. Not demonstrated in the room: facets and trendlines, animation, the dual-axis chart and `write_image`. Say so, and say where they are in the lesson; the animation section's table (when animation helps) is worth two sentences in the wrap-up.

MENTAL MODEL TO SAY (12 MIN FRAME)

"A static chart is a **map printed on the wall**: beautifully composed, the same for everyone, fixed. An interactive chart is a **map app**: the reader pinches to zoom, taps a pin, switches layers. You, the mapmaker, still decide what is on the map and what the default view is. The reader decides where to look next."

Then the mechanism, in one breath, with the lesson's flow diagram on screen: "Your Python does not draw anything. It builds a **figure**: traces (the data) plus a layout (titles, axes, legend). That description goes, as JSON text, to a JavaScript library, plotly.js, which draws it in a browser. A notebook is a web page. An HTML file is a web page. That is why the chart works in both, and why **the data is in the file**." Write on the board: **figure = data + layout** and **aggregate first**.

LIVE DEMO (YOU TYPE, THEY FOLLOW) · 28 MIN

New notebook, `session19_plotly.ipynb`. In a notebook `fig.show()` draws under the cell. The outputs below were produced on a machine with no screen, where `fig.show()` does nothing; the printed lines are what learners should see under each cell as well as the chart. If charts do not appear in the room's notebooks, see Before the Session.

1. **Imports and a monthly table (2m).** Say: "Two interfaces. Plotly Express, `px`, is the seaborn-style one: name the columns. We start there."

```
import numpy as np
import pandas as pd
import plotly
import plotly.express as px

sales = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
sales["month_start"] = sales["date"].dt.to_period("M").dt.to_timestamp()
monthly = sales.groupby(["store", "month_start"], as_index=False)[
    ["revenue", "visitors", "transactions"]].sum()
print(plotly.__version__, monthly.shape)
```

```
7.1.0 (96, 5)
```

2 **The first figure, and why nothing appears (3m).** Run the first two lines only. Wait. Say: "Nothing. `px.line` returned a Figure object. Python has built a description; nothing has drawn it yet." Then run `fig.show()` and spend a full minute on the live chart: hover, click a legend entry, double-click one, drag a box, double-click to reset.

```
fig = px.line(monthly, x="month_start", y="revenue", color="store",
              title="Monthly revenue by store")
print(type(fig).__name__, "with", len(fig.data), "traces")
fig.show()
```

```
Figure with 4 traces
```

3 **A figure is data (2m).** Say: "Because a figure is a description, you can inspect it like any object. This habit is what makes Plotly debuggable."

```
trace = fig.data[0]
print(trace.type, "|", trace.name, "|", trace.mode, "|", len(trace.x), "points
print(fig.layout.title.text, "|", fig.layout.xaxis.title.text)
print(list(fig.to_dict()))
```

```
scatter | Asheville | lines | 24 points
Monthly revenue by store | month_start
['data', 'layout']
```

A line chart is a `scatter` trace in `lines` mode. The whole figure is two things, `data` and `layout`, and everything else today changes one or the other.

4 **px.bar does not aggregate (3m).** Ask first: "`sns.barplot(data=sales, x=\"store\", y=\"revenue\")` gave four bars at the mean. What will `px.bar` do with the same table?" Take guesses, then run.

```
wrong = px.bar(sales, x="store", y="revenue")
print(len(wrong.data[0].x), "bar segments")
by_store = sales.groupby("store", as_index=False)["revenue"].mean()
by_store = by_store.sort_values("revenue", ascending=False)
right = px.bar(by_store, x="store", y="revenue", text_auto=",.0f")
print(len(right.data[0].x), "bars:", by_store.round(0).to_dict("records")[0])
```

```
2903 bar segments
4 bars: {'store': 'Boise', 'revenue': 10144.0}
```

Say: "No error. The first chart looks like four bars and is 2,903 slivers stacked up. In Plotly you aggregate with pandas first, then plot."

5 **The default hover is rarely the one you want (3m).** Say: "Here is what Express wrote for a scatter. It is a small template with placeholders. It does not say the one thing I want to know about an odd point: which day is it?"

```
fig = px.scatter(sales, x="visitors", y="revenue", color="store")
print(fig.data[0].type, "|", fig.data[0].hovertemplate)
```

```
scattergl | store=Asheville<br>visitors=%{x}<br>revenue=%{y}<extra></extra>
```

Note the trace type: above a thousand points Express switches to a WebGL trace (`scattergl`) by itself.

6 **Design the hover (4m).** Say: "`hover_name` is the bold first line. `hover_data` is a dictionary: True adds a column, a string starting with a colon adds it with a number format." Show the chart and hover the top point: the Saturday of the 2024 Labor Day sale, in Boise.

```
stores = ["Boise", "Bend", "Flagstaff", "Asheville"]
sales["day"] = sales["date"].dt.strftime("%a %b %d, %Y")
fig = px.scatter(sales, x="visitors", y="revenue", color="store", opacity=0.5,
                 category_orders={"store": stores}, hover_name="day",
                 hover_data={"store": True, "revenue": ":$,.0f", "visitors": "
                 labels={"visitors": "Visitors", "revenue": "Revenue"})
print(fig.data[0].hovertemplate)
best = sales.loc[sales["revenue"].idxmax()]
print(best["day"], "|", best["store"], "|", f"${best['revenue']:,.0f}")
fig.show()
```

```
<b>%{hovertext}</b><br><br>store=%{customdata[0]}<br>Visitors=%{x:,}<br>Revenu
Sat Aug 31, 2024 | Boise | $27,677
```

7 **Time series: one box per date (3m).** Say: "For lines compared at the same dates, write the template yourself and unify the hover."

```
fig = px.line(monthly, x="month_start", y="revenue", color="store",
              category_orders={"store": stores}, template="plotly_white",
              labels={"month_start": "", "revenue": "", "store": ""})
fig.update_traces(hovertemplate="%{y:$,.0f}")
fig.update_layout(hovermode="x unified")
print(fig.layout.hovermode, "|", fig.data[0].hovertemplate)
fig.show()
```

```
x unified | %{y:$,.0f}
```

8 **Finish the layout (3m).** Say: "Three methods do the finishing. `update_layout` for the whole figure, `update_xaxes` / `update_yaxes` for axes, `update_traces` for the series. The title is still the message."

```
fig.update_layout(
    title_text="Flagstaff has overtaken Bend; Asheville is drifting down",
    height=450, legend=dict(orientation="h", yanchor="bottom", y=1.0, x=0))
fig.update_yaxes(tickprefix="$", tickformat=",.0f", rangemode="tozero")
fig.update_xaxes(dtick="M3", rangeslider_visible=True)
y_axis = fig.layout.yaxis
print(y_axis.tickprefix, fig.layout.xaxis.dtick, y_axis.rangemode)
fig.show()
```

```
$ M3 tozero
```

9 **Break it on purpose: a misspelled property (2m).** Say: "Plotly checks every property name. The real message is about five hundred lines. Read the first lines and the last three, nothing in between."

```
try:
    fig.update_layout(tittle="Monthly revenue")
except ValueError as error:
    lines = str(error).splitlines()
    print(lines[0])
    print([ln for ln in lines if "Did you mean" in ln][0])
    print(len(lines), "lines in the full message")
```

```
Invalid property specified for object of type plotly.graph_objs.Layout: 'titt
Did you mean "title"?
511 lines in the full message
```

10 **Point at something: shapes and annotations (2m).** Say: "Reference lines and shaded regions live in the layout, so they zoom and pan with the data."

```
boise = sales[(sales["store"] == "Boise") & (sales["date"] >= "2025-10-01")]
fig = px.line(boise, x="date", y="revenue", template="plotly_white",
              title="Boise, fourth quarter of 2025")
fig.add_hline(y=boise["revenue"].mean(), line_dash="dash", line_color="gray",
              annotation_text="quarter average")
fig.add_vrect(x0="2025-11-28", x1="2025-12-01", fillcolor="orange", opacity=0.
              line_width=0)
print(len(fig.layout.shapes), "shapes,", len(fig.layout.annotations), "annotat
fig.show()
```

```
2 shapes, 1 annotation
```

11 **Save it, and look inside the file (4m).** Say: "One file, no Python needed to open it. Where the plotly.js library comes from decides the size." Then open the small file in a text editor and search for Asheville.

```
from pathlib import Path
fig = px.line(monthly, x="month_start", y="revenue", color="store",
              title="Monthly revenue by store")
fig.write_html("revenue_full.html")
fig.write_html("revenue_cdn.html", include_plotlyjs="cdn")
for name in ["revenue_full.html", "revenue_cdn.html"]:
    print(f"{name:<20} {Path(name).stat().st_size / 1024:8,.0f} KB")
text = Path("revenue_cdn.html").read_text(encoding="utf-8")
print("Asheville" in text, '"x":' in text)
```

```
revenue_full.html       4,716 KB
revenue_cdn.html           13 KB
True True
```

Say, slowly: "An HTML chart is not a picture of your data. It **contains** your data. Plot one point per customer and you have sent the reader your customer table."

THREE MISTAKES TO EXPECT

- **Handing `px.bar` or `px.line` unaggregated rows.** Fresh from seaborn, learners expect a mean. They get 2,903 stacked slivers, or a line that scribbles back and forth. No error. Coach: "Print `len(fig.data[0].x)`. How many bars did you mean to draw?" Then: "Which pandas line makes that table?"
- **Seaborn words in a Plotly call:** `hue=` for `color=`, `col=` for `facet_col=`, `order=` for `category_orders={...}`. The TypeError names the unexpected argument. Coach: point at the workbook's two-column translation table; do not supply the word.
- **Nothing appears, or the wrong thing is sent.** A cell that ends with `fig = px...` shows nothing (no `fig.show()`); a script that calls `fig.show()` in the lab opens browser tabs for every run. And at the end, a learner writes the full `sales` table into an HTML file when the chart needed three columns. Coach: "Open your file in a text editor and search for a column you did not plot."

GUIDED LAB · 25 MIN

Lab 7.2-A `labs/module-19/l7_2-plotly/product_map_starter.py` (everyone; reference: `product_map_solution.py`). Question: do pricier products get better ratings, and which products are the odd ones out? One Express call with a designed hover, a finished layout, `write_html` to `OUTPUT`, and a look inside the file. The lab files never call `fig.show()`: learners open `output/product_map.html` from the file manager.

- **Minutes 0 to 4:** run the starter as shipped. Ask: "Why does the starter cut the table to five columns before the chart exists?" (Whatever goes into the figure goes into the file. `cost` is commercially sensitive.)
- **Minutes 4 to 16:** TODO 1 to 3. The common stall is the `hover_data` format strings: the colon is part of

the string (`":$,.2f"`). Have learners print `fig.data[0].hovertemplate` and read it.

- **Minutes 16 to 25:** TODO 4 and 5, then everyone opens the HTML file and finds the lowest-rated product by hovering. Debrief: the title. "Price tells you almost nothing about rating" is a finding; "Price vs rating" is a topic.
- **Expected numbers:** 242 products, 8 with no rating, 234 rows in the chart. Figure with 9 traces of type scatter. Spearman correlation of price and rating: −0.01. Lowest rated: Juniper Vane Trailhead Down Jacket, $372.95, rating 2.8, 4 reviews. product_map.html is about 31 KB with the CDN link; a product name is inside the file (True) and the cost column is not (False).
- **Lab 7.2-B** `labs/module-19/l7_2-plotly/promo_share_trend_starter.py` (fast finishers now, everyone at home; reference: `promo_share_trend_solution.py`). 20,302 orders, 2,148 with a promo code. Weekly table (210, 4); median weekly share 6.6% online, 7.9% in store. 2 traces, hover mode x unified. Six sale weekends shaded, each four days (Friday to Monday). Share of orders using a code: 51.0% on sale-weekend days, 8.0% on all other days. January 2024 is 25.3% with no sale on: the data starts there and WELCOME10 marks a customer's first order in the data.
- The solution's last cell exports a PNG with `write_image` only when it can find Chrome (the environment variable `BROWSER_PATH` , or a Chrome on the PATH) and prints `static image skipped: ...` otherwise. That message is expected on most classroom machines and under `check_labs.py` .

START THE LESSON PROJECT · 10 MIN

Open the lesson at **Project: The Sales Explorer**. Read the brief: the manager liked the store brief and sent five follow-up questions; send her one file she can explore. Everyone runs **Step 1** (check: `(96, 3) (2897, 5) (7, 12)`) and **Step 2**, the trend chart with a unified hover and a range slider, and looks at it. Ask: "Why does `points` have 2,897 rows when `sales` has 2,903?" (`dropna` : six store-days have no revenue.) Steps 3 to 5 are homework; tell them the finished file is about 174 KB with three charts and one library link.

CHECKS FOR UNDERSTANDING

Workbook self-check, items Q7 to Q12.

Answers: **Q7** A) The call only builds a Figure object; it is displayed when you call `fig.show()` (or write it to a file); **Q8** `8` ; **Q9** C) `"revenue": ":$,.0f"` ; **Q10** B) One hover box appears for the x position under the mouse, listing all four stores; **Q11** `bar 4` ; **Q12** D) Aggregate to what you mean to publish, pass only the columns the chart needs, then `fig.write_html("chart.html", include_plotlyjs="cdn")` . Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give the Express call in lab 7.2-A with the `hover_data` dictionary left empty, so the whole task is the hover. Let pairs keep the lesson's seaborn-to-Express table open. **Stretch:** the lesson's bonus challenge (a dropdown that switches the trend chart between revenue and visitors with `updatemenus`); or in lab 7.2-

B add `facet_row="channel"` and clean the facet titles with `for_each_annotation`; or export the chart with `write_image` after running `plotly_get_chrome`.

CLOSING

EXIT TICKET · 5 MIN (ANSWERS)

1. You call `sns.barplot(data=sales, x="store", y="revenue")` and black lines appear on the bars. What are they, by default, and which argument changes them? **Answer:** A bootstrapped 95% confidence interval for the mean of each bar (uncertainty of the estimate, not spread of the days). `errorbar=` changes it: `"sd"`, `"se"`, `("pi", 50)` or `None`. Say which one in the caption.
2. Give one job for an axes-level function and one for a figure-level function. **Answer:** Axes-level (`scatterplot`, `histplot`, `barplot`): one panel inside your own `plt.subplots()` layout, with `ax=`. Figure-level (`relplot`, `displot`, `catplot`): seaborn builds a grid of panels from a column with `col=` / `row=` and returns a FacetGrid.
3. You plot one point per customer with `email` in `hover_data` and send the HTML file to a colleague. What have you sent? **Answer:** The customer table: every x, y and hover column is stored as readable text inside the file. Aggregate first and pass Plotly only the columns the chart needs.

Collect the cards. Anyone who misses the first question gets a one-line note back next session; anyone who misses the third hears it again before the capstone.

HOMEWORK BEFORE SESSION 20 (ABOUT 3 HOURS)

- Finish **The Store Performance Brief** (Lesson 7.1 project): four panel functions, `store_brief.png`, and three sentences for the manager, each with a number, saying what the bars in panels B and C are.
- Finish **The Sales Explorer** (Lesson 7.2 project): `sales_explorer.html` with three charts. Email it to yourself and open it on a phone.
- Lesson exercises: 7.1 Exercises 1 to 4 (5 is a stretch), 7.2 Exercises 1 to 3 (4 is a stretch). Both lesson quizzes and journals.
- Whichever kit lab you did not finish: `website_by_age_starter.py`, `promo_share_trend_starter.py`.
- **Read the four capstone questions** at the top of Lesson 8.1 (`capstone_project.html`, the section Choose Your Question). You will choose one at the end of Session 20. Do not start the analysis yet.

BRING LIST FOR SESSION 20 (BOKEH AND STREAMLIT)

- `store_brief.png` and `sales_explorer.html` from the homework (the Streamlit dashboard reuses both ideas).
- Check the two installs: `python -c "import bokeh, streamlit; print(bokeh.__version__, streamlit.__version__)"` should print versions starting with 3 and 1 (the course environment has 3.10.0 and 1.65.0). Both are in the course `requirements.txt`; nothing new to install.
- A terminal you are comfortable in: Streamlit apps are started with `streamlit run dashboard.py` from the course folder, not from a notebook cell. Port 8501 must be free.
- A web browser. No accounts, no API keys, no servers to start beforehand. The practice API from Module 6 is not needed again.
- One sentence, written down: which of the four capstone questions you are leaning toward, and why.

IF YOU RUN LONG OR SHORT

Long, in 7.1: cut demo step 7 (the skewed distribution: it is Lesson 4.2 again) and step 6 (aggregate yourself: say it in one sentence instead). Never cut step 8 (error bars) or the lab's merge check. **Long, in 7.2:** cut step 10 (shapes and annotations) and step 9 (the misspelled property); shorten step 8 to the title and the dollar axis. Never cut step 4 (`px.bar`) or step 11 (the data is in the file). If the whole session is ten minutes behind at the break, start each project by reading the brief only and move Step 1 to homework. **Short:** run lab 7.1-B and 7.2-B in the room; then the lesson's facet-and-trendline example (`trendline="ols"`, the slopes are about $21 per visitor in every store) with the question "does one more visitor cause $21 of revenue?"; or have pairs trade `product_map.html` files and find one thing in the other pair's hover that a reader would not understand.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 20: Bokeh and Streamlit

How to teach the last tools session. Before the break, Bokeh: several charts looking at one table, so that selecting rows in one lights them up in the others. After it, Streamlit: an app whose controls run pandas, built on one idea, the script reruns. The session ends with the capstone kickoff: every learner chooses a question and leaves with a written plan.

2 lessons | 3 hrs session | Lessons 7.3-7.4 | Session 20 of 21 | Capstone question chosen

HOW TO USE THIS GUIDE

- **Same rhythm, one addition.** Warm-up; for each lesson a frame, a live demo, a guided kit lab and the first step of the lesson project; then, in place of the usual last stretch, a **20-minute capstone kickoff** before the wrap-up.
- **The lessons are the reference.** Keep `visualization_bokeh.html`, `visualization_streamlit.html` and the top of `capstone_project.html` open in browser tabs. Labs are in `labs/module-20/`.
- **Two tools, two working styles.** Bokeh is demonstrated from a notebook or script that writes HTML files. Streamlit is demonstrated from an editor and a terminal: it cannot run in a notebook cell.
- **Every output in this guide is from a real run** on the course data (Bokeh 3.10.0, Streamlit 1.65.0). Streamlit outputs come from its testing tool, `AppTest`, so you can reproduce them without a browser.
- **Checkpoint 2 is not in this session.** The Modules 5 to 6 data-access checkpoint sits in Session 18. Tonight's assessment moment is the capstone plan.

> **THE ONE RISK OF SESSION 20: RUNNING OUT OF TIME BEFORE THE KICKOFF**
>
> Both tools are new, both are fun, and each demo can swallow forty minutes. The capstone kickoff at 2:30 is not optional: learners who leave without a chosen question and a written plan arrive at the Session 21 studio with nothing to work on. Protect it by cutting demo steps (the lists under If You Run Long name them), never by shortening the kickoff. The second risk is conceptual and lives after the break: learners carry the notebook's mental model (variables persist) into Streamlit, where **the whole script reruns**. Say that sentence until they finish it for you.

BEFORE THE SESSION

- ☐ Run the kit check: `python3 labs/check_labs.py module-20`. Expect `9 passed, 0 failed` (four starters, four solutions, one pytest file). It starts no servers and opens no browser.

- ☐ Hand out **starters only** (`margin_brush_starter.py`, `category_picker_starter.py`, `gear_shortlist_starter.py`, `catalog_browser_starter.py`) with `labs/labs_paths.py`. Keep the solutions and `test_catalog_browser.py` on your machine until the lab is done.
- ☐ Versions: `python -c "import bokeh, streamlit; print(bokeh.__version__, streamlit.__version__)"` should print versions starting with 3 and 1. `streamlit version` in the terminal is the same check for learners.
- ☐ **Streamlit's first run** may ask for an email address in the terminal: press Enter to skip. Do one `streamlit run` on the projector machine before class so the prompt does not appear live. To silence it on shared machines, create `~/.streamlit/config.toml` with `[browser]` and `gatherUsageStats = false`.
- ☐ **Port 8501.** `streamlit run` serves on `http://localhost:8501`. If a learner has an app still running, the next one takes 8502 and the old tab shows the old app: teach Ctrl+C early. Venue firewalls are not an issue: everything is on localhost. If the browser does not open by itself, copy the Local URL from the terminal.
- ☐ **No internet in the room?** Bokeh's default `mode="cdn"` files open blank offline: have learners pass `mode="inline"` to `output_file` (about 1.4 to 1.8 MB per file). Streamlit works offline; its charts do too.
- ☐ Projector: editor and terminal side by side for the second half, font 18 pt or larger; a notebook or second terminal for the `AppTest` checks.
- ☐ Whiteboard: "share a range → same view", "share a source → same rows"; for after the break, "the whole script reruns".
- ☐ Capstone: print the four questions (from Lesson 8.1, Choose Your Question) one set per table, bring sticky notes, and read the answer notes in `../../final-assessment.html` beforehand so that you can ask good questions without giving anything away.
- ☐ Skim the Session 19 homework in the warm-up: `store_brief.png` (four panels) and `sales_explorer.html` (three charts, about 174 KB).

SESSION 20 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 19 (seaborn error bars, Plotly figures), no notes. Homework check: store_brief.png and sales_explorer.html.
0:10	10	**7.3** Frame and mental model	One table in the middle of the room, each chart a window onto it.
0:20	25	**7.3** Live demo	Figure and glyphs, ColumnDataSource, hover, linked panning, linked brushing, a dropdown that needs no server.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:45	22	**7.3** Guided lab	`labs/module-20/l7_3-bokeh/margin_brush_starter.py`, then `category_picker_starter.py`.
1:07	8	**7.3** Start the lesson project	The Linked Store Explorer: Step 1 (one minimal source and a view) and the plan for three plots.
1:15	10	Break	Leave the brushed two-plot page on the projector for learners to try.
1:25	10	**7.4** Frame and mental model	A spreadsheet that recalculates: the whole script reruns.
1:35	25	**7.4** Live demo	hello.py, the two counters, keys and a reset callback, st.cache_data, layout, a half-chosen date range, a five-line test.
2:00	22	**7.4** Guided lab	`labs/module-20/l7_4-streamlit/gear_shortlist_starter.py`, then `catalog_browser_starter.py`.
2:22	8	**7.4** Start the lesson project	The Pinecrest Sales Dashboard: Step 1 (page setup and cached loading), app left running.
2:30	20	Capstone kickoff	Learners choose one of the four questions in Lesson 8.1 and write a one-paragraph plan: question, tables, method, the chart they expect to make.
2:50	10	Wrap-up	Exit ticket (3 questions), homework (finish 7.3 and 7.4; capstone milestones 1 to 4), bring list for Session 21.
Total	180		Ends at 3:00.

The lesson pages are timed at 90 + 95 = 185 minutes for a solo learner; the room gives each lesson 65. **Protect:** both mental models, linked brushing (7.3 demo step 8), the two counters (7.4 demo step 3), one lab per lesson, the first project step of each, and the full 20-minute kickoff. **Goes home:** both lesson projects, the lesson exercises, quizzes and journals (about 3 hours), plus capstone Milestones 1 to 4 (195 minutes by the lesson's estimates, 20 of them started tonight). That is a heavy week: say so, and tell learners the capstone milestones come first if they must choose.

WARM-UP

0:00 Warm-up: Retrieval from Session 19 — 10 min

HOW TO RUN IT

Six minutes on paper, no notes; two minutes of answers; two for the homework check.

FIVE QUESTIONS (WITH ANSWERS)

1. Which kind of seaborn function takes `ax=`, and what does the other kind return? **Answer:** Axes-level functions (`scatterplot`, `barplot`, `histplot`, ...) take `ax=` and return that Axes. Figure-level functions (`relplot`, `displot`, `catplot`) make their own figure and return a FacetGrid.

2. `sns.barplot` draws a black line on each bar. With default settings, what is it? Name one setting that shows spread instead. **Answer:** A bootstrapped 95% confidence interval for the mean (uncertainty of the estimate). `errorbar="sd"` or `errorbar=("pi", 50)` shows spread of the data.

3. You pass 2,903 daily rows to `px.bar(sales, x="store", y="revenue")`. What do you get, and what should you have done? **Answer:** 2,903 bar segments stacked into four piles, with no error: Plotly Express does not aggregate. Group with pandas first (`groupby("store", as_index=False)["revenue"].mean()`), then plot four rows.

4. `monthly` has one row per store and month (4 stores, 24 months). What does this print?

```
fig = px.line(monthly, x="month_start", y="revenue", color="store")
print(len(fig.data), fig.data[0].type, len(fig.data[0].x))
```

   Answer: `4 scatter 24`: one trace per store, each a `scatter` trace in lines mode holding 24 points. A figure is data: you can count it before you look at it.

5. You wrote a chart with `fig.write_html("chart.html", include_plotlyjs="cdn")`. Name two things that are true of that file. **Answer:** It is small because the reader's browser fetches plotly.js from a public server, so the reader must be online; and it contains the data you passed to the figure (every x, y and hover column) as readable text.

HOMEWORK CHECK (2 MIN)

Two volunteers put `sales_explorer.html` on the projector. Ask the room: "Find the best single day. Now find that same day in the trend chart." Let them struggle for ten seconds. "Three charts that ignore each other. Fixing that is the first half of today."

LESSON PLANS

7.3 Interactive Charts with Bokeh

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Build a Bokeh figure by adding glyphs (lines, markers, bars) and save it with `output_file`
- Put data in a `ColumnDataSource` and refer to its columns by name
- Configure a hover tool with formatted fields, including dates
- Link plots by shared ranges (panning) and by a shared data source (brushing), and arrange them with layouts
- Add a widget that works in a standalone file using `js_link` or `CustomJS`, and choose between Bokeh and Plotly for a job

IN THE ROOM / AT HOME

Room (65 min): the frame, the ten-step demo, lab 7.3-A (7.3-B for fast finishers) and Step 1 of the Linked Store Explorer. **Home:** the rest of the project (three linked plots, the store picker, the live selection summary, `store_explorer.html`, about 154 KB), Exercises 1 to 4, quiz and journal. Not demonstrated: bars on a categorical axis beyond the last step, `gridplot`, `js_link` sliders (lab 7.3-B uses one), date formatters in tooltips and static export. Point at the lesson's Bokeh-or-Plotly table in one sentence: "Plotly for a chart, Bokeh for charts that talk to each other."

MENTAL MODEL TO SAY (10 MIN FRAME)

"Picture your data as **one large table in the middle of a room**, and each chart as a **window in the wall** looking at that table from a different side. One window shows revenue against visitors, another shows revenue against the date. Walk in and put red stickers on thirty rows of the table: every window shows those thirty rows in red, because there is only one table. In Bokeh the table is a `ColumnDataSource`, the windows are plots, and the stickers are a selection."

Start from the homework: "Your `sales_explorer.html` has three charts that ignore each other. You found the best day in the scatter; then you hunted for it in the trend by eye." Write on the board: **share a range → same view** and **share a source → same rows**. Show the lesson's two-branch diagram (what should stay in sync?) while you say it.

LIVE DEMO (YOU TYPE, THEY FOLLOW) · 25 MIN

One notebook or one script, `session20_bokeh`, started from the course folder. `show(...)` writes the HTML file named by `output_file` and opens it in a browser tab; on the machine that produced the outputs below no browser opens, and the printed lines are identical. To draw inside a notebook instead, run `from bokeh.io import output_notebook; output_notebook()` once and skip the `output_file` lines.

1 **A figure, then a glyph (2m).** Say: "Two moves. Make an empty plot, then add marks to it. If Plotly Express felt like ordering from a menu, this feels like Matplotlib." In a notebook or script, `show` writes the file and opens it in a browser tab.

```
import numpy as np
import pandas as pd
import bokeh
from bokeh.plotting import figure, output_file, save, show

sales = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
boise = sales[sales["store"] == "Boise"]
p = figure(width=800, height=320, x_axis_type="datetime",
           title="Boise daily revenue")
p.line(boise["date"], boise["revenue"], line_width=1.5, color="#667eea")
p.yaxis.axis_label = "revenue ($)"
output_file("first.html", title="Boise daily revenue")
show(p)
print(bokeh.__version__, type(p).__name__, "with", len(p.renderers), "renderer
```

```
3.10.0 figure with 1 renderer
```

Walk the toolbar on the right: pan, box zoom, wheel zoom (click it on first), save, reset. Everything on the plot is an object with properties: `p.yaxis.axis_label`, `p.title.text`.

2 **Layers: one line and one set of markers per store (3m).** Say: "Bokeh does not pick colors for you. A palette is a list of color codes. `legend_label` builds the legend." Ask before the print: "Four stores. How many renderers? How many legend entries?"

```
from bokeh.models import NumeralTickFormatter
from bokeh.palettes import Colorblind

stores = ["Boise", "Bend", "Flagstaff", "Asheville"]
store_palette = [Colorblind[8][i] for i in (0, 1, 3, 6)]
colors = dict(zip(stores, store_palette))
sales["month_start"] = sales["date"].dt.to_period("M").dt.to_timestamp()
monthly = sales.groupby(["store", "month_start"], as_index=False)["revenue"].s
p = figure(width=800, height=360, x_axis_type="datetime",
           title="Monthly revenue by store")
for store in stores:
    one = monthly[monthly["store"] == store]
    p.line(one["month_start"], one["revenue"], line_width=2, color=colors[stor
           legend_label=store)
    p.scatter(one["month_start"], one["revenue"], size=6, color=colors[store],
              legend_label=store)
p.yaxis.formatter = NumeralTickFormatter(format="$0,0")
p.legend.location = "top_left"
p.legend.click_policy = "hide"
output_file("monthly.html", title="Monthly revenue by store")
show(p)
print(len(p.renderers), "renderers,", len(p.legend.items), "legend items")
```

```
8 renderers, 4 legend items
```

3 **The table in the middle of the room: ColumnDataSource (3m).** Say: "Until now I passed columns straight to each glyph. Making the source myself unlocks everything else today. And look at what I do first: I keep only the columns the page needs."

```
from bokeh.models import ColumnDataSource

sales["day"] = sales["date"].dt.strftime("%a %b %d, %Y")
sales["promo"] = sales["promo_day"].map({0: "no", 1: "yes"})
tidy = sales[["date", "day", "store", "visitors", "revenue", "promo"]].dropna(
source = ColumnDataSource(tidy)
print(source.column_names)
print(len(source.data["revenue"]), "rows;", type(source.data["revenue"]).__nam
```

```
['index', 'date', 'day', 'store', 'visitors', 'revenue', 'promo']
2897 rows; ndarray
```

A source is a dictionary of equal-length columns. The DataFrame's index came along as a column named `index`. Six store-days with no revenue were dropped: a marker with no y cannot be drawn.

4 **Glyph arguments become column names (3m).** Say: "With a source, a string means: look this column up. A number still means: this value for every mark." Ask: "One call to `scatter`. How many renderers, how many markers?"

```
from bokeh.transform import factor_cmap

store_color = factor_cmap("store", palette=store_palette, factors=stores)
p = figure(width=800, height=400, title="Daily revenue against foot traffic",
           x_axis_label="visitors per day", y_axis_label="revenue ($)")
p.scatter("visitors", "revenue", source=source, size=5, alpha=0.5,
          color=store_color, legend_group="store")
p.legend.location = "top_left"
p.yaxis.formatter = NumeralTickFormatter(format="$0,0")
output_file("scatter.html", title="Revenue and foot traffic")
show(p)
print(len(p.renderers), "renderer draws", len(tidy), "markers")
```

```
1 renderer draws 2897 markers
```

5 **A hover that names the day (3m).** Say: "`@name` is this row's value in that column. A format goes in braces. Any column in the source can appear, plotted or not." Hover the top point live: the Saturday of the 2024 Labor Day sale, in Boise. Note the fresh source: a new page needs new objects.

```
from bokeh.models import HoverTool

tips = [("Day", "@day"), ("Store", "@store"),
        ("Visitors", "@visitors{0,0}"), ("Revenue", "@revenue{$0,0}")]
hover = HoverTool(tooltips=tips)
source = ColumnDataSource(tidy)
p = figure(width=800, height=400, title="Hover a point to identify the day",
           tools=["pan", "wheel_zoom", "box_zoom", "reset", hover])
p.scatter("visitors", "revenue", source=source, size=5, alpha=0.5,
          color=factor_cmap("store", palette=store_palette, factors=stores))
output_file("hover.html", title="Hover")
show(p)
print([type(t).__name__ for t in p.toolbar.tools])
```

```
['HoverTool', 'PanTool', 'WheelZoomTool', 'BoxZoomTool', 'ResetTool']
```

6 **Break it on purpose: a typo that raises nothing (2m).** Say: "Watch. I misspell a column." Run it. "No exception. When I show this plot I get axes and no points, and one line in the terminal: E-1001, BAD_COLUMN_NAME. An empty Bokeh plot is nearly always this."

```
p = figure(width=400, height=250)
renderer = p.scatter("visitors", "revenu", source=ColumnDataSource(tidy))
print("no exception raised; y =", renderer.glyph.y,
      "| column exists:", "revenu" in tidy.columns)
```

```
no exception raised; y = revenu | column exists: False
```

7 **Linked panning: share a range (3m).** Say: "Two ways to connect plots, and they connect different things. First, the view. Give the second plot the first plot's range object." Zoom the top plot with the wheel; the bottom follows.

```
from bokeh.layouts import column

boise_source = ColumnDataSource(tidy[tidy["store"] == "Boise"])
top = figure(width=800, height=230, x_axis_type="datetime", title="Boise: reve
             tools="xpan,xwheel_zoom,reset", active_scroll="xwheel_zoom")
top.line("date", "revenue", source=boise_source, color="#667eea")
bottom = figure(width=800, height=230, x_axis_type="datetime",
                title="Boise: visitors", x_range=top.x_range,      # the link
                tools="xpan,xwheel_zoom,reset", active_scroll="xwheel_zoom")
bottom.line("date", "visitors", source=boise_source, color="#d97706")
output_file("linked_pan.html", title="Linked panning")
show(column(top, bottom))
print(bottom.x_range is top.x_range)
```

```
True
```

8 **Linked brushing: share a source (5m). This is the reason to learn Bokeh.** Say: "Second, the rows. Two plots, one source, a selection tool. I write no linking code at all." Drag a box around the busiest days on the left and wait. Ask: "What are the orange days on the right?" (Sale weekends, summer Saturdays, the weeks before Christmas.) Then let two learners drive: select December on the right.

```
from bokeh.layouts import row

boise_2025 = tidy[(tidy["store"] == "Boise") & (tidy["date"] >= "2025-01-01")]
brush_source = ColumnDataSource(boise_2025)
select_tools = "box_select,lasso_select,tap,reset"
style = dict(source=brush_source, size=6, color="#667eea",
             nonselection_alpha=0.15, selection_color="#d97706")
left = figure(width=400, height=360, title="Drag a box here...", tools=select_
              active_drag="box_select", x_axis_label="visitors",
              y_axis_label="revenue ($)")
left_dots = left.scatter("visitors", "revenue", **style)
right = figure(width=500, height=360, title="...and see when those days were",
               tools=select_tools, active_drag="box_select", x_axis_type="date
right_dots = right.scatter("date", "revenue", **style)
output_file("linked_brush.html", title="Linked brushing")
show(row(left, right))
print(len(brush_source.data["revenue"]), "rows shared by 2 plots |",
      "same source:", left_dots.data_source is right_dots.data_source)
```

```
363 rows shared by 2 plots | same source: True
```

Say: "One drag answered: when were the busiest days? No filter, no query, no new chart." Then the opposite slip: "If I had written `ColumnDataSource(boise_2025)` twice, I would have two tables. No error, and no brushing."

9 **A dropdown that works in a plain HTML file (4m).** Say: "When the reader moves a widget in a saved file, what code runs? There is no Python on their machine. Only JavaScript can run. Read these two lines of JavaScript as you would read Python." The plot draws through a view that shows only rows passing a filter; the dropdown changes the filter.

```
from bokeh.models import CDSView, CustomJS, GroupFilter, Select

source = ColumnDataSource(tidy)
store_filter = GroupFilter(column_name="store", group="Boise")
view = CDSView(filter=store_filter)
p = figure(width=800, height=360, title="Boise: daily revenue against foot tra
           tools="pan,wheel_zoom,reset")
p.scatter("visitors", "revenue", source=source, view=view, size=5, alpha=0.5)
picker = Select(title="Store", value="Boise", options=stores, width=200)
picker.js_on_change("value", CustomJS(args=dict(flt=store_filter, plot=p), cod
    flt.group = cb_obj.value;
    plot.title.text = cb_obj.value + ": daily revenue against foot traffic";
"""))
output_file("select.html", title="Store picker")
show(column(picker, p))
print(view.filter.column_name, "=", view.filter.group)
```

```
store = Boise
```

`cb_obj` is the widget that changed. `flt` and `plot` exist because they were passed in `args`. A Python callback (`picker.on_change`) is silently ignored in a saved file: it needs a running Bokeh server, or the next lesson's tool.

10 **Two ways to save, and what travels (2m).** Say: "Same choice as Plotly. `cdn` is small and needs the internet; `inline` works offline. And the same warning: every column of every source is in the file."

```
from pathlib import Path

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
sales["weekday"] = sales["date"].dt.day_name().str[:3]
by_day = sales.groupby("weekday")["revenue"].mean().loc[days]
for mode in ["cdn", "inline"]:
    name = f"weekday_{mode}.html"
    p = figure(width=600, height=300, x_range=days, title="Mean revenue by wee
    p.vbar(x=days, top=by_day.values, width=0.8, color="#667eea")
    output_file(name, title="Weekday", mode=mode)
    save(p)
    print(f"{name:<20} {Path(name).stat().st_size / 1024:8,.0f} KB")
print(by_day.round(0).tolist())
```

```
weekday_cdn.html            7 KB
weekday_inline.html     1,362 KB
[6138.0, 5716.0, 5985.0, 6321.0, 7713.0, 10933.0, 9211.0]
```

THREE MISTAKES TO EXPECT

- **A blank plot from a misspelled column.** No Python exception: axes, no points, and `E-1001 (BAD_COLUMN_NAME)` in the terminal that nobody reads. Coach: "Print `source.column_names`. Read your glyph call aloud, string by string."
- **`RuntimeError: Models must be owned by only a single document`** when a cell is rerun or a source is reused on a second saved page. Coach: "Which objects were made in an earlier cell? A new page needs new objects." The habit: create figures and sources in the same cell (or function) that saves them.
- **Two sources where one was meant** (no brushing, no error), and its cousin, **old-tutorial arguments** (`plot_width`, `p.circle(size=...)`). Coach the first with: "How many times does `ColumnDataSource(` appear on this page?" For the second, read the message: it names the replacement (`width`, `scatter`).

GUIDED LAB · 22 MIN

Lab 7.3-A `labs/module-20/l7_3-bokeh/margin_brush_starter.py` (everyone; reference: `margin_brush_solution.py`). Question: are Pinecrest's high-margin products also the well-reviewed ones? Two scatter plots on one source: price against margin, and reviews against rating. Labs use `save`, never `show`: learners open `output/margin_brush.html` from the file manager.

- **Minutes 0 to 4:** run the starter as shipped. Ask: "The starter computes margin from `cost` and then leaves `cost` out of `table`. Why?" (Every column of the source is in the file. Margin is what the page needs; cost per product is not for publishing.)
- **Minutes 4 to 16:** TODO 1 to 5. Stall to expect: `source=source` left out of the second scatter, or a second `ColumnDataSource(table)` created for it. Ask them to print `left_dots.data_source is right_dots.data_source`.
- **Minutes 16 to 22:** everyone opens the file and drags a box around the top band of margins. Then TODO 6 and the debrief: "What did the brush show? What does pandas say?" The honest answer is nothing much: high-margin products are rated about the same as the rest.
- **Expected numbers:** 234 products with a rating; margin runs from 34% to 58%. Source columns: index, name, category, price, margin, rating, review_count (no cost). `renderers: 1 + 1 | same source: True`. The left plot has seven tools (BoxSelect, LassoSelect, Tap, WheelZoom, Pan, Reset, Hover). margin_brush.html is about 29 KB. Margin of 50% or more: 67 products, mean rating 4.17; under 50%: 167 products, mean rating 4.12; Spearman correlation of margin and rating +0.01.
- **Lab 7.3-B** `labs/module-20/l7_3-bokeh/category_picker_starter.py` (fast finishers now, everyone at home; reference: `category_picker_solution.py`). 234 rated products in 9 categories. `view shows: category = Tents | 24 products to start with`. category_picker.html (cdn) is about 21 KB; category_picker_offline.html (inline) about 1,750 KB. Counts the page should show: Accessories 30, Apparel 35, Camp Kitchen 28, Climbing 24, Footwear 25, Packs 22, Sleeping 24, Snow 22, Tents 24.

- If a callback does nothing in the browser, open the developer tools (F12) and read the Console tab: JavaScript errors appear there, not in Python.

START THE LESSON PROJECT · 8 MIN

Open the lesson at **Project: The Linked Store Explorer**. Read the brief: a page where the reader picks a store, selects days in any chart and sees those days everywhere, with a one-line summary. Everyone types **Step 1** (check: `2897 rows; ['index', 'date', 'day', 'store', 'visitors', 'revenue', 'promo']`). Then on paper, three boxes: which three plots, which share a source, which share a range. Steps 2 to 5 are homework; tell them Step 4 holds the longest piece of JavaScript in the course and that it is a plain loop.

CHECKS FOR UNDERSTANDING

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) Reads each marker's size from the `visitors` column of the source; **Q2** `4 2`; **Q3** D) Create the second plot with `x_range=top.x_range`; **Q4** A) The selection is stored in a source, and the two plots have two separate sources; **Q5** `['index', 'x', 'y'] 3`; **Q6** C) `("Revenue", "@revenue{$0,0}")`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give the complete left plot of lab 7.3-A and ask only for the right one; keep the lesson's tooltip table open (`@col`, `@col{$0,0}`, `@col{0%}`). **Stretch:** in lab 7.3-A color the markers by category with `factor_cmap` and `legend_group`; in 7.3-B add a second widget, a checkbox that keeps only products with at least 50 reviews (the lesson's bonus challenge shows the filter-combining idea); or lesson Exercise 4, the threshold counter.

7.4 Dashboards with Streamlit

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Write a Streamlit app as a plain Python script and run it locally with `streamlit run`
- Explain the rerun model, and predict what a variable holds after a widget is used
- Keep values across reruns with `st.session_state` and avoid repeated work with `st.cache_data`
- Lay out a dashboard with a sidebar, columns and tabs, showing metrics, charts, a table and a download button
- Test an app without a browser, and describe what it takes to put an app in front of other people

IN THE ROOM / AT HOME

Room (65 min): the frame, the nine-step demo, lab 7.4-B (the shortlist) and then 7.4-A (the catalog browser) as far as time allows, and Step 1 of the Pinecrest Sales Dashboard. **Home:** the rest of the project (`dashboard.py` with filters, four metrics, three tabs, a download and the AppTest check), Exercises 1 to 3 (4 is a stretch), quiz and journal. Deployment is two minutes of talk in the room and a careful read at home: the lesson's six conditions and its three checks before anyone else can open an app.

MENTAL MODEL TO SAY (10 MIN FRAME)

"Think of a **spreadsheet** with an input cell at the top and formulas below it. Change the input and every formula recalculates at once: you never tell the sheet which cells to update. A Streamlit app behaves the same way, except that the sheet is your script. Whenever a reader changes any widget, Streamlit runs **the whole script again**, from the first line to the last, with the widget's new value, and replaces what is on the page."

Bridge from before the break: "Your Bokeh dropdown ran two lines of JavaScript in the reader's browser. That was the ceiling. The moment someone asks to pick their own date range and download those rows, the controls must run pandas, and that means Python has to be running while they use the page." Show the lesson's comparison table (HTML file against app) and its rerun loop diagram. Board: **the whole script reruns**; under it, **session_state remembers · cache_data does not repeat**.

LIVE DEMO (YOU TYPE, THEY FOLLOW) · 25 MIN

Editor and terminal, not a notebook: Streamlit runs `.py` scripts with its own server. Start from the course folder so `data/` resolves. Each step shows the file to type, the terminal command, and a check with Streamlit's testing tool whose output is from a real run. Stop each app with Ctrl+C before starting the next (or pass `--server.port 8502`).

1. **An app is a script (3m).** Work in an editor with a terminal open in the course folder, not in a notebook. Type the file, then start it. Say: "Not `python hello.py`. `streamlit run`." Change the store in the dropdown. "The metric, the chart and the table all changed. I wrote no code to connect them."

File `hello.py`:

```
import pandas as pd
import streamlit as st

st.title("Pinecrest store sales")
sales = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
store = st.selectbox("Store", ["Boise", "Bend", "Flagstaff", "Asheville"])
one = sales[sales["store"] == store]
st.metric("Total revenue", f"${one['revenue'].sum():,.0f}")
st.line_chart(one, x="date", y="revenue")
st.write("The five best days:")
best = one.nlargest(5, "revenue")[["date", "visitors", "revenue"]]
st.dataframe(best, hide_index=True)
```

Terminal:

```
streamlit run hello.py
```

Check without a browser (notebook or second terminal):

```
from streamlit.testing.v1 import AppTest

at = AppTest.from_file("hello.py", default_timeout=30).run()
print("errors:", len(at.exception), "| title:", at.title[0].value)
print(at.selectbox[0].value, "->", at.metric[0].value)
at.selectbox[0].select("Asheville").run()
print(at.selectbox[0].value, "->", at.metric[0].value)
```

```
errors: 0 | title: Pinecrest store sales
Boise -> $7,364,698
Asheville -> $4,140,977
```

The check is Streamlit's own testing tool, `AppTest`: it runs the script in memory with no server and no browser. Keep a notebook or a second terminal open for these checks; you return to them in step 8.

2. **Edit while it runs, then stop it (2m).** Change the title text and save the file. The page offers **Rerun**: choose **Always rerun**. Say: "Edit, save, look: that is how you build one of these." Then press Ctrl+C in the terminal and reload the browser tab. "Gone. The page exists only while the Python process runs. Remember that when we talk about sharing."

3. **Predict: two counters (4m). This is the heart of the lesson.** Type the file. Before running, everyone writes down what both numbers say after three clicks. Then click three times. Say: "Trace one click. The

script reruns. `plain = 0`. The button returns True, so plain becomes 1. Next click: back to 0, up to 1. It never survives the end of a run."

File `counter.py`:

```
import streamlit as st

st.title("Two counters")
plain = 0                                   # an ordinary variable
if st.button("Add one"):
    plain += 1
    st.session_state["kept"] = st.session_state.get("kept", 0) + 1
left, right = st.columns(2)
left.metric("Ordinary variable", plain)
right.metric("Session state", st.session_state.get("kept", 0))
```

Terminal:

```
streamlit run counter.py
```

Check without a browser (notebook or second terminal):

```
at = AppTest.from_file("counter.py", default_timeout=30).run()
print("start    plain:", at.metric[0].value, " kept:", at.metric[1].value)
for n in range(1, 4):
    at.button[0].click().run()
    print(f"click {n}  plain:", at.metric[0].value, " kept:", at.metric[1].val
```

```
start    plain: 0  kept: 0
click 1  plain: 1  kept: 1
click 2  plain: 1  kept: 2
click 3  plain: 1  kept: 3
```

Write on the board: **the whole script reruns**. "That one sentence explains why apps are easy to write, why a plain variable forgets, and why caching matters."

4 **Keys and a reset callback (3m).** Say: "Give a widget a key and its value lives in session state under that name. Starting values are set through session state. The reset happens in a callback, `on_click=reset`, which Streamlit runs before the rerun begins."

File `reset_demo.py`:

```
import streamlit as st

STORES = ["Boise", "Bend", "Flagstaff", "Asheville"]

def reset():
    st.session_state["stores"] = STORES
    st.session_state["min_revenue"] = 0

if "stores" not in st.session_state:        # first run of this session
    reset()
st.multiselect("Stores", STORES, key="stores")
st.slider("Minimum daily revenue ($)", 0, 20000, step=1000, key="min_revenue")
st.button("Reset filters", on_click=reset)
state = st.session_state
st.write("Showing", state["stores"], "at or above", state["min_revenue"])
```

Terminal:

```
streamlit run reset_demo.py
```

Check without a browser (notebook or second terminal):

```
at = AppTest.from_file("reset_demo.py", default_timeout=30).run()
at.multiselect[0].set_value(["Bend"])
at.slider[0].set_value(5000).run()
print("changed:", at.multiselect[0].value, at.slider[0].value)
at.button[0].click().run()
print("reset:  ", at.multiselect[0].value, at.slider[0].value)
```

```
changed: ['Bend'] 5000
reset:   ['Boise', 'Bend', 'Flagstaff', 'Asheville'] 0
```

5 **Do not repeat slow work: st.cache_data (3m).** Say: "Everything is recomputed on every rerun, including reading the file. Here is a load that takes two seconds." Run it and change the store three times: the first run waits two seconds (plus start-up), the rest are instant and the caption keeps saying 1.

File `cache_demo.py`:

```
import time
import pandas as pd
import streamlit as st

@st.cache_data
def load_sales(path):
    time.sleep(2)                          # stand-in for a slow file or query
    st.session_state["loads"] = st.session_state.get("loads", 0) + 1
    return pd.read_csv(path, parse_dates=["date"])

sales = load_sales("data/daily_sales.csv")
store = st.selectbox("Store", ["Boise", "Bend", "Flagstaff", "Asheville"])
total = sales.loc[sales["store"] == store, "revenue"].sum()
st.metric("Total revenue", f"${total:,.0f}")
st.caption(f"The function body has run {st.session_state.get('loads', 0)} time
```

Terminal:

```
streamlit run cache_demo.py
```

Check without a browser (notebook or second terminal):

```
import time

at = AppTest.from_file("cache_demo.py", default_timeout=30)
for store in ["Boise", "Bend", "Flagstaff"]:
    started = time.perf_counter()
    at.run()
    if store != "Boise":
        at.selectbox[0].select(store).run()
    seconds = round(time.perf_counter() - started)
    print(f"{store:<10} {at.metric[0].value:>12}  about {seconds} s  body runs
          at.session_state["loads"])
```

```
Boise        $7,364,698  about 3 s  body runs: 1
Bend         $5,045,467  about 0 s  body runs: 1
Flagstaff    $4,988,177  about 0 s  body runs: 1
```

Cache the slow steps that do not depend on widgets. Leave the filtering uncached. The cache is shared by every user of the app; session state belongs to one browser tab.

6 **Layout: sidebar, columns, tabs (3m).** Say: "By default elements stack down the page in script order. Four containers cover almost every dashboard: sidebar for filters, columns for things side by side, tabs for alternative views, an expander for details."

File `layout_demo.py`:

```
import pandas as pd
import streamlit as st

# set_page_config must be the first st call
st.set_page_config(page_title="Pinecrest Store Sales", layout="wide")

@st.cache_data
def load_sales(path):
    return pd.read_csv(path, parse_dates=["date"])

sales = load_sales("data/daily_sales.csv")
with st.sidebar:                                    # filters go in the sidebar
    store = st.selectbox("Store", ["Boise", "Bend", "Flagstaff", "Asheville"])
one = sales[sales["store"] == store]

st.title(f"{store} store")
left, right = st.columns(2)                         # side by side
left.metric("Revenue", f"${one['revenue'].sum():,.0f}")
right.metric("Visitors", f"{one['visitors'].sum():,}")
tab_trend, tab_data = st.tabs(["Trend", "Data"])  # alternative views
with tab_trend:
    st.line_chart(one, x="date", y="revenue")
with tab_data:
    st.dataframe(one[["date", "visitors", "revenue"]], hide_index=True)
    st.download_button("Download these rows as CSV", data=one.to_csv(index=Fal
                       file_name="one_store.csv", mime="text/csv")
```

Terminal:

```
streamlit run layout_demo.py
```

Check without a browser (notebook or second terminal):

```
at = AppTest.from_file("layout_demo.py", default_timeout=30).run()
print("errors:", len(at.exception), "| sidebar widgets:", len(at.sidebar.selec
print("metrics:", [(m.label, m.value) for m in at.metric], "| tabs:", len(at.t
at.sidebar.selectbox[0].select("Flagstaff").run()
print(at.title[0].value, "|", at.metric[0].value)
```

```
errors: 0 | sidebar widgets: 1
metrics: [('Revenue', '$7,364,698'), ('Visitors', '352,716')] | tabs: 2
Flagstaff store | $4,988,177
```

Tabs organize the page; they do not skip code. Every tab's content is computed on every rerun.

7 **Break it on purpose: a half-chosen date range (3m).** Say: "When a reader picks a new range there is a moment after the first click when the widget holds one date, and the script reruns right then." Show the red box in the browser, then the fix: check the length, show a message, `st.stop()`.

File `dates_good.py`:

```
import datetime as dt
import streamlit as st

dates = st.date_input("Range", value=(dt.date(2025, 1, 1), dt.date(2025, 12, 3
if len(dates) != 2:
    st.info("Pick an end date to complete the range.")
    st.stop()
start, end = dates
st.write(start, end)
```

Terminal:

```
streamlit run dates_good.py
```

Check without a browser (notebook or second terminal):

```
import datetime as dt

for name in ["dates_bad.py", "dates_good.py"]:
    at = AppTest.from_file(name, default_timeout=30).run()
    at.date_input[0].set_value((dt.date(2025, 3, 1),)).run()    # a start date
    problem = at.exception[0].message if at.exception else "none"
    notes = [i.value for i in at.info]
    print(f"{name:<14} error: {problem} | info shown: {notes}")
```

```
dates_bad.py   error: not enough values to unpack (expected 2, got 1) | info s
dates_good.py  error: none | info shown: ['Pick an end date to complete the ra
```

`dates_bad.py` is the same file without the three-line `if` block. `st.stop()` ends the current run cleanly; the lesson project uses exactly this for the date range and for an empty store list.

8 **A five-line safety net (2m).** Say: "The check I have been running is a test. Save it as a file and it guards the app after every edit." Run it from the terminal.

File `test_hello.py`:

```python
from streamlit.testing.v1 import AppTest


def test_hello_runs_and_switches_store():
    at = AppTest.from_file("hello.py", default_timeout=30).run()
    assert not at.exception
    at.selectbox[0].select("Asheville").run()
    assert at.metric[0].value == "$4,140,977"
```

Terminal:

```
python -m pytest -q test_hello.py
```

```
1 passed
```

9 **What sharing an app takes (2m, no code).** Say: "A Plotly file you send. An app you **run**. For a colleague to use it while your laptop is closed, six things must be true: a machine that runs Python all the time, the same packages (`requirements.txt`), the code and the data, a start command (`streamlit run dashboard.py --server.headless true`), a way in with a decision about who may open it, and secrets kept out of the code. Start local. Deploy when people ask to use it without you."

THREE MISTAKES TO EXPECT

- **`python app.py` in place of `streamlit run app.py`.** The terminal fills with `missing ScriptRunContext` warnings and nothing opens. Coach: "Read the first word of your command." (The kit's two Streamlit labs are the one exception, by design: run with `python` they test themselves; see the lab notes.)
- **Expecting a variable to remember.** A list, a counter or a flag assigned in the script body is assigned again on every rerun. There is no error, only a number that will not grow. Coach: "Put your finger on line 1 and trace one click to the last line. What is the list at the top?" Then: "Where must it live?"
- **Changing a widget's value after it was drawn** (`st.session_state["x"] = 0` inside `if st.button(...)`), and **two identical widgets** without keys. Both give a red box whose first sentence says exactly what happened. Coach: have the learner read the first sentence aloud, then ask "what runs before the script does?" (A callback.)

GUIDED LAB · 22 MIN

Lab 7.4-B `labs/module-20/l7_4-streamlit/gear_shortlist_starter.py` first (everyone; reference: `gear_shortlist_solution.py`): a tent shortlist that forgets. It is the counter demo turned into a task, and it is the lab that makes the rerun model stick. Then **Lab 7.4-A** `labs/module-20/l7_4-streamlit/catalog_browser_starter.py` (reference: `catalog_browser_solution.py`, test file

`test_catalog_browser.py`): sidebar filters with keys, a reset callback, `st.stop()` on an empty result, three metrics, a table and a download.

- **Each lab file is an app and its own test.** `streamlit run gear_shortlist_starter.py` opens it in the browser; `python gear_shortlist_starter.py` runs a built-in check with `AppTest` and prints what the app shows, with no server and no browser. Learners keep the app open in a browser tab (Always rerun) and run the check in the terminal after each TODO.
- **Minutes 0 to 3:** before fixing anything, everyone predicts what `python gear_shortlist_starter.py` prints. It prints 0, 1, 0, 1, 0 in the first column: the list forgets.
- **Minutes 3 to 12:** TODO 1 to 5 in the shortlist. Stall to expect: keeping `if left.button(...):` and also adding `on_click=`, which adds the tent twice in one click. One or the other.
- **Expected (shortlist):** 24 tents. The first metric reads 0 at the start, 1 after adding the first tent, still 1 after adding it again and after choosing a second tent, 2 after adding the second ($259.90 in total: $41.95 + $217.95), and 0 after Clear. The third metric, the ordinary counter, reads 1 on every line. The unfinished starter prints 0, 1, 0, 1, 0: the list forgets.
- **Minutes 12 to 22:** the catalog browser, TODO 1 to 6. The starter's check counts progress: `metrics: 3`, `widgets in the sidebar: 4`, `empty selection handled: True`.
- **Expected (catalog):** Start (Tents): Products 24, Median price $298.45, Mean rating 4.25. Tents and Sleeping under $100: 12 products, median $46.95, mean rating 4.07; with In stock only: 11. No category: the warning shows, 0 metrics, 0 errors. After Reset: ['Tents'], (0, 620), False, 24 products. `test_catalog_browser.py`: 3 tests pass.
- **A trap worth one sentence:** text passed to `st.caption`, `st.write` and `st.markdown` is Markdown, and a pair of dollar signs is read as math. `"$0 to $620"` renders as *0 to 620* in italics. Write `\$`, or put the amount in words, as the lab's caption does. `st.metric` values are not Markdown, so `$298.45` is safe there.

START THE LESSON PROJECT · 8 MIN

Open the lesson at **Project: The Pinecrest Sales Dashboard**. Read the brief: choose stores and a date range, see the key numbers, look at the trend and the weekly pattern, download the rows. Everyone creates `dashboard.py`, types **Step 1** (page setup, constants, the cached `load_sales`) plus one temporary line, `st.write(sales.shape)`, and starts it with `streamlit run dashboard.py`. Check: the page shows `(2903, 10)`. Leave the app running with Always rerun: Steps 2 to 7 are homework, built by saving and looking. The final AppTest check should print Revenue $21,539,320 for all stores.

CHECKS FOR UNDERSTANDING

Workbook self-check, items Q7 to Q12.

Answers: **Q7** B) 11; **Q8** `` `20` ``; **Q9** A) In a function passed as `on_click=`, which Streamlit runs before the rerun starts; **Q10** D) Loading and preparing the CSV file, which is slow and the same whatever the widgets say;

Q11 `1 0`; **Q12** C) True only during the one rerun caused by a click, otherwise False. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: for the shortlist, hand out a card with the three places state can live (a variable: one run; `st.session_state`: one browser tab; `@st.cache_data`: everyone, until restart) and let pairs trace one click on paper before typing. In the catalog browser give TODO 1 complete. **Stretch:** add a `st.download_button` to the shortlist; add a fourth filter to the catalog browser (brand) with its own key and include it in the reset; write one more test in `test_catalog_browser.py`; or lesson Exercise 2 (a delta against the previous period), reading its caution about seasonal comparisons.

CAPSTONE KICKOFF

2:30 Capstone Kickoff: Choose a Question, Write the Plan

20 min

PURPOSE

Every learner leaves with a chosen capstone question and a one-paragraph plan in the workbook. The plan is Milestone 1 of Lesson 8.1 started in the room; the homework is Milestones 1 to 4, up to a first finding. Session 21 is a studio: it only works if people arrive with a prepared table and something they have noticed.

RUN IT ON A CLOCK

Q	Question (Lesson 8.1)	Main data	Leans on
A	Why is Asheville falling behind?	daily_sales.csv (store_weather.csv, survey.csv, sales_2025.xlsx possibly useful)	time series, ratios, comparison between stores, careful wording about cause
B	Does it matter how we found the customer?	customers.csv, orders.csv	merging, more than two groups, skewed data, effect sizes
C	Does the welcome discount bring people back?	orders.csv (customers.csv possibly useful)	per-customer features, two proportions, spotting an unfair comparison
D	What did the March 2025 price increase do to sales?	order_items.csv, orders.csv, products.csv	multi-table merges, seasonality, reasoning about what would have happened otherwise (the most demanding)

1. **Minutes 0 to 4 · the brief.** Put the lesson's memo on screen ("We have four questions that keep coming up in meetings, and opinions on all of them. We would like evidence."). Say what is handed in: a notebook that runs top to bottom, one or two figures, a one-page `report.md`; a dashboard is optional. Say the sentence from the lesson that matters most: "A well-supported *no clear difference* scores exactly as well on the rubric as a dramatic discovery."

2. **Minutes 4 to 8 · choose.** Learners reread the four questions (they read them for homework) and write a letter on a sticky note. Tell them the lesson's advice: choose by curiosity, not by ease. D is the most demanding. A learner may propose their own question if it passes the lesson's three checks (a decision, the data exists in `data/`, no lesson project already answered it).

3. **Minutes 8 to 16 · write the paragraph.** In the workbook, complete the plan frame: the question and the decision it informs, the tables and what one row of the analysis table will be, the comparison and the method, why the comparison might be unfair and how they will check, the chart they expect to make, and what result would change their mind. Circulate. Your only questions: "What is being compared with what?" and "What else differs between those two groups?"

4. **Minutes 16 to 20 · partner check.** Pairs swap workbooks. The reader answers one thing in writing under the plan: "From this plan, I predict your notebook will contain ...". That is the lesson's own done-when test for Milestone 1. Collect the sticky notes: they tell you how to split your coaching time in Session 21.

WHAT TO SAY AND WHAT NOT TO SAY

- **Do not hint at any answer.** The lesson deliberately has no answer key. Your notes for each question (sound method, key numbers, traps, what an honest null or an over-claim looks like) are in `final-assessment.html`; read them before Session 21, not aloud tonight.
- **Do point at the lesson's own "Think about" lines.** Each question's paragraph already names its trap in general terms (time behind each customer, what else changed in March, which factor of revenue moved). Ask the learner to read theirs to you and say how the plan handles it.
- **A plan that is a topic** ("look at Asheville") is the Beginning level of the rubric's first row. Ask for the comparison. **A plan with six comparisons** will not finish: ask which one is the headline.
- **Spread is not required.** If everyone chooses A, let them: the showcase is then a study in how the same data yields different, defensible reports. Note it for the gallery walk grouping.
- **Tools and help.** The lesson allows AI assistants and web code on working-analyst terms: the learner is responsible for every line and every claim, checks outputs another way, and says in the README what help was used. Say it now so nobody is surprised by the disclosure line at submission.

CLOSING

EXIT TICKET · 4 MIN (ANSWERS)

1. Two Bokeh plots on one page. What must they share for a box selection in one to highlight the same rows in the other? What must they share to pan together? **Answer:** The same `ColumnDataSource` object for linked selection (brushing). The same range object (`x_range=first.x_range`) for linked panning and zooming.

2. A reader changes one widget in your Streamlit app. What runs? Name the place for a value that must survive, and the decorator for a slow step that must not repeat. **Answer:** The whole script reruns from the top with the widget's new value. `st.session_state` keeps values across reruns (per browser tab); `@st.cache_data` stores a function's result so the slow body runs once.

3. Write your capstone question letter (A, B, C or D) and the one comparison your plan is built on. **Answer:** No single answer. Check that it is a comparison (two or more groups, or before and after against something), not

a topic, and that the learner can say what would make it unfair. Use the cards to balance your coaching time in Session 21.

HOMEWORK BEFORE SESSION 21

- **Capstone first: Milestones 1 to 4 of Lesson 8.1, up to a first finding** (M1 Question and plan 30 min, M2 Load and check 45 min, M3 Clean and prepare 45 min, M4 Explore 75 min; 195 minutes in all). Bring: the plan as the first Markdown cell of `pinecrest_report.ipynb`, the loaded and checked tables, one function that builds the analysis table, a "What I have noticed" list, and two sentences on what you expect the answer to be and what could still prove you wrong.
- Finish **The Linked Store Explorer** (7.3): `store_explorer.html`. Finish **The Pinecrest Sales Dashboard** (7.4): `dashboard.py` and its AppTest check.
- Lesson exercises (7.3: 1 to 3; 7.4: 1 to 3), both quizzes and journals; the kit lab you did not finish.
- Read the rest of Lesson 8.1 before you start: the worked mini-example, the one-page report template, the rubric and the five common mistakes.
- If the week is short: capstone milestones, then the two lesson projects, then exercises.

BRING LIST FOR SESSION 21 (CAPSTONE STUDIO AND SHOWCASE)

- Your capstone folder `pinecrest-capstone/` with `pinecrest_report.ipynb` that **runs from a fresh kernel** as far as you have got (Restart and Run All before you leave home), and the course `data/` folder beside it.
- Your one-paragraph plan (workbook) and your "What I have noticed" list.
- Lessons 4.5, 4.7 and 3.3 within reach, plus your own helpers: `overview`, `cohens_d`, your chart helpers.
- A way to show one figure to others: your laptop screen is enough for a gallery walk; a single PNG on a shared drive or USB stick for lightning talks.
- Nothing to install, no servers, no accounts, no API keys.

IF YOU RUN LONG OR SHORT

Long, in 7.3: cut demo step 2 (layers and legend: it is Matplotlib thinking again) and step 10 (file sizes: say the two modes in a sentence); shorten step 9 to running the finished cell. Never cut step 3 (the source) or step 8 (brushing). **Long, in 7.4:** cut step 6 (layout: the lesson project teaches it) and step 4 (keys and reset: lab 7.4-A covers it); never cut step 3 (two counters) or step 5 (caching). If you are ten minutes behind at 2:20, drop "start the project" for 7.4 (it becomes homework from Step 1) and keep the kickoff whole. **Short:** run the second lab of each lesson in the room;

have pairs open each other's `margin_brush.html` and state one thing the brush showed; or take five extra minutes in the kickoff for a second partner check, where the reader names the weakest sentence of the plan.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 21: Capstone Studio and Showcase

How to run the last session: a studio where learners take their own capstone from a first finding to a quantified, visualized answer, a timed showcase, and the close of the course. The instructor answer notes and the rubric with weighting are in final-assessment.html.

`1 lesson` `3 hrs session` `Lesson 8.1` `Session 21 of 21` `Studio + showcase`

HOW TO USE THIS GUIDE

- **One lesson, no new tools.** Session 21 is a studio and a showcase. The lesson page is `capstone_project.html` (The Pinecrest Report, 420 minutes); learners arrive with Milestones 1 to 4 done, work Milestones 5 and 6 here, and finish 7 and 8 within a week.
- **Your instructor answer notes are in `../../final-assessment.html`**: for each of the four questions the tables and steps of a sound analysis, the key numbers, the traps, what an honest null looks like (Questions B and C; for D, what an over-claim looks like), follow-up questions and what weak work gets wrong, plus the rubric with weighting and the submission rules. Read it before class. Nothing from it is said aloud as an answer or copied into a learner page: the lesson deliberately has no answer key.
- **Coach by asking.** Every coaching move in this guide is a question. If a learner asks "is my answer right?", ask what would make it wrong and whether they checked.
- **The optional lab** `labs/module-21/l8_1-capstone/` holds a filled-in version of the lesson's notebook outline on a practice question that is not a capstone question. Point stuck learners at its structure, not its question.

> **THE ONE RISK OF SESSION 21: A ROOM OF STUCK PEOPLE WAITING FOR YOU**
>
> Twenty learners on four questions will want you all at once. The clock below rations your attention: a check-in every 15 minutes in which every learner reports a status in one word, then coaching for the reds first. Learners who are green help ambers at their table. Never sit with one learner for more than four minutes in a round.

BEFORE THE SESSION

- ☐ Read `final-assessment.html` end to end, especially the notes for the questions your learners chose (the Session 20 sticky notes tell you the mix).
- ☐ Run `python3 labs/check_labs.py module-21`: `2 passed, 0 failed`.

- ☐ Decide the showcase format from the size of the room (table below) and print the gallery-walk rotation cards or set up the second circle and its timekeeper.
- ☐ Traffic-light cards (green, amber, red) or three colors of sticky note for each table; a visible countdown timer on the projector.
- ☐ Projector: the rubric from the lesson page (The Rubric section) and the report template, both readable from the back.
- ☐ No servers, no internet, no installs are needed today. Learners need their capstone folder with `data/` beside the notebook.
- ☐ Prepare the course-close slide content: what next (machine learning, open data, regression), the feedback link or form you use, the submission deadline (one week from today).

SESSION 21 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 20 (Bokeh, Streamlit) and a one-line plan check: each learner says their question and headline comparison.
0:10	10	**8.1** Frame: the rubric and the strong sentence	The six rubric rows, the five-part sentence, how the studio clock works today.
0:20	45	**8.1** Studio 1 · Milestone 5: test and quantify	Check-ins at 0, 15 and 30 minutes; coaching rounds by question; everyone leaves with the main result as one strong sentence.
1:05	10	Break	Learners restart the kernel and Run All before they leave the room.
1:15	40	**8.1** Studio 2 · Milestone 6: visualize, then the Answer	One finding-first figure saved from code; draft the Answer paragraph of report.md. Check-ins at 0, 20 and 35 minutes.
1:55	10	**8.1** Showcase prep	Choose the one figure; write three sentences (question, answer with uncertainty, one caveat).
2:05	35	**8.1** Showcase	Three-minute lightning talks or a gallery walk, timed for the size of the room.
2:40	10	Course close and what next	What the course built, the last journal entry, where to go next, feedback.
2:50	10	Wrap-up	Exit ticket (3 questions), submission instructions and deadline, the after-class milestones.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
Total	180		Ends at 3:00.

The 420 minutes of Lesson 8.1, split. Nothing about the milestones changes; the kit only says where each one happens.

When	Milestones (lesson estimates, minutes)	Min	Notes
Before Session 21	Milestones 1 to 4: question and plan (30), load and check (45), clean and prepare (45), explore (75)	195	Started in the Session 20 kickoff (about 20 minutes of Milestone 1 in the room), finished at home.
In the room, Session 21	Milestone 5 test and quantify (45), Milestone 6 visualize (40 of 45), first draft of the Answer (5)	90	Studio 1 and Studio 2. Showcase prep and the showcase itself are presentation, not milestone time.
After Session 21, within one week	Finish Milestone 6 (5), the rest of Milestone 7 write the page (40), Milestone 8 review and reproduce (30)	75	Plus the optional dashboard or interactive file (60): 135 minutes with the option.
Lesson total	Eight milestones = 360 required minutes (195 + 90 + 75) + the optional dashboard (60)	420	Milestone 6 is split 40 in the room and 5 after; the 5-minute Answer draft in the room is the start of Milestone 7.

WARM-UP

0:00 Warm-up: Retrieval from Session 20 and a Plan Check — 10 min

FIVE QUESTIONS (WITH ANSWERS)

1. Two Bokeh plots on one page: what must they share for a box selection in one to highlight the same rows in the other? **Answer:** The same `ColumnDataSource` object. (Sharing a range object links panning and zooming instead.)
2. A Bokeh file saved with `output_file` has a dropdown. Which kind of callback works for a reader who opens the file? **Answer:** JavaScript: `CustomJS` (or `js_link`). A Python `on_change` callback needs a running Bokeh server.
3. A reader moves a Streamlit slider. What runs? **Answer:** The whole script, from the top, with the slider returning its new value.
4. Where does a value live that must survive reruns? And what stops a slow CSV load repeating on every click? **Answer:** `st.session_state` (per browser tab); `@st.cache_data` on the loading function (shared by all users).
5. Your capstone: say the question letter and the headline comparison in one sentence: "I am comparing ___ with ___ using ___." **Answer:** No single answer. Listen for a comparison rather than a topic, and note who has none yet: they get the first coaching round.

HOW TO RUN IT

Questions 1 to 4 on paper for four minutes, answers called out in two. Then question 5 around the room, ten seconds each: write the comparisons on the board under A, B, C, D. Anyone without a comparison is your first coaching visit in Studio 1.

LESSON PLAN

8.1 The Pinecrest Report

175 min

OBJECTIVES (FROM THE LESSON PAGE)

- Turn a business question into a written analysis plan before touching the data
- Carry an analysis from raw files to a tested, quantified answer, working independently
- Report a result with its estimate, confidence interval, test, effect size and caveats
- Present the answer in a one-page report with one or two figures that state their finding
- Make the work reproducible, review it against a rubric, and share it responsibly

FRAME · 10 MIN: THE RUBRIC AND THE STRONG SENTENCE

Put the rubric on screen. Say: "Six rows. Proficient in all six is a successful capstone. Exemplary is what makes a reviewer remember your name. And a well-supported "no clear difference" scores exactly as well as a discovery." Then write the five-part sentence on the board, using the lesson's example: "On promotion days 25.8% of visitors bought, against 23.4% on comparable ordinary days: 2.4 points more (95% CI 1.9 to 2.9; Welch t-test $p < 0.001$; $d = 0.93$)." Label the five parts: two estimates, the difference, the interval, the test, the effect size. "By the break, your main result is a sentence like this one." Explain the clock: check-ins at fixed times, one word each, reds first.

STUDIO 1 · 45 MIN · MILESTONE 5, TEST AND QUANTIFY

- **0:00 stand-up check-in (3 min).** Each learner holds up a card: green (the analysis table exists and I know my comparison), amber (table exists, method unclear), red (no table yet). Reds come to you first, ambers pair with a green at their table.
- **Coaching round 1 (12 min).** Reds: "Show me one row of your analysis table and say what it is." If it does not exist, do Milestone 3 together in five minutes using the scaffold lab's `build_table()` pattern, then move on.
- **0:15 check-in (2 min).** One word each: "estimate", "interval", "test", "effect", "stuck". Coaching round 2 (13 min): stuck learners first; ask which method fits the data type (proportions, skewed amounts, several groups).
- **0:30 check-in (2 min).** Everyone tries to break their result: one robustness check (another definition, without the extremes, within each store or year). Coaching round 3 (10 min).
- **0:42 (3 min).** Everyone writes the main result as one five-part sentence in a Markdown cell. Spot-read four sentences aloud (with permission) and name the missing part, if any.
- **Done when:** the main result is one sentence in the strong form, and at least one attempt to make it disappear is recorded (the lesson's own done-when).

COACHING NOTES BY QUESTION (NO ANSWERS ALOUD)

For each question: what strong work does, the traps to listen for, and three questions to ask. The numbers that go with these notes are in `final-assessment.html`.

QUESTION A · WHY IS ASHEVILLE FALLING BEHIND?

- **Strong work** writes revenue as visitors × conversion × basket for each store and year, finds which factor moved, and shows whether the pattern is steady through the year (month by month against the same month a year earlier) and unusual against the other three stores.
- **Traps to listen for:** reading the decline as "customers spend less" without checking; comparing seasons; treating 363 days as 363 independent customers; tying the October to November 2025 survey (one snapshot) to a two-year trend; using `orders.csv` (account holders only) as if it were the store total; ignoring the one missing Asheville revenue day.
- **Questions to ask:** "Which factor moved, and by how much in each month?" "What do the other three stores' same factors do?" "What data would tell you why fewer people come in?"

QUESTION B · DOES IT MATTER HOW WE FOUND THE CUSTOMER?

- **Strong work** builds one row per customer (including those who never bought, with zero), defines "worth" before looking, compares five channels with a method that suits skewed spend, counts its pairwise comparisons, and makes the time window fair.
- **Traps to listen for:** customers who signed up during 2024 and 2025 have had less time to spend; inner merges silently drop the 1,112 customers with no completed order (or keep the 739 orders of closed accounts with no channel); ten pairwise tests with no adjustment; "worth twice as much" answered from one ratio of means.
- **Questions to ask:** "Does every customer in your comparison have the same amount of time?" "How many comparisons did you look at?" "How large a difference could your data have missed?"

QUESTION C · DOES THE WELCOME DISCOUNT BRING PEOPLE BACK?

- **Strong work** defines a true first order (the data start on 2024-01-01, so a customer who signed up in 2019 has no first order in this file), uses a fixed follow-up window, compares two proportions with an interval and effect size, and words the result without "because".
- **Traps to listen for:** the time-to-return trap (late first orders have had no time to come back); counting long-standing customers' first order in the data as a first order; customers chose to use the code (self-selection); counting cancelled first orders.
- **Questions to ask:** "What is your follow-up window, and who is excluded by it?" "Whose first order is really a first order?" "What would a fair test of the code look like?"

QUESTION D · WHAT DID THE MARCH 2025 PRICE INCREASE DO TO SALES?

- **Strong work** identifies the repriced products from `order_items.unit_price` (not today's `products.price`), compares units of repriced against unchanged products over the same months a

year apart, checks the comparison before the change (a placebo period), reports the drop in units with its interval (about 14% fewer than trend), and states that the products were not chosen at random.

- **Traps to listen for:** before-and-after across seasons (January and February are slow, so repriced products look busier afterward); company growth (repriced units flat year over year look like "no effect" until the unchanged products are beside them); valuing old orders at today's price; bulk lines and discontinued SKUs; category mix between the two groups. This question has a real effect, so also listen for over-claims: "sales fell 14%" for the whole company, or a price elasticity the data do not show.
- **Questions to ask:** "How did you decide which products were repriced?" "What happened to the unchanged products over the same months, and before March?" "Is your number about units or revenue, and about which products?"

STUDIO 2 · 40 MIN · MILESTONE 6, THEN THE ANSWER

- **0:00 check-in (2 min).** "Which one chart carries your answer?" Each learner names it in a sentence.
- **Work (18 min).** Redraw that chart to the Module 3 standard: a title that states the finding, labeled axes with units, purposeful color, error bars identified in a caption, saved from code with `savefig(..., dpi=150)`.
- **0:20 check-in (3 min).** Pairs swap screens for 60 seconds each: "Without reading anything but the figure, what is the answer?" If the partner cannot say, the title or the comparison needs work.
- **Work (12 min).** Fix the figure; then draft the **Answer** paragraph of `report.md`: two or three sentences a manager could repeat, leading with the answer and the main number with its uncertainty.
- **0:35 (5 min).** Restart the kernel and Run All. Anyone whose notebook fails fixes it now with a neighbor; note their name for a follow-up message this week.

SHOWCASE PREP · 10 MIN

Each learner prepares the one figure and three sentences: the question, the answer with its uncertainty, one specific caveat. Lightning talk or gallery station, the content is the same. A learner with a null result (most often Question B or C) says so first and states what the interval rules out: model that sentence for the room once.

SHOWCASE · 35 MIN

Learners	Lightning talks (35 min)	Gallery walk (35 min)
8	One circle, everyone presents: 8 × (3 min talk + 1 min one question and changeover) = 32 min, 3 min spare. You keep time.	Two halves of 4. Half A at their screens, half B visits: 4 rounds of 3 min + 30 s to move = 14 min; swap (2 min); half B presents for 14 min; 5 min debrief. Every visitor sees every presenter in the other half.

16	Two circles of 8 in parallel (two corners or two rooms), the same 32-minute clock; you sit in one, a co-facilitator or a learner timekeeper runs the other. If you must keep one circle: 2-minute talks (one figure, one sentence) × 16 = 32 min.	Two halves of 8. Each visitor makes 4 visits of 3.5 min (14 min) to 4 of the 8 stations, assigned so every station has a visitor every round; swap; 14 min; 5 min debrief. Each presenter gives the 3-minute version four times.
24	Three circles of 8 in parallel, each with a timekeeper (the third can be a learner); 32 min. One circle of 24 does not fit in 35 minutes: use the gallery walk.	Two halves of 12. Each visitor makes 4 visits of 3.5 min to 4 of the 12 stations (a printed rotation card per visitor); swap; 5 min debrief. Recommended at this size.

Audience job (both formats): each listener writes one question per presenter on a sticky note and gives it to them; the presenter keeps the notes for the Next questions section of the report. **Your job:** keep time and say nothing evaluative in public. Note one strength per learner for their written feedback.

COURSE CLOSE · 10 MIN

- **What the course built (2 min).** Point at the list from the lesson's summary: from installing Python and loading a CSV to answering a business question nobody had answered for them, and knowing how sure they are.
- **The last journal entry (4 min, started here, finished at home).** The lesson's end-of-course prompt: read the first entry from Lesson 0.1 and write to the person who wrote it. Three things you can do now; the hardest lesson and how it looks from here; what the capstone taught you about how you work; one question about data in your own life.
- **What next (2 min).** Prediction (machine learning) builds directly on this course, starting from the `customer_features.csv` table of Lesson 2.9; regression is the bridge; a second capstone on public open data; deeper SQL. Repeat the lesson's rule about data: never practice on private or work data without written permission.
- **Feedback (2 min).** Your usual form. Ask one spoken question: "Which session would you change first, and how?"

CHECKS FOR UNDERSTANDING

Workbook self-check, all twelve items, as homework with the submission (or as a five-minute pair activity if the showcase runs short).

Answers: **Q1** C) Did products repriced on 2025-03-01 sell fewer units, March to December, than comparable products whose price did not change?; **Q2** A) The difference in medians with a bootstrap 95% confidence interval, reported next to the means; **Q3** `4 350.0`; **Q4** B) We found no clear difference: the data are consistent with anything from 1.5 points lower to 4.0 points higher, so a gain larger than about 4 points is unlikely; **Q5** D) Give everyone the same window (for example 180 days) and keep only customers whose first order was at least 180 days before the data end; **Q6** `10`; **Q7** A) The change in the affected products, against the change in

unaffected products over the same months a year earlier (a difference in differences); **Q8** `0.034`; **Q9** C) Customers chose whether to use the code, so those who used it may differ from those who did not in ways the data do not record; **Q10** B) The answer, with the main number and its uncertainty; **Q11** `[10, 40, 30]`; **Q12** D) The question, then the answer with its uncertainty, pointing at the one figure. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: learners who arrive without Milestones 1 to 4 work from the scaffold lab's structure in Studio 1 and narrow their question to one comparison; they present a plan and a first finding in the showcase, and the rubric still applies at submission. **Stretch:** the Exemplary column: a pre-stated result that would change the reader's mind, a robustness check reported in the page, multiple comparisons or non-independence addressed, an alternative explanation tested; or the optional dashboard (adapt `dashboard.py` from Lesson 7.4) after the page is done.

CLOSING

◆ EXIT TICKET · 5 MIN (ANSWERS)

1. Write your main result as one sentence with all five parts. **Answer:** Estimate for each group, the difference, its 95% interval, the test and p-value, and the effect size, in words a manager can repeat. A null result also says how large a difference the data can rule out.
2. Name one caveat that is specific to your question and data (not "more data is needed"). **Answer:** Check it is specific: who or what is missing, what could not be separated, or why "because" is too strong for this design.
3. What will you do in the 75 minutes of capstone work left after today, in order? **Answer:** Finish the figure, write report.md (Answer first, then Caveats), then Restart and Run All and check every number in the report against the fresh output, score yourself on the rubric, submit.

SUBMISSION (WITHIN ONE WEEK) AND WHAT GOES HOME

- **After class: 75 minutes of required work.** Finish the figure, write `report.md` from the template (Answer first, then Caveats), then Milestone 8: Restart and Run All, check every number in the report against the output, score yourself on the six rubric rows, work through the self-review checklist. Optional: a dashboard or interactive file (60 minutes), only after the page is done.
- **Hand in** the project folder: `pinecrest_report.ipynb` (runs top to bottom with relative paths), one or two figures, `report.md` (one page), `README.md` (what this is, how to run it, where the data is from, and what help was used: AI assistants, web code, a classmate), and the self-scored rubric. Deadline: one week from today. The full rules, late options and how the two checkpoints are recorded are in `final-assessment.html`.
- **Bring list:** there is no Session 22. For the submission: the course `data/` folder is not handed in; link to it as the lesson's Sharing Your Work section says.

IF YOU RUN LONG OR SHORT

Long: shorten Showcase prep to five minutes and Course close to the journal prompt and the deadline; never cut the 0:42 strong-sentence write-up in Studio 1 or the Restart and Run All at the end of Studio 2. With 24 learners and lightning talks, switch to the gallery walk rather than cutting talks to one minute. **Short:** do the twelve self-check items in pairs; or run a second gallery round with the sticky-note questions answered in writing; or start Milestone 7 in the room (draft the Caveats section next, then Next questions).